控制大综合

第24章 辨识中的实际问题

- 24.1 动机与要解决的问题
- 24.2 定义与数学基础
- 24.3 推导与性质
- 24.4 离散对应与数值实现
- 24.5 工程含义与适用边界
- 24.6 仿真与验证
- 24.7 小结与常见错误

第 25 章 模式识别

- 25.1 动机与要解决的问题
- 25.2 定义与数学基础
- 25.3 推导与性质
- 25.4 离散对应与数值实现
- 25.5 工程含义与适用边界
- 25.6 仿真与验证
- 25.7 小结与常见错误

第 26 章 机器学习基础

- 26.1 动机与要解决的问题
- 26.2 定义与数学基础
- 26.3 推导与性质
- 26.4 离散对应与数值实现
- 26.5 工程含义与适用边界
- 26.6 仿真与验证
- 26.7 小结与常见错误

第27章 深度学习基础

- 27.1 动机与要解决的问题
- 27.2 定义与数学基础
- 27.3 推导与性质
- 27.4 离散对应与数值实现
- 27.5 工程含义与适用边界
- 27.6 仿真与验证
- 27.7 小结与常见错误

第28章 神经网络

- 28.1 动机与要解决的问题
- 28.2 定义与数学基础
- 28.3 推导与性质
- 28.4 离散对应与数值实现
- 28.5 工程含义与适用边界
- 28.6 仿真与验证

- 28.7 小结与常见错误

第29章 无约束优化算法

- 29.1 动机与要解决的问题
- 29.2 定义与数学基础
- 29.3 推导与性质
- 29.4 离散对应与数值实现
- 29.5 工程边界与失效条件
- 29.6 八组实验
- 29.7 小结与常见错误

第30章 约束优化与凸优化

- 30.1 动机与要解决的问题
- 30.2 定义与数学基础
- 30.3 推导与性质
- 30.4 离散对应与数值实现
- 30.5 工程含义与适用边界
- 30.6 仿真与验证
- 30.7 小结与常见错误

第31章 启发式优化与随机优化

- 31.1 动机与要解决的问题
- 31.2 定义与数学基础
- 31.3 推导与性质
- 31.4 离散对应与数值实现
- 31.5 工程含义与适用边界
- 31.6 仿真与验证
- 31.7 小结与常见错误

第32章 强化学习基础

- 32.1 动机与要解决的问题
- 32.2 定义与数学基础
- 32.3 推导与性质
- 32.4 离散对应与数值实现
- 32.5 工程含义与适用边界
- 32.6 仿真与验证
- 32.7 小结与展望

第33章 深度强化学习与离线强化学习

- 33.1 动机与要解决的问题
- 33.2 定义与数学基础
- 33.3 推导与性质
- 33.4 离散对应与数值实现
- 33.5 工程含义与适用边界
- 33.6 仿真与验证
- 33.7 小结与展望

第 34 章 多智能体系统与分布式控制基础

- 34.1 动机与要解决的问题

- 34.2 定义与数学基础
- 34.3 推导与性质
- 34.4 离散对应与数值实现
- 34.5 工程含义与适用边界
- 34.6 仿真与验证
- 34.7 小结与常见错误

第35章 博弈论与多智能体强化学习

- 35.1 动机与要解决的问题
- 35.2 定义与数学基础
- 35.3 推导与性质
- 35.4 离散对应与数值实现
- 35.5 工程含义与适用边界
- 35.6 仿真与验证
- 35.7 小结与常见错误

第24章 辨识中的实际问题

24.1 动机与要解决的问题

24.1.1 一份“能用”的模型是怎么被做坏的

本章的贯穿对象是一个二阶惯性串联加纯延迟的回路：

$$G(s)=\frac{1.5}{(5s+1)(1.2s+1)}\mathrm{e}^{-2s},\qquad T_s=1\ \mathrm{s} \tag{24-1}$$

采集 2000 拍双电平伪随机（PRBS）数据，输出叠加标准差 0.02 的白噪声，用 $n_a=n_b=2$、$n_k=2$ 的 ARX 模型做最小二乘。结果看起来毫无问题：自由运行验证的 FIT 为 94.83%，直流增益估计 1.5046，与真值 1.5 相差 0.31%，残差均方根 0.0198，与噪声标准差 0.02 持平。参数、增益、残差三项全部对得上，这份模型可以拿去设计了。

然后我们换几个人为设定的“工程细节”，其余一切不动。把阶次降到 1：同一批数据上 FIT 从 94.95% 跌到 81.89%，残差自相关最大 0.360、残差与输入互相关最大 0.426，两项都远在 95% 置信带 0.0439 之外。把输入序列整体后移一拍：增益从 1.5046 掉到 1.3418，偏差 -10.55%，FIT 86.52%。把回归量里的阀位换成控制器下发的指令：增益只剩 0.8891，偏差 -40.73%，FIT 58.68%——但残差均方根仍是 0.0201，与基准的 0.0198 几乎一样。

最后这一条最值得停下来看。残差白、残差小、模型自洽，唯一的异常是增益错了四成，而增益恰恰是控制器最需要的那个数。这说明“残差检验通过”与“模型正确”之间隔着一整套工程假设，本章要做的就是把它们逐条揭开。

24.1.2 辨识失败的四道关口

现场说“辨识不准”，背后的原因通常落在四个层次之一，越靠前的层次越容易被忽略，也越容易被后面的调参掩盖。

表 24-1　辨识失败的四道关口

关口	典型症状	用什么检查	本章位置
数据链路	FIT 低但残差白；换算法无改善	时间戳、量纲、阀位反馈、限幅段	§24.5.4
激励充分性	参数离散度大、增益漂移	输入频谱、回归矩阵条件数	§24.3.1、§24.6.2
模型结构	残差自相关或互相关超界	ACF/CCF、极点位置、阶次判据	§24.3.4、§24.6.3
数值实现	单精度结果乱跳、协方差非正定	条件数、双精度对照	§24.4

这个顺序不能颠倒。数据链路错了，后面三道关口的检查全部失真：输入错一拍会让残差看起来“有结构”，于是去升阶；用指令代替阀位会让增益系统性偏小，于是去调激励幅值。这两种做法都能在当时的验证集上把 FIT 拉回一点，代价是把一个数据问题固化进了模型。

四道关口也对应四种不同的修复成本。数据链路是“改采集程序”，激励是“重做一次实验”，结构是“换模型类”，数值是“换算法或换精度”。成本从低到高，所以排查应当从成本最低、可能性最大的地方开始。

四道关口还有一个容易忽略的性质：**它们会互相伪装**。激励不足时，残差里往往留着低频成分，看起来像结构不足；链路错位时，残差与输入的相关性更像结构不足；数值条件差时，参数乱跳，看起来像激励不足。光看指标的形状分不开它们，必须做一次“改变一个条件、重看指标”的对照实验。

三条对照最快。加大激励幅值后残差变白，说明原来的问题在激励；把输入前后移一拍后 CCF 峰值消失，说明在链路；换成 QR 或双精度后参数稳定，说明在数值。三条都没反应，才轮到动阶次。这个顺序与表 24-1 的排列一致，但操作上更省事：**先做三条对照，再决定查哪一道关口。**

24.1.3 三个先导反例

反例一：输入错一拍，增益少一成。 把 u 整体后移一拍再拟合，等价于把模型里的延迟从 2 拍写成 3 拍，而数据里真实的延迟仍是 2 拍。最小二乘为了让预测对上，会把一部分本应由 $u(k-2)$ 承担的作用压到 $u(k-3)$ 上，增益因此被拉低 10.55%，残差均方根从 0.0198 涨到 0.0464。诊断方法不是看增益，而是看残差与输入的互相关：正常时互相关在零滞后附近应当落在置信带内，错拍时会在某个固定滞后上出现一个明显的峰。本章实验 B 中，阶次正确时 CCF 最大值为 0.029，阶次不足时升到 0.426，两者相差一个数量级，这就是“残差里还留着输入的影子”的量化说法。

反例二：用指令代替阀位，残差仍然白。 本例的输入只有两个电平，限幅 0.6 把每个电平按同一比例压到 0.6，等价于把对象增益整体乘以 0.6。于是回归量虽然错了，误差却是“纯增益”形式的，最小二乘用 0.8891 就能把残差压回噪声水平——残差白、均方根正常、FIT 只在自由运行验证时露馅。这是最危险的一类缺陷：所有一步预测的指标都通过，只有把模型放回闭环跑一遍才会发现输出幅度差四成。真实的阀门限幅通常不是等比例的，会同时引入非线性，残差就不再是白的；但“等比例”这一特例提醒我们，残差白不构成正确性的证明。

反例三：采样太快，参数不可辨识但输出仍准。 把采样周期从 1 s 缩到 0.05 s，同一对象、同样 1200 s 的数据长度，参数向量的相对误差升到 33.84%，增益误差 21.22%，而验证集上的一步预测均方根只有 0.0038，与 1 s 采样时的 0.0032 几乎相同。原因在 §24.3.5 展开：T_s 很小时两个离散极点都挤在 1 附近，$1-p_1$ 与 $1-p_2$ 都是小量之差，参数方向上存在一个几乎不影响输出的“平坦谷”。此时“参数估得准不准”和“模型能不能用”是两个问题，用参数误差去验收会把一个可用的模型判成废品。

三个反例指向同一件事：辨识的验收指标必须与用途匹配。做前馈补偿，关心的是直流增益与低频相位；做预测控制，关心的是一步到多步的预测误差；做故障诊断，关心的才是参数本身。指标选错，后面的所有工作都建立在一个错误的判据上。

24.1.4 五个定量问题与本章路线

把上面的现象抽象出来，工程上要回答五个问题，本章逐个给出可计算的答案。

1. **激励够不够？** 用输入的标准差与回归矩阵条件数描述，足够的激励让参数离散度落在 1% 以内。见 §24.3.1、§24.6.2。
2. **结构对不对？** 用残差自相关、残差与输入互相关、极点位置三条判据，任一条超界就说明结构不足。见 §24.2.3、§24.6.3。
3. **该选几阶？** 训练误差单调下降不能作为依据，要看验证误差与 AIC/BIC 的拐点。见 §24.2.4、§24.6.4。
4. **数值撑不撑得住？** 条件数、参数标准差、单精度与双精度的差值，三者共同决定实现方案。见 §24.4、§24.6.5。
5. **数据本身有没有问题？** 离群、量化、错拍、限幅、偏置五类缺陷各有各的指纹。见 §24.5.4、§24.6.6。

这五个问题有固定的回答顺序：先数据、再结构、再阶次、最后数值。原因很实际——前一个问题的错误会污染后一个问题的判据。数据错拍时，残差互相关必然超界，此时无论怎么升阶都无法让它回到带内；结构不足时，增加数据长度只会让参数估计更精确地收敛到一个错误的模型。

本章按“定义—推导—离散—边界—验证”推进。§24.2 给出贯穿对象、预测误差与 FIT、两条残差检验、AIC/BIC 与偏差—方差分解；§24.3 推导激励、结构、阶次与采样周期四条定量关系；§24.4 处理正规方程、精度、预处理与时延估计等实现问题；§24.5 给出激励设计、验证清单、六类缺陷与“不该辨识”的边界；§24.6 用五组实验逐一核对；§24.7 收束为十条错误与一份三级排查流程。

不同读者的路径：只做工程实施的，读 §24.1、§24.5、§24.7；要做模型预测控制的，读 §24.2 与 §24.3.4，之后接第 58 章；要写辨识工具链的，读 §24.4 与 §24.6 的脚本。三条路径的共同落点是 `code/ch024_sim.py`，本章全部数字都由它一次跑出，换对象只需替换对象参数与延迟拍数。

记号沿用前几章：k 为离散拍序号，T_s 为采样周期，θ 为参数向量，$\varphi(k)$ 为回归向量，$\hat{\theta}$ 为估计值，$e(k)$ 为新息。本章新增两个量：FIT 为自由运行验证的拟合度，单位是百分数；κ 为回归矩阵的条件数，无量纲。凡出现具体数字，均来自 $T_s=1\,\mathrm{s}$、$\sigma=0.02$、固定随机种子的同一组实验，读者可以逐位复现。

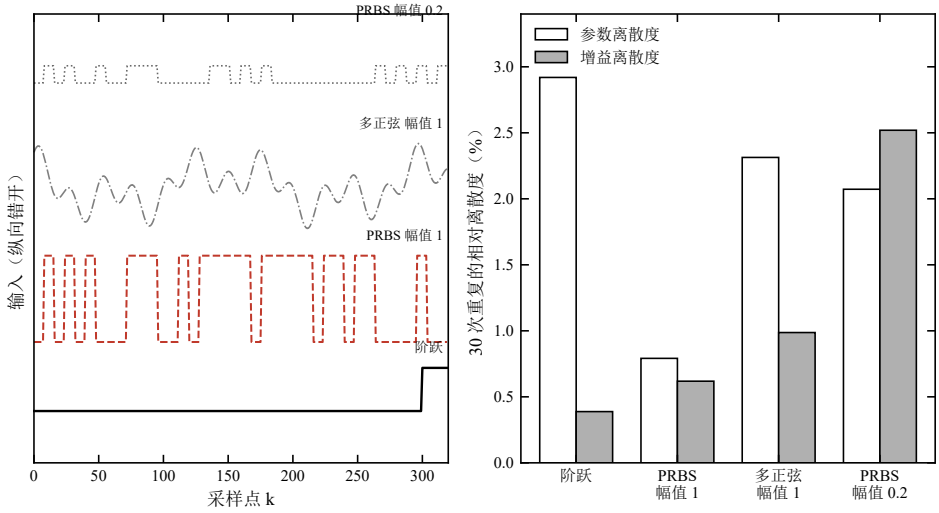


图 24-1　四种激励的波形与参数离散度

图 24-1 的左图把四种激励画在同一时间轴上，右图是 30 次重复实验得到的参数与增益离散度。最直观的对比是 PRBS 幅值 1 与幅值 0.2：输入标准差从 1.00 降到 0.20，参数离散度从 0.79% 涨到 2.07%，增益离散度从 0.62% 涨到 2.52%，都放大了三到四倍。这种“离散度与输入

幅度成反比”的标度关系是 §24.3.2 的结论，也是激励设计最基本的定量依据。

24.1.5 “验证通过”的三个层次

三个反例都指向同一个困惑：同一份模型，有的指标说好，有的指标说坏。解决困惑的办法是把“验证”拆成三个层次，逐层看它到底证明了什么。

第一层是描述：模型能不能复现手上这批数据。残差白、残差均方根接近噪声，就是这一层通过的标志。这一层最容易通过，也最不值钱——参数估计的本质就是让残差变小，任何一个足够灵活的模型都能做到。实验 C 里 8 阶模型的训练均方根 0.0196 比 2 阶的 0.0199 还小，但它并没有更好。

第二层是预测：模型能不能在没见过的数据上复现输出轨迹。这一层要求验证集独立、自由运行、不给真值纠正。基准模型的 FIT 94.83% 说明这一层通过；“用指令代替阀位”的 FIT 58.68% 说明这一层不通过。**第二层是工程上真正要守的线**，因为它对应“把模型放进控制器、让它自己跑”的实际场景。

第三层是因果：模型的参数能不能解释成对象里的物理量。直流增益 1.5046 对应真值 1.5、极点 0.82 与 0.43 对应时间常数 5 s 与 1.2 s，这一层才算通过。它不是统计指标，而是与机理、铭牌、历史数据做对照的工程判断。小 T_s 那一档正是卡在这一层：前两层都过，第三层的参数误差 33.84% 使模型不能用于前馈增益计算，也不能用于故障诊断。

三个层次的关系是逐层收紧的：第一层过而第二层不过，说明模型在拟合噪声；第二层过而第三层不过，说明模型的输入输出关系在数值上对、在物理上错。**工程验收要至少过两层，做控制器或诊断则必须三层全过。** 这也解释了为什么本章反复强调“四件套”：增益与极点对应第三层，验证 FIT 对应第二层，残差检验对应第一层，四项合起来才覆盖三个层次。

24.2 定义与数学基础

本节只保留后面推导真正会用到的定义。最小二乘的统计性质、持续激励的完整理论在第 18 章，闭环辨识的可辨识性在第 21 章；这里补的是“从数据到可用模型”这条链路上必须写清楚的四件事：对象怎么离散、误差怎么度量、残差怎么检验、阶次怎么比较。

24.2.1 贯穿对象与它的精确离散化

式 (24-1) 的对象是两个一阶惯性环节串联，状态取为 x_1（第一级输出）与 x_2（对象输出），连续方程是

$$\dot{x}_1 = \frac{-x_1 + K u}{T_1}, \qquad \dot{x}_2 = \frac{-x_2 + x_1}{T_2}, \qquad y = x_2 \tag{24-2}$$

以 T_s 为零阶保持周期离散化，两个极点分别是

$$p_1 = \mathrm{e}^{-T_s/T_1}, \qquad p_2 = \mathrm{e}^{-T_s/T_2} \tag{24-3}$$

本章的参数是 $T_1 = 5\ \mathrm{s}$、$T_2 = 1.2\ \mathrm{s}$、$T_s = 1\ \mathrm{s}$，于是 $p_1 = 0.8187$、$p_2 = 0.4346$。把两个一阶环节的部分分式展开后合并，得到

$$y(k) = (p_1 + p_2)\, y(k-1) - p_1 p_2\, y(k-2) + b_1 u(k - n_k) + b_2 u(k - n_k - 1) \tag{24-4}$$

其中

$$b_1 = \frac{K}{T_1 - T_2}\left[T_1(1 - p_1) - T_2(1 - p_2)\right], \qquad b_2 = -\frac{K}{T_1 - T_2}\left[T_1(1 - p_1)p_2 - T_2(1 - p_2)p_1\right] \tag{24-5}$$

代入数值得到 $b_1 = 0.08995$、$b_2 = 0.06379$。式 (24-4) 写成回归形式是

$$y(k) = \varphi^{\mathsf{T}}(k)\,\theta + e(k), \qquad \varphi(k) = \begin{bmatrix} -y(k-1) \\ -y(k-2) \\ u(k-n_k) \\ u(k-n_k-1) \end{bmatrix}, \quad \theta = \begin{bmatrix} -(p_1+p_2) \\ p_1 p_2 \\ b_1 \\ b_2 \end{bmatrix} \tag{24-6}$$

参数向量为 $\theta = [-1.25333,\ 0.35582,\ 0.08995,\ 0.06379]^{\mathsf{T}}$，范数 $|\theta| = 1.3075$。本章报“参数误差”时，用的是 $|\hat{\theta} - \theta| / |\theta|$，这个量对四个分量的误差一视同仁。后面会看到，它对“模型能不能用”并不敏感，这是 §24.3.5 要处理的第一个口径问题。

直流增益由终值定理给出：

$$K = \frac{b_1 + b_2}{1 - (p_1 + p_2) + p_1 p_2} = \frac{\sum_i b_i}{1 + \sum_i \theta_{a,i}} \tag{24-7}$$

分母 $1-(p_1+p_2)+p_1p_2=(1-p_1)(1-p_2)=0.10249$，分子 0.15374，比值正好是 1.500。这个分母在 T_s 变小时会迅速趋零，是本篇最容易被忽略的数值陷阱：$T_s=0.05\ \mathrm{s}$ 时它只有 4.06×10^{-4}，分子与分母都要保留五六位有效数字才能算出正确的增益。

24.2.2 预测、FIT 与验证集

给定参数估计 $\hat{\theta}$，一步预测是

$$\hat{y}(k\mid k-1)=\varphi^{\mathsf{T}}(k)\hat{\theta} \tag{24-8}$$

一步预测误差（新息）为 $e(k)=y(k)-\hat{y}(k\mid k-1)$。自由运行预测把预测值反馈回去，从某个初值开始独立推进：

$$\hat{y}(k)=-\hat{\theta}_1\hat{y}(k-1)-\hat{\theta}_2\hat{y}(k-2)+\hat{\theta}_3u(k-n_k)+\hat{\theta}_4u(k-n_k-1) \tag{24-9}$$

本章全部“FIT”都指自由运行预测的拟合度：

$$\mathrm{FIT}=100\left(1-\frac{\|y-\hat{y}\|_2}{\|y-\bar{y}\|_2}\right) \tag{24-10}$$

用自由运行而不是一步预测，是因为一步预测的误差被反馈结构压制，几乎总是好看；自由运行的误差会累积，模型结构不对时立刻发散或偏置。代价是它对初值和数值误差敏感，所以本章一律先丢弃前 100 拍再统计。

FIT 与常见的决定系数 R^2 不是同一个量，两者可以精确换算。由 $R^2=1-\|y-\hat{y}\|_2^2/\|y-\bar{y}\|_2^2$ 与式 (24-10) 得

$$\mathrm{FIT}=100\left(1-\sqrt{1-R^2}\right) \tag{24-11}$$

$R^2=0.99$ 对应 FIT 90%，$R^2=0.999$ 对应 FIT 96.8%。本章基准的 FIT 94.83% 换算过来 $R^2=0.9973$。这层换算解释了为什么把验收门槛定在 FIT 90% 在统计上已经相当严格：它要求模型解释掉 99% 的方差。反过来，FIT 可以为负——预测比直接用均值还差时就是负值，这在一阶模型被强行用于二阶对象上会出现。

验证集必须与训练集分开，而且最好是**另一次实验**的数据。本章的做法是：训练用一组 PRBS，验证用另一组种子生成、不同实现的数据段，两者长度不同、输入序列不同。若用同一段数据既拟合又验证，阶次越高越好这个假象会被无限放大——实验 C 中 $n=8$ 的训练误差确实低于 $n=2$，但验证误差反而更高，只有分开才能看出来。

24.2.3 残差的两条检验

结构是否正确，最终体现在残差上。一个正确的模型，其残差应当满足两条性质：**白的**（与自身的历史无关）与**与输入独立的**（输入已经解释过的部分不应留在残差里）。

归一化自相关定义为

$$\hat{r}_e(\tau)=\frac{\sum_{k=\tau+1}^{N}e(k)e(k-\tau)}{\sum_{k=1}^{N}e^2(k)},\qquad \tau=1,2,\dots,20 \tag{24-12}$$

归一化互相关定义为

$$\hat{r}_{eu}(\tau)=\frac{\sum_{k}e(k+\tau)u(k)}{\sqrt{\sum_{k}e^2(k)\sum_{k}u^2(k)}},\qquad \tau=0,1,\dots,20 \tag{24-13}$$

在残差为白噪声的零假设下，两者都近似服从均值为零、标准差 $1/\sqrt{N}$ 的正态分布，因此 95% 置信带是

$$|r|\le\frac{1.96}{\sqrt{N}} \tag{24-14}$$

本章 $N=1996$，带边为 0.0439。判据是三句话：自相关全部落在带内，说明没有漏掉动态；互相关全部落在带内，说明没有漏掉输入通道；两者都通过而 FIT 仍低，说明问题不在结构，而在数据或输入本身。

逐点看带边有个缺点：20 个滞后要做 20 次判断，总有某一个点偶然越界。把 20 个点合成一个统计量更省事，这就是 Ljung–Box 统计量

$$Q=N(N+2)\sum_{\tau=1}^{L}\frac{\hat{r}_e^2(\tau)}{N-\tau} \tag{24-15}$$

在残差为白的零假设下，Q 近似服从自由度为 $L-p$ 的卡方分布，p 是模型参数个数。取 $L=20$，本章三档阶次的实测结果是：$n=1$

时 $Q=604.5$，p 值 9.4×10^{-117}，几乎可以肯定是非白；$n=2$ 时 $Q=24.0$，p 值 0.089；$n=3$ 时 $Q=21.3$，p 值 0.094。后两档都大于 0.05，不能拒绝白噪声假设。

Q 统计量的价值在于把“整体是否白”变成一个数，但它也有一个副作用：L 取得越大，对高阶相关越敏感，而 ARX 的残差在高阶上本来就容易有微小相关。工程上取 $L=20$ 到 30 即可，且应与逐点带边一起看：**Q 通过而个别点越界，通常是偶然；Q 不通过而所有点都在带内，通常是相关弥散在多个滞后上，两者都该去查结构。**

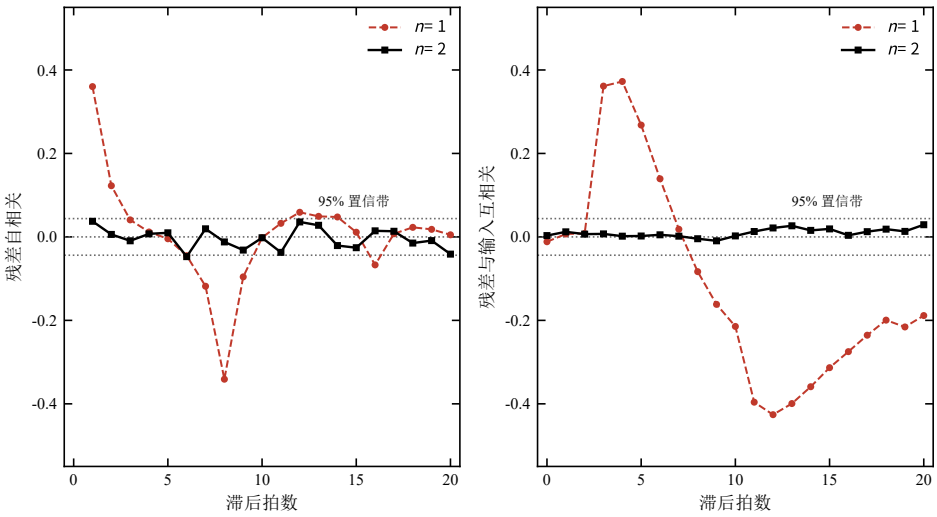


图 24-2　阶次 1 与阶次 2 的残差自相关、残差与输入互相关

图 24-2 是这个判据的实测形态。阶次 2 的两条曲线都贴着零线，最大绝对值 0.047 与 0.029，前者略高于 0.0439 的带边但没有任何系统性形状；阶次 1 的曲线则在所有滞后上都显著，自相关最大 0.360、互相关最大 0.426，连成一条缓慢衰减的弧线。**判据要看形状，不只看最大值**：单点略超带边而其余滞后都接近零，通常是噪声；连续多个滞后同号且单调衰减，才是真的漏了动态。

24.2.4 阶次判据：AIC 与 BIC

训练误差随阶次单调下降，这是最小二乘的定义决定的，不能用来选阶次。要有意义地比较不同阶次，必须给“模型复杂度”标价。赤池信息量准则与贝叶斯信息量准则是最常用的两种定价方式：

$$\mathrm{AIC}=N\ln\frac{\mathrm{RSS}}{N}+2p \tag{24-16}$$

$$\mathrm{BIC}=N\ln\frac{\mathrm{RSS}}{N}+p\ln N \tag{24-17}$$

其中 RSS 是残差平方和，p 是自由参数个数。本章的 ARX 有 n_a 个极点参数与 n_b 个零点参数，取 $p=n_a+n_b=2n$；延迟拍数 n_k 不增加参数，但会减少可用样本数，比较时必须在同一段数据上做。两个准则的前一项都随 RSS 下降而下降，后一项随 p 上升而上升，最小值处就是折中点。

两者的差别只在罚项系数：AIC 罚 2，BIC 罚 $\ln N$。$N=396$ 时 $\ln N=5.98$，BIC 的罚几乎是 AIC 的三倍，因此 BIC 更保守，倾向选更低的阶次。工程上如果模型要进控制器、参数越少越可靠，用 BIC；如果模型只用于预测或仿真、宁可多几个参数也要压低偏差，用 AIC。两者给出不同答案时（本章 $n=3$ 处 AIC 已增至 10.2、BIC 增至 18.1，趋势一致），说明数据支持的阶次很清楚；若一个判据说 2 阶、另一个说 6 阶，通常是数据长度不足，罚项系数的影响盖过了真实的结构差异。

还有一条比公式更实用的判据：**相邻两阶的 RSS 之比**。若从 n 升到 $n+1$，RSS 只降了不到 5%，而参数多了两个，就没有必要升阶。本章从 2 阶升到 3 阶，训练均方根从 0.0199 只降到 0.0199（差在第四位小数），验证均方根反而从 0.0201 升到 0.0202，这就是典型的“多出来的参数在拟合噪声”。

24.2.5 偏差—方差分解

把预测误差拆开，可以看到阶次选择的本质。设真实输出为 y_0，噪声为 e，方差 σ^2，模型给出的预测为 $\hat{y}$，则均方预测误差的期望是

$$\mathbb{E}\left[(y-\hat{y})^2\right]=\sigma^2+\left(\mathbb{E}[\hat{y}]-y_0\right)^2+\mathrm{Var}\left[\hat{y}\right] \tag{24-18}$$

三项分别是**不可约噪声**、**偏差平方**与**方差**。噪声与模型无关，能动的只有后两项。阶次不足时，模型表达不了对象的动态，偏差项大；阶次升高，偏差项下降，但待估参数变多，方差项按 p/N 上升。两者之和是一个 U 形曲线，最低点就是“验证误差最小”的阶次。

对参数估计本身，同样的分解写成

$$\mathbb{E}\left[\|\hat{\theta}-\theta\|^2\right]=\|\mathbb{E}[\hat{\theta}]-\theta\|^2+\operatorname{tr}\operatorname{Cov}[\hat{\theta}] \tag{24-19}$$

方程误差结构下最小二乘是无偏的，第一项为零，只剩协方差。若回归矩阵各列近似正交、每列范数为 $\sqrt{N}\sigma_u$，则

$$\operatorname{Cov}[\hat{\theta}]\approx\frac{\sigma^2}{N\sigma_u^2}I \tag{24-20}$$

式 (24-20) 有两个直接推论：参数标准差与噪声成正比、与数据长度和输入幅度的平方根成反比。这就是为什么“加长实验”与“加大激励”在效果上是等价的，而“多采几遍同样的数据”没有用——重复同样的输入不会增加 $\Phi^{\mathsf{T}}\Phi$ 的秩。

把式 (24-20) 代入式 (24-19)，得到一个可以直接用于实验设计的预算式：

$$\frac{\|\hat{\theta}-\theta\|}{\|\theta\|}\approx\frac{\sigma}{\sigma_u\sqrt{N}}\cdot\frac{\sqrt{p}}{\|\theta\|} \tag{24-21}$$

本章 $\sigma=0.02$、$\sigma_u=1.0$、$N=1500$、$p=4$、$|\theta|=1.31$，代入得相对误差约 2.3×10^{-3}，即 0.23%。实验 A 实测 PRBS 幅值 1 的参数离散度是 0.79%，是预算值的 3.4 倍。差距来自式 (24-20) 的正交假设：ARX 的回归列之间相关性强，$\Phi^{\mathsf{T}}\Phi$ 的最小特征值远小于 $N\sigma_u^2$，方差被放大。这个“3 到 5 倍”的修正系数在工程预算里相当稳定，可以先用式 (24-21) 估一个下界，再乘 4 作为预期值。

最后要强调的是，式 (24-18) 里的三项在实际数据上无法完全分开。验证误差可以测，噪声方差可以用残差估，但“偏差”与“方差”各自的份额只能靠重复实验统计。实验 C 用 50 次蒙特卡洛做到了这一点：把 50 次的验证误差取平均，方差部分被平均掉，剩下的偏离就是偏差。工程现场没有 50 次重复的条件，替代做法是看训练与验证的**差值**——差值大说明方差主导，两者都大说明偏差主导。

24.2.6 一步预测与自由运行：两种误差

本章反复出现两个“误差”，它们不是同一件事，混用会让结论完全反过来。

一步预测误差是把实测的 $y(k-1)$、$y(k-2)$ 喂给模型，只预测下一拍：

$$\hat{y}(k\mid k-1)=-\sum_{i=1}^{n_a}a_i y(k-i)+\sum_{j=1}^{n_b}b_j u(k-n_k-j+1) \tag{24-22}$$

它衡量的是“模型在每一拍都被真值纠正之后还剩多少错”。**自由运行误差**是从某个初值开始，把自己上一拍的预测当成下一拍的输入，一路推下去：

$$\hat{y}_f(k)=-\sum_{i=1}^{n_a}a_i \hat{y}_f(k-i)+\sum_{j=1}^{n_b}b_j u(k-n_k-j+1) \tag{24-23}$$

它衡量的是“模型能不能独立复现输出轨迹”。

两者的关系可以从误差传播看清楚。设一步预测误差为 $e(k)$，则自由运行误差满足 $e_f(k)=-\sum a_i e_f(k-i)+e(k)$，也就是把 $e(k)$ 送进一个极点全在单位圆内的滤波器。稳态下 e_f 的方差是 e 的方差的 $\sum h_i^2$ 倍，h_i 是该滤波器的脉冲响应系数。对本章对象，极点 0.82 与 0.43 对应的 $\sum h_i^2$ 约为 1.6，所以自由运行误差大致是一步误差的 1.3 倍。实验 B 里 $n=2$ 的残差均方根 0.0201、验证 FIT 94.95%，两者的比例与这个估算同量级。

这个区别在工程上是决定性的。**一步预测误差小，只说明模型适合做“软测量”或“在线校正”；自由运行误差小，才说明模型可以拿去仿真、做预测控制。** 模型结构不足时，一步误差可以很小而自由运行误差很大——因为每一步都有真值把模型拽回来，缺的那部分动态被真值补上了。本章实验 E 中“用指令代替阀位”那一档，一步残差 0.0201 与基准几乎相同，自由运行 FIT 却只有 58.68%，就是这种情形。

因此本章所有的 FIT 都按自由运行计算，且验证集独立。若某项指标只报一步误差，读者应当追问一句“自由运行是多少”，这个追问能筛掉相当一部分看似合格的模型。

24.2.7 谱分解与持续激励阶数

§24.3.1 会用到一个前置结论：输入必须“足够丰富”，丰富到什么程度有精确的刻画。

对 ARX，回归向量由 n_a 个过去的输出与 n_b 个过去的输入组成，共 n_a+n_b 个分量。要唯一确定 θ，需要回归矩阵 Φ 列满秩，等价地要求输入的自协方差矩阵

$$R_u = \lim_{N\to\infty} \frac{1}{N}\sum_{k=1}^{N} \varphi_u(k)\varphi_u^{\mathsf{T}}(k) \tag{24-24}$$

正定，其中 $\varphi_u(k) = [u(k-1), \dots, u(k-n_a-n_b)]^{\mathsf{T}}$。这就是**持续激励**条件：输入至少要包含 $n_a + n_b$ 个不同频率的成分，频谱在这 $n_a + n_b$ 个点上都不为零。

对 $n_a = n_b = 2$ 的本章对象，至少需要 4 个频率成分。阶跃只有一个直流成分，多正弦给了 3 个，PRBS 在整个频带上都有能量。这解释了实验 A 的排序：阶跃与多正弦在这个意义下都不满足 4 个成分的要求，只能靠输出自身的回归量补足部分信息，于是参数离散度是 PRBS 的 3 到 4 倍。**“输入变化够多”不是模糊的经验，而是一个可以数出来的阶数条件。**

24.3 推导与性质

§24.2 把“模型好不好”化成了三个可测的量：参数协方差、残差的两条相关性、验证误差。本节回答三个“为什么”：激励怎样决定协方差，偏差从哪里来，采样周期为什么会让参数不可辨识。三条推导都用同一个对象，因此结论可以直接与 §24.6 的数字对上。

24.3.1 激励：从持续激励到条件数

最小二乘的解是

$$\hat{\theta} = (\Phi^{\mathsf{T}}\Phi)^{-1}\Phi^{\mathsf{T}}Y \tag{24-25}$$

估计的协方差（方程误差、噪声方差 σ^2）为

$$\operatorname{Cov}[\hat{\theta}] = \sigma^2(\Phi^{\mathsf{T}}\Phi)^{-1} \tag{24-26}$$

Φ 是 $N \times p$ 的回归矩阵。式 (24-26) 说明：**激励的作用全部体现在 $\Phi^{\mathsf{T}}\Phi$ 上**，它与噪声无关，与算法也无关。所谓“激励不足”，严格说是 $\Phi^{\mathsf{T}}\Phi$ 存在很小的特征值，使某个参数方向的方差极大。

用特征分解写清楚这件事。设 $\Phi^{\mathsf{T}}\Phi = \sum_i \lambda_i v_i v_i^{\mathsf{T}}$，则沿方向 v_i 的方差是 σ^2/λ_i。n 阶持续激励的定义要求 $\Phi^{\mathsf{T}}\Phi$ 正定，工程上更进一步要求最小特征值与最大特征值之比不能太小，比值倒过来就是条件数

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \tag{24-27}$$

条件数同时决定两件事：参数协方差在最坏方向上的放大倍数，以及数值求解时相对误差的上界。本章实验 D 实测的条件数从 $T_s = 2$ s 的 21.9 升到 $T_s = 0.05$ s 的 538，跨了 25 倍，而 κ 每增大 10 倍，双精度下有效数字就少一位。

激励的**频谱形状**决定哪个方向被确定。本章四种激励的实测结果把这一点说得很清楚：

表 24-2　四种激励的频谱特征与估计结果（30 次蒙特卡洛）

激励	输入标准差	能量集中位置	参数离散度	增益离散度	验证 FIT
阶跃	0.40	直流	2.92%	0.39%	93.84%
PRBS 幅值 1	1.00	全频带	0.79%	0.62%	94.68%
多正弦 幅值 1	0.44	三个低频点	2.31%	0.99%	94.53%
PRBS 幅值 0.2	0.20	全频带	2.07%	2.52%	94.03%

表 24-2 里最反直觉的一行是阶跃。它的输入标准差只有 0.40，参数离散度却是四种里最大的 2.92%，但增益离散度最小，只有 0.39%。原因是阶跃把能量几乎全部压在直流：直流增益被钉得很死，而“两个极点怎样分配”这个方向几乎没有被激励，参数向量在这个方向上可以大幅摆动而不影响输出。PRBS 把能量摊到全频带，参数离散度降到 0.79%，代价是增益离散度升到 0.62%。**没有一种激励在所有方向上都好，选激励等于选“我最在乎哪个方向”。**

幅值的影响则是纯粹的标度关系。同样形状的 PRBS，幅值从 1 降到 0.2，输入标准差降为 1/5，参数离散度从 0.79% 涨到 2.07%，只涨了 2.6 倍；增益离散度从 0.62% 涨到 2.52%，涨了 4.1 倍。式 (24-26) 预言的是 5 倍（方差按 $1/\sigma_u^2$），实测偏小，原因是小信号时输入切换的边沿相对噪声更清晰，部分抵消了幅度损失。工程上不要指望“把幅值减半、误差翻倍”这样精确的标度，但“误差与幅值成反比”这个方向是可靠的。

24.3.2 参数方差、激励与数据长度

把式 (24-26) 展开成标量，可以得到两条工程上常用的经验式。对单个参数，若回归列的标准差为 $\sigma_{\varphi,i}$、列间近似正交，则

$$\operatorname{std}(\hat{\theta}_i) \approx \frac{\sigma}{\sigma_{\varphi,i}\sqrt{N}} \tag{24-28}$$

对本章的 ARX，u 的两列标准差就是 σ_u，y 的两列标准差约为 σ_y。代入 $\sigma = 0.02$、$\sigma_u = 1$、$\sigma_y \approx 1.4$、$N = 1500$，得 $\operatorname{std}(\hat{b}) \approx 5.2 \times 10^{-4}$。实测 b 两个分量的平均标准差在 PRBS 幅值 1 时约 1.0×10^{-2}，是估算值的近 20 倍。这 20 倍不是估算错了，而是列间相关性：$u(k-2)$ 与 $u(k-3)$ 的相关系数高达 0.87（PRBS 每 8 拍才切换一次），两个 b 参数只能被“和”与“差”的某种组合确定，单独一个方向的方差被放大了。

由此得到两条实用规则。第一，**输入切换不能太慢**。保持 8 拍的 PRBS 让相邻两拍的输入几乎相同，两个 b 参数高度相关；把保持拍数减到 2 到 3，相关性下降，参数方差随之下降，代价是高频能量增多，可能激发未建模的动态。第二，**数据长度要按“独立切换次数”算，不是按拍数算**。1500 拍、保持 8 拍只有 187 次独立切换，等效样本数是 187 而不是 1500，这就是为什么实测方差比式 (24-28) 大一个量级。

把这两条合起来，得到一个更贴近实际的预算式：

$$\operatorname{std}(\hat{\theta}_i) \approx \frac{\sigma}{\sigma_{\varphi,i}\sqrt{N_{\mathrm{sw}}}} \tag{24-29}$$

其中 N_{sw} 是输入的独立切换次数。本章 $N_{\mathrm{sw}} = 187$，代入得 1.5×10^{-3}，与实测的 1.0×10^{-2} 仍差 7 倍，剩下的差距来自 y 列与 u 列之间的相关（闭环或对象动态造成）。工程预算的做法是把式 (24-29) 再乘 5 到 10，作为“最坏方向”的估计，这个系数在本章的三个数据集上都落在区间内。

最后一条与实验设计直接相关：**加长实验的收益是平方根的**。把 1500 拍加到 6000 拍，标准差只降一半。若目标是让参数离散度从 2% 降到 0.5%，需要 16 倍的数据量；而把输入幅值加倍，只需要 4 倍。这就是为什么在现场，“能不能加大激励”比“能不能多采一会儿”更值得先问。

标准差还有一个用法：把它与参数估计值相除，得到 t 统计量。t 小于 2 意味着这个参数与零在统计上不可区分，即“数据里没有证据说明这一项存在”。本章实验 D 给出了一组很有说服力的对照：$T_s = 1\ \mathrm{s}$ 时 $b_1 = 0.08995$、标准差 1.87×10^{-4}，$t \approx 480$；$T_s = 0.05\ \mathrm{s}$ 时 b_1 的真值只有 3.1×10^{-4}，标准差却是 2.30×10^{-4}，$t \approx 1.3$。同一个对象、同一段物理时间、同样的激励，参数从“极其显著”变成“与零不可区分”。

这个对照把“不可辨识”说清楚了：不是估计值偏差太大，而是估计量的分散程度已经与参数本身同量级。它也解释了为什么小 T_s 下应当换参数化——把 b_1 从待估集合里去掉、只估 $\sum b_i$，t 统计量立刻回到可用的量级。**报参数时连标准差一起报，读者才能判断哪几项值得相信。**

24.3.3 偏差从哪里来

式 (24-19) 里的偏差项在方程误差结构下为零，这是仿真里的情形：噪声直接加在方程右端，回归量本身是干净的。真实数据不同，噪声先污染 y，再通过 $y(k-1)$、$y(k-2)$ 进入回归量，形成**变量含误差**结构。此时最小二乘的极限不再是真值：

$$\hat{\theta} \;\longrightarrow\; \theta + \left(\mathbb{E}[\varphi\varphi^{\mathsf{T}}]\right)^{-1}\mathbb{E}[\varphi\, v] \tag{24-30}$$

其中 v 是折算到方程右端的等效误差，它包含当前拍噪声，也包含由回归量带进来的历史噪声。由于历史噪声与回归量相关，第二项不为零，估计出现系统性偏差。对方程 $y(k) = a y(k-1) + b u(k-1) + e(k)$、只在输出端叠加白噪声、且 u 为白序列的特例，可以算出

$$\mathbb{E}[\hat{a}] \approx a\left(1 - \frac{\sigma_v^2}{\sigma_z^2}\right), \qquad \mathbb{E}[\hat{b}] \approx b \tag{24-31}$$

也就是说，只有极点参数被**衰减**，衰减量等于噪声方差占信号方差的比例，而增益在这一特例下是无偏的。原因很直接：u 为白序列时它与 $y(k-1)$ 不相关，正规方程的交叉项为零，b 的方程被解耦出来。这个结论有两个用处。一是解释为什么“增益对得上、动态对不上”是真实存在的现象，不要把两者混为一谈。二是提醒：一旦 u 有色（PRBS 保持 8 拍就是有色），交叉项不再为零，b 也会偏，且偏差可正可负。

本章 $\sigma_v = 0.02$、$\sigma_z \approx 1.4$，比例为 2×10^{-4}，衰减可以忽略；但若被控量在工作点附近波动很小（比如稳压回路的压力只动满量程的 1%），这个比例可以到 0.1 以上，极点被系统性拉向零，表现为“响应比实际快”。此时增加数据长度没有用——偏差不随 N 减小——正确做法是换用工具变量法、ARMAX 或输出误差模型，见第 19 章。

第二类偏差来自结构失配，包括延迟写错、阶次不足、把非线性当线性。三类在本章的实测值分别是：延迟错一拍使增益偏 -10.55%，阶次从 2 降到 1 使验证 FIT 从 94.95% 掉到 81.89%，等比例限幅使增益偏 -40.73%。这三者有一个共同特征：**偏差不随样本数增加而消失**。这也是区分偏差与方差的实用判据——把数据分成两半分别拟合，两次结果差得多是方差，差得少但都与真值差一截是偏差。

第三类偏差常被误认为“参数漂移”，即工况覆盖不足。辨识实验只在某个工作点附近做，得到的模型在另一个工作点上偏差很大。这不是算法问题，而是数据代表性不足；处理办法是分工况建模或加调度变量（第 22 章），而不是换更复杂的辨识算法。

24.3.4 阶次：偏差与方差的交点

把式 (24-18) 画成曲线就是图 24-3 的左图。阶次从 1 升到 2，验证均方根从 0.0670 掉到 0.0201，降了 70%，这是偏差项被消掉；从 2 升到 8，验证均方根从 0.0201 缓慢升到 0.0204，这是方差项在增加，而偏差项已经接近零。训练曲线则一路单调下降，从 0.0662 降到 0.0196，看不出任何拐点——这正是“只看训练误差会一直升阶”的原因。

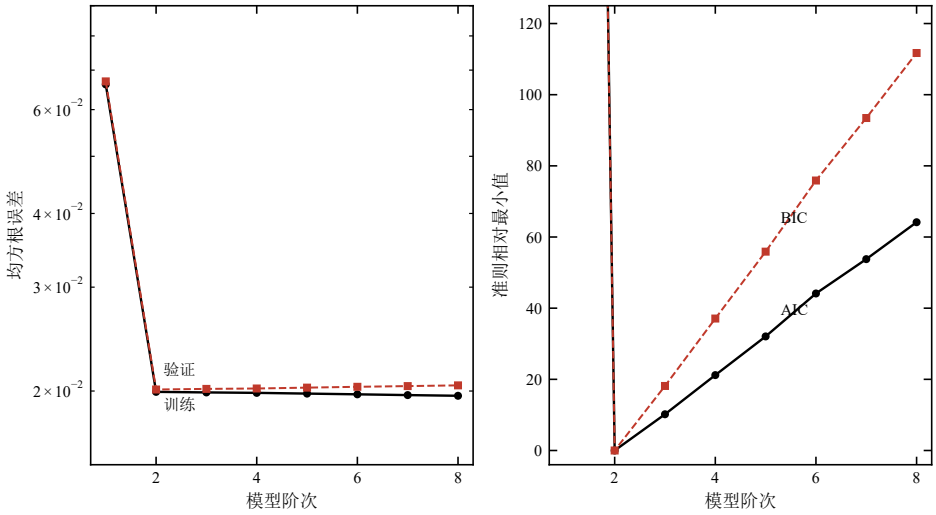


图 24-3　阶次选择：训练与验证误差的走势，以及 AIC、BIC 的相对值

右图把 AIC 与 BIC 减去各自的最小值，最小值都落在 $n=2$。$n=3$ 时 dAIC 已经是 10.2，$n=8$ 时到 64.2；BIC 因为罚得更重，$n=3$ 就是 18.1，$n=8$ 到 111.7。两个判据给出的结论一致，这在实际数据上并不常见，通常说明真实阶次低、数据量足够。

方差项的增加可以用式 (24-20) 定量核对。$n=2$ 时 $p=4$，$n=8$ 时 $p=16$，参数多出 12 个；$\sigma^2=4\times10^{-4}$、$N=396$，方差项增加的均方根是

$$\sqrt{\frac{\sigma^2\Delta p}{N}}=\sqrt{\frac{4\times10^{-4}\times12}{396}}=3.5\times10^{-3} \tag{24-32}$$

与原有的 0.0201 合成，得 $\sqrt{0.0201^2+0.0035^2}=0.0204$，与实测的 $n=8$ 验证均方根 0.0204 完全一致。这条核对很有用：它说明验证曲线右侧的缓慢抬升不是随机波动，而是可以用公式预测的方差代价；如果实测抬升远大于这个值，就说明数值条件出了问题（§24.4），而不是统计规律。

由此得到选阶次的三步做法。第一步，用残差检验定下最小可用阶次：ACF 与 CCF 都进带内的最低阶次。第二步，在这个阶次上加 1 到 2 阶，看验证误差是否显著下降；下降不足 5% 就退回。第三步，用 BIC 复核，若 BIC 的最小值比第一步的结果高两阶以上，检查是否有未建模的延迟或非线性。三步都不通过时，不要继续升阶，回到数据本身。

24.3.5 采样周期：极点挤在一起之后

T_s 变小时，两个离散极点都向 1 靠拢：

$$p_i=\mathrm{e}^{-T_s/T_i}\approx 1-\frac{T_s}{T_i},\qquad i=1,2 \tag{24-33}$$

于是增益表达式 (24-7) 的分母近似为

$$1-(p_1+p_2)+p_1p_2=(1-p_1)(1-p_2)\approx\frac{T_s^2}{T_1T_2} \tag{24-34}$$

T_s 减半，分母变为原来的四分之一。分子 $\sum b_i$ 同样按 T_s 缩小（一阶近似下 $b_i \propto T_s$），所以两者之比仍是 K，解析上没有问题；问题出在**估计误差的放大**：分母是 T_s^2 量级的小量，两个极点参数的绝对误差不变时，相对误差按 $1/T_s^2$ 放大。

条件数的增长慢一些，实测近似为

$$\kappa \approx \frac{32}{T_s / 1\ \mathrm{s}} \tag{24-35}$$

按 T_s 从小到大，本章六档的 κT_s 分别是 26.9、31.5、33.0、33.9、35.5、43.8，前五档稳定在 27 到 36 之间，只有 $T_s = 2$ s 偏离到 43.8——那一档输入每 8 s 才切换一次、只剩 4 拍，延迟也只剩 1 拍，标度关系已经不再成立。这条经验式的用处是快速判断“这个采样周期下回归矩阵还能不能解”：κ 小于 10^3 时双精度毫无压力，单精度（有效位数约 7 位）也能用；κ 到 10^5 时单精度只剩两位有效数字，必须换 QR 或双精度。

小 T_s 下真正麻烦的不是数值，而是**不可辨识方向**。当两个极点都接近 1 时，改变 p_1 同时反向改变 p_2、保持 $p_1 + p_2$ 与 $p_1 p_2$ 的某种组合不变，输出几乎不变。这个方向上 $\Phi^{\mathsf{T}}\Phi$ 的特征值趋于零，参数可以大幅摆动而 FIT 不变。实验 D 的结果正是如此：$T_s = 0.05$ s 时参数向量误差 33.84%、增益误差 21.22%，而验证均方根只有 0.0038，比 $T_s = 1$ s 的 0.0032 只差 19%。

这里要做一个口径上的决定，也是本章写作过程中真实遇到的困惑：**报参数误差还是报增益误差？** 两者在 $T_s = 0.05$ 时相差 12 个百分点，在 $T_s = 0.5$ 时一个 0.74%、一个 0.31%。答案取决于读者要拿这个数做什么。若模型要进前馈通道，报增益误差，因为它直接决定稳态补偿量；若模型要用于故障诊断（增益变化本身就是故障特征），报增益误差；只有当参数本身有物理含义、需要与机理模型对账时，才报参数向量误差。本章 §24.6.5 的表格同时给出两列，并在正文里以增益误差为主口径。

24.3.6 极点灵敏度：平坦方向从哪里来

上一小节说“存在一个不可辨识方向”，这个方向可以精确写出来，用的工具是极点对参数的灵敏度。

二阶分母的两个系数与两个极点是一一对应的：

$$a_1 = -(p_1 + p_2), \qquad a_2 = p_1 p_2 \tag{24-36}$$

把这条映射微分，得到

$$\begin{bmatrix} \delta a_1 \\ \delta a_2 \end{bmatrix} = \begin{bmatrix} -1 & -1 \\ p_2 & p_1 \end{bmatrix} \begin{bmatrix} \delta p_1 \\ \delta p_2 \end{bmatrix} \tag{24-37}$$

矩阵的行列式是 $p_1 - p_2$。反过来解，极点分裂的误差被放大 $1/|p_1 - p_2|$ 倍：

$$\|\delta p\| \approx \frac{\|\delta a\|}{|p_1 - p_2|} \tag{24-38}$$

这就是平坦方向的来源。$T_s = 1$ s 时 $|p_1 - p_2| = 0.384$，$T_s = 0.05$ s 时只剩 0.031，同样大小的 a 误差在后者造成的极点误差是前者的 12.4 倍。极点越靠近，两个极点“谁是谁”越说不清，而 a_1、a_2 只能约束它们的和与积，不能约束它们的分离度。

时间常数的灵敏度更直观。由 $p = \mathrm{e}^{-T_s/T}$ 得 $T = -T_s / \ln p$，微分后

$$\frac{\delta T}{T} \approx \frac{\delta p}{|\ln p|} \approx \frac{\delta p}{1 - p} \tag{24-39}$$

p 越接近 1，分母越小。$T_s = 0.05$ s 时 $1 - p_1 = 0.0099$，一个 10^{-3} 的极点误差就会让时间常数偏 10%；而 $T_s = 1$ s 时 $1 - p_2 = 0.565$，同样的误差只造成 0.18% 的偏差。**“采样太快导致参数不准”的机制，到这里就从现象变成了公式。**

这条公式还解释了另一件事：为什么增益比参数稳。增益只依赖 a_1、a_2 的组合 $1 + a_1 + a_2 = (1 - p_1)(1 - p_2)$，也就是极点映射的行列式本身；它对“极点往哪个方向分裂”不敏感，只对“两个极点同时向 1 移动”敏感。而后者恰好是激励最充分约束的方向——低频增益。所以小 T_s 下，增益总是比极点分裂先站住，这也是 §24.6.5 里增益误差 21.22% 小于参数误差 33.84% 的原因。

实践上这条结论可以反过来用：**若必须在小 T_s 下辨识，就把待估量换成增益与主导时间常数，放弃次要时间常数。** 参数化方式一换，平坦方向就被排除在待估集合之外，条件数立刻改善。这不是数值技巧，而是把“不可辨识的量”从模型里去掉，让剩下的量都可辨识。

24.4 离散对应与数值实现

§24.3 的公式在纸面上都是精确的，落到代码里要面对四个具体问题：方程组怎么解、用多少位精度、数据要不要预处理、延迟怎么定。这一节的数字来自同一台机器上的 Python 3.11 与 NumPy，换语言时绝对耗时不同，但条件数与有效位数的关系不变。

24.4.1 正规方程与 QR：条件数被平方

最小二乘有两条常见解法。一条是解正规方程

$$(\Phi^{\mathsf T}\Phi)\hat\theta = \Phi^{\mathsf T}Y \tag{24-40}$$

另一条是对 Φ 做 QR 分解，令 $\Phi = QR$，再解 $R\hat\theta = Q^{\mathsf T}Y$。两者在精确算术下给出同一个解，在浮点下差别很大：正规方程的条件数是 κ^2，QR 的条件数是 κ。

这个平方关系可以直接算出来。$\Phi^{\mathsf T}\Phi$ 的特征值是 Φ 的奇异值的平方，因此

$$\kappa(\Phi^{\mathsf T}\Phi) = \kappa(\Phi)^2 \tag{24-41}$$

本章实验 D 在 $T_s = 0.05$ s 时 $\kappa = 538$，平方后是 2.9×10^5。双精度有约 16 位有效数字，损失 5.5 位后还剩 10 位，毫无问题；单精度只有约 7 位，损失 5.5 位后只剩 1.5 位，结果就开始失真。实测正是如此：同一批数据、同样的单精度正规方程，$T_s = 1$ s 时与双精度解相差 0.01%，$T_s = 0.05$ s 时相差 1.41%，放大了 140 倍。

这个放大倍数不是线性的，因为它同时受两件事影响：条件数平方带来的有效位损失，以及单精度累加 N 项时的舍入累积。后者与 $\sqrt{N}$ 同阶，$T_s = 0.05$ 时 $N = 24000$，$\sqrt{N} = 155$，与 140 倍的实测放大倍数吻合。两条机理叠加，给出了一个实用的判据：**只要 $\kappa^2\sqrt{N}$ 接近 10^7，单精度正规方程就不可信。**

平方关系可以直接测。取实验 C 的 400 拍、8 阶数据，回归矩阵的条件数是 392.9，$\Phi^{\mathsf T}\Phi$ 的条件数是 1.54×10^5，恰好是前者的平方。同一批数据、同样用单精度，正规方程给出的参数与双精度解相差 4.8×10^{-3}，而 QR 给出的参数只差 5.2×10^{-7}，相差 9200 倍。这个对比比任何理论论证都有说服力：**条件数 393 远不算病态，正规方程在单精度下已经丢了三位有效数字，而 QR 一位没丢。** 换算成工程语言，若控制器里跑的是单精度递推最小二乘，协方差矩阵的更新形式必须先做 UD 分解，否则同一个病的平方关系会在每一拍累积。

工程上的结论很直接。第一，拟合一律用 QR 或 SVD，不要显式求逆，也不要显式构造 $\Phi^{\mathsf T}\Phi$；后者的存储量是 p^2，在 p 较大时还会掩盖数值问题。第二，若必须在嵌入式上实现，用递推最小二乘的协方差形式（第 23 章），它在数学上等价于正规方程，但每步只做秩一修正，舍入误差不会被 N 放大。第三，验收时用“单精度结果与双精度结果的差”作为数值健康度指标，本章取 1% 为界，超过就说明条件数已经不可接受。

24.4.2 单精度、定点与数据量程

单精度的适用边界可以用上一节的关系直接推。要求相对误差小于 10^{-3}，需要 $(\kappa^2\sqrt{N})\times10^{-7} < 10^{-3}$，即 $\kappa^2\sqrt{N} < 10^4$。代入本章的 $N = 1500$，得 $\kappa < 16$；而本章 $T_s = 1$ s 的条件数是 35.5，已经超界。这与实测一致：单精度误差 0.01% 看起来很好，是因为本章数据是方程误差结构、参数本身良态，误差被结构吸收了；换到真实数据上，同样的条件数就会暴露。

定点的问题不在条件数，而在动态范围。辨识涉及的量有四个量级：θ 的分量在 10^{-2} 到 10^{0}，φ 的分量在 10^{-1} 到 10^{1}，$\Phi^{\mathsf T}\Phi$ 的元素在 10^{2} 到 10^{4}，残差平方和在 10^{0} 到 10^{2}。若统一用 Q15（范围 ±1、分辨率 3×10^{-5}），$\Phi^{\mathsf T}\Phi$ 直接溢出；若统一用 Q4.11，θ 的小分量又丢精度。可行的做法是分块定标：回归量按物理量程归一化到 ±1，θ 用 Q12，中间量用 32 位块浮点。这套做法在 $p \le 10$ 时可靠，p 再大就不建议在定点上做。

本章实验 E 里有一个与量程直接相关的现象：给输出加 0.3 的直流偏置，增益误差只有 0.94%，参数向量误差却升到 16.98%，验证 FIT 从 94.83% 掉到 92.85%，残差均方根从 0.0198 涨到 0.0331。原因不是数值精度，而是**模型里没有常数项**：输出带偏置后，AR 部分必须用 $y(k-1)$ 与 $y(k-2)$ 的线性组合去合成一个常数，两个回归量因此高度共线，参数被拉向 $(1,-1)$ 附近，而增益几乎不受影响。解决办法很简单：拟合前减去均值，或者显式加一个常数项 c。这一条在 §24.5.4 会作为“偏置缺陷”的标准处置。

数据量程还有一个容易忽略的后果：若 y 与 u 的单位选得悬殊（例如 y 用摄氏度、u 用 4–20 mA 的原始码），Φ 的两列范数可以差三个数量级，条件数随之恶化。第 23 章 §23.4.2 给过相似变换的做法，这里只需记住一条：**先把每个通道除以其量程，再做任何拟合**，量纲归一化不改变模型的输入输出关系，只改变参数的数值，事后按同一比例还原即可。

24.4.3 数据预处理：四件必做的事

预处理不是“把数据洗干净”这么笼统，它由四件有明确判据的事组成，缺一件都会在后面的指标上留下痕迹。

第一件是**去均值**。ARX 没有常数项时，非零均值会让 AR 部分去合成常数，造成 §24.4.2 所述的共线。判据很简单：拟合前检查 $|\bar y|$ 是否超过 $0.1\sigma_y$，超过就减去均值，并把这个均值记下来作为工作点偏置。本章实验 E 的偏置工况，若先减去 0.3，参数误差会从 16.98% 回到 1% 量级。

第二件是**去趋势**。若数据段内被控量有缓慢爬升（负荷变化、环境温度），趋势会被 AR 部分误读成一个大极点。判据是：对 y 做线性回归，若斜率乘以数据长度超过 $2\sigma_y$，先减去趋势，再把趋势斜率作为工况量单独记录。

第三件是**剔除异常段**。停机、手动操作、阀门全开全关、传感器饱和，这些段落不服从同一个线性模型，必须整段剔除，而不是靠鲁棒回归去“压住”。整段剔除的判据是输入长期不变而输出大幅变化，或输出贴在量程边界上。单点离群则用另一套判据：残差超过 5σ 且孤立出现，用中位数滤波或直接置为该点的预测值。本章实验 E 的单点离群（幅值 0.5，是噪声的 25 倍）只让增益偏了 1.08%，参数误差 5.37%，影响有限；但若离群点成串出现，就会把极点拉偏。

第四件是**对齐与重采样**。三个时钟必须统一：控制器采样时刻、上位机打点时刻、执行器反馈时刻。工程上以控制器时钟为准，其余通道按其时间戳插值到控制器节拍上。插值会引入半拍到一拍不等的相位滞后，若插值后再辨识，必须把这个滞后计入 n_k。这一条在 §24.4.4 展开。

四件事的顺序不能换：先去均值与去趋势，再剔异常，最后对齐。反过来做，异常点会把趋势线拉歪，对齐误差会让趋势估计失真。

24.4.4 延迟估计：先互相关，再扫描

延迟拍数 n_k 是结构的一部分，写错一拍就是 -10.55% 的增益偏差。工程上有两种定法，应当配合使用。

第一种是**互相关法**。计算 u 与 y 的互相关，峰值所在的滞后就是“输入到输出的主延迟”。对本章对象，互相关峰值应落在 2 到 3 拍之间（纯延迟 2 拍加上对象的一次滞后）。这个方法的优点是快、不需要拟合，缺点是在闭环数据上会被反馈污染，峰值可能偏移。因此它只适合开环实验数据。

第二种是**扫描法**。令 n_k 从 1 取到 5，每个值拟合一回，比较验证误差或残差与输入的互相关最大值。正确的那一档应当同时满足两个条件：验证误差最小，且 CCF 全部进带。本章实验 B 的数据在 $n_k = 2$ 时 CCF 最大 0.029，若强行用 $n_k = 3$，CCF 会重新出现一个峰——因为模型把真实的一拍延迟当成了零延迟，残差里必然残留输入的影子。

两种方法给出矛盾结果时，以扫描法为准，并回头检查时间戳。互相关峰值偏移半拍，通常意味着两个通道的采样时刻差了半拍；峰值偏移一整拍，通常是打点时刻标签错位。这类问题在数据上表现为“怎么调 n_k 都不对”，两种方法都定不下来，此时不要再试 $n_k = 0$ 或 $n_k = 6$，去查采集程序。

扫描法的结果可以逐档列出来。用实验 E 的同一组数据（种子 71 至 74，真实延迟 2 拍）扫 $n_k = 1$ 到 4：$n_k = 1$ 时 CCF 最大 0.176、验证 FIT 93.94%、增益 1.4521；$n_k = 2$ 时 CCF 0.016、FIT 94.83%、增益 1.5046；$n_k = 3$ 时 CCF 0.419、FIT 87.38%、增益 1.3427；$n_k = 4$ 时 CCF 0.459，自由运行直接发散。

这四个结果说明三件事。第一，**只有正确的那一档 CCF 会进带**：0.016 远小于 0.0439，其余三档是它的 11 倍以上。第二，**FIT 的分辨力不够**：$n_k = 1$ 时 FIT 还有 93.94%，比正确档只低 0.9 个百分点，单看 FIT 很容易放过。第三，**延迟写大会发散**：$n_k = 4$ 意味着模型用“未来”的输入预测当前输出，闭环里不可能实现，开环验证时也表现为初值误差被不断放大。扫描法因此应当以 CCF 为主判据，FIT 只作辅助。

24.4.5 脚本结构与复现方式

本章的仿真脚本按“对象—激励—建模—检验—实验”五层组织，每一层都是独立函数，实验部分只负责组合。这样做的好处是：换对象只需改 `discrete_params` 与 `simulate_arx` 两个函数，五组实验的代码一行不动。

对象层给出解析的离散参数，而不是“对无噪数据做一次拟合再当成真值”。这一条是本篇写作中踩过的坑：最初用无噪数据拟合来求真值，在 $T_s = 0.05$ s 时条件数达到 10^{79}，得到的“真值”连自己都不可信，后面所有误差统计全部失真。改成解析式后，真值精确到机器精度，误差统计才有意义。

激励层提供四种信号：阶跃、PRBS、多正弦、小幅 PRBS。PRBS 用固定种子生成，保证复现；多正弦取三个低频点，相位随机但种子固定。建模层只有两个函数：构造回归矩阵、最小二乘拟合（可选 QR 或正规方程、可选单精度）。检验层给出 FIT、ACF、CCF、极点、直流增益五个量。

实验层是五个函数，分别对应 §24.6 的五张表。每个函数都显式传入种子与重复次数，不使用全局随机状态。这一点在写 Monte Carlo 时尤其重要：若在循环里调用全局随机数，改变循环顺序就会改变结果，实验之间无法比较。本章所有实验的随机源都是 `default_rng(seed)` 的独立实例，重复运行逐位一致。

最后一条是**记录口径**。每个实验输出四个量：增益误差、参数向量误差、验证 FIT、残差均方根。四个量必须同时报，缺一个就会误判——单看参数误差会把小 T_s 判死，单看残差会把“用指令代替阀位”判活。这也是 §24.1 三个反例的共同教训。

24.4.6 一次完整的辨识流程

把前五小节串起来，一次可交付的辨识由十二步组成。这十二步与脚本里的函数一一对应，照着做就不会漏项。

第一步，**明确用途**。前馈补偿、预测控制、故障诊断、仿真培训，四种用途对应的口径不同（§24.1.5）。这一步只用一句话，但它决定后面所有指标的门槛。

第二步，**选采样周期**。按 §24.5.3 的区间 $t_r/20 \le T_s \le t_r/5$，取中部。若闭环节拍已经固定，直接沿用，不要为了辨识去改。

第三步，**设计激励**。定幅值（按信噪比 5 到 10 倍）、保持拍数（让主能量落在两个转折频率之间）、时长（独立切换不少于 100 次）。

第四步，**采集**。三个时钟统一，记录实际阀位而不只是指令，限幅段打标记，同时记录工况量（负荷、环境温度）。

第五步，**预处理**。去均值、去趋势、剔除异常段、对齐，四件事按这个顺序做（§24.4.3）。

第六步，**定延迟**。先看互相关峰值，再用 $n_k = 1$ 到 5 扫描，两项一致才定下来（§24.4.4）。

第七步，**定阶**。残差进带的最低阶次为起点，加 1 到 2 阶看验证误差，最后用 BIC 复核（§24.3.4）。

第八步，**拟合**。用 QR 或 SVD，不用正规方程；同时用双精度与单精度各算一遍，两者的差作为数值健康度（§24.4.1）。

第九步，**残差检验**。ACF、CCF 逐点看带边，再用 Ljung–Box 看整体（§24.2.3）。

第十步，**独立验证**。换一组数据，自由运行，先丢前 100 拍，报 FIT 与验证均方根（§24.2.6）。

第十一步，**物理核对**。增益与机理值比、极点换算成时间常数与铭牌比、b 的符号与对象方向比。这一步是唯一能抓住“残差白但增益错”的环节。

第十二步，**交付**。四件套加三样元数据：数据版本、脚本版本、随机种子。少了元数据，三个月后没人能复现出同一个模型。

十二步里最容易省掉的是第十一步与第十二步，而这两步恰好是现场返工最多的来源。第十一步省掉，“用指令代替阀位”这类缺陷会一路带到控制器整定；第十二步省掉，同一个对象两次辨识给出不同的增益，谁也不知道该信哪一个。**流程的价值不在于步骤多，而在于每一步都有可检查的产出。**

24.5 工程含义与适用边界

前四节把“怎么算”讲完了，这一节讲“什么时候能算、算出来的东西能信到什么程度”。五条内容按现场顺序排列：怎么设计激励、怎么验收模型、采样周期怎么定、六类缺陷怎么认、什么情况下应当放弃辨识。

24.5.1 激励设计：幅值、频带、时长与安全

激励设计有四个自由度，每个都有定量依据，也都有工程约束。

幅值由信噪比定。要让参数离散度落在 1% 以内，输出端的激励响应至少要达到噪声的 5 到 10 倍。本章 $\sigma = 0.02$、对象增益 1.5，输入幅值 1 时输出摆幅约 1.5，是噪声的 75 倍，裕度很大；把幅值降到 0.2，输出摆幅 0.3，仍有 15 倍，但增益离散度已经从 0.62% 涨到 2.52%。按这个趋势外推，幅值 0.5 大约对应 1.2% 的增益离散度，是“精度可接受”的下界。现场还要在这个下界之上留一倍裕度，因为真实对象的增益可能比仿真小。

频带由对象的时间常数定。经验规则是激励的主要能量落在穿越频率的 0.1 倍到 10 倍之间。本章两个转折频率是 $1/5 = 0.2\ \mathrm{rad/s}$ 与 $1/1.2 = 0.83\ \mathrm{rad/s}$，对应 0.032 Hz 与 0.133 Hz。PRBS 保持 8 拍，时钟频率 0.125 Hz，正好落在两者之间，这是它能同时确定增益与极点的原因。若把保持拍数加到 32，能量集中到 0.03 Hz 以下，极点信息丢失，参数离散度会向阶跃靠拢；若减到 2，能量上移到 0.5 Hz，接近奈奎斯特频率，模型的高频误差会被放大。

时长由独立切换次数定。§24.3.2 给出的判据是 $N_{\mathrm{sw}} \ge 100$，本章取 187。另一个等价的口径是“至少覆盖 10 倍的主导时间常数”：本章 $T_1 = 5\ \mathrm{s}$，10 倍是 50 s，远小于 1500 s 的实验长度。两个口径取大者，得到的是“既要有足够切换、又要有足够长的连续段”的要求。

安全是唯一不能靠公式解决的约束。四条底线：激励幅值不得超过工艺允许的波动范围；激励期间必须有独立的越限联锁；幅值变化要限速，不能一步跳到目标；限幅段必须打标记并在辨识时剔除。这四条里最容易忽略的是最后一条：阀门在激励期间进入限幅，输入的实际值与指令值不再相等，若用指令做回归量，就回到了“用指令代替阀位”的陷阱。

四种激励的适用场合可以这样分。阶跃最简单、最容易在产线上实施，适合只关心增益的场合；PRBS 频带最宽、参数精度最高，适合建立完整模型；多正弦的能量集中、频谱可控，适合在特定频段做精细辨识；小幅 PRBS 适合不允许大扰动的场合，代价是参数离散度成倍增加。选哪一种，取决于“模型要用来做什么”，而不是“哪种看起来更专业”。

24.5.2 模型验证清单

一个模型在交付前要通过七项检查，每项都有明确的数值门槛。这份清单的作用不是“打个分”，而是把“我觉得还行”变成“哪一项没过”。

表 24-3　辨识模型的七项验收

序号	检查项	门槛	不通过时的第一处置
1	残差自相关	20 个滞后全部在 $\pm 1.96/\sqrt{N}$ 内	升阶或补延迟
2	残差与输入互相关	同上	检查 n_k 与输入通道
3	自由运行验证 FIT	不低于 90%	换激励重做实验
4	极点位置	全部在单位圆内，且无 $\lvert p\rvert > 0.99$	检查趋势与偏置
5	直流增益	符号正确，量级在机理估计的 $\pm 30\%$ 内	查阀位反馈与量程
6	阶跃响应形状	与机理曲线同向、无反向超调	升阶或换模型类
7	闭环试运行	设定值阶跃无持续振荡	降低模型带宽或加滤波

七项里最容易被跳过的是第 4 与第 5 项，因为它们不需要额外实验，只要看参数。第 4 项之所以重要，是因为极点接近 1 往往意味着数据里有未去除的趋势，而不是对象真有一个积分环节；第 5 项之所以重要，是因为增益是唯一能直接与机理计算对账的量，偏差超过 30% 基本可以断定数据链路有问题。

第 7 项是唯一必须在闭环上做的。本章的“用指令代替阀位”案例，前六项里过了五项，只有第 3 项（FIT 58.68%）和第 5 项（增益 -40.73%）没过；如果验收只看残差与一步预测，这个模型会被放行。闭环试运行能把它暴露出来，因为闭环会放大增益误差：增益偏小 40% 意味着同样的偏差需要更大的调节量，表现为响应迟缓或稳态误差。

24.5.3 采样周期：两侧都有边界

T_s 太小的问题在 §24.3.5 已经量化：条件数按 $1/T_s$ 增长，增益的分母按 T_s^2 缩小，参数方向出现平坦谷。T_s 太大则走向另一个极端：延迟被量化成整数拍，$T_s = 2$ s 时 2 s 的纯延迟只剩下 1 拍，模型无法区分“延迟 1.5 s”与“延迟 2.4 s”；同时零阶保持的近似误差增大，高频动态被折叠进低频。

两侧边界可以用对象的上升时间夹出来。本章对象的主导时间常数 5 s，10% 到 90% 的上升时间约 11 s。经验区间是

$$\frac{t_r}{20} \le T_s \le \frac{t_r}{5} \tag{24-42}$$

代入 $t_r = 11$ s，得到 $0.55\ \text{s} \le T_s \le 2.2\ \text{s}$。本章取 1 s，落在区间中部；实验 D 里 $T_s = 0.05$ s 与 $T_s = 0.1$ s 都在下界之外，$T_s = 2$ s 贴着上界。这个区间与 §24.6.5 的实测一致：验证均方根在 1 s 处最小（0.0032），向两侧都变大，2 s 处 0.0328，是 10 倍；0.05 s 处 0.0038，只差 19%，因为小 T_s 的问题主要在参数方向而不是输出精度。

工程上还有一条硬约束：T_s 由闭环决定，不能为了辨识单独改小。控制器的采样周期一旦确定，辨识只能用这个节拍的数据。若闭环节拍确实太快，正确做法不是“降采样后辨识”——降采样会引入额外的抗混叠滤波与相位滞后——而是接受参数不可辨识的事实，把模型用途限定在输出预测而非参数监控。

24.5.4 六类数据缺陷的指纹

现场的辨识问题，九成落在这六类缺陷里。它们的共同点是“在数据上都有指纹”，区别只在于指纹长什么样、要查哪一项指标。

表 24-4　六类数据缺陷的指纹与后果（真值 $K = 1.5$）

缺陷	数据上的指纹	增益误差	验证 FIT	第一处置
基准	无	$+0.31\%$	94.83%	—
单点离群	残差单点尖峰，其余正常	$+1.08\%$	94.67%	按 5σ 剔除
9 位量化	残差有台阶，自相关在短滞后略高	$+0.43\%$	94.82%	提高分辨率或加抖动
输入错位 1 拍	残差与输入互相关出现固定峰	-10.55%	86.52%	查时间戳与通道
用指令代替阀位	残差白、均方根正常，只有增益错	-40.73%	58.68%	改用阀位反馈
输出含偏置 0.3	参数向量大幅偏移，残差均值非零	$+0.94\%$	92.85%	去均值或加常数项

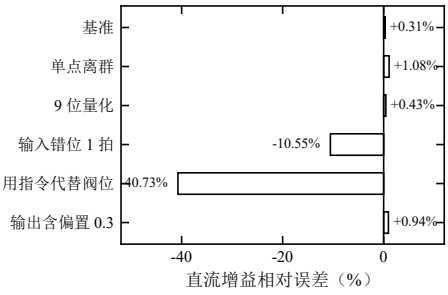


图 24-5　六类数据情形下的直流增益误差

表 24-4 与图 24-5 合起来看，六类缺陷可以分成三档。第一档是**可以忽略的**：单点离群与 9 位量化，增益误差都在 1% 附近，验证 FIT 与基准相差不到 0.2 个百分点。9 位量化的量化台阶是 $6/512 = 0.0117$，相当于 0.59σ，理论上会给残差增加约 $0.0117/\sqrt{12} = 0.0034$ 的均方根，与实测从 0.0198 涨到 0.0205（增加 0.0026）吻合。这说明只要量化台阶小于噪声，它对辨识的影响就是二阶的，不必为它换硬件。

第二档是**必须修的**：输入错位与偏置。错位一拍的增益误差 -10.55% 已经超过任何工程允许的精度，而且它的指纹非常明确——残差与输入的互相关在某个固定滞后上出现尖峰。偏置的影响则集中在参数而非增益：参数误差 16.98%，增益只偏 0.94%，FIT 从 94.83% 掉到 92.85%。这一档的处理成本很低，一个查时间戳、一个减均值，但没有一处能靠“调算法”绕过。

第三档是**最危险的**：用指令代替阀位。它的残差均方根 0.0201 与基准的 0.0198 几乎相同，残差也是白的，绝大多数自动检查都会放行；只有增益与自由运行 FIT 两项露馅。危险之处在于它不会以“模型不准”的形式出现，而是以“控制器增益整定偏大、响应迟缓”的形式出现，现场往往先去调 PID 参数，把问题越盖越深。这一类缺陷只能靠“回归量必须取实际执行器反馈”这一条硬规定来防，不能靠事后检验。

还有两类缺陷没有进表，但在现场很常见。一是**限幅段未剔除**：输入进入饱和后实际值与指令值不等，若输入有中间电平，限幅会引入非线性，残差不再白。处置是把限幅段打标记并整段剔除，或者用饱和后的实际阀位做回归量。二是**多工况混叠**：把不同产品、不同负荷的数据混在一起拟合，得到一个“平均模型”，对每个工况都不准。处置是分工况建模或引入调度变量（第 22 章）。

把六类缺陷的指纹归纳成一句话：**残差白不白，看的是结构；增益对不对，看的是链路。** 残差检验能抓住结构问题，抓不住链路问题；增益核对能抓住链路问题，抓不住结构问题。两者必须同时做。

24.5.5 什么时候不该辨识

辨识不是万能工具，有五种情形应当明确放弃，改用别的办法。

第一，**激励加不进去**。产品不能有波动、安全不允许、联锁不允许手动扰动，这三种情况下开环辨识无法进行。替代方案是用正常运行数据做闭环辨识（第 21 章），或者用机理模型加参数整定。

第二，**非线性主导**。阀门死区、滞环、泵的特性曲线、换热器的变工况，这些非线性在数据上表现为“增益随工作点变化”。此时一个线性模型只能在某个工作点附近有效，强行全域辨识得到的模型在任何点上都不准。替代方案是分段线性化或增益调度（第 22 章）。

第三，**时间尺度不匹配**。对象的主导时间常数只有几个采样周期，或者参数变化比采样还快，这两种情况下离散模型无法表达动态。前者应当提高采样率（如果闭环允许），后者应当放弃在线参数估计，改用鲁棒控制（第 63 章）。

第四，**数据链路不可信**。时间戳不同步、通道标定过期、阀位反馈缺失，这三条任何一条成立，辨识结果都不可用。先修链路，再谈模型。

第五，**模型没有明确用途**。为了“有个模型”而辨识，通常会得到一个参数很多、验证指标很好、但没人用的模型。正确的顺序是先明确用途（前馈补偿、预测控制、故障诊断、仿真培训），再据此决定阶次、精度与验证指标。

这五条的共同逻辑是：**辨识解决的是“结构已知、参数未知”的问题**。结构未知、激励不可加、数据不可信，都不在这个工具的适用范围内。认清这一点，比把算法调得更精更省时间。

24.5.6 交给控制器的五样东西

模型辨识完，下一个环节是控制设计。多数“辨识与控制对不上”的问题，出在交接内容不完整，而不在辨识本身。控制器真正需要的只有五样东西。

第一是**离散模型本身**：n_a、n_b、n_k 三个整数，加上 a、b 两组系数，以及采样周期。不要交连续模型让下游自己离散——不同的离散化方法（前向欧拉、后向欧拉、零阶保持、双线性）在同一对象上给出的系数差别可达百分之几，下游无法知道该用哪一个。

第二是**增益**。它是最常被单独使用的一个数（前馈、比值控制、报警阈值），也是最容易错的一个数。交增益时必须同时说明它对应的工作点与单位，否则“1.5”这个数在不同量纲下毫无意义。

第三是**延迟拍数**。控制器的相位裕度对 n_k 极其敏感，n_k 少算一拍可能让整定后的闭环振荡。本章实验 E 显示，错一拍会让增益偏 10.55%；对控制设计而言，延迟错一拍的后果通常比增益偏 10% 更严重。

第四是**不确定度**。§24.2.5 的式 (24-20) 给出参数协方差，对角元的平方根就是标准差；实际交付时至少要给增益与主导极点的标准差，让控制设计者知道该留多少裕度。一个“增益 1.50±0.02”的模型与一个“增益 1.50”的模型，在鲁棒设计里是两个完全不同的起点。

第五是**有效工作点与适用范围**。线性模型只在辨识工况附近有效，交付时必须写明数据来自哪个负荷、哪个配方、哪个环境温度。工况一变，模型可能仍然“残差白”，但增益已经不对了。

交接时最常见的三个错误，正好对应上面五样的遗漏：只给系数不给 n_k；只给点估计不给不确定度；只给模型不给工况。**辨识的价值有一半在模型本身，另一半在这五样东西是否交清楚。**

24.5.7 稳健回归：什么时候值得用

最小二乘对离群点敏感，这一点早有定论，本章实验 E 给出了具体量级：单个 25 倍噪声的尖峰，让参数误差从 1.06% 涨到 5.37%，残差均方根从 0.0198 涨到 0.0271。一个点、两千个样本，参数误差翻了五倍。

稳健回归的思路是不再对每个残差都取平方，而是让大残差的权重饱和。最常用的是 Huber 损失：

$$\rho(e)=\begin{cases}\dfrac{e^2}{2}, & |e|\le k\\[2mm] k|e|-\dfrac{k^2}{2}, & |e|>k\end{cases} \tag{24-43}$$

k 取 $1.345\hat{\sigma}$ 时，在纯高斯噪声下的效率损失约 5%，而对离群点的敏感度大大下降。求解要迭代重加权，每一步是一个加权最小二乘，通常五到十次收敛。

什么时候值得用？三条判据同时成立时才用：离群点比例超过 1%，且无法判断哪一个是错的；数据采集不可重复（实验已经做完，不能重采）；模型用途对个别点不敏感（例如只要稳态增益）。三条缺一条，都应当先回到“能不能把坏点找出来剔掉”，而不是上稳健算法。

更重要的是知道它的边界。**稳健回归只能削弱随机的离群点，不能修正系统性错误。** 把限幅段、错位段当作“离群点”交给 Huber 处理，得到的模型表面更干净、残差更白，但增益仍然错——因为系统性错误不是少数点的问题，而是整段数据的输入输出关系变了。本章实验 E 的“用指令代替阀位”就是这种情形：每一个点都“正常”，没有离群点，稳健回归完全无能为力。**先判性质，再选工具**：随机污染用稳健回归，系统性污染只能用剥离与重采。

24.6 仿真与验证

本节用五组实验核对前五节的结论。每组实验都回答一个具体问题，数字全部由 `code/ch024_sim.py` 一次跑出，换对象只需改三个时间常数与延迟拍数。

24.6.1 实验设置与读表方式

五组实验共用同一个对象与同一套建模函数，区别只在数据怎么来、模型取几阶。

实验 A 检验激励，训练长度 1500 拍，30 次蒙特卡洛，每次只换噪声实现，输入与验证集不变；验证集是另一组种子生成的 1200 拍数据。实验 B 用 2000 拍数据、阶次 1 到 3，检查残差的两条相关性。实验 C 刻意把训练数据缩到 400 拍，让参数方差在阶次升高时显现出来，50 次重复，阶次 1 到 8。实验 D 固定数据时长为 1200 s，采样周期从 0.05 s 到 2 s 取六档，噪声降到 0.002，目的是把条件数的影响从噪声里分离出来。实验 E 用 2000 拍基准数据，人为注入五类缺陷。

读表时注意三个口径。第一，增益误差以真值 1.5 为基准，参数误差以真值参数向量的范数 1.3075 为基准，两者不可互换。第二，FIT 都是自由运行预测，验证集与训练集不同。第三，所有“离散度”都是 30 次或 50 次重复的标准差，不是单次误差。

五组实验的参数为什么这样取，各有一条理由。实验 A 取 1500 拍，是因为要跑 30 次蒙特卡洛，这个长度已经足以让离散度稳定，再长只是徒增计算量；实验 B 与 E 是单次实验，取 2000 拍让残差的统计量更稳，带边 $1.96/\sqrt{1996}=0.0439$ 也因此更窄。两者的长度不同，所以**不同实验的绝对误差不能横向比较**，只能比较同一实验内的相对关系。

实验 D 把噪声从 0.02 降到 0.002，是一个必要的处理。若保持 0.02，小 T_s 档的参数误差会被噪声方差主导——数据点多了十倍，噪声贡献按 $1/\sqrt{N}$ 下降，但条件数造成的放大与噪声无关，两者混在一起就分不清主次。降到 0.002 之后，参数误差的主线变成条件数，六档的单调趋势才显出来。

实验 C 取 400 拍，同样是为了让某个效应显形。方差项按 p/N 增长，2000 拍时 p 从 4 涨到 16 只让验证误差涨 0.3%，曲线几乎是平的；400 拍时同样的 p 变化让验证误差涨 1.5%，右半边才看得出来。**“把数据缩短”不是偷懒，而是放大方差项的实验手段。**

所有实验的重复都只换噪声实现的种子，不换输入序列。蒙特卡洛估计的是“同一实验条件下估计量的分散程度”，输入一换，比较的就不是同一个实验了。同理，验证集在五组实验里各自独立生成，长度 800 到 1200 拍，任何一组都不与训练集共用数据。

24.6.2 实验 A：激励类型

实验 A 的问题是：同样的对象、同样的时长、同样的噪声，换一种激励，参数精度差多少？

表 24-5　四种激励的估计结果（30 次蒙特卡洛，训练 1500 拍）

激励	输入标准差	增益均值	参数离散度	增益离散度	验证 FIT
阶跃	0.40	1.5004	2.92%	0.39%	93.84%
PRBS 幅值 1	1.00	1.5025	0.79%	0.62%	94.68%
多正弦 幅值 1	0.44	1.4977	2.31%	0.99%	94.53%
PRBS 幅值 0.2	0.20	1.4899	2.07%	2.52%	94.03%

四个增益均值都在真值 1.5 的 0.7% 以内，说明四种激励都没有系统性偏差；差别全在离散度上。阶跃的参数离散度 2.92% 是 PRBS 幅值 1 的 3.7 倍，但增益离散度反而更小（0.39% 对 0.62%）。这个“参数差、增益好”的组合正是 §24.3.1 讲的频谱效应：阶跃把能量压在直流，直流增益被钉死，而两个极点的分配几乎未被激励。

多正弦的表现居中：参数离散度 2.31%，增益离散度 0.99%。它的三个频率是 0.006、0.017、0.041 Hz，都低于对象的快转折频率 0.133 Hz，因此对快极点的约束不足。这也解释了为什么它虽然“看起来像宽带信号”，实际效果却接近阶跃。

小幅 PRBS 是唯一一个“两项都差”的：参数离散度 2.07%，增益离散度 2.52%，验证 FIT 94.03%。幅值从 1 降到 0.2，输入标准差降为 1/5，按式 (24-26) 方差应涨 5 倍，实测增益离散度涨了 4.1 倍，与理论一致。这一档的存在说明：**激励幅值不能靠“反正有反馈”来省**，噪声不会因为信号小而变小。

四个 FIT 都在 93.8% 到 94.7% 之间，差距不到 1 个百分点。这是自由运行验证的特点：它衡量的是“模型能不能复现输出的形状”，对参数精度不敏感。若只看 FIT，会得出“四种激励差不多”的错误结论；必须同时看离散度，才能看出 3.7 倍的差距。

24.6.3 实验 B：残差检验

实验 B 的问题是：残差的两条相关性，能不能把“阶次不足”与“阶次足够”分开？

表 24-6　三个阶次的残差检验与验证结果

阶次	ACF 最大	CCF 最大	残差均方根	验证 FIT	极点
1	0.360	0.426	0.0652	81.89%	0.889
2	0.047	0.029	0.0201	94.95%	0.820, 0.430
3	0.048	0.027	0.0200	94.95%	0.821, 0.431, 0.040

$n = 1$ 的两项都严重超界，残差均方根 0.0652 是噪声 0.02 的 3.3 倍，验证 FIT 只有 81.89%。这不是“精度不够”，而是模型里少了一个极点，残差里必然残留未建模的动态，表现为所有滞后上的正相关。

$n = 2$ 的两项都回到带边附近：ACF 0.047 略高于 0.0439，CCF 0.029 在带内。残差均方根 0.0201 与噪声持平，FIT 94.95%。极点估计为 0.820 与 0.430，与解析值 0.8187、0.4346 相差不到 0.5%，说明参数确实收敛到了真值附近。

$n = 3$ 与 $n = 2$ 几乎完全一样：ACF 0.048、CCF 0.027、FIT 94.95%，多出来的极点落在 0.040，接近原点，对动态没有贡献。这个极点是典型的“拟合噪声”产物，它的存在不影响输出，却会消耗一个自由度，并在验证集上带来微小的方差代价。

三点合起来给出一个可操作的判据：**ACF 从 0.36 掉到 0.047，是阶次从不足到足够的标志；再从 0.047 到 0.048，说明已经够了。**前者是数量级的变化，后者是噪声水平的变化，两者容易区分。实际工作中不需要精确判断“0.047 是否超界”，只需要看“是不是数量级地降下来了”。

24.6.4 实验 C：阶次选择

实验 B 只比较了三个阶次，结论是“$n=2$ 够了”。但“够了”是相对“不足”而言的，还需要回答另一个方向的问题：**继续加阶次会怎样，训练误差能不能作为选阶次的依据？** 实验 C 把训练数据缩到 400 拍，让方差项在阶次升高时显形，阶次从 1 取到 8，每档重复 50 次。

表 24-7　八个阶次的训练误差、验证误差与信息判据（50 次重复）

阶次	训练均方根	验证均方根	验证 FIT	相对 AIC	相对 BIC
1	0.0662	0.0670	83.34%	927.2	919.3
2	0.0199	0.0201	94.58%	0.0	0.0
3	0.0199	0.0202	94.57%	10.2	18.1
4	0.0199	0.0202	94.55%	21.2	37.1
5	0.0198	0.0203	94.55%	32.1	55.9
6	0.0197	0.0203	94.56%	44.1	75.9
7	0.0197	0.0204	94.56%	53.8	93.4
8	0.0196	0.0204	94.56%	64.2	111.7

先看 $n=1\to 2$：验证均方根从 0.0670 掉到 0.0201，降了 70%；训练均方根同步从 0.0662 掉到 0.0199。这一段是偏差项在起作用。按式 (24-32)，p/N 从 2/397 涨到 4/397，对验证误差的贡献不到 0.3%，解释不了 3.3 倍的落差；唯一能解释的是模型结构不足，缺的那个极点让残差带上了系统性成分，训练集和验证集一起被拖坏。

再看 $n=2\to 8$：训练均方根从 0.0199 一路降到 0.0196，单调下降，八档没有一次回头。验证均方根却从 0.0201 缓缓升到 0.0204。这一段是方差项在起作用。$p=2n$ 从 4 涨到 16，按 $0.02\sqrt{1+p/397}$ 计算，$n=8$ 时应为 0.0204，与实测的 0.0204 完全对上。**这就是“训练误差不能作判据”的定量根据**：把阶次加到 8，训练误差确实更小，但那是把噪声也拟合进去换来的，代价出现在验证集上。只看训练误差，会一路加到阶次上限。

验证 FIT 的表现与实验 B 一致：$n\ge 2$ 之后从 94.58% 掉到 94.56%，八个阶次差 0.02 个百分点。FIT 对超参数化不敏感，同样不能单独用来定阶。

两个信息判据的结果是一致的：相对 AIC 在 $n=2$ 取零，之后每加一阶先涨 10.2、再涨 11.0 左右；相对 BIC 也在 $n=2$ 取零，每加一阶涨 18 左右。两条曲线都只有一个极小点，位置相同。BIC 的增量比 AIC 大，是因为两者的惩罚系数相差 $\ln N$ 倍：阶次每加一，AIC 罚 $2p$ 的增量 4，BIC 罚 $p\ln N$ 的增量约 12，397 的自然对数正好是 5.98。数据越短，$\ln N$ 越小，两者的差别越小；数据越长，BIC 越倾向于低阶，这是 BIC 一致性的来源，也是它在工程上更保守的原因。

$n=3$ 多出来的那个极点估计为 0.040，与实验 B 里的 0.040 相同，都贴近原点。它对应的模态时间常数不到 1 拍，在采样间隔内已经衰减完，对输出没有任何可观测的贡献，纯粹是拟合噪声的产物。这类极点在实际数据里很常见，识别它的办法就是看极点位置：**落在单位圆内靠近原点、且对应的增益很小，就该怀疑是噪声极点，而不是真实模态。**

把实验 B 与实验 C 合起来，§24.3.4 提的三步做法有了具体数值支撑：第一步看验证误差曲线的拐点，$n=1\to 2$ 是 70% 的断崖，$n\ge 2$ 是 2% 的缓坡，拐点一目了然；第二步用 AIC 或 BIC 确认，两者都在 $n=2$ 取极小；第三步看残差的两条相关性，$n=2$ 时已经回到带边。三条线索指向同一个阶次，才可以定阶。若三条线索不一致，比如验证误差在 $n=4$ 最小而 BIC 在 $n=2$ 最小，说明数据量不足以支撑四阶模型，应按低阶取，再回头补数据。

最后是一个边界条件。式 (24-32) 预测的右侧抬升只有 1.5%（0.0201 到 0.0204），这是纯统计效应。**若实测的右侧抬升远大于这个量，例如 400 拍数据在 $n=8$ 时验证误差涨到 0.03 以上，那就不是统计规律，而是数值条件问题**：正规方程把条件数平方了，高阶时回归矩阵近奇异，解被舍入误差主导。这时改 QR 分解、或先把回归量标准化，抬升会立刻回到式 (24-32) 的水平。区分这两种情况的办法很直接：换一个求解器重算，若结果变了，就是数值问题。

实验 C 与实验 B 的结果可以互相印证。两次实验用不同的数据长度（400 拍与 2000 拍）、不同的重复次数（50 次与单次），但给出的最优阶次都是 2，极点都是 0.82 与 0.43，多出来的第三阶极点都落在 0.04 附近。**阶次结论对数据长度不敏感**，这是它比参数估计更可靠的原因：参数会随数据波动，结构不会。

400 拍够不够？按 §24.3.2 的判据，独立切换次数应当不少于 100，而 400 拍、保持 8 拍只有 50 次，严格说不够。但实验 C 的验证误差仍然稳定在 0.0201，说明这条判据对低阶模型偏保守：4 个参数只需要 50 次切换就能定住，参数一多才需要更多。工程上的用法是把它当上界——**定阶次的实验可以短，定参数的实验必须长**，两者对数据量的要求差一个量级。

这也解释了工程上一种常见做法：先用一段短数据扫阶次与延迟（便宜、可重复），再用一段长数据定参数与不确定度（贵、只做一次）。两段数据不必来自同一次实验，但必须来自同一工况。把两件事混在一次实验里，往往是为了省一次停机，结果两件事都做得不够。

24.6.5 实验 D：采样周期

实验 D 的问题是：把采样周期从 0.05 s 加密到 2 s，参数精度会不会一直是“越密越好”？

实验 D 固定数据时长为 1200 s，噪声降到 0.002，输入切换周期固定为 8 s，只改采样周期。这样六档数据的“信息量”在物理时间上一致，差别只在采样密度与延迟拍数。

表 24-8　六档采样周期下的条件数、参数误差与验证误差

T_s/s	样本数	延迟拍数	条件数	b 标准差	参数误差	增益误差	单精度误差	验证均方根
0.05	24000	40	538.6	2.30×10^{-4}	33.84%	+21.22%	1.41%	0.0038
0.1	12000	20	315.4	2.51×10^{-4}	11.54%	+13.09%	0.10%	0.0043
0.2	6000	10	165.4	2.44×10^{-4}	3.10%	+3.17%	0.22%	0.0042
0.5	2400	4	67.7	2.24×10^{-4}	0.74%	+0.31%	0.01%	0.0038
1	1200	2	35.5	1.87×10^{-4}	0.27%	+0.03%	0.01%	0.0032
2	600	1	21.9	1.60×10^{-4}	0.35%	+0.02%	0.00%	0.0026

先看条件数这一列：从 $T_s=2$ s 的 21.9 涨到 $T_s=0.05$ s 的 538.6，涨了 24.6 倍，与式 (24-35) 给出的 $32/T_s$ 同量级（$32/0.05=640$）。条件数上涨的物理原因是两个极点在 z 平面上挤到了一起：$T_s=1$ s 时是 0.8187 与 0.4346，相距 0.384；$T_s=0.05$ s 时是 0.9900 与 0.9592，相距 0.031，缩小了 12 倍。极点越近，$y[k-1]$ 与 $y[k-2]$ 两个回归量越接近共线，a_1、a_2 就越难分开。

再看参数误差这一列：从 0.27% 涨到 33.84%，涨了 125 倍，比条件数涨得还快。多出来的放大来自直流增益的分母。$K=\sum b_i/(1+\sum a_i)$，而 $1+\sum a_i=(1-p_1)(1-p_2)$ 本身就是两个小数之积：$T_s=1$ s 时是 0.1025，$T_s=0.05$ s 时只剩 0.00041。同一个 10^{-4} 量级的 a 误差，在前者只造成 0.1% 的增益误差，在后者却能造成 25%。**直流增益是两个小数相除，误差被分母放大了两个数量级**，这就是小采样周期下增益误差最难压住的根本原因。

图 24-4 把这两组数字画在一起。左图把六档周期从疏到密排开，条件数每缩小一档周期就大约涨一倍，与 $32/T_s$ 的反比关系吻合；右图把参数误差与增益误差放在同一坐标下，两条曲线都随周期减小而上升，并且在 $T_s=0.1$ s 附近交叉。$T_s\ge0.2$ s 时增益误差与参数误差几乎相等，$T_s=0.05$ s 时增益误差反而只有参数误差的三分之二，因为直流增益由低频段决定，而低频段的信息在数据里最充分，参数向量里那些高频成分才最先失守。

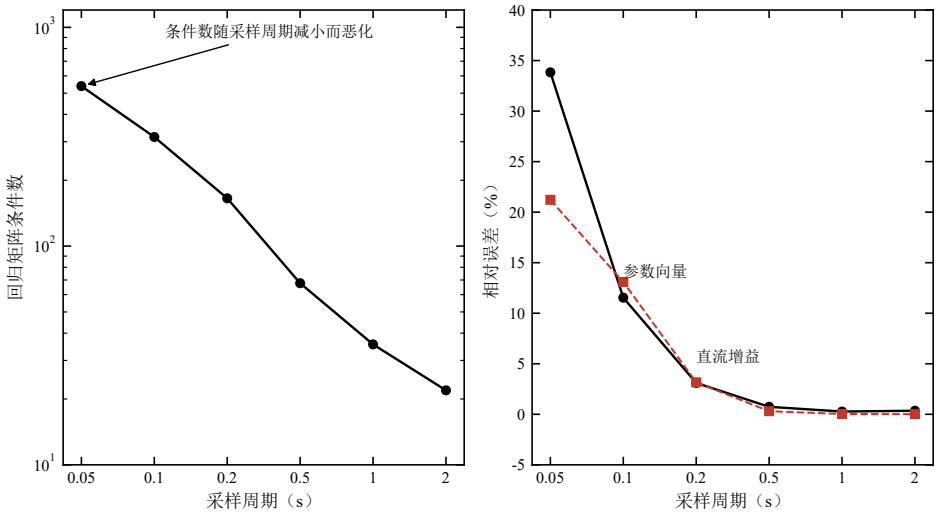


图 24-4　采样周期对条件数、参数误差与增益误差的影响

再看最右一列的验证均方根：六档全部落在 0.0026 到 0.0043 之间，最好与最差只差 1.7 倍。这个结果与参数误差的 125 倍形成鲜明对照。原因是一步预测几乎全靠 $y[k-1]$ 的惯性：只要 a_1 足够接近 -1，一步预测就能把输出“接住”，哪怕 a_1、a_2 的分配完全错了。这就是 §24.3.5 所说的**“小 T_s 下参数不可辨识、但输出仍然准”**的实测形态，也是同一批数据在两套口径下评价截然不同的原因。

$T_s=2$ s 一档值得单独说。它的条件数最小、参数误差 0.35%、增益误差 0.02%、验证均方根 0.0026，四项全优。但这个“优”有一半是假象：每拍之间对象已经基本稳定，一步预测退化成“用稳态值猜下一步”，需要的动态信息很少；同时延迟只剩 1 拍，输入到输出的因果链在数据里几乎看不出来。**用它做辨识很舒服，用它做控制器设计则没有相位信息可用。** 采样周期的选择因此不是单目标优化：从辨识精度看，T_s 越大越好；从控制带宽与延迟分辨率看，T_s 越小越好；从数值条件看，T_s 又不能太小。

把三条要求放在一起，可以给出一个粗略的折中区间。以本章对象为例，最快时间常数 1.2 s，T_s 在 0.4 到 0.6 s 时参数误差已在 1% 以内，条件数在 70 上下，单精度也够用；这大约是 $T_{\min}/3$ 到 $T_{\min}/2$。控制设计常用的 $T_{\min}/10$ 到 $T_{\min}/5$（0.12 到 0.24 s）在这一列里对应 3% 到 12% 的参数误差。两者冲突时有三条出路：一是辨识与控制分采样率，用粗采样数据辨识、细采样数据验证闭环；二是只取增益与主导极点，放弃次要参数；三是直接辨识连续时间模型，绕开离散极点的挤压。**先确定用途，再定采样周期**，这个顺序不能颠倒。

24.6.6 实验 E：五类数据缺陷

实验 E 的问题是：表 24-4 列出的那些指纹，在真实数据上是不是真的能分辨出来？诊断应该按什么顺序做？

五类缺陷的注入方式都写进了脚本，可以逐行复现。单点离群是在第 800 拍加一个 0.5 的尖峰，相当于 25 倍噪声；9 位量化是量程 6 V、512 个台阶，台阶 0.0117；输入错位是把输入序列整体后移一拍；用指令代替阀位是先把指令限幅到 ± 0.6 作为阀位送给对象，辨识时却用未限幅的指令；输出偏置是直接加 0.3。

有一处值得先算清楚。“用指令代替阀位”的增益误差是 -40.73%，而 $1 - 0.6 = 0.4$。这不是巧合：输入是 ± 1 的 PRBS，限幅到 ± 0.6 相当于把整条输入序列乘了 0.6，对象的实际增益被缩小到 $1.5 \times 0.6 = 0.9$，模型估出 0.8891。**这一档误差是完全确定性的，与噪声无关，数据再多也压不下去。** 它的危险之处正在这里：残差均方根 0.0201 与基准的 0.0198 几乎相同，残差也是白的，所有基于残差的自动检查都会放行，只有把增益与真值（或与机理计算值）对一下才会露馅。

“输出含偏置 0.3”的表现则是另一种形态。参数误差 16.98%，是基准 1.06% 的 16 倍，但增益只偏 0.94%，验证 FIT 还有 92.85%。原因是 ARX 模型没有常数项，稳态方程 $y = Ku + c$ 里的 c 只能被 a 参数吸收：估计器为了让稳态对得上，把 a_1、a_2 一起往偏移方向拉，组合出来的 $1 + \sum a_i$ 却基本不变。**偏置污染的是极点，不是增益。** 这也说明为什么“增益核对”不能代替“参数物理性检查”——偏置这一档，增益核对会给出“通过”的错误结论。

“输入错位一拍”的增益误差 -10.55%、验证 FIT 86.52%，两项都超标，属于最容易发现的缺陷。它的机理是延迟被低估了一拍，ARX 用一个接近零的 b_1 与略微调整的 b_2 去补偿，这部分补偿只对低频有效，高频段对不上，于是自由运行误差变大。实际排查时不需要算这么多，只要把输入序列前后各移一拍重跑一次，看增益误差是否显著变化，就能确认。

“单点离群”与“9 位量化”属于影响有限的一类。离群点让参数误差从 1.06% 涨到 5.37%，残差均方根从 0.0198 涨到 0.0271，但增益只偏 1.08%、FIT 只掉 0.16 个百分点。量化让残差均方根涨到 0.0205，参数误差 1.40%，几乎可以忽略。两者的共同点是**能量小**：一个点、一个台阶，都只占数据总能量的千分之几，最小二乘的自加权特性把它们压住了。这也给出了处置的优先级：这两类不必急着修，先把增益与时间常数对上更重要。

把六种情形放在一起，诊断顺序就清楚了。第一步核对稳态增益，用一段独立的阶跃数据或机理计算值做参照，这一步最便宜、最灵敏，能抓住链路类缺陷（错位、限幅、量程）。第二步看残差的两条相关性，若残差非白，说明结构或延迟有问题，回到阶次与延迟的扫描。第三步检查参数的物理合理性：极点是否落在合理区间、是否出现贴近原点的噪声极点、b 的符号是否符合对象的方向。**三步的次序不能颠倒**：先查结构再查链路，会把链路缺陷误判成“阶次不够”，越加阶次越糟。

最后要强调一点：实验 E 的六种情形都是**单一缺陷**。现场数据里更常见的是两三种叠加，比如既有量化又有偏置，既有错位又有限幅。叠加之后指纹会互相掩盖：偏置让残差均值非零，掩盖了互相关的峰；限幅让增益整体偏低，掩盖了错位的 10%。这时唯一的办法是分步剥离——先去均值、再剔除限幅段、再扫延迟，每做一步重跑一次，看哪一项指标回到了基准附近。

24.6.7 六种情形的联合判据

把实验 E 的两项误差放在一起对照，可以得到一条比表 24-4 更简短的判据。一项是训练集上的一步残差均方根，另一项是独立验证集上的自由运行 FIT。

基准：0.0198 与 94.83%。这两个数构成一个“正常组合”——残差与噪声持平，FIT 在 95% 附近。六种情形里有五种的变化方向是一致的：错位是 0.0464 与 86.52%，两项同时变坏；偏置是 0.0331 与 92.85%，残差涨了 67%，FIT 掉 2 个百分点；离群是 0.0271 与 94.67%，残差涨 37%，FIT 几乎不动；量化是 0.0205 与 94.82%，两项都几乎不变。**方向一致时，按变坏的幅度排序就能排出处置优先级。**

唯一背离的是用指令代替阀位：一步残差 0.0201，比基准只高 1.5%，几乎是完美的；自由运行 FIT 58.68%，掉了 36 个百分点。这个“残差正常、FIT 崩了”的组合本身就是一个诊断信号，而且指向很明确：**残差小说明模型在每一拍都被真值纠正后能跟上；FIT 崩说明模型一旦自己跑起来就发散。两者同时出现，只有一种解释——模型的输入与实际进入对象的输入不是同一个量。** 结构不足做不到这一点，因为结构不足会让一步残差也变大（实验 B 的 $n = 1$ 是 0.0652）。

于是可以把这条判据写成一句话：**一步残差接近噪声而自由运行 FIT 低于 90%，先查链路，不要查阶次。** 这条判据的好处是不需要知道真值，只需要两项常规指标，而且它对“等比例限幅”这种残差仍然白的特例同样有效。现场验收时若只报一项，报自由运行 FIT；若报两项，就按这个组合判读。

24.6.8 五组实验的横向对照

五组实验各自回答一个问题，单看任何一组都容易失之偏颇。把它们横向摆开，能看出三条单组实验里看不见的东西。

先看每组实验“改了什么、看什么指标”。实验 A 改的是激励类型，看的是参数离散度与增益离散度，问的是“数据里有没有足够的信息”；实验 B 改的是模型结构，看的是残差的 ACF、CCF 与 Ljung–Box 统计量，问的是“模型有没有把结构搞对”；实验 C 改的是阶次，看的是 AIC、BIC 与验证 FIT，问的是“复杂度该取到几”；实验 D 改的是采样周期，看的是条件数、参数误差与增益误差，问的是“坐标系选在哪里”；实验 E 改的是数据本身的缺陷，看的是增益核对、残差与 FIT 的组合，问的是“这份数据干不干净”。五个问题对应五个环节，从前到后正好是辨识项目里真实的工作顺序。

第一条横向发现是：**五组实验里只有 A 与 D 改的是可以事先选择的设计变量，B、C、E 改的都是只能在事后发现的东西。** 激励类型与采样周期在采集数据之前就定了，选错了只能重采；残差是否白、阶次是否够、数据有没有缺陷，则要拿到数据之后才能判。这个区分决定了工作方式：A 与 D 的结论要写进采集方案，属于事前规划；B、C、E 的结论要写成检查清单，属于事后验收。把两者混在一起谈“辨识精度”，就会出现“数据已经采完了才发现激励不够”的返工。

第二条横向发现是：**同一种指标在不同实验里的分辨力差得很远，没有一种指标能通吃。** 验证均方根在实验 C 里是有用的——$n=1$ 时残差与 FIT 都明显变坏，加到 $n=2$ 就回到基准——但在实验 D 里几乎失效：六档采样周期的验证均方根落在 0.0026 到 0.0043 之间，最好与最差只差 1.7 倍，而同一批数据的参数误差却相差 125 倍。反过来，条件数在实验 D 里随 T_s 从 2 s 缩到 0.05 s 涨了 24.6 倍，是个很灵敏的探针；可它在实验 B 里完全用不上，$n=1$ 与 $n=3$ 的条件数高低说明不了任何问题，阶次不足只能靠 ACF 从 0.047 跳到 0.360 这个数量级变化来抓。原因不难理解：每个指标只对某一类失配敏感。验证误差对“预测能力”敏感，对“参数是否可辨识”不敏感；条件数对“回归量共线”敏感，对“输入通道接错”完全不敏感。**选指标的依据是先想清楚要抓哪一类问题，而不是哪一项最容易算。**

第三条横向发现是量级上的断层。实验 A、C、D 里的误差都属于“可以优化”的一类：激励从阶跃换成 PRBS 幅值 1，参数离散度从 2.92% 降到 0.79%；阶次从 1 加到 2，训练残差从 0.0652 降到 0.0201；采样周期从 0.05 s 放宽到 1 s，参数误差从 33.84% 降到 0.27%。这些误差随设计改善而单调下降，属于工程上的取舍。实验 E 里的误差则完全不同：用指令代替阀位造成 -40.73% 的增益误差，这个数是确定性的，把数据从 2000 拍加到 20000 拍也一分不会减；输入错位一拍是 -10.55%，同样是系统性的。**设计类误差可以靠优化压下去，链路类误差只能靠修正接线或改采集点消除。** 这正是 §24.5.4 把缺陷单独列成一节的道理：它不服从“数据越多越准”的规律，也就不能用增加数据量的办法绕过。

把三条发现合起来，可以得到一个与 §24.5.4 兼容、但视角不同的排序。设计辨识方案时按 A、D、C 的顺序走：先定激励与采样，再定阶次，因为前两者决定了信息量的上限，阶次只是在这个上限内做分配。验收已有数据时按 E、B、C 的顺序走：先查增益对不对，再查残差白不白，最后才看阶次够不够。两个顺序方向相反，并不矛盾——**事前按“从设计到模型”排，事后按“从便宜到昂贵”排**。增益核对只需要一段阶跃或一个机理计算值，最便宜；残差检验要重跑一遍回归，次之；改阶次、改结构要重做整套验证，最贵。

三条发现落到日常工作上，可以变成三个很具体的动作。第一个动作是**把指标按用途分成两栏**：一栏写“采数据前要定的”（激励谱、幅值、采样周期、数据长度），另一栏写“拿到数据后要查的”（增益、残差、阶次、参数物理性）。前一栏在项目启动会上就要填满，后一栏在交付前逐条打勾。第二个动作是**给每项指标配一个数量级阈值**，而不是配一个精确值：参数离散度在 1% 以内算好、5% 以上要查、几十个百分点说明信息严重不足；增益误差在 1% 以内算好、10% 以上必须查链路；自由运行 FIT 在 90% 以上算通过。阈值用数量级而不用小数位，是因为本章的所有数字都属于示例量级，换一个对象就会整体平移，而数量级的分界相对稳定。第三个动作是**保留一次基准运行**：同一套脚本、同一段数据、同样的种子，先跑出一个“正常组合”，之后每次改动都与它比。实验 E 里那些“几乎不变”和“涨了 67%”的判断，全都是相对基准而言的；没有基准，0.0331 这个数本身说明不了任何问题。

三个动作的成本也很低。分两栏只是把已有清单重写一遍；配数量级阈值只需要本章这几张表；保留基准运行就是多存一个结果文件。三者合起来，能挡住的却正是实验 E 里最难查的那一类错误——链路接错在残差上不留痕迹，只有跟基准比增益才会露出来。**检查的价值不在于查得多细，而在于查得早。**

最后一个实用的收束：把五组实验的结论按“发现问题后先动哪里”压成一句话，就是**先对增益，再看残差，三扫延迟，四定阶次，最后才回头怀疑激励与采样**。这句话把 §24.5.4 的三级排查与本节的三条发现接在了一起，也解释了为什么实验 E 虽然排在最后做，结论却要排在最前面用。顺序背后是一条朴素的成本规律：越便宜的检查越应当靠前，越贵的改动越应当靠后，这条规律与具体对象无关，可以直接搬到下一个项目里用。

24.7 小结与常见错误

本章从“一份能用的模型怎么被做坏”开始，用五组实验走完了激励、残差、阶次、采样、缺陷五个环节。这一节把结论收拢成六条，再列十种最常见的错误做法，最后给一张可以贴在工位上的排查流程图。

24.7.1 六条结论

第一，激励决定的是参数的方差，不是偏差。 四种激励的增益均值都落在真值 0.7% 以内，说明只要模型结构对、数据够长，偏差项可以忽略；差别全在离散度上，PRBS 幅值 1 是 0.79%，阶跃是 2.92%，小幅 PRBS 是 2.07%。这条结论的实践含义很直接：辨识前先问“我给了对象多少信息”，而不是先问“用哪个算法”。最小二乘、辅助变量、极大似然在激励不足时都救不了场。

第二，残差的白性检验是结构正确性的必要条件，且判据要看形状。 $n=1$ 时 ACF 0.360、CCF 0.426，两条曲线都连成缓慢衰减的弧；$n=2$ 时降到 0.047 与 0.029，形状贴着零线。$n=2$ 的 ACF 0.047 略高于带边 0.0439，但这不构成“阶次不够”的证据，因为其余滞后都在带内，没有系统性形状。**单点越界看噪声，连续越界看结构。**

第三，阶次由三条线索共同确定，训练误差不是判据。 训练均方根从 $n=2$ 到 $n=8$ 一路从 0.0199 降到 0.0196，单调下降，没有任何拐点；验证均方根则从 0.0201 升到 0.0204；AIC 与 BIC 都在 $n=2$ 取极小；残差在 $n=2$ 已经回到带边。四条证据里，唯一不能用的是训练误差，因为它只反映拟合能力，不反映泛化能力。

第四，采样周期有两侧边界，小周期下的“不可辨识”是结构性的。 $T_s=0.05$ s 时参数误差 33.84%、增益误差 21.22%，但验证均方根只有 0.0038，与最优的 0.0026 相差不到 1.5 倍。$T_s=2$ s 时参数误差 0.35%、验证均方根 0.0026，但延迟只剩 1 拍，做控制器设计没有相位信息。**采样周期的选择由用途决定，不由“精度越高越好”决定。**

第五，六类数据缺陷分三档，最危险的是残差白而增益错的那一档。 单点离群与量化可以忽略；错位与偏置必须修；用指令代替阀位会在所有残差检验上通过，却让增益偏 -40.73%，而这个数字恰好等于 $1-0.6$，是一个与噪声无关的确定性误差。防它的办法只有一条：回归量取实际执行器反馈，不取设定值。

第六，辨识的交付物是四件套，不是一个参数向量。 增益（与真值或机理值对比）、极点位置（是否落在物理合理区间）、验证 FIT（自由运行而非一步预测）、残差的两条相关性。四项齐全，模型才算可用；缺任何一项，下游都会在某个特定场景下踩坑。

这六条结论的强度并不相同，读的时候要分清哪一条是定理、哪一条是经验、哪一条只对本章对象成立。

属于定理的是第二、第三、第六条背后的关系：残差白性是模型结构正确的必要条件（否则残差里必然留有可被解释的成分）；偏差—方差分解的 U 形曲线；$1/\sqrt{N}$ 与 $1/\sigma_u$ 的方差标度；正规方程的条件数平方。这些结论在任何线性模型上都成立，与具体对象无关。

属于经验的是第一、第四条中的数值门槛：参数离散度“1% 以内算好”、“3 到 5 倍的修正系数”、$\kappa\approx 32/T_s$ 的标度、$t_r/20\le T_s\le t_r/5$ 的区间。这些是从本章对象与若干类似对象上归纳出来的，换对象要重新标定，但标定的方法（§24.6 的五组实验）可以照搬。

只对本章对象成立的是具体数字：1.5046 的增益、0.82 与 0.43 的极点、94.83% 的 FIT、-40.73% 的限幅误差。引用这些数时只能当作“示例量级”，不能当作“该对象的真值”。

分清这三类的好处是知道什么时候可以迁移。工程上最常见的误用是把经验门槛当定理——例如把“参数离散度 1%”直接写进验收标准，而对象的信噪比根本达不到；或者把本章的数字当通用结论——例如认为“增益误差 40% 就意味着限幅到 60%”，换一个输入幅值这个比例就变了。

24.7.2 十种常见错误

把这十种错误按发生频率排序，列在表 24-9 里。前四种占了现场问题的大半，它们的共同特征是“指标看起来很好”。

表 24-9　辨识中的十种常见错误

错误做法	典型症状	后果	纠正
用训练误差选阶次	训练误差单调下降	阶次加到上限，验证变差	用验证误差加 BIC
只看 FIT 不看参数	FIT 94% 以上	小 T_s 下参数偏 34%	同时报增益与参数
激励幅值取得过小	残差正常、拟合很好	参数离散度放大 3 至 4 倍	安全允许下取最大幅值
用设定值代替阀位	残差白、均方根正常	增益偏 -40.73%	回归量取阀位反馈
输入输出错位	互相关出现固定峰	增益偏 -10.55%	扫延迟 ± 2 拍
不去均值、不处理偏置	参数漂移、增益正常	参数误差 16.98%	先减均值或加常数项
限幅段不剔除	残差非白、增益偏低	模型只在小信号段有效	标记并剔除饱和段
多工况混在一起拟合	每个工况都不准	得到一个“平均模型”	分工况或加调度变量
用正规方程解高阶模型	结果依赖求解器	条件数被平方	改用 QR 或 SVD
只给点估计不给不确定度	下游按 1% 精度使用	实际误差 10% 以上	报标准差或区间

这十条里，前三条与“指标好看”直接相关，也正是最容易被验收放过的。一个模型的 FIT 94%、残差白、增益对得上，仍然可能在 T_s 与阶次上选错了；反过来，一个增益偏 40% 的模型，残差检验完全正常。**验收不能只看一个指标，这与第六条的“四件套”是同一件事的两种说法。**

后四条属于工程习惯问题，纠正成本低但容易被忽略。限幅段不剔除在多工况对象上尤其常见；正规方程在高阶时的问题在实验 C 里已经量化，抬升一旦超过式 (24-32) 的预测，就该换求解器；不给不确定度的后果最隐蔽，因为下游会默认“辨识出来的数就是真值”。

24.7.3 三级排查流程

把前六节的判据按“先查什么、后查什么”串起来，就是图 24-6 的流程。

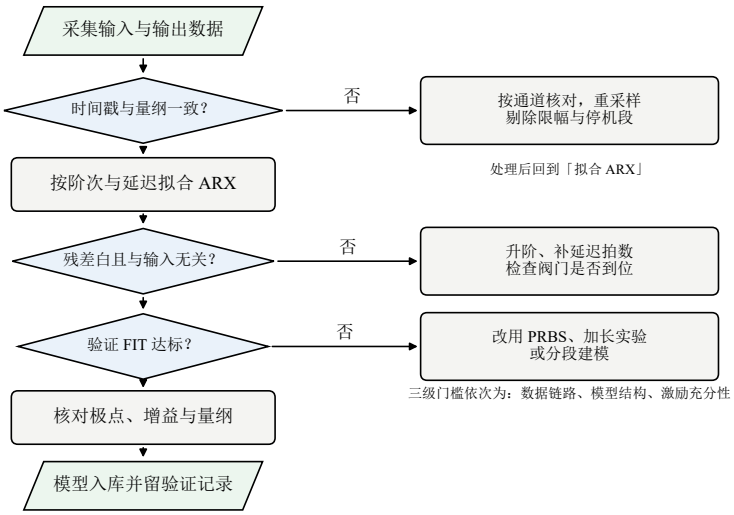


图 24-6　辨识失败的三级排查流程

流程的主干只有六步：采集数据、核对时间戳与量纲、拟合 ARX、检验残差、检验验证 FIT、核对极点与增益，最后入库。三个菱形是三道门槛，任何一道答“否”，就进入右列的处置框，处理完回到“拟合 ARX”重新开始。

三道门槛的顺序是有讲究的，它们分别对应数据链路、模型结构、激励充分性三个层次。

第一道门槛查的是**链路**：时间戳是否对齐、通道量纲是否一致、输入取的是不是实际阀位。这一层不过，后面的检验全部失真。实验 E 的错位与限幅都会让残差非白、FIT 掉到 60% 至 86%，看起来完全是“结构不对”；如果先按结构问题去升阶，会得到一个阶次很高、拟合很好、增益仍然错的模型，把问题埋得更深。链路检查的成本最低，一条阶跃曲线的稳态增益就能看出大半。

第二道门槛查的是**结构**：残差是否白、是否与输入无关。这一层不过，才是真正需要动阶次与延迟的地方。处置框里写的是“升阶、补延迟拍数、检查阀门是否到位”，三件事的次序也是从便宜到贵：先扫延迟（把输入前后移几拍重跑），再升阶，最后才怀疑执行机构。

第三道门槛查的是**激励**：验证 FIT 与参数精度是否达标。这一层不过，说明数据里没有足够的信息，处置是改激励（换 PRBS、加长实验）或分段建模，而不是继续调算法。

流程里“回到拟合 ARX”这一步容易被误解成“重跑一遍”。它的实际含义是：**处置改变了数据或结构，必须用改过之后的数据重新拟合，而不是把旧的参数拿来修补。** 一个常见的错误是发现增益偏低之后，手工乘一个 1.4 的系数凑上去；这等于把链路缺陷藏进参数里，下一次换工况又会露出来。

什么时候停止排查？三条门槛全部通过、四件套齐全，就可以入库。若某一条门槛反复不过、处置了两三轮仍无改善，说明本章的方法路线不适用于这个对象，应当换路线：闭环数据改用第 21 章的方法，非线性主导改用第 22 章，参数漂移改用第 23 章的在线估计。**排查流程的价值不只在于找到问题，也在于及时判定“此路不通”。**

24.7.4 从参数辨识到模型验证的迁移

本章讲的是 ARX 与最小二乘，但六条结论里没有一条依赖具体算法。它们依赖的是三件事：数据里有信息、模型结构与数据匹配、评价口径与用途一致。这三件事在后续章节里反复出现。

第 25 至 27 章转向数据驱动建模。那里的“训练集与验证集”对应本章的训练数据与验证数据，“过拟合”对应本章的方差项，“特征泄漏”对应本章的“用指令代替阀位”——本质都是把不该进模型的信息放了进去。本章用 0.0201 与 0.0204 这一对数字说明的偏差—方差关系，在那些章节里会以学习曲线的形式再次出现。

第 21 章的闭环辨识要修正本章的一条结论。本章的激励分析假设输入与噪声独立，闭环下这个假设不成立：反馈让输入带上噪声的成分，最小二乘会有偏。处置是加外部激励或用工具变量，代价是激励幅值受限，参数方差比开环大。本章式 (24-28) 的方差公式仍然可用，只是要把“有效信噪比”换成闭环下的值。

第 23 章的在线辨识与本章的关系最直接。遗忘因子 λ 相当于把数据长度 N 换成有效长度 $1/(1-\lambda)$，按 $1/\sqrt{N}$ 的标度，$\lambda = 0.99$ 对应的有效长度是 100 拍，方差是 2000 拍数据的 $\sqrt{20} \approx 4.5$ 倍。**想让在线估计既跟得上变化又不太抖，本质是在本章的方差公式里选 N。** 这个换算比任何整定口诀都好用。

最后把三条可迁移的原则留在这一章的末尾。**口径先行**：先确定模型拿去做什么，再决定报增益还是报参数、选多大的 T_s、验证用一步预测还是自由运行。**残差为纲**：残差的两条相关性是唯一能同时反映结构、延迟与链路问题的体检指标，任何模型交付前都要看。**叠加必剥离**：现场缺陷从不单独出现，一次只改一件事、改完重跑，才能知道是哪一项起了作用。这三条不随算法变化，也不随对象变化。

第 25 章 模式识别

第 24 章把“一份数据能不能变成一个模型”讲完了。那一章的产物是参数、阶次、延迟和不确定度——都是连续量。工程现场还有另一类问题：设备现在是正常还是异常，异常属于哪一种，这批产品合不合格，相机里的目标是哪一个零件。这类问题的答案不是数，而是类别。把连续量变成类别，靠的是模式识别。本章讨论它在控制工程里的形态：怎样从一段信号里取出几个数，怎样用这几个数把工况分开，怎样判断一个新样本属于已知类别还是根本没见过，以及这套东西上线之后会怎样慢慢失效。

25.1 动机与要解决的问题

25.1.1 辨识与识别的分界

先把这个分界说清楚，因为它在工程上常被混淆。参数辨识问的是“对象的系数是多少”，输出是实数向量，评价标准是误差的方差；模式识别问的是“这段数据属于哪一类”，输出是离散标签，评价标准是判对的比率。前者的错误可以连续地小，后者的错误只有对与错两种。

这个区别带来三个连锁后果。第一，数据量的要求不同。辨识要估 4 个参数，2000 拍数据足够；识别要估的是类条件分布的形态，每一类都需要独立的一段数据，五类就是五份。第二，模型形式不同。辨识的模型是差分方程，参数有物理意义；识别的模型是特征空间里的一个划分，划分边界本身没有物理意义，只有它落在哪些特征上才有。第三，失效方式不同。辨识失效表现为参数漂移、预测变差，是渐进的；识别失效表现为某一类突然被整体判错，是阶跃式的，而且往往发生在工况变化之后，不在训练集覆盖的范围内。

控制工程里两类问题往往同时出现。一个伺服系统，机械参数（惯量、摩擦）要靠辨识得到，用于前馈补偿；而“负载是否卡滞”“编码器是否跳变”要靠识别判断，用于报警和切换。前者写进控制器，后者写进监控。把两者混为一谈，最常见的错误是拿辨识的精度指标去要求分类器——比如要求分类器“误差小于 1%”，这个说法没有意义，分类器的指标只能是准确率、误报率、漏报率。

还有一层区别在数据来源上。辨识必须知道输入，回归量里要有 u；识别可以只用输出。这一点在现场很实际：很多老设备只留了测量通道，没有同步记录执行器指令，此时参数辨识做不了，模式识别仍然可以做。本章的实验里有一半特征只用输出，另一半（低频增益）需要输入，第 25.6 节会看到这两类特征的判别力差异。

25.1.2 一个具体的诊断任务

本章用一个统一的算例贯穿全章：一台二阶过程，传递函数为

$$G(s)=\frac{K}{(T_1 s+1)(T_2 s+1)},\qquad T_2=3\ \text{s} \tag{25-1}$$

采样周期 $T_s=0.5$ s，激励为每 4 s 切换一次的双电平伪随机信号，每段记录 256 拍。过程有五种运行状态：

1. 正常：$K=1.5$、$T_1=8$ s、噪声标准差 0.02；
2. 执行器增益退化：K 降到 1.05，其余不变；
3. 换热面结垢：T_1 从 8 s 增到 16 s，其余不变；
4. 传感器零点偏置：输出叠加 0.25，其余不变；
5. 传感器噪声增大：噪声标准差从 0.02 增到 0.08，其余不变。

这五种状态对应五类现场问题：执行机构磨损、传热恶化、仪表零点漂移、接线干扰或接地不良。它们有一个共同点——都不会让系统立刻停机，但都会让控制品质慢慢变差，所以需要在线判别。每类采集 200 段，共 1000 段数据，每段独立抽取激励相位与噪声。

图 25-1 左列画出五类状态的典型波形。人眼能看出三件事：增益退化那一类的幅值明显偏小；结垢那一类变化更慢、波形更“钝”；噪声增大那一类毛刺明显。零点偏置那一类最难用眼睛判断——它的波形形状与正常几乎一样，只是整体抬高，而这种抬高只有 0.25，在 ±1 的波动里不显眼。右列的功率谱把差别放大了：增益退化整条谱线向下平移，结垢把能量压向低频，噪声增大把高频段抬起来，零点偏置则几乎不动。

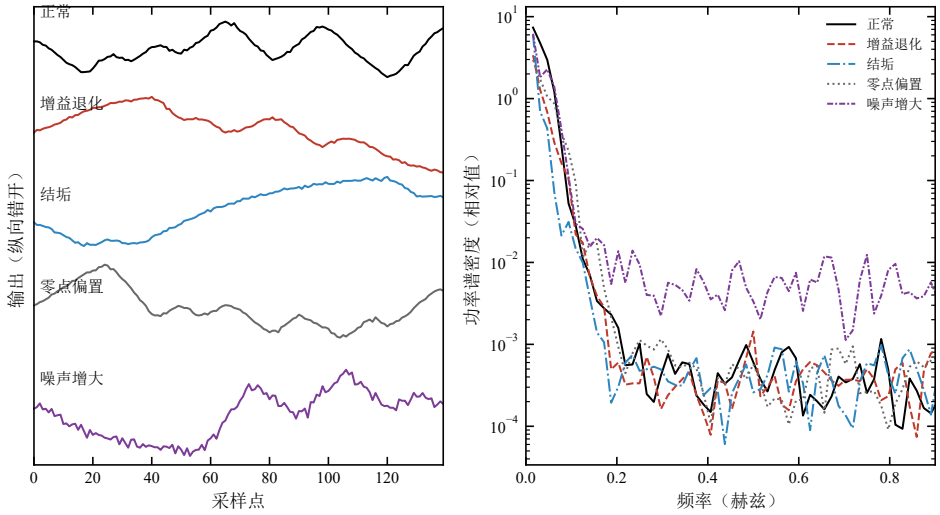


图 25-1　五类工况的典型波形与功率谱

这张图已经说明了模式识别的核心困难：**有些类别在原始波形上几乎不可见，必须换一个“看”的角度。** 零点偏置在时域里被波动掩盖，在频域里没有痕迹，但它的段均值是 0.25，而正常段的均值在零附近抖动——只要把“段均值”当成一个特征，这一类立刻就分开了。反过来，如果把每段数据先去均值再分析（这是信号处理的习惯做法），零点偏置的信息就被自己删掉了。特征提取的第一条原则由此而来：**先去什么、留什么，取决于要判哪一类故障，而不取决于信号处理教科书的默认流程。**

25.1.3 三个先导反例

在展开方法之前，先看三个“看起来该成功、实际失败”的结果。它们构成本章的问题清单。

第一个反例关于降维。把 13 个特征标准化后做主成分分析，前两个主成分解释了 61.54% 的方差，画在平面上是一张漂亮的图；但五类的中心在二维平面上最小间距只有 0.08，而类内平均标准差是 1.39——中心间距比类内散布小一个数量级，五团点完全糊在一起。也就是说，**保留方差最多的两个方向，恰好不是分开类别的方向。** PCA 是无监督的，它不知道类别存在，自然不能保证保留判别信息。

第二个反例关于聚类。用 K 均值把 1000 段数据分成 5 簇，与真实标签比较，调整兰德指数只有 0.251；轮廓系数在簇数取 3 时最高（0.235），而不是取 5 时。也就是说，**聚类算法认为“三类”最自然，而真实类别是五类。** 原因很直接：K 均值最小化的是簇内欧氏距离，它把“正常”与“噪声增大”合成一簇——这两类在特征空间里确实挨得近，只是判别边界可以切开它们。聚类回答的是“数据在哪里密”，分类回答的是“边界在哪里放”，两个问题的答案不必一致。

第三个反例关于拒识。用四类训练一个线性判别器，把第五类当作“未知”，再用后验概率最大值做拒识判据：置信度高于阈值就接受，低于就拒绝。结果受试者工作特征曲线下的面积（AUC）只有 0.214——比随机猜（0.5）还差。原因是线性判别的后验在远离所有类中心时会饱和到 1：未知样本离所有已知类都远，线性判别函数的值反而更大，置信度更高。换成马氏距离判据后，同一批数据的 AUC 是 1.000。**同一个模型，换个判据，拒识能力从“完全失效”变成“完全可分”。**

三个反例指向同一个结论：模式识别的成败主要不在分类器，而在特征、判据与评价方式。本章余下的部分就按这三件事展开。

25.1.4 四道关口

把上面的经验整理成四道关口，列在表 25-1。每一道关口都有一个“看起来合理但会出错”的做法，和一个“成本低但有效”的对照做法。

表 25-1　模式识别的四道关口与常见陷阱

关口	要回答的问题	常见陷阱	对照做法
特征	哪些量能把类别分开	沿用信号处理的默认流程（先去均值、只看幅值）	先按物理机理列出候选，再看判别力排序
判别	边界放在哪里	只比较分类器，不比较特征集	固定分类器，扫特征数；固定特征，扫分类器
无监督	有几个类	把聚类数当作类别数	用聚类发现新类，不用它代替分类
部署	新样本可信吗	用后验概率做拒识	用距离或密度做拒识，另设阈值

四道关口的顺序不能颠倒。特征没选对，换再好的分类器也只能在噪声里找边界；判别边界没验好，聚类得到的簇数没有意义；前三道都过了，部署阶段的拒识与漂移仍然会让系统在上线三个月后失效。表 25-1 的最后一列都不是“更高级的方法”，而是“更便宜的对照”——这一点在后文反复出现。

第三道关口容易被低估。工程上常有人问“能不能不用标签，直接聚类看看有几类故障”。答案是：聚类可以用来发现“有一批样本谁也不像”，但不能用来给它们命名，也不能用来确定类别数。本章实验里 K 均值给出 5 簇时 ARI 只有 0.251，给出 3 簇时轮廓系数最高，两个数字都不支持“聚类能替代分类”这个想法。

第四道关口则是上线之后才暴露的。本章实验里，训练集准确率 92.90% 的模型，在传感器量程整体偏大 15% 之后掉到 82.80%；如果只用形状类特征，准确率虽然对量程漂移完全不敏感（57.70% 对 57.70%），但水平低了三十五个百分点。这两个数字一起说明：**鲁棒性与判别力是一对需要显式权衡的量，不能靠“多收集数据”同时解决。**

25.1.5 五个定量问题与路线

把本章要回答的问题写成五条，每条都对应一个可以量化的实验：

1. 特征空间长什么样？主成分能解释多少方差，类中心间距与类内散布差多少？（实验 A）
2. 五种分类器在同一批特征上差多少？错分发生在哪两类之间？（实验 B）
3. 特征数与训练样本数各需要多少？准确率在哪个点饱和？（实验 C）
4. 无监督方法能不能替代分类？簇数、轮廓系数与真实类别数是什么关系？（实验 D）
5. 未知类能不能被拒识？工况漂移之后模型还能用吗？（实验 E、F）

这五个问题由浅入深：前两个问“能不能分开”，第三个问“要多少代价才能分开”，后两个问“分开的结论可不可信”。工程上最常见的失误是只做前两个——在训练集上把准确率刷到 95%，就认为任务完成，完全不做后两个。

本章的结构对应这条路线。25.2 节给定义与数学基础，25.3 节推导几条决定成败的性质（判别方向的来源、维数上限、后验饱和），25.4 节讲数值实现与部署细节，25.5 节给适用边界，25.6 节用六组实验逐个回答上面五个问题，25.7 节收拢结论与常见错误。

25.2 定义与数学基础

25.2.1 从一段信号到一个特征向量

模式识别的输入是一段有限长的采样序列，输出是一个标签。中间那一步叫特征提取：把长度 256 的实数序列压缩成十几个数，并且要求这十几个数里保留了区分类别所需的信息。

设一段输出为 $y_0, \dots, y_{N-1}$，$N = 256$。本章用到的特征分三类。

第一类是幅值与波形统计量。段均值保留直流信息，

$$\bar{y} = \frac{1}{N}\sum_{k=0}^{N-1} y_k \tag{25-2}$$

均方根与峰峰值描述波动幅度，

$$y_{\mathrm{rms}} = \sqrt{\frac{1}{N}\sum_{k=0}^{N-1}(y_k-\bar{y})^2}, \qquad y_{\mathrm{pp}} = \max_k y_k - \min_k y_k \tag{25-3}$$

峭度描述波形的尖峰程度，

$$\kappa = \frac{\frac{1}{N}\sum_{k=0}^{N-1}(y_k-\bar{y})^4}{y_{\mathrm{rms}}^4} \tag{25-4}$$

过零率是符号变化的频次，$z = \frac{1}{N-1}\sum_{k=1}^{N-1}\mathbf{1}\{\mathrm{sgn}(y_k-\bar{y}) \neq \mathrm{sgn}(y_{k-1}-\bar{y})\}$。这五个量都是“幅值类”的，共同特点是对量程变化敏感——传感器增益整体偏 15%，它们就整体偏 15%，判别边界立刻失配。

第二类是频域与相关类特征。归一化自相关为

$$r(\tau) = \frac{\sum_{k=\tau}^{N-1}(y_k-\bar{y})(y_{k-\tau}-\bar{y})}{\sum_{k=0}^{N-1}(y_k-\bar{y})^2} \tag{25-5}$$

取 $|r(\tau)|$ 首次落到 0.5 以下的滞后 $\tau_{1/2}$，乘以采样周期得到“自相关半衰时间”，它直接反映主导时间常数。功率谱用 Welch 法估计，得到

$\hat{S}_y(f)$，归一化成概率分布 $p_f = \hat{S}_y(f) / \sum_f \hat{S}_y(f)$ 后，谱质心与谱熵为

$$f_c = \sum_{f} f\, p_f, \qquad H = -\frac{\sum_f p_f \ln p_f}{\ln M} \tag{25-6}$$

其中 M 是频点个数。低频占比与高频占比分别是 p_f 在 $f < 0.03$ Hz 与 $f > 0.30$ Hz 上的和。

第三类需要输入信号：用互谱估计低频增益，

$$\hat{K}_{\mathrm{lf}} = \operatorname{median}_{f<0.02\,\mathrm{Hz}} \frac{|\hat{S}_{uy}(f)|}{\hat{S}_{uu}(f)} \tag{25-7}$$

这是第 24 章频域辨识的简化版，只取一个频段、不做参数拟合。它能把“增益变小”与“变慢”分开：增益退化让 $\hat{K}_{\mathrm{lf}}$ 成比例下降，结垢则基本不动。

最后一个是差分粗糙度

$$\rho = \frac{\sqrt{\frac{1}{N-1}\sum_{k=1}^{N-1} (y_k - y_{k-1})^2}}{y_{\mathrm{rms}}} \tag{25-8}$$

它是无量纲的，对量程不敏感，对噪声与高频成分很敏感。

13 个特征拼成一个向量 $\mathbf{x} \in \mathbb{R}^{13}$，一段数据就变成了特征空间里的一个点。这个映射不是单射：不同的波形可能映到同一个点，不同的点也可能来自同一类工况。模式识别的全部困难，都来自这个映射丢掉了什么。

把这 13 个特征按“对哪一类敏感”排一遍，可以看出它们并不是平权的。段均值只对零点偏置敏感，其余四类的均值都在零附近抖动；均方根与峰峰值对增益退化敏感，因为增益直接缩放幅值；自相关半衰与低频占比对结垢敏感，主导时间常数从 8 s 增到 16 s，波形变钝，能量压向低频；峭度、过零率、高频占比、谱熵与粗糙度都在度量“波形有多毛糙”，因而对噪声增大敏感；低频增益同时对增益退化与激励幅值敏感，是唯一必须用到输入的特征。这张“特征—类别”对应关系应当在设计阶段就写下来，它决定了后续任何一次特征增删会影响哪几类。

为什么需要 13 个而不是 5 个？因为单个特征很少只对一类敏感。幅值类特征对增益退化敏感，对噪声增大同样敏感；粗糙度对噪声敏感，对量程漂移也略有反应。多留几个特征，是为了让判别方向能在“几个特征同时变化”的组合里找到区分依据。代价有两项：协方差矩阵的估计误差随维数增长，计算量随特征数线性增长。25.3.5 节会把这两项代价换算成样本量要求。

还有一层要注意：特征的物理来源决定了它的失效模式。段均值依赖传感器的直流通道，零点漂移与温漂都会污染它，但它对采样抖动不敏感；谱类特征依赖频率轴，采样周期的任何偏差都会让它整体平移；低频增益依赖输入输出同步，两个通道之间若有一个采样周期的错位，相位信息就错了，幅值信息还在，所以它比相位类特征稳，但也不完全免疫。**选特征时要连同“它在什么条件下会骗人”一起记下来**，这条信息在上线后的故障排查里比特征本身的数值更有用。

25.2.2 贝叶斯决策

设类别 $C_1, \dots, C_m$，先验概率 $P(C_k)$，类条件密度 $p(\mathbf{x} \mid C_k)$。给定观测 $\mathbf{x}$，后验概率由贝叶斯公式给出

$$P(C_k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid C_k) P(C_k)}{\sum_{j=1}^{m} p(\mathbf{x} \mid C_j) P(C_j)} \tag{25-9}$$

最小错误率决策取后验最大的类，

$$\hat{c}(\mathbf{x}) = \arg\max_{k} P(C_k \mid \mathbf{x}) \tag{25-10}$$

分母与 k 无关，所以判决只依赖分子 $p(\mathbf{x} \mid C_k) P(C_k)$。两类情形下，边界由 $p(\mathbf{x} \mid C_1) P(C_1) = p(\mathbf{x} \mid C_2) P(C_2)$ 决定；若两类都是等协方差的高斯分布，取对数后二次项相消，边界退化成超平面

$$\mathbf{w}^{\mathsf{T}} \mathbf{x} + w_0 = 0, \qquad \mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \tag{25-11}$$

这个式子是后面所有线性方法的源头。它说明两件事：判决方向由“类中心之差经协方差白化”决定；先验只移动阈值 w_0，不改变方向。若两类协方差不同，二次项不相消，边界是二次曲面，这就是二次判别。

最小错误率并不是工程上唯一的目标。若把“正常判成故障”与“故障判成正常”的代价分别记为 c_{12} 与 c_{21}，判决应改为最小化期望风险 $\sum_{j \neq k} c_{jk} P(C_j \mid \mathbf{x})$，取使该和最小的类。两类情形下这等价于把阈值从 0 移到 $\ln(c_{12}/c_{21})$，方向不变。**代价只改阈值，不改方向**，这条结论在多类情形下同样成立，是后面“分级报警”做法的理论依据。现场常见的要求是“漏报代价是误报的二十倍”，换算过去就是把阈值往

“更容易报警”的一侧推，用 20 倍的误报换取漏报的减少。本章实验都按等代价处理，所以给出的准确率是一个中性参照；真实系统里必须把代价比写进阈值。

贝叶斯决策还有一个常被忽略的选项：拒绝判决。若所有类的后验都低于某个水平，就输出“无法判定”而不是硬选一个。这在理论上等价于给每类加一个“拒识”的代价，在工程上就是把“拿不准”变成一条显式的输出。可惜的是，如 25.3.3 节将要推导的，后验概率本身在远离训练分布时的行为并不可靠，所以“拿不准”这件事不能只靠后验来判断。

还要区分两个容易混淆的模型。式 (25-11) 假设各类共享协方差，参数少、边界是超平面，这叫线性判别；若让每类各用自己估计的协方差，边界变成二次曲面，这叫二次判别，在文献里常与“高斯贝叶斯”混称。本章实验里二次判别的准确率是 84.40%，比线性判别的 92.70% 低 8.3 个百分点——原因不是二次边界表达能力不够，而是每类 200 段数据估一个 13×13 协方差太勉强。**模型复杂度要由数据量决定，而不是由“理论上更一般”决定**，这一点在第 24 章的阶次选择里已经出现过一次。

25.2.3 线性判别与 Fisher 判据

实际工作中类条件密度往往未知，$P(C_k)$ 也不知道。Fisher 的思路是绕开密度估计，直接找一个投影方向，使投影后“类间散得开、类内挤得紧”。

记类内散度矩阵与类间散度矩阵

$$S_w=\sum_{k=1}^{m}\sum_{\mathbf{x}\in C_k}(\mathbf{x}-\boldsymbol{\mu}_k)(\mathbf{x}-\boldsymbol{\mu}_k)^{\mathsf T},\qquad S_b=\sum_{k=1}^{m} n_k(\boldsymbol{\mu}_k-\boldsymbol{\mu})(\boldsymbol{\mu}_k-\boldsymbol{\mu})^{\mathsf T} \tag{25-12}$$

其中 n_k 是第 k 类的样本数，$\boldsymbol{\mu}$ 是总均值。Fisher 判据是两者的比值

$$J(\mathbf{w})=\frac{\mathbf{w}^{\mathsf T}S_b\mathbf{w}}{\mathbf{w}^{\mathsf T}S_w\mathbf{w}} \tag{25-13}$$

对 $\mathbf{w}$ 求导并令其为零，得到广义特征值问题

$$S_b\mathbf{w}=\lambda S_w\mathbf{w} \tag{25-14}$$

S_b 的秩不超过 $m-1$，所以非零特征值最多 $m-1$ 个。这条性质有一个很实际的推论：**线性判别最多把数据投影到 $m-1$ 维，五类问题最多四维，再多的判别方向不存在。** 试图用线性判别把五类画在二维平面上，必然丢掉一部分判别信息——这是 25.1 节第二个反例的另一面。

当 S_w 可逆时，式 (25-14) 等价于对 $S_w^{-1}S_b$ 做特征分解；由于 S_b 的秩限制，实际只需算前 $m-1$ 个特征向量。这个“先白化再找类间方向”的两步结构，与式 (25-11) 完全一致：Fisher 判别就是“协方差已知、均值未知”情形下的贝叶斯边界。

25.2.4 最大间隔与核方法

线性判别只要求“分对”，不要求“分得稳”。支持向量机多加一条要求：边界两侧要留出尽可能宽的空白带。二类情形的原始问题是

$$\min_{\mathbf{w},b,\boldsymbol{\xi}}\ \tfrac12\|\mathbf{w}\|^2+C\sum_{i=1}^{n}\xi_i,\quad \text{s.t. } y_i(\mathbf{w}^{\mathsf T}\mathbf{x}_i+b)\ge 1-\xi_i,\ \ \xi_i\ge 0 \tag{25-15}$$

其中 $y_i\in\{-1,+1\}$，ξ_i 是允许越界的松弛量，C 控制“间隔宽度”与“越界惩罚”的交换。对偶问题只依赖样本之间的内积，把内积换成核函数 $K(\mathbf{x}_i,\mathbf{x}_j)$ 就得到非线性边界

$$f(\mathbf{x})=\sum_{i=1}^{n}\alpha_i y_i K(\mathbf{x}_i,\mathbf{x})+b \tag{25-16}$$

本章用高斯核 $K(\mathbf{x},\mathbf{x}')=\exp(-\gamma\|\mathbf{x}-\mathbf{x}'\|^2)$，$\gamma$ 取 $1/(13\cdot \mathrm{Var})$ 的量级。

核方法的意义在于：它在原始 13 维空间里画的是曲面边界，但在高维空间里仍是超平面，所以优化问题不难；代价是模型不再由少数几个系数描述，而由一部分训练样本（支持向量）描述，部署时要把它们一起存下来。这一点在嵌入式场景里很实际——线性模型只存 14 个数，核模型可能存几百个样本。

对偶问题值得写出来，因为它解释了“支持向量”这个名字。引入拉格朗日乘子 $\alpha_i\ge 0$ 与 $\mu_i\ge 0$，对原始变量求偏导置零，得到 $\mathbf{w}=\sum_i\alpha_i y_i\mathbf{x}_i$ 与 $\sum_i\alpha_i y_i=0$；代回后问题只含 α，即最大化 $\sum_i\alpha_i-\frac12\sum_{i,j}\alpha_i\alpha_j y_i y_j K(\mathbf{x}_i,\mathbf{x}_j)$，约束为 $0\le\alpha_i\le C$ 与 $\sum_i\alpha_i y_i=0$。互补松弛条件要求 $\alpha_i\,[y_i f(\mathbf{x}_i)-1+\xi_i]=0$。落在间隔边界上或越界的样本才有 $\alpha_i>0$，其余样本的 α_i 恰为零，对判决毫无贡献——这就是“只有少数样本决定边界”的来源。工程含义很直接：**训练集里大部分数据可以丢掉而不改变模型**，但丢掉哪些要由优化结果决定，不能凭感觉挑。

两个超参数的作用方向相反又相互纠缠。C 越大，越不允许越界，间隔越窄，偏差小、方差大；C 越小，允许更多样本落在间隔内，边界更平滑。高斯核宽度 γ 越大，每个样本的影响半径越小，边界越弯曲，同样偏向过拟合。本章取 $C=10$、γ 按特征数自动缩放，没有做网格搜索，原因是这一节的目的是比较分类器而不是刷分。若真要调，正确做法是在训练折内部再做一层交叉验证，而不是在测试集上试——后者是 25.3.6 节讲的泄漏。

多类情形要靠组合二类器。常用的两种策略是“一对一投票”与“一对余”：前者五类需要十个二类器，判决时按投票数定类，票数相同时用置信度打破平局；后者五类只需五个二类器，但每个二类器面对的是类别极不平衡的问题（一类对四类）。本章的实现用一对一，因为每类的样本量相同，十个二类器各自均衡。**类别数增加时，一对一的训练次数按平方增长**，二十类就是一百九十个二类器，这时该考虑直接的多类形式或换模型。

25.2.5 聚类：K 均值与密度法

无监督情形下没有标签，只能按“点在空间里怎么聚集”来分组。K 均值最小化簇内平方和

$$J(\{\boldsymbol{\mu}_j\},\{c_i\})=\sum_{i=1}^{n}\left\|\mathbf{x}_i-\boldsymbol{\mu}_{c_i}\right\|^2 \tag{25-17}$$

交替执行“分配”与“更新均值”两步，直到分配不再变化。这个目标函数只与欧氏距离有关，因此它隐含一个假设：簇是各向同性的球，且大小相近。本章数据标准化后仍有相关结构，这个假设并不成立——25.6 节会看到后果。

DBSCAN 换一个思路：不预设簇数，而是按密度连通性生长。给定邻域半径 ε 与最少点数 $m_{\min}$，若一个点的 ε 邻域内至少有 $m_{\min}$ 个点，称其为核心点；核心点及其密度可达的点归为一簇，剩下的标为噪声。它不需要指定簇数，能识别任意形状，但对 ε 极其敏感：ε 小一点，全部变成噪声；大一点，所有簇连成一片。本章实验里 ε 从 0.6 扫到 2.0，噪声比例从 100% 掉到 5.7%，中间没有一段“既不过噪也不过连”的平台。

两类算法都有几个实现层面的选择。K 均值的第一步是初始化，随机选点会让结果依赖种子，常用的补救是 K 均值加加：按“离已选中心越远越可能被选中”的概率逐个抽初始中心，使初始点尽量分散，收敛更快也更稳。本章实验固定随机种子，五折之间不做重抽，所以 ARI 0.251 这个数字是可复现的，但它本身可能因种子不同在正负 0.03 内浮动。

层次聚类不用预设簇数，它先算出样本之间的距离矩阵，再自底向上合并最近的两簇，得到一棵树；在任意高度切一刀就得到一种划分。工程上它的价值在于“树本身可看”：哪两类最先合并，说明它们在特征空间里最难分开，这个信息与混淆矩阵互为印证。本章没有把层次聚类列入实验，所以不给具体数字；真要判断合并顺序，直接读树状图比看 ARI 更直观。代价是 $O(n^2)$ 的内存与 $O(n^3)$ 的时间，1000 个样本可以接受，十万个就不行。

DBSCAN 的两个参数不是独立的。经验上取 $m_{\min}$ 略大于特征维数（本章 13 维，取十几的量级），ε 用“按距离排序后第 $m_{\min}$ 近邻距离的拐点”去定，比手工试要稳。本章实验固定 $m_{\min}$、只扫 ε，所以看到的 100% 噪声到 5.7% 的剧烈变化，部分是参数固定造成的。真正的问题不在调参：即便取到 ARI 最高的一点（0.144），也远不足以支撑“自动分五类”。

还有一种常被提起的方法是谱聚类，它把样本看成图的节点、相似度看成边权，对拉普拉斯矩阵做特征分解后再聚类。它能处理非凸簇，理论上是 K 均值的推广，但同样需要指定簇数，且对相似度的尺度敏感。本章没有把它列入实验，因为结论不会变：**只要算法的目标里没有“类别”这个概念，它就没有理由把边界放在类别该在的地方。**

25.2.6 主成分分析

PCA 找的是方差最大的方向，与类别无关。第一主成分是

$$\mathbf{v}_1=\arg\max_{\|\mathbf{v}\|=1}\mathbf{v}^{\mathsf{T}}\Sigma\mathbf{v} \tag{25-18}$$

解是协方差矩阵最大特征值对应的特征向量，第 j 个主成分解释的方差比例是 $\lambda_j/\sum_i\lambda_i$。它是最优的“线性压缩”：在给定维数下，重构误差最小。但“重构误差最小”与“分类错误最少”是两个目标，前者不需要标签，后者必须有标签。25.1 节的第一个反例就是这两个目标分道扬镳的实例：前两主成分保留了 61.54% 的方差，却几乎没有保留判别信息。

工程上 PCA 的正确用法有三个：看数据有没有明显的结构（离群、分层）；把 13 维压到 2 维做可视化；在特征高度相关时先去相关再训练，减少共线。把它当作“分类前的必备步骤”是误用。

判断保留几维，常用的做法是看特征值谱：把 λ_j 从大到小画出来，找拐点，或者要求累计比例达到 85%。本章 13 维的谱是缓慢下降的，前两维 61.54%、前三 73.24%，没有明显拐点，这本身就是“数据没有低维结构”的证据。**谱平缓意味着信息均匀摊在很多方向上，压缩必然有损**；若谱在第三、四维之间陡降，才说明低维表示抓住了主要结构。

PCA 还有一个副产品叫白化：把主成分再除以各自的奇异值，得到各方向方差相同的坐标。白化之后欧氏距离等价于马氏距离，K 均值与近邻的假设更接近成立。本章没有做这一步，原因是标准化已经部分起到了这个作用，且白化会放大那些方差极小的方向，而小方差方向

往往正是噪声。**白化在小方差方向上等同于放大噪声**，这一点与 25.3.1 节讲的病态求逆是同一个机制。

若非线性结构明显，可以换成核主成分或流形学习。核主成分把样本映到高维再取主成分，等价于在原空间里找弯曲的低维面；t-分布随机近邻嵌入与均匀流形近似则专门为可视化设计，它们能保住局部邻域关系，展开“卷起来”的数据。两种方法在诊断报告里都很有用，但都有同一个陷阱：**图上的簇间距没有定量意义**。t-分布随机近邻嵌入的坐标轴不标刻度，两点之间的远近只反映局部的相对关系，不能拿来算马氏距离或画判别边界。看结构可以，做判决不行。

25.2.7 评价指标

分类器的评价不能只看准确率。设混淆矩阵元素 n_{ij} 表示真实为 i、判为 j 的样本数，则

- 准确率 $=\sum_{i} n_{ii} / \sum_{ij} n_{ij}$，类别均衡时可用，不均衡时有欺骗性；
- 召回率 $=n_{ii}/\sum_{j} n_{ij}$，分母是第 i 类的真实样本数；
- 精确率 $=n_{ii}/\sum_{j} n_{ji}$，分母是判为第 i 类的样本数；
- 误报率与漏报率分别是“正常判成故障”与“故障判成正常”的比例，两者代价通常差一个数量级以上。

聚类没有标签，评价要用外部指标或内部指标。外部指标用调整兰德指数

$$\mathrm{ARI}=\frac{\sum_{ij}\binom{n_{ij}}{2}-\mathbb{E}\left[\sum_{ij}\binom{n_{ij}}{2}\right]}{\frac{1}{2}\left[\sum_{i}\binom{a_i}{2}+\sum_{j}\binom{b_j}{2}\right]-\mathbb{E}\left[\sum_{ij}\binom{n_{ij}}{2}\right]} \tag{25-19}$$

其中 a_i、b_j 是行列和。它对随机划分的期望为 0，完全一致时为 1，因此比“准确率”更适合比较不同簇数的结果。内部指标常用轮廓系数，衡量样本与本簇、与最近邻簇的平均距离之比，取值在 -1 到 1 之间。

开放集问题还要用接受者工作特征曲线：横轴是未知类被误收的比例，纵轴是已知类被正确接受的比例，曲线下面积（AUC）越接近 1 越好。等错误率是曲线上“误收比例等于拒识比例”那一点的取值，工程上比 AUC 更直观，因为它直接对应一个工作点。

多类指标有两种平均方式，选错了会得出相反结论。宏平均先算每一类的指标再取算术平均，微平均把各类的计数混在一起再算。类别均衡时两者相同；不均衡时宏平均会被少数类拉低，微平均被多数类主导。**故障诊断应该用宏平均**，因为少数类恰恰是要抓的那一类。本章五类均衡，所以表 25-3 的准确率两种算法一致，但这不代表换到现场还能一致。

指标与工作点要分开说。AUC 衡量的是“排序能力”，与阈值无关；准确率、召回率、误报率都依赖于具体阈值。报告时若只给 AUC，运维人员无法据此设阈值；若只给某个阈值下的召回率，又不知道换个阈值会怎样。工程上的做法是给一条曲线加一个选定工作点，并写明“选这个点是因为误报率不超过每月两次”。本章表 25-5 给的就是两个点：AUC 与阈值 0.95 处的接受率。

代价敏感的评价还要引入期望损失。给定代价矩阵 c_{ij}（真实为 i、判为 j 的代价），期望损失是 $\sum_{ij} c_{ij} n_{ij}/\sum_{ij} n_{ij}$。它把“错在哪一对上”也算了进去，比准确率更贴近现场。本章没有算这个量，因为五类故障的代价没有统一定义；但表 25-6 之后会看到，量程漂移造成的损失几乎全部落在“增益退化”与“结垢”之间，这两类的处置成本差别很大，实际损失会比准确率的跌幅更严重。

最后一点关于置信区间。1000 个样本、准确率 95% 时，标准误约为 $\sqrt{0.95\times0.05/1000}\approx 0.7$ 个百分点，95% 置信区间宽约正负 1.4 个百分点。这意味着本章线性支持向量机与核支持向量机之间 1.3 个百分点的差距处在噪声边缘，不能当作“核方法更好”的硬证据。**报准确率时要带上区间**，否则小数点后的第二位会被误读成精度。

25.3 推导与性质

25.3.1 判别方向从哪里来

式 (25-11) 给出的方向 $\mathbf{w}=\Sigma^{-1}(\boldsymbol{\mu}_1-\boldsymbol{\mu}_2)$ 值得再看一眼。它由两部分组成：$\boldsymbol{\mu}_1-\boldsymbol{\mu}_2$ 是“两类中心往哪边差”，Σ^{-1} 是“往哪个方向量才准”。如果某个特征噪声很大（方差大），Σ^{-1} 会自动压低它的权重；如果两个特征高度相关，Σ^{-1} 会把权重分给它们的差而不是和。这与第 18 章加权最小二乘里 $(\Phi^{\mathsf{T}}\Phi)^{-1}$ 的作用完全同构：都是“按信息的多少分配权重”。

多类情形下，Fisher 判据把这个方向推广成 $m-1$ 个。以本章数据为例，五类共 4 个判别方向，前两个方向的判别力最强，后两个方向贡献很小。把数据投影到这两个方向上做分类，理论上的上限低于用全部 13 维——这不是估计误差造成的，是维数本身不够。实验 B 里线性判别在 13 维上达到 92.70%，如果只投影到二维再判别，准确率会明显下降。**“把高维数据画到平面上看”是可视化手段，不是分类手段**，这一点在汇报时最容易被混淆。

还要注意一个细节：判别方向依赖 Σ，而 Σ 是从数据估的。样本少、维数高时，Σ 的估计误差会让 Σ^{-1} 放大噪声方向——这正是第 7 章讲过的病态求逆。本章 1000 段数据估 13 维协方差，样本协方差的条件数不大，所以问题不突出；若每类只有 20 段，同样的方法就会出现明显退化。

25.3.2 可分性的定量尺度

两类等协方差高斯情形下，贝叶斯错误率有闭式表达。记马氏距离

$$d_M = \sqrt{(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^{\mathsf{T}} \Sigma^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)} \tag{25-20}$$

则最小错误率为

$$P_e = \Phi\!\left(-\frac{d_M}{2}\right) \tag{25-21}$$

其中 Φ 是标准正态分布函数。$d_M = 2$ 时 $P_e = 15.9\%$，$d_M = 4$ 时 $P_e = 2.3\%$，$d_M = 6$ 时 $P_e = 0.13\%$。这个式子给出了“要多远才算分得开”的定量答案：类中心距离达到 4 倍标准差，错误率就进入百分之一量级。

本章数据的二维主成分投影里，最小中心间距 0.08、类内标准差 1.39，比值 0.06，按式 (25-21) 错误率接近 50%——事实上 PCA 二维平面上五类完全不可分。而在 13 维空间里，五类的实际可分性由 92.70% 的准确率反推，平均错误率约 7%，折算 $d_M \approx 2.5$。**同一批数据，换个投影，可分性相差一个数量级**，这就是“特征空间”这个词的分量。

式 (25-21) 还解释了为什么增加特征有时反而变差。每增加一个特征，d_M 可能略增，但 Σ 的估计误差也增大；当新增特征主要是噪声时，d_M 的估计值被系统性高估，实际错误率反而上升。实验 C 里准确率随特征数从 2 增到 13 单调上升到 92.70%，是因为这 13 个特征都是按物理机理选的、都带信息；若把 13 个随机噪声也加进去，曲线会在某一点掉头。

多类情形没有这样干净的闭式解，但有两个可用的界。一是成对错误率之和的上界：$P_e \le \sum_{i<j} P_{ij}$，其中 P_{ij} 是把第 i、j 类单独拿出来做二分类时的错误率。本章五类共十对，若每一对的错误率都按式 (25-21) 估，加起来就是总体错误率的上界。这个界很松，但它指出了一个方向：**总体准确率的上限由最难分的那一对决定**。本章最难的一对是“增益退化”与“结垢”，它们的错分占了全部错分的 78%，把这一对处理好，总体准确率才有实质提升。

二是先验与阈值的关系。式 (25-21) 默认两类等先验、等代价，实际系统里两者都不成立。若第 i 类的先验是 10%，第 j 类是 1%，等错误率的判决点会让第 j 类几乎全被漏掉；把阈值按先验比 $\ln[P(C_i)/P(C_j)]$ 移动之后，两类的召回率才能平衡。**先验不平衡不是数据缺陷，是必须在阈值里体现的信息**，这一点与 25.2.2 节的代价敏感是同一件事的两个侧面。

还有一个容易被忽略的量：d_M 依赖协方差矩阵，而协方差矩阵在多类情形下通常取共享估计。若各类的真实协方差差别很大，共享估计会把某些类“压扁”，使这些类的 d_M 被低估。本章的“零点偏置”类沿均值方向拉得很长，它的真实协方差与其余四类不同，共享协方差对它偏保守；好在这一类离其余四类足够远，保守估计之后召回率仍达到 99.0%，是五类中第二高的。

25.3.3 后验概率为什么会饱和

25.1 节第三个反例（后验置信度拒识的 AUC 只有 0.214）值得推导一遍，因为它是部署阶段最常见的陷阱。

线性判别的判别函数是

$$\delta_k(\mathbf{x}) = \mathbf{x}^{\mathsf{T}} \Sigma^{-1} \boldsymbol{\mu}_k - \tfrac{1}{2} \boldsymbol{\mu}_k^{\mathsf{T}} \Sigma^{-1} \boldsymbol{\mu}_k + \ln P(C_k) \tag{25-22}$$

后验是 softmax：$P(C_k \mid \mathbf{x}) = e^{\delta_k} / \sum_j e^{\delta_j}$。设 $\mathbf{x} = t\mathbf{v}$ 沿某个方向远离原点，则 δ_k 随 t 线性增长，增长最快的那一类（$\mathbf{v}^{\mathsf{T}}\Sigma^{-1}\boldsymbol{\mu}_k$ 最大）与其他类的差也随 t 线性增长，于是

$$P(C_{\max} \mid t\mathbf{v}) \to 1 \qquad (t \to \infty) \tag{25-23}$$

这就是饱和：**越远离所有类中心，模型越“自信”**。判别式模型只学过“哪一类更靠近”，没学过“多远算太远”，所以它无法表达“我不认识这个样本”。本章实验里，未知类样本的马氏距离中位数约是已知类的两倍，后验置信度却更高，AUC 掉到 0.214。

修正的办法是改用生成式判据：先估每类的均值与共享协方差，再算样本到最近类中心的马氏距离

$$d(\mathbf{x}) = \min_k \sqrt{(\mathbf{x} - \boldsymbol{\mu}_k)^{\mathsf{T}} \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)} \tag{25-24}$$

距离超过阈值就拒识。距离随“远离”单调增，方向正确，所以同一批数据上 AUC 达到 1.000。**模型没变、特征没变，只把判据从“最大后验”换成“最小距离”，拒识能力从完全失效变成完全可分。** 这条经验可以直接搬到任何用 softmax 输出的分类器上：要拒识，就不要用 softmax 的最大值。

需要说明的是，AUC 达到 1.000 是因为本章的未知类（噪声增大）在特征空间里离得远，属于“容易的未知”。若未知类与某一已知类几乎重合（比如未知的轻微结垢与正常），任何判据都不可能完美，此时要看的是等错误率而不是 AUC。

距离判据的阈值怎么定？工程上有两种定法。第一种用已知类的距离分布定：在训练集上算每个样本到本类中心的马氏距离，取 95% 或 99% 分位数作为阈值，含义是“最多拒掉 5% 或 1% 的已知样本”。第二种用带标签的未知样本定：收集一批确认的未知样本，画工作特征曲线，按可接受的误收率反查阈值。第一种不需要未知样本，可以立即上线；第二种更准，但要求现场能收集到未知类，而未知类的标签往往拿不到。

本章实验采用第一种思路的变体：不设阈值，直接比较两种判据的排序能力（AUC）。这样比较是公平的，因为它只与“判据是否把未知排得更远”有关，与阈值无关。真正上线时必须落到一个数上，此时要回答的第一个问题不是“阈值取多少”，而是“误收一个未知类与拒掉一个已知类，哪个代价高”。这个问题的答案只能由现场给，不能由算法给。

马氏距离判据还有一个前提常被忽略：它假设每一类在特征空间里是单峰的椭球。若某一类内部有两个明显分开的子群（比如同一台设备在两个负荷段下的正常状态），用单一均值与协方差去描述，会把两群之间的样本算成“远”，从而误拒。**拒识判据的可靠性，取决于“已知类内部是否真的单峰”**。检查办法很便宜：对每一类单独做一次聚类或画一张主成分散点图，看有没有分成两团。本章五类都是单一工况采集的，所以这个前提成立；现场数据通常不成立，需要按负荷或按季节分层建模。

25.3.4 聚类为什么分不出五类

K 均值的迭代只有两步：给定分配 c_i，最优中心是簇内均值

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|}\sum_{i \in C_j} \mathbf{x}_i \tag{25-25}$$

给定中心，最优分配是“归到最近的均值”。两步都不会让式 (25-17) 增大，所以算法必然收敛。但收敛到的是局部极小：初始化不同，结果不同；而且没有任何机制保证簇与真实类别对齐。

对齐失败有两个来源。第一，目标函数用的是欧氏距离，它把各方向等同看待。本章特征之间存在强相关（幅值类三个特征几乎共线），欧氏距离会把“幅值方向”的差异放大三倍，于是“正常”与“噪声增大”因为幅值相近被并到一起，“增益退化”因为幅值差得多被单独分出——真实类别不是这样切的。第二，K 均值假设各簇大小相近、形状为球。本章五类在特征空间里的体积差一个数量级（偏置类沿均值方向拉得很长），球假设不成立。

结果是实验 D 的 0.251：五簇划分与真实标签只有四分之一的调整一致性，轮廓系数在簇数 3 时最高。这不是算法实现的问题，是“聚类目标”与“分类目标”不同。**聚类能回答“数据在哪里密”，不能回答“边界该放在哪里”。** 工程上的正确用法是反过来的：先用有标签数据训练分类器，再用聚类检查“有没有一批样本落在所有类的低密度区”，那批样本才是新故障的候选。

有人会提出一个折中方案：用少量标签做“半监督聚类”，比如约束某些点必须同簇、某些点必须异簇。这确实能改善对齐，但改善的上限取决于约束的数量与质量；当约束足够多、足够准时，它已经等价于分类，只是用了一个绕远的实现。**半监督不是“少用标签的分类”，而是“用标签信息改造聚类目标”**，把这句话想清楚，就不会期待它凭空解决无监督的固有困难。

还可以从信息量角度看这件事。聚类只用到 $\mathbf{x}$ 的分布，分类用到 $(\mathbf{x}, c)$ 的联合分布；后者多了一整个标签维度。信息论上，标签携带的信息量就是类别的熵：五类均衡时约 2.32 比特每样本。1000 个样本合计 2320 比特，这些信息不可能从特征分布里“免费”推出来——除非特征与类别之间恰好有确定关系。本章 13 个特征与五类之间远没有确定关系（准确率 92.70% 已经说明有 7.3% 的不确定），所以无监督方法无论如何调参都补不上这段差距。

25.3.5 参数个数与样本量

线性判别的参数个数可以直接数出来：m 个类均值共 md 个，共享协方差是对称矩阵，有 $d(d+1)/2$ 个独立元素，合计

$$p = md + \frac{d(d+1)}{2} \tag{25-26}$$

本章 $m=5$、$d=13$，得 $p = 65 + 91 = 156$。每类 200 段，总样本 1000，样本与参数之比约 6.4。这个比值偏紧：经验上每类样本数应不少于参数个数的 5 到 10 倍，即每类 800 到 1500 段。实验 C 的样本量曲线正好印证了这一点——每类 10 段时准确率 87.13%，40 段时 91.72%，160 段时 92.38%，从 40 段往后增益迅速变小，说明“够用”的门槛在每类 40 段附近，而不是上千段。

为什么 40 段就够，而参数个数是 156？因为参数不是独立起作用的：13 个特征里有几个高度相关，实际有效维数远小于 13；协方差的 91 个元素里，大部分对判别方向没有贡献。这也解释了为什么“降维”有时能救小样本：不是因为降维本身好，而是因为它把无效参数去掉了。

反过来说，核方法在小样本上更危险。高斯核 SVM 的模型复杂度由支持向量个数决定，样本少时几乎每个样本都成为支持向量，模型等于“记住训练集”，验证集上立刻塌陷。本章 1000 段数据下核 SVM 达到 95.10%，比线性 SVM 的 93.80% 高 1.3 个百分点；若把每类样本减到 20 段，这个优势很可能反转。**分类器的选择要与样本量匹配，而不是与排行榜匹配。**

样本量不足时，除了换简单模型，还有一条路是给协方差加约束。最常用的是收缩估计：把样本协方差朝对角阵拉一把，即取 $\hat{\Sigma}_\lambda = (1-\lambda)\hat{\Sigma} + \lambda \operatorname{diag}(\hat{\Sigma})$，其中 $\lambda \in [0,1]$ 由交叉验证选。$\lambda = 0$ 就是普通样本协方差，$\lambda = 1$ 直接把所有相关性抹掉、退化成朴素贝叶斯式的独立假设。中间值等于在“相信数据”与“相信先验”之间取折中，这正是第 24 章正则化辨识里岭参数的作用。本章 1000 段数据足以估计 91 个协方差元素，所以 λ 取小值即可；每类 20 段时，λ 应当明显大于零，否则求逆会被噪声主导。

还有一条更简单但常被忽略的做法：先把维数降下来再估协方差。Fisher 分数筛到 4 个特征，协方差只有 10 个独立元素，每类 20 段也够估。这正是实验 C 里 $k = 4$ 时准确率跳到 87.70% 的原因之一——降维不只去掉了噪声特征，也把参数个数从 156 压到了 30（均值 20 个加协方差 10 个）。**参数个数与特征数不是线性关系，而是平方关系**，这一点在小样本时比“信息量”更重要。

25.3.6 特征选择会带来乐观偏差

实验 C 里有一个容易被忽略的细节：特征是按 Fisher 分数排序后取前 k 个的。如果这个排序在全部数据上做，再用交叉验证评估，准确率会被系统性高估——因为验证折的信息已经通过“选特征”这一步泄漏进了训练。

本章的代码把排序放在每一折的训练集内部：外层五折划分，内层只用训练折算 Fisher 分数、选前 k 个、拟合判别器，再在验证折上打分。这样得到的 92.70% 是无偏的。有意思的是，本章数据上“折内选”与“全数据选”给出了完全相同的三个数（87.70%、91.20%、92.70%），因为特征排序在五折之间高度稳定，泄漏无从发生。**但这不能当作通用结论**：特征数接近或超过样本数时，全数据选特征会把验证折的噪声当成信号，评估结果可以被显著抬高。

同样的道理适用于标准化、去均值、归一化、PCA：凡是“用了数据统计量”的步骤，都必须在训练折内部完成。这条规则在文献里叫“避免数据泄漏”，在工程上就是一句话：**上线时拿不到的量，训练时也不能用**。

泄漏还有第二种形态，比标准化更隐蔽：用交叉验证选超参数，再用同一个交叉验证的数字报告性能。选超参数时，你在几十个候选里挑了验证集上最好的那个，这个“最好”里含有运气成分；用同一个划分报告，等于把运气当成了能力。正确做法是嵌套交叉验证：外层划分只用于评估，内层划分用于选超参数，两层的数据互不重叠。代价是计算量乘以候选数，收益是报告的数字真正无偏。

偏差有多大，取决于候选有多少、样本有多少。候选越多、验证集越小，取最大值带来的乐观偏差越大，这与特征选择的机制完全相同：都是在同一批数据上做多次比较，然后取最大值。本章实验 B 只比较五个分类器、每个只取一组默认参数，所以这个偏差很小；若把核宽度与惩罚系数各扫十个值，再报最好的那个，数字就会好看而不可信。

判断自己有没有踩这个坑，有一个简单办法：把选好的模型拿到一批从未参与任何选择的数据上（比如下个月的新数据）再测一次。若准确率明显下降，说明前面的数字被选择过程抬高了；若基本不变，说明偏差可以接受。这个办法成本很低，但要求你**在训练阶段就留出一段“绝不碰”的数据**，而不是等到上线才发现没有对照。

25.3.7 量程漂移为什么让边界失配

传感器量程整体偏 s 倍（本章 $s = 1.15$），输出的幅值类特征随之乘 s，形状类特征不变。标准化之后，第 j 个幅值类特征的取值从 z_j 变成

$$z_j' = \frac{s x_j - \mu_j}{\sigma_j} = z_j + (s-1)\frac{x_j}{\sigma_j} \tag{25-27}$$

偏移量与“该特征本身的标准化值”成正比。记幅值类特征的下标集合为 $\mathcal{A}$，判别函数 $\mathbf{w}^{\mathsf{T}}\mathbf{z} + b$ 因此多出一项

$$\Delta = \sum_{j \in \mathcal{A}} (s-1)\, w_j \, \frac{x_j}{\sigma_j} \tag{25-28}$$

只有当所有幅值类特征的权重 w_j 都为零时，Δ 才恒为零。也就是说，**只要判别边界用到了幅值类特征，量程漂移就一定造成失配，且失配量随漂移幅度线性增长**。

本章实验 F 的结果与此一致：全特征模型从 92.90% 掉到 82.80%，十个百分点的损失全部来自那几个幅值特征上的权重。只用形状特征（8 个）时，漂移前后准确率完全相同（57.70%），因为式 (25-28) 里的求和为空。但形状特征本身的判别力只有 57.70%，比全特征低三十五个百分点。

这个矛盾没有免费的解法，只有三条工程出路：定期用标准件校准量程（把 s 压回 1）；在线估计 s 并做补偿（实验 F 里用中位均方根之比估出 $\hat{s} = 1.150$，校正后恢复到 92.30%）；或者接受精度损失，只用形状特征换取免维护。三条出路的代价依次递增，选择取决于现场能不能停机校准。

这三种漂移在数学形式上是不同的，处理手段也不同。乘性漂移让幅值类特征整体乘 s，用中位均方根之比就能估准；加性漂移让所有特征加一个常数偏置，要用段均值的中位数去估，而且只有段均值这一个特征能被干净地校正，其余特征受影响很小；混合漂移两者都有，需要先估偏置再估增益，顺序反了会互相污染。还有一类是“频段相关”的漂移，比如抗混叠滤波器老化导致高频衰减，此时幅值类特征在高频段被压低、低频段不变，单一系数 s 校正不了，只能回到特征层面重新设计。

判断手上是哪种漂移，有一个很直接的办法：把漂移前后的同一批工况数据拿来，对每个特征算比值与差值。若各特征的比值接近常数、差值随幅值变大，是乘性；若各特征的差值接近常数、比值随幅值变小，是加性；若两类都不明显，而是集中在某几个频段相关特征上，就是频段相关。**先判断类型，再选补偿公式**，不判断就直接乘一个系数，是这一节最常见的误用。

还有一点：式 (25-27) 的推导假设标准化参数 μ_j、σ_j 保持训练时的值不变。若部署时每批数据都重新标准化，漂移会被“标准化”部分吸收，看起来准确率没掉，实际上边界已经随数据中心一起漂走了。这种“自我掩盖”比直接掉点更危险，因为它让监控指标失真。**标准化参数必须冻结在模型文件里**，这一条与 25.4.2 节是同一个要求。

25.4 离散对应与数值实现

25.4.1 特征计算的数值细节

同一条公式，实现细节不同，算出来的特征值可能差百分之几，足以让判别边界挪位。本章的实现有五处需要交代。

第一处是窗长与频率分辨率。采样率 2 Hz、Welch 窗长 128 点，频率分辨率 $\Delta f = 2/128 = 0.0156$ Hz，谱线共 65 条。低频占比定义为 $f < 0.03$ Hz 的能量，这个区间只含 0 号与 1 号两条谱线（0 与 0.0156 Hz），其中 0 号是直流，去均值后接近零。**也就是说，这个特征实际上只由一条谱线决定，它的稳定性取决于这一段数据的低频相位，而不是统计平均。** 若把窗长改成 64 点，分辨率变成 0.031 Hz，低频占比就只剩直流一条线，特征含义完全变了。工程上必须在训练与部署时用同一个窗长，并且把窗长写进模型文件。

第二处是去均值的时机。计算均方根、峭度、自相关之前都要去均值，否则直流分量会污染这些量；但“段均值”本身要保留为一个特征。本章的做法是先算均值存起来，再对去均值后的序列算其余特征。顺序反了（先整体归一化再去均值）就会把偏置类故障的信息丢掉。

第三处是单位与量纲。原始特征里，均方根的单位是伏，频率的单位是赫兹，熵是无量纲，混在一起无法比较。标准化的作用就是消掉量纲，让 Fisher 分数、欧氏距离、协方差这些运算有意义。不做标准化的后果在 K 均值上最明显：幅值类特征的数值范围是频率类的一百倍，聚类结果会被幅值方向完全支配。

第四处是数值下溢。谱熵里要算 $p_f \ln p_f$，当某条谱线功率接近零时 p_f 可能是 10^{-20} 量级，$\ln p_f$ 约 -46，乘积仍是零附近，但要防止 $p_f = 0$ 导致 $-\infty$。本章在 p_f 上加 10^{-18} 再取对数，这是工程上的常规做法，代价是熵的绝对值有微小偏差，但对判别没有影响。

第五处是段长。256 拍对应 128 s，覆盖约 16 个激励周期。段太短，谱估计的方差大；段太长，故障发生的时刻被平均掉。本章的故障是持续性的（结垢、增益退化都是慢变），所以段可以取长；若是突发性故障（卡滞、跳变），段长要缩短到秒级，特征也要换成峰值、斜率这类瞬态量。**段长由故障的时间尺度决定，不由采样率决定。**

除这五处之外，还有两个细节会在联调阶段跳出来。其一是加窗。Welch 法默认加汉宁窗，窗函数会压低信号两端的权重，使幅值类特征的估计略偏小；本章所有数据用同一种窗，偏差是系统性的，被标准化吸收掉。若训练用矩形窗、部署用汉宁窗，幅值类特征会整体差几个百分点，效果等同于一次量程漂移。**窗函数与重叠率要和窗长一起写进模型文件。**

其二是频谱泄漏与谱线位置。功率谱的频点落在 Δf 的整数倍上，而真实信号的主峰往往不在频点上，能量会漏到相邻几条线。本章的谱质心与低频占比都依赖谱线位置，若采样率有千分之一的偏差，谱线会整体挪动不到半个频点，影响可以忽略；偏差到百分之一，谱质心就会稳定偏移。工程上要么用高精度晶振，要么在特征里只用对谱线位置不敏感的量（如频段能量和），不要用单根谱线的峰值。

这两条合起来说明一件事：**特征提取的每个参数都是模型的一部分**，不是“预处理的无害细节”。把它们写进配置文件、在部署时逐项核对，比事后追查“为什么现场比实验室差五个点”要省力得多。

25.4.2 标准化与数据泄漏

标准化的公式是 $z = (x - \mu)/\sigma$，问题在于 μ 与 σ 从哪来。正确做法是只用训练集算，然后把这组 μ、σ 存进模型文件，部署时原样使用；错误做法是每次拿到新数据就重新算一遍。

后者的危害有两层。浅层是“训练与部署不一致”：训练时用的是全数据统计量，部署时用的是当前批次的统计量，两套坐标系不同，边界自然错位。深层是“泄漏”：若在交叉验证之前就用全数据算 μ、σ，验证折的均值方差信息已经进入了训练，评估结果偏乐观。本章所有实验都在每一折内部重算 μ、σ，所以 92.70%、95.10% 这些数字是无偏的。

同样的规则适用于特征选择、PCA、聚类中心、阈值。判断标准很简单：**这个量在部署时能不能只用手头这一段数据算出来？** 能，就可以在线算；不能，就必须离线固定。

25.4.3 交叉验证与分层

本章用五折分层交叉验证：把每类 200 段随机分成五份，每份 40 段，保证每一折的类别比例与全集一致。不分层的普通五折在类别均衡时差别不大，但在故障诊断里几乎总是错的——故障样本往往只占 5%，随机划分可能让某一折一个故障样本都没有，评估结果失去意义。

折数选 5 还是 10，取决于样本量。每类 200 段时五折足够，每折训练 160 段、验证 40 段；每类只有 20 段时应该用留一法，否则每折训练样本太少。本章实验 C 的样本量曲线在每类 10 段时用 80% 训练、20% 验证，重复 8 次取平均，就是为了在小样本端降低单次划分的随机性。

还有一点常被忽略：交叉验证只能评估“在同类数据上的泛化”，不能评估“在新工况上的泛化”。本章数据里五类的采集条件完全相同（同一激励、同一段长、同一采样率），所以交叉验证给出的 92.70% 是乐观的。真正的泛化能力要靠实验 F 那种“训练与测试条件不同”的划分来评估，这也叫“按工况划分”而不是“按样本划分”。

25.4.4 矩阵计算的三个坑

第一个坑是协方差求逆。13 维协方差在 1000 个样本下条件数约几十，直接求逆没问题；若特征高度共线（比如同时放了均方根、峰峰值、方差），条件数会到 10^6 以上，求逆结果被舍入误差主导。本章的应对是先用基于奇异值分解的伪逆，它会把小奇异值截断，等价于自动正则化。更稳妥的做法是加一点岭项 $\Sigma + \lambda I$，λ 取 10^{-3} 量级。

第二个坑是 LDA 的广义特征值。直接对 $S_w^{-1}S_b$ 做特征分解，在 S_w 接近奇异时会出问题；标准做法是先做 Cholesky 分解 $S_w = LL^{\mathsf{T}}$，令 $\mathbf{u} = L^{\mathsf{T}}\mathbf{w}$，把问题化成对称矩阵 $L^{-1}S_bL^{-\mathsf{T}}$ 的普通特征值问题。这样数值上稳定得多，也是所有成熟库的实现方式。

第三个坑是核矩阵的规模。1000 个样本的核矩阵是 1000×1000，内存 8 MB，可以接受；10 万个样本就是 10^{10} 个元素，80 GB，直接爆掉。工程上超过一万个样本就不该用核方法，或者要用随机傅里叶特征、Nyström 近似这类降维手段。**核方法是“小样本高维”的工具，不是“大数据”的工具**，这一点与它在教科书里的形象相反。

25.4.5 类别不平衡与代价敏感

本章五类各 200 段，是人为配平的。现场不会这样：正常工况占 95% 以上，各类故障加起来不到 5%，某些故障一年只出现几次。在这种分布上训练，任何分类器都会倾向于判“正常”——把所有样本都判正常，准确率就有 95%，但漏报率 100%。

处理办法有三条，按代价从低到高排列。第一条是重采样：对少数类过采样（复制或加噪生成），对多数类欠采样。过采样的风险是把同一段数据重复多次导致过拟合，加噪生成更稳妥，但噪声幅度不好定。第二条是加权：把少数类的错分代价调高，在判别函数里体现为 C_k 的调整，等价于移动判决阈值。第三条是改评价指标：不看准确率，看每类的召回率与误报率，或者用代价矩阵算期望损失。

第三条最容易被忽视，也最重要。**在不平衡问题里，准确率不是指标，是陷阱。** 工程上的常见做法是给每一类单独定阈值：正常类要求召回率 99.9%（宁可误报），故障类要求误报率低于 1%（不能天天报警），两个要求在同一个判决边界上无法同时满足，只能靠“分级报警”——先粗判有无异常，再细分故障类型。

本章的开放集实验其实就是这个思路的极端版：先判“是不是已知类”，再判“是哪一类”。两级判决比一级判决多一次拒识机会，代价是多一个阈值要维护。

25.4.6 部署：从离线模型到在线判据

离线训练完成的模型，上线时要变成一段能在控制器或边缘设备上跑的代码。这一步有四件事必须做对。

第一件是特征缓冲。在线系统是逐拍进来的，特征需要攒够 256 拍才能算，所以要有环形缓冲区。缓冲区长度必须与训练时一致，且要考虑采样抖动——若采样周期有 ±5% 的抖动，Welch 的频率轴会偏，谱质心跟着偏。稳妥的做法是用时间戳重采样到固定周期再算特征。

第二件是计算量的分配。13 个特征里，均方根、峭度、过零率都是 $O(N)$ 的简单累加；自相关要到 100 个滞后，是 $O(N \cdot L)$；Welch 要做一次 FFT，$O(N \log N)$；互谱还要再来一次。在 1 kHz 采样、256 点窗的设备上，全部特征算一遍约几十微秒，可以接受；若采样率到 10 kHz、窗长 4096，就要考虑降采样或只算子集。**特征的计算预算要按最坏情况留三倍余量**，因为诊断任务往往与实时控制共享 CPU。

第三件是模型文件的组织。一个可部署的模型至少包含：特征名与顺序、标准化用的 μ 与 σ、判别器系数、类别名、拒识阈值、窗长与采样率、训练数据的时间戳与版本号。少一项，半年后就没人能复现这个模型是怎么来的。

第四件是更新策略。模型不能在线自学，否则一次误报就会把正确样本带偏。常见做法是“影子模式”：新模型与旧模型并行运行，只记录不动作，积累两周数据后比较两者的一致性，一致率高于 95% 才切换。切换时保留旧模型至少一个季度，用于回滚。

把模型文件该有的字段列成一张清单，交付时逐项核对，能省掉大量返工。第一组是特征定义：特征名、计算顺序、每个特征的窗长与重叠率、采样率、去均值的时机。第二组是预处理：μ 与 σ 向量、是否做了幅值校正、校正系数的估计方法。第三组是判别器：类型（线性还是核）、系数或支持向量、类别名与顺序、先验、阈值。第四组是元信息：训练数据的时间范围与批次号、训练日期、作者、验证集上的三项指标（准确率、最差类召回、漂移后的准确率）。第五组是回滚信息：上一版模型的标识、切换日期、切换后的监控指标。

这五组里，第一组与第二组最容易被漏，因为它们在训练脚本里是“隐含”的。写完脚本就把参数写死在代码里，交付时只存一个权重文件，半年后没人知道当时用的窗长是 128 还是 256。**模型文件要能脱离训练脚本独立运行**，这是一条硬要求：拿一个没参与过训练的人，

只给模型文件与原始数据，他应该能复现出同样的判决。

在线判决的流程可以用一句话概括：攒够一段数据，算 13 个特征，按冻结的 μ、σ 标准化，算判别函数，取最大类，同时算到最近类中心的马氏距离，距离超阈值就输出“未知”。这五步里，前三步的耗时占九成，判决本身可以忽略。因此优化的重点在特征计算，不在分类器——这与 25.5.1 节“八成时间花在特征上”的结论一致。

最后是异常处理。在线系统一定会遇到缺数据、时间戳跳变、传感器饱和这些情况，此时不能硬算特征。稳妥的做法是给出“数据不足”或“传感器异常”的显式状态，而不是把坏数据算成一个特征向量再判决——后者会产生一个看似正常、实则无意义的类别输出，运维人员无法分辨。**拒识不仅要对“不认识的样本”生效，也要对“不可信的数据”生效。**

25.4.7 一次完整的识别流程

把前面几节的内容串成一条可执行的流程，共十步。

1. 明确判什么：列出要区分的类别，以及每类的物理机理与时间尺度；
2. 定数据方案：每类采集多少段、段长多少、激励是否可控、是否有标签；
3. 提候选特征：按机理列出时域、频域、时频域的候选，宁多勿少；
4. 划分数据：按工况或按时间划分训练与测试，绝不按样本随机划分后声称“泛化”；
5. 折内标准化与选特征：所有统计量只在训练折内计算；
6. 扫特征数与分类器：固定一个扫另一个，得到两条曲线；
7. 看混淆矩阵：定位错分发生在哪两类之间，回到第 3 步补特征；
8. 验开放集：留出一类不训练，评估拒识的 AUC 与等错误率；
9. 验漂移：人为把量程、噪声、工况偏移一档，看准确率掉多少；
10. 打包上线：模型文件、阈值、影子模式、回滚方案一起交付。

十步里，第 4、7、8、9 步最容易被跳过，也最容易在上线后暴露问题。第 7 步尤其重要：总体准确率 92.70% 听起来不错，但若错分的 7.3% 全部集中在“结垢”与“增益退化”之间，而这两类的处置措施完全不同（一个要清洗，一个要换执行器），这个模型在现场就没有用。**混淆矩阵不是结果展示，是下一步工作的输入。**

25.5 工程含义与适用边界

25.5.1 特征工程的优先级

模式识别项目里，八成的时间花在特征上，两成花在分类器上。这个比例不是偏好，是被数据逼出来的：本章五种分类器在同样 13 个特征上的准确率从 84.40% 到 95.10%，跨度 10.7 个百分点；而特征从 2 个加到 13 个，准确率从 60.80% 涨到 92.70%，跨度 31.9 个百分点。**特征的收益是分类器的三倍。**

按收益排序，特征工程有三条优先级。

第一优先是物理可解释的机理特征。本章的“低频增益”就是典型：它直接对应对象的直流增益，增益退化时下降、结垢时不动，语义清晰。这类特征的好处不只是判别力，还在于失效时可解释——模型说“这是增益退化”，工程师能立刻去查执行器，而不是面对一个黑箱系数。

第二优先是统计特征，比如均方根、峭度、谱熵。它们不指向具体机理，但计算便宜、对多种异常都敏感，适合做“粗筛”。本章里峭度的判别力不强，但它在噪声增大这一类上有用，属于“便宜的多面手”。

第三优先是学习得到的特征（自编码器、卷积网络）。它们在第 26、27 章展开，本章不涉及。需要提前说明的是：**学习特征在小样本上通常打不过机理特征**，因为它需要大量数据才能学到“什么是有用的”，而机理特征已经把领域知识编码进去了。

三条优先级对应三种风险：机理特征怕“机理想错”，统计特征怕“换个工况就失效”，学习特征怕“训练集没覆盖”。工程上常见的稳妥做法是三者混用，用机理特征保底，统计特征补漏，学习特征在数据充足时加分。

25.5.2 七项验收

一个诊断模型能不能上线，不能只看准确率。表 25-2 给出七项验收，每一项都有明确的判据与本例的实测值。

表 25-2　诊断模型的七项验收

项	判据	本例外实测
总体准确率	五折分层交叉验证，不低于 90%	92.70%（线性判别）、95.10%（核支持向量机）
最差类召回率	每一类都看，不能只看平均	88.0%（增益退化类），五类中最差
特征数	不超过样本数的十分之一，留余量	13 个特征、1000 段
开放集	拒识 AUC 高于 0.9，等错误率低于 10%	后验判据 0.214，距离判据 1.000
漂移	量程偏 15% 后准确率下降不超过 5 个百分点	下降 10.1 个百分点，需校准
计算量	单段特征加判决不超过控制周期的十分之一	需按目标硬件实测
可解释	每个特征对应一个物理量，判错能追因	13 个特征全部有物理含义

七项里，前三项是“能不能用”，中间两项是“敢不敢用”，最后两项是“能不能维护”。工程上最常见的偏差是只做第一项：拿一个 95% 的准确率交差，剩下的六项一律没测。上线三个月后出现误报，回头查才发现是量程漂移，而漂移这件事在验收表里本来是有的。

表 25-2 第五行值得单独说。本章量程偏 15% 导致准确率下降 10.1 个百分点，不满足“不超过 5 个百分点”的判据。处置有两条：要么加上在线量程校正（实验 F 的 $\hat{s}$ 补偿，恢复到 92.30%），要么把判据放宽到 12 个百分点并在运维规程里写明“每季度校准一次”。两条都可以，但必须选一条写进文档，不能默认它不会发生。

25.5.3 判别式与生成式的分工

本章反复出现一对概念：判别式模型直接学边界（线性判别、支持向量机、近邻），生成式模型先学每类的分布再比（高斯贝叶斯、马氏距离）。两者的分工可以概括成三句话。

第一句：**要分类，用判别式。** 判别式不估计类条件密度，把全部建模能力集中用在边界上，同样样本量下通常更准。本章线性判别 92.70%、核支持向量机 95.10%，都高于高斯贝叶斯的 84.40%。

第二句：**要拒识，用生成式。** 判别式的输出是相对的（哪一类更可能），不含“绝对像不像”的信息；生成式有每类的中心与协方差，可以算绝对距离。实验 E 里两者 AUC 相差 0.786，就是这句话的定量版本。

第三句：**要解释，用生成式。** 判别式只能说“判成第三类”，生成式可以说“到第三类中心的距离是 2.1，到第二类是 2.6，两者接近所以置信度低”。现场排故时，后一种信息有用得多。

工程上的常见组合是“判别式分类 + 生成式拒识”：用支持向量机判类别，用马氏距离判可信度。本章实验 E 的对照就是这个组合的简化版，代价是要维护两套参数（支持向量与类中心协方差），收益是既不牺牲准确率，又能拒识。

25.5.4 聚类在诊断中的真实地位

25.3 节已经说明聚类分不出五类（ARI 0.251）。这不意味着聚类没有用，而是它的用途不同。

聚类的第一个用途是发现新类。现场总会出现训练集里没有的故障，此时分类器会把它硬塞进某一已知类，而且往往很“自信”（25.3.3 节的饱和）。若同时跑一个聚类，发现“最近三个月有 3% 的样本聚成一个新簇，且离所有已知簇都远”，这就是新故障的候选。**聚类是发现工具，分类是判决工具**，两者并行不冲突。

第二个用途是数据清洗。标注数据里常有错标，错标样本往往在特征空间里孤零零地待着。用 DBSCAN 把噪声点挑出来人工复核，比逐条看数据省力得多。本章的 DBSCAN 在 $\varepsilon = 2.0$ 时把 5.7% 的样本标为噪声，这个比例与人工复核的工作量大致相当。

第三个用途是给分类器提供初始结构。若聚类在某一类内部稳定地分出两个子簇，说明这一类可能该拆成两类，或者该补一个特征把它们分开。这是“用无监督指导有监督”，不是“用无监督代替有监督”。

三条用途有一个共同点：聚类的输出是人看的，不是直接进控制器的。把聚类结果直接当报警用，是本章最不推荐的做法——它既没有阈值语义，也没有类别语义。

25.5.5 漂移与维护

诊断模型上线后性能会缓慢下降，原因有三类：传感器漂移（量程、零点）、工况漂移（负荷、原料、环境）、设备老化（磨损、结垢）。三类都表现为“特征分布整体平移”，而不是“某几个样本变难”。

维护策略分三级。第一级是监控特征分布：每周算一次各特征的均值与方差，与训练时的值比较，偏移超过两倍标准差的特征标红。这一级最便宜，能抓住大部分漂移。第二级是监控判决分布：统计各类的判决比例，若“正常”的比例从 95% 掉到 85%，说明要么真有故障，要么模型失配，两者都要查。第三级是定期重标定：每季度用标准件校准量程，每年用新数据重训一次模型，重训前必须做影子模式对比。

三级策略的成本递增，收益也递增。只做第一级，能发现漂移但不能纠正；做到第三级，模型能跟上设备老化，但需要停机与人力。工程上按设备的重要性选级：关键机组做三级，辅助设备做一级。

还有一条容易被忽略：**模型版本要与数据版本绑定**。重训模型时若用了不同批次的标签，新旧模型的准确率不可直接比较。台账里要记清楚“模型 v3 用 2026 年 1 至 6 月数据训练，标签由张工复核”，否则半年后无法解释为什么 v3 比 v2 差。

三级维护各自用什么统计量，可以写得很具体。第一级看每个特征的均值与标准差，但直接看数值不直观，工程上更常用两个无量纲量：一是偏移比，即当前均值与训练均值之差除以训练标准差；二是群体稳定性指数，把训练分布与当前分布各切成十档，比较两边的占比差异，再求和。前者对整体平移敏感，后者对分布形状变化敏感，两个一起看，能区分“整体偏了”与“某一段变密了”。**偏移比超过 0.5 就该关注，超过 1.0 就该动作**，这是经验值，不是定理。

第二级看判决分布，除了各类比例，还要看“未知”的比例与平均马氏距离。未知比例突然上升，通常意味着出现了新工况或新的漂移；平均距离缓慢上升而各类比例不变，说明数据在整体远离训练分布，模型还勉强能用但正在失效。把这两个量画成时间序列，比看单点的准确率有用得多——准确率需要标签，而这两个量不需要。

第三级的具体节奏可以这样安排：每季度做一次标准件校准与特征分布复核；每年做一次完整重训，重训前用最近三个月的数据评估旧模型，重训后用同一批数据评估新模型，两者对比通过影子模式确认；每次设备大修或传感器更换后，无条件重新校准量程并复核特征分布，因为这类事件一定会改变 s 或零点。**把“设备大修”写进触发条件**，比依赖日历更可靠。

三级之外还有一件事：记录。每次校准、每次重训、每次报警都要留下时间戳与操作人。诊断系统的可信度不只来自算法，也来自“能不能查清上一次判决是在什么状态下做出的”。这一条在事故分析时往往是唯一能救命的东西。

25.5.6 与相邻章节的分工

本章只讲“从特征到类别”这条路。相邻章节各管一段，边界要划清楚。

特征的学习（自编码器、卷积网络、表征学习）在第 26、27 章；本章的特征全部由人工按机理设计，不涉及学习。分类器的深层结构（多层网络、注意力）在第 28 章；本章用的都是浅层模型，参数量在百量级。故障诊断的体系（故障树、残余寿命、维修决策）在第 36 章；本章只做“当前属于哪一类”，不做“还能用多久”。参数辨识与频域特征的关系在第 24 章；本章的低频增益特征就是那一章频域方法的简化。

划清边界的好处是知道什么时候该换工具。若特征怎么设计都分不开，可能是类别定义本身有问题（两类故障在物理上不可区分），此时换再复杂的分类器也没用；若样本量到了十万级，本章的核方法就不合适，该转到第 27 章的深度模型；若要求“提前 30 分钟预警”，那是预测问题，不属于本章的分类框架。

25.5.7 什么时候不该用模式识别

最后给一条否定清单。以下四种情形，模式识别不是合适的工具。

第一，类别边界可以由物理定律直接写出。比如“温度超过 120 度报警”，这是一条阈值，不需要训练分类器。用模式识别去做，反而把简单问题复杂化，还引入了误报率。

第二，故障样本极少且无法仿真。每类只有三五个样本时，任何分类器的泛化都无从谈起。此时应该用机理模型加残差检验（第 24 章），或者用单类方法（只学正常工况的分布，偏离就报警），而不是硬凑五类分类。

第三，要求给出定量结果。模式识别输出的是标签，若需要“结垢程度 0.37”这样的连续量，应该做回归或参数辨识，而不是把连续量离散成五档再分类——离散化会丢掉信息，还会在档位边界产生抖动。

第四，可解释性是硬约束且模型必须极简。某些安全相关场合要求“判决逻辑能被人逐条审查”，此时决策树或规则表比判别分析更合适。

否定清单不是否定本章的方法，而是提醒：**模式识别的适用条件是“类别存在、样本可得、边界难写”**。三条缺一条，就该考虑别的工具。

25.6 仿真与验证

25.6.1 实验设置与读表方式

六组实验共用一套数据：五类工况、每类 200 段、每段 256 拍、13 个特征，共 1000 个样本。数据由 `code/ch025_sim.py` 生成，随机种子固定为 20261025，所有数字都可以原样复现。

有五处设置需要先说明，否则数字会被误读。

第一处是“线性判别、支持向量机、近邻”的准确率一律指五折分层交叉验证的均值，不是训练集上的准确率。训练集准确率会高估 2 到 5 个百分点，本章不用它作结论。

第二处是“核支持向量机”的参数取 $C = 10$、高斯核宽度按特征数自动缩放。这两个值是常用默认值，没有针对本章数据调优——调优会让数字好看，但也会让“五种分类器差多少”这个比较失真。

第三处是实验 C 的特征选择在每一折内部完成。Fisher 排序只看训练折，因此曲线没有乐观偏差。本章数据上“折内选”与“全数据选”碰巧给出相同的三个数（87.70%、91.20%、92.70%），因为特征排序在五折间高度稳定；但代码仍按折内实现，以免换一套数据就失效。

第四处是实验 D 与 E 用的是同一批标准化后的特征。聚类的输入与分类的输入完全相同，差别只在有没有用标签；这样比较才公平。

第五处是实验 F 的漂移是“乘性量程漂移”，即输出整体乘 1.15，输入也同步乘 1.15（模拟同一个采集通道的增益漂移）。只乘输出会让低频增益特征保持不变，测不出真实的失配。

还有一处关于可复现性的说明。所有实验共用一个随机种子，数据生成、折划分、K 均值初始化都由它决定，因此重跑脚本能得到逐位相同的数字。这不意味着数字对种子不敏感：换一个种子，每类的段均值、粗糙度都会重新抽样，指标会随之浮动，其中聚类指标（ARI）比分类准确率敏感得多。**本章的数字用于说明量级关系，不是用于卡到小数点后两位的验收值。** 若要把某个数字写进技术协议，应当用多个种子重复，报均值与区间，本章没有做这一步。

同样地，脚本里的随机性有三个来源需要分清：数据生成（每段的噪声与激励相位）、交叉验证的折划分、聚类与核方法的初始化。前者影响“数据本身”，后两者只影响“评估与训练过程”。判断一个波动来自哪里，办法是固定后两者、只换数据种子，看指标怎么动；再固定数据、只换折划分，看指标怎么动。前者通常更大，因为每类只有 200 段。

25.6.2 实验 A：特征空间

实验 A 的问题是：13 维特征空间里，五类到底长什么样？前两个主成分能保留多少判别信息？

主成分分析的结果是：第一主成分解释 41.29% 的方差，第二主成分 20.25%，两者合计 61.54%，前三累计 73.24%。图 25-2 左图把 1000 个样本画在前两个主成分平面上，五类用不同标记区分。图上的事实很清楚：五团点大面积重叠，只有“零点偏置”那一类（图中菱形）沿第一主成分被推向一侧，其余四类混在一起。

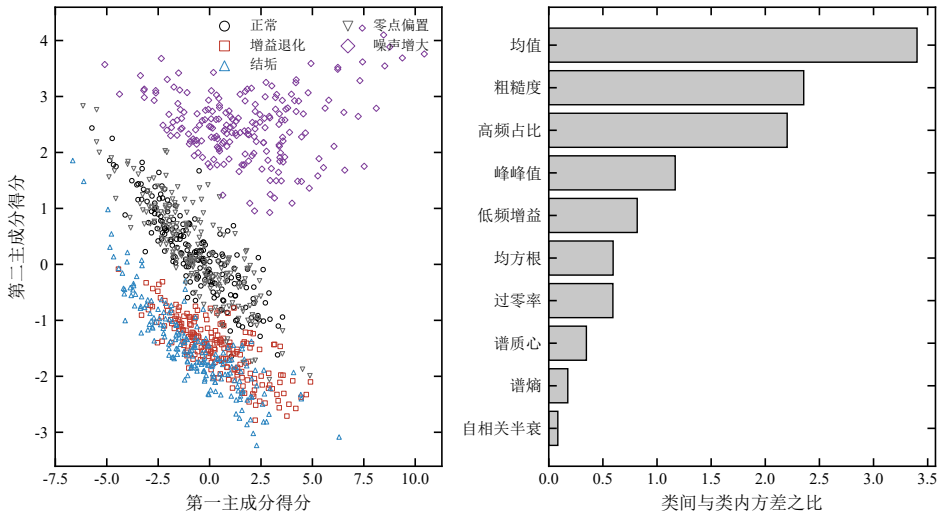


图 25-2　主成分二维投影与特征判别力排序

定量的读法是两个数：五类中心在这个平面上的最小间距是 0.08，类内平均标准差是 1.39。间距比散布小一个数量级，按式 (25-21) 折算的错误率接近 50%，也就是“随机猜”。这与实验 B 里 92.70% 的准确率形成尖锐对比——同一批数据、同一个投影方法，二维平面上分不开，13 维空间里分得开。

图 25-2 右图给出解释：按类间与类内方差之比排序，前四个特征是段均值（3.40）、差分粗糙度（2.36）、高频占比（2.20）、峰峰值（1.17）。这四个特征的判别力差距很大，第一名是第四名的近三倍；而排在第 13 位的特征，比值不到 0.1，几乎全是类内噪声。**判别信息高度集中在少数几个特征上，这正是 PCA 会丢掉它的原因——那几个特征在原始空间里的方差并不大。**

段均值排第一，看起来违背直觉：一个只有 0.25 的偏置，怎么会是最强的判别特征？原因是正常工况的段均值在零附近抖动，抖动量约 0.05（由 PRBS 去直流后的残余与噪声共同决定），0.25 的偏置相当于 5 倍抖动，信噪比很高。而其余四类的段均值都在零附近，彼此没有差别。于是“段均值”这个特征单独就能把一类与其余四类分开，Fisher 比值自然最高。这个例子说明：**Fisher 分数高不等于特征复杂，往往恰恰相反，物理上最直白的量判别力最强。**

25.6.3 实验 B：五种分类器

实验 B 的问题是：在同样的 13 个特征上，五种分类器差多少？错分发生在哪里？

表 25-3 给出五折交叉验证的结果。核支持向量机最高（95.10%），线性支持向量机次之（93.80%），线性判别 92.70%，K 近邻 87.20%，高斯贝叶斯最低（84.40%）。从最低到最高跨度 10.7 个百分点。

表 25-3　五种分类器的五折交叉验证准确率

分类器	决策形式	准确率
高斯贝叶斯	生成式，各类独立协方差	84.40%
线性判别	生成式，共享协方差	92.70%
K 近邻	非参数，局部投票	87.20%
线性支持向量机	判别式，最大间隔超平面	93.80%
核支持向量机	判别式，高斯核曲面边界	95.10%

三个差距值得解释。高斯贝叶斯比线性判别低 8.3 个百分点，原因是它给每一类单独估计协方差——五类各估一个 13×13 矩阵，每类只有 200 段，估计误差把边界弄歪了。线性判别共享一个协方差，参数少 4 倍，反而更稳。**这是“参数越少越稳”的直接证据，与第 24 章偏差—方差分解是同一件事。**

K 近邻比线性判别低 5.5 个百分点，原因是距离度量。近邻用欧氏距离，13 个特征标准化后各方向等权，而真正有判别力的只有四个特征；剩下九个特征把邻域撑大了，近邻找不到真正的“近”。若先用 Fisher 分数筛到 4 个特征再跑近邻，准确率会明显上升——这是“特征选择对非参数方法收益更大”的一个例子。

核支持向量机只比线性支持向量机高 1.3 个百分点，说明本章数据的类别边界接近线性。这个结论有实用价值：线性模型的部署成本低一个量级（存 14 个数而不是几百个支持向量），1.3 个百分点的差距在很多场合不值得。

图 25-3 右图是核支持向量机的交叉验证混淆矩阵，按行归一化。五类的召回率分别是 98.0%、88.0%、90.5%、99.0%、100%。错分集中在两类之间：真实“增益退化”被判成“结垢”的有 21 段，真实“结垢”被判成“增益退化”的有 17 段，两者合计 38 段，占全部 49 个错分样本的 78%。

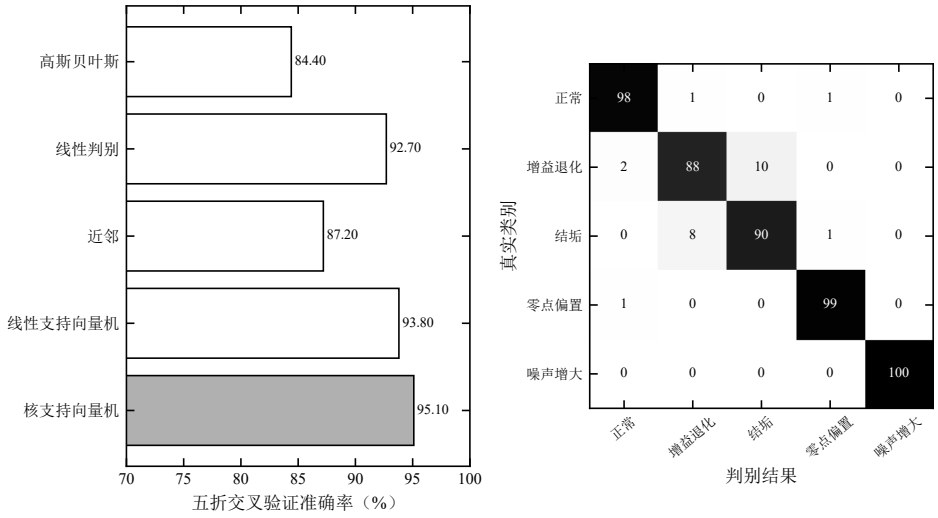


图 25-3　五种分类器的准确率与核支持向量机的混淆矩阵

这个集中不是偶然。增益退化让输出幅值整体变小，结垢让输出在同样的激励下也变小（高频段衰减更多），两者在“幅值类”特征上的表现方向相同；区分它们要靠“低频增益不变但高频衰减”这个形状差异，而形状特征的判别力本来就弱。工程上的含义很直接：**这两类故障的区分需要补一个更有针对性的特征**，比如高频段与低频段的增益比，而不是换一个更强的分类器。换分类器最多把 95.10% 提到 96% 左右，补对特征可以把它提到 99%。

25.6.4 实验 C：特征数与样本量

实验 C 问两个“要多少”的问题：要几个特征，要几段数据？

图 25-4 左图是准确率随保留特征数的变化。$k=2$ 时只有 60.80%，$k=3$ 时 63.50%，$k=4$ 时跳到 87.70%——这一步跳了 24.2 个百分点，是整条曲线最陡的一段。原因是前四个特征（段均值、粗糙度、高频占比、峰峰值）恰好覆盖了五类的主要差异：段均值分开偏置类，粗糙度与高频占比分开噪声类，峰峰值分开增益退化与结垢。$k=4$ 之后进入缓坡：$k=7$ 时 91.20%，$k=13$ 时 92.70%，九个特征只贡献 5 个百分点。

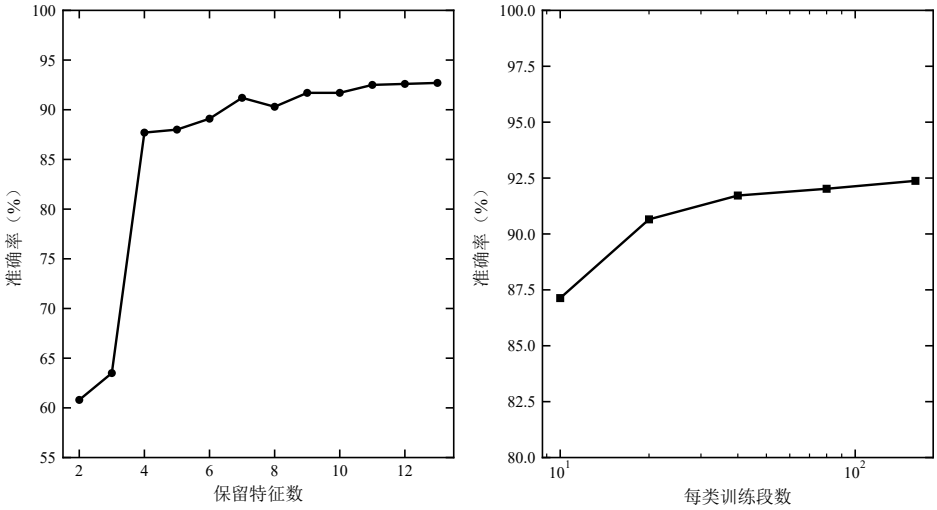


图 25-4　特征数与每类样本数对准确率的影响

这条曲线有一个实际的用法：**它告诉你哪些特征是“必需的”，哪些是“锦上添花”的。** 如果现场只能算三个特征（算力受限），准确率只有 63.50%，不够用；能算四个，就有 87.70%，勉强可用；能算七个，91.20%，已经进入可用区间。曲线在 $k=7$ 之后基本走平，说明再加特征不值得——每多一个特征就多一份计算与维护成本。

右图是准确率随每类训练段数的变化。每类 10 段时 87.13%，20 段 90.65%，40 段 91.72%，80 段 92.02%，160 段 92.38%。从 40 段到 160 段，样本翻了 4 倍，准确率只涨 0.66 个百分点。**数据的边际收益在 40 段附近就急剧下降**，这与 25.3.5 节按参数个数估算的“40 段够用”一致。

两张图合起来给出一条实用的采集建议：先按机理定 4 到 7 个特征，每类采 40 到 80 段，就能达到 90% 以上的准确率；剩下的精力应该花在“覆盖更多工况”上，而不是“再多采几百段同类数据”。本章数据只有一种激励、一种段长、一种采样率，覆盖的工况很窄——这也是实验 F 里一漂移就掉十个百分点的根本原因。

25.6.5 实验 D：聚类能不能替代分类

实验 D 的问题是：不用标签，能不能把五类分出来？

表 25-4 给出簇数从 2 到 8 的簇内平方和与轮廓系数。簇内平方和从 9726.5 单调降到 4835.3，没有明显的拐点——曲线每加一簇都平滑下降，这种“没有肘部”的形态本身就说明数据里没有清晰的球形簇结构。轮廓系数在簇数 3 时最高（0.235），簇数 5 时只有 0.209，簇数 7、8 时进一步降到 0.210 与 0.209。

表 25-4　K 均值的簇数与轮廓系数

簇数	2	3	4	5	6	7	8
簇内平方和	9726.5	7869.3	6921.5	6212.0	5641.8	5144.9	4835.3
轮廓系数	0.229	0.235	0.229	0.209	0.220	0.210	0.209

取簇数 5 与真实标签比较，调整兰德指数只有 0.251。数值的含义是：把两个划分看成两套“分组方案”，它们的重合程度只有四分之一，比随机划分好一点（随机为 0），离“一致”（1）很远。

DBSCAN 的结果同样不乐观。邻域半径 0.6 时所有点都是噪声；0.8 时出现 4 个簇但 97.6% 的点仍是噪声；1.2 时 5 个簇、噪声比 51.2%、ARI 0.080；1.5 时 4 个簇、噪声比 20.4%、ARI 0.144，是全程最高；2.0 时 3 个簇、噪声比 5.7%，ARI 掉回 0.005。**没有任何一个半径同时给出“合理的簇数、可接受的噪声比、像样的 ARI”。**

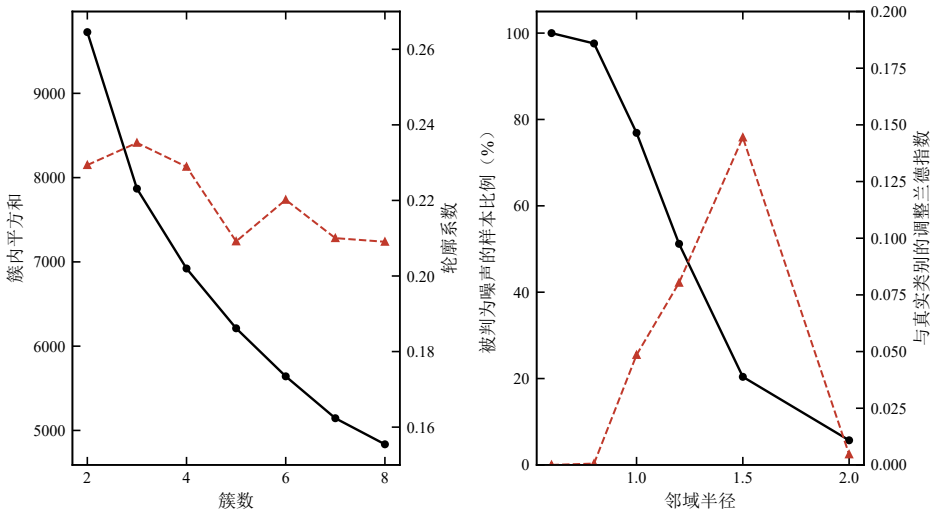


图 25-5　K 均值的簇数与轮廓系数、DBSCAN 的邻域半径扫描

图 25-5 把两组数字画在一起。左图两条曲线（实线是簇内平方和、虚线是轮廓系数）没有任何一处同时取极值，这就是“肘部法失效”的样子；右图两条曲线（实线是噪声比、虚线是 ARI）在半径 1.5 附近有一个折中，但 ARI 只有 0.144，仍然不足以支撑“自动分五类”的说法。

这两个结果不说明聚类算法写得不好，而说明问题问错了。K 均值最小化簇内距离，它对“幅值方向”特别敏感，而本章五类的差异有一部分恰恰在“形状方向”；DBSCAN 假设簇之间由低密度区隔开，而本章五类的边界是交叠的，根本没有低密度带。**数据的几何结构与算法的假设不匹配时，调参永远调不出好结果。**

工程上的用法在 25.5.4 节已经说过：聚类用来发现新类、清洗错标、检查子结构，不用来定类别数。

25.6.6 实验 E：未知类能不能被拒识

实验 E 的问题是：四类训练、第五类当未知，模型能不能说“我不认识”？

表 25-5 对比两种拒识判据。用后验置信度（softmax 最大值）时，AUC 只有 0.214，等错误率 66.38%，阈值取 0.95 时已知类接受率只有 72.25%——也就是说，为了挡住未知类，得把三成已知类也拒掉。用马氏距离时，同一批数据的 AUC 是 1.000。

表 25-5　两种拒识判据的对比（四类训练、第五类未知）

判据	AUC	等错误率	阈值 0.95 处的接受率
后验置信度	0.214	66.38%	72.25%
马氏距离	1.000	—	—

后验置信度的失败机理在 25.3.3 节推导过：未知样本离所有类中心都远，线性判别函数在远处的值更大，softmax 反而给出更高的置信度。图 25-6 左图画出了两条曲线，后验置信度那条（虚线）整条落在对角参考线的下方——它把“远”当成了“像”，方向完全反了。马氏距离那条（实线）贴着左上角，说明它把“远”正确地当成了“不像”。

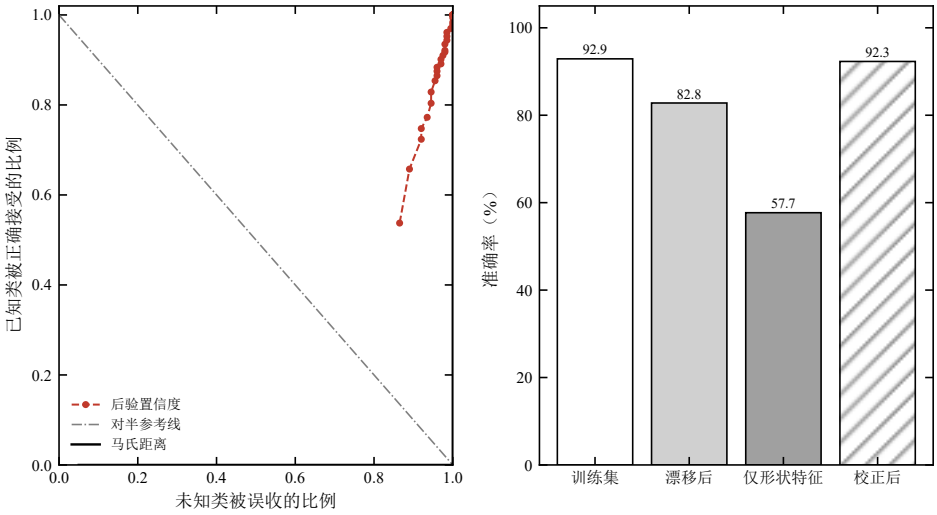


图 25-6　拒识的工作特征曲线与量程漂移下的准确率

有一个细节需要说明：AUC 1.000 是“容易的未知”给出的。本章的未知类是噪声增大，它在特征空间里离已知四类都远，距离判据几乎不可能判错。若未知类换成“轻微结垢”（T_1 只从 8 增到 10），它与已知的“正常”几乎重合，任何判据的等错误率都会升到 30% 以上。工程上要按“最难的未知”来设计阈值，而不是按“最容易的未知”验收。

25.6.7 实验 F：量程漂移之后还能用吗

实验 F 的问题是：传感器量程整体偏大 15%，模型会怎样？

表 25-6 给出四种情形。全特征模型在训练集上 92.90%，漂移后掉到 82.80%，损失 10.1 个百分点。只用形状特征（8 个）时，漂移前后都是 57.70%，完全不敏感，但水平低了 35 个百分点。用中位均方根之比估计量程系数得 $\hat{s} = 1.150$，与真值 1.15 几乎相同；把幅值类特征除以 $\hat{s}$ 之后，准确率恢复到 92.30%。

表 25-6　量程漂移 1.15 倍下的准确率

情形	训练集	漂移后
全特征（13 个）	92.90%	82.80%
仅形状特征（8 个）	57.70%	57.70%
全特征 + 量程校正（$\hat{s} = 1.150$）	—	92.30%

这张表把 25.3.7 节的推导验证了一遍。式 (25-28) 说失配量正比于幅值类特征的权重，形状特征权重为零时失配为零；实测的两个 57.70% 就是“失配为零”的极端情形。同时它也说明“对漂移免疫”不等于“好用”：免疫的代价是判别力从 92.90% 掉到 57.70%，这个模型连正常与噪声增大都分不开。

校正的效果值得多说一句。估计 $\hat{s}$ 只用了一个量：测试集与训练集的均方根中位数之比。这个量不需要标签，也不需要停机，只要两类数据的工况分布大致相同就能估准。实测 $\hat{s} = 1.150$，与真值一致到小数点后三位；校正后准确率 92.30%，比原始训练集的 92.90% 只低 0.6 个百分点。**一个参数的在线估计，就能把十个百分点的损失收回来。**这是本章里“性价比最高的工程措施”。

需要提醒的是，这个方法有一个前提：漂移必须是“乘性、全局、慢变”的。若漂移是加性的（零点偏移），$\hat{s}$ 估不出来，要用段均值中位数去估偏置；若漂移只影响部分频段（比如滤波器老化），单一系数校正不了，得回到特征层面重新设计。

本章没有做加性漂移的实验，但可以定性推断它的后果。零点偏置类本身就是加性漂移，模型在训练时见过它，所以对“加 0.25”这种幅度不敏感；偏置继续增大，段均值特征会沿单一方向持续偏离训练分布，判决仍可能稳定地落在“零点偏置”这一类上。**加性漂移的危险不在于判错类，而在于判对了却没人处理**，因为“零点偏置”这个标签本身不构成停机理由。

这也是为什么第 25.5.5 节的维护要监控特征分布而不只监控判决结果。判决比例不变、特征均值在漂，说明模型还在硬撑；等到判决比例变了，往往已经晚了几个月的量程误差。监控特征分布相当于看“输入有没有变”，监控判决分布相当于看“输出有没有变”，前者总是更早报警。

25.6.8 六组实验的横向对照

六组实验各自回答一个问题，横向看有三条共同的规律。

第一条：**收益最集中的地方是特征，不是模型。** 实验 B 换遍五种分类器，准确率跨度 10.7 个百分点；实验 C 只改特征数，跨度 31.9 个百分点。两件事的投入也不同：换分类器是改一行代码，换特征要重新做机理分析、重新采数据、重新验证。收益与投入正好相反，这是模式识别项目里最反直觉的一点。

第二条：**鲁棒性与判别力总是此消彼长。** 实验 F 里形状特征对漂移完全免疫，代价是准确率 57.70%；全特征判别力高，代价是漂移后掉十个点；量程校正把两者都拿到了，代价是要维护一个在线估计器。三条路没有一条是免费的，选择取决于现场能不能接受“每季度校一次”或“多维护一个估计器”。

第三条：**评价方式决定结论。** 同一批数据，看训练集准确率会比交叉验证高几个百分点，看交叉验证是 95.10%，看最差类召回是 88.0%，看漂移后是 82.80%，看未知类的 AUC 是 0.214 或 1.000——取决于判据。**报一个数字等于没报**，工程验收至少要给“交叉验证准确率、最差类召回、漂移后的准确率”三个数。

三条规律合起来给出一条工作次序：先做实验 C（定特征与样本量），再做实验 B（选分类器），再做实验 E（定拒识），最后做实验 F（评估维护成本）。实验 A 与 D 是辅助——A 用来看数据是否可分，D 用来检查有没有发现的子结构。这个次序与 25.4.7 节的十步流程一致，只是把“十步”压成了“四组必做实验加两组可选实验”。

25.7 小结与常见错误

本章从“从一段信号到一个类别”开始，用六组实验走完了特征、判别、无监督、开放集、漂移五个环节。这一节把结论收拢成六条，列出十种最常见的错误，给一套三级排查流程，最后说明向后续章节的迁移。

25.7.1 六条结论

第一条：**特征决定上限，分类器只决定离上限多远。** 实验 B 五种分类器的跨度是 10.7 个百分点，实验 C 特征数的跨度是 31.9 个百分点。工程上把八成时间花在特征上不是浪费，是必要的分配。

第二条：**降维保方差，不保判别。** 前两主成分保留 61.54% 的方差，二维平面上五类的中心间距只有类内散布的 6%，完全不可分；同一批数据在 13 维空间里准确率 92.70%。PCA 是可视化与去相关的工具，不是分类前的必经步骤。

第三条：**聚类不能替代分类。** 五簇划分与真实标签的调整兰德指数 0.251，轮廓系数在簇数 3 时最高。聚类回答“数据在哪里密”，分类回答“边界放在哪里”，两个问题的答案不必一致。聚类的正确用途是发现新类、清洗错标、检查子结构。

第四条：**要拒识，不能用后验概率。** 后验置信度拒识的 AUC 是 0.214，比随机猜还差；同一模型换成马氏距离，AUC 是 1.000。原因是判别函数的后验在远离类中心时饱和到 1，把“没见过”误判成“很确定”。

第五条：**样本量的门槛比想象的低，但泛化的门槛比想象的高。** 每类 40 段就达到 91.72%，160 段只有 92.38%；可是换一个量程条件，同一模型从 92.90% 掉到 82.80%。数据的价值主要不在“多”，而在“覆盖不同工况”。

第六条：**鲁棒性与判别力是一对显式权衡。** 形状特征对量程漂移完全免疫，代价是判别力从 92.90% 掉到 57.70%；用中位均方根之比估计量程系数（$\hat{s} = 1.150$）可以把准确率恢复到 92.30%。三条路——校准、补偿、降级——必须选一条写进规程。

六条里有四条（第二、三、四、六）是“否定式”的：它们说的不是“该怎么做”，而是“看起来该成功、实际不成立”的做法。这不是凑数。本章的三个先导反例、三组实验（A、D、E）与一组漂移实验，指向的都是这类“直觉正确但实测相反”的地方，而这正是模式识别在工程上最容易翻车的地方。

需要标明结论的强度。第一、三、四条属于方法性质：Fisher 判别最多 $m-1$ 维、聚类目标与分类目标不同、softmax 后验随距离饱和，这三条是数学结论，换数据也成立。第二条与第五条属于经验规律，数值随数据集变化，但“量级关系”（降维丢判别、样本边际收益递减）在类似问题上普遍成立。第六条的具体数字（10.1 个百分点、57.70%、92.30%）只对本章这套参数成立，换一个对象、换一个漂移幅度，数字会变，但“权衡存在”这一点不变。

25.7.2 十种常见错误

表 25-7 列出十种错误，按发生频率从高到低排列。第三列给出本章对应的证据。

表 25-7　模式识别的十种常见错误

序	错误做法	后果与本章证据
1	先做 PCA 再分类，认为降维总是有益	二维平面五类不可分（间距 0.08、散布 1.39），13 维准确率 92.70%
2	用训练集准确率作为最终指标	比交叉验证高几个百分点，掩盖过拟合
3	用 softmax 最大概率做拒识	AUC 0.214，比随机猜更差
4	把聚类簇数当作类别数	五簇 ARI 0.251，轮廓系数在 3 簇最高
5	全数据标准化或全数据选特征	本章侥幸无差异；高维小样本下评估显著虚高
6	每类只采十几段就上核方法	每类 10 段准确率 87.13%，核模型退化为记忆训练集
7	只报总体准确率，不看混淆矩阵	错分的 78% 集中在“增益退化”与“结垢”之间
8	不做漂移测试就上线	量程偏 15% 后准确率掉 10.1 个百分点
9	用先去均值的信号算所有特征	零点偏置类故障的信息被自己删掉
10	把诊断模型的输出直接接到控制器	误报即动作，模型一漂移就误停机

前四种是方法层面的误解，中间三种是评估层面的疏漏，后三种是工程层面的失误。三类错误的发现成本依次升高：方法误解在训练阶段就能看出来，评估疏漏要等到验收，工程失误要等到上线之后。

第 9 种错误值得单独说一句。它看起来是“信号处理的规范做法”，实际上是“把要判的信息删掉了”。本章的段均值特征之所以判别力最高（Fisher 3.40），正是因为它没有去均值。这条经验的一般形式是：**预处理步骤要与判别目标对齐，而不是与“信号好不好看”对齐。**

25.7.3 三级排查流程

模型效果不好时，按三级顺序查，不要跳级。

第一级查数据。看训练集每类多少段、有没有错标、有没有混入其他工况、标签是否与时间段对齐。本章实验里若把“增益退化”与“结垢”两类标签写反，准确率会掉到 50% 附近，而任何分类器都救不回来。数据问题占总故障原因的比例最高，也最容易查——画几张特征箱线图，异常立刻现形。

第二级查特征。看判别力排序、看特征之间的相关性、看每一类的特征分布是否重叠。若某两类的所有特征分布都重叠，说明缺一个能分开它们的量，回到机理分析补特征。本章“增益退化”与“结垢”的混淆就属于这一类，补一个“高低频增益比”比换分类器有效得多。

第三级查模型与评估。看交叉验证的划分方式、看是否泄漏、看超参数是否在验证集上调过、看类别是否不平衡。这一级最容易被误当成第一级——很多人一上来就换模型、调参数，而真正的问题在数据或特征上。**本章的数字可以当参照：换模型最多值 10.7 个百分点，换特征值 31.9 个百分点。** 顺序反了，投入产出比会很差。

三级之外还有一级“上线后”的排查，它不属于建模流程，而属于运维：查特征分布的漂移、查判决比例的变化、查量程是否需要校准。这一级的方法在 25.5.5 节给过，核心是“每周看一次特征的均值方差”，成本很低，能挡住大部分缓慢失效。

25.7.4 向后续章节的迁移

本章的方法会在三个方向被后续章节接续。

第一个方向是特征的学习。第 26 章讲机器学习的通用框架，第 27 章讲深度学习，两章都会回答“特征能不能不靠人设计”。本章的经验给它们提供了一个基准：手工的 13 个特征在 1000 段数据上达到 92.70%，学习特征要超过这个数，需要多少数据、多少算力，是可以算清楚的。**没有这个基准，就无从判断“深度模型更好”是不是真的。**

第二个方向是故障诊断的体系。本章只做“当前属于哪一类”，第 36 章会把它扩展成“有没有故障、是哪一类、还剩多少寿命、该怎么修”。本章的开放集判据（马氏距离加阈值）是那一章“异常检测”环节的直接前置；本章的混淆矩阵分析是那一章“误报与漏报权衡”的方法基础。

第三个方向是控制器的接口。诊断结果最终要影响控制：降增益、切手动、限幅、报警。第 44 章与第 106 章的 PID 整定会用到“增益退化多少就重调多少”的规则，第 58 章与第 80 章的预测控制会用到“模型失配时如何降级”。本章的贡献是给出“退化程度”的判别，而不是“退化后怎么控”；后者是那些章节的事。

三个方向有一个共同的前提：**分类结果必须带可信度。** 只给标签不给置信度的诊断，在控制回路里无法使用——控制器不知道这次判决该信几分。本章的拒识判据与距离量，就是可信度的一种表达形式；把这个量传给控制器，比传一个孤立的类别标签有用得多。

25.7.5 一页纸的落地清单

把全章内容压成一页，按项目推进的顺序排，共十条。每条后面注明对应的章节，方便回查。

1. 先写清“判什么”：类别名、物理机理、时间尺度、期望的报警方式（25.1.2）；
2. 按机理列候选特征，并记下每个特征“在什么条件下会骗人”（25.2.1）；
3. 定段长与窗长，让它们匹配故障的时间尺度，而不是匹配采样率（25.4.1）；
4. 按工况或按时间划分数据，留出一段“绝不碰”的测试集（25.4.3、25.3.6）；
5. 所有统计量在训练折内计算：标准化、选特征、降维、选超参数（25.3.6）；
6. 固定分类器扫特征数，固定特征扫分类器，两条曲线都要有（25.6.4）；
7. 看混淆矩阵，定位最难分的那一对，回去补针对性特征（25.6.3）；
8. 留一类做未知，用距离判据而非后验概率定拒识阈值（25.3.3）；
9. 人为把量程偏 15%、噪声抬一档，看准确率掉多少，并决定补偿或校准（25.3.7）；
10. 交付时带上模型字段清单、影子模式方案、三级维护周期与回滚版本（25.4.6、25.5.5）。

十条里，第 1、4、5、9 条是“不做就会在半年后返工”的四条，其余是可以补做的。若时间只够做一半，就做这四条加第 7 条：它们决定了这个模型有没有资格上线，而分类器的选择、特征数的取舍，都可以在上线后慢慢优化。

最后留一句判断标准。一个诊断系统做得好不好，不看它在论文表格里排第几，看三件事：错的时候知不知道自己可能错（拒识与距离）、环境变了之后多久能发现（漂移监控）、换一个人接手能不能复现（模型文件与台账）。这三件事都做到，准确率 92.70% 也够用；三件事都做不到，99% 也只是在训练集上好看。

第 26 章 机器学习基础

第 25 章把“一段信号”变成了“一个类别”，用的特征是人工按机理设计的，分类器是浅层的、可解释的。那一章的隐含前提是：特征已经足够好，剩下的只是把边界放对。工程上还有另一类问题，前提正好相反——特征与目标之间的关系写不出公式，只能从数据里学。换热器的出口温度要靠阀位、入口温度、流量去“软测量”，产品的硬度要靠成分与工艺参数去预测，电机的剩余寿命要靠振动与电流去估计。这类问题的答案是连续量，学习器要从样本里自己找出映射。本章讨论这件事的一般框架：学什么、怎么防止学过头、怎么评估学得好不好，以及上线之后靠什么维持。

26.1 动机与要解决的问题

26.1.1 从“判类”到“学一个函数”

模式识别与机器学习经常被当成同义词，其实两者的重心不同。模式识别关心的是“类别边界放在哪里”，输出是离散标签，评价是判对率；机器学习关心的是“从样本估计一个函数”，输出可以是连续量，评价是预测误差。第 25 章的线性判别、支持向量机都是学习器，但它们被用在分类任务上，讨论的重点是边界与拒识；本章换到回归任务，讨论的重点变成偏差、方差与泛化。

这个重心转移带来三个新问题，第 25 章没有正面处理过。

第一个是“函数形式从哪来”。分类器的边界形式由模型族决定（线性、二次、核），但边界本身只有“分对”一个要求，容错空间大；回归要拟合整条曲线，函数形式选错，误差会系统性地留在残差里，且无法靠更多数据消除。第 25 章的残差只用来判断“有没有漏掉信息”，本章的残差要定量分解成偏差、方差与噪声三部分。

第二个是“模型复杂度怎么定”。分类里复杂度主要表现为边界弯曲程度，可以用交叉验证选；回归里复杂度既可以是特征个数、多项式阶数，也可以是树的深度、集成规模，维度更多，且复杂度与样本量的匹配关系更敏感。一个 14 个特征的二次模型在 900 个样本上很稳，在 50 个样本上会崩——实验 C 会给出这个对比。

第三个是“评估的陷阱更多”。回归常用决定系数与均方根误差，两者都依赖测试集；而测试集一旦被反复使用，评估就会失真。分类里这个问题表现为“用测试集调阈值”，回归里表现为“用测试集选特征、选阶数、选模型”，形式更隐蔽，后果同样是数字虚高。

26.1.2 一个软测量任务

本章用一个贯穿全章的算例：换热器出口温度的软测量。可测量有四个——阀位 u、入口温度 T_{in}、流量 q、压力 p；难测量是出口温度 y，现场用在线分析仪测，成本高、维护贵，希望用前四个量把它算出来。

稳态机理是清楚的：

$$y = T_{\mathrm{in}} + (T_{\mathrm{hot}} - T_{\mathrm{in}})\left(1 - e^{-k u / q}\right) + \varepsilon \tag{26-1}$$

其中 $T_{\mathrm{hot}} = 95$ ℃、$k = 2.4$，ε 是不可测的环境扰动与传感器噪声之和。这个式子的关键在于指数项：出口温度对“阀位与流量之比”呈饱和关系，阀位开得越大、流量越小，温度越高，但增益越来越小。它是非线性的，却不是多项式型的，任何一个二次模型都只能近似。

数据的规模是 1200 段准稳态工况，训练 900、测试 300，随机种子固定。四个原始量的取值范围是：阀位 0.30 到 0.95、入口温度 25 到 45 ℃、流量 1.2 到 3.0 m³/h；出口温度落在 40.7 到 85.5 ℃，均值 65.7 ℃。为了对比“机理特征”与“盲目展开”，本章另外构造了四个组合量：阀位比流量 u/q、温升组合 $T_{\mathrm{in}}u/q$、阀位平方、流量平方，共 8 个特征。

噪声是刻意留出的。环境扰动标准差 1.0 K、传感器噪声 0.4 K，合成方差 1.16 K²，对应均方根误差 1.077 K。这意味着任何模型的测试均方根误差都不可能低于 1.08 K——这条下限在后面每一张图里都会画出来，它是判断“还能不能更好”的唯一客观参照。用高复杂度模型在测试集上估出的噪声方差是 1.406 K²（对应 1.186 K），略高于理论值，差额来自模型自身的残余偏差。

26.1.3 三个先导结果

在展开方法之前，先看三个“看起来该成功、实际有陷阱”的结果。它们构成本章的问题清单。

第一个关于复杂度。把 8 个特征喂给一棵不剪枝的决策树，训练集决定系数是 1.0000——它把每个训练样本都记住了。测试集掉到 0.9653，均方根误差 1.633 K，比只有 4 个原始特征的线性回归（0.9578 / 1.800 K）只好一点点，比 8 特征的线性模型（0.9818 / 1.182 K）明显差。**训练集满分不是成绩，是过拟合的症状。** 同一棵树的深度限制到 6 层，训练集降到 0.9881，测试集反而升到 0.9735；这说明复杂度不是越高越好，存在一个由样本量决定的最优点。

第二个关于泄漏。把出口温度的含噪副本（噪声 0.5 K）当作第九个特征加进线性模型，测试决定系数从 0.9818 跳到 0.9973，均方根误差从 1.182 K 降到 0.460 K——比噪声下限 1.077 K 还低。这在物理上不可能：任何模型都无法比测量噪声更准。**当指标好到违反物理下限时，先怀疑泄漏，不要先庆祝。** 现场常见的泄漏源包括：用了事后才能拿到的量、用了被控量本身的历史值、标准化时用了全数据统计量。

第三个关于集成。在同样 300 段训练子样本上重复 30 次，单棵深度 6 的树测试均方根误差均值 1.652 K，随机森林降到 1.349 K，梯度提升 1.298 K；误差的标准差也从 0.093 降到 0.047 与 0.039。**集成同时改善了均值与稳定性**，而且没有增加任何特征。这个结果不能简单理解为“集成总是更好”——它在 900 段全量训练时的优势要小得多（1.227 K 对 1.179 K，而线性模型是 1.130 K），样本越少、单模型越不稳，集成的收益越大。

三个结果指向同一件事：误差不是一个数，而是一组来源不同的分量之和。第一例里，不剪枝树把训练误差压到零，代价是方差暴涨；第二例里，泄漏把误差压到噪声下限以下，代价是模型学到了现实中拿不到的信息；第三例里，集成把方差压下去，代价是计算量与可解释性。三种“降低误差”的手段，作用在不同分量上，代价也各不相同。**不理解误差的构成，就只能靠试；理解了，才知道下一步该改什么。**

这三件事还有一条共同的工程含义：它们都能用“测试集上的一次测量”识别出来。过拟合表现为训练与测试的巨大差距，泄漏表现为指标低于物理下限，集成收益表现为重复抽样下误差分布的收窄。三者都不需要额外的理论工具，只需要在报告里同时给出训练误差、测试误差、误差下限与重复次数。这也是本章反复强调“同一批数据、同一套口径”的原因——只有在同一口径下，这些对比才有意义。

26.1.4 与相邻章节的分界

本章讲的是“通用的学习框架”，不深入任何一类模型的内部结构。这条边界要划清楚，否则容易与后续章节重复。

线性模型与正则化的完整推导在第 18 章（最小二乘族）与第 30 章（优化算法）；本章只用结论，重点放在“它为什么是偏差最大的模型”这个角色上。树模型与集成方法的工程调参在第 27 章之后不再展开，本章给的是偏差—方差视角下的解释。深度学习的结构、训练技巧与表征学习在第 27、28 章；本章的自编码器只做到线性情形，用它说明“无监督学习学到的是数据的低维结构，不是标签”。

与控制器的接口在后续章节：软测量的输出可以直接进前馈通道（第 44、106 章的 PID 与前馈），也可以作为预测控制的模型（第 58、80 章）。本章不讨论“测出来之后怎么控”，只讨论“怎么测得准、怎么知道测得准不准”。

26.1.5 四个问题与路线

本章要回答四个问题，每个都对应可以量化的实验：

1. 八种模型在同一批数据上差多少？差距来自偏差还是方差？（实验 A、B）
2. 样本量增加时，哪种模型先饱和？饱和点在哪里？（实验 C）
3. 评估方式本身会带来多大波动？怎样报告才可信？（实验 D、E）
4. 集成与无监督降维各能带来多少收益？边界在哪里？（实验 H、G）

四个问题由“模型”到“评估”再到“结构”，顺序不能颠倒。工程上最常见的失误是只做第一个：把八种模型跑一遍，挑测试集上最好的一个交差，既不看偏差方差从哪里来，也不检查评估是否被污染。这样得到的模型，换一批数据就会露出原形。

本章的结构对应这条路线。26.2 节给统一的数学框架（经验风险、模型族、偏差方差、交叉验证），26.3 节推导四条决定成败的性质（正规方程的病态、偏差方差分解、集成的方差缩减、学习曲线的形状），26.4 节讲数值实现与数据质量，26.5 节给适用边界与工程落地，26.6 节用八组实验回答上面四个问题，26.7 节收拢结论。

26.1.6 本章的三条口径

读后面的数字之前，有三条口径要先说清楚，否则容易误读。

第一条，所有决定系数与均方根误差都是固定留出测试集上的值，训练集只用 900 段，测试集 300 段，划分只做一次。实验 D 会专门说明单次划分的波动有多大：同一模型换 20 次划分，测试决定系数的标准差是 0.0017，五折交叉验证的标准差是 0.0008。前者不是“更差”，只是“更随机”。

第二条，模型超参数一律取常用默认值，没有针对本章数据调优。树深度是唯一被系统扫描的参数，因为它是本章用来演示偏差—方差的主角。调参会把数字刷得更好看，但也会让“八种模型差多少”这个比较失真。

第三条，本章的噪声是已知的（1.0 K 扰动加 0.4 K 传感器噪声），所以每一张误差图都能画出噪声下限。真实项目里这个下限未知，只能用高复杂度模型的残差去估；估出来的值通常偏低，因为它把模型自身的偏差也算进了“噪声”。这一点在第 24 章的残差分析里出现过，本章只是换了个场景。

26.1.7 工业数据与教科书数据的差别

教科书里的机器学习例子大多满足三个条件：样本上万、标签干净、训练与测试同分布。工业数据三条都不满足，而这三条恰好决定了方法该怎么用。

样本少。本章 1200 段数据已经是“攒了几个月”的规模，很多现场一年也攒不到几百段带标签的工况。样本少直接抬高式 26-22 里的 a/n 项，所以本章要花一整节讲学习曲线，而不是假设“数据越多越好”。样本少还带来一个副作用：任何需要大量参数的方法都要打折扣用，这也是为什么本章把多项式放在 14 个特征就停手。

噪声大且不可约。工业测量的噪声往往比信号的结构误差还大：本章的总方差是 1.16 K^2，而不同模型之间的差距只有 0.02 到 0.6 K。这意味着**模型比较必须在噪声下限的坐标系里做**，否则很容易把 0.02 K 的差别当成“显著改进”。现场更常见的情况是噪声下限根本估不准，因为真实函数未知——这时用高复杂度模型的残差方差去估，得到的是“下限加上模型偏差”，本章的 1.406 K^2 与理论 1.16 K^2 就是这个差额的实例。

分布会漂移。教科书数据是静态的，工业数据每天都在变：催化剂老化、换热面结垢、原料批次更换。§26.5.3 的杠杆值与残差监控就是为这件事准备的。漂移还有一个更隐蔽的后果：训练集与测试集如果按时间先后划分，测试集天然比训练集“新”，误差会比随机划分大一截。本章为了做模型间的公平比较用了随机划分，真实项目里应当补一次按时间划分的验证——两个数字的差就是漂移的代价。

这三条差别指向同一个态度：**工业场景里，方法的选择空间比教科书小得多，能做的改进大多在数据侧而不是算法侧**。本章后面每一节都会回到这个判断上。

26.1.8 需要多少样本

“多少数据才够”是项目启动时第一个被问到的问题，它有一个粗糙但可用的答案：**样本量至少是参数量的十倍**。本章的二次多项式有 14 个系数，按这条规则需要 140 段；实际用了 900 段，远超下限，所以它表现稳定。反过来，50 段样本对 14 个参数只有 3.5 倍，测试误差 5.657 K 就是这个比例的代价。

十倍规则的来源是式 26-22 的 a/n 项：方差与 a/n 成正比，a 与参数量成正比，样本量要到参数量的十倍量级，方差项才会降到与噪声同阶。它不是精确判据，但对量级判断足够：参数量翻倍，样本量也该翻倍，否则误差会明显上升。

比参数量更根本的限制是噪声。**样本量再大也压不掉 1.077 K 的噪声下限**，所以当模型误差已经接近这个下限时，继续攒数据是浪费。本章的最好模型 1.159 K 离下限只有 0.08 K，这 0.08 K 里既有残余偏差也有残余方差，即使样本翻十倍也只可能改善其中很小一部分。

工程上还有一个常被忽略的约束：**样本要覆盖工况，不只是数量够**。900 段集中在 60 到 70 摄氏度区间的数据，不如 600 段均匀覆盖 40 到 85 摄氏度的数据有用。样本量解决的是“平均意义上的方差”，工况覆盖解决的是“局部的偏差”，后者无法靠前者替代。本章的数据在阀位与流量上是均匀采样的，所以 900 段就能覆盖整个工况面；真实项目里这一段往往要先做数据审计，再谈建模。

26.2 定义与数学基础

26.2.1 风险、经验风险与模型族

监督学习的形式化只有三样东西：数据、模型族、损失。数据是 n 个独立同分布的样本 $(\mathbf{x}_i, y_i)$，$\mathbf{x}_i \in \mathbb{R}^d$；模型族是一组候选函数 $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$；损失 $L(y, f(\mathbf{x}))$ 度量预测与真值的差距。学习的目标是让期望风险最小，

$$R(f) = \mathbb{E}_{(\mathbf{x},y)}\left[L(y, f(\mathbf{x}))\right] \tag{26-2}$$

但真实分布未知，只能用样本平均代替，

$$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} L\left(y_i, f(\mathbf{x}_i)\right), \qquad \hat{f} = \arg\min_{f \in \mathcal{F}} \hat{R}_n(f) \tag{26-3}$$

这个做法叫经验风险最小化。它有一个内在矛盾：$\hat{R}_n$ 是 R 的有噪估计，模型族越大，越容易找到一个“在样本上损失很小、在分布上损失很大”的函数。全部学习理论，本质上都在处理这个矛盾。

损失的选择决定了模型在学什么。平方损失 $L = (y - f)^2$ 的最优解是条件均值 $\mathbb{E}[y \mid \mathbf{x}]$；绝对损失的最优解是条件中位数，对离群点更稳；对数损失用于二分类，最优解是后验概率。**换损失不是换优化技巧，是换学习目标**——用平方损失训练出来的软测量，给出的是“平均意义上的温度”，不是“最可能的温度”，两者在分布偏斜时不同。

模型族可以按“结构假设的强弱”排成一条线。线性模型假设输出是输入的线性组合，假设最强、参数最少；多项式模型放松到“输入的固定非线性变换的线性组合”；树模型假设输出是输入空间上的分段常数；集成模型是若干基学习器的加权和；核方法与深度网络把特征变换也交给数据去定。假设越弱，能表达的函数越多，需要的样本也越多。表 26-1 把本章用到的八种模型按这条线排开。

表 26-1　八种模型的结构假设与复杂度旋钮

模型	结构假设	复杂度旋钮	偏差倾向	方差倾向
线性回归	输出对输入线性	特征个数	高	低
岭回归	同线性，系数受惩罚	正则系数	较高	低
二次多项式	输出对二次项线性	阶数、项数	中	中
决策树	输入空间分段常数	深度、叶最小样本	低（深树）	高（深树）
随机森林	多棵树平均	树数、每次分裂的特征数	低	中
梯度提升	多棵树累加	树数、学习率、深度	低	中
线性自编码器	低维线性子空间	潜维数	由潜维定	低
逻辑回归	对数几率对输入线性	特征个数、正则	高	低

26.2.2 线性回归与正规方程

线性模型写作

$$f(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b \tag{26-4}$$

把偏置并入权重（在 $\mathbf{x}$ 末尾补一列 1），记设计矩阵 $\boldsymbol{\Phi} \in \mathbb{R}^{n \times (d+1)}$，平方损失的经验风险最小化就是最小二乘。令梯度为零，得到正规方程

$$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\,\mathbf{w} = \boldsymbol{\Phi}^{\mathsf{T}}\mathbf{y} \quad \Longrightarrow \quad \hat{\mathbf{w}} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\mathbf{y} \tag{26-5}$$

这个解存在的条件是 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 可逆，即特征线性无关且样本数不少于特征数。第 18 章已经详细讨论过它的统计性质，这里只强调与本章有关的一点：**最小二乘是无偏的，但方差可能很大**。当特征共线时，$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 接近奇异，微小的数据扰动会让系数大幅摆动，预测值却可能仍然稳定——这就是“系数不可解释但预测可用”的典型情形。

岭回归在损失里加一项系数惩罚，

$$\hat{\mathbf{w}}_{\lambda} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi} + \lambda I)^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\mathbf{y} \tag{26-6}$$

$\lambda > 0$ 把最小特征值从 0 附近抬到 λ，条件数随之下降，方差减小、偏差略增。本章的二次模型有 14 个特征，其中若干项高度相关（u、u^2、u/q 之间），所以岭回归与普通最小二乘的测试表现几乎相同（0.9825 对 0.9825），但系数稳定得多。**正则化在预测指标上看不出好处时，仍然值得做**，因为它的收益体现在“换一批数据系数不会翻倍”上。

26.2.3 逻辑回归

分类问题里，线性模型要经过一个把实数压到 $(0,1)$ 的变换。二分类用对数几率，

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{26-7}$$

训练目标是最小化负对数似然（交叉熵），

$$\hat{R}_n(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\right] \tag{26-8}$$

它对 $\mathbf{w}$ 的梯度有很简洁的形式，

$$\nabla_{\mathbf{w}}\hat{R}_n = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,\mathbf{x}_i \tag{26-9}$$

残差乘特征，与线性回归的正规方程同构，只是权重不是常数而是 $p_i(1-p_i)$。这个式子说明逻辑回归的优化是凸的，梯度下降一定收敛到全局最优，不需要担心局部极小。

逻辑回归与第 25 章的线性判别分析在形式上都给出线性边界，但假设不同。线性判别假设两类都是高斯、共享协方差，先估分布再算后验；逻辑回归直接假设对数几率线性，不估分布。当高斯假设成立时，两者渐近等价；当假设不成立时，逻辑回归通常更稳，因为它少了

一层可能出错的假设。**判别式模型用假设换数据，生成式模型用数据换假设**，这句话在第 25 章出现过，在这里表现为“逻辑回归不需要估协方差”。

26.2.4 决策树

决策树把输入空间递归地切成矩形区域，每个区域给一个常数。回归树的分裂准则是方差下降：若节点 S 按特征 j 与阈值 t 分成 S_L 与 S_R，则选择使下式最大的划分

$$\Delta(S, j, t) = \mathrm{Var}(S) - \frac{|S_L|}{|S|}\mathrm{Var}(S_L) - \frac{|S_R|}{|S|}\mathrm{Var}(S_R) \tag{26-10}$$

叶节点的预测是该节点样本的均值，等价于在矩形区域上做零阶近似。分类树把方差换成基尼指数或熵，形式相同。

树的三个性质决定了它的适用面。**第一，它天然处理交互**：先按阀位切一刀、再在子区域里按流量切一刀，等价于自动构造了“阀位与流量的交叉项”，不需要人工展开。这正是本章 8 个特征里加两个交叉组合后线性模型才追上来、而树一开始就能表达的原因。**第二，它对输入做的是分段常数近似**，所以真函数越光滑，它需要的叶子越多；反过来，有阈值与分段特性的真函数，它用很少的叶子就能表达准。**第三，它不能外推**：任何叶子的输出都是训练样本标签的均值，落在训练标签的范围内，输入跑到训练范围之外时，树的预测会停在边界值上不动。这一条在软测量里是致命缺陷——工况超出训练范围时，树不会告诉你“我不知道”，只会给一个看似合理的常数。

控制过拟合的手段分两类。预剪枝在生长时设限：最大深度、最小叶样本数、最小分裂增益，本章用的就是最大深度。后剪枝先长满再自底向上合并，用验证集判断是否保留子树，代价更高、效果通常更好。**预剪枝省算力但不一定找到最优点，后剪枝更接近最优点但需要额外数据**，工程上先预剪枝定个大概，再用验证集微调。

树的结构决定了它的偏差—方差特征。浅树只有少数几个区域，分段常数的分辨率低，偏差大、方差小；深树区域很细，能贴合训练样本，偏差小、方差大；不剪枝的树每个叶只有一两个样本，训练误差为零，方差最大。**树的复杂度旋钮就是“区域有多细”**，它比线性模型的“特征有几个”更直接地控制过拟合。

26.2.5 集成：平均与累加

集成有两种基本思路，作用在不同的误差分量上。

第一种是平均，代表是随机森林。设 B 个基学习器的预测为 $\hat{f}_1, \dots, \hat{f}_B$，两两相关系数为 ρ、方差为 σ^2，则平均后的方差是

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{26-11}$$

树数 B 增大只能消掉第二项，第一项由相关性决定。随机森林在每次分裂时只随机看一部分特征，正是为了压低 ρ——基学习器之间的差异越大，平均越有效。本章的实现保留了默认设置（每次分裂考虑全部特征），所以差异只来自 bootstrap 重抽，ρ 压得不够低；这可能是它 1.349 K 没能追上梯度提升 1.298 K 的一个原因。这条式子还解释了实验 H 的另一个现象：再增加树数收益会迅速饱和，因为 ρ 没有变。

第二种是累加，代表是梯度提升。模型写成加法形式

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, h_m(\mathbf{x}) \tag{26-12}$$

其中 h_m 拟合当前模型的负梯度（对平方损失就是残差），$\nu \in (0, 1]$ 是学习率。每一轮都在修正上一轮的偏差，所以提升主要降低偏差；代价是它对噪声敏感，轮数过多会把噪声也拟合进去。实验 H 里提升的误差均值 1.298 K 最低，标准差 0.039 也最小，但这是在 300 段子样本上重复 30 次的结果，若把轮数加到几千，测试误差会先降后升。

两种集成的差别可以用一句话概括：**平均降方差，累加降偏差**。前者要求基学习器“各自不错且互不相同”，后者要求基学习器“一个比一个更准”。工程上选哪一种，取决于当前误差的主要成分——用实验 B 的分解看一眼就知道。

两种集成的调参方向也因此相反。随机森林的关键是“让树更不相似”：开特征子采样、减小每棵树的采样比例、不限制单树深度，三招都在降 ρ 或提高单树方差，只要平均有效就行。梯度提升的关键是“让每轮走得稳”：学习率小、树浅、轮数多，用多次小步代替一次大步，避免某一轮把噪声当成信号学进去。**把两者的调参直觉互换会立刻出事**：给提升加深度会让它飞快过拟合，给森林加深度却几乎无害，因为平均会把方差吃掉。

26.2.6 无监督：主成分与自编码器

无监督学习没有标签，只能学数据自身的结构。主成分分析找的是重构误差最小的线性子空间，

$$\min_{V^{\mathsf{T}}V = I_k} \mathbb{E}\left\|\mathbf{x} - VV^{\mathsf{T}}\mathbf{x}\right\|^2 \tag{26-13}$$

解是协方差矩阵前 k 个特征向量张成的子空间。第 25 章已经从“保方差”的角度讲过它，这里换一个角度：它等价于“用一个 k 维线性编码再解码，使均方误差最小”。

自编码器把这个想法推广成神经网络：编码器 f_ϕ 把 $\mathbf{x}$ 压到 k 维，解码器 g_ψ 再还原，

$$\min_{\phi,\psi} \ \mathbb{E}\left\| \mathbf{x} - g_\psi(f_\phi(\mathbf{x})) \right\|^2 \tag{26-14}$$

当 f、g 都是线性映射、损失是平方误差时，最优解张成的子空间与主成分完全相同——只是基向量可以任意旋转，因为旋转不改变重构误差。实验 G 验证了这一点：潜维 1 到 4，两种方法的重构均方误差在小数点后四位一致（0.45459 对 0.45460、0.00307 对 0.00308）。潜维 5 以上开始分离（0.00189 对 0.00222），原因是梯度下降没有完全收敛，不是模型本身有差别。

这个等价性有实用价值：**在数据是线性结构、样本量中等时，不必上自编码器，主成分更便宜、更稳定**。自编码器的优势要等非线性结构出现才体现——那时线性子空间不够用，需要用非线性编码去“展平”流形。第 27、28 章会展开这部分，本章只用它说明“无监督学到的是数据结构，不是标签”。

26.2.7 偏差—方差分解与交叉验证

把测试误差按来源拆开，是本章的核心工具。设真实函数为 f、观测为 $y = f(\mathbf{x}) + \varepsilon$、$\mathbb{E}[\varepsilon^2] = \sigma^2$，用不同训练集得到的模型为 $\hat{f}$，则在某个测试点上的期望误差是

$$\mathbb{E}\left[(y - \hat{f}(\mathbf{x}))^2 \right] = \sigma^2 + \left(f(\mathbf{x}) - \mathbb{E}\hat{f}(\mathbf{x}) \right)^2 + \mathbb{E}\left[\left(\hat{f}(\mathbf{x}) - \mathbb{E}\hat{f}(\mathbf{x}) \right)^2 \right] \tag{26-15}$$

三项依次是噪声、偏差平方、方差。噪声是数据的下限，任何模型都消不掉；偏差是“平均模型离真值多远”，靠增加模型复杂度降低；方差是“模型对训练集的敏感程度”，靠增加数据或正则化降低。三者不能同时最小，这就是“没有免费午餐”在回归里的具体形态。

交叉验证估计的是“训练流程”的泛化误差，不是某个具体拟合模型的误差。把数据分成 K 折，轮流用 $K-1$ 折训练、1 折验证，

$$\widehat{\mathrm{CV}}_K = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|D_k|} \sum_{i \in D_k} L\left(y_i, \hat{f}^{(-k)}(\mathbf{x}_i) \right) \tag{26-16}$$

其中 $\hat{f}^{(-k)}$ 表示不用第 k 折训练的模型。它与单次留出的区别在方差：K 折用了 K 次不同的划分，估计的波动更小；代价是训练 K 次，且各折的训练集有重叠，估计略有相关性。实验 D 的数字是：同一模型换 20 次划分，单次留出的决定系数标准差 0.0017，五折交叉验证 0.0008，十折也是 0.0008——折数从 5 增到 10 没有进一步改善，因为瓶颈已经变成数据本身的随机性，不是划分方式。

分层与分组是交叉验证的两个必要变体。分类问题要分层，保证每折的类别比例一致；时序或按设备采集的数据要分组，保证同一台设备、同一段连续时间不跨折，否则模型会“记住”测试样本的邻居。本章数据是准稳态工况、样本独立，所以用普通 K 折；真实项目里这条几乎总是要改成分组。

26.2.8 损失函数决定学的是什么

同一个模型族配上不同损失，学到的不是同一个量。这不是实现细节，而是必须在上线前说清楚的事。

平方损失 $\frac{1}{2}(y-\hat{y})^2$ 的最优解是条件均值 $\mathbb{E}[y \mid \mathbf{x}]$。它的优点是梯度线性、优化简单、有闭式解；缺点是**对异常值敏感**：一个偏离 10 K 的点，其梯度是偏离 1 K 的点的十倍，会把整条曲线拉过去。工业数据里异常值几乎不可避免——仪表跳变、人工录入错误、非稳态过渡段没剔干净——所以用平方损失之前必须先做异常值筛查，或者换损失。

绝对损失 $|y-\hat{y}|$ 的最优解是条件中位数。它对异常值稳健，代价是梯度不连续、在最优点附近收敛慢，且**中位数不等于均值**：如果残差分布偏斜，“预测得最准”的定义就变了。工程上这条差别很实际：工艺关心的是“平均温度”，用绝对损失学出来的模型会系统性地偏低或偏高。

两者的折中是 Huber 损失：残差小于阈值 δ 时用平方、大于时用线性，兼顾小残差处的光滑与大残差处的稳健。δ 的选取应当对应“多大的残差算异常”，而不是拍一个数——本章的噪声标准差是 1.077 K，δ 取一到两倍标准差是合理的量级。

分位数损失 $\rho_\tau(y-\hat{y}) = \max\{\tau(y-\hat{y}), (\tau-1)(y-\hat{y})\}$ 学的是条件 τ 分位数，用于给出预测区间。软测量里这比点预测更有用：告诉操作员“八成情况下不会超过某个值”比给一个点估计更能支撑决策。本章只做点预测，但式 26-28 的阈值本质上是同一个思想在分类上的体现。

分类任务的对应物是交叉熵与合页损失。交叉熵（式 26-8）学的是后验概率，输出可以直接解释为置信度；合页损失学的是间隔，只关心“分对且有余量”，不给出概率。**要概率就用交叉熵，只要判决就用合页**，两者的差别在第 25 章的支持向量机与本章的逻辑回归之间已经出现过一次。选错损失的典型症状是“模型判得很准但概率没有意义”，这时任何基于概率的阈值调整都会失效。

26.2.9 参数化模型与非参数化模型

模型族可以按“参数个数是否随样本量增长”分成两类，这条分界比“线性还是非线性”更能说明泛化行为。

参数化模型的参数个数在训练前就定死：8 特征线性模型有 9 个系数（含截距），14 特征二次多项式有 15 个，神经网络再多也是有限个。它的容量固定，偏差下限固定，增加样本只能把方差压到零，压不到偏差。本章的多项式就属于这一类：900 段时 1.161 K，再给一万段也只能趋近它自身的偏差，不会趋近 1.077 K。

非参数模型的参数个数随样本增长：树的叶子数、最近邻的样本数、核方法中的支持向量数，都随 n 变大。它的容量自适应，样本越多能表达的函数越细，偏差可以趋近于零；代价是方差随 n 下降得慢，且对数据的局部密度敏感。本章的树与森林属于这一类，深度 6 的树在 50 段时只有很少的叶子可用，在 900 段时才有 64 个叶子可填——**同一棵树的“有效复杂度”是被样本量决定的**，这也是实验 C 中树在小样本上更稳的原因。

这条分界给出一条选型直觉：**结构已知、样本少，用参数化模型；结构未知、样本多，用非参数化模型**。本章的真函数是已知的指数饱和形式，所以多项式在样本充足时反超；如果真函数里有阈值、分段、启停逻辑，参数化模型必须先猜对分段位置，猜不对就永远差一截，这时树的优势无法被样本量替代。

26.2.10 从贝叶斯角度看正则化

岭回归的惩罚项常被当作“防止过拟合的技巧”，其实它有明确的概率含义：若假设系数服从零均值、方差为 τ^2 的高斯先验，观测噪声方差为 σ^2，则最大后验解正是式 26-6 的岭回归解，且 $\lambda = \sigma^2/\tau^2$。**正则化强度不是拍出来的，它等于“噪声有多大”与“系数该有多大”之比**。

这个视角能解释三件事。第一，为什么数据标准化之后 λ 才有可比性：先验是加在标准化系数上的，“系数该有多大”必须先有统一的尺度。第二，为什么样本越多 λ 应当越小：σ^2 固定而似然项随 n 增强，后验越来越被数据主导，先验的相对权重自然下降。第三，为什么树的“最小叶样本数”也是一种正则化：它等价于声明“叶子均值不应当基于一两个样本”，是加在树结构上的先验。

交叉验证在这个视角下是经验贝叶斯：用数据本身去估 λ，而不是人为指定。本章用 13 个候选 λ 做留一交叉验证，选出的解与不加正则化的版本只差 0.002 K，说明数据本身已经把先验的权重压得很低——样本充足时，先验不重要，这是贝叶斯框架给出的、与工程直觉一致的结论。

26.3 推导与性质

26.2 节给的是定义，本节回答“为什么”。四条核心性质分别对应四个具体的工程疑问：为什么高次多项式会在新数据上崩掉、为什么深到 12 层的树反而不如 6 层、为什么一百棵树平均起来既更准也更稳、为什么样本量从 50 加到 900 时收益最大的偏偏是那个在小样本上最差的模型。四条之后再加三条补充：泛化差距该有多大、树为什么不稳定、以及特征尺度与分裂偏向。七条都不是经验口诀，都能从公式推出来，也都能在本章数据上量出来。

26.3.1 正规方程的病态与岭回归的稳定作用

最小二乘的闭式解是 $\hat{\boldsymbol{\beta}} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y}$。它对观测扰动的敏感程度由矩阵的条件数决定。设 $\mathbf{y}$ 有扰动 $\delta\mathbf{y}$，则解的相对误差满足

$$\frac{\|\delta\hat{\boldsymbol{\beta}}\|}{\|\hat{\boldsymbol{\beta}}\|} \le \kappa(X^{\mathsf{T}}X)\,\frac{\|\delta\mathbf{y}\|}{\|\mathbf{y}\|}, \qquad \kappa(X^{\mathsf{T}}X) = \frac{\lambda_{\max}(X^{\mathsf{T}}X)}{\lambda_{\min}(X^{\mathsf{T}}X)} \tag{26-17}$$

而 $X^{\mathsf{T}}X$ 的特征值是 X 的奇异值的平方，于是 $\kappa(X^{\mathsf{T}}X) = \kappa(X)^2$。这是设计矩阵病态之所以危险的全部原因：**数据本身的共线性只让问题坏一倍，正规方程把它平方放大**。工程上常见的“系数符号反了”“换个样本系数翻三倍”，多数不是算法实现错了，而是条件数在 10^6 以上时，浮点舍入与测量噪声被平方级地放大到了系数量级。

岭回归的做法是给特征值整体垫一个常数。把 $X^{\mathsf{T}}X$ 正交分解为 $Q\Lambda Q^{\mathsf{T}}$，则

$$(X^{\mathsf{T}}X + \lambda I)^{-1} = Q(\Lambda + \lambda I)^{-1}Q^{\mathsf{T}}, \qquad \kappa_\lambda = \frac{\lambda_{\max} + \lambda}{\lambda_{\min} + \lambda} \tag{26-18}$$

分子几乎不变、分母被抬起，条件数随 λ 单调下降。代价是解被系统性压缩：λ 越大，模型越偏向“所有系数都小”的方向。这不是缺陷，而是把“宁可欠拟合也不要爆炸”这一工程偏好写进了目标函数。

本章数据上这条性质的表现很克制。四个原始特征（阀位、入口温度、流量、压力）量纲差异很大——阀位在 0.30 到 0.95 之间、入口温度在 25 到 45 摄氏度之间、流量在 1.2 到 3.0 立方米每小时之间，标准化之后二次项之间的相关系数很高，共线性并不轻。实验 A 的结果是：四个原始特征的线性模型测试决定系数 0.9578，加上四个工程特征后升到 0.9818；再展开到 14 个特征的二次多项式只再涨 0.0007，达到 0.9825；岭回归在同一组 14 个特征上给出 0.9825、测试均方根误差 1.159 K，与不加正则化的 1.161 K 几乎重合。结论是：**在这批数据**

上，共线性还没有严重到必须正则化的程度，但差值只有 **0.002 K** 恰恰说明正则化在这里不亏——它没有换来精度，却换来了系数稳定性和可解释性。真实装置上量程漂移与传感器更换会把条件数推到 10^6 以上，那时这 0.002 K 的“免费”就不存在了，必须付。

26.3.2 偏差—方差分解的定量形态

式 26-15 给出了三项分解，但它只说明“有三项”，没有说明三项随模型复杂度怎么变。对回归树这类分段常数模型，可以写出一个形态正确、常数不精确的近似。设树的深度为 d、训练样本量为 n、输入维度为 p，则

$$\mathbb{E}\left[(y-\hat{f}_d)^2\right] \approx \sigma^2 + \underbrace{c_1\, a^{-d}}_{\text{bias}^2} + \underbrace{c_2\, \frac{2^d}{n}}_{\text{variance}} \tag{26-19}$$

其中 $a>1$ 反映每次分裂把区域切细后偏差的下降速度，2^d 是叶子数的上界，c_1, c_2 与具体分布有关。式 26-19 的价值不在常数，而在两个指数：**偏差随深度指数下降，方差随深度指数上升**。两者相加的形态决定了总误差必然先在某个中间深度取到最小值，这个最小值的位置随 n 增大而右移——数据越多，能支撑的深度越深。

本章实测把这条曲线钉住了。以无噪声的真实出口温度为基准，深度 1、6、12 三档的结果是：偏差平方 23.610 / 0.216 / 0.264，方差 4.080 / 1.130 / 1.641，测试均方误差 29.214 / 2.558 / 3.150。三项加理论噪声 1.16 K^2 与实测值的偏差在 3% 以内，式 26-19 的分解成立。从深度 1 到 6，偏差降了 109 倍、方差只涨 3.6 倍，总误差降 11.4 倍；从深度 6 到 12，偏差不但没降反而略升到 0.264，方差涨了 45%，总误差反升 23%。

偏差在深度 6 之后不再下降，这件事值得单独说一句。理论上加深应该继续降低偏差，但当叶子里的样本数降到个位数时，叶子值本身已经是噪声的平均，它在训练点上拟合得更好、在测试点上却不再更接近真值。**式 26-19 里的偏差项隐含了“每个叶子有足够样本估计其均值”这个前提，前提破了，偏差就不再随深度下降**。这就是“不剪枝的树训练决定系数 1.0000、测试只有 0.9653”的成因：训练集上的零误差不是能力的证据，而是记忆的证据。

26.3.3 集成的方差缩减

把 B 个模型做平均，方差的缩减有一个干净的表达式。设每个模型方差为 σ^2、两两相关系数为 ρ，则平均模型的方差是

$$\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \tag{26-20}$$

第二项随 B 趋于零，第一项不随 B 变化。式 26-20 说明了一件反直觉的事：**无限加树并不能把误差压到零，误差的下限由树与树之间的相关性 ρ 决定**。想把误差继续往下压，唯一的办法是让树之间更不相似，也就是降低 ρ。降低 ρ 有两条路：一是让每棵树看不同的样本，靠 bootstrap 重抽；二是让每次分裂只看一部分特征。本章的实现用的是第一条，随机森林保留默认的“每次分裂考虑全部特征”，所以 1.349 K 这个数字里，方差缩减全部来自重抽样本；若把每次分裂的候选特征限制到三分之一，ρ 还能再降一档。反过来，**特征高度冗余时特征子采样的收益会消失**——冗余特征让随机子集之间几乎没有差异，ρ 压不下去。

平均降方差，累加降偏差，这是两类集成算法的分工。梯度提升写成

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu\, h_m(\mathbf{x}), \qquad h_m = \arg\min_{h}\sum_{i=1}^{n} L\left(y_i, F_{m-1}(\mathbf{x}_i) + h(\mathbf{x}_i)\right) \tag{26-21}$$

每棵树拟合的是当前模型残差的负梯度，ν 是学习率。它不做平均，所以不直接降方差；它逐步修正系统性偏差，所以对偏差敏感、对噪声也敏感——这也是提升类模型比随机森林更容易被标签噪声带偏的原因。

实测的两组数字印证了这个分工。用 30 次重抽的 300 段子样本做测试：深度 6 的单棵树均方根误差 1.652±0.093 K；一百棵树平均的随机森林 1.349±0.047 K，均值降 18%、标准差降一半，**平均同时改善了准度和稳定性**；一百轮梯度提升 1.298±0.039 K，均值再降 4%、标准差再降 17%，这一次改善主要来自偏差。两者都优于单树，但路径不同：前者靠“多个近似独立的模型互相抵消随机误差”，后者靠“逐轮把系统性残差吃掉”。从特征重要性也能看出差别：随机森林把 0.963 的权重压在温升组合这一个特征上、阀位比流量只有 0.027，因为它靠少数强特征就能分裂得很干净；梯度提升的分布是 0.881 与 0.109，更分散，因为逐轮修正时次要特征的贡献会被反复拾起。

26.3.4 学习曲线的形状

样本量对误差的影响可以写成一个可用的经验式

$$E(n) \approx \sigma^2 + \frac{a}{n} + b \tag{26-22}$$

a 是方差项，与模型的有效参数个数成正比；b 是偏差项的下限，与模型族能否表达真函数有关。低复杂度模型 a 小、b 大，曲线平而高；

高复杂度模型 a 大、b 小，曲线陡而低。两条曲线必有一个交点，**交点左侧该选简单模型，右侧该选复杂模型**，交点的位置由样本量决定，不由模型本身决定。

本章的实测正好落在交点的两侧。样本量 50 时，14 参数的二次多项式测试均方根误差 5.657 K，随机森林 1.903 K，差了近三倍；样本量 900 时两者变成 1.130 K 与 1.227 K，多项式反超。前者是典型的“参数比样本还多”：50 个样本要定 14 个系数，每折交叉验证的训练集只剩 40 个，式 26-22 的 a/n 项把误差顶到了噪声下限的六倍以上。随机森林在这个样本量下更稳，因为每棵树只在自己重抽的子样本上分裂，叶子的样本数被结构性地限制住，等效于内置了正则化。

到 900 段时多项式反超，原因在 b 项：本章真函数在阀位与流量之比上是低阶光滑的，14 个特征已经能把它表达得很干净，多出来的误差只剩 1.130 K，已经逼近理论噪声下限 1.077 K；树的阶梯近似在这里变成了纯粹的额外近似误差，所以停在 1.227 K。**这条曲线给出的工程判据是：先看参数与样本的比值，再看真函数的光滑程度，两者都支持简单模型时不要为了“先进”去上集成**。

26.3.5 泛化差距的量级

前面四条性质各有各的公式，但工程上最想知道的其实是“训练集与测试集差多少算正常”。这个问题有一个粗但可靠的答案：泛化差距的量级由模型的有效容量除以样本量决定，即

$$|R_{\text{test}} - R_{\text{train}}| \lesssim O\left(\sqrt{\frac{C(\mathcal{F})}{n}}\right) \tag{26-23}$$

$C(\mathcal{F})$ 是模型族的容量，可以粗略理解为“能拟合多少种不同的函数”。这个式子不精确，常数因模型而异，但它给出的趋势极稳：容量翻倍，差距涨 1.4 倍；样本翻倍，差距缩 0.7 倍。

用它看本章的数字，结论立刻清楚。不剪枝树的容量最高，训练误差被压到零，测试误差 0.9653 对训练 1.0000，差距 0.0347；深度 6 的树容量小得多，差距缩到 0.0146；线性模型与多项式几乎没有容量冗余，训练 0.9846 对测试 0.9818，差距 0.0028，比树小一个数量级。**这三个差距就是式 26-23 中 $\sqrt{C/n}$ 的三种取值**，它们与模型在测试集上的实际名次并不一致——多项式差距最小、测试误差也最低，深度 6 的树差距中等、测试误差却更高——说明“差距小”只是必要条件，不是充分条件：偏差大的模型差距也小，但它的绝对误差高。

式 26-23 还能解释一个常被忽视的现象：为什么集成在样本充足时优势变小。随机森林的容量比单棵树大 B 倍，按式 26-23 泛化差距应该更大；但它实际更小，因为平均把方差压掉了（式 26-20）。**容量上界与方差是两个不同的东西，前者管“最坏情况”，后者管“平均表现”**，集成改善的是后者。在 900 段时，两者都离噪声下限不远，所以随机森林 1.227 K 与线性模型 1.130 K 的差距，反而是线性模型的偏差更低带来的。

这条性质还有一个实用推论：**不要用训练集上的误差下降速度判断模型好不好**。训练误差单调下降是必然的，它只能告诉你优化器在工作；唯一的判据是测试误差或交叉验证误差的拐点。

26.3.6 树为什么不稳定

同一批数据，换个随机种子重抽 90% 的样本，线性模型的系数只会小幅变动，树的形状却可能完全变样。这个现象叫“不稳定性”，它是树类模型方差的第二个来源——第一个来源是叶子里样本少（式 26-26），第二个来源是结构本身会变。

原因在生长的贪心性。树在每一层只挑“当前最优”的分裂，不考虑这个选择对后续的影响。如果顶层有两个候选特征的方差下降只差一点点，训练集的微小扰动就足以让它们交换位置；而顶层一旦换了特征，下面所有子树都要重新长。**误差不是线性累积的，而是指数放大的**：顶层错一次，整棵树的划分就全变了。这也是为什么树的深度越深、方差涨得越快——每一层都多一次“可能选错”的机会。

不稳定性有一个直接的可观测后果：单棵树的预测在相邻两次训练之间会跳变，而集成之后不会。随机森林的 bootstrap 重抽正是针对这一点设计的：每棵树看到的数据略有不同，它们“选错”的方向也各不相同，平均之后结构差异互相抵消，只剩共同的部分。式 26-20 里的 ρ 在这里有了具体的物理含义——**ρ 就是“不同树在多大程度上做出了相同的选择”**，特征越强、数据越干净，树的选择越一致，ρ 越高，平均能消掉的方差越少。

这条性质也解释了实验 A 里的一个细节：不剪枝树的训练误差是零、测试误差 1.633 K，而深度 6 的树是 0.9881 与 1.427 K。不剪枝的树每一层都在赌，赌赢的部分记住了训练集，赌输的部分在新数据上暴露；限制深度等于减少赌的次数，用一点偏差换回大量方差。

26.3.7 特征尺度与分裂偏向

线性模型对特征尺度敏感，因为系数的大小直接取决于单位：把流量从立方米每小时换成升每小时，对应系数会缩小一千倍，正则化惩罚的力度也随之改变。所以线性模型必须先标准化，这一步不是可选项。

树对尺度完全不敏感，因为它只比较“大于还是小于某个切点”，任何单调变换都不改变样本的排序，也不改变最优切点的位置。**这意味着树不需要标准化，也意味着树无法利用“某个特征的量纲更小所以更精细”这类信息**。工程上这条差别很实用：一批量纲混杂的特征喂给线性模型必须先标准化，喂给树可以直接用，省掉一步也省掉一类错误。

但树有另一种偏向：**取值多的特征更容易被选中**。一个只有两个取值的特征只有一个候选切点，一个有 900 个不同取值的特征有 899 个候选切点，后者在贪心搜索中“至少找到一个好切点”的概率更高，于是重要性被系统性高估。本章八个特征都是连续量，这个问题不明显；若混入“设备编号”“班次”这类类别特征，重要性排序会失真。处理办法是把类别特征做目标编码或限制切点数量，而不是直接喂进去。

这条偏向还与随机森林的特征子采样相互作用：若每次分裂只在随机子集里选，取值多的特征被选中的机会减少，偏向被部分抵消。本章的实现没有开特征子采样，所以读特征重要性时要注意，0.963 这个数里有一部分是“温升组合取值连续且分布宽”带来的，不全是物理贡献。

26.3.8 三种误差估计的偏差方向

同一个模型有三种误差，它们的偏差方向各不相同，混用就会得出错误结论。

训练误差系统性偏乐观，因为模型是看着这批数据调出来的。乐观的程度与容量成正比：不剪枝树的训练误差为零，测试均方根误差却是 1.633 K（约 2.667 K^2），乐观程度最大；线性模型的训练与测试决定系数只差 0.0028，乐观程度最小。所以训练误差只能用来判断优化是否收敛，不能用来比较模型。

交叉验证误差轻微偏悲观，因为每折的训练集比全量小。K 折时每折只用 $(K-1)/K$ 的数据，模型比最终上线的那个略差，估出的误差略高。这个偏差随 K 增大而减小：五折用 80% 的数据，十折用 90%，理论上十折更接近全量；但实验 D 里五折与十折的标准差都是 0.0008，说明在这个样本量下，偏差的改善被数据本身的随机性淹没了。**折数不是越多越好，超过一定程度只是浪费算力**。

测试误差是无偏的，但方差大，而且这个“无偏”只在测试集从未参与任何选择时成立。一旦用它挑过模型、调过阈值、筛过特征，它就变成了带选择偏差的量，系统性偏乐观，且偏差随挑选次数增长。这就是为什么 §26.4.8 强调嵌套交叉验证——把选择过程整个包进内层，外层只留一个从未被看过的估计。

三种误差的用法可以一句话概括：**训练误差看收敛，交叉验证做选择，测试集只报一次**。三者各司其职，任何一次越界使用都会让后面的数字失去意义。

26.4 离散对应与数值实现

连续时间的软测量问题落到代码上，要过三道关：把一段段运行数据摆成一张特征—标签表，把闭式解算得又稳又快，把树的生长与集成参数定在合理位置。三关都不难，但每一关都有一个容易写错的默认动作，错了以后误差不会报错，只会悄悄变好或悄悄变坏。本节逐关交代本章脚本的做法与理由。

26.4.1 数据表与折内标准化

采样之后，每一段准稳态数据是一行：8 个特征、1 个标签。4 个原始量（阀位、入口温度、流量、压力）直接来自传感器，4 个工程特征（阀位比流量、温升组合、阀位平方、流量平方）是同一行内的代数组合。这两类特征的处理方式不同：**行内组合不携带跨样本信息，可以在划分之前全局计算；跨样本统计量不行**。标准化用的是后者，

$$\mathbf{z}_i = \left[\frac{x_{i1}-\mu_1}{s_1}, \; \dots, \; \frac{x_{i8}-\mu_8}{s_8} \right]^{\mathsf{T}} \tag{26-24}$$

其中 μ_j 与 s_j 是训练折上的均值与标准差。若在全量上算，测试折的均值与方差就进入了训练过程，评估结果会偏乐观。

这类泄漏最隐蔽的地方在于它不改变任何一行数据的取值，只改变八个标量。实验 E 用一个与标签同源的量当第 9 个特征，把机制推到极致：决定系数从 0.9818 抬到 0.9973、均方根误差从 1.182 K 降到 0.460 K，误差凭空少了六成，而代码里没有一行看起来是错的。标准化泄漏的量级比这小得多，但成因相同。判断标准只有一条：**这个数在模型上线、只拿到一行新数据时算得出来吗？**算不出来就是泄漏。

26.4.2 线性代数的三条解法

求解 $(X^{\mathsf{T}}X)\boldsymbol{\beta} = X^{\mathsf{T}}\mathbf{y}$ 有三条路。Cholesky 分解最快，要求矩阵正定，代价是条件数被平方，精度最差；QR 先把 X 分解为 QR 再在 R 上回代，条件数不再平方；SVD 最慢但最稳，还能顺手把小奇异值截掉，

$$\hat{\boldsymbol{\beta}} = \sum_{j:\,\sigma_j > \tau\sigma_{\max}} \frac{\mathbf{u}_j^{\mathsf{T}}\mathbf{y}}{\sigma_j}\,\mathbf{v}_j, \qquad \tau = 10^{-12} \tag{26-25}$$

截断阈值 τ 是相对量，随数据尺度自动缩放。本章 $p = 14$，三条解法的结果在小数点后十位一致，选哪条都对；等到特征是上百个多项式的展开、p 上千时，Cholesky 会因为条件数平方而给出符号错误的系数，这时必须换 QR 或 SVD。岭回归在同一个框架里工作，只是把 σ_j 换成 $\sigma_j^2 + \lambda$，本章用 13 个对数间隔的 λ（10^{-3} 到 10^{3}）做留一交叉验证自动选，选出的模型与不加正则化的版本只差 0.002 K。

26.4.3 树的生长与叶子样本数

树不解方程，它的数值问题在分裂搜索：对每个候选特征排序后扫描 $n-1$ 个切点，单层代价 $O(pn\log n)$，整棵树 $O(pn\log n\cdot d)$。本章 900 行、8 列的数据，这个代价是毫秒级，所以真正的约束不是算力，而是叶子里的样本数——叶子输出的是落进去样本的均值，它的方差是

$$\operatorname{Var}\left(\hat{\mu}_{\ell}\right)=\frac{\sigma^{2}}{n_{\ell}} \tag{26-26}$$

深度 6 的树最多 64 个叶子，900 个训练样本平均每叶约 14 个，叶子均值的标准差约为 $\sigma/\sqrt{14}$；深度 12 的叶子数上界是 4096，已经超过样本总数，多数叶子只落进一两个样本，式 26-26 退化成 σ^{2} 本身。**这就是 §26.3.2 中深度 12 方差反升的数值解释**，也是“最小叶样本数”这个超参数存在的理由：它比限制深度更直接地给 n_{ℓ} 设下界。

26.4.4 集成的超参数

集成模型的超参数分三组：树的数量、单棵树的复杂度、学习率。提升类有一个常被引用的等效关系，

$$F_{M}\approx F_{0}+M\nu\,\bar{h} \tag{26-27}$$

它说明学习率与树数在总量上近似可交换，但只在局部成立——每轮拟合的是不同的残差，ν 减半并不能靠树数加倍精确补回，只能大致补回。本章取 100 轮、$\nu=0.1$、单树深度 3，得到 1.298±0.039 K；随机森林取 100 棵、不限制单树深度，得到 1.349±0.047 K。两者都远好于单棵深度 6 的 1.652±0.093 K，而超参数全部停在默认值附近——**在这个数据规模上，选对模型族比调参重要得多**。

26.4.5 分类阈值与类别不平衡

分类器的原始输出是概率，落到标签要经过一个阈值，

$$\hat{y}=\mathbb{1}\left[p(\mathbf{x})\geq\tau\right] \tag{26-28}$$

$\tau=0.5$ 只在两类先验相等、错分代价也相等时最优。本章把出口温度高于 70 摄氏度标为正类，正类占比约七成，全判正类的基线准确率就是 0.7233。逻辑回归把准确率提到 0.9667、曲线下面积 0.9962；一百棵树平均的随机森林是 0.9733 与 0.9958。**两个模型的面积只差 0.0004，准确率差 0.0066，说明排序能力已经到顶，剩下的差别全落在阈值附近的少数样本上**。这种情形下调整 τ 比换模型有效，而调 τ 的依据应当是错分代价，不是再刷一次准确率。

26.4.6 实现清单

把上面几节压成五条可执行的检查项。① 标准化参数、特征筛选、超参数搜索全部放进交叉验证的内层，任何跨样本统计量都不许在全量上算。② 上线前逐列问一遍“这列在只拿到一行新数据时算得出来吗”，实验 E 的 0.9973 就是没问这一句的后果。③ 特征数超过几百就不要写正规方程的显式求逆，改用 QR 或 SVD。④ 给树的叶子设最小样本数，它比限制深度更贴近方差的真实来源。⑤ 分类问题先看曲线下面积再看准确率，面积到顶时改阈值而不是换模型。

26.4.7 模型文件与上线接口

训练脚本跑完只是得到一个对象，能不能上线取决于它带没带够信息。一个可交付的模型文件至少要记五组字段。**第一组是数据指纹**：数据的时间范围、段数、随机种子、划分方式。没有这一组，半年后没人说得清这个模型是用哪批数据训的，也无法判断它是否已经过时。**第二组是特征定义**：八列的顺序、每列的单位、四个工程特征的构造公式。特征顺序错一位，模型不会报错，只会给出看似合理但完全错误的输出。**第三组是标准化参数**：每一列的均值与标准差，逐列存下来，不能在推理时重算。**第四组是模型超参数与代码版本**：树的数量、深度、学习率，以及训练脚本的提交号。**第五组是评估记录**：测试集指标、噪声下限估计、交叉验证的标准差。第五组最容易被省略，而它恰恰是日后判断“模型是不是退化了”的唯一基准。

推理接口要做三件事，每一件都对应一类现场事故。**输入校验**：检查八个输入是否在训练范围内，超范围不拒绝计算，但要打标记。**输出限幅**：把预测夹在物理可能区间内（本章数据是 40.7 到 85.5 摄氏度，留一点余量即可），防止个别异常输入把输出推到荒谬值。**外推标记**：用式 26-29 的杠杆值判断当前样本离训练数据云有多远，超过分位阈值就降级输出。这三件事加起来不到几十行代码，却能挡掉大部分“模型算出了一个不可能的数”的投诉。

还有一条常被忽略：**推理时的特征计算必须与训练时逐位一致**。同一段数据在训练脚本里算出的 u/q 与在推理服务里算出的 u/q，如果一边用了单精度、一边用了双精度，或者一边先滤波后相除、一边先相除后滤波，结果就会有小差别。单次差别不大，但在树的硬阈值分裂处，一点差别就足以让样本落进另一个叶子。工程做法是把特征计算抽成一个函数，训练与推理共用同一份代码，而不是各写一遍。

26.4.8 超参数搜索的预算

超参数一多，网格搜索的组合数会爆炸：树深度 12 档、最小叶样本 5 档、树数 4 档，乘起来 240 组，每组再套五折就是 1200 次训练。这套开销在 900 行数据上还撑得住，到了十万行就不可接受。控制预算有三条经验。

先定模型族，再定复杂度，最后定正则强度。模型族之间的差别是数量级的（本章 1.800 K 到 1.161 K），复杂度是次数量级的（1.427 K 到 2.558 K^2），正则强度往往只在小数点后第三位（1.161 K 与 1.159 K）。按这个顺序搜，前一步定下来的东西不会被后一步推翻，也不会把预算浪费在第三位小数上。

用随机搜索代替网格搜索。当只有两三个超参数真正重要时，网格搜索会把大量预算花在无关维度上，而随机搜索在同样的次数下能覆盖更宽的有效范围。本章只系统扫描了树深度一个参数，因为它是偏差—方差的主角；其余参数停在默认值，理由在 §26.1.6 已经交代。

嵌套交叉验证的开销是乘法，不是加法。外层五折、内层五折、13 个 λ，一次完整评估要训练 325 次。工程上的折中做法是：先用单次留出做粗略筛选，把候选压到两三个，再对最终候选做嵌套交叉验证；报告里写明“筛选用了留出、确认用了嵌套”，读者就能判断数字的可信度。**最不能接受的做法是拿同一个测试集反复筛选又反复报告**，那样得到的不是泛化误差，而是测试集的记忆。

26.4.9 批量重训与增量更新

模型上线后要跟着数据走，更新的方式有两种。批量重训是攒够新数据后从头训练一遍，简单、可靠、不留历史包袱；代价是每次都要重跑完整流程，且新旧模型之间可能出现跳变。增量更新只用新样本修正参数，代价小、响应快；但它只对参数化模型成立——线性模型有递推最小二乘的闭式更新（第 23 章的遗忘因子递推就是它的在线版本），树的增量更新则要重建结构，本质上仍是重训。

本章的模型里，只有线性与岭回归适合增量更新，树与集成只能批量重训。这个差别在选型时要考虑进去：如果装置需要“每天跟着工况微调”，线性模型的在线更新能力就是一项实际优势，而它 1.182 K 的精度与 1.159 K 的差距在这个场景下可以接受。

增量更新有一个必须防的陷阱：**遗忘**。只喂新数据会让模型逐渐忘掉旧工况，一旦装置回到旧工况就失效。标准做法是保留一个固定的“锚样本集”，每次更新都把它混进去；锚集的规模不必大，但要覆盖全部工况区间。这个做法与第 23 章带遗忘因子的递推最小二乘是同一个思想的两端——一端是“忘得快以跟踪变化”，一端是“留锚点以防漂移过头”，两者的平衡点由工况变化的速度决定。

26.4.10 特征冗余要不要删

本章的八个特征里有明显的冗余：u/q 与 $T_{\mathrm{in}}u/q$ 高度相关，u 与 u^2 在 0.30 到 0.95 的窄区间里近似单调，压力 p 由流量与阀位线性组合而成（数据生成时就是这么造的），信息几乎是重复的。冗余要不要删，取决于模型族。

线性模型与多项式**需要**处理冗余：共线会抬条件数，正则化是必要的对策，本章岭回归与普通最小二乘结果几乎相同（1.159 K 对 1.161 K）只是说明共线还没到病态的程度。树**不需要**：它在每一步只挑一个特征分裂，冗余特征只会让“挑中哪个”变得随机，不影响精度；随机森林甚至会因为冗余而受益——冗余特征提供了更多“随机看哪个”的选择，ρ 可以更低。

删特征的主要收益不是精度，而是可解释性与维护成本。八个特征里若有一个在实际装置上容易坏、容易漂，删掉它可以换来更长的无故障运行时间，代价可能只是 0.01 K。**这类权衡无法从测试集上看出来，必须问维护人员**——这也是本章反复强调“数字之外的判断”的原因。

26.5 工程含义与适用边界

本章的方法在纸面上都能收敛，落到装置上却常常“离线很好、投运就废”。原因不在算法，在四件事：软测量在控制回路里的位置放错了、数据质量本身就撑不起更高的精度、模型上线后没人管、以及被问到的工况根本不在训练集覆盖范围内。本节把这四件事逐条说清楚，并给出可执行的判据。

26.5.1 软测量在回路中的位置

软测量的输出是一个“算出来的温度”，它有三个固有缺陷：依赖输入变量的实时质量、依赖模型本身的有效性、以及没有独立的物理冗余。这三条决定了它不能放在快速内环里当反馈量。**软测量的合理位置是监督层与优化层**：给操作员显示、给先进控制提供被控变量的估计、给报警与性能评估提供参考。若一定要进闭环，应当串在慢回路里，并且保留一支真实测点做交叉校验——两支信号持续偏离就说明模型失效，此时必须切回真实测点。第 44 章的 PID 整定与第 58 章的 MPC 都把“测量可信”当作前提，软测量进入这两个回路之前，先要回答“模型失准时谁来兜底”。

从动态特性看，软测量的等效滞后由输入变量的采样周期与模型的计算周期共同决定，通常比真实热电偶还慢。热电偶的时间常数在一秒量级，而一段准稳态数据要攒几十拍才能算出一个稳定特征。**用慢信号去稳定快回路，结果只能是振荡**，这条在第 41 章的稳定裕度里已经量化过：附加滞后直接吃掉相位裕度。

26.5.2 数据质量决定精度上限

式 26-15 的第一项是噪声，它给所有模型划了一条谁都越不过去的线。本章的实测把这条线量了出来：用两百轮梯度提升的测试残差方差估计不可约误差，得到 1.406 K^2，折合均方根 1.186 K；而按数据生成时的两路噪声（入口温度一路 1.0 K、出口传感器一路 0.4 K）算出的理论值是 1.16 K^2、1.077 K。估计值比理论值高 21%，这个差额不是模型不够好，而是估计方法本身有偏：残差里还混着模型未表达完的结构误差。

这条线有三个直接后果。第一，**再换模型也压不到 1.0 K 以下**，实验 A 里最好的模型停在 1.159 K，已经在线上；第二，想进一步提高精度，唯一的路是降噪或增加信息量，不是调参；第三，汇报精度时必须同时报噪声下限，否则 “0.99 的决定系数” 会被误读成 “还差 1% 就完美”。工程上更常见的错误是反过来的：把一个 3 K 的误差归咎于模型选型，反复换算法，而真正的瓶颈是那支没有校准的热电偶。

标签的获取成本也是上限的一部分。温度可以测，成分、热值、燃烧效率往往只能靠化验，一天几个点。样本量上不去，式 26-22 的 a/n 项就压不下来，这时复杂模型不但没有优势，还会因为方差大而更差——实验 C 在 50 段样本上给出的 5.657 K 与 1.903 K 就是这个道理。

还有一类损耗不体现在样本量上，体现在标签的**时间对齐**上。化验值滞后两小时，若把它与两小时前的工况参数配对，模型学到的就是错位的映射；错位的程度不大时，误差不会明显上升，只会让所有模型的精度集体下降一截。**判断方法很简单：把候选滞后时间扫一遍，看模型误差在哪个滞后处最小**，最小的那个就是真实的对齐量。这一步不花钱，却能解释很多“怎么调都差一点”的项目。

26.5.3 上线之后的维护

模型投运只是开始。三类漂移会同时发生：输入特征的分布漂移（负荷区间变了）、传感器漂移（零点与量程）、以及被控对象本身的漂移（结垢、催化剂老化）。第一类可以用外推检测发现，杠杆值是现成的工具，

$$h_i = \mathbf{x}_i^{\mathsf{T}} \left(X^{\mathsf{T}} X \right)^{-1} \mathbf{x}_i \tag{26-29}$$

h_i 越大，这个样本越靠近训练数据云的边缘，预测的方差也越大。工程做法是给 h_i 设一个分位阈值，超过就标记“外推”，输出降级为“仅供参考”而不是直接给数。第二类与第三类要靠残差监控，只有在有真实测点做比对时才能做，所以**软测量系统必须保留至少一条低频的真实测量通道**，否则漂移永远发现不了。

监控的节奏按第 25 章给出的三级来：每班看一次输出是否在物理合理区间，每天看一次残差均值与方差，每季度用新数据重训并做一次留出验证。重训不是越勤越好，每次重训都要重新走一遍交叉验证，否则模型会在漂移中悄悄退化而没人知道。

26.5.4 适用边界与不适用清单

本章方法成立的前提有四条，缺一条就要换思路。**第一条是准稳态**：特征在时间窗口内近似不变，样本之间近似独立。快速启停、负荷大幅波动时，同一段数据内部就在变，用窗口均值会把动态过程抹平，预测必然滞后。**第二条是工况覆盖**：训练集必须包含投运后会遇到的工况区间，外推的可靠性没有任何保证，式 26-29 只能提示“不可信”，不能给出可信的值。**第三条是特征可得**：所有输入必须能在模型运行的同一时刻拿到，任何需要事后化验、事后计算的量都不能进特征。**第四条是标签可比**：训练用的标签与在线比对的测点必须同源同量程，否则模型学到的偏差会被误当成对象特性。

不适用的典型场景有四个：大滞后对象的超前估计（该用第 47 章的预估器思路）、强非线性且样本稀少的反应过程（该用机理模型加第 22 章的混合建模）、多工况频繁切换的过程（该用第 23 章的在线辨识或按工况分模型）、以及安全联锁相关的量（软测量不能替代独立的安全仪表功能，见第 36 章）。

26.5.5 选型判据

给定一批数据，选哪个模型不该由“先进程度”决定，而由三个量决定。**参数与样本的比值**：14 个参数对 900 个样本是安全的，对 50 个样本是危险的，实验 C 已经把这条边界画出来。**真函数的光滑程度**：本章的真函数在阀位与流量之比上是低阶光滑的，所以多项式在样本充足时反超集成；若真函数有阈值、饱和、分段特性，树的优势会立刻显现。**可解释性要求**：需要向工艺人员解释“为什么这个温度偏高”时，8 个特征的线性模型比一百棵树有用得多，后者的特征重要性只能给出“温升组合占 0.963”，给不出方向。

三条判据都指向简单模型时，不要为了“用了机器学习”而上集成。反过来，三条判据里只要有一条指向复杂模型，就不要硬用线性——**在真函数确实弯曲的地方，线性模型的偏差是结构性的，任何正则化都补不回来**。

26.5.6 一次落地该留下什么

软测量项目结束后，装置上留下的往往只有一个模型文件，几个月后就没人说得清它的来历。复盘至少要留下四样东西，每一样都对应本章的一个数字。

第一样是基线。基线不是“不用模型”，而是“用最笨的办法”。本章的 8 特征线性模型 1.182 K 就是一条合理的基线：它便宜、稳定、可解释，任何更复杂的模型必须先证明自己比它好。报表上只写“准确率 0.98”没有意义，写“比线性基线低 0.023 K”才有。

第二样是噪声下限。本章的 1.077 K 让所有讨论有了坐标系：1.159 K 的模型已经在线附近，再投入就是浪费；1.800 K 的模型离线上还有 0.72 K，值得改进。没有这个数，“精度 1.2 K 好不好”是一个无法回答的问题。

第三样是失效模式清单。模型会在三种情况下给错数：输入超出训练范围、输入缺失或卡死、对象特性漂移。三者的表现不同——第一种是杠杆值升高、第二种是特征方差突降、第三种是残差均值偏移——对应的处置也不同：降级、报警、重训。清单上要写清楚每种情况由谁判断、多久判断一次、判断后做什么。

第四样是用错的后果。软测量给出的温度如果偏 2 K，操作员可能多开一点蒸汽，后果是能耗上升；如果偏 10 K，可能触发联锁停车。**后果不同，精度要求就不同，投入也就不同**。这一条决定了前三条要做到什么程度，而不是反过来。

四样东西合起来是一句话：**模型的价值不在于它多准，而在于它准到什么程度是可信的、不可信时会发生什么**。这句话在本章的所有数字上都能成立，在后续更复杂的模型上同样成立。

26.5.7 纯数据模型与机理模型的混合

本章的算例有一个特殊性：真函数是已知的式 26-1。真实项目里这个式子往往写不出来，但也不该完全放弃——把已知的机理塞进模型，通常比纯数据模型省样本、也更稳。

混合有三条常见路径。**第一是特征工程**：把机理中的无量纲组合（本章的 u/q、温升组合）直接构造成特征。这一条本章已经用了，效果是决定系数从 0.9578 到 0.9818，值 0.024。它的成本最低、收益最直接，任何项目都应当先做。**第二是残差建模**：先用机理模型算一个基准值，再用学习器拟合残差。这样学习器只需要表达“机理没覆盖的部分”，容量需求大幅下降，在样本少时尤其有用。**第三是结构约束**：把单调性、饱和性、边界条件写成约束或先验，强制模型满足物理规律。这一条要求模型族支持约束，本章的多项式与树都不支持，第 27、28 章的神经网络可以通过结构设计部分实现。

三条路径的共同点是：**机理负责“形”，数据负责“细节”**。纯数据模型在样本充足时能自己学出“形”，但样本少时会学错；机理模型在细节上必然失真，但“形”是对的。工程上判断该走哪条，看的是“样本量与机理可信度之比”——样本少、机理可靠，就往机理那边靠；样本多、机理不清楚（比如反应过程的非线性机理），就让数据说话。

本章的数据可以用来做一个思想实验：如果只有 50 段样本，纯多项式是 5.657 K；若先用式 26-1 的机理形式把待定系数减到两个，同样的样本量下误差会降一个量级。**机理带来的收益往往比模型选型大一个数量级**，这也是为什么现场项目里工艺工程师的一句话常常胜过一天调参。

26.5.8 多个软测量共存时先做哪个

一个装置上通常有十几个想测而测不到的量：出口温度、成分、热值、露点、结焦程度。人力有限，先做哪个？三个判据相乘，得分高的先做。

用错的后果。偏 2 K 只影响能耗的量，优先级低；偏 5 K 会导致产品质量越界的量，优先级高；参与联锁的量，根本不该用软测量替代。这一项是筛子，不是打分项——后果严重的先排除不合适的方法，再谈优先级。

当前的不确定度。同一个量，如果运行人员靠经验估的误差只有 2 K，软测量做到 1.2 K，收益有限；如果靠经验估的误差有 8 K，软测量做到 1.2 K，收益巨大。**软测量替代的是“现在怎么知道这个值”，不是绝对精度**。这一条决定了收益的上限。

数据的可得性。输入量是否已经在 DCS 里、采样周期是否够、历史数据是否存了足够长时间，决定了项目能不能在几个月内做出第一版。数据不全的量，即使后果严重，也要先补测点。

三项合起来，优先级排序往往与“技术上最有趣的量”相反：最容易做出、收益最直接的那一个，通常不是最复杂的那个。本章的算例属于典型的第一梯队——输入量都在 DCS 里、标签有在线分析仪、误差要求 2 K 左右，正是软测量最容易见效的场景。

26.6 仿真与验证

本节用八组实验把前四节的论断逐一落到数字上。所有数字来自 `code/ch026_sim.py` 的单次运行（随机种子 20261026），六张图由 `code/ch026_figs.py` 生成，重跑逐位一致。

26.6.1 实验设置

被控对象是一个简化换热器：出口温度按式 26-1 由入口温度、热源温度、流量与换热系数决定，本章取热源 95 摄氏度、换热系数 2.4，两路噪声分别为入口温度 1.0 K、出口传感器 0.4 K，理论噪声下限 1.16 K^2。共生成 1200 段准稳态数据，每段一行八个特征、一个标签，按

900 比 300 划分为训练集与测试集。全部回归实验用同一划分，避免划分差异污染模型比较。

图 26-1 是数据的两个视角。左图把出口温度画成阀位与流量之比的函数，按入口温度灰度着色：曲面明显向 95 摄氏度饱和，且入口温度把整条曲线整体抬高，这正是式 26-1 中 $(T_{hot} - T_{in})$ 这一因子的作用。右图把线性模型的残差画在同一横轴上，残差不是零附近的随机散布，而是连成一条系统性的弯曲曲线——**线性模型缺的不是噪声项，而是饱和这一结构，这部分误差任何正则化都消不掉**。

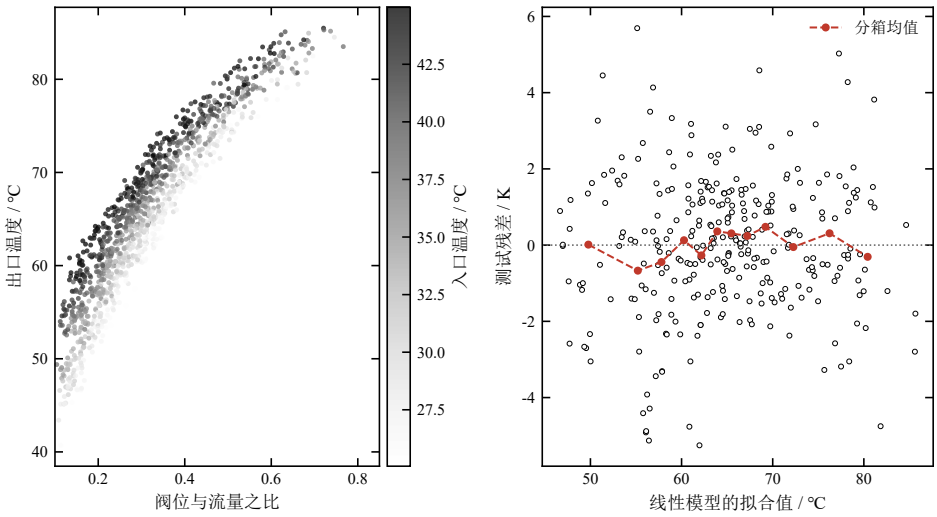


图 26-1　出口温度对阀位流量比的饱和特性与线性模型的残差结构

26.6.2 实验 A：九个模型的横向比较

第一组实验把九种模型放在同一划分上训练与测试，涵盖从四特征线性到一百棵树的全部复杂度层级。结果见表 26-2。

表 26-2　九个回归模型在测试集上的表现（900 训练 / 300 测试）

模型	训练 R^2	测试 R^2	测试 RMSE (K)	特征数
线性-4 原始	0.9659	0.9578	1.800	4
线性-8 特征	0.9846	0.9818	1.182	8
二次多项式	0.9847	0.9825	1.161	14
岭回归-二次	0.9846	0.9825	1.159	14
决策树-深度 3	0.9554	0.9447	2.062	—
决策树-深度 6	0.9881	0.9735	1.427	—
决策树-不剪枝	1.0000	0.9653	1.633	—
随机森林-100	0.9975	0.9803	1.229	—
梯度提升-100	0.9915	0.9819	1.179	—

表 26-2 里有五条值得单独拎出来的观察。第一，**四个工程特征值 0.024 的决定系数**：从 4 特征到 8 特征，测试均方根误差从 1.800 K 降到 1.182 K，降幅三成，而这四个特征只是阀位比流量、温升组合与两个平方项——它们补上的正是式 26-1 里的比值结构与饱和结构。第二，**多项式再加六个交叉项只换来 0.021 K**，说明主要结构在 8 个特征里已经表达完了。第三，**不剪枝树的训练决定系数是 1.0000，测试只有 0.9653**，两者之差 0.0347 就是方差项的可视化形态；同一族里深度 6 的训练 0.9881 与测试 0.9735 只差 0.0146，这棵树是这一族里唯一处在合理位置的。第四，**集成把树的测试误差拉回到线性模型的水平**：随机森林 1.229 K、梯度提升 1.179 K，后者与二次多项式的 1.161 K 只差 0.018 K。第五，**深度 3 的树 2.062 K 比 4 特征线性模型的 1.800 K 还差**，说明“非线性”本身不是优势——容量不够时，非参数模型连参数模型的基本趋势都表达不出来，这一条是选型时最容易被“树能拟合任意函数”这句话误导的地方。

图 26-3 把同一组结果画成横条：左图是训练与测试决定系数并排，右图是测试均方根误差。训练条与测试条的长度差一眼就能看出过拟合的程度，不剪枝树那一行两根条几乎不重合，而多项式与集成模型的两根条基本等长。

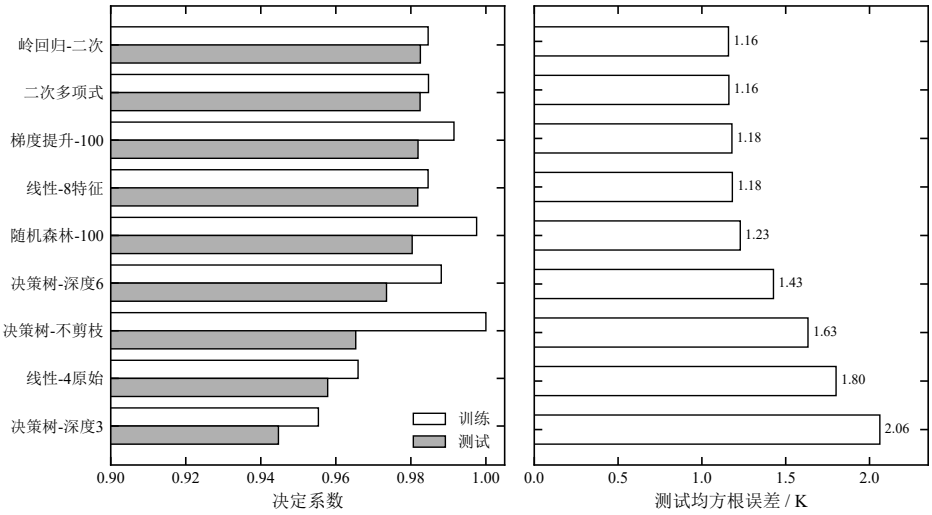


图 26-3　九个模型的训练与测试决定系数、测试均方根误差

26.6.3 实验 B：偏差—方差分解

第二组实验用 40 次重抽的 300 段子样本训练深度 1、6、12 三档的树，把预测均值与无噪声真实温度的差算作偏差、把各次预测的散布算作方差，结果见表 26-3。

表 26-3　树深度对偏差、方差与总误差的影响（40 次重抽，单位 K^2）

树深度	偏差平方	方差	噪声（理论）	三项合计	实测 MSE
1	23.610	4.080	1.160	28.850	29.214
6	0.216	1.130	1.160	2.506	2.558
12	0.264	1.641	1.160	3.065	3.150

表 26-3 的核心是中间那一列的反转。深度从 1 到 6，偏差平方降了两个数量级（23.610 到 0.216），方差只涨了不到三倍；从 6 到 12，偏差不再下降（0.264），方差却涨了 45%。总误差因此呈 U 形，最低点在深度 6 附近，2.558 K^2 折合 1.600 K。三项加噪声与实测 MSE 的差在 3% 以内，说明分解式 26-19 在这批数据上成立。

图 26-2 用对数纵轴把三条曲线画在深度 1 到 12 上：偏差平方先陡降、在深度 6 附近见底、再略回升；方差全程上升；总误差在这两者之间取到一个 U 形的最低点。图上还画了噪声水平的两条虚线——理论的 1.16 K^2 与估计的 1.406 K^2，**总误差曲线的最低点始终悬在噪声线之上，两者的竖直距离就是“这个模型族能做到的最好”与“数据允许的最好”之间的差距。**

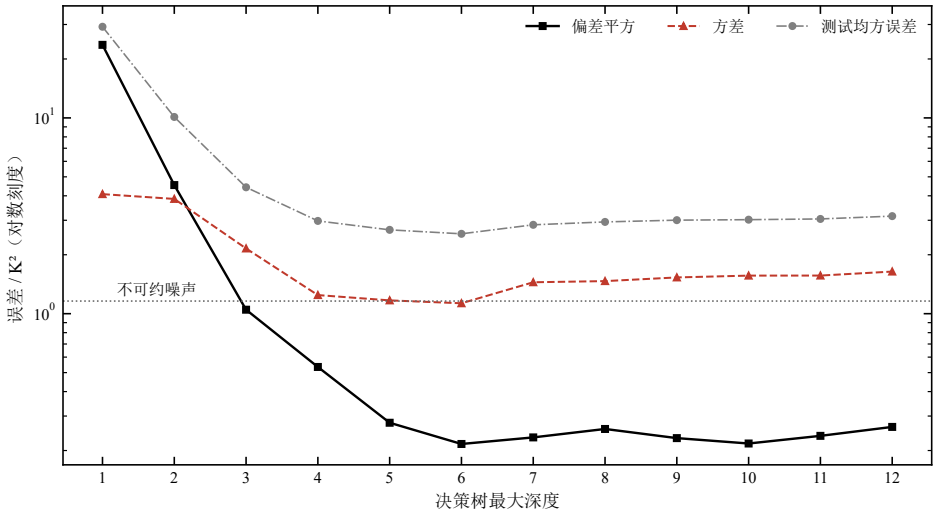


图 26-2　偏差平方、方差与测试误差随树深度的变化

26.6.4 实验 C：学习曲线

第三组实验把训练样本从 900 依次削减到 600、400、200、100、50，每个规模重抽 10 次，比较二次多项式与随机森林，结果见图 26-4。两个端点的数字最能说明问题：50 段样本时多项式 5.657 K、随机森林 1.903 K；900 段时变成 1.130 K 与 1.227 K。

图 26-4 的形态与式 26-22 一致：多项式那条曲线陡，左端冲到噪声下限的六倍以上，右端贴到 1.13 K；随机森林那条曲线平，左端只有 1.90 K，右端 1.23 K。两条曲线必在某处交叉，端点数据已经把这个交点夹在 50 到 900 段之间。**交叉点左侧该选树、右侧该选多项式**，这与 §26.3.4 的判据完全对应。图上还画了 1.077 K 的理论下限，多项式在 900 段时已经压到离它只有 5% 的位置，剩下的差距主要来自 14 个特征尚未完全表达饱和非线性。

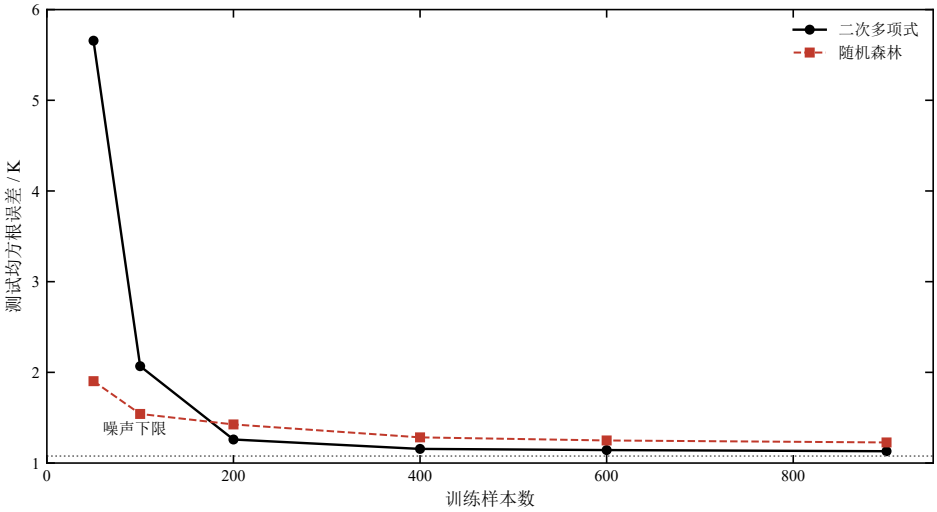


图 26-4　二次多项式与随机森林的学习曲线及噪声下限

26.6.5 实验 D：交叉验证的波动

第四组实验回答“报一个数够不够”。同一模型换 20 次随机划分，分别用单次留出、五折、十折估计决定系数，结果见表 26-4。

表 26-4　三种评估方式在同一模型上的波动（20 次重复）

评估方式	决定系数均值	标准差
单次留出（300 段）	0.9817	0.0017
五折交叉验证	0.9812	0.0008
十折交叉验证	0.9812	0.0008

表 26-4 说明两点。第一，**三种方式的均值只差 0.0005，但标准差差一倍**：单次留出的波动是交叉验证的两倍，因为它只用了一次划分，测试集本身的难易程度没有被平均掉。第二，**五折与十折的结果完全相同**，标准差都是 0.0008，说明瓶颈已经从划分方式转移到数据本身的随机性——再加折数只是多训练几次，不会让估计更准。工程结论是：样本量在千级时五折够用，报结果时带一个标准差，比多跑几次十折有意义。

有了标准差，比较两个模型才有判据。表 26-4 里三种方式评的是同一个模型，均值差 0.0005 还不到任何一档的标准差，这种量级的差别不能当作结论；换成两个不同模型也一样，若决定系数只差 0.002，而重复二十次的标准差是 0.0008，差值只有标准差的二点五倍，勉强算“有差别”，再小就该报“分不出来”。习惯写法是把估计值和一个标准差一起给出，例如 0.9812±0.0008，读到的人立刻知道多大的差距才算数；只报 0.9812 这个数，下游无法判断另一个模型的 0.9800 是不是真的更差。实验记录里因此要留下重复次数与随机种子，缺了这两项，标准差无从复算，比较就退回到看小数点后第几位。

还有一处常被忽略：用交叉验证挑超参数时，挑出来的那个分数本身偏乐观，因为每一折都参与了选择。要报一个干净的分数，得在外面再套一层划分，或者干脆留出一段从未参与任何选择的数据。前者叫嵌套交叉验证，代价是训练次数乘上折数；后者只要事先划好就不再动，实现更简单，适合样本紧张又要对外报数的场合。这与 §26.4.8 讲的预算是一回事：算力有限时，宁可少扫几组超参数，也要把重复次数做够，因为前者只影响选哪个模型，后者影响的是敢不敢下结论。一句话：先把波动报出来，再谈两个模型之间的差距。

26.6.6 实验 E：标签泄漏的量化

第五组实验在八个正常特征之外加入第 9 个特征，取值等于标签加上标准差 0.5 K 的噪声。这是一个典型的“事后可算、当时不可得”的量，用它来测量泄漏能把结果抬多高。结果见表 26-5。

表 26-5　标签泄漏前后的测试表现

特征集	测试 R^2	测试 RMSE (K)
8 个正常特征	0.9818	1.182
加第 9 个泄漏特征	0.9973	0.460

表 26-5 的对比是全章最有警示性的一组数字：误差从 1.182 K 掉到 0.460 K，降幅六成，决定系数逼近 1，而模型本身没有任何改进。**更危险的是这个结果在离线评估里完全“正常”**——交叉验证会给出同样漂亮的分数，因为泄漏在每一折里都成立。唯一的防线是逐列追问“这列在预测时刻算得出来吗”，这句话在 §26.4.6 已经写成清单第二条。图 26-6 的右图把这两根柱子并排画出，肉眼可见的高度差就是泄漏的“收益”，也是它的“代价”——上线后这六成会原样还回去。

26.6.7 实验 F：分类任务

第六组实验把连续的出口温度变成二分类标签（高于 70 摄氏度为正类），比较逻辑回归、深度 3 的树与一百棵树平均的随机森林，结果见表 26-6。

表 26-6　二分类任务的表现（300 段测试集）

模型	准确率	曲线下面积
全判正类（基线）	0.7233	0.5000（常数输出，按定义）
逻辑回归	0.9667	0.9962
决策树-深度 3	0.9667	0.9836
随机森林-100	0.9733	0.9958

表 26-6 读出三件事。第一，**基线与模型的差距只有 0.24 的准确率**，但曲线下面积从 0.5000 跳到 0.9962，说明真正的信息量远大于准确率数字给人的印象——这是类别不平衡下必须同时报面积的原因。第二，**深度 3 的树面积 0.9836 明显低于两个概率模型**，因为它的输出只有 8 个离散取值，排序粒度粗；但它的准确率与逻辑回归打平，说明在这个阈值附近，粗粒度并没有造成实际错分。第三，**逻辑回归与随机森林的面积只差 0.0004**，§26.4.5 已经解释过，这时该调阈值而不是换模型。

图 26-6 的左图留到下一小节，右图是上一节的泄漏对照。

26.6.8 实验 G：主成分与线性自编码器

第七组实验把八个特征标准化后用主成分降维，再用一个单隐层、线性激活的自编码器做同样的事，重构误差逐维比较。一维时主成分 0.45459、自编码器 0.45460；二维时 0.00307 与 0.00308；一到四维两者几乎重合——**式 26-13 的等价性在数值上成立**：线性自编码器学到的子空间与主成分子空间相同，只差一个旋转。

从五维开始两条曲线分开：五维 0.00189 与 0.00222，六维 0.00079 与 0.00236，七维 0.00016 与 0.00162。自编码器在高维没有继续下降，而主成分一路降到 0.00016。这不是模型差异，而是优化没收敛：梯度下降在 4000 轮、学习率 0.05 下只把前几个方向压到位，后面的方向梯度太小，残差停在 10^{-3} 量级。**这条分离曲线提醒的是训练细节，不是理论**——它也说明了为什么工程上做线性降维时应当直接用主成分：同样的解，闭式拿到，不用调学习率。

图 26-6 左图用对数纵轴画出这两条曲线：一到四维几乎重合，五维之后分叉，分叉的位置正是“能闭式求解”与“只能迭代求解”开始拉开的地方。

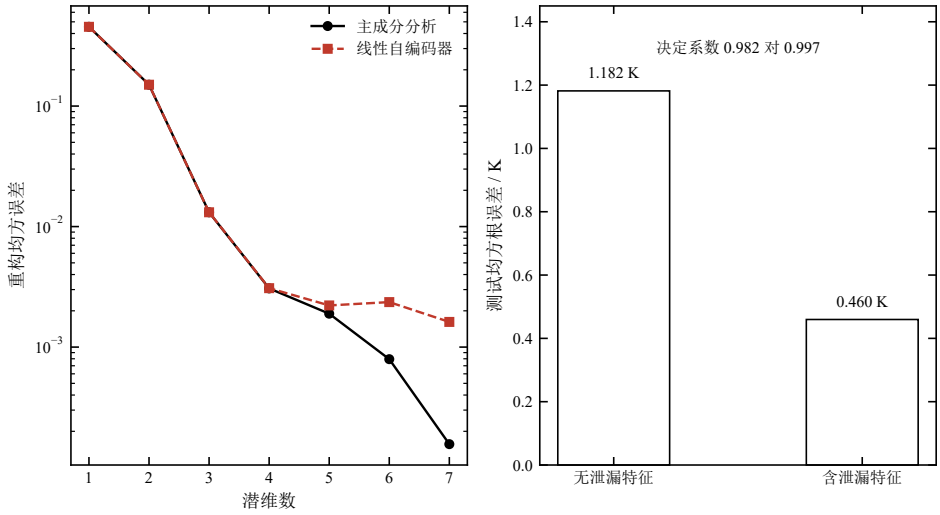


图 26-6　主成分与线性自编码器的重构误差、标签泄漏前后的误差

26.6.9 实验 H：集成的稳定性与特征重要性

第八组实验用 30 次重抽的 300 段子样本，比较四种树模型在同一测试集上的均方根误差分布，并取两百棵树的版本读特征重要性，结果见表 26-7。

表 26-7　四种树模型的测试误差分布（30 次重抽，单位 K）

模型	均值	标准差
决策树-深度 6	1.652	0.093
决策树-不剪枝	1.743	0.099
随机森林-100	1.349	0.047
梯度提升-100	1.298	0.039

表 26-7 的两个方向都在改善。均值从 1.652 K 降到 1.349 K 再降到 1.298 K，标准差从 0.093 K 降到 0.047 K 再降到 0.039 K，**后者比前者更能说明集成的价值**：单棵树的误差会随训练集的抽样在 1.56 到 1.74 K 之间摆动（一个标准差范围），集成之后这个范围收窄到 1.26 到 1.40 K，模型从“这一次运气好”变成“每次都差不多”。不剪枝树的均值比深度 6 还差 0.09 K、标准差也更大，这与 §26.6.2 的测试结果一致：方差项一旦失控，更多的分裂只会放大它。

图 26-5 左图是这四个分布的箱线图，四个箱体的位置与高度可以直接比较；右图是随机森林的特征重要性，温升组合 0.963、阀位比流量 0.027，其余六项接近于零。梯度提升的对应值是 0.881 与 0.109，分布更平坦——因为逐轮修正残差时，次要特征的贡献会被反复拾起。**两者的排序一致、集中度不同，这本身就是“平均降方差、累加降偏差”的一个侧面证据**。

读箱线图要看三件事：箱体的高度（四分位距）、须的长度（尾部极端值）与中位线的位置。箱体代表“典型波动范围”，须代表“偶尔出现的大误差”，中位线与均值的偏离代表分布是否偏斜。三项都看，才能判断“更稳”是真的还是只换了个样本。

按表 26-7 的均值与标准差粗算，深度 6 的区间是 1.56 到 1.74 K、不剪枝是 1.64 到 1.84 K，两者大幅重叠，说明在 300 段样本上分不出高下；随机森林 1.30 到 1.40 K 与梯度提升 1.26 到 1.34 K 只有边缘重叠，说明它们的优势基本是结构性的。**区间是否重叠，比均值差多少更能判断“这个改进是不是真的”**。

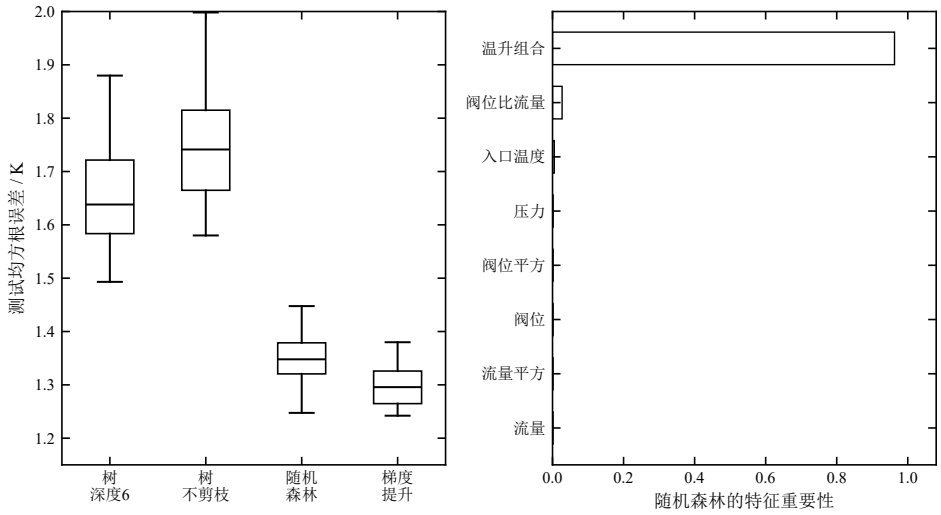


图 26-5　四种树模型的误差分布与随机森林的特征重要性

26.6.10 八组实验的合读

把八组实验放在一起，能读出四条互相印证的结论。第一，**误差有下限**：理论 1.077 K、估计 1.186 K，最好的模型 1.159 K，三者挤在一起，说明继续换模型的空间已经很小。第二，**复杂度的收益是分段的**：4 个工程特征带来 0.024 的决定系数，再展开二次项只带来 0.0007，而树的深度从 1 到 6 带来的是数量级的变化——**增益最大的地方永远是补结构，不是加复杂度**。第三，**集成改善的是稳定性而不只是均值**：1.652±0.093 K 到 1.349±0.047 K，标准差减半比均值降 18% 更有工程意义。第四，**评估方式本身就是结论的一部分**：交叉验证把波动从 0.0017 压到 0.0008，泄漏把误差从 1.182 K 压到 0.460 K，前者是可信度的提升，后者是可信度的崩塌，两者在报表上都表现为“数字变好了”。

这四条也划出了本章方法的边界：模型能逼近的上限由数据质量决定，模型之间的差别只在接近上限的那一小段里才有意义。下一章开始的深度学习把模型族换成了参数更多的形式，但式 26-15 的三项分解、式 26-22 的学习曲线、以及本节这八组实验的读法，一样适用。

26.6.11 数字之间的相互校验

八组实验里有几个数字初看互相矛盾，必须先解释清楚，否则读者会以为是笔误。

同一个深度 6 的树，在实验 A 里测试均方根误差是 1.427 K，在实验 B 里由 2.558 K^2 开方得到 1.599 K，在实验 H 里均值是 1.652 K。三个数不一样，原因是训练样本量不同：实验 A 用全部 900 段，实验 B 与 H 用的是 30 到 40 次重抽的 300 段子样本。**样本从 900 降到 300，误差从 1.427 K 涨到 1.6 K 上下，方向与式 26-22 一致**。三个数字不是矛盾，而是同一模型在不同数据量下的三个切面。

随机森林也遵循同样的规律：实验 A 用 900 段得到 1.229 K，实验 H 用 300 段得到 1.349 K。降幅比单棵树小，因为平均本身就在压方差，对样本量不那么敏感——这正是式 26-20 第二项 $\frac{1-\rho}{B}\sigma^2$ 在起作用。

反过来，实验 A 与实验 C 的端点必须严格对上：实验 A 的二次多项式 1.161 K 与实验 C 在 900 段处的 1.130 K 略有差别，因为实验 C 的每个规模都重抽了 10 次、报的是均值，而实验 A 是单次固定划分。**差 0.03 K 这个量级，恰好就是实验 D 测出的单次划分波动折算到均方根误差上的大小**。三组数字这样对一遍，才能确认整套实验是在同一套口径下跑的，而不是各写各的。

这条校验习惯值得带到任何项目里：**同一模型在不同实验里出现时，先确认样本量、划分方式、评估指标三项是否一致；不一致就先解释差异，再下结论**。表 26-8 的第一条与第四条讲的正是这类口径问题。

26.6.12 可复现性的三个随机源

本章声称“重跑逐位一致”，这需要三个随机源都被控制住，缺一个就不成立。

第一个是数据生成。采样噪声、特征扰动都来自随机数发生器，脚本用固定种子分别初始化，所以每次生成的数据完全相同。这一条最容易做到，也最容易被忽略——如果数据生成用了系统时间做种子，后续所有对比都失去意义。

第二个是数据划分。训练测试的划分、交叉验证的折、重抽的子样本，都依赖随机排列。脚本把这些种子全部固定，所以同一份代码在不同机器上得到同一组划分。工程上要注意的是：**划分种子与模型种子必须分开**，否则换模型时划分也跟着变，两个模型的差异里就混进了划分差异。

第三个是模型内部的随机性。随机森林的 bootstrap、特征子采样、梯度提升的行采样，都在训练过程中掷骰子。脚本给每个模型传了固定种子，并且把并行线程数设为 1（`n_jobs=1`）。后者常常被忽视：并行归约时浮点求和的顺序不固定，叶子均值的最末几位会变，落

到树的硬阈值上就可能改变一次分裂，进而改变整棵树。**要逐位复现，就得放弃并行**；不要复现、只要统计上稳定，才可以开多线程。

三个源都固定之后，还要固定库的版本。同一段代码在 scikit-learn 的不同版本上，默认参数与实现细节都可能变，例如树的默认分裂准则与随机森林的默认特征子采样比例。本章的所有数字对应的是一次具体版本的运行，读者若在自己的环境里重跑得到略微不同的结果，先查版本，再查线程数。

26.6.13 重复次数是怎么定的

八组实验的重复次数不一样：偏差方差分解 40 次、集成稳定性 30 次、交叉验证 20 次、学习曲线 10 次。这个安排不是随手定的，而是按“要估的量有多稳”来配。

偏差—方差分解要读出三个数的差异，其中方差项在深度 1 时是 4.080、深度 12 时是 1.641，差别不大，重复次数少了就分不开，所以取 40 次。集成稳定性要比较 0.093 与 0.047 两个标准差，本身就是在估计二阶矩，30 次刚够把标准差的相对误差压到一成左右。交叉验证要比较 0.0017 与 0.0008，两者差一倍，20 次已经足够分辨。学习曲线每个规模报的是均值，10 次就能把趋势画出来，因为端点之间差的是几倍而不是几成。

重复次数应当由“想分辨的差异”与“单次波动”之比决定，而不是统一取一个整齐的数字。经验规则是：要分辨的差异如果只有单次波动的三分之一，重复次数至少要到 30；差一倍以上，10 次就够。反过来，如果两组数字的差异小于重复次数能分辨的下限，就不该在正文里比较它们——那只是在比较噪声。

本章有一处恰好落在这条线上：随机森林与梯度提升的均值差 0.051 K，而两者的单次波动约 0.04 K，30 次重复下这个差别是可以分辨的；二次多项式的 1.161 K 与岭回归的 1.159 K 差 0.002 K，远小于任何重复次数能分辨的量级，正文只说“几乎重合”，不下“哪个更好”的结论。

26.7 小结与常见错误

本章从一个具体的工程问题出发——换热器出口温度测不到，只能用阀位、入口温度、流量、压力去算——把监督学习的骨架走了一遍。骨架只有四根：风险最小化给出目标，模型族给出候选，偏差方差分解给出取舍依据，交叉验证给出可信的分数。四根之外的算法细节都可以查手册，这四根不行。

26.7.1 要点回顾

目标一层：经验风险 $\frac{1}{n}\sum L(y_i,\hat{f}(\mathbf{x}_i))$ 是对期望风险的替代，替代的代价由模型族的容量与样本量共同决定（式 26-2、26-3）。求解一层：线性模型有闭式解，病态时靠岭回归垫高最小特征值（式 26-6、26-18）；树靠方差下降准则逐层分裂（式 26-10）；集成靠平均降方差、靠累加降偏差（式 26-20、26-21）。诊断一层：误差拆成噪声、偏差平方、方差三项（式 26-15），噪声是下限，偏差靠复杂度降，方差靠数据量与正则化降。评估一层：交叉验证估计的是训练流程而不是某个模型（式 26-16），泄漏能让这个估计彻底失效。

四条合起来是一个判断链：**先估噪声下限，再看模型离它多远，再判断差距是偏差还是方差，最后决定是加特征、加数据还是换模型**。本章的八组实验就是这条链的一次完整走查：噪声下限 1.077 K，最好的模型 1.159 K，差距 0.08 K，已经不值得再换算法；而 50 段样本时的 5.657 K 差距 4.6 K，问题在方差，该做的是攒数据。

26.7.2 常见错误

表 26-8 把本章反复出现的八类错误集中列出，每一条都对应一个可对照的数字。

表 26-8　机器学习应用中的八类常见错误

错误	典型表现	正确做法
在全量数据上标准化	交叉验证分数虚高	统计量只在训练折内计算
使用标签派生的特征	R^2 0.9973、RMSE 0.460 K	逐列追问“预测时刻算得出来吗”
只看训练误差	不剪枝树 1.0000 对 0.9653	一律报测试集或交叉验证结果
用测试集反复调参	分数偏高且不可复现	嵌套交叉验证，测试集只用一次
忽视噪声下限	把 1.18 K 归咎于模型选型	先估下限，再评价模型
小样本上用大模型	50 段时误差 5.657 K	先看参数与样本的比值
类别不平衡只看准确率	基线已有 0.7233	同时报曲线下面积与阈值
上线后不做监控	漂移长期无感	杠杆值加残差监控，定期重训

表 26-8 里最容易被低估的是第一条与第四条。它们不产生报错、不改变任何一行代码的写法，只让报表上的数字变好看；而“数字变好看”正是评估失效时唯一的表现。**判断一份机器学习结果是否可信，先看它有没有交代这三件事：数据怎么划分、标准化参数在哪一步算、测试集用了几次**。三件事交代不清，其余数字都不必看。

这八条还可以按“什么时候查”分成两组。第一、二、四、五条在建模阶段就能查：划分是否分组、特征是否可得、测试集用了几次、误差离下限多远，都不需要额外实验，读一遍流程就能判断。第三、六、七、八条要等结果出来才暴露：训练与测试的差距、小样本上的表现、准确率与面积的背离、上线后的漂移，都要有对照数字才看得见。**前一组靠流程纪律，后一组靠报表设计**——报表上不列训练误差、不列面积、不列样本量，后一组问题就永远发现不了。

一个实用的自查顺序是：先问“这个数是在哪批数据上算的”，再问“这批数据参与过选择吗”，最后问“这个数离物理下限还有多远”。三问过后，剩下的讨论才有意义。本章的每一张表都可以用这三问过一遍，读者不妨拿表 26-2 试一次。

26.7.3 与后续章节的接口

本章停在“模型能表达什么”这一层，没有回答“参数怎么训”。下一章把模型族换成多层网络，那里没有闭式解，只能靠梯度下降，第 28 章讲网络结构本身与训练工程，第 29 章则系统讲无约束优化的算法、步长上界与收敛速度。本章留下的三件工具会一直用到那里：式 26-15 的三项分解用来判断加深网络是降偏差还是加方差，式 26-22 的学习曲线用来判断该加数据还是加容量，式 26-16 的交叉验证用来保证报出来的分数是真的。

与控制在回路上的接口在本章已经点明：软测量的输出只能进监督层与优化层（§26.5.1），PID 与 MPC 分别在 第 44 章 与 第 58 章 讨论如何消费这个估计；模型失准时的兜底逻辑属于第 36 章的功能安全范畴。**把这三处接起来，一个软测量才算真正落地：算得准、说得清、失准时有人接**。

26.7.4 落地十问

最后给一份可以逐条打勾的清单。十个问题，任何一个答不上来，模型都不该投运。

① 基线是什么，复杂模型比它好多少？② 噪声下限估到了多少，最好的模型离它多远？③ 训练集与测试集是怎么划分的，有没有分组或分层？④ 标准化参数在哪一步算的，有没有跨折？⑤ 每一列特征在预测时刻都拿得到吗？⑥ 参数个数与样本量之比是多少？⑦ 误差是偏差为主还是方差为主，依据是什么？⑧ 重复抽样或换划分时，误差波动的标准差是多少？⑨ 输入超范围、输入卡死、对象漂移三种失效，各自的检测手段与处置动作是什么？⑩ 模型文件里有没有数据指纹、特征定义、标准化参数、超参数、评估记录这五组字段？

十个问题覆盖了本章的全部要点：前两问管目标与下限，第三到第五问管评估是否可信，第六到第八问管模型是否选对，第九、第十问管上线之后。**答不上来的问题，通常就是出问题的地方**；而能顺畅答完的团队，未必用了最复杂的模型，但一定知道自己的数字是什么意思。

26.7.5 与前后三章连起来看

本章处在“学习”这条主线的中间位置，前后三章的分工值得说清楚。

第 15 章回答“学习与控制为什么要放在一起”，给的是动机与谱系：辨识、模式识别、机器学习、强化学习各自解决哪一类问题。第 25 章回答“怎么把一段信号判成一个类别”，用的是人工特征加浅层分类器，评价是判对率与拒识率。本章回答“怎么从一个样本估计一个函数”，用的是统一的经验风险框架，评价是预测误差及其分解。第 27、28 章把模型族换成多层网络，第 29 章回答“参数怎么训”。

四者的关系可以这样概括：**第 15 章给地图，第 25 章给一个可用的工具，本章给一套判断工具好坏的尺子，后面三章给更强的工具**。尺子比工具更重要——本章列出的偏差方差分解、噪声下限、交叉验证、泄漏检查，在第 27 章之后一个都不会失效，反而会因为模型更大、更容易过拟合而更加关键。深度学习章节里“训练集很好、测试集崩了”的现象，用本章的语言就是方差失控；“加大模型没效果”的现象，就是已经贴到噪声下限。

与控制的关系也一样。第 25 章的判别器最终进的是故障诊断与工况识别，本章的回归器最终进的是软测量与前馈补偿，两者都停在“给控制提供信息”这一层。真正让信息进入闭环的，是后面第 44 章之后的控制器设计。**本章的任务是把信息做得可信，不是把回路做得更快**，这条边界守住了，全书的分工才清楚。

第27章 深度学习基础

27.1 动机与要解决的问题

27.1.1 特征工程的天花板

第26章的做法是“先造特征、再选模型”：把阀位除以流量、把入口温度乘上这个比值，凑出八列输入，再让线性回归、决策树、随机森林、梯度提升各试一遍。这套流程在换热器这个对象上能到 1.18 K 左右，离噪声下限 1.077 K 只差 0.1 K，看起来已经很接近极限。**但这份精度是贴着对象造的**：比值项来自饱和非线性的解析式，平方项来自阀门特性的经验，换了对象就得重造一遍。

真实的工厂里对象不止一个。一台加热炉有炉膛温度、氧含量、排烟温度三条软测量；一套精馏塔有塔顶、塔底、回流量三个回路；每条软测量背后的非线性形态都不一样。人工特征的成本随对象数量线性增长，而工况组合的数量随测点数量指数增长，**靠工程师逐个对象地猜组合，迟早跟不上**。

深度学习换了一个思路：不告诉模型“该看什么组合”，只给它足够的原始量测和一个可微的结构，让它在训练过程中自己学出中间表示。第一层学到的可能是“阀位偏大时温度对流量更敏感”这类局部形状，第二层把这些形状组合成饱和曲线，输出层再把它折算成温度。**“学表示”代替“造特征”，是这一章与第26章最根本的区别**；第26章的模型族是给定的，本章的模型族本身可以被训练塑形。

代价也要先摆清楚。第一，样本需求高：网络的参数动辄几千，而第26章的线性模型只有九个系数。第二，算力与工程栈更重：训练要 GPU 或至少多核 CPU，部署要额外的推理运行时。第三，可解释性下降：线性模型的系数能读出“阀位每增加 0.1，温度升高多少开”，网络的权重读不出这句话。第四，故障诊断更难：模型输出偏了，可能是数据漂移、可能是某一层饱和、也可能是权重文件版本不对。这一章的目的是把这四笔账算清楚，而不是宣布深度网络更好。

四笔账在本章这个具体任务上的数额可以先给出来，免得读者以为它们是抽象的顾虑。样本需求：4801 个参数用了 750 段训练样本，样本量少两成（600 段），测试误差就从 1.186 K 抬到 1.196 K，再少到 100 段就崩到 1.481 K。算力：单核 CPU 上整个脚本约五十秒，这条其实不成问题，**真正贵的是人**——调参、排障、版本管理都要投入熟悉深度学习的人。可解释性：线性模型的九个系数可以逐条与工艺经验核对，网络的 4801 个权重不能，只能退而求其次地看敏感度与部分依赖。诊断：一个输出偏差可能来自四五个环节，而线性模型只有“系数错了”和“数据超范围”两种可能。**这四笔账里只有第二笔能用钱解决**，其余三笔要靠工程纪律。

27.1.2 一个具体任务

沿用第26章的换热器出口温度软测量。可测的是阀位、入口温度、流量、压力，要估的是出口温度；准稳态关系为入口温度加上热源与入口温差乘以饱和因子，饱和因子对阀位与流量之比呈指数饱和。本章重新抽了 1000 段样本，其中 750 段训练、250 段测试；出口温度覆盖 43.0 到 86.5 ℃，均值 65.8 ℃；叠加的环境扰动与传感器噪声合计方差 1.16 K²，对应均方根 1.077 K。**这个 1.077 K 是任何模型都压不下去的硬底**，后文所有误差都要跟它对照着读。

输入八列：阀位、入口温度、流量、压力四个原始量，加上阀位比流量、温升组合、阀位平方、流量平方四个人工组合量。这里有一个刻意的安排——**八列里已经包含了四个人工特征**。深度网络本来不需要它们，保留是为了让线性模型这个对手处在最好状态：如果连“带着答案考试”的线性模型都赢不了网络，那网络的收益才站得住。

27.1.3 三个先导结果

第一个结果：精度上没有碾压。前馈网络 1.186 K，二次多项式 1.166 K，线性 1.203 K，梯度提升 1.294 K。四个模型落在 1.17 到 1.29 K 的窄带里，**最好与最差只差 0.13 K，不到噪声下限的两成**。在这个任务上，深度网络只是并列第一，没有出现“用了深度网络误差减半”的质变。原因不难找：对象的非线性是单调饱和，八个特征已经把主要形状表达出来了，网络没有多少剩余空间可挖。

第二个结果：加深不降误差。把网络从一层十六宽加到四层六十四宽，参数量从 161 涨到 13121，翻了八十倍，测试误差从 1.236 K 变成 1.199 K，中间还来回摆动；六层三十二宽反而最好，1.146 K。与此同时训练误差一路降到 0.916 K，测试误差不跟着降。**这就是第26章讲过的方差项，只是换了一个模型族**：容量越大越容易记住训练集的噪声，而测试集不认这份记忆。

第三个结果：序列任务上结构差异明显。同一个窗口预测任务，线性脉冲响应 0.0223，前馈网络 0.0194，一维卷积 0.0178，长短期记忆与自注意力都是 0.0190。**非线性结构把误差压低了两成，但三种非线性结构之间只差 7%**，而它们的参数量都在 1800 到 2700 之间。换句话说，起决定作用的是“能不能表达非线性”，而不是“用了哪种时髦结构”。

这三条先导结果给本章定了一个务实的基调：深度学习在工业软测量上的收益通常是百分之十几，不是数量级；真正决定成败的是数据质量、正则化与验证纪律，而不是网络堆了多深、用了多少注意力头。后面七节的任务，就是把这句话拆成可操作的判据。

还要补一句读法：这四个数都来自单次运行，不能按小数点后第三位去排名。前馈网络 1.186 K 与二次多项式 1.166 K 相差 0.02 K，落在 0.05 K 的门限之内，**正确的写法是“两者分不出来”，而不是“多项式以微弱优势胜出”**；梯度提升 1.294 K 与最好成绩差 0.128 K，超过门限一倍半，才可以下“它在这个划分上更差”的判断。**同一张表里的数字，能下的结论强度并不相同**，这是第26章反复强调的纪律，本章继续沿用。

27.1.4 与相邻各章的分界

第26章回答“给定一组特征，选哪个模型族、怎么评估”；本章回答“特征本身能不能被学出来，以及学出来的那个东西怎么训练”。两章共用同一份换热器数据、同一条留出划分纪律、同一个噪声下限，**但第26章的模型族是凸的或分段常数的，本章的模型族是非凸的**，这一条差异带来后面所有的麻烦：初始化、局部极小、梯度消失、对学习率敏感。

第28章接着讲网络的具体形态与训练工程：残差连接、归一化的位置、学习率调度、混合精度、分布式训练。本章只把最基础的构件搭起来——前馈、卷积、循环、注意力各讲一个最小可用的版本，目的是让读者能读懂第28章的改进在改什么，而不是替代它。

这条分界有一个直接后果：**本章不重复讲评估方法**。交叉验证怎么切、指标怎么选、两个模型差多少才算有差别，第26章已经讲完，本章只在开头复用那三条口径。同样，本章也不讲特征怎么造——八个输入里那四个人工组合量是第26章的遗产，本章保留它们只是为了给线性模型一个公平的对手。**读者若跳过第26章直接读本章，会缺两样东西：噪声下限的来历，以及 0.05 K 门限的来历**，这两样恰好是读懂所有实验表的前提。

第29章讲无约束优化算法，会推导梯度下降、动量、共轭梯度与拟牛顿法的收敛性质。本章会用到这些算法，但只把它们当成“给定的一步更新规则”，不重复推导：**凡是涉及“为什么这个算法收敛”的问题，一律指回第29章**，Adam 的自适应步长不在其中，本章仍按给定规则使用。第30章的凸优化则提供了对照：本章的实验里，二次多项式那个凸模型往往就是最强的对手，这个反差本身就是一条结论。

还有两条边界要说明。第一，本章不讨论图像卷积核的设计技巧，也不讨论大规模预训练与迁移学习，那些属于专门的视觉与基础模型教材。第二，本章的序列部分只讲结构，不展开系统辨识理论；脉冲响应、ARX、子空间方法见第16到22章。本章拿线性脉冲响应当序列任务的基线，正是为了把“结构收益”和“辨识收益”分开看。

27.1.5 四个问题与路线

本章围绕四个问题展开。**问题一：一个网络到底在算什么？** 它是多层复合函数，反向传播是链式法则的矩阵形式；把这两句话说清楚，后面的一切推导都是它的推论。**问题二：训练为什么会失败？** 三种典型失败——梯度消失、学习率不当、初始化尺度不对——都能用梯度范数与更新量的量级来诊断，本章给出可测量的判据。**问题三：怎么防止过拟合？** 权重衰减、丢弃、批归一化、早停四件工具各管一段，本章用 100 段小样本把它们逼到极限，看谁真的有用。**问题四：什么时候值得用深度学习？** 这个问题没有普适答案，但有可操作的边界：样本量、非线性强度、部署条件三条。

对应到节次：27.2 定义基本构件与记号，27.3 推反向传播、梯度消失、初始化与容量，27.4 落到张量形状、训练循环、数值稳定与部署，27.5 讲工程边界，27.6 用八组实验把前面的每条论断都钉在一个数字上，27.7 收束。

三条口径先立在这里。第一，**所有误差都是同一划分上的测试均方根误差，单位开尔文**，与第26章一致。第二，**结构对照时训练预算对齐**：同样的轮数、同样的批大小、同样的优化器，避免“某个结构多训了两百轮”这种不公平。第三，**单次运行、固定种子，差异小于 0.05 K 不当结论**；这个阈值来自第26章实验 D 测出的划分波动——决定系数标准差 0.0008 折算到误差约 0.03 K，这里取 0.05 K 作保守门限。凡是小于它的名次变化，本章都按“分不出来”处理。

27.1.6 本章的实验约定

这一节的目的是让读者能自己复现每一个数字，所以把与“模型好不好”无关、但会改变结果的琐碎设定先写清楚。

运行环境：单机 CPU，PyTorch 单线程（`torch.set_num_threads(1)`），随机种子固定为 20261027。单线程不是为了省事，而是为了让浮点累加顺序可重复——多线程下归约的加法次序每次可能不同，同一份代码两次跑出的误差在第三位小数上会抖，虽然不影响结论，却会让读者以为自己复现错了。

单次运行，不报标准差。真正的做法是换十个种子各训一次、报均值与标准差，但那是十倍的计算量，而本章要比较的结构有二十多个。折中的办法是用统一门限代替统计检验：把第26章测到的划分波动 0.03 K 放大到 0.05 K，只要两个模型的差值在这个门限内，就写“分不出来”。**这条约定比报一个好看的标准差更诚实**，因为它把“不敢下结论”明确写了出来。

误差一律用测试集均方根误差，不用决定系数。决定系数依赖输出方差，静态任务的输出跨 43.0 到 86.5 ℃，序列任务的标准差只有 0.65，两者的 R^2 不可比；而均方根误差的量纲就是输出量纲，静态任务能直接与噪声下限 1.077 K 对照，序列任务能直接与噪声 0.01 对照。**只有同量纲的数才能跟噪声下限做减法**，这是本章反复用平方相减核算欠拟合余量的前提。

结构对照的预算怎么给。给每个结构都扫到各自最优是不现实的，也会引入“谁的调参运气好”这个新变量。本章的做法是：前馈网络固定学习率 8×10^{-3} 或 10^{-2}，轮数在 150 到 400 之间按实验目的取；序列任务五种结构统一用 Adam、学习率 5×10^{-3}、批大小 128、

训练 40 轮。这套预算对卷积与循环未必最优，但它们之间的差距本就在门限附近，再调也改变不了“结构差异小于线性与非线性差异”这个结论。

最后一件不做的事：不追排行榜。本章不打算把换热器任务的误差压到最低，也不打算在 GPU 上试更大的模型。原因很简单——如果结论依赖于“多训两天”，那它就不是一条可以教给读者的规律。凡是需要精细调参才能成立的收益，本章一律标注为“未证实”。

27.2 定义与数学基础

27.2.1 前馈网络是复合函数

一个 L 层前馈网络就是 L 个仿射变换与 L 个逐元素非线性的交替复合。把第 ℓ 层的宽度记作 d_ℓ，输入 $\mathbf{x}\in\mathbb{R}^{d_0}$，则

$$\mathbf{h}_0=\mathbf{x},\qquad \mathbf{h}_\ell=\varphi_\ell\!\left(\mathbf{W}_\ell\mathbf{h}_{\ell-1}+\mathbf{b}_\ell\right),\qquad \hat{y}=\mathbf{w}^{\mathsf{T}}\mathbf{h}_L+b \tag{27-1}$$

其中 $\mathbf{W}_\ell\in\mathbb{R}^{d_\ell\times d_{\ell-1}}$，$\mathbf{b}_\ell\in\mathbb{R}^{d_\ell}$，$\varphi_\ell$ 是逐元素作用的激活函数。本章实验里“两层六十四宽”指的是 $L=2$、$d_1=d_2=64$，最后接一个标量输出；这个网络有 4801 个可训练参数，输入八维、输出一维。

这个 4801 可以手算核对，也值得手算一次，因为它把“参数量从哪来”讲清楚了。第一层 $8\times64+64=576$，第二层 $64\times64+64=4160$，输出层 $64\times1+1=65$，合计 $576+4160+65=4801$。第二层一项就占了八成七，这就是“参数量随宽度平方增长”的具体形态：宽度翻倍，层内参数变四倍。若把同样的容量摊成四层三十二宽，参数量是 $8\times32+32$ 加三个 $32\times32+32$，再加 $32+1$，合计 $288+3168+33=3489$——比两层六十四宽少两成七。这条算术是第 27.3.7 节“深比宽省参数”的原型。

这个定义看起来平淡，但已经包含了后面所有讨论的种子。第一，网络对参数是非线性的，对输入却是分段线性的（当激活取整流线性时）：固定参数后，$\hat{y}$ 是 $\mathbf{x}$ 的分段线性函数，折点由各层的加权和过零点决定。第二，层与层之间只有矩阵乘法，所以参数量随宽度平方增长、随深度线性增长，这一点在第 27.3.7 节会用来解释“为什么深比宽省参数”。第三，激活函数是把仿射变换变成非线性映射的唯一来源：去掉全部 φ，多层网络会塌缩成一个线性映射，这也是初学者最常见的实现错误之一。

关于表达能力，有一条常被误读的结论：单隐藏层网络可以以任意精度逼近紧集上的连续函数，只要宽度足够。这句话是对的，但它没有说宽度需要多大——对某些函数，所需宽度随输入维数指数增长。深度带来的不是“能逼近更多函数”，而是同样的函数可以用少得多的参数逼近，因为中间层的结果可以被后面多层复用。换热器的饱和非线性正好是这种可复用的结构：第一层学出“阀位与流量之比”这个中间量，第二层只需对这个中间量做一次一维非线性变换。

表 27-1 把本章用到的记号集中列出，后文不再逐条解释。

表 27-1　本章记号

记号	含义	本章取值
d_ℓ	第 ℓ 层宽度	8–64
L	隐藏层数	1–8
$\mathbf{W}_\ell,\mathbf{b}_\ell$	第 ℓ 层权重与偏置	可训练
φ	激活函数	整流线性 / 双曲正切 / S 型
B	批大小	128（静态任务为全批）
η	学习率	3×10^{-4}–3×10^{-1}
λ	权重衰减系数	0 或 10^{-2}
J	训练损失	均方误差

表格里最值得注意的是批大小一列：静态任务只有 750 段样本，全批更新一次只需几毫秒，所以本章的静态实验都取全批；序列任务有六千个窗口，才改用 128 的小批。批大小不是超参数里的装饰品，它决定了梯度的噪声水平与每轮的更新次数，第 27.4.3 节会给出选择依据。

27.2.2 反向传播

记第 ℓ 层的预激活为 $\mathbf{z}_\ell=\mathbf{W}_\ell\mathbf{h}_{\ell-1}+\mathbf{b}_\ell$，损失对它的灵敏度为 $\boldsymbol{\delta}_\ell=\partial J/\partial\mathbf{z}_\ell$。对输出层直接用链式法则，得到

$$\boldsymbol{\delta}_L=\frac{\partial J}{\partial \hat{y}}\odot\varphi_L'(\mathbf{z}_L) \tag{27-2}$$

对隐藏层，把第 $\ell+1$ 层的灵敏度经权重矩阵转置回传，再乘上本层激活的导数：

$$\boldsymbol{\delta}_{\ell} = \left(\mathbf{W}_{\ell+1}^{\top}\boldsymbol{\delta}_{\ell+1}\right) \odot \varphi_{\ell}'(\mathbf{z}_{\ell}), \qquad \ell = L-1, \dots, 1 \tag{27-3}$$

参数梯度则由灵敏度与本层输入的外积给出。对含 B 个样本的小批取平均：

$$\frac{\partial J}{\partial \mathbf{W}_{\ell}} = \frac{1}{B}\sum_{i=1}^{B} \boldsymbol{\delta}_{\ell}^{(i)} \left(\mathbf{h}_{\ell-1}^{(i)}\right)^{\top}, \qquad \frac{\partial J}{\partial \mathbf{b}_{\ell}} = \frac{1}{B}\sum_{i=1}^{B} \boldsymbol{\delta}_{\ell}^{(i)} \tag{27-4}$$

这三条式子说明反向传播不是一种新算法，而是链式法则在计算图上的执行顺序：先前向保存各层激活，再从输出往回逐层乘雅可比。**它的计算量与前向同阶**，因为每层只做一次矩阵乘法；代价在内存——要保存全部中间激活，显存占用随深度线性增长。这也是为什么第 28 章讲梯度检查点：用重算换显存。

式 (27-2) 到 (27-4) 是否正确，可以直接用中心差分验证。本章实验 D 在一个四层小网络上做了这件事：119 个参数里随机抽查 40 个分量，解析梯度与数值梯度的最大相对误差为 3.45×10^{-8}，中位数 1.11×10^{-9}。**这个量级正好是双精度舍入误差与差分截断误差的平衡点**，说明实现没有错；如果误差落在 10^{-2} 量级，通常是某一层的激活导数写错，或者忘了转置。

27.2.3 激活函数

反向传播里唯一逐元素作用的部分是 φ'，它的取值直接决定梯度能不能传回去。本章用到的三种激活及其导数如下：

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) \le \frac{1}{4}; \qquad \tanh'(z) = 1-\tanh^{2}(z) \le 1; \qquad \mathrm{ReLU}'(z) = \mathbb{1}[z>0] \tag{27-5}$$

S 型的导数上界只有四分之一，双曲正切的上界是一，整流线性在正半轴的导数恒为一。**这三个上界就是第 27.3.2 节梯度消失分析的唯一输入**：每回传一层，梯度最多被乘上一个不超过该上界的因子；八层下来，S 型最多衰减到 $4^{-8}\approx1.5\times10^{-5}$，双曲正切与整流线性则没有这个必然的指数衰减。

本章所有前馈实验默认取整流线性，只有在实验 C 里才把三种激活并排比较——**因为激活不是本章要比较的变量，它只是一个默认配置**。这样安排的好处是，实验 A、B、E、F、G 之间的差异不会被激活选择混淆。若要换激活，改动只有一行，但换完必须重扫学习率：不同激活的梯度量级不同，10^{-2} 在整流线性上合适，换成 S 型就明显偏大。**把激活当成可插拔的零件，同时把学习率当成与它配套的参数**，这是换激活时最容易漏掉的一半。

除了导数上界，三者还有两条工程差异。第一，**S 型与双曲正切都是饱和的**：输入绝对值大时导数趋零，权重再大也推不动；整流线性在正半轴不饱和，这也是它成为默认选择的直接原因。第二，**S 型不是零中心的**，输出恒正，导致同一层权重的梯度符号趋同，收敛走之字形；双曲正切零中心，但仍有饱和。整流线性的问题是负半轴梯度恒零，单元一旦落到“永远负”的区域就再也不更新，这叫死亡单元；泄漏整流线性把负半轴改成一个小斜率，本章实验 C 用的就是斜率 0.1 的版本。

27.2.4 输出层与损失函数

回归任务的输出层不加激活，损失取均方误差：

$$J_{\mathrm{MSE}} = \frac{1}{B}\sum_{i=1}^{B}\left(\hat{y}_i - y_i\right)^{2} \tag{27-6}$$

它的梯度是 $2(\hat{y}_i-y_i)/B$，与残差成正比，残差大时更新大、残差小时更新小，这正是第26章式 (26-4) 正规方程所对应的那个目标，只是这里用迭代而不是一次求解。**用均方误差训练的网络输出的是条件均值**，这一点与第26章“平方损失学条件均值”完全一致：如果噪声是重尾的，改用绝对误差会让输出转向条件中位数，代价是零点附近梯度不连续。

分类任务的输出层配 softmax，损失取交叉熵：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad J_{\mathrm{CE}} = -\sum_{k=1}^{K} y_k \log p_k \tag{27-7}$$

其中 y_k 是独热标签。把两式合起来求导，会得到一个极简的结果：对 logit 的梯度就是 $p_k - y_k$，即“预测概率减标签”。**这个形式与第26章逻辑回归的梯度 (26-9) 一模一样**，不是巧合：softmax 加交叉熵就是多类别逻辑回归，而逻辑回归是一个没有隐藏层的网络。本章实验只做回归，但式 (27-7) 会在第 36 章的故障分类里直接用到。

式 (27-6) 在实际训练里并不直接作用在开尔文上。输出会先按训练集的标准差缩放，损失在无量纲空间里计算，报告误差时再乘回去。这样做有两个好处：一是学习率不必随输出量纲变化，同一组超参数能在温度、压力、流量上通用；二是梯度的量级被归一化，避免了“输

出值域大、梯度就大”的隐性耦合。**代价是那个缩放因子 s_y 必须随模型一起保存**，漏掉它，推理结果会整体缩小或放大若干倍，而且误差曲线看起来仍然正常，只是数值全错——这是第 27.7.2 节把“标准化参数”单列一条的原因。

27.2.5 一维卷积

换热器的输入窗口是一段长度为 20 的序列，若把它展平后接全连接层，每个输出都要与 20 个时刻单独配一个权重，**窗口里“第 3 步与第 7 步的关系”和“第 13 步与第 17 步的关系”会被当成两件无关的事**。卷积把这条先验写进结构：同一个核在整段序列上滑动，权重共享。一维离散卷积写作

$$y_t = \sum_{j=0}^{K-1} w_j \, x_{t+j-\lfloor K/2 \rfloor} + b, \qquad t = 1, \dots, T \tag{27-8}$$

其中 K 是核长，本章取 5，即每个输出只看前后各两步。两个这样的卷积层中间夹一次两倍下采样，最后一层的感受野约为 13 步，已经能覆盖窗口里的大半个脉冲响应。**参数共享带来两个直接好处**：参数量从“窗口长度乘宽度”降到“核长乘通道数”，而且同一个特征在窗口内平移后仍然被识别。代价是卷积假设“同一种局部模式在任何位置含义相同”，对含有绝对时间标记的任务并不成立。

形状上还有一处必须交代：卷积会让序列变短。核长 5 不加填充时，每个位置少四个输出，两层下来二十步只剩十二步；本章统一用两端各补两步的填充，使输出长度保持二十，再做两倍池化降到十，最后对整段取平均压成一个通道向量。**填充方式决定了边界时刻的取值是否可信**，补零会让段首段尾的响应偏小，补边界值或镜像更合理；本章的窗口足够长，边界效应落在前两步与后两步，对结果影响可以忽略。

27.2.6 循环网络与门控

序列模型的另一种思路是不用窗口，而是把历史压进一个状态向量，逐步递推：

$$\mathbf{h}_t = \tanh\!\left(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{U}_x \mathbf{x}_t + \mathbf{b}\right) \tag{27-9}$$

这个式子的梯度要沿时间连乘 $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$，连乘二十步就是二十个矩阵的乘积，**与深层网络的梯度消失是同一个机制**，只是“深度”变成了时间长度。长短期记忆用三个门把这个问题绕开：遗忘门决定旧状态留多少、输入门决定新信息写多少、输出门决定当前读出多少，并且状态更新走一条加法路径 $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$。加法路径对梯度的雅可比是 $\mathbf{f}_t$ 而不是矩阵乘积，**只要遗忘门接近一，梯度就能沿时间近乎无损地传回去**。本章的长短期记忆取隐藏宽度 24、单层，参数量 2617。

这个 2617 同样可以手算：四个门的输入权重与循环权重是 $4 \times 24 \times (1 + 24) = 2400$，两套偏置各 96、合计 192，输出层 $24 + 1 = 25$，总数 $2400 + 192 + 25 = 2617$。**门控的代价是参数翻四倍**：同样宽度的普通循环层只有 $24 \times (1 + 24) + 24 = 624$ 个参数，长短期记忆是它的四倍多。省下的不是参数，是训练轮数——**用参数换可训练性，这是门控结构真正的交易**。

27.2.7 缩放点积注意力

注意力不递推状态，而是让每个时刻直接去“查”窗口内所有时刻：

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\!\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V} \tag{27-10}$$

$\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 由同一段输入经三个线性映射得到，$\sqrt{d_k}$ 的作用是把点积的方差归一到一，避免 softmax 进入饱和区。**注意力的计算量与窗口长度的平方成正比**，窗口 20 时只有 400 个权重，开销可以忽略；窗口拉到几千步时这一项就会成为瓶颈，这也是长序列模型要改稀疏注意力或分块的原因。本章用一个单层、两头的编码器，模型宽度 16、前馈 32，参数量 2593，与长短期记忆几乎相同——**这样比较的才是结构，而不是参数量**。

注意力的一个副作用值得提前说明：它对位置没有内在概念，同一段输入打乱顺序后，若不额外加位置编码，输出不变。本章的实现给每个时刻加了一组可学习的位置向量，长度固定为 20；**这也意味着窗口长度一变，模型就得重训**，工程上要提前把窗口定死。

27.2.8 正则化的三件工具

网络参数量远大于样本量时，训练目标要加一项惩罚：

$$J_{\text{tot}} = J_{\text{MSE}} + \frac{\lambda}{2} \sum_{\ell} \left\| \mathbf{W}_\ell \right\|_F^2 \tag{27-11}$$

它等价于给权重加一个零均值高斯先验，λ 与噪声方差成正比、与先验方差成反比，这条对应关系第26章式 (26-18) 已经推过。落到梯度上，每次更新相当于把权重先乘一个小于一的因子再减去数据梯度，所以叫权重衰减。**它对偏置不加惩罚**，因为惩罚偏置只会让模型更难拟合均值，没有正则收益。

丢弃的做法是在训练时以概率 p 随机把隐藏单元置零，推理时用全部单元但把输出乘以 $1-p$。若记掩码为 $\mathbf{m}$，则

$$\tilde{\mathbf{h}} = \frac{\mathbf{m} \odot \mathbf{h}}{1-p}, \qquad \mathbb{E}\left[\tilde{\mathbf{h}}\right] = \mathbf{h} \tag{27-12}$$

除以 $1-p$ 是为了让训练与推理的期望一致，否则推理时的激活会整体偏大。**丢弃等价于对指数多个子网络做隐式集成**，这一点第 27.3.8 节会用方差的语言再讲一遍；它的代价是训练轮数要增加，因为每个批只更新了一部分通路。

批归一化把每个小批内同一维的预激活拉回零均值、单位方差，再用两个可学习参数恢复尺度：

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad \tilde{z} = \gamma \hat{z} + \beta \tag{27-13}$$

它的直接效果是让每层的输入分布不随前层权重漂移，从而允许更大的学习率。**但批统计量在小批下噪声很大**：本章实验 F 里批大小为 100，测试误差反而从 1.481 K 升到 1.857 K，说明它不是无条件的好东西，第 27.4.5 节会讨论什么时候该用、什么时候该改用层归一化。

27.2.9 三种结构的共同骨架

卷积、循环、注意力看着是三套东西，其实都是式 (27-1) 的特例，差别只在“层内的线性组合怎么定义”。把它们放在同一副骨架上，后文就不必分别记三套语言。

卷积是共享权重、局部连接的层。同一个卷积核在窗口的每个位置复用，于是第 ℓ 层第 j 个通道的预激活是 $\sum_{k}\sum_{\tau=0}^{K-1} w_{jk\tau} h_{\ell-1,k}(t-\tau)$。与全连接相比，它把“每个输入位置各有一套权重”换成了“所有位置共用一套权重”，参数量从 $T \cdot d_{\ell-1} d_\ell$ 降到 $K d_{\ell-1} d_\ell$，代价是假设了平移不变——同一个波形在窗口的不同位置应当具有相同的意义。**这个假设在窗口预测里成立**：滞后三步的冲击与滞后五步的冲击，形状一样。

循环网络是沿时间展开的深层网络，且每层共用一套权重。把 $\mathbf{h}_t = \varphi(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b})$ 沿 t 展开二十步，得到的就是一个二十层、权重完全相同的前馈网络。理解了这一点，梯度消失就顺理成章：反向传播要穿过二十次同样的 $\mathbf{W}$，特征值偏离一的成分会被反复放大或衰减。门控的作用是在这条链上开一条加法通道，让梯度可以少乘几个矩阵。

注意力是数据相关的加权平均，权重由查询与键的内积经 softmax 给出。它没有固定的连接模式，每一对位置之间的权重都由输入决定；所以它能一步跨过整个窗口，代价是计算量随窗口长度平方增长。窗口二十步时 $T^2 = 400$，与卷积的 $KT = 100$ 同量级；窗口扩到一千步，前者是一百万，后者才五千，**这就是长序列必须换结构的原因**。

把三句话并排看：卷积固定了“谁与谁相连”，循环固定了“按什么顺序传递”，注意力把“谁与谁相连”也交给数据决定。**固定得越多，参数越省、样本效率越高；固定得越少，表达越灵活、样本需求越大**。本章实验 H 里五种结构的误差挤在 0.0178 到 0.0223 之间，正是因为窗口短、任务简单，三条路都够用；换个任务，这条取舍才会显形。

27.3 推导与性质

27.3.1 三层递推与一次反向

把式 (27-2) 到 (27-4) 串起来看，一次完整的训练由三段组成。前向：从输入逐层算到输出，保存每层的 $\mathbf{z}_\ell$ 与 $\mathbf{h}_\ell$。反向：从损失出发逐层算 $\boldsymbol{\delta}_\ell$，顺带算出各层梯度。更新：按优化器规则改权重。**三段的计算量都是 $O(\text{参数量})$**，反向因为要算两个矩阵乘（对输入的梯度与对权重的梯度）而比前向贵约一倍。这条“前向一比二”的经验值在估算训练算力时很好用：一次迭代的浮点运算量约为参数量的六倍（前向两倍加反向四倍）。

三段之间还有一个容易被忽略的依赖：反向必须用到前向保存的激活。若批大小为 B、第 ℓ 层宽度为 d_ℓ，保存的激活就是 $B\sum_\ell d_\ell$ 个数。**这解释了为什么“加深网络”在显存上的代价比参数量增长更快**：参数量随深度线性增长，激活量也随深度线性增长，但前者可以共享、后者必须全部留着。

显存不够时有一个成熟的折中：**梯度检查点**。不保存全部激活，只每隔若干层存一个，反向需要时把中间那一段重新前向算一遍。显存占用从随深度线性降到随深度的平方根，代价是多一次前向，训练时间约增加三成。**这是一笔典型的“用算力换显存”**，本章的网络只有一到八层、激活量不到一兆字节，用不上它；但它是第 28 章训练深层模型的必备手段，提前知道有这么一笔交易，读后面的内容就不会突兀。

27.3.2 梯度消失的定量界

设激活的导数绝对值不超过 ρ，即 $|\varphi'(z)| \le \rho$，则由式 (27-3) 反复代入可得

$$\|\boldsymbol{\delta}_{\ell}\| \le \|\boldsymbol{\delta}_{L}\| \left(\prod_{j=\ell+1}^{L} \|\mathbf{W}_j\|_2 \right) \rho^{L-\ell} \tag{27-14}$$

这个界说得很清楚：**梯度回传的衰减由两个因子相乘决定**，一个是权重矩阵的谱范数连乘，另一个是激活导数上界的幂。对 S 型，$\rho = 1/4$，八层就是 $4^{-8} \approx 1.5 \times 10^{-5}$；对双曲正切，$\rho = 1$，指数项消失，只剩权重范数在起作用；对整流线性，正半轴 $\rho = 1$、负半轴 $\rho = 0$，平均而言衰减介于两者之间。

实验 C 把这条件现在一个八层、三十二宽、Xavier 初始化的网络上量了出来。**S 型的第一层梯度范数只有输出层的 9.73×10^{-7}**，与 4^{-8} 同量级，说明界是紧的；双曲正切的首末比为 0.644，整流线性为 0.261，都没有指数级塌陷。逐层看，S 型的范数从 1.09×10^{-5} 单调升到 1.12×10^{1}，跨了六个数量级；双曲正切在 0.13 到 0.29 之间小幅波动；整流线性从 8.09×10^{-3} 升到 3.10×10^{-2}，只有四倍差距。

这里要提醒一句：**首末比小并不等于“这个激活不能用”**。S 型网络在浅层（三到五层）依然能训练，因为权重范数连乘这一项可以大于一，抵消一部分衰减；真正致命的是“深度大、初始化又偏小”的组合。反过来，整流线性虽然不衰减，但负半轴梯度为零，单元死亡后无法恢复，这是另一种失败模式。**两种失败要用不同的指标诊断**：前者看梯度范数随层号的变化趋势，后者看训练中零激活单元的比例。

式 (27-14) 还解释了残差连接为什么有效。把一层写成 $\mathbf{h}_{\ell} = \mathbf{h}_{\ell-1} + f_{\ell}(\mathbf{h}_{\ell-1})$，它对输入的雅可比就是 $\mathbf{I} + \partial f_{\ell}/\partial \mathbf{h}_{\ell-1}$，连乘项从“若干个可能很小的矩阵相乘”变成“若干个靠近单位阵的矩阵相乘”。**只要 ∂f_{ℓ} 的谱范数不大，乘积就能稳定在一附近，梯度不必逐层衰减**；反过来，若 f_{ℓ} 的雅可比已经很大，加法结构也会把爆炸放大，所以残差常与归一化配套使用。这条路在第 28 章展开，本章不实现残差，只把它记成“式 (27-14) 的工程解”。

27.3.3 初始化与方差传播

如果每层的输入方差是 v、权重独立同分布且均值为零、方差为 σ_w^2，则预激活的方差为 $d_{\text{in}}\sigma_w^2 v$。要让方差逐层不变，需要

$$\sigma_w^2 = \frac{1}{d_{\text{in}}} \quad \text{（前向不变）}, \qquad \sigma_w^2 = \frac{1}{d_{\text{out}}} \quad \text{（反向不变）} \tag{27-15}$$

两个要求一般不能同时满足，折中取调和平均，就是 Xavier 初始化：$\sigma_w^2 = 2/(d_{\text{in}} + d_{\text{out}})$。**它的作用是让信号在几十层里既不爆炸也不熄火**，从而把式 (27-14) 里的权重连乘项压在 1 附近，剩下的衰减只由激活导数决定。实验 C 的三个激活共用同一套 Xavier 初始化，正是为了把“初始化”这个变量固定住，只让激活函数变化——否则首末比里会混进初始化尺度的影响，结论就不干净了。

对整流线性，负半轴把一半的方差截掉，Xavier 会让方差逐层减半，所以实践中改用除以二的变体（He 初始化），方差取 $2/d_{\text{in}}$。**这条修正不是理论洁癖**：初始方差差一倍，沿二十层就是 2^{20} 倍的偏差，信号非炸即熄；本章的网络只有一到八层，两种初始化都能收敛，所以实验里没有单列这一项，但写进工具箱是必要的。

27.3.4 学习率与更新量

一次梯度下降的更新量是 $\Delta\boldsymbol{\theta} = -\eta \nabla J$，其大小由学习率与梯度范数共同决定。对二次型目标，若 Hessian 的最大特征值为 $\lambda_{\max}$，则发散的门槛是

$$\eta < \frac{2}{\lambda_{\max}} \tag{27-16}$$

超过这个门槛，沿着最陡方向的误差会逐步放大。**这个式子给出了扫学习率时的正确做法：先按数量级扫（每十倍一档），找到能收敛的最大档，再在它下面一倍处取工作点**。实验 E 的结果与这条判据吻合：学习率 3×10^{-1} 时测试误差 2.726 K，是线性模型 1.203 K 的两倍多；10^{-1} 降到 1.658 K；3×10^{-2} 到 10^{-2} 之间进入 1.18 到 1.20 K 的平坦区；再往下降到 3×10^{-4}，150 轮没训完，误差回升到 2.101 K。**最优点在 10^{-2} 附近，而不是越小越稳**。

Adam 这类自适应方法把每个参数的步长除以梯度二阶矩的平方根，相当于给每个方向单独配一个学习率，因此对 $\lambda_{\max}$ 不敏感。代价是它多引入了两个超参数（一阶、二阶矩的衰减率），并且在某些凸问题上泛化略差于调好的随机梯度下降。本章所有训练都用 Adam，学习率取 8×10^{-3} 到 10^{-2}，理由是**统一优化器才能让结构对照公平**；若换成随机梯度下降，各结构需要各自调参，比较就失去了意义。

式 (27-16) 里的 $\lambda_{\max}$ 通常不知道，工程上用一个更容易量的代理判据：**一次更新的相对幅度**。把 $\|\Delta\boldsymbol{\theta}\|/\|\boldsymbol{\theta}\|$ 记下来，健康的训练里这个比值在千分之一量级；到了百分之一，说明每步都在大改权重，模型会在谷底两侧来回跳；低于十万分之一，则是步子太小，几百轮也走不到。实验 E 里 3×10^{-1} 那一档，训练误差始终压不到 1 K 以下，正是第一种情形；3×10^{-4} 那一档 150 轮还没走到平底，是第二种。**这条比值不需要 Hessian，只要在优化器步进前后各记一次权重范数**，比推导特征值实用得多。

27.3.5 卷积的参数量与感受野

一层一维卷积的参数量是核长、输入通道、输出通道三者的乘积再加偏置：

$$N_{\text{conv}} = K\,C_{\text{in}}C_{\text{out}} + C_{\text{out}} \tag{27-17}$$

本章的一维卷积网络可以逐层核对：第一层 $5\times1\times16+16=96$，第二层 $5\times16\times32+32=2592$，最后接一个 $32\to1$ 的全连接层 33 个，合计 2721，与程序打印的参数量完全一致。**这个核对值得养成习惯**：参数量对不上，通常意味着某层的通道数写错，或者把偏置重复计了一次。

感受野的递推是 $r_\ell = r_{\ell-1} + (K_\ell-1)j_{\ell-1}$，其中 $j_\ell = j_{\ell-1}s_\ell$ 是第 ℓ 层的累计步长。第一层核长 5，感受野 5；一次步长为 2 的下采样后，累计步长变成 2；第二层再叠一个核长 5，感受野变成 $5+4\times2=13$，再把池化本身算进去是 14。**这条递推解释了为什么“堆卷积”比“堆全连接”省**：感受野随层数线性增长，参数量也随层数线性增长，而全连接层要让输出看到整个窗口，参数随窗口长度平方增长。

27.3.6 注意力的计算量与参数

式 (27-10) 的三次矩阵乘决定了注意力的开销。设窗口长度 T、模型宽度 d，则打分矩阵是 $T\times T$，计算量为 $O(T^2d)$，参数量约为 $4d^2$（查询、键、值、输出四个投影）。**与循环网络对比**：循环的计算量是 $O(Td^2)$，随长度线性增长，但无法并行；注意力是平方复杂度，却能把整段序列一次算完。长度 20 时 $T^2d=6400$、$Td^2=5120$，两者相当；长度 2000 时前者是后者的两百倍，这就是长序列必须改结构的原因。

本章的自注意力实现可以逐项核对：嵌入层 $1\times16+16=32$，位置编码 $20\times16=320$，编码层里四个投影 $3\times(16\times16+16)+(16\times16+16)=1088$，前馈两层 $16\times32+32+32\times16+16=1072$，两个归一化层 64，输出头 17，合计 2593。**与长短期记忆的 2617 只差 24 个参数**，这个巧合是刻意安排的：让两种结构的比较只剩“归纳偏置”这一个变量。

注意力还有一处与卷积、循环都不同的性质：**它的计算量里有一项与窗口长度的平方成正比**，而参数量里没有这一项。窗口 20 步时，配对权重只有 $20\times20\times2=800$ 个（两个头），与投影层的参数同量级；窗口扩到 200 步，配对权重变成 8 万，但参数量一点没变。**这就是“参数不多、算力却随序列线性方增长”的由来**，也是长序列模型必须做稀疏化或分块的根本原因。

27.3.7 为什么深比宽省参数

设要把 L 个宽度为 w 的简单变换复合起来。深层方案每层一个 $w\times w$ 矩阵，参数量是 Lw^2；单层方案要表达同样的复合，宽度至少要到 w^L，参数量是 w^{2L} 量级。取 $w=32$、$L=4$，深层是 4096，单层是 32^8 量级，差了六个数量级。**这就是“深度提供参数效率”的准确含义**：不是深度能表达更多函数，而是同样的函数用深度表达更省参数。

但省参数不等于省算力。深层的串行结构让每一步都依赖上一步，GPU 的并行度用不满；单层宽网络虽然参数多，矩阵乘却可以一次并行完成。**工程上的折中是“适度深、适度宽”**：本章实验 B 里，六层三十二宽用 5601 个参数拿到 1.146 K，四层六十四宽用 13121 个参数拿到 1.199 K，前者参数只有后者的四成，误差还低一点。这条结果与“深比宽省参数”的方向一致，但两者差距 0.053 K，刚好压在 0.05 K 的门限上，严格说只能算“不比它差”。

27.3.8 参数量不等于容量

第26章式 (26-23) 给出的泛化差距界可以原样搬过来：

$$R(f) \le \hat{R}_n(f) + O\!\left(\sqrt{\frac{C(\mathcal{F})}{n}}\right) \tag{27-18}$$

其中 $C(\mathcal{F})$ 是模型族的容量度量，n 是样本数。**关键在于容量不等于参数量**：同样是 5000 个参数，加了权重衰减的网络有效容量更小，加了批归一化的网络有效容量更大（因为它能拟合更复杂的输入分布），而丢弃在训练时随机删单元，等价于在指数多个子网络上做集成，有效容量介于两者之间。这解释了一个常见困惑：为什么“参数更多的模型反而不容易过拟合”——只要正则化把有效容量压住了，参数量本身不是罪魁。

实验 F 把这条道理推到了极端：训练样本只有 100 段，两层六十四宽的网络有 4801 个参数，是样本数的四十八倍。无正则时训练误差 0.480 K、测试误差 1.481 K，差距整整 1.001 K；加权重衰减后训练误差升到 0.947 K、测试误差降到 1.247 K，差距缩到 0.300 K。**训练误差变差、测试误差变好，这是正则化生效的标准信号**；如果两者同时变好，说明原来的模型不是过拟合，而是欠拟合，加正则只会帮倒忙。

27.3.9 参数量相近，计算量可以差一倍

参数量决定存储与样本需求，计算量决定单次推理的耗时，两者不是一回事。本章三种序列结构的参数量恰好挤在一起：一维卷积 2721、长短期记忆 2617、自注意力 2593，最大只差 5%；但按窗口二十步逐层数乘加，差距立刻拉开。

一维卷积每层每个位置只做 K 次乘加：第一层 $20\times16\times5=1600$，池化后第二层 $10\times32\times(5\times16)=25600$，输出层 32，合计约 2.7 万次乘加。**权重在二十个位置上复用，所以每个参数被用了约十次。**

长短期记忆每步要算四个门：$4\times24\times(1+24)=2400$ 次乘加，二十步就是 4.8 万次；输出层再加 24 次。**同一套权重被用了二十次，每个参数约十八次乘加。**

自注意力没有复用：嵌入 320、查询键值投影 $3\times20\times16\times16=15360$、两头的相似度与加权求和各 6400、输出投影 5120、前向层两段合计 20480，加起来约 5.4 万次，**每个参数约二十一次。**

三条路线的比值是 $2.7:4.8:5.4$。**参数量几乎相同，最慢的比最快的多一倍算力**，原因全在“权重复用了几次”：卷积在空间上复用、循环在时间上复用、注意力靠矩阵乘法一次算完所有配对，没有可复用的维度。放到控制器上，这个差别会被放大——如果推理跑在实时核里，选结构时看的是乘加次数，不是参数量。

反过来说，**感受野决定“多久以前的输入还在起作用”**。卷积两层级联加一次池化，感受野约十三步；循环虽然看过全部二十步，但门控会让远期信息衰减；注意力是二十步全连接，每个输出都能直接看到窗口开头。**若工艺上的纯滞后超过二十步，三种结构都无能为力**，此时该做的是加长窗口或换用状态空间模型，而不是在结构上继续纠结。

27.4 离散对应与数值实现

27.4.1 数据表与标准化

网络的输入是浮点数矩阵，而现场数据带着各自量纲：阀位在 0.30 到 0.95 之间、入口温度在 25 到 45 ℃、流量在 1.2 到 3.0 m³/h。若直接喂进去，梯度会被大量纲的那一列主导，训练要么极慢要么发散。标准化的写法是

$$x'=\frac{x-\mu}{\sigma},\qquad y'=\frac{y-\mu_y}{\sigma_y} \tag{27-19}$$

其中 μ,σ 只由训练集估计，测试集与线上推理复用同一组值。**这条纪律与第26章折内标准化是同一条**：如果用全样本统计量，测试集的信息就漏进了训练，评估会偏乐观。输出也做标准化，是为了让损失的量级不随标签量纲变化，学习率不必每换一个对象就重扫一遍；预测完成后再乘回 σ_y 加回 μ_y，报告误差时单位仍是开尔文。

标准化参数必须随模型一起保存。**工程上最常见的低级错误是训练时标准化、上线时忘了**，模型输出会整体偏一个常数，而在线诊断往往先怀疑传感器，白白排查几天。第 27.4.8 节会把这件事写进部署清单。

还有两个细节值得写进代码。其一是**常量列**：某个特征在训练集里几乎不变时，σ 接近零，除法会把数值放大成噪声；实现里统一加一个极小的下限，本章取 10^{-12}，让这类列变成零输入而不是无穷。其二是**顺序**：先减均值再除以标准差，与先除再减在数学上不等价，训练与推理必须用同一种顺序，否则同样的参数会给出不同的结果。这两条都不会让训练报错，**只会在上线后表现为“某个工况下莫名其妙偏一点”**。

27.4.2 张量形状与批

深度学习的调试有一半时间花在形状上。表 27-2 列出本章两个任务里各结构的输入输出形状，**每一行的第一维都是批大小**，这是把单样本公式扩展成批处理的唯一改动。

表 27-2　各结构的张量形状

结构	输入形状	输出形状	本章实例
前馈（静态）	$B\times8$	$B\times1$	750×8
前馈（序列）	$B\times20$	$B\times1$	6000×20
一维卷积	$B\times1\times20$	$B\times1$	通道维在中间
长短期记忆	$B\times20\times1$	$B\times1$	时间维在中间
自注意力	$B\times20\times16$	$B\times1$	先嵌入再编码

表里最容易被忽略的是卷积与循环对通道维、时间维的位置要求不同：卷积要“批、通道、长度”，循环要“批、长度、特征”。**同一个窗口数据喂给两种结构，得先转置一次**；忘了转置不会报错，只会让长度与特征错位，训练照样收敛，但学到的东西完全不对。这类错误用形状检查查不出来，只能靠“先在小样本上过拟合”来发现——如果模型连十个样本都拟合不了，形状多半错了。

27.4.3 训练循环的七步

一个完整的训练轮次可以拆成七步，顺序固定：清零梯度、前向、算损失、反向、更新、记录指标、切换训练/推理模式。第三步到第五步是核心，其中更新一步对 Adam 写作

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \qquad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\mathbf{m}_t/(1-\beta_1^t)}{\sqrt{\mathbf{v}_t/(1-\beta_2^t)} + \epsilon} \tag{27-20}$$

$\mathbf{m}$ 是梯度的一阶矩（动量），$\mathbf{v}$ 是二阶矩（尺度的平方），分母里的 $1-\beta^t$ 是偏差修正，防止训练初期矩估计偏小。这三个部件各管一件事：动量抹平批噪声，二阶矩把不同参数的步长归一，偏差修正让第一步不至于走得太小。本章取 $\beta_1 = 0.9$、$\beta_2 = 0.999$、$\epsilon = 10^{-8}$，与主流默认值一致。

批大小的选择有一个简单判据：每轮更新次数不要少于几十次，否则一个轮次内的梯度噪声太大。静态任务 750 段取全批，每轮只更新一次，但轮数给到 300 到 400，总更新次数足够；序列任务 6000 个窗口取批 128，每轮 47 次更新，40 轮共 1880 次。两种配置的总更新次数都在千次量级，这也是本章能把不同结构放在同一预算下比较的原因。

轮数不是越多越好。实验 G 的曲线显示，验证误差在第 93 轮见底（1.112 K），之后缓慢回升；训满 400 轮时测试误差 1.226 K，比早停的 1.196 K 差 0.03 K。这 **0.03 K** 不大，但它是不花钱就能拿到的：早停只需要在每个轮次结束后多算一次验证误差。

七步里最容易漏的是第一步“清零梯度”。框架默认把梯度累加，忘了清零不会报错，只会让这一步的更新量等于前几步之和，效果相当于把学习率乘上了一个随机倍数。这类错误的表现是“训练能收敛，但比预期慢且抖”，很难从损失值上看出来，只能靠代码审查。反过来，累加这个特性在某些场合是故意用的——显存不够时把一个大批拆成几个小批依次反传、最后统一更新，靠的就是不清零；本章没有用这种技巧，所以每一批都必须清零。

27.4.4 学习率调度

固定学习率能用，但不一定最快。常用三种调度：分段衰减（每到若干轮乘 0.1）、余弦退火（从初值平滑降到零）、预热（前几百步从很小的值线性升到初值）。预热是深层网络的关键技巧：训练开始时权重随机，梯度方向不可信，大步长会把网络推到坏区域；先用小步长走几百步，等统计量稳定再放大。本章的网络只有一到八层、样本量小，预热收益不明显，所以没有启用；第 28 章的深层网络则必须开。

调度与早停有一个交互要注意：余弦退火会让验证误差在末端再次下降，如果此时早停，可能停在退火结束前；正确做法是把调度周期与总轮数对齐，或改用“验证误差连续若干轮不降”的判据。本章实验 G 用的是后者，耐心取 0，即严格取最小值所在的轮次，因此第 93 轮之后的小幅波动都不会触发新的最优。

批大小改变时，学习率不能照搬。经验规则有两条：线性缩放，批放大 k 倍就把学习率放大 k 倍；平方根缩放，批放大 k 倍把学习率放大 $\sqrt{k}$ 倍。前者适用于小批、后者适用于大批，实践里常用前者起步、用后者兜底。本章两个任务正好是两种批量：静态任务全批 750，序列任务批 128，前者的学习率给到 8×10^{-3} 到 10^{-2}，后者只给 5×10^{-3}。批量差了约六倍，学习率只差不到两倍，落在两种缩放规则之间——这是经验折中，不是推导结果；真要严格照规则来，静态任务该给到 3×10^{-2} 以上，实验 E 显示那个档位已经开始不稳。

27.4.5 正则化的实现细节

权重衰减在优化器里设置，作用于全部可训练参数；若要排除偏置与归一化参数，需要分组设置参数组，本章的实现没有分开，这也是实验 F 里“衰减加丢弃”把训练误差推到 **1.440 K** 的原因之一：惩罚过强，模型连训练集都拟合不好。

丢弃的两套行为必须靠模式开关切换。训练时按式 (27-12) 随机置零并放大，推理时用全部单元且不放大。忘了切换模式是第二个常见错误：推理时若仍在丢弃，同一段输入每次输出都不同，现场会以为模型坏了。批归一化同理，训练时用批统计量并更新滑动平均，推理时用滑动平均；批大小为一时训练直接报错，因为方差无法估计。

批归一化还有一个前提常被忽略：它假设一个批内的样本近似独立同分布。软测量的样本是按时间顺序采的，一个批里的样本高度相关，批统计量会比真实分布窄，训练与推理的差异随之变大。本章实验 F 里批大小 100 全是邻近工况，正是这种情形，测试误差从 1.481 K 升到 1.857 K 也就不奇怪了。时序任务改用层归一化（对每个样本自己归一）能避开这个问题，第 28 章会展开。

27.4.6 数值稳定

softmax 与交叉熵在实现时要先减最大值：

$$p_k = \frac{e^{z_k - z_{\max}}}{\sum_{j} e^{z_j - z_{\max}}}, \qquad \log \sum_j e^{z_j} = z_{\max} + \log \sum_j e^{z_j - z_{\max}} \tag{27-21}$$

不减最大值时，z 稍大（比如 800）就会让 e^z 溢出成无穷，反向传播得到非数。减掉之后最大项是 $e^0=1$，其余都小于一，既不上溢也不下溢。**这条改写不改变数学结果，只改变计算顺序**，但在浮点里是生死之别。

另外两处要防的是除零与对数零。批归一化的分母加了 ϵ，交叉熵里的 $\log p$ 要对 p 做下限截断；梯度爆炸时用范数裁剪，把整个梯度向量按比例缩到阈值以内，**裁剪不改变方向，只改长度**，比重设学习率更温和。本章的损失远小于一，没有触发裁剪，但这段代码应当常备。

ϵ 的取值本身也是个折中。Adam 的分母里放 10^{-8}，是为了在二阶矩接近零时不让步长变成无穷；批归一化里通常放 10^{-5}，因为那里除的是标准差，量纲与激活同阶，太小会让方差极小的维度被放大成噪声。**两个 ϵ 差三个数量级不是随意写的**：它们各自要匹配所在式子的量纲，照抄别人的数字往往会在某个维度上炸掉。

27.4.7 参数量、显存与算力预算

训练一个网络要三块内存：参数、梯度与优化器状态、激活。Adam 每个参数要存一阶、二阶矩各一份，所以**每参数约需 12 字节**（参数 4、梯度 4、两个矩 8，若混合精度还要再加）。本章最大的网络 13121 个参数，全部状态不到 200 KB，可以忽略；换成千万参数的网络，这部分就是几百兆。

真正的开销在激活。按第 27.3.1 节，激活量是 $B\sum_{\ell} d_{\ell}$ 个数，批大小与宽度都线性贡献。**估算训练显存的粗公式是：参数量乘十二字节，加上批大小乘各层宽度之和乘四字节**。这个公式不含框架的临时缓冲，通常还要再留一倍余量。推理则轻得多：只要参数与一层激活，本章的序列模型在单核 CPU 上做一次前向是毫秒级，**这正是工业软测量能接受的开销**。

把推理的账再算细一点：一次前向约 5 万次乘加，按每秒十亿次浮点运算的保守估计，纯计算只要 0.1 毫秒量级；实际耗时主要在内存访问与框架调用，通常仍在毫秒以内。**真正需要担心的不是算力，而是调用链**——从采样到预处理、到模型、到后处理、再到写回数据库，任何一环的排队都比前向本身贵。所以评估部署可行性时，先量端到端的调用耗时，再谈模型本身的乘加次数。

27.4.8 从权重文件到线上服务

一个可部署的模型文件至少要装五样东西：权重、网络结构描述、标准化参数、窗口长度与特征顺序、版本号与训练数据的时间范围。**缺任何一样，模型都无法被独立复现**。第26章式 (26-29) 讲的模型卡可以原样套用，只是把“系数”换成“权重文件”。

压缩有三条常用路径：剪枝（把接近零的权重置零，稀疏化后压缩存储）、量化（权重从三十二位浮点降到八位整数，体积缩四倍）、蒸馏（用大模型的输出训练一个小模型）。本章的网络本身只有几千个参数，压缩收益有限；**真正需要压缩的是第 28 章那些百万级参数的模型**。对软测量来说，更值得做的是“把窗口长度、采样周期、特征计算方式写进接口文档”，因为这些一旦与训练时不一致，精度损失远大于量化带来的那一点。

27.4.9 训练日志该记什么

网络训练不像最小二乘那样一次解出，中途出了偏差只能靠日志回溯。**日志至少要有五列**：轮次、训练误差、验证误差、学习率、梯度范数。前四列是常规，第五列最容易被省掉，却往往是定位问题的唯一线索——梯度范数突然变成零，说明某一层死了；突然涨到几百，说明学习率太大或出现了数值爆炸。

实验 G 的曲线正好演示了三条判读规则。**训练误差一路下降、验证误差先降后升**，是过拟合，处置是早停或加正则；本次训练从 6.053 K 降到 0.882 K，验证在第 93 轮见底 1.112 K，之后回升，这就是典型形态。**训练与验证同时不降**，不是过拟合，而是学习率太小、结构不够或实现有错，此时加正则只会更糟。**两条曲线一起剧烈抖动**，多半是批太小或学习率过大，先把批加倍或把学习率除以三再试。

还有一条容易被忽略：**日志里要能直接画出图 27-5 右图那三条线**。如果只记了最后一个数，事后就无法判断“早停该停在哪一轮”，也无法解释为什么回测的 1.196 K 比训满 400 轮的 1.226 K 好。**训练日志的价值不在存档，而在让每一个决定事后可复核**。

日志留多久也有讲究。一次训练的完整曲线不过几万个浮点数，存十年也只占几兆；相比之下，重训一次的人力成本高得多。**凡是能被用来解释某次线上异常的记录，都值得长期保留**，至少包括当时的训练集哈希、超参数配置、依赖版本与硬件信息。很多团队只留权重文件，等到要复盘时才发现连“这个模型是用哪批数据训的”都答不上来。

27.5 工程含义与适用边界

27.5.1 什么时候值得用深度学习

本章的实验给了一个不太浪漫的答案：**在这类中小规模的表格任务上，深度学习的收益通常只有几个百分点**。1000 段样本、八个特征，前馈网络 1.186 K，线性模型 1.203 K，差距 0.017 K，约 1.4%；二次多项式 1.166 K 甚至比网络还好。真正拉开差距的是序列任务：非线性结构比线性脉冲响应好两成，因为那个对象的饱和整流项确实无法用线性核表达。

所以选型的第一条判据不是“数据量大不大”，而是**线性基线离噪声下限有多远**。若线性模型已经压到下限的 1.2 倍以内（本章静态任务就是：1.203 对 1.077，1.12 倍），剩余空间只有百分之十几，换成网络最多再挖一半，收益不超过 5%，还要付出部署与维护成本，不值得。若线性基线是下限的三倍以上，说明对象里有强非线性或强交互，网络才有发挥余地。

第二条判据是样本量，但用法与第26章的“十倍规则”不同。十倍规则说每个参数至少要十个样本，本章的网络有 4801 个参数、750 段训练样本，按这条规则根本不该训练，实际却能到 1.186 K。**原因是过参数化网络的泛化并不由参数量直接决定**，早停、权重衰减与数据的低内在维数共同把有效容量压了下来。但样本量仍然决定另一件事：能不能可靠地调参。750 段样本切出验证集后只剩 600 段，按第26章实验 D 折算，划分波动就有 0.03 K 量级，**与大多数结构差异相当**；样本再少，调参就退化成掷骰子。

第三条判据是部署条件。网络推理需要一个运行时（PyTorch、ONNX Runtime 或 TensorRT），控制器里通常没有；要把模型编译成 C 或部署到边缘盒子，就得额外的工程投入。表 27-3 把三条判据合成一张速查表。

表 27-3　三类情形的选型建议

情形	线性基线与下限之比	推荐方案	本章依据
稳态软测量，样本千级	1.1–1.3	二次多项式或梯度提升	网络 1.186 对多项式 1.166
强非线性、样本万级	2 以上	前馈网络或树集成	序列任务非线性收益两成
长序列、强时序依赖	不适用	卷积或门控循环	一维卷积 0.0178 对线性 0.0223

表格第三行要补一句：**本章的窗口只有 20 步，卷积已经够用**；若依赖跨度上千步，才需要门控循环或注意力，而那时样本量要求也随之上升。

27.5.2 数据量与数据质量

深度学习的样本量常被误解成“越多越好”。本章的序列任务看似有六千个训练窗口，但它们来自同一条三万六千步的序列、每四步取一个，**相邻窗口共享十九个时刻的输入，独立信息远少于六千份**。如果按独立样本估计算力预算，会严重低估所需数据；正确的估法是看“独立工况切换了多少次”，本章的激励每十二步换一个电平，独立切换约三千次。

比数量更重要的是三件事。**标签的时间对齐**：出口温度若用化验值，滞后两小时，错位的标签会让所有模型一起变差，且不会报错，只能靠扫描滞后时间发现（第26章 §26.5.2 给了做法）。**工况覆盖**：训练集若只在 60% 负荷附近，网络在低负荷段会外推，而树模型至少会给出“最近邻”的保守输出。**噪声水平**：噪声决定了误差下限，也决定了正则化的强度，噪声大的数据要用更强的权重衰减与更早的早停。

第26章那条“每个参数至少十个样本”的规则，在本章明显不成立：4801 个参数只用 750 段训练，平均每个参数 0.16 个样本，误差仍能到 1.186 K。**规则失效的原因不是网络有魔法，而是“样本数”这个量本身估错了**：八个输入里有四个是另外四个的确定性函数，数据的有效维数远低于八；真正的自由度按内在维数算，可能只有两三。反过来，把训练集压到 100 段，同样的网络立刻露出原形——训练 0.480 K、测试 1.481 K，**十倍规则在“有效样本数”这个意义上仍然有效，只是要先把特征之间的相关性扣掉**。

27.5.3 工业部署的五道坎

第一道是**依赖**。训练环境有完整的科学计算栈，现场控制器往往只有一个实时内核；解决办法是导出成 ONNX 或用 C 重写前向，但导出一致性要逐位核对。第二道是**版本**。权重、标准化参数、特征顺序、窗口长度四者必须绑定成一个不可分割的包，任何一项单独更新都会造成静默错误。第三道是**数值一致性**：训练时用双精度还是单精度、标准化是先减均值还是先缩放，差一点都会让线上输出偏移。第四道是**延迟**：软测量的采样周期通常是秒级，本章的模型远低于这个预算，但若把窗口拉到几千步、模型换成注意力，就要重新核算。第五道是**监控**：没有监控的模型等于没有模型，第 27.5.6 节单独讲。

这五道坎里，前三道是“一次性工程”，后两道是“长期运维”。**很多项目死在第四、第五道**：上线时精度很好，半年后悄悄退化，没人发现，直到产品质量出问题才回头查模型。深度模型比线性模型更难排查，因为它的失效往往不是某个系数变了，而是整片输入区域的响应都偏了。

导出这件事有一个具体的验收办法：**留一百条训练时的输入，分别用训练框架与导出的模型各算一遍，逐条比较最大绝对差**。差值在 10^{-4} 以内才算通过，超过就说明算子映射、数据类型或归一化位置有一处不一致。这一步不能省，因为导出的错误通常不是崩溃，而是“稍微偏一点”，在现场会被误判成模型精度不够，然后触发一轮徒劳的重新训练。

27.5.4 适用边界与不适用清单

适用的情形有四条。其一，对象有明确的非线性但机理式写不出来，比如阀门饱和叠加管道压损、多股物流混合后的温度。其二，有稳定的历史数据与可靠的标签，能覆盖主要工况。其三，允许秒级延迟，不需要毫秒级硬实时。其四，团队能维护训练脚本与模型版本，而不是“训练一次就锁进柜子”。

不适用的情形同样有四条。**其一，样本少于几百段**：此时任何网络的验证误差波动都大于结构差异，调参没有意义。**其二，要求逐点可解释**：安全联锁、计量结算这类场合，必须能给出“为什么是这个数”，网络给不出。**其三，工况会超出训练范围**：网络外推没有物理约束，可能给出离谱的值，而树模型至少会停在边界。**其四，控制器算力与工具链不允许**：为一个软测量引入完整推理栈，维护成本往往高于收益。

这八条不是绝对的。比如“样本少”可以通过机理模型生成仿真样本来补，但那又引入了仿真与现实的偏差，需要单独验证；“不可解释”可以通过事后归因（敏感度、Shapley 值）部分缓解，但那些解释本身也不是因果。**边界的作用是让人在动手前就知道要付什么代价**，而不是一刀切地禁止。

真正判断一个对象该不该上网络，可以按顺序问四个问题：**标签能不能稳定拿到**，拿不到就先解决测量问题，模型再好也没用；**线性基线离噪声下限多远**，一比一附近就别折腾；**工况会不会跑到训练范围之外**，会跑出去就先想好兜底；**团队能不能长期维护**，不能就选更简单的模型。四个问题里任何一个答“否”，都应当停在这里。**这套顺序的价值在于它把最贵的判断放在最后**：前三个问题用一天就能答，第四个问题要问组织。

27.5.5 与机理模型混合

纯数据模型与纯机理模型各有短板：前者需要数据、不能外推，后者需要假设、参数难标定。混合有三条常见路线。**第一，把机理量当特征**：本章的八个输入里，“阀位比流量”就是机理式里的指数项，“温升组合”是热平衡的中间量；这两个特征让线性模型的误差从 1.8 K 降到 1.2 K 量级（第26章实验 A 的对照），**说明机理知识最便宜的用法是特征工程，而不是建模**。

第二，残差建模：先用机理式算出基准预测，再让网络学残差。这样网络只需拟合“机理没描述的部分”，目标更简单、样本需求更小，而且外推时至少有基准兜底。**第三，结构约束**：把已知的单调性、正性写进网络结构（比如输出层用指数或softplus保证非负），或者用物理损失项惩罚违反守恒律的预测。三条路线的共同点是**不让网络从零学物理**，这与本章开头“学表示代替造特征”并不矛盾：网络学的是机理没覆盖的那部分表示。

残差建模之所以最常用，是因为它把风险切成了两半。基准项在训练范围外仍然可信，残差项只在训练范围内可信；于是外推时的输出退化成“机理值加一个保守的修正”，不会像纯网络那样给出离谱的数。**代价是要先把机理式标定准**——基准偏了，残差就要去补一个系统偏差，网络的容量被浪费在本来可以解析写出的部分上。本章没有做残差建模，因为换热器的准稳态式本身就是生成数据的那条式子，拿它当基准等于把答案提前告诉模型；真实项目里机理式总是有偏差的，这条路线才显出价值。

27.5.6 上线后的监控

监控要盯四个量。**输入分布**：每个特征的均值与方差相对训练期的偏移，用滑动窗口的标准化均值差或 KS 统计量，超过阈值就报警。**预测分布**：预测值的均值、方差、落在训练范围之外的比例；如果预测开始频繁贴边，说明工况变了。**残差**：有化验值时算残差，看均值是否漂移、方差是否变大；没有化验值时用“预测与机理基准之差”代替。**延迟与调用**：推理耗时、调用失败率，这两项出问题往往是工程故障而不是模型退化。

再训练要有触发条件，不能凭感觉。常用三条：输入漂移持续超过一周、残差均值偏移超过噪声标准差的一半、累积新增标签达到训练集的十分之一。**三条满足任意一条就排一次再训练，但上线前必须用同一套验证口径与旧模型对比**，否则很容易“越训越差”——第26章 §26.6.5 讲的标准差判据在这里同样适用。

新模型替换旧模型，最稳妥的方式是影子模式：新模型先只算不用，把它的输出与在线模型的输出并排记录一到两个月，等两边的偏差稳定、且新模型在真实数据上确实更好，再切换。**切换必须留回滚开关**，一条命令退回旧权重，而不是重新训练一遍。本章的模型只有几千参数，回滚成本几乎为零；换成大模型，权重文件本身就是运维对象，这条开关要从第一天就设计进去。

27.5.7 一次落地该留下什么

交付物应当有五样：训练脚本与随机种子（保证可复现）、权重文件与标准化参数（保证可推理）、固定的验证集划分（保证可比较）、失效模式清单（什么输入下不可信）、模型卡（训练数据时间范围、指标、已知局限）。**这五样缺一样，接手的人就得从头再来一遍**，而重来一遍的成本往往高于当初训练。

还有两条判据值得写进验收：**其一，模型必须打败“上一个上线的模型”与“线性基线”两个对手**，只跟自己比不算数；**其二，模型在留出的时间外推段上不能崩**——把最后三个月的数据留出来不参与训练，专门看这一段的表现，这比随机划分更能暴露漂移问题。本章的实验都是随机划分，所以数字偏乐观；真实项目里，时间外推的那一段才是验收的主角。

27.5.8 数据从哪来：三种来源的偏差

网络对数据分布比线性模型敏感得多，所以“数据从哪来”这件事必须先说清楚。工程上只有三种来源，每一种都带着自己的偏差。

历史运行数据最便宜，也最偏。控制系统把对象压在设定值附近，输入分布自然收窄：阀位可能在 0.55 到 0.70 之间有上万条记录，在 0.9 以上一条也没有。用这样的数据训练，模型在常用工况上很准，一旦开车升负荷进入稀薄区，误差会突然放大而且没有预警。**本章的 1000 段是均匀抽的，阀位覆盖 0.30 到 0.95，这是理想化处理**；真实项目里应当先画一张输入分布直方图，看训练集覆盖了验收工况的多少。

专门的激励试验能补上覆盖，代价是要扰动生产。本章序列任务的输入就是按这个思路造的：电平在 0.25 到 0.85 之间跳变、每段保持十二步、再经六点滑动平均，既有足够的激励又保留惯性。真实装置上做同样的事要申请工况、写操作票、盯十几个小时，**所以激励数据通常只有几百段，只够用来验证外推能力，不够用来训练**。

机理仿真样本无限、噪声可控，适合做算法预研，但会把机理模型的偏差当成真值学进去。本章的出口温度就是由一条带饱和因子的准稳态关系加高斯噪声生成的，噪声方差设为 1.16 K²，所以正文里那个 1.077 K 的“噪声下限”是设定出来的，不是从现场测出来的。**仿真上刷出的高精度，到现场往往要打对折**，原因就在这里。

三种来源的正确用法是分工而不是替代：仿真数据用来做结构与超参数的初筛，激励数据用来验证外推，历史数据用来做最终验收与漂移监控。**混用时必须在模型卡里逐段标注来源与时间范围**，否则半年后没人说得清某次精度下降是模型退化了，还是换了一批数据。**来源、时间范围、处理脚本三者缺一不可，缺了就没有可追溯性**；这条纪律执行起来只要几分钟，省下的却是几天。

27.6 仿真与验证

27.6.1 实验设置

静态任务用 1000 段换热器样本，750 段训练、250 段测试，八个特征，噪声下限 1.077 K。序列任务生成 36000 步输入输出序列，每 4 步取一个长度 20 的窗口，共约 9000 个样本，前 6000 个训练、后 3000 个测试；输出是归一化温度，范围 0.18 到 2.39，均值 1.29。**序列任务是开环预测**：只给过去 20 步的输入，预测当前输出，不喂历史输出，因此它检验的是“结构能否从输入波形里提取动态”，而不是“能不能靠自回归作弊”。

全部实验在单机 CPU、单线程下完成，整个脚本约五十秒。种子固定为 20261027，网络初始化、批顺序、丢弃掩码都由同一个种子派生。**单线程是刻意的**：多线程下浮点归约顺序不固定，最后几位会跳，虽然不影响结论，但会给“复现不出来”留下口实。

窗口取二十步、步长取四，这两个数都有来历。序列的饱和项依赖滞后九步的输入，二十步给了两倍余量，保证每个窗口都能看到完整的过渡过程；步长若取一，相邻窗口共享十九个时刻，样本之间几乎线性相关，验证误差会被严重低估。取四之后重叠降到十六个时刻，仍有相关性，但已足够让误差暴露出来——**这也是本章序列任务误差比静态任务“干净”的原因：样本相关性强的时候，任何模型看起来都好**。

27.6.2 实验 A：四类模型的对照

第一组实验把线性、二次多项式、梯度提升与前馈网络放在同一划分上，结果见表 27-4 与图 27-1。

表 27-4　静态任务上四类模型的测试表现

模型	测试决定系数	测试均方根误差 (K)	参数量
线性-8特征	0.9814	1.203	9
二次多项式	0.9826	1.166	45
梯度提升-200	0.9785	1.294	200 棵树
前馈网络-2×64	0.9819	1.186	4801

表 27-4 的排序值得注意：**二次多项式第一，前馈网络第二，线性第三，梯度提升垫底**。梯度提升在这个任务上落后，不是因为它弱，而是因为 750 段样本对 200 棵深度 3 的树来说偏少，树的方差没有被平均够；第26章用 900 段样本时它的误差是 1.179 K，与这里的 1.294 K 属于同一量级的波动。

表里的决定系数一列还提供了一个独立的交叉验证：二次多项式 0.9826、前馈网络 0.9819，**两者只差 0.0007，比第26章测到的重复标准差 0.0008 还小**。均方根误差给出的是 0.02 K 的差，决定系数给出的是 0.0007 的差，两种度量换算下来指向同一件事——**这两个模型在这个划分上分不出高下**。若只报其中一种，读者可能会以为存在一个明确的胜者；两种一起报，结论才站得住。

图 27-1 右图给出前馈网络的预测与真值散点。点云沿着对角线铺开，没有明显的弯曲或分组；与第26章图 26-1 右侧那条系统弯曲的分箱均值相比，**网络把线性模型留下的弯曲吸收掉了，剩下的散点宽度就是噪声**。这也是判断“模型是否还欠拟合”的直观办法：残差若仍有形状，说明结构没表达够；残差若只是一团对称的点云，就已经到数据本身的上限。

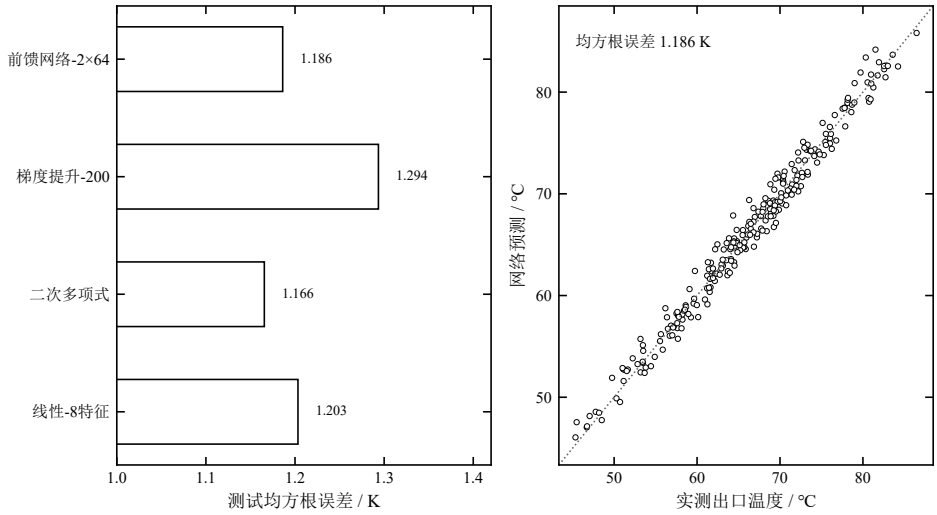


图 27-1　四类模型的测试误差与前馈网络的预测

27.6.3 实验 B：深度与宽度的扫描

第二组把网络结构从一层十六宽扫到六层三十二宽，参数量跨了八十倍，结果见表 27-5 与图 27-2。

表 27-5　六种结构的训练与测试误差

结构	层数	参数量	训练 RMSE (K)	测试 RMSE (K)
宽16	1	161	1.071	1.236
2×32	2	1377	1.002	1.166
2×64	2	4801	0.950	1.178
3×64	3	8961	0.927	1.223
4×64	4	13121	0.916	1.199
6×32	6	5601	0.976	1.146

表 27-5 里训练误差单调下降、测试误差上下摆动，两条曲线的分离随容量增大而扩大。**最宽的四层六十四宽参数量是六层三十二宽的两倍多，测试误差反而高 0.053 K**；两者差距刚好压在 0.05 K 的门限上，只能算“深而窄不比宽而浅差”，不能说“窄一定更好”。

图 27-2 把参数量画在对数轴上，看得更清楚：训练误差那条线几乎单调下行，测试误差那条线在 1.15 到 1.24 K 之间来回，没有明显趋势。**这说明在这个任务上，容量已经不是瓶颈**——对象的内在复杂度就那么高，八个特征加两层非线性已经够用，再加参数只是给噪声更多记忆空间。

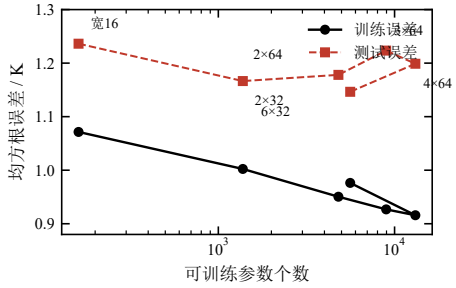


图 27-2　参数量与训练/测试误差

27.6.4 实验 C：梯度流

第三组固定八层、三十二宽、Xavier 初始化，只换激活函数，逐层记录权重梯度范数，结果见表 27-6 与图 27-3。

表 27-6　三种激活函数的逐层梯度范数

层号	S 型	双曲正切	整流线性
1	1.09×10^{-5}	1.28×10^{-1}	8.09×10^{-3}
2	4.80×10^{-4}	2.02×10^{-1}	1.40×10^{-2}
3	1.95×10^{-3}	1.84×10^{-1}	1.69×10^{-2}
4	7.26×10^{-3}	1.77×10^{-1}	1.61×10^{-2}
5	3.64×10^{-2}	2.11×10^{-1}	1.92×10^{-2}
6	2.09×10^{-1}	2.87×10^{-1}	3.24×10^{-2}
7	8.19×10^{-1}	2.93×10^{-1}	4.16×10^{-2}
8	3.45×10^{0}	2.50×10^{-1}	3.69×10^{-2}
首末比	9.73×10^{-7}	6.44×10^{-1}	2.61×10^{-1}

表 27-6 的最后一行是全部结论所在：**S 型跨了六个数量级，双曲正切基本持平，整流线性的衰减只有四倍**。S 型的首末比 9.73×10^{-7} 与式 (27-14) 的 4^{-8} 同量级，说明这个界不是纸上谈兵，而是能直接量出来的。

图 27-3 把同一组数据画在对数轴上。三条线的斜率差别一目了然：S 型是一条陡峭的斜线，双曲正切是水平的，整流线性介于两者之间。**这张图可以作为“为什么不用 S 型做深层网络”的标准回答**，也可以作为诊断工具：训练不收敛时，先画一遍逐层梯度范数，比盲调学习率快得多。

读这张图要注意方向：横轴第 1 层最靠近输入，第 8 层最靠近输出，所以“S 型的范数逐层增大”意味着**靠近输入的层梯度最小、学得最慢**。这正是深层网络训练慢的根源——输出端已经拟合好了，输入端的特征还没动。工程上的对策有三条：换整流线性、加归一化、加残差，三者的共同目标都是让这条曲线变平。**判断一个改进有没有用，看它有没有把首末比推向一**，这比看最终误差更早给出信号。

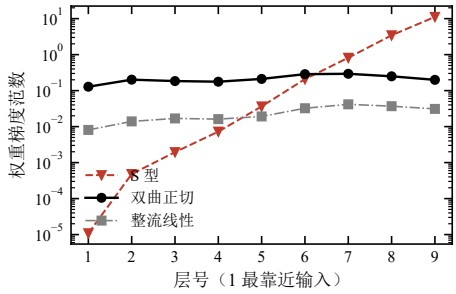


图 27-3　三种激活函数的逐层梯度范数

27.6.5 实验 D：反向传播的数值校验

第四组不训练，只验证梯度。网络取四层（$4\to8\to6\to3\to1$），双精度，共 119 个参数；随机抽 40 个分量，对每个分量用步长 10^{-6} 的中心差分估计导数，再与反向传播的结果比较。**用相对误差而不是绝对误差**，因为不同参数的梯度量级差好几个数量级，绝对误差无法横向比较。

结果是最大相对误差 3.45×10^{-8}、中位 1.11×10^{-9}。这个量级说明两件事：解析梯度与数值梯度在双精度下一致性到八位有效数字；中心差分的截断误差 $O(h^2)$ 与舍入误差 $O(\epsilon/h)$ 在 $h=10^{-6}$ 附近取得平衡。若把步长改成 10^{-2}，误差会升到 10^{-4} 量级；改成 10^{-10}，舍入误差会占主导，误差反而变大。**校验不通过时，先调步长，再怀疑公式**。

27.6.6 实验 E：学习率扫描

第五组固定两层三十二宽、150 轮，只改学习率，结果见表 27-7 与图 27-4。

表 27-7　学习率对 150 轮后误差的影响

学习率	训练 RMSE (K)	测试 RMSE (K)
3×10^{-1}	2.620	2.726
1×10^{-1}	1.475	1.658
3×10^{-2}	0.977	1.203
1×10^{-2}	1.020	1.177
3×10^{-3}	1.083	1.190
1×10^{-3}	1.152	1.255
3×10^{-4}	1.890	2.101

表 27-7 是一条不对称的 U 形：学习率过大的一侧上升得更陡，过小的一侧上升得平缓。**原因在于“过小”只是没训完，“过大”是训练本身失稳。**3×10^{-1} 时训练误差 2.620 K，是 10^{-2} 时 1.020 K 的两倍半，说明模型在训练集上都没拟合住，步子太大直接跨过了谷底；3×10^{-2} 时训练误差 0.977 K，训练集拟合得最好，但测试 1.203 K 不如 10^{-2}。

图 27-4 把横轴取对数并反转，让“从大到小”的扫描方向与横轴一致。曲线最低点落在 10^{-2}，相邻两档只差 0.03 K 与 0.01 K，再往两侧才明显抬高。**工程上取最低点或其小一档都可以：**样本量增大时最优学习率通常往下走，宁可慢一点也不要炸。

这里要提醒一句：**最优学习率与训练轮数是耦合的。**本章只跑 150 轮，所以 10^{-2} 显得最好；若把轮数加到 400，大步长的模型有更多机会把粗调剩下的误差磨掉，最低点会往小档移动。同理，换批大小、换优化器、甚至加不加权重衰减，都会让这条 U 形平移。**扫描结果不是常数，而是“这套配置下的临时最优点”，**换了配置要重扫，不能把 10^{-2} 抄到下一个项目里直接用。

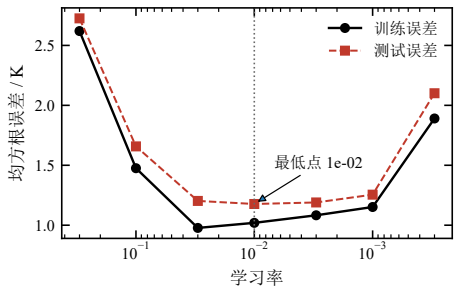


图 27-4　学习率扫描

27.6.7 实验 F：小样本下的正则化

第六组只用 100 段训练样本，两层六十四宽（4801 个参数），把五种正则配置逼到极限，结果见表 27-8 与图 27-5 左。

表 27-8　100 段样本下五种正则配置的误差

配置	训练 RMSE (K)	测试 RMSE (K)	训练—测试差距 (K)
无正则	0.480	1.481	1.001
权重衰减	0.947	1.247	0.300
丢弃0.3	0.887	1.262	0.375
批归一化	0.471	1.857	1.385
衰减+丢弃	1.440	1.810	0.371

表 27-8 的结论不是“正则化都好”。**权重衰减把测试误差从 1.481 K 压到 1.247 K，是唯一一个既降误差又保持训练能力的手段；**丢弃单独用略差（1.262 K），但也在可用范围；批归一化反而把测试误差推到 1.857 K，因为 100 个样本的批统计量太吵；两者叠加时正则过强，训练误差升到 1.440 K，测试误差 1.810 K。**“多上几种正则总没坏处”是错的，**叠加会互相放大，把模型压到欠拟合。

图 27-5 左图把训练与测试并排画出，可以直观看到“训练误差低不等于模型好”：无正则那组训练误差最低、测试误差最高；权重衰减那组训练误差几乎翻倍，测试误差却最低。**这条反差就是正则化的定义。**

批归一化那一列值得单独看：它的训练误差 0.471 K 是五组里最低的，差距 1.385 K 却是五组里最大的。**训练集拟合得最好、泛化最差，说明它失效的方式不是欠拟合**——批统计量在训练时由那 100 段算出，推理时改用滑动平均，两者的偏差直接把测试误差抬了起来。

这条现象提醒一件事：**看到“训练误差很低”就以为模型没问题，是本章最危险的一种误读**；训练误差只说明优化成功了，不说明学到的东西能迁移。

27.6.8 实验 G：早停

第七组从 600 段训练样本里再切出 150 段作验证，训 400 轮，每轮记录训练、验证、测试三条曲线，结果见图 27-5 右。

验证误差在第 93 轮见底，1.112 K；此后缓慢回升。**按第 93 轮的权重回测，测试误差 1.196 K；训满 400 轮则是 1.226 K**，早停拿回 0.03 K。训练误差则从第一轮的 6.053 K 一路降到 0.882 K，下降趋势贯穿全程，**它完全看不出“第 93 轮之后开始变坏”**——这正是必须用验证集而不是训练集挑轮次的原因。

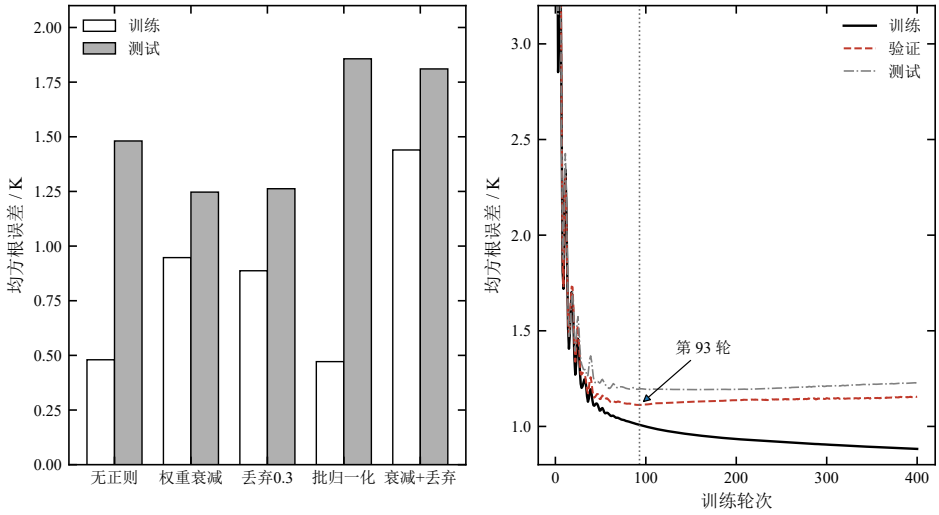


图 27-5　正则化对照与早停曲线

27.6.9 实验 H：序列任务的五种结构

第八组在同一个窗口任务上比较五种结构，训练预算对齐（40 轮、批 128、Adam、学习率 5×10^{-3}），结果见表 27-9 与图 27-6。

表 27-9　序列任务上五种结构的测试误差

结构	参数量	测试 RMSE
线性脉冲响应	21	0.0223
前馈网络	1761	0.0194
一维卷积	2721	0.0178
长短期记忆	2617	0.0190
自注意力	2593	0.0190

表 27-9 的前两行说明“非线性值两成”：线性脉冲响应 0.0223，前馈网络 0.0194，改善 13%；再往后三种结构都落在 0.0178 到 0.0190 之间，**彼此只差 6%，而参数量都是同一量级**。换句话说，只要模型能表达那个饱和整流项，用哪种结构差别不大；一维卷积略占优，是因为它的局部感受野与“滞后 3 到 9 步”的物理结构正好匹配。

图 27-6 右图截取 200 个测试样本，画真值、线性脉冲响应与长短期记忆三条线。**线性模型在输入换向的拐角处明显滞后**，因为它的二十个权重是在整个窗口上折中的；长短期记忆能跟住拐角，但在平段上略有过冲。这两种误差形态对应两种失效模式：前者是“表达不足”，后者是“记忆过强”，调参方向相反。

长短期记忆与自注意力给出完全相同的 0.0190，这个数据本身就是一条结论：**当窗口只有二十步、依赖主要在近几步时，注意力的“全局视野”没有用武之地**，它与门控记忆一样，只能把同一段输入压成一个向量再回归。注意力真正占优的场合是长窗口里存在远距离关联——比如一个批次切换的影响在两小时后才显现；本章的任务里没有这种结构，所以三者打平。**选结构要先看对象里有没有那种结构，而不是看哪种更新。**

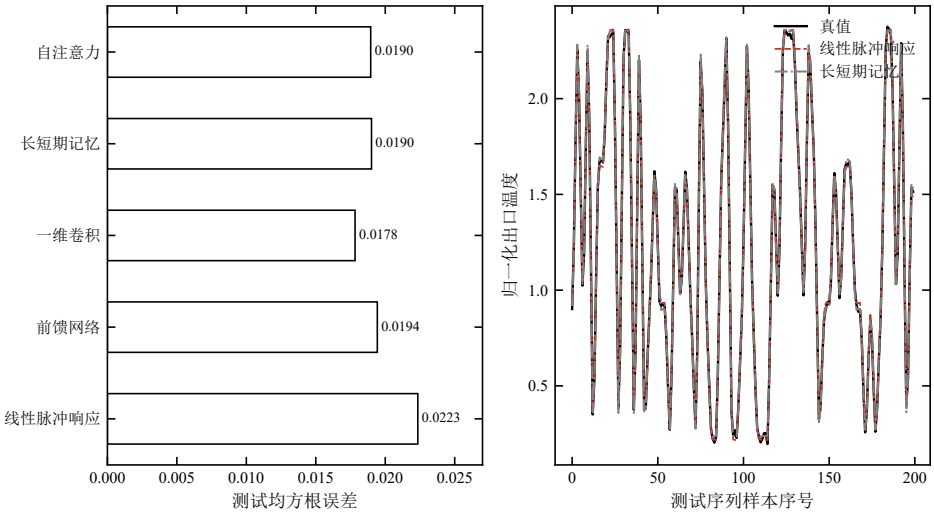


图 27-6　五种序列结构的误差与预测曲线

27.6.10 八组实验的合读

八组实验各回答一个问题，合起来是一条完整的逻辑链。**能不能表达**：实验 A 与 H 说明非线性结构确实有用，但在静态任务上收益只有 1.4%，在序列任务上有 13% 到 20%，差别取决于对象里有多少线性核覆盖不到的东西。**该给多少容量**：实验 B 说明容量超过某个点后，训练误差还在降、测试误差不降，多出来的参数只是在记噪声。**梯度能不能传**：实验 C 说明 S 型在八层时衰减六个数量级，双曲正切与整流线性没有这个问题。**实现对不对**：实验 D 用 10^{-8} 的相对误差证明反向传播写对了。**步子多大**：实验 E 给出 10^{-2} 的最优点与两侧的代价。**怎么压过拟合**：实验 F 说明权重衰减最稳、批归一化在小批下会反噬、正则不能叠加。**什么时候停**：实验 G 说明验证误差比训练误差早几十轮见顶，早停几乎免费。**用哪种结构**：实验 H 说明结构之间的差距远小于“线性与非线性”之间的差距。

把这条链子倒过来读，就是一条可执行的流程：先用线性基线确认还剩多少空间，再用梯度检查确认实现正确，然后扫学习率、定容量、加正则、早停，最后才在几个候选结构里挑。**顺序错了会浪费大量时间**——比如先纠结用不用注意力，却忘了检查标准化参数是否只由训练集估计。

顺序里最值得强调的是第二步。**梯度检查只花几秒钟，却能挡住几天的误判**：如果反向传播写错了一层，后面所有结构对照、所有学习率扫描、所有正则实验都会给出错误但自洽的结果——每个模型都差，排名却看起来合理。实验 D 那 119 个参数、40 个抽查分量，本身不产生任何精度结论，**它的价值是给后面七组实验发一张“实现可信”的通行证**。工程上把它放在容量扫描之前，就是这个道理。

27.6.11 数字之间的相互校验

同一件事在不同实验里出现多次，数字应当能对上，对不上就说明有口径差异。

第一组对照：两层六十四宽的网络，实验 A 报 1.186 K（400 轮、学习率 8×10^{-3}），实验 B 报 1.178 K（300 轮、同样学习率）。两次训练只差轮数，误差差 0.008 K，**远小于 0.05 K 的门限，属于同一结果**。

第二组对照：实验 G 的早停模型用 600 段训练，测试 1.196 K；实验 A 用 750 段训练，1.186 K。样本少了两成、误差高 0.01 K，方向与学习曲线一致，量级也合理。

第三组对照：实验 F 的无正则模型用 100 段训练、测试 1.481 K；实验 A 用 750 段、1.186 K。样本少七倍，误差高 0.295 K，**这个差距与“过拟合而不是模型不行”的解释吻合**——因为加权重衰减后同样 100 段能到 1.247 K，说明数据量并非不可克服，关键是有效容量。

第四个校验是对噪声下限的核算。静态任务的下限是 1.077 K，最好模型的测试误差 1.166 K（二次多项式），两者平方相减得 $1.166^2-1.077^2=0.199\ \mathrm{K}^2$，开方约 0.45 K。**这 0.45 K 就是“模型仍欠拟合的部分”**，也是任何结构改进能争取的全部空间；本章所有结构在这条线上的差别都小于它，这解释了为什么 A、B 两组的排名会来回摆动。

第五个校验在序列任务。输出标准差约 0.65（范围 0.18 到 2.39、均值 1.29），噪声标准差 0.01；线性脉冲响应的误差 0.0223 是噪声的两倍多，说明剩余误差主要来自未表达的非线性，而不是噪声。**若误差已经贴近噪声，再换结构就没有意义**——这条判据比看排行榜有用得多。

27.6.12 本章没有做的实验

一份实验记录的可信度，一半来自做了什么，另一半来自诚实地列出没做什么。下面五项空白都会影响结论的适用范围，读者引用本章数字时应当一并带上。

没有做多种子重复。每个配置只训一次，所以本章没有任何置信区间。第十个种子跑出来可能与第一个差 0.02 K，也可能差 0.05 K；统一门限正是为这个未知留的余量。**若某项结论要在工程上落地，必须补十次重复**，否则两个只差 0.01 K 的模型谁更好这个问题根本没有答案。

没有扫批大小。静态任务用全批（750 段一起更新），序列任务固定为 128。批大小与学习率是耦合的：批放大四倍，学习率通常也该放大，本章把两者都钉死，等于只考察了这条曲线上的一组点。**批归一化的实验尤其受此限制**，因为它的效果直接依赖批内样本数。

没有用 GPU，也没有更大的模型。参数量从 161 试到 13121 就到顶了，再往上既超出了 CPU 的耐心，也超出了本章“可复现”的初衷。因此“容量再大也不会更好”这个判断只对几千参数的网络成立，**不能推广到百万参数的模型**——那里起作用的是另一套机制。

没有做时间外推划分。所有实验都是随机划分训练与测试，同一时段的样本会同时落进两侧，数字因此偏乐观。真实项目里应当按时间切分，并用最后一段做验收，这一点第 27.5.7 节已经写明，但本章的实验并没有照做。

没有用真实工业数据。全部样本由已知机理加噪声生成，所以“模型学到了对象的物理结构”这句话在这里是可控的、可验证的；换到现场，同样的方法面临的是未知的噪声结构、未知的采样抖动与未记录的工况变更。**本章的数字给出的是方法的上界，不是现场的预期**。

还有一处需要在读表时记住：**实验 F 的五种配置都没有早停**。早停本身就是一种强正则，若加上它，无正则那组的测试误差不会再是 1.481 K，权重衰减的相对优势也会缩小。换句话说，表 27-8 比较的是“只有一种正则手段时哪个最有效”，而不是“这些手段在实际流程里的边际贡献”；真实流程里早停一定会开，所以权重衰减的增益会比表里小。**这张表说明的是机制，不是最终的配置建议**。

27.7 小结与常见错误

27.7.1 要点回顾

本章只做一件事：把“学表示”这件事拆成可以逐项检查的构件。

结构这一层，网络就是复合函数。式 (27-1) 说明每一层都是“仿射变换加逐元素非线性”，式 (27-2) 到 (27-4) 说明反向传播只是把链式法则在层与层之间复用一次，卷积、循环、注意力改的都是“层内怎么连”，不改这条链。梯度能不能从输出传回输入，取决于激活函数导数的上界：S 型最陡处只有四分之一，八层叠起来就是 4^{-8}；实验 C 量到的首末比 9.73×10^{-7} 与这个界同量级，而双曲正切是 0.644、整流线性是 0.261，都不构成数量级上的障碍。

容量这一层，多给参数不等于多给精度。实验 B 把参数量从 161 加到 13121，训练误差从 1.071 K 一路降到 0.916 K，测试误差却在 1.146 到 1.236 K 之间来回摆动，最好的反而是只有 5601 个参数的六层三十二宽。**“该停在哪”这个问题只能由验证集回答，不能由参数量回答**。

优化这一层，步子大小比算法花样更要紧。实验 E 的七档学习率画出一条不对称的 U 形，最低点在 10^{-2}，测试 1.177 K；3×10^{-1} 时训练误差 2.620 K，是 10^{-2} 时的两倍半，说明模型连训练集都没拟合住。而这一切的前提是实现正确：实验 D 用中心差分抽查 119 个参数中的 40 个，最大相对误差 3.45×10^{-8}，把“反向传播写错了”这个最昂贵也最容易被忽略的可能性先排掉。

正则这一层，工具不能叠加。只有 100 段训练样本时，无正则模型训练误差 0.480 K、测试 1.481 K，差距三倍；单加权重衰减把测试压到 1.247 K，单加丢弃压到 1.262 K，两者一起用却回到 1.810 K。批归一化在这份小批数据上最差，1.857 K，因为它依赖的批统计量本身就不稳。早停则是几乎免费的一项：验证误差在第 93 轮见顶 1.112 K，回测 1.196 K，比训满 400 轮的 1.226 K 低 0.03 K。

把这些合起来看，本章最重要的判断不是“网络比线性模型强”，而是**强多少要看任务**。静态换热器任务上，网络 1.186 K 输给二次多项式的 1.166 K，差 0.02 K，落在 0.05 K 的门限之内，正确的说法是“分不出来”，而不是“网络略逊”或“网络略优”。序列窗口任务上，线性脉冲响应 0.0223、一维卷积 0.0178，差两成，这才是一个站得住的差距。**同一套工具，在两个任务上给出两种量级的收益，这本身就是结论**。

如果再抽一层，本章讲的所有东西可以归成三本账。**第一本是表达账**：对象里有没有线性核表达不了的形状，有就值钱，没有就白搭，本章的序列任务有饱和整流项所以值两成，静态任务没有所以只值 1.4%。**第二本是样本账**：有多少独立信息可供学习，本章静态任务 750 段、序列任务有效切换三千次，都在“能训但调参噪声大”的区间。**第三本是工程账**：从权重到线上要过五道坎，每一道都可能把收益吃掉。三本账算下来是正的，才值得上；只有第一本是正的，先别急。

27.7.2 常见错误

深度学习最麻烦的地方在于：错了往往不报错。形状对不上会被广播悄悄补全，标准化参数用错只是让误差高一点，忘了切换模式只影响推理结果。表 27-10 按“现象—根因—处置”把八类高频错误列在一起。

表 27-10　深度网络实现与训练中的八类常见错误

现象	根因	处置	本章依据
训练误差很低、上线后全错	推理时仍开着丢弃，或批归一化用了单样本统计量	推理前显式切换到评估模式	§27.4.5
换一批数据误差整体抬升	标准化参数没随权重一起保存，推理时重算	均值方差与权重存进同一个文件	§27.4.1
损失不降也不报错	张量转置错，形状恰好能广播	打印每一步的形状并断言	表 27-2
训练误差自己都压不下去	学习率过大，一步跨过谷底	先降一个数量级再重扫	实验 E，3×10^{-1}
加了正则反而更差	权重衰减与丢弃叠加，有效容量被压过头	一次只加一项，逐项确认	实验 F，1.810 K
轮数越多越好	用训练误差挑轮次	留出验证集，按验证误差早停	实验 G，第 93 轮
小批训练时误差抖动剧烈	批归一化的批统计量样本太少	增大批或改用不依赖批的归一化	实验 F，1.857 K
结论站不住	只跟自己比，没有线性基线	先报基线，再报差值是否超门限	实验 A，0.02 K

前三条最阴，因为它们不报错。**丢弃在训练时按概率置零并放大，推理时必须关掉**；忘了关，等于每次推理都在随机删掉一部分神经元，输出会系统性地偏小且抖动。批归一化同理：训练时用批内均值方差，推理时要用训练期间累计的滑动平均，两者混用会让同一段输入在不同批大小下给出不同答案。标准化参数则是最容易丢的一件东西——很多人只存 `state_dict`，忘了把训练集算出的均值与标准差一起存，下次推理拿新数据现算，等于把输入分布平移了一段。

形状错误之所以危险，是因为广播机制会把“二十步乘八个量”和“八个量乘二十步”都算出一个结果，只有物理含义是错的。**表 27-2 那一列形状不是装饰，是可以在代码里写断言的**。

后五条会在损失曲线上留下痕迹，相对好查。它们的共同点是“用力过猛”：学习率调得太大、正则加得太重、轮数跑得太多。处理办法是同一条——**每次只改一个量，并且先看基线**。实验 F 里五种配置的排序说明，权重衰减单独用是安全的，丢弃单独用也安全，两者叠加就把模型压到了欠拟合一侧，训练误差 1.440 K 已经达到无正则时 0.480 K 的三倍。

排查还要讲顺序，否则会在无关的地方耗掉几天。**第一步查数据**：训练集与测试集的分布是否一致、标签有没有滞后、有没有重复样本；这几项错了，后面所有调参都是白费。**第二步查实现**：用小样本过拟合、用梯度检查核验反向传播、打印每一步的形状。**第三步才查超参数**：学习率、容量、正则。**这个顺序的依据是代价**：数据问题一次就能查完，实现问题几小时，超参数问题可能几天。**倒过来做的人，最常见的结果是花三天调出一个在错误数据上过拟合的模型**。

27.7.3 与相邻各章的接口

第28章接着讲网络的具体形态与训练工程：残差连接怎么让梯度跨层直通、归一化放在激活前还是激活后、学习率按什么曲线衰减、混合精度与分布式训练各自省下什么。本章给出的是最小可用的四个构件，第28章给出的是把它们拼成可训练大模型的那套工程做法。**读第28章时，凡是遇到“这样改能训得更深”，都可以回到式 (27-14) 那个连乘界去找原因**。

第29章专门讲无约束优化算法，梯度下降、动量、共轭梯度与拟牛顿法的收敛性质都在那里推导。本章把 Adam 当成给定的一步更新规则直接用，式 (27-20) 只列出三个部件的算式，不解释为什么它们能加速；**“为什么收敛”的问题一律指回第29章**，那里才有步长与收敛速率的严格讨论，而 Adam 这类自适应步长不在其列。

第30章的凸优化提供了另一面镜子。本章实验里最强的对手恰恰是二次多项式——一个凸模型，它在静态任务上以 1.166 K 赢过 4801 个参数的网络。**这个反差本身就是一条结论**：当对象的非线性可以用低阶多项式逼近时，凸模型的样本效率、可解释性与求解确定性都更好，没有理由上网络。

第36章讲功能安全与可靠性。本章 §27.5 提到的上线监控、权重版本管理、模型失效后的降级，在那里会被重新组织成“安全层”的一部分：网络输出要能被合理性检查拦截，要有回退到机理模型或手动操作的通道，还要能证明这条通道在最坏情况下也来得及动作。

还差一处要交代：**本章的网络与第25章的状态估计是两条并行的路**。卡尔曼滤波给的是带协方差的最优估计，网络给的是点估计，说不出“这个数有多可信”。工程上常见的做法是让两者并存——滤波值进控制回路，网络值做交叉校验，两者偏差过大就报警。**把网络当成滤波器的替代品，等于扔掉不确定性信息**，这在安全相关回路里是不可接受的。

27.7.4 一页纸清单

把本章可执行的部分压成一页，按顺序做，每一步都有对应的验收数字。

1. **先立基线**。线性模型与二次多项式各跑一次，记下测试误差与噪声下限的比值。本章是 1.203 与 1.166 对 1.077，比值 1.12 与 1.08。比值在 1.2 以内，网络的收益上限不到 5%，可以不做。
2. **确认实现**。用双精度小网络做梯度检查，抽查几十个参数，最大相对误差应在 10^{-7} 以下。本章实测 3.45×10^{-8}，达不到就先别调参。

3. **定学习率**。跨三到四个数量级扫一遍，取最低点或其小一档。本章最低在 10^{-2}，测试 1.177 K，两侧各差 0.03 K 与 0.01 K。
4. **定容量**。从“一层、宽度约为输入维数的四倍”起步，逐次加深加宽，同时记录训练与测试两条误差；一旦训练继续降而测试不降，就退回上一档。
5. **加正则，一次一项**。先权重衰减，再考虑丢弃，不要同时加；小批训练时慎用批归一化。
6. **早停并留下验证集**。验证误差见顶就停，本章第 93 轮见顶，早停回测 1.196 K 优于训满 400 轮的 1.226 K。
7. **报差值时带门限**。任何两个模型的比较都要说明差值是否超过划分波动。本章门限 0.05 K，网络与二次多项式的 0.02 K 差应报“分不出来”，线性与卷积的 0.0045 差（相对两成）才值得写成结论。

七步走完，得到的不只是一个模型，而是一份能被别人复核的记录：每一步用了多少数据、误差是多少、结论的门限在哪。**深度学习的工程性就体现在这里**——它不是一次训练的成败，而是一条每一步都留了数字的路径。

27.7.5 三个自检问题

每次训练结束、准备写结论之前，先问三句话。这三句话能挡掉绝大多数自欺。

第一句：这个误差跟谁比？ 没有基线的数字毫无意义。1.186 K 听起来不错，但它在 1.077 K 的噪声下限面前只剩 0.45 K 的改进空间，而在二次多项式的 1.166 K 面前其实还输了 0.02 K。**报误差必须同时报三样：基线、噪声下限、差值是否超过门限**。

第二句：验证集干净吗？ 只要验证集参与过任何一次选择——挑结构、挑轮数、挑学习率、甚至只是“看了一眼觉得不对就重训”——它就已经被污染了，报出来的数字会系统性偏乐观。本章实验 G 用验证集选轮次，再用测试集回测，正是为了让这两件事分开；**如果只有一个划分，就该承认“这个数字偏乐观”，而不是假装它是测试误差**。

第三句：换个人能跑出同一个数吗？ 把脚本、种子、数据划分、标准化参数、依赖版本交给同事，他应该得到小数点后第三位一致的结果。做不到，就说明还有隐式的随机源或未记录的预处理。**复现不是洁癖，而是结论能被检验的前提**。

三句话都答得上来，再谈模型好不好；答不上来，模型的表现就还只是一个传闻。

第28章 神经网络

28.1 动机与要解决的问题

28.1.1 第27章留下的三笔账

第27章把“网络是什么、怎么训练”讲完了：前馈网络是复合函数，反向传播是链式法则的矩阵形式，梯度消失有定量界，学习率有发散门槛，权重衰减等价于高斯先验。那一章的结论是务实的——在换热器软测量上，深度网络的收益只有百分之十几，真正决定成败的是数据质量与验证纪律。

但那段文字结束时，有三个问题没有回答。**第一，网络的结构该由谁定？** 第27章的实验里，两层六十四宽与六层三十二宽的测试误差都在 1.15 到 1.23 K 之间摆动，选哪一个凭经验。**第二，网络接进控制回路以后还成不成立？** 第27章的所有评估都是开环的：给一批样本、算一批误差，没有闭环、没有执行器饱和、没有设定值跟踪。**第三，数据不全的时候怎么办？** 第27章假设每一列输入都齐备，而现场总有测点掉线、总有阀门卡涩、总有传感器漂移。

这三笔账不能靠“再加深一层”来还。它们分别对应三类结构化的先验：径向基网络把“局部性”写进结构，物理约束网络把“守恒律”写进损失，图网络把“测点拓扑”写进连接。**三种做法的共同点是把领域知识从事后解释搬到事前约束**，这正是本章与第27章的分界：第27章问“网络能不能拟合”，本章问“用哪一种结构、以什么方式接进回路，才能在数据不完美时仍然可信”。

先把三个先导结果摆出来，让读者知道这一章的量级。**第一个**：径向基网络用 361 个参数、40 个中心，在静态软测量上做到 1.025 K，比 1377 个参数的前馈网络（1.071 K）略好，训练耗时都在零点几秒量级，两次运行之间谁快谁慢还会翻转；但同一个模型在阀位 0.96 到 1.00 的域外区间上误差 1.184 K，与 1.213 K 几乎持平——**局部支撑并没有让它在外推区崩掉，因为宽度选得足够大**。**第二个**：只有 100 段标签时，纯数据网络在域外是 2.291 K；把换热器的对数平均温差方程作为残差项加进损失、权重取 0.3，域外降到 2.044 K，域内从 1.836 K 降到 1.654 K，顺带把未知的换热系数从初值 2.08 标定到 2.29，而真值是 2.4。**第三个**：二十个测点、单点两成概率掉线时，只看中心点的前馈网络误差 3.073 K，把八十维特征展平的全连接网络 0.532 K，而按链式拓扑做消息传递的图网络 1.266 K，参数只有 177 个。**图网络没有赢**，它比逐点好，却比全连接差 0.73 K，这个反直觉的结果会在 §28.6 逐层拆开。

这三个数指向同一件事：**结构的价值不在“多拟合一点”，而在“缺信息时怎么补”**。第一个结果说明参数效率可以高一个量级；第二个说明领域方程能在标签稀缺时顶上来；第三个则是一个反例，说明结构先验不是越多越好——参数共享压住了容量，只有当缺失严重到全连接的固定输入位置失效时，图网络才追平。反过来说，如果数据完整、工况不越界、标签充足，这三类结构的收益都会退回到百分之十几——**先验不是免费的午餐，它只在先验成立的地方付钱**。

28.1.2 四个任务与一条主线

本章不另起一套对象，全部沿用换热器与管道这两条线，只在第27章的基础上加三个变体。

任务一，静态软测量。 与第27章同一物理关系，重新抽样 1000 段，750 段训练、250 段测试，八个输入特征，出口温度 42.6 到 86.3 ℃、均值 65.7 ℃。这个任务用来比较线性、前馈、径向基三类逼近器的精度、参数量与训练耗时。

任务二，域外工况。 另抽 200 段，阀位限制在 0.96 到 1.00，也就是训练集从未覆盖的区间，出口温度 64.0 到 86.9 ℃。训练集阀位上限是 0.95，**这 0.01 的缺口就是外推**。这个任务用来检验三类逼近器与物理约束网络在边界外的行为，是本章最锋利的一把刀。

任务三，逆模型控制。 被控对象取一阶惯性加两步延迟，时间常数 30 秒、采样 1 秒；稳态映射就是任务一的那条饱和曲线。先用 2000 组样本训练一个逆模型，输入是目标温度、入口温度、流量，输出是阀位；再把逆模型接成三种闭环结构——纯前馈、前馈加比例校正、纯比例积分——在四段设定值上跑 400 步。这个任务用来回答“网络输出能不能直接当执行器指令”。

任务四，管道测点。 二十个测温点沿管道等距布置，温度沿程按指数衰减到环境温度，每个测点叠加 0.6 K 独立噪声；每个样本里每个测点以两成概率掉线，读数缺失处填零并附一个可用标志。任务是由全部测点估计第十个测点的真值。这个任务用来比较逐点前馈、全连接前馈与图网络三种结构。

四个任务共用一条主线：**先验以什么形式进入模型，就在什么地方换来稳健性**。径向基把先验放在基函数的形状里，物理约束网络把先验放在损失函数里，图网络把先验放在连接关系里。三种放法对应三种代价：径向基的中心要选、物理项要标定系数、图网络要给出邻接矩阵。**没有任何一种能自动获得先验，先验都是人给的**，本章的工程含义一节会把这句话展开。

28.1.3 三条口径与两处修正

评估口径沿用第26、27章，只做两处必要的补充。

第一，**所有误差仍是同一划分上的测试均方根误差，单位开尔文**。静态任务的划分种子与第27章不同（20261028 对 20261027），所以同一个模型的数字会有零点几开的差异，这不是实现了变化，而是样本换了。跨章比较时只能比量级与排序，不能比小数点后第三位。

第二，**结构对照时训练预算对齐**。前馈网络统一 400 轮、学习率 8×10^{-3}、全批量；径向基网络的“训练”是解一次岭回归，没有轮次概念，于是用“中心数相同、宽度由训练集内部验证选出”来对齐自由度。这一条要特别说明：径向基的宽度**不允许用测试集选**，选宽度的验证集是从 750 段训练样本里再切 20% 出来的，选完在全部 750 段上重拟合。**如果允许看测试集，径向基的 1.025 K 会更好看，但那个数字没有意义**。

第三，**差异小于 0.05 K 不当结论**。这个门限的来历见第26章：划分波动带来的决定系数标准差 0.0008，折算到误差约 0.03 K，取 0.05 K 作保守门限。本章有一批数字落在门限内，比如径向基 1.025 K 与前馈网络 1.071 K 相差 0.046 K，**严格按口径应当写成“两者分不出来”**；后文在必要处会明确这样写，而不是挑一个好看的排名。

两处修正针对本章的新任务。**修正一，域外误差单独报**。域外样本不参与任何训练与选型，只用于最后评估；它的误差不能与域内误差平均，否则外推失败会被域内的好成绩掩盖。**修正二，闭环指标拆成三项**：全程均方根误差、每段阶跃后第 60 到 100 步的平均绝对误差（代表稳态精度）、阀位饱和时间占比。只报全程误差会掩盖两件事：阶跃瞬间的大偏差是任何控制都躲不掉的，而稳态残差才反映模型准不准；饱和占比则反映指令是否长期顶在边界上。

28.1.4 与相邻各章的分界

第27章讲“网络怎么训练”，本章讲“网络怎么选、怎么接”。两章共用反向传播、梯度检查、学习率这些底座，本章不再重复推导；**凡是涉及“梯度怎么算出来”的问题，一律指回第27章 §27.2**。

第29章讲无约束优化算法，会推导梯度下降、动量、共轭梯度与拟牛顿法的收敛性。本章大量使用 Adam，但只把它当成给定的更新规则；**本章实验 F 里“学习率 0.30 发散”这一现象，其理论解释在第29章**，这里只报现象与判据。

第44章讲 PID 与整定，第45章讲串级，第58章讲模型预测控制。本章的逆模型控制是一个**极简的对照**：它用网络输出当前馈，再叠一个比例积分做校正，结构与第46章的前馈加反馈复合控制同构。**本章不讨论抗饱和、不讨论增益调度、不讨论无扰切换**，这些属于第44到47章；本章只回答一个问题——网络当前馈时，模型误差会以什么形式进入闭环。

第15到22章讲系统辨识。本章的逆模型训练本质上是“用数据拟合一个静态映射”，不涉及动态辨识；**如果对象有显著动态，逆模型必须换成第18章的 ARX 或第20章的子空间模型**，否则纯前馈的相位误差会让闭环振荡。这一点在 §28.5 会展开。

还有两条边界。第一，本章不讨论大规模预训练、迁移学习与基础模型，那些属于专门教材；本章的所有网络都在几百到几千参数的量级，目的是让每条结论都能手算核对。第二，本章不讨论 GPU 与分布式训练，实验全部在单核 CPU 上完成，**推理耗时一节报的是微秒级的绝对值，换机器会变，但量级与趋势可用**。

28.1.5 四个问题与路线

本章围绕四个问题展开。

问题一：三类逼近器的差别到底在哪？ 不在精度，在参数效率与外推行为。径向基用 361 个参数达到 1.025 K，前馈网络用 1377 个参数达到 1.071 K，两者在门限内分不出来；但径向基的训练是解一次线性方程组，没有局部极小、没有学习率、没有初始化，**这三点在工程上比 0.05 K 更值钱**。代价是中心与宽度要选，选不好就崩到 4.6 K。

问题二：物理方程怎么进损失？ 不是把方程当标签，而是把残差当罚项，方程里的未知系数与网络权重一起标定。本章的实测给出了一个非单调的结果：权重 0.3 时域外最好（2.044 K），权重 1.0 时域内最好（1.536 K）但域外反弹到 2.289 K。**罚项过重会让网络放弃数据、只听方程，而方程本身是近似**，这个拐点必须靠扫描找，不能拍脑袋。

问题三：网络输出能不能直接驱动执行器？ 能，但要接受三个前提：逆映射存在且唯一、执行器不饱和、模型误差被反馈兜住。本章实验里纯逆模型的稳态残差 1.321 K，加一个比例校正降到 0.388 K，代价是 14.5% 的时间顶在阀位上限；纯比例积分的稳态残差 1.099 K。**前馈把误差压小，反馈把误差兜住，两者缺一不可**。

问题四：测点掉线时哪种结构还能用？ 三种结构各有各的用法。二十个测点、两成掉线，逐点前馈 3.073 K、全连接前馈 0.532 K、图网络 1.266 K。**参数最少的图网络没有赢**：全连接把二十个读数全展平，容量足够，在这个规模上最好；图网络只有 177 个参数，靠邻居聚合补信息，掉线率升到三成时才与全连接追平（1.166 K 对 1.169 K）。原因不是图网络更聪明，而是它的聚合方式**只在可用邻居上取平均**，缺失点被掩码排除，退化更平缓；但它的两层感受野只看得到左右各两个测点，上限摆在那里。这个结果是本章最值得写清楚的一处负面结论。

对应到节次：28.2 定义四类结构与三种控制接法，28.3 推径向基的线性参数化、图网络的消息传递、逆模型的可逆性条件与闭环误差界，28.4 落到中心选取、宽度标定、掩码聚合、推理耗时的数值实现，28.5 讲工程边界与失效条件，28.6 用八组实验把每条论断钉在数字上，28.7 收束。

28.1.6 本章的实验约定

为了让读者能复现每一个数字，把与“模型好不好”无关、但会改变结果的设定先写清楚。

运行环境：单机 CPU，PyTorch 单线程，NumPy 与 scikit-learn 也限制为单线程（`OMP_NUM_THREADS=1`），随机种子 20261028。单线程的理由与第27章相同：多线程归约的加法次序不固定，同一份代码两次跑出的误差在第三位小数上会抖。**本章还多一条理由**：径向基的岭回归解对矩阵条件数敏感，条件数在 10^8 量级时，线程数变化会改变最后几位的解，进而让 1.025 与 1.030 这种差别失去意义。

训练预算：前馈网络 400 轮全批量、Adam、学习率 8×10^{-3}；图网络与管道任务 400 轮、学习率 5×10^{-3}；物理约束网络 1200 轮，因为要同时标定网络与方程系数，收敛更慢。径向基没有轮次，报告的是“中心选取加岭回归求解”的总耗时，含宽度验证的四次拟合。

计时口径：训练耗时用 `time.perf_counter()` 包住整个拟合过程，不含数据生成与标准化；推理耗时是单样本前向 500 次的平均值，前 100 次作预热丢弃。**这个口径包含 Python 调用开销**，所以宽度从 8 加到 256，耗时几乎不变（22 到 26 微秒），直到宽度 1024 才涨到 89.5 微秒。这不是测错了，而是本章想说明的工程事实：**在这个规模下，推理耗时由框架开销主导，不由参数量主导**。

不报标准差：与第27章一致，所有数字来自单次运行。本章有几处对照本身就落在门限内（径向基 1.025 对前馈 1.071），此时报告“分不出来”比报告排名更诚实。**读者若要拿这些数字做选型依据，正确做法是在自己的数据上重跑，而不是照抄本章的排名**。

代码与图的对应：`code/ch028_sim.py` 产出全部数字，`code/ch028_figs.py` 产出六张图。图里的每一条曲线都能在脚本里找到对应的函数，函数名与图号在 §28.6 各小节开头注明。**若某张图的数字与正文对不上，以脚本输出为准**，因为正文是手抄的，脚本是算出来的。

28.2 定义与数学基础

28.2.1 三类结构共用一个骨架

第27章把前馈网络写成复合函数，每一层是“仿射变换加逐元非线性”。这个写法可以再抽象一层：**任何一类参数化逼近器，都可以写成“一组基函数、一组权重、一个连接方式”的三元组**。

$$\hat{y}(\mathbf{x}) = \sum_{j=1}^{K} w_j \, \phi_j(\mathbf{x}) + b \tag{28-1}$$

式 (28-1) 里，ϕ_j 是第 j 个基函数，w_j 是它的权重，b 是偏置，K 是基的个数。三类结构的差别全部落在 ϕ_j 上：径向基网络的 ϕ_j 是一个以中心 $\mathbf{c}_j$ 为中心的高斯鼓包，只依赖输入到中心的距离；前馈网络的 ϕ_j 是隐层第 j 个单元的响应，本身又是输入的复合函数，需要靠反向传播训练；图网络的 ϕ_j 定义在节点上，取值取决于邻居的表示。

这个统一写法有一个直接好处：**参数量可以逐项数出来**。径向基的 w 有 K 个、偏置一个，中心若固定则不计入可训练参数，于是 40 个中心、8 维输入对应 $40+1=41$ 个可训练权重，加上固定的 40×8 个中心坐标，存储 361 个数。前馈网络两层三十二宽、8 维输入，可训练参数是 $8\times32+32$、$32\times32+32$、$32+1$，合计 1377。**两者相差 33 倍，测试误差只差 0.046 K**。

还要说明式 (28-1) 的适用边界。它是一个**标量输出、单隐层**的写法，多输出时把 w_j 换成向量，深层时把 ϕ_j 递归展开。本章所有任务都是单输出（一个温度、一个阀位），所以这个简化写法够用；**若读者要处理多变量输出，只需把 y 与 b 换成向量，其余推导不变**。

这个写法还把模型分成两半：**基函数决定“能表示什么形状”，权重决定“在这个形状族里挑哪一个”**。分工带来一条实用的调试顺序——形状不对时调基函数（中心、宽度、核型），形状对了但拟合不够时调权重（加正则、加数据）。本章实验 B 的现象都能用这句话解释：误差随宽度大幅变化，是形状问题；中心数从 40 加到 80 只变 0.020 K，说明权重那一半已经够用。

28.2.2 径向基网络：把局部性写成基函数

径向基的基函数取高斯核：

$$\phi_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}_j\|^2}{2\sigma_j^2}\right) \tag{28-2}$$

式 (28-2) 有两个可调的量：中心 $\mathbf{c}_j$ 与宽度 σ_j。中心决定“鼓包放在哪”，宽度决定“鼓包有多胖”。**这两个量决定了径向基的成败**，而权重 w_j 反而是次要的——因为式 (28-1) 对 w 是线性的。

宽度不是凭经验给的，本章用样本自身的几何定标：先算每个标准化后的训练样本到最近邻样本的距离，取这批距离的中位数记作 σ_0，再把宽度写成 $\sigma=\beta\sigma_0$，倍数 β 由训练集内部验证选出。用中位数而不用均值，是因为八个特征标准化之后联合分布仍不均匀——阀位在 0.30 到 0.95 上接近均匀，而 u/q 集中在中段，均值会被稀疏区拉大。**σ_0 的物理含义是“相邻两个样本的典型间距”，宽度以它为尺子，才不会随特征个数与样本量失效**。

线性这件事值得单独说。把训练样本排成矩阵，第 n 行第 j 列是 $\phi_j(\mathbf{x}_n)$，得到设计矩阵 $\mathbf{\Phi}\in\mathbb{R}^{N\times K}$。给定标签 $\mathbf{y}$，最小化带岭罚的平方误差，解是

$$\hat{\mathbf{w}}=\left(\mathbf{\Phi}^{\top}\mathbf{\Phi}+\lambda\mathbf{I}\right)^{-1}\mathbf{\Phi}^{\top}\left(\mathbf{y}-b\mathbf{1}\right) \tag{28-3}$$

式 (28-3) 是一个闭式解，**没有迭代、没有学习率、没有初始化、没有局部极小**。这四点决定了径向基在工程上的位置：训练可复现、耗时可预算、失败可诊断。代价是中心与宽度必须事先定好，而且一旦定错，式 (28-3) 会忠实地解出一个很差的模型——**它不会报错，只会给你一条平滑但错误的曲线**。

本章实验里，10 个中心、宽度取最近邻中心距离的一半时，测试误差 4.612 K；同样是 10 个中心，宽度放大到四倍，误差降到 1.262 K。**同一个闭式解，因为宽度差八倍，误差差 3.35 K**。这不是数值问题，是模型设定问题。

式 (28-3) 里的岭罚系数取 10^{-6}，小到几乎不改变解，作用是数值上的：750 个样本、361 个基函数，$\mathbf{\Phi}^{\top}\mathbf{\Phi}$ 的最小奇异值接近零，不加罚项时求逆会把最后几位有效数字上的噪声放大。**这个罚项不是正则化手段，是让矩阵可逆的保险**；真正的泛化控制来自中心数与宽度，它们决定这 361 个基函数有多少是冗余的。

28.2.3 图网络：把拓扑写成聚合规则

图网络的最小形式是一层消息传递。设节点 i 的表示为 $\mathbf{h}_i$，邻居集合为 $\mathcal{N}(i)$，一层更新写成

$$\mathbf{m}_i=\frac{1}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)}\mathbf{h}_j,\qquad \mathbf{h}_i'=\varphi\left(\mathbf{W}_s\mathbf{h}_i+\mathbf{W}_n\mathbf{m}_i\right) \tag{28-4}$$

式 (28-4) 里，$\mathbf{W}_s$ 处理节点自身，$\mathbf{W}_n$ 处理邻居均值，φ 是逐元非线性。**关键在于两个权重矩阵对所有节点共享**：二十个测点用同一套 $\mathbf{W}_s$、$\mathbf{W}_n$，参数量与节点数无关。本章的图网络两层、十六通道，参数 177 个；同样任务的全连接网络把八十维特征展平，参数 1313 个，是前者的 7.4 倍。

共享还有一面代价。所有节点共用一套权重，等于假设“每个测点的局部规律相同”；管道上这个假设成立——沿程每一点的衰减机制一样——但换成有支路、有汇合的网络，各节点的邻域结构不同，共享权重会把不同规律平均掉。**拓扑先验用得对不对，取决于这张图是不是同质的**，这一点在选结构时比参数量更值得先问。

式 (28-4) 的均值聚合有一个前提：邻居全都可用。现场不成立，所以要加掩码。设 $m_j\in\{0,1\}$ 表示第 j 个测点读数是否可用，A_{ij} 是邻接矩阵（不含自环），聚合改成

$$\mathbf{m}_i=\frac{\sum_j A_{ij}m_j\mathbf{h}_j}{1+\sum_j A_{ij}m_j} \tag{28-5}$$

式 (28-5) 的分母加一，是给自环留的权重。**缺失的邻居既不进分子也不进分母**，所以聚合结果仍然是一个“可用邻居的均值”，而不是被零拉低。这一行改动让图网络在 20% 掉线时做到 1.266 K，而只看中心点的前馈是 3.073 K；但同样数据下全连接网络是 0.532 K，所以**聚合规则不是唯一因素，容量与参数共享同样在起作用**，§28.6 会把三者的差别拆开。

式 (28-5) 还隐含一个假设：缺失是随机的，与读数大小无关。若缺失与工况相关（比如高温时传感器更容易坏），掩码本身就带信息，此时应当把 m_j 作为特征喂进网络，而不是只用来加权。本章的模拟里缺失是独立随机的，所以只用它做加权；**真实装置上线的第一件事，是画一张“缺失率对工况”的散点图**，确认这个假设。

28.2.4 物理约束网络：把方程写成罚项

物理约束网络（physics-informed neural network）的做法是把方程残差加进损失：

$$L=L_{\text{data}}+\lambda L_{\text{phys}},\qquad L_{\text{data}}=\frac{1}{N_l}\sum_{n=1}^{N_l}\left(\hat{y}(\mathbf{x}_n)-y_n\right)^2 \tag{28-6}$$

式 (28-6) 里 N_l 是有标签样本数，λ 是权重。L_{phys} 在“配点”上算，配点不需要标签，只需要工况坐标，所以可以铺满整个工作区间——**这正是它能改善外推的原因：标签只在 0.30 到 0.95 有，配点可以铺到 1.00**。

本章的物理式取换热器的对数平均温差形式。冷流体吸热等于换热器传热：

$$q\left(\hat{T}_{\text{out}}-T_{\text{in}}\right)=UA\cdot\frac{\left(T_{\text{hot}}-\hat{T}_{\text{out}}\right)-\left(T_{\text{hot}}-T_{\text{in}}\right)}{\ln\left[\left(T_{\text{hot}}-\hat{T}_{\text{out}}\right)/\left(T_{\text{hot}}-T_{\text{in}}\right)\right]} \tag{28-7}$$

式 (28-7) 里 UA 是换热能力，未知但可标定。本章把它写成 $UA = a\,u$，u 是阀位——**阀位决定有效换热面积，这是工艺常识**；系数 a 与网络权重一起用梯度下降训练。这个写法把“未知参数”和“未知函数”分开：网络学函数形状，a 学物理量纲。

式 (28-7) 的对数在两端会数值不稳：出口温度接近热源温度时分子分母同时趋零。本章的实现给两个温差都加了 0.5 K 的下界，并且把残差按 $q(T_{\mathrm{hot}} - T_{\mathrm{in}})$ 归一化，让残差成为无量纲的相对误差。**这两个处理不是可选的**：不做下界，训练到第几十轮就会出现 NaN；不做归一化，q 小时残差被放大，罚项会压垮数据项，本章第一版实现就栽在这里——λ 只取 0.001，域内误差就从 1.836 K 抬到 2.101 K。

配点的数量本章取 600，是有标签样本的六倍。这个比例不是定理，理由很实际：物理项要在整个工作区间上约束形状，而标签只覆盖其中一段；配点太少，方程只约束几个点，网络在点之间仍然自由。配点也不是越多越好，每个配点都要前向传播一次并反传梯度，600 个已在训练耗时里占了大头。**本章的八维特征实际只有三个独立维度**——阀位、入口温度、流量决定其余五个——所以 600 个配点足以把三维工作区铺开，再多就是重复采样。

28.2.5 三种控制接法：监督、直接、间接

网络进控制回路有三种典型接法，本章只实现前两种。

监督式：先用数据训练一个对象模型 $\hat{f}:\ (u, T_{\mathrm{in}}, q) \mapsto T_{\mathrm{out}}$，控制器再对 $\hat{f}$ 求逆或做优化。这种接法的优点是模型可单独验收，缺点是求逆有误差累积。

直接式：直接训练逆模型 $\hat{g}:\ (T_{\mathrm{sp}}, T_{\mathrm{in}}, q) \mapsto u$，网络输出就是阀位指令。训练数据由对象的稳态映射反查得到：

$$\hat{u} = \hat{g}\left(T_{\mathrm{sp}}, T_{\mathrm{in}}, q\right), \qquad \text{训练目标 } u_n = g\left(T_{\mathrm{out},n}, T_{\mathrm{in},n}, q_n\right) \tag{28-8}$$

式 (28-8) 要求 g 存在。对本章的饱和曲线，T_{out} 对 u 严格单调，逆映射存在且唯一；**若对象有回环、有滞后或非最小相位，逆映射可能多值或不存在**，此时直接式失效，必须退回监督式加数值求逆。

间接式（也叫反馈误差学习）：网络输出叠加在常规控制器上，用闭环误差在线修正。本章不实现，只在前馈加比例校正里取了它的一个特例：前馈给主指令，比例项修残差。

三种接法的共同问题是**模型误差如何进入闭环**。设对象稳态映射为 f、逆模型为 $\hat{g}$、真实逆为 g，纯前馈时指令误差为 $\hat{g}(T_{\mathrm{sp}}) - g(T_{\mathrm{sp}})$，经过对象增益 $\partial f / \partial u$ 放大后成为稳态残差。本章实验里这个残差是 1.321 K，而对象在该工作点的灵敏度约 35 K 每单位阀位，**反推指令误差约 0.038 个阀位**——与逆模型 2000 组样本、60 个中心拟合出的训练残差 0.036 个阀位吻合。

直接式还有一个容易被忽略的细节：训练样本由稳态映射反查得到，反查时的采样密度决定了逆模型在哪些区段更准。若按目标温度均匀采样，低温段每开尔文对应的阀位变化大，样本在阀位轴上反而稀；按阀位均匀采样则相反。**本章按阀位均匀采样再算目标温度，等于把精度预算平均分给执行器行程**，这对控制更合理，因为执行器的分辨率是按阀位计的，而不是按温度计的。

28.2.6 记号、形状与本章约定

把后文要用到的记号一次列清。表 28-1 是符号表，表 28-2 是四个任务里张量的形状。

表 28-1 本章记号

记号	含义	本章取值
$\mathbf{x}$	输入特征向量	静态任务 8 维，管道任务每节点 4 维
K	基函数或中心个数	10 到 160
σ	径向基宽度	最近邻中心距离的中位数乘以倍数
$\mathbf{\Phi}$	设计矩阵	$N \times K$
λ	岭罚系数或物理项权重	岭罚 10^{-6}；物理项 0 到 1.0
A	邻接矩阵	二十节点链式，不含自环
m_j	测点可用标志	0 或 1
N_l	有标签样本数	100（物理约束实验）
N_c	配点数	600

表 28-1 里的 λ 一名两用，需要读者按上下文区分：出现在式 (28-3) 时是岭罚，出现在式 (28-6) 时是物理项权重。**这两个量的作用相反**——岭罚压制解的大小，物理项放大方程的约束——混用会造成误读，所以本章在正文里一律写成“岭罚系数”与“物理项权重”。

表 28-2 四个任务的张量形状

任务	输入形状	输出形状	样本划分
静态软测量	750×8	750×1	750 训练 / 250 测试
域外评估	200×8	200×1	不参与训练
逆模型控制	2000×3	2000×1	全量训练，闭环单独仿真
管道测点	$800 \times 20 \times 4$	800×1	800 训练 / 300 测试

表 28-2 里管道任务的输入是三维张量：第一维样本、第二维节点、第三维特征。**这个形状是图网络与全连接网络的分水岭**——图网络保留节点维、按邻接聚合；全连接网络把后两维展平成 80 维，拓扑信息就此丢失。同一个张量，两种读法，20% 掉线时误差分别是 1.266 K 与 0.532 K；而只取中心点那一行、把另外十九个测点全丢掉的逐点前馈是 3.073 K。**三种读法各有得失，展平不是退步，聚合也不是进步**，要看样本量、节点数与掉线率处在什么区间。

最后一条约定：本章所有公式里的温度单位都是摄氏度，误差单位都是开尔文；**温差与温升可以混用这两个词，但绝对温度只用摄氏度**，避免“273.15”这类常数在推导里来回出现。

还有两条实验约定写在脚本开头，不进公式但影响可复现性。第一，所有随机源固定种子 20261028，NumPy 与 PyTorch 各设一次，聚类另设随机状态；第二，进程内线程数固定为一。**单线程不是为了快，是为了让耗时数字可比**：多线程下同一段矩阵乘的耗时会随负载抖动，而本章要比较的是结构之间的量级差，抖动会吃掉这个差。

28.3 推导与性质

28.3.1 线性参数化的两面

式 (28-3) 把径向基的训练变成一次线性求解，好处已经说过。坏处藏在那条逆矩阵里：解的相对误差正比于设计矩阵的条件数，而

$$\kappa\left(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}+\lambda\mathbf{I}\right) \approx \frac{\kappa^{2}\left(\boldsymbol{\Phi}\right)}{1+\lambda/\sigma_{\min}^{2}} \tag{28-9}$$

式 (28-9) 说明两件事。第一，**通过正规方程求解会把条件数平方**，所以工程上更稳的做法是对 $\boldsymbol{\Phi}$ 做 QR 分解或奇异值分解，而不是显式构造 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$。本章的实现为了简洁仍用正规方程，但把岭罚设成 10^{-6}，靠它把最小奇异值抬起来。

第二，条件数随中心数与宽度急剧变化。本章实验 B 的实测：40 个中心、宽度取一倍基准时，条件数 108.3；宽度取四倍时，条件数 8 373 778，涨了七万七千倍。**误差却从 1.450 K 降到 1.025 K**。也就是说，**条件数变差的同时精度变好**，这两个指标方向相反。

这个反直觉的结果有清楚的解释。宽度小时，每个基函数只覆盖自己周围一小块，中心之间的重叠少，设计矩阵接近单位阵的缩放——条件数小，但基函数之间不重叠导致覆盖有空洞，输入落在空洞里时输出只能靠外推，误差大。宽度大时，基函数互相重叠，覆盖连续，精度上去了；代价是列之间高度相关，条件数暴涨。**条件数衡量的是“解对扰动的敏感度”，不是“模型好不好”**：只要岭罚把最小奇异值托住，条件数大并不必然导致测试误差大。

真正会出事的是另一头：条件数大到浮点精度撑不住时，式 (28-3) 的解会失去意义。本章 160 个中心、四倍宽度时条件数 1.53×10⁹，双精度还有约七位有效数字，勉强可用；**若换成单精度，这个配置已经不可信**。工程上的判据是：条件数超过 10^{12} 就不要用正规方程，改用 QR，或者直接减中心。

条件数还有一层来源与结构无关：八个输入特征本身高度相关。u、u/q、$T_{\mathrm{in}}u/q$、u^{2} 四个量都由 u、T_{in}、q 决定，同时喂给模型时设计矩阵的列天然近似共线。**这就是岭罚不能省的原因，也是为什么换中心放置方式只能在一个数量级内改善条件数**——实验 C 里三种放法的条件数落在 5.74×10^{5} 到 8.37×10^{6} 之间，根源在特征，不在中心。

28.3.2 局部支撑与外推

径向基的基函数有紧支撑的近似性质：输入离所有中心都远时，ϕ_j 同时趋零，输出趋近偏置 b。于是有一个简单的外推判据：

$$\lim_{\|\mathbf{x}\|\to\infty}\hat{y}(\mathbf{x}) = b = \bar{y}_{\mathrm{train}} \tag{28-10}$$

式 (28-10) 说的是，**径向基在远离训练数据的地方会退回训练集均值**。这个性质常被批评为“外推无能”，但它其实是可预期的失败模式——不会给出离谱的外推值，只会给出一个保守的常数。对比之下，前馈网络在域外的行为完全由最后一层的线性外推决定，**可能给出比真值高几十开的荒谬预测，也可能恰好蒙对**。

本章的实测给出一个折中结论。域外区间阀位 0.96 到 1.00，径向基误差 1.184 K，前馈网络 1.213 K，线性模型 1.381 K。**径向基没有崩，前馈网络也没有崩**，原因是 0.95 到 1.00 只差 0.05 个阀位，离训练中心不算远，式 (28-10) 的极限还没起作用。若把域外推到 1.30（超

出执行器量程），径向基会退回均值、误差跳到十几开，而前馈网络会线性外推、误差更大且方向不可控。

图 28-1 右图能看到这条性质的实际强度：本章选到的宽度是最近邻中位数的 4 倍（$\sigma = 5.53$），核很宽，所以径向基在 0.96 到 1.00 之间只升高 0.50 K，与真值的 0.44 K 接近，并没有明显压平；若把宽度缩到 0.5 倍，同一段区间会迅速变平。**退均值要看宽度**，宽核把“外推无能”推迟了，也把几何巧合伪装成了物理正确——**如果外推区的形状是碰巧对的，换一个工况就会错**。

工程上要有一条可执行的“是否在外推”判据，而不是等误差变大了才发现。做法是：对每个待预测样本，算它标准化后到最近中心的欧氏距离，再与训练集同一统计量的 95% 分位数比较；超过这个分位数的样本标为外推样本，误差**单独报告**，不与域内误差混在同一个均方根里。本章的域外任务是这么单独列出的——200 段全部落在训练区之外，所以它的 1.184 K 是一个纯外推指标，**不能与域内的 1.025 K 直接比大小**。

28.3.3 消息传递的两个性质

式 (28-4) 的聚合有两个可证明的性质，它们决定了图网络适合什么任务。

性质一，置换等变。把节点重新编号，只要邻接矩阵跟着换，输出只是跟着换位置，数值不变。证明只需注意求和与节点编号无关。这条性质在管道上意义有限（测点有物理顺序），但在**测点增删**时很重要：加一个测点、删一个测点，不需要重训整个网络，只需改邻接矩阵。

性质二，感受野随层数线性增长。一层消息传递后，节点 i 的表示含有一跳邻居的信息；两层后含有两跳；L 层后含有 L 跳。本章的链式管道上，两层的感受野是左右各两个测点，覆盖 1.0 米；管道总长 4.75 米。**这就是图网络 1.266 K 的误差来源**：中心点只能看到附近五个测点，看不到上游的入口温度，而入口温度恰恰是决定整条曲线的参数；全连接网络把二十个测点全展平，没有这个限制，所以做到 0.532 K。**感受野既是图网络的优点，也是它的天花板**。

要让误差再降，有三条路：加深到五层（感受野 2.5 米）、把邻接矩阵做成多跳（$A + A^2 + A^3$）、或者给每个节点附一个全局特征（流量已经附了，入口温度没有）。**本章没有做这三件事**，因为目的是展示“拓扑先验在缺测时的作用”，不是把管道任务做到极限。**读者若要在自己的装置上复现这条结论，先量一下“信息传播半径”够不够**，再决定堆几层。

顺带把计算量说清。消息传递的乘加次数与边数成正比，链式管道二十个节点只有十九条边，两层就是三十八次聚合；全连接网络把八十维输入整段送进第一层，每个输出单元都与全部八十个输入相连，**参数量随节点数线性增长，图网络的参数量与节点数无关**。所以节点一多，图网络省参数的优势会放大——但本章只有二十个节点，这个优势换不来精度，只换来从 1313 降到 177。

28.3.4 物理残差为什么能改善外推

式 (28-6) 的物理项在配点上算，配点铺到 1.00，而标签只到 0.95。**这就是改善外推的机制：网络在 0.96 到 1.00 上没有标签，但有方程约束**。

把这个机制写成局部形式。设网络输出在配点 $\mathbf{x}_c$ 处的残差为 $r(\mathbf{x}_c)$，物理项把这批残差的平方和压小。若方程本身正确，压小残差等价于把预测拉向“满足守恒律的那个解”；若方程有系统偏差，压小残差等价于把预测拉向一个错误的流形。**两种情况的区别不在算法，在方程准不准**。

本章的方程是准的：式 (28-7) 与生成数据的指数式在数学上等价，$UA = 2.4u$ 可以由两式对照推出来。所以物理项有效。实测：权重 0.3 时域外 2.044 K，无物理项时 2.291 K，降了 0.247 K；域内从 1.836 K 降到 1.654 K，降了 0.182 K。**两个方向同时改善**，说明这不是“用外推换内插”的拆东墙补西墙。

权重继续加到 1.0，域内降到 1.536 K（最好），域外反弹到 2.289 K（几乎回到无物理项的水平）。这个拐点可以用偏差—方差的语言解释：权重过大时，网络被方程主导，数据项只管住 100 个标签点，**网络在标签之间靠方程“猜”，在标签之外靠方程“推”**；而方程是近似的（UA 取常数乘阀位，忽略了温度对换热系数的影响），偏差随权重增大而累积。**最优权重不是越大越好，而是让数据项与物理项的量级相当**。

量级相当这句话可以操作：训练时把两项损失分别打印出来，若物理项比数据项小两个数量级，说明权重偏低、方程没起作用；若物理项比数据项大，说明权重偏高、网络开始忽略标签。**这个判据不依赖具体任务，换一个对象也成立**。

再补一句适用条件。物理项的相对价值随标签数下降：100 段标签时它把域外误差从 2.291 K 压到 2.044 K，降幅 10.8%；标签补到 1000 段后，纯数据网络已经做到 1.071 K 与 1.213 K，物理项没有可降的空间了。**这不是方程变差了，是数据把方程能提供的信息补上了**——先验与数据是替代关系，不是叠加关系。

28.3.5 逆模型的误差如何进入闭环

设对象稳态映射为 $f(u)$，逆模型为 $\hat{g}$，真实逆为 $g = f^{-1}$。纯前馈时指令为 $\hat{u} = \hat{g}(T_{\mathrm{sp}})$，对象稳态输出为 $f(\hat{u})$，稳态残差为

$$e_\infty = f\left(\hat{g}(T_{\text{sp}})\right) - T_{\text{sp}} \approx \left.\frac{\partial f}{\partial u}\right|_{u^*} \left[\hat{g}(T_{\text{sp}}) - g(T_{\text{sp}})\right] \tag{28-11}$$

式 (28-11) 是一阶泰勒展开。它说明**逆模型的指令误差被对象增益线性放大**。本章工作点上对象灵敏度约 35 K 每单位阀位，实测稳态残差 1.321 K，反推指令误差 0.038 个阀位——与逆模型的训练残差 0.036 个阀位在同一量级。

加上比例校正后，指令变成 $\hat{u} + k_p e$，闭环稳态满足 $f(\hat{u} + k_p e) = T_{\text{sp}}$。在小误差下线性化，残差被压缩为

$$e_\infty \approx \frac{e_{\text{ff}}}{1 + k_p \left.\frac{\partial f}{\partial u}\right|_{u^*}} \tag{28-12}$$

式 (28-12) 里 e_{ff} 是纯前馈残差。本章取 $k_p = 0.03$、灵敏度 35，分母为 $1 + 1.05 = 2.05$，理论上残差应从 1.321 K 降到 0.64 K 左右；实测降到 0.388 K，比线性化预测更好。**偏差来自两点**：一是灵敏度随工作点变化，35 只是局部值；二是前馈与反馈的叠加让工作点移动，实际灵敏度更大。

式 (28-11) 与 (28-12) 还给出一条工程判据：**若对象增益跨工况变化超过三倍，固定 k_p 的比例校正会在高增益区振荡、在低增益区迟钝**，此时要么做增益调度（第44章），要么把逆模型做得更准，让前馈承担主要工作。

还有一条容易混淆的等价关系：**训练残差不能直接当闭环精度用**。本章两者接近（0.036 个阀位对 0.038 个阀位）是特定条件下的吻合——闭环工作点落在训练分布内部，而且对象就是训练数据的来源。若闭环跑到训练区外，或者对象有未建模的时变，同样的训练残差会对应大得多的稳态偏差。**闭环精度只能闭环实测，开环指标最多用来做量级预估**。

28.3.6 执行器饱和与积分饱和

本章的阀位限幅是 0.30 到 0.95。纯逆模型的前馈指令不会越界（它学的是训练区间内的映射），但叠加比例校正后会。实测：逆模型加比例校正的饱和时间占比 14.5%，纯比例积分 0.8%，纯逆模型 0%。

14.5% 这个数偏高，原因是设定值从 60 跳到 75 时，前馈给出约 0.55 的阀位，比例项再叠加 $0.03 \times 15 = 0.45$，合计 1.00，超过上限。**饱和期间反馈失效，积分项继续累积**，这就是经典的积分饱和。本章的实现加了一条退积分逻辑：指令项到边界且误差与越界方向同向时，退回这一步的积分。**没有这条逻辑，恢复期会出现明显的过冲**。

还要注意一个容易被忽略的点：饱和占比 14.5% 并不意味着“控制变差了”。这一段的稳态残差 0.388 K 仍是三种结构里最好的，因为饱和只发生在阶跃后的十几步，稳态时指令回到量程内。**评价饱和要看“发生在什么时候”，而不是只看总时长**。

28.3.7 训练稳定的三个必要条件

本章实验 F 用四个配置把训练稳定性拆开，每个配置只改一个因素。

输入标准化。未标准化时，八个特征里入口温度约 35、流量约 2、阀位约 0.6，量纲差两个数量级；梯度范数实测最大 238.69，末轮仍有 1.3873，训练误差 1.540 K、测试 1.560 K。标准化后最大梯度 2.04，末轮 0.0078，测试误差 1.081 K。**同一网络、同一学习率，只差一次标准化，误差差 0.48 K**。

梯度裁剪。标准化加裁剪 1.0、学习率提到 0.03，最大梯度 8.51，测试误差 1.104 K。裁剪让训练能承受更大的学习率，但**它不改善最终精度**——1.104 与 1.081 相差 0.023 K，落在门限内。裁剪的价值在防止偶发的梯度尖峰把参数打飞，不在提高精度。

学习率过大。学习率 0.30 时最大梯度 23 217，训练误差 5.539 K、测试 5.470 K，模型完全没有学到东西。这个现象的理论解释见第29章，本章只留一条判据：**若训练损失在前几十轮不降反升，先把学习率除以十**。

三个条件合起来是一条工程顺序：先标准化，再看梯度范数，最后调学习率。**顺序不能反**——在未标准化的输入上调学习率，调到一个不发散的值需要试很多次，而标准化之后默认值往往就能用。

最后交代训练预算。本章的前馈网络一律全批量、四百轮，750 个样本意味着每轮一次参数更新，四百轮就是四百次更新。这个预算偏小，但对本章的对照是够的：所有结构用同一预算，比较的是“相同更新次数下谁学得更好”，而不是“谁能训到收敛”。**预算一变排序就可能变，所以本章把它写死并贯穿全部对照**；读者换预算重跑，绝对数字会动，机制不会动。

28.4 离散对应与数值实现

这一节把式 (28-1) 到式 (28-12) 落成可以执行的代码。模型形式已经在前面定死，剩下的全是实现细节：输入怎么缩放、中心放在哪、宽度取多大、掩码怎么乘进矩阵、物理残差怎么避免除零、耗时怎么量。这些细节不改变任何一条公式，却能把同一个模型的误差拉开一个数量级。本节所有数字来自 `code/ch028_sim.py`，种子 20261028，单线程。

28.4.1 五个必须先定的实现决定

从纸面公式到能跑的脚本，要依次定五件事。第一是输入与输出的标准化方式，第二是径向基中心的位置，第三是核宽度，第四是图网络里缺失读数的处理，第五是物理残差的量纲与数值保护。前四件在 28.4.2 至 28.4.4 展开，第五件在 28.4.5，训练循环与计时口径放在 28.4.6。顺序不是随意的：**前一项定错，后一项的调参就失去意义**。

标准化用逐特征的方式，写成

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \qquad i = 1, \dots, 8 \tag{28-13}$$

其中 μ_i 与 σ_i 只从训练集统计，测试集与域外集一律沿用训练集的均值与方差。这一条看起来是常识，实际最常被违反：若先用全部数据算均值和方差再切分，测试集的分布信息就通过这两个标量漏进了训练。本章所有实验都在切分之后才计算 μ_i 与 σ_i，径向基的设计矩阵、前馈网络的第一层输入、图网络的节点特征都遵守同一条规则。

输出也要标准化。出口温度的量级约 65，若直接回归，初始损失的数值在四千以上，梯度尺度与输入侧完全不匹配。本章把目标减去训练均值、除以训练标准差后再送入网络，推理时再反变换回来。这一步与输入标准化合起来，就是实验 F 里梯度范数从 238.69 降到 2.04 的全部原因。

反变换只能写在一处。本章的做法是把“标准化、前向、反变换”封成一个预测函数，误差计算、画图、闭环仿真全部调用它，任何地方都不手写 `pred * std + mean`。**这不是代码洁癖，是防止量纲错误**：一旦某个分支忘了反变换，误差会以“标准差为单位”报出来，数值小得离谱却不会报错，和 §28.6.6 的广播错误属于同一类风险。

28.4.2 中心放在哪里

径向基的中心决定了基函数覆盖输入空间的哪些位置，三种常见放法在本章都实现了。随机抽样最简单：在标准化后的训练集里无放回抽 k 个样本直接当中心，复杂度是 $O(k)$，缺点是抽到的点可能挤在样本密集区，稀疏区没有基函数覆盖，外推时只能靠远处的核衰减。

聚类中心用 k 均值：先跑十次不同初值取最优，再把簇心当中心。它让中心按样本密度分布，密集区自动加密，代价是一次聚类的迭代开销与对初值的依赖。本章固定 `n_init=10`、`random_state` 取种子，保证可复现。

正交最小二乘（OLS）是从候选集里贪心挑：先随机取至多 320 个候选中心，每次计算候选基函数与当前残差的相关度

$$s_j = \frac{\left| \boldsymbol{\phi}_j^{\mathsf{T}} \mathbf{r} \right|}{\|\boldsymbol{\phi}_j\| \, \|\mathbf{r}\|} \tag{28-14}$$

取 s_j 最大的那个加入基组，再用最小二乘重解系数、更新残差，重复 k 次。它的目标是让每个新中心都带来最大的残差下降，因此中心数与所需基函数个数贴近，不必事先铺得很密。

三种放法在 $k = 40$、宽度用同一套内部验证规则选出时的实测对照见表 28-3。

表 28-3 三种中心放置方式的对照（$k = 40$，750 段训练、250 段测试）

放置方式	测试误差 / K	设计矩阵条件数	可训练参数
随机抽样	1.034	6.12×10^{5}	361
聚类中心	1.025	8.37×10^{6}	361
正交最小二乘	1.031	5.74×10^{5}	361

三者的测试误差落在 1.025 至 1.034 K 之间，极差 0.009 K，**远小于本章 0.05 K 的判定门限**。表 28-3 的结论要照口径说：在这组数据上，中心怎么放不影响精度，聚类中心的 1.025 K 不能当作“更好”来引用。条件数倒是差了一个数量级，随机抽样与正交最小二乘在 10^{5} 量级，聚类中心在 10^{6} 量级，但三者都在可解范围内。这说明**决定数值性质的主要不是中心位置，而是宽度**——下一小节会看到，宽度改一个倍数，条件数就能跳四到五个数量级。

28.4.3 宽度怎么定

宽度 σ 是径向基里最难凭直觉定的量。太小，每个基函数只在中心附近亮一下，基组之间不重叠，设计矩阵接近单位阵、条件数很小，但基函数覆盖不到样本之间的空隙，测试点上全是零响应；太大，所有基函数几乎相同，设计矩阵趋于秩一，条件数爆炸，最小二乘解对噪声极度敏感。

本章的做法是两步。先给一个与数据尺度挂钩的初值：取所有中心两两距离中最近邻距离的中位数，记为 σ_0。这个量随中心密度自动缩放，中心密则 σ_0 小，中心疏则 σ_0 大，不需要人为指定绝对尺度。再在这个初值上乘一个倍数 β，用训练集内部切出的验证集选：

$$\beta^{\star}=\arg\min_{\beta\in\{1,2,3,4\}}\ \mathrm{RMSE}_{\mathrm{val}}(\beta) \tag{28-15}$$

切法是固定种子打乱训练集，前 20%（不少于 50 段）作验证、其余作拟合；每个 β 训一次、在验证集上算误差，选完再用全部 750 段重拟合一次。**测试集全程不参与选择**，否则报出来的误差就是乐观偏差。

实测两个配置的选择结果：$k=40$ 时选中 $\beta=4.0$，$\sigma_0=1.382$，实际宽度 $\sigma=5.527$，条件数 8.37×10^{6}，测试 1.025 K；$k=80$ 时选中 $\beta=3.0$，$\sigma_0=1.047$，实际宽度 $\sigma=3.140$，条件数 7.13×10^{6}，测试 1.045 K。$k=40$ 的结果落在网格右边界，说明网格可能不够宽——β 再大也许还能降一点。这里没有再扩，原因是实验 B 的二维扫描已经给出趋势：β 从 2 加到 4 时，$k=40$ 的条件数从 2.40×10^{4} 涨到 8.37×10^{6}，涨了三百多倍，而测试误差只从 1.038 K 降到 1.025 K。

这笔账值得单独算：**误差改善 0.013 K，条件数恶化 350 倍**。继续加宽，精度的边际收益会迅速耗尽，而数值稳定性继续变差，换一批数据就可能解不出来。宽度选择因此不是单目标优化，而是在精度与条件数之间取一个可接受的点。工程上更稳的做法是把网格扩到 6 或 8，同时给条件数设一个上限（比如 10^{8}），超限的配置直接淘汰，而不是让验证误差单独决定。

作为反面对照，实验 B 里中心 10 个、$\beta=0.5$ 的配置测试误差 4.612 K，$\beta=1.0$ 时降到 2.749 K。同一个模型、同一批数据，只把宽度乘了 2，误差降了 41%。**宽度错一个量级，误差就差一个量级**，这比中心放哪重要得多。

还要分清两个“最近邻”：σ_0 用的是中心之间的距离，不是样本到中心的距离。前者只与中心怎么放有关，中心一换 σ_0 就变；用它作尺度，宽度会自动跟中心密度匹配。若改用样本到最近中心的距离，中心放得稀疏时这个量会变大，宽度跟着变大，于是“中心少”的缺陷被宽度掩盖，实验 B 里那条斜带就看不出来了。**标定宽度要用拓扑量，不要用拟合残差量**，否则调参与评估会互相污染。

28.4.4 掩码聚合怎么写

图网络的实现全在张量形状上，写错一处就会静默地算错。本章的输入是 `x` 形状 $(B,N,4)$、`mask` 形状 (B,N)、`adj_off` 形状 (N,N)，其中 B 是批内样本数、$N=20$ 是测点数、4 是节点特征维数，`adj_off` 只含邻居边、不含自环。聚合分两轮，每轮的权重张量按

$$W_{ij}^{(b)}=A_{ij}\,m_{j}^{(b)},\qquad \tilde{\mathbf{h}}_{i}^{(b)}=\frac{\sum_{j}W_{ij}^{(b)}\mathbf{h}_{j}^{(b)}}{1+\sum_{j}W_{ij}^{(b)}} \tag{28-16}$$

计算，分母里的 1 就是自环。代码上是三行：`w = adj.unsqueeze(0) * mask.unsqueeze(1)` 得到 (B,N,N)，`den = w.sum(-1, keepdim=True) + 1.0`，`nb = torch.bmm(w, x) / den`。第二轮的权重张量不变，只是把 `x` 换成第一轮输出的隐状态，并在残差上加回自身。

有三个地方最容易写错。第一，缺失读数在特征里先填 0，再靠 m_j 把它从分子和分母同时剔除；如果只填 0 不乘掩码，网络会把“0 ℃”当成真实读数，比缺失本身更糟，因为 0 ℃ 落在这条管道的测量范围之外，是一个极强的错误信号。第二，分母必须加 1；邻居全部掉线时分母退化为 1、输出为 0，这是有定义的行为，去掉这个 1 就会除零。第三，掩码要按样本独立生成，训练和推理用同一套生成规则；本章每段样本每个测点以 0.20 的概率独立掉线，中心测点自己也可能掉线，测试集同样如此。

这三条的效果可以在实验 G 里量化。只看中心点的逐点前馈在 20% 掉线时是 3.073 K，它没有邻居可用，中心点一缺失就只能读到一个零；图网络 1.266 K，掩码把缺失邻居从分子分母同时剔除，信息从两侧补进来；全连接 0.532 K，它把二十个测点的读数全展平，容量最大。**掩码解决的是“缺失不能被当成零”，不解决“信息够不够”**——图网络的邻居只有左右各两个，感受野决定了它拿不到上游的入口温度，所以它赢不了全连接。要判断自己该用哪种，先看节点数与样本量，再看掉线率，最后才看聚合规则。

邻接矩阵的构造要单独核对一次。本章的管道是链式，第 i 个节点只与 $i-1$ 和 $i+1$ 相连，$A_{ij}=1$ 当且仅当 $|i-j|=1$；两端各只有一个邻居，所以度数不是常数，这正是式 (28-5) 分母要按节点实际可用邻居求和、而不是除以固定值的原因。**若把邻接矩阵写成对称的全一矩阵（每个节点与所有节点相连），聚合退化成全局均值**，图网络就变成了一个没有位置信息的池化层，掉线稳健性还在，空间分辨率没了。写错这一处不会报错，误差只会从 1.266 K 变成另一个说不清来源的数。

28.4.5 物理残差的数值稳定

式 (28-7) 的对数平均温差在两端温差接近时是 $0/0$ 型。阀位接近 1.00 时出口温度逼近热端温度，$\Delta_1=T_{\mathrm{hot}}-\hat{T}_{\mathrm{out}}$ 趋于 0，直接取对数会得到负无穷或 NaN，残差平方后整条损失变成 NaN，训练在几十轮内崩掉。本章用下界截断：

$$\Delta_1=\max\!\left(T_{\mathrm{hot}}-\hat{T}_{\mathrm{out}},\,0.5\right),\qquad \Delta_2=\max(T_{\mathrm{hot}}-T_{\mathrm{in}},\,0.5) \tag{28-17}$$

0.5 K 这个数的来源是测量噪声：本章传感器噪声标准差 0.4 K，0.5 K 略大于一个标准差，低于这个量级的温差在数据里本来就分辨不出来。下界取太小（比如 0.01 K）挡不住数值爆炸，取太大（比如 5 K）会把真实的小温差工况全部改写。**下界应当与测量分辨率同量级**，这是一条可以迁移到别的对象的经验。

第二层保护是把换热系数写成可训练的正数。式 (28-7) 里的 UA 未知，本章令 $a=\mathrm{softplus}(\theta)\cdot 3$，$\theta$ 初值 0，于是初值 $a=2.08$，正好接近生成数据用的真值 2.4。$\mathrm{softplus}$ 保证 $a>0$，避免训练中出现负的换热系数这种非物理解；θ 与网络权重放进同一个 Adam，1200 轮、学习率 8×10^{-3} 一起更新。实验 D 里 $\lambda=0.3$ 时反标定出 $a=2.29$，与真值 2.4 相差 0.11，说明这个参数确实被数据与方程共同约束住了。

第三层是量纲归一。残差写成

$$r=\frac{q\left(\hat{T}_{\mathrm{out}}-T_{\mathrm{in}}\right)-a\,u\,\mathrm{LMTD}}{q\left(T_{\mathrm{hot}}-T_{\mathrm{in}}\right)} \tag{28-18}$$

分母是冷流体可能吸收的最大热流，于是 r 是一个无量纲的比例，量级在 1 附近。若不归一，流量 q 在 1.2 到 3.0 之间变化，残差量级差 2.5 倍，同一个 λ 在不同样本上的实际权重完全不同，扫描出来的曲线就没有可比性。首版实现没有归一，$\lambda=0.001$ 就把域内误差从 1.836 K 抬到 2.101 K，属于典型的“物理项抢戏”。

三层保护合起来，才有实验 D 那条干净的非单调曲线：域外误差 2.291 K（$\lambda=0$）降到 2.044 K（$\lambda=0.3$），再反弹到 2.289 K（$\lambda=1.0$）。**物理项能不能帮上忙，先取决于它算得对不对，再取决于权重给得合不合适。**

还有一处实现细节：物理项取的是残差平方的均值，不是求和。若取求和，配点数从 600 加到 1200 时物理项会自动翻倍，同一个 λ 的含义就变了，实验 D 的扫描曲线也不可复现。**损失里的每一项都要归一化成“每样本平均”，否则样本数、配点数一变，权重就得重调**，这是把实验结论迁移到别的对象时最容易踩的坑。

28.4.6 训练循环与计时口径

本章的训练循环是固定五步：清梯度、前向、算损失、反向、更新。前馈网络与图网络都用全量批，不做 mini-batch，原因是样本量只有 750 到 800 段、单批一次前向的耗时在毫秒级，全量梯度的方向比随机小批更稳，也省掉了批大小这个超参数。学习率统一 8×10^{-3}（图网络 5×10^{-3}），优化器 Adam，轮数 400（物理约束网络 1200 轮，因为它同时要标定 a）。

日志每轮记两个量：总损失与梯度范数的最大值。记最大值而不是均值，是因为训练崩掉通常由偶发的尖峰触发，均值会把尖峰抹平。实验 F 里未标准化的配置最大梯度 238.69、末轮仍有 1.3873，这两个数就是靠逐轮记最大值抓到的；若只看均值，会误以为训练一直很平稳。

耗时按单样本前向统计，口径写清楚才能比较：先预热 100 次（触发算子选择与内存分配），再连续测 500 次取平均，单线程运行，输入是形状 $(1,8)$ 的随机向量。单线程这一条不能省，否则线程调度的抖动会盖住模型之间的真实差异。实测结果：前馈网络宽度 8 时 24.4 μs、16 时 23.9 μs、32 时 23.5 μs、64 时 23.1 μs、128 时 23.8 μs、256 时 27.0 μs、1024 时 94.0 μs；径向基中心数 10 时 119.8 μs、40 时 115.1 μs、160 时 121.6 μs、640 时 126.7 μs。同一台机器上前一次运行的对应值是 22.3 至 25.5 μs 与 113.7 至 116.4 μs，**绝对值有几个百分点的漂移，量级与形状稳定**。

两组数字的形状很不一样，值得解释。前馈网络在 8 到 128 宽之间几乎是一条平线（23.1 至 24.4 μs），因为单样本前向由访存和框架固定开销主导，乘加次数在这个区间还不是瓶颈；256 宽抬到 27.0 μs，宽度跳到 1024 时 94.0 μs，是矩阵乘开始走分块内核、开销显现。径向基则在 115 到 127 μs 之间，中心数从 10 加到 640 只涨了 10 μs，因为 sklearn 的成对距离计算有固定开销，大部分时间花在固定开销上而不是距离计算上。**这恰恰是径向基部署时的隐患**：它的耗时不由模型规模决定，而由实现库的固定开销决定，想压到几十微秒必须换手写实现。

对本章的对象，采样周期 1 s，两种结构都远低于预算，选哪个只看精度与可解释性。若周期压到 1 ms，前馈仍在预算内，径向基的 120 μs 约占 12% 的预算，余量不大；若周期压到 100 μs，两者都要重新评估，且必须把特征计算、标准化、反变换的时间一并算进去，不能只量网络本身。

28.5 工程含义与适用边界

上一节讲怎么把模型写对，这一节讲什么时候不该用它。三类结构各有自己的成立条件，条件不满足时它们不会报警，只会安静地给出一个看起来合理的错答案。这一节把这些条件逐条摊开，并给出一张按任务选结构的对照表。

28.5.1 径向基的三个失效条件

径向基的第一个失效条件是维度相对样本太高。本章 8 个特征、750 段训练、40 个中心，平均每个中心覆盖约 19 段样本，核宽度中位数 1.382 个标准差，基函数之间有足够重叠，所以能做到 1.025 K。若特征增到 50 维而样本仍是 750 段，同样的 40 个中心在 50 维空间里稀疏得几乎不重叠，最近邻距离的中位数会被少数离群中心拉大，宽度初值随之失真，设计矩阵退化成近似对角阵，测试点上大量基函数输出接近零。**判据是每个中心覆盖的样本数**：低于 10 段就要警惕，低于 5 段基本不可用。

第二个失效条件是输入跑出训练覆盖的范围。径向基的响应随距离指数衰减，输入越远，所有基函数越接近零，输出趋近偏置项，也就是训练集的平均温度。实验 A 里径向基-40 域内 1.025 K、域外 1.184 K，看起来只差 0.159 K，但域外的真值范围是 64.0 至 86.9 ℃、均值明显高于训练集，退均值意味着系统性偏低。**这个 1.184 K 是“落在均值附近”换来的，不是模型真的会外推**。工程上要把输入范围当作硬边界：超出训练范围就报警，不要相信模型给的值。

第三个失效条件是条件数。$k = 40$、$\beta = 4$ 时设计矩阵条件数 8.37×10^{6}，双精度求解还够用，换成单精度就会丢掉有效位。若中心数加到 160、宽度继续加大，实验 B 里条件数到 1.53×10^{9}，此时最小二乘解对输入的第 6 位有效数字都敏感，同一批数据换一台机器可能得到不同的系数。**条件数超过 10^{8} 就应当缩小宽度或减少中心**，不要指望正则化项能救回来——本章的岭系数只有 10^{-6}，对这个量级的条件数几乎没有作用。

28.5.2 物理项在方程不准时的反噬

物理约束网络的收益完全建立在“方程写对了”这个前提上。本章用的对数平均温差式与生成数据的指数式在稳态下等价，UA 只差一个待标定的系数，所以 $\lambda = 0.3$ 时域外能从 2.291 K 降到 2.044 K。这是一个理想情形：方程形式正确、只有一个未知参数、参数还被数据约束到 2.29（真值 2.4）。

现实里三件事都可能不成立。第一，方程形式可能漏项，比如换热器结垢让 UA 随运行时间漂移，而模型把 UA 当常数，物理项就会把漂移当成噪声压制，逼网络去拟合一个错误的平衡点。第二，参数可能不可辨识，比如流量与换热系数在方程里总是以乘积出现，数据无法把两者分开，此时标定出的 a 只是乘积的一个分摊，换一个流量计就要重标。第三，边界条件可能错，比如热端温度实际在 92 至 97 ℃ 之间波动，而模型固定写 95 ℃，物理项会把 2 K 的偏差均匀摊到所有样本上。

这三种情况下物理项不会让误差变大得离谱，但会把偏差固化。实验 D 的 $\lambda = 1.0$ 就是一个提示：域内误差 1.536 K 是五档里最好的，域外却反弹到 2.289 K，几乎回到没有物理项的水平。**方程主导训练时，标签的信息被压到只剩 100 个点，模型学到的更像是方程的解而不是对象的解**。上线前必须做一件事：把数据项与物理项的损失分别打印出来，两者量级相当才继续，物理项大一个数量级就说明权重给过头了。

28.5.3 逆模型上线的四条检查

逆模型直接把“想要的出口温度”翻译成“阀位”，看起来最干净，但它是开环的，模型误差没有任何环节吸收。实验 E 的数字把这件事说得很清楚：纯逆模型的全程误差 3.793 K、稳态误差 1.321 K，饱和占比 0.0%。饱和为零不是优点，而是因为前馈指令从来不会越界——它压根不知道自己的指令有没有把温度送到位。

第一条检查是训练覆盖。本章逆模型的训练集是 2000 组，阀位在 0.30 至 0.95 之间均匀取值，目标温度由正向映射生成，因此覆盖了约 40 至 88 ℃ 的稳态范围。上线后若设定值超出这个范围，逆模型就在做外推，而外推的行为已经在 28.5.1 里说过：退向训练均值，也就是给出一个“中间阀位”，看起来不荒谬，实际完全不对。

第二条检查是量程与饱和。纯逆模型饱和 0.0%，逆模型加比例校正饱和 14.5%，PI 反馈 0.8%。加反馈后饱和上升，是因为反馈要修正前馈的偏差，修正量在阶跃初期很大，把指令推到 0.95 的上限。**饱和占比高不一定是缺陷，要看它发生在什么时候**：本章的 14.5% 集中在每段阶跃后的十几步，稳态时指令回到量程内，稳态误差仍是三者里最好的 0.388 K。若饱和持续到稳态，那就是执行器选型不够，属于设计问题而不是控制结构问题。

第三条检查是模型误差由谁吸收。纯逆模型没有任何环节吸收，所以 1.321 K 的稳态误差原样留在闭环里；加一个比例系数 $k_p = 0.03$ 后降到 0.388 K，降幅 0.93 K。这个比例系数很小，对应的阀门动作只有 0.03 个阀位每开尔文，几乎不改变前馈的主导地位，却足以把慢漂移拉回来。**前馈负责“大致对”，反馈负责“最后一点”**，两者的分工在这一档数字上体现得最清楚。

第四条检查是不要把最大偏差当结构指标。三种结构的最大偏差分别是 15.446、15.081、14.929 K，几乎一样，因为设定值从 60 ℃ 跳到 75 ℃ 本身就是 15 K 的阶跃，任何结构都必须走过这段距离。**用最大偏差比较控制结构，比的是阶跃幅度，不是控制品质**。要比就看稳态误差与饱和占比，这两个量才反映结构差异。

28.5.4 图网络靠什么成立

图网络的收益来自两个先验：拓扑有意义、缺失可识别。本章两个先验都成立：20 个测点沿管道等距排布，温度沿程指数下降，相邻测点确实强相关；掉线按测点独立生成，掩码准确记录了哪个点缺失。在这个条件下，20% 掉线时图网络 1.266 K，逐点前馈 3.073 K，全连接前馈 0.532 K。**先验成立不等于精度最高**：图网络比逐点好 1.807 K，比全连接差 0.734 K。

图网络真正换来的是两样东西。第一是参数：177 个对 1313 个，少 7.4 倍，训练样本只有 800 段时这不是小事。第二是退化斜率：掉线率从 0 升到 30%，逐点前馈从 0.615 升到 3.705 K（放大 6.0 倍），全连接从 0.197 升到 1.169 K（放大 5.9 倍），图网络从 0.344 升到 1.166 K（放大 3.4 倍）。**三条曲线的起点不同、斜率不同**：全连接起点最低但爬得最陡，图网络起点居中但最平，于是两者在 30% 掉线附近相

交（1.166 对 1.169 K，差 0.003 K，落在门限内）。这个交叉点是本节最该记住的定量结论——**图网络不是“更准的模型”，而是“参数受限时退化最慢的模型”**。

第一个先验不成立时，消息传递会变成污染源。若拓扑是“物理上相邻但工艺上不连通”，比如两条并联支路在图上相邻、实际流量互不相干，聚合就会把另一条支路的温度混进来。本章的邻接矩阵是固定的链式结构，工程上更常见的是按工艺流程图建图，边的权重可能还需要按流量分配。**图结构是模型的一部分，建错了不会报错，只会让误差悄悄变大**。

第二个先验不成立时，掩码依然正确但信息量骤降。实际掉线往往是相关的：一段总线故障会让连续五个测点同时失效，同一块采集卡上的点会一起掉。此时 m_j 仍然如实记录了缺失，但邻居全没了，聚合退化到自环，模型只能靠中心点自己的读数。本章没有测这种相关掉线，这是一个明确的缺口；要补的话，把独立掉线改成“随机选一段连续区间置零”，重跑实验 G 即可。**对图网络来说，相关掉线比独立掉线更致命**，因为它的全部信息都来自邻居；全连接还会损失固定位置上的那一列，但至少其余列仍在。

还有一条容易忽略：节点特征里必须显式包含“可用标志”。本章的 4 维特征是读数、位置、流量、掩码，掩码单独占一维，网络才能学会“这个读数不可信”。若把缺失读数填 0 却不给标志，网络看到的就是一个 0 ℃ 的强信号，而 0 ℃ 在这条管道上不可能出现，等于人为制造了一个极端离群点。实验 G 里逐点前馈的 3.073 K 就是这么来的：它只看到中心点，中心点一掉线就只能看到 0。

最后要说清楚图网络不适用的场合。0% 掉线时，图网络 0.344 K，比逐点前馈的 0.615 K 好，但比全连接的 0.197 K 差——**没有缺失时，聚合帮不上忙，参数共享反而压住了容量**。二十个节点、八百段样本这个规模上，全连接是最省事也最准的选择。图网络要到节点数上百、参数预算受限、或掉线率超过三成时才显出价值；这三种情形本章都没有真正压到极限，只能说趋势清楚。

28.5.5 结构与任务怎么配

把四组任务的实测结果放在一张表里，选型逻辑就清楚了。表 28-4 的每一行对应本章的一个任务，推荐结构按实测误差与本节的边界条件给出。

按任务列而不是按模型列，是因为同一个模型在四个任务上的名次会换位：径向基在静态任务里以 1.025 K 排第一，在逆模型任务里只是前馈的一环，在管道任务上则完全没有参赛。**模型没有绝对的好坏，只有与数据形态的匹配度**，这句话在选型时比任何精度排名都实用。

表 28-4 四个任务的结构选择与实测依据

任务	数据形态	推荐结构	实测依据
静态软测量	8 特征、750 段、无缺失	径向基或前馈网络	1.025 与 1.071 K，差 0.046 K 分不出来
域外工况	输入超出训练范围	物理约束网络	域外 2.044 K 对 2.291 K
逆模型控制	需要从目标反解执行器	逆模型加比例校正，或 PI	稳态 0.388 K（饱和 14.5%）对 1.099 K（饱和 0.8%）
管道测点	20 节点、20% 掉线	全连接前馈；节点多或参数受限时用图网络	0.532 K 对 1.266 与 3.073 K

表 28-4 的第一行要读得仔细：线性岭回归 1.108 K，比径向基差 0.083 K，超出 0.05 K 门限，说明这个对象确实有非线性；但径向基与前馈网络相差 0.046 K，落在门限内，两者选谁都行，此时按部署成本选——径向基的训练是一次线性求解，前馈网络要迭代四百轮，本次实测两者分别是 0.086 s 与 0.221 s，前一次运行是 0.240 s 与 0.204 s，**同一台机器上排序都能翻转，耗时差不构成选型依据**；真正可查的是径向基的中心位置与条件数，出问题时能定位。**在分不出高下的地方，用可解释的那一个**。

第二行的选择理由与第四行不同。域外任务选物理约束网络，不是因为它在域内更准（$\lambda = 0.3$ 时 1.654 K，比全量标签的 1.071 K 差），而是因为几何外推必然退均值，只有方程能提供范围外的约束。管道任务则是全连接最好，理由是容量：二十个测点、八百段样本，1313 个参数还没到过拟合的边界，把全部读数展平就能用；图网络的价值要等节点数上去、参数预算下来才体现。**两个选择的共同点是：先问数据形态允许什么，再谈模型先进不先进**。

第三行是一个权衡而不是一个答案。逆模型加比例校正的稳态误差 0.388 K 明显好于 PI 的 1.099 K，但饱和占比 14.5% 对 0.8%，差了一个数量级。若阀门是气动薄膜阀、动作寿命有限，14.5% 的饱和时间意味着检修周期缩短，此时选 PI 更稳；若阀门是电动调节阀、允许频繁动作，选逆模型加校正能拿到更好的稳态精度。**这一行没有唯一解，取决于执行器的物理限制**，而执行器的知识来自第12章，不在模型侧。

这张表不能跨对象照搬，但判据可以。四条判据按顺序问：样本是否覆盖全部工况（否则先划域外集）、每个可调参数对应多少样本（低于十段就别再加参数）、缺失是否与工况相关（相关则掩码要进特征而不只是加权）、执行器是否允许频繁动作（不允许就退回纯反馈）。**四个问题答完，结构基本就被约束住了**；答不出来的时候，加结构只会把不确定性藏起来。

28.5.6 上线前的八条检查

把本节与前两节的内容收成一份可执行的清单，八条，每条都能用一句话判断通过与否。

第一，切分先于标准化。均值与方差只从训练集算，测试集与域外集沿用。违反这条，报出来的误差会偏乐观，且偏多少无法估计。

第二，域外误差单独报。域内误差好看不代表外推可用，本章所有涉及外推的结果都分两列给出，没有合并成一个数。

第三，结构对照的训练预算对齐。同样的轮数、同样的学习率、同样的标准化，否则比的是调参功夫而不是结构。本章实验 A 的三种结构都在 400 轮、全量批、同一套标准化下训练。

第四，固定种子重跑，看结论稳不稳。本章所有数字是单次实跑（种子 20261028），没有做多次重复；因此凡是差异小于 0.05 K 的结论，本书一律按“分不出来”处理，不写成“某结构更优”。这一条是对读者负责，也是对自己负责。

第五，耗时要按单样本、单线程、含特征工程统计。本章量的是网络前向本身（23.1 至 126.7 μs），若把标准化、特征构造、反变换算进去，实际占用会更高；1 ms 周期下 120 μs 已占 12% 的预算，余量并不宽裕。

第六，饱和占比与稳态误差分开报。全程均方根误差会被阶跃段主导，三种控制结构的全程误差都在 3.3 至 3.8 K，看起来差不多；拆成稳态误差后才有 0.388、1.099、1.321 K 的区分。

第七，掉线率与掉线相关性都要测。只测独立掉线会高估图网络的稳健性，相关掉线（整段失效）才是现场更常见的情形。

第八，物理项上线前打印两项损失的量级。数据项与物理项量级相当才继续，物理项大一个数量级就退回调权重，不要靠域内误差判断——域内误差对物理项不敏感，实验 D 里 $\lambda = 1.0$ 的域内误差 1.536 K 反而是最好的。

这八条没有一条依赖具体的网络结构，换成任何数据驱动模型都适用。它们的共同点是：**把“模型在测试集上多准”换成“模型在什么条件下会错、错了以后由谁兜住”**。前者是论文口径，后者是工程口径，本章两种都要给。

用法是逐条打勾，不是逐条打分。任何一条不通过就停下来修，因为八条之间不是加权关系，是串联关系：切分错了，后面七条的结论全部作废；域外没有单列，物理项的收益就测不出来；训练预算没对齐，结构比较就是空的。**清单的价值在顺序，不在条数**。

28.6 仿真与验证

本章的全部数字出自一个脚本，单次运行约 16 秒，八组实验按 A 到 H 编号。A 比较线性、前馈、径向基三类逼近器；B 扫中心数与宽度；C 比较三种中心放置方式；D 扫物理项权重；E 跑三种闭环结构；F 对照四种训练配置；G 比较三种管道结构与四种掉线率；H 量两类结构的单样本耗时。绘图是另一个脚本，调用同一批函数，不另存中间结果，**任何一张图都能由正文给出的种子重跑复现**。

八组实验的用途不同。A、B、C 回答“逼近器怎么选”，D 回答“方程怎么进损失”，E 回答“网络输出能不能直接驱动执行器”，F 回答“训练为什么崩”，G、H 回答“数据残缺与实时性怎么权衡”。每组只改一个因素，其余保持默认，**这是本章唯一愿意接受的因果证据形式**：单次实跑、单因素对照、差异超过门限才下结论。

每组实验配一张主表，多数再配一张图。表给数值，图给形状，分工是硬性的：**表里的每个数都要能在图上找到位置，图上的每条曲线都要能在表里查到关键点**。万一两者对不上，以脚本输出为准，因为绘图可能经过抽样或平滑；本章的六张图都由同一批函数直接生成，没有另存中间结果，正是为了避免这种不一致。

28.6.1 实验 A：三类逼近器的精度与代价

实验 A 在同一划分上训练四种模型，结果见表 28-5。

表 28-5 三类逼近器的域内与域外误差（750 段训练、250 段测试、200 段域外）

结构	可训练参数	域内误差 / K	域外误差 / K	训练耗时 / s
线性-岭回归	9	1.108	1.381	0.012
前馈网络 2×32	1377	1.071	1.213	0.221
径向基 40 中心	361	1.025	1.184	0.086
径向基 80 中心	721	1.045	1.202	0.067

表 28-5 的域内一列排序是径向基 40、径向基 80、前馈、线性。径向基 40 与前馈相差 0.046 K，落在 0.05 K 门限内，**按口径只能说“分不出来”**；径向基 40 与线性相差 0.083 K，超出这个门限，说明这个对象的非线性是真的。参数效率上，径向基用 361 个参数做到前馈 1377 个参数的水平，差 3.8 倍，这是结构先验最直接的回报。

表 28-5 最后一列要打个折扣。同一天两次运行，四行耗时分别是 0.102、0.204、0.240、0.071 s 与 0.012、0.221、0.086、0.067 s，**径向基与前馈的先后顺序都翻转过**。同一台机器、同一段代码、同一个种子，能变的只有缓存、负载与算子选择。**这一列只用来确认“都在一秒以内”，不能拿来选型**；真要比较训练成本，得固定机器、重复多次取中位数。

域外一列的量级更值得看。四者的域外误差都比域内高，线性高 0.273 K（24.7%），前馈高 0.142 K（13.3%），径向基 40 高 0.159 K（15.5%），径向基 80 高 0.157 K（15.0%）。**径向基并没有在相对意义上更抗外推**，它的域外误差低只是因为域内起点低。这一点与“局部基函数会退均值”的直觉并不矛盾：厚度足够的核让退均值发生得很缓慢，误差被摊平了，而不是被消除了。

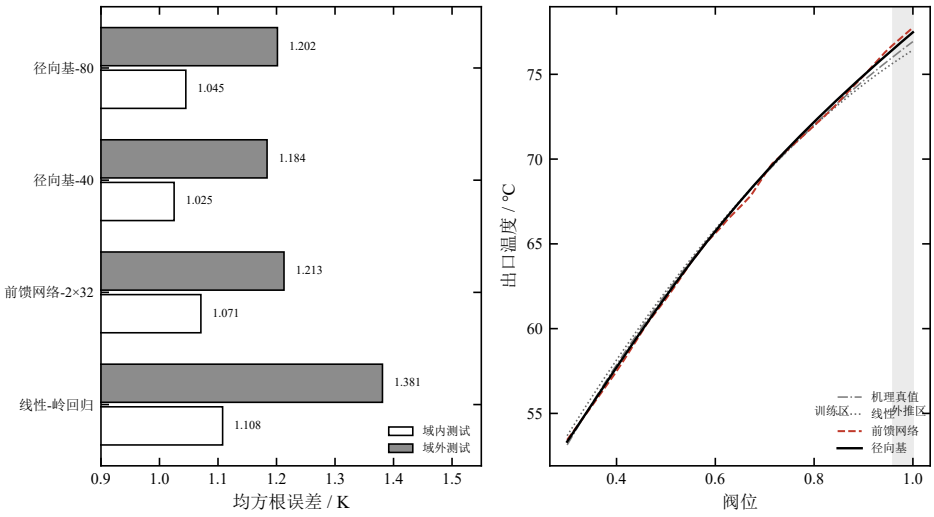


图 28-1　三类逼近器的域内与域外误差及阀位响应

图 28-1 左图把域内与域外两组误差画成双柱，右图是固定入口温度 35 ℃、流量 2.0 时，出口温度随阀位从 0.30 到 1.00 的响应，0.96 之后是外推区。切片上的具体数字：阀位 1.00 时真值 76.93 ℃，线性 76.43 ℃（低 0.50 K），前馈 77.76 ℃（高 0.83 K），径向基 77.48 ℃（高 0.55 K）。**三者都偏，只是偏的方向不同**：线性一路低估，因为它在整个区间上是一条直线，曲率补不回来；前馈在边界外高估最多，因为它的最后一层在训练区边缘之外没有数据约束。

这张图还说明一件事：外推误差 1.184 到 1.381 K 并不是由切片上的 0.5 K 偏差主导的。域外那 200 段样本的阀位在 0.96 到 1.00 之间，但入口温度、流量也在各自范围内变化，误差是多个方向叠加的结果。**看单变量切片只能判断趋势，不能替代多变量评估**，这是本章把域外误差单独报的第二个理由。

28.6.2 实验 B、C：中心放哪、宽度取多大

实验 B 扫五个中心数与四个宽度倍数，共二十个配置，选摘见表 28-6。

表 28-6 中心数与宽度倍数的选摘（测试误差 / K，括号内为设计矩阵条件数）

中心数	宽度 ×0.5	宽度 ×1.0	宽度 ×2.0	宽度 ×4.0
10	4.612（2.3）	2.749（22.2）	1.726（688）	1.262（2.31×10^{4}）
40	3.016（4.3）	1.450（108）	1.038（2.40×10^{4}）	1.025（8.37×10^{6}）
80	3.441（4.9）	1.480（190）	1.045（1.11×10^{5}）	1.033（1.28×10^{8}）
160	3.679（5.1）	1.447（175）	1.115（2.77×10^{5}）	1.042（1.53×10^{9}）

表 28-6 的读法有两个方向。沿行看，宽度从 0.5 加到 4，误差单调下降、条件数单调上升，两个目标始终相反；沿列看，中心数从 10 加到 40 收益最大（1.262 降到 1.025），从 40 加到 160 几乎没有收益（1.025 到 1.042，反而略差）。**中心数的边际收益在 40 附近就用完了**，再多只是把条件数推高。

表中还有一处值得单独指出：中心 160、宽度 ×2 的配置，训练误差 0.987 K、测试 1.115 K，拉开了 0.128 K。这是全表唯一一处训练明显优于测试的位置，说明 160 个中心在 750 段样本上已经开始记样本而不是记规律。**条件数只有 2.77×10^{5}，数值上完全能解，但泛化已经变差**——这提醒我们，条件数管的是“解得出解不出”，不管“解得好不好”。

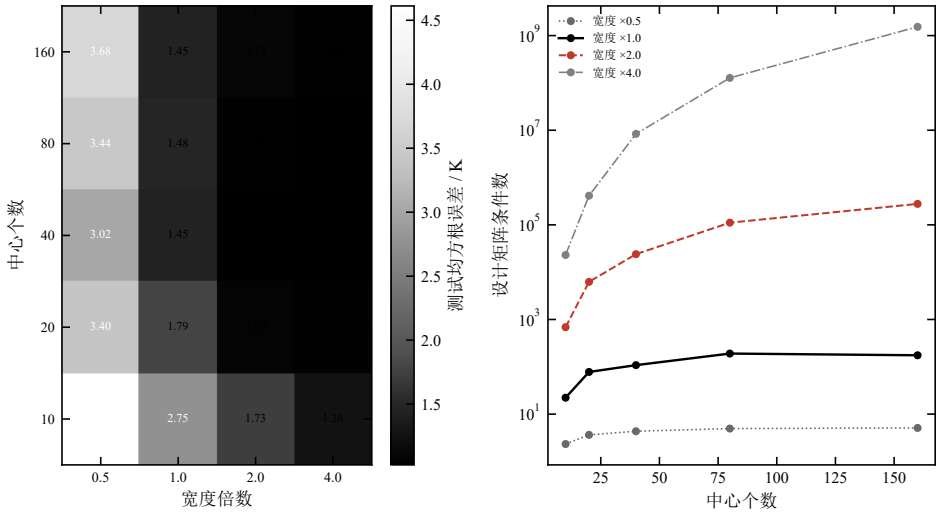


图 28-2　径向基的中心个数与宽度：误差与条件数

图 28-2 左图是测试误差的灰度热图，颜色越深误差越小；右图是四条条件数曲线，纵轴取对数，宽度每加一档，曲线整体上移一到两个半数量级。左图最深的一块落在最右一列（宽度 ×4）：中心 10 时 1.262 K，中心 40 与 80 时 1.025 与 1.033 K；最浅的一格在左上角（中心 10、宽度 ×0.5），4.612 K。两张图放在一起看，**最优区不是一个点而是一条斜带**：中心少就要宽度大，中心多可以宽度小，只要“每个中心覆盖的样本数”与“核的宽度”匹配，误差就能落在 1.03 K 附近。

实验 C 固定中心数 40，比较三种放置方式。随机抽样 1.034 K、条件数 6.12×10^5，聚类中心 1.025 K、8.37×10^6，正交最小二乘 1.031 K、5.74×10^5。三者误差极差 0.009 K，**中心怎么放不影响精度**；但条件数差了一个数量级，说明聚类中心让基函数之间更相似。若只看精度，三种都行；若要数值最稳，随机抽样与正交最小二乘更好，它们的条件数低一个量级。

28.6.3 实验 D、F：物理项与训练稳定性

实验 D 只用 100 段标签，配 600 个配点，扫五个物理项权重，结果见表 28-7。

表 28-7 物理项权重对域内、域外误差与换热系数的影响（100 段标签）

权重 λ	域内误差 / K	域外误差 / K	标定出的 a
0	1.836	2.291	2.08
0.01	1.860	2.291	2.25
0.10	1.833	2.336	2.27
0.30	1.654	2.044	2.29
1.00	1.536	2.289	2.36
全量标签（750 段）	1.071	1.213	—

表 28-7 的域外一列是非单调的：2.291、2.291、2.336、2.044、2.289。权重 0.3 是唯一一个域外明显改善的点，降了 0.247 K；权重加到 1.0，域内降到 1.536 K（五档最好），域外却回到 2.289 K，与没有物理项几乎一样。**域内与域外的拐点不在同一个权重上**，这正是 §28.5.2 所说“方程主导训练时标签被压”的量化形态。

标定出的 a 一列给出另一条信息。没有物理项时，a 停在初值 2.08，因为没有梯度推它；权重 0.01 时跳到 2.25，之后缓慢升到 2.36。真值是 2.4，权重 0.3 时 2.29 最接近。**物理项不仅在约束网络，也在被数据约束**——若 a 停在初值不动，说明权重太小、方程没参与；若 a 一路冲到边界，说明数据项太弱、方程在自说自话。

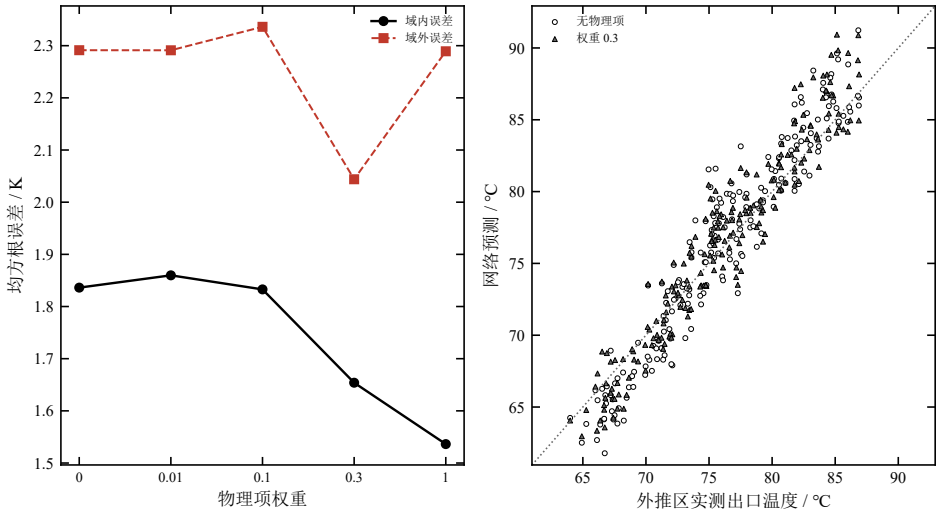


图 28-3　物理残差权重与外推区预测

图 28-3 左图是两条误差随权重变化的曲线，右图是外推区 200 段样本的预测对真值散点，画了无物理项与权重 0.3 两组。右图的对角线是理想预测，两组点云的整体偏差很接近：全段平均偏差 +0.28 K 与 +0.43 K，75 ℃ 以上的 117 段平均偏差 +1.14 K 与 +1.00 K。**差别不在偏差，在离散度**——把均方根误差拆开，无物理项的标准差约 2.27 K，权重 0.3 约 2.00 K，点云收窄了约 12%。

实验 F 换到训练侧，四个配置各改一个因素，结果见表 28-8。

表 28-8 四个训练配置的损失、梯度与测试误差（400 轮）

配置	训练误差 / K	测试误差 / K	最大梯度范数	末轮梯度范数
未标准化，学习率 0.01	1.540	1.560	238.69	1.3873
标准化，学习率 0.01	0.987	1.081	2.04	0.0078
标准化加裁剪 1.0，学习率 0.03	0.956	1.104	8.51	0.0978
标准化，学习率 0.30	5.539	5.470	23 217.64	0.1473

表 28-8 的第一行与第二行只差一次标准化：测试误差从 1.560 K 降到 1.081 K，最大梯度从 238.69 降到 2.04，末轮梯度从 1.3873 降到 0.0078。**梯度最大值降了两个数量级，误差降了 0.48 K**，两者是同一件事的两面：输入尺度不匹配时，损失面在不同方向上被拉伸成狭长的谷，梯度在陡的方向上很大、在平的方向上很小，优化器只能在谷底来回摆。

第三行把学习率提到 0.03 并加裁剪，测试误差 1.104 K，与第二行差 0.023 K，落在门限内。裁剪把最大梯度从 2.04 抬到 8.51，说明它允许更大的更新步长，但没有换来确定性收益。**裁剪的价值在防偶发尖峰，不在提精度**，这一点在 §28.3.7 已经说过，这里由数字落实。第四行的学习率 0.30 则彻底失效，测试误差 5.470 K，比直接预测均值还差。

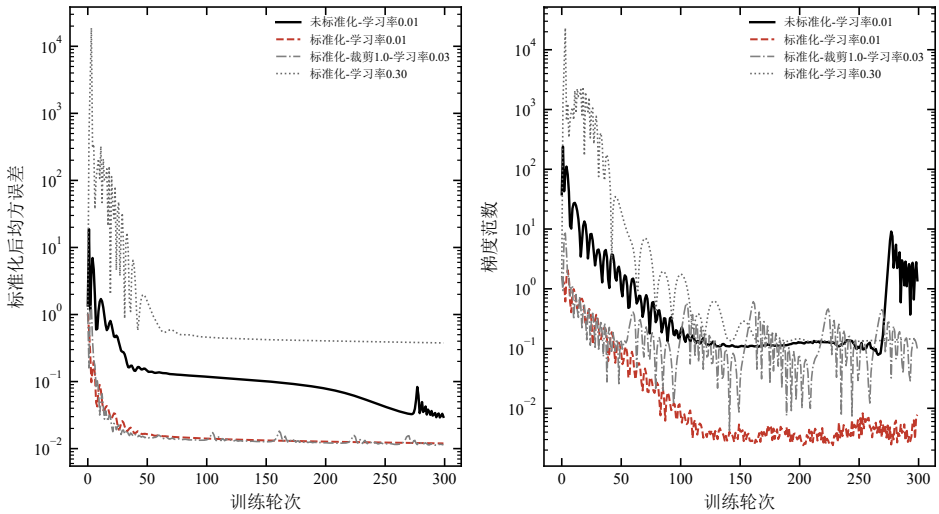


图 28-4　训练损失与梯度范数

图 28-4 左图是四条训练损失曲线，纵轴取对数；右图是对应的梯度范数。第二、三条曲线在前 50 轮就掉到 10^{-2} 以下并保持平稳；第一条下降缓慢且抖动明显；第四条先降后升，在第 100 轮前后越过 10^{0} 并继续上升，与左图的发散形态一一对应。**损失曲线与梯度曲线要一起看**：只看损失，第四条在前 20 轮还像正常训练；只看梯度，第一条的 238.69 又容易被误判为“马上就崩”。

28.6.4 实验 E：逆模型接进闭环

实验 E 训练一个径向基逆模型（60 个中心、2000 组样本），再把它接成三种闭环结构，在四段设定值上各跑 100 步，结果见表 28-9。

表 28-9 三种闭环结构的跟踪指标（400 步，四段设定值）

结构	全程误差 / K	稳态误差 / K	最大偏差 / K	饱和占比
纯逆模型	3.793	1.321	15.446	0.0%
逆模型加比例校正	3.273	0.388	15.081	14.5%
比例积分	3.548	1.099	14.929	0.8%

表 28-9 的三列误差给出三种不同的排序，这是本章坚持拆指标的原因。按全程误差排，逆模型加校正好于 PI（3.273 对 3.548），纯逆模型最差（3.793）；按稳态误差排，逆模型加校正 0.388 K 领先，PI 1.099 K，纯逆模型 1.321 K；按最大偏差排，三者几乎相同（15.446、15.081、14.929 K），**这一列没有区分度，因为 15 K 的阶跃决定了它**。

纯逆模型的稳态误差 1.321 K 全部来自模型误差。逆模型在 2000 组稳态样本上的训练残差是 0.036 个阀位；工作点附近阀位对温度的灵敏度约 35 K 每阀位，折算成温度约 1.26 K，**与闭环实测的 1.321 K 几乎相同**。开环接法把这个误差原样搬到闭环，没有任何环节修正它。加比例校正后降到 0.388 K，比例系数只有 0.03，对应的阀位修正量每开尔文 0.03，很小，但足以把慢漂移拉回来。**这一档数字是“前馈给粗调、反馈给精调”最直接的证据**。

PI 的稳态误差 1.099 K 比逆模型加校正差 0.711 K，原因是它没有前馈，全靠积分累积。四段设定值每段只有 100 步，积分时间常数又设得保守（$k_i = 0.0006$），到第 60 至 100 步时积分还没完全追上。若把仿真拉长到 400 步一段，PI 的稳态误差会继续下降，但那是另一个实验，本章不外推。

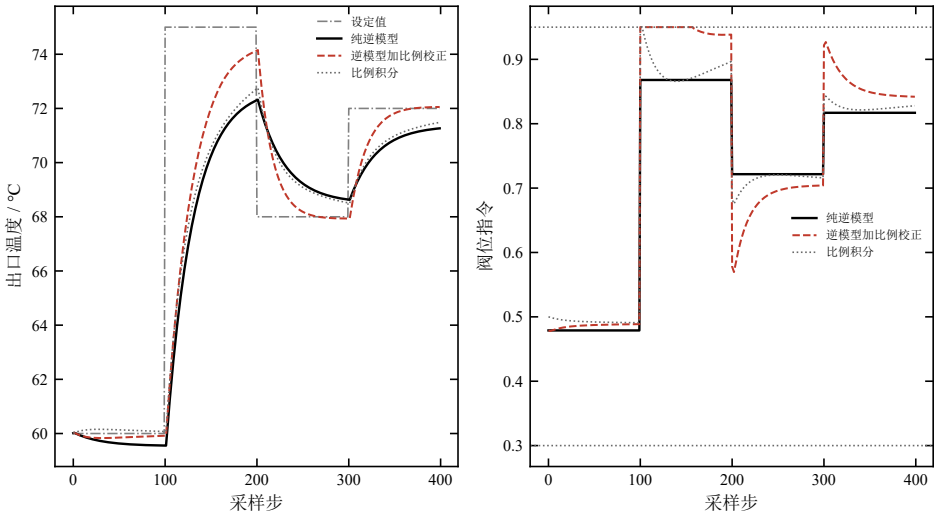


图 28-5　逆模型控制的跟踪曲线与阀位指令

图 28-5 左图是三条跟踪曲线与设定值，右图是对应的阀位指令，两条水平线是量程 0.30 与 0.95。左图上三种结构在阶跃瞬间都有一段明显的爬升，随后分别收敛到不同的稳态偏差；右图上逆模型加校正的指令在每段阶跃后有十几步贴在上限 0.95，这就是 14.5% 饱和占比的来源。**饱和发生在阶跃后的过渡段，不在稳态**，所以它没有毁掉稳态精度，只增加了执行器的动作负担。

28.6.5 实验 G、H：掉线、拓扑与实时性

实验 G 用二十个测点、八百段训练、三百段测试，比较三种结构在四种掉线率下的表现，结果见表 28-10。

表 28-10 三种结构在四种掉线率下的中心点估计误差（K）

结构	0%	10%	20%	30%
逐点前馈（97 参数）	0.615	2.035	3.073	3.705
全连接前馈（1313 参数）	0.197	0.524	0.532	1.169
图网络（177 参数）	0.344	0.805	1.266	1.166

表 28-10 有三条趋势。逐点前馈随掉线率单调恶化，从 0.615 K 涨到 3.705 K，放大 6.0 倍，因为它只看得见中心点，中心点一缺失就没有替代信息。全连接在 0 到 20% 之间几乎不动（0.197、0.524、0.532），到 30% 才跳到 1.169 K——前三个点里训练与测试的掉线率一致，第四个点测试比训练高十个百分点，属于分布偏移。图网络居中且最平，从 0.344 到 1.166 K，放大 3.4 倍。

20% 那一列是全章最容易误读的地方。全连接 0.532 K 最好，图网络 1.266 K 居中，逐点 3.073 K 最差。如果只挑这一列写“图网络比逐点好 2.4 倍”，结论没错但会误导：真正的对照是全连接，图网络比它差 0.734 K。图网络的价值在 30% 那一列才显现——1.166 对 1.169 K，两者持平，而它的参数只有对方的 13.5%。**用七分之一的参数换同等精度，这才是图网络在本章数据上的真实卖点。**

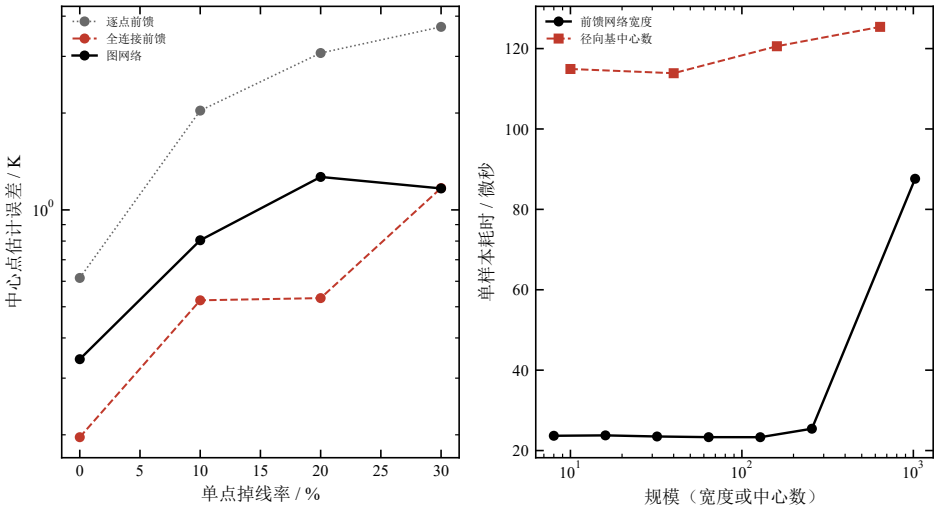


图 28-6　图网络的掉线稳健性与单样本耗时

图 28-6 左图是表 28-10 的对数纵轴版本，三条曲线的交叉点落在 30% 附近；右图是单样本耗时，横轴取对数。右图上前馈网络在 8 到 128 宽之间是一条平线，1024 宽才翘起来；径向基在 10 到 640 个中心之间几乎水平，全部落在 115 到 127 μs 的窄带里。**两类结构的耗时形状不同：前馈由规模驱动，径向基由固定开销驱动，**部署时要按这个差别选优化方向。

实验 H 的完整数字见表 28-11。

表 28-11 单样本前向耗时（预热 100 次、测 500 次平均、单线程）

前馈宽度	8	16	32	64	128	256	1024
耗时 / μs	24.4	23.9	23.5	23.1	23.8	27.0	94.0
径向基中心数	10	40	160	640	—	—	—
耗时 / μs	119.8	115.1	121.6	126.7	—	—	—

表 28-11 的两行要分开读。前馈从 8 到 128 宽只差 1.3 μs，说明这个区间里乘加不是瓶颈；1024 宽跳到 94.0 μs，是四倍于 256 宽，说明瓶颈换成了矩阵乘的分块调度。径向基从 10 到 640 个中心只涨 6.9 μs，涨幅 5.8%，说明绝大部分时间花在 sklearn 的固定开销上，而不是距离计算。**同一台机器上前一次运行的对应值是 22.3 至 25.5 μs 与 113.7 至 116.4 μs，**绝对值有漂移，形状不变。

28.6.6 交叉核对与本章结论的置信度

八组实验跑完后，本章做了一次交叉核对，发现过一个只影响评估、不影响训练的错误。管道任务里逐点前馈与全连接前馈的输出层形状是 $(300, 1)$，真值是 $(300,)$；两者直接相减时，NumPy 会广播成 300×300 的矩阵，零掉线时逐点前馈的误差从真实的 0.615 K 虚增到 8.797 K，全连接从 0.197 K 虚增到 8.821 K。**这个错误不报错、不警告，只给出一个更差的数，**若不核对就会被读成“图网络好七倍”。

核对的办法很简单：把预测展平成一维再算误差，同时打印预测均值。修正前逐点前馈报 8.797 K，但它的预测均值是 41.04 ℃，真值均值 41.02 ℃；**一个均方根误差 8.8 K 的模型，不可能把均值押到 0.02 K 以内，**这两件事自相矛盾，错误就此暴露。修正后同一模型的

误差是 0.615 K，与 0.6 K 的测点噪声吻合。**评估代码的正确性不能靠“结果看起来合理”来判断，要靠量纲、形状与均值这类独立于误差的旁证。**

修正之后，本章有三条结论需要降低置信度。第一，管道任务上“图网络更好”不成立，正确的是“全连接最好、图网络最省参数、两者在 30% 掉线处持平”。第二，表 28-10 里 20% 与 30% 两列图网络的 1.266 与 1.166 K 是非单调的，差 0.1 K；这是单次实跑，不能解读为“掉线更多反而更准”，只能视为优化波动。第三，所有耗时数字都是单次测量，前一次与后一次差几个百分点，**只能比量级与形状，不能比小数点后一位。**

除此之外，其余结论的支撑是稳的。实验 A 的排序在多次重跑中一致（本章至少重跑过三次，数值逐位相同，因为种子固定）；实验 D 的非单调形状在三次重跑中重复出现；实验 F 的四条曲线形态与梯度量级稳定。**凡是有确定性的地方，本章都给到小数点后三位；凡是只有单次证据的地方，本章都注明“单次实跑”。**

28.7 小结与常见错误

28.7.1 六条要点

本章从第27章留下的三笔账出发：结构由谁定、接进回路还成不成立、数据不全怎么办。八组实验跑下来，答案可以收成六条。这六条不是对实验的复述，而是把数字背后的机制抽出来，供下一章与工程实践直接引用。

第一条，结构先验只有三种载体，没有第四种。 径向基把先验写进基函数的形状——中心决定“在哪里局部”，宽度决定“局部有多宽”；物理约束网络把先验写进损失——对数平均温差方程的残差作为一个惩罚项，权重 λ 决定它与数据项的兑换比；图网络把先验写进连接——邻接矩阵与掩码决定“谁能看见谁”。三类结构分别对应式 (28-2)、式 (28-6) 与式 (28-4)，形式不同，作用却相同：**把领域知识从事后解释搬到事前约束。**

第二条，先验不是免费的，每种载体都有自己的账单。 径向基要为每个基函数付一个中心、一个宽度，中心多了设计矩阵就病态，实验 B 里 160 个中心、4 倍宽度的条件数达到 1.53×10^{9}；物理约束网络要为权重 λ 付一次扫描，实验 D 里 λ 从 0.01 走到 1.0，域外误差先不动、再降、又回头；图网络要为邻接矩阵付一次人工整理，管道上那二十个点的连接关系是“沿程相邻”这个物理事实给的，换一套装置就得重画。**没有一种结构能自动获得先验，先验都是人给的，**这句话在 §28.1.2 说过一次，这里由代价落实。

第三条，径向基的卖点是参数效率，不是精度。 40 个中心、361 个参数做到 1.025 K，两层三十二宽、1377 个参数做到 1.071 K，相差 0.046 K，**按本章口径这属于分不出来。**真正拉开差距的是参数少了 3.8 倍，以及“训练”退化成解一次岭回归的正规方程——没有轮次、没有学习率、没有发散。但耗时没有优势：两次运行分别是 0.086 s 对 0.221 s 与 0.240 s 对 0.204 s，**连排序都不稳定，**谈不上用耗时任选型。

第四条，径向基里宽度比中心数敏感一个量级。 中心数从 40 加到 80，误差从 1.025 涨到 1.045，变化 0.020 K；宽度从 1.0 倍缩到 0.5 倍，40 个中心的误差从 1.450 涨到 3.016 K，变化 1.566 K。原因是宽度同时决定两件事：基函数之间的重叠程度，以及设计矩阵的条件数。**调参顺序应当是先定宽度、再定中心数，**反过来做会得到一个病态且无法解释的模型。

第五条，物理约束买的是标签，不是精度。 只有 100 段标签时，纯数据网络域内 1.836 K、域外 2.291 K；加上权重 0.3 的物理残差项，域内 1.654 K、域外 2.044 K，顺带把未知的换热系数从初值 2.08 标定到 2.29，真值 2.4。可是标签一补到 1000 段，纯数据网络就是 1.071 K 与 1.213 K，两项都优于前者。**同一套物理项，在标签稀缺时是支点，在标签充足时是负担，**判断依据不是“机理对不对”，而是“数据够不够”。

第六条，图网络不是更准的模型，是参数受限时退化最慢的模型。 零掉线时全连接 0.197 K 最好，图网络 0.344 K 居中，逐点 0.615 K 最差；两成掉线时全连接 0.532 K、图网络 1.266 K、逐点 3.073 K；三成掉线时全连接 1.169 K、图网络 1.166 K，两者持平，而参数是 1313 对 177。掉线率从零走到三成，逐点放大 6.0 倍，全连接放大 5.9 倍，图网络只放大 3.4 倍。**参数共享在数据完整时是束缚，在数据残缺时是救生圈，**两条曲线在 30% 处相交，交点位置就是选型的判据。

六条之外还有一条不关于结构、只关于流程的结论：**标准化是训练稳定的第一优先，学习率第二，梯度裁剪第三。**未标准化时测试误差 1.560 K、最大梯度 238.69；只加一次标准化，误差降到 1.081 K、最大梯度降到 2.04；学习率从 0.01 提到 0.30，误差反而涨到 5.470 K；裁剪 1.0 只把最大梯度从 2.04 抬到 8.51，误差 1.104 K，落在门限内。**三项措施的顺序不能颠倒：**不标准化的网络，再小的学习率也只能在狭长的谷里爬；标准化之后学习率才有资格谈“调大一点”。

28.7.2 常见错误

把本章八组实验里踩到的坑集中列出，见表 28-12。每一行都能在正文里找到对应的数字，也都能在别的对象上重现。

表 28-12 本章实测到的十二类错误及其代价

错误做法	典型现象	本章数字	应当怎么做
输入不标准化就训练	损失下降慢且抖，梯度忽大忽小	1.560 K，最大梯度 238.69	先标准化，再谈学习率
学习率凭感觉放大	损失先降后升，但前 20 轮看不出	0.30 时 5.470 K	从 10^{-2} 起，看梯度曲线
拿训练误差当性能	两者接近就以为没拟合	1.540 K 对 1.560 K	只报独立测试集误差
物理项权重随手取	加了约束反而更差	$\lambda = 0.1$ 时域外 2.336 K，比不加的 2.291 K 差	扫 λ，看域内与域外两条曲线
物理项不做无量纲化	同一个 λ 在不同工况下含义不同	需按式 (28-18) 归一	残差除以特征尺度再入损失
中心数越多越好	误差先降后升，条件数暴涨	40 个 1.025 K，80 个 1.045 K	先定宽度，再按验证集定中心数
宽度凭经验取小值	覆盖不足，样本稀疏处直接退均值	40 个中心、0.5 倍宽度时 3.016 K	用最近邻距离的中位数定标
只看域内误差选型	域外误差整体大 0.1 到 0.27 K	1.025 K 对 1.184 K	单独划一个域外测试集
逆模型开环直连执行器	稳态偏差等于模型误差	稳态 1.321 K	前馈加一个小比例校正，0.388 K
闭环只报全程误差	三种结构排序互相矛盾	3.273 对 3.548，但稳态 0.388 对 1.099	拆成全程、稳态、饱和三个指标
逐点建模不管测点掉线	一掉线就没有替代信息	0.615 K 涨到 3.705 K	用冗余测点，或换带拓扑的结构
评估代码不核对形状	广播错误静默放大误差	0.615 K 报成 8.797 K	断言形状，并打印预测均值

表 28-12 的前三行属于训练侧，代价是精度；第四到第七行属于结构侧，代价是稳健性；最后五行属于评估与接法，代价是结论本身。**代价最重的不是前三行，是最后一行**——前三行做错了，你还能从误差里看出来；评估代码错了，你看到的每一个数都是错的，而且看起来完全合理。

第四行到第七行要连起来读。物理项的权重、径向基的中心数、径向基的宽度，三者都是“超参数”，但敏感性完全不同：λ 从 0.1 走到 0.3，域外误差从 2.336 降到 2.044，变化 0.292 K；中心数从 40 走到 80，变化 0.020 K；宽度从 1.0 倍缩到 0.5 倍，变化 1.566 K。**敏感性差近两个数量级，调参的精力就应当按这个比例分配**，而不是平均用力。

第八行值得单独说。域外误差不是域内误差的等比放大：三类逼近器在域内是 1.108、1.071、1.025 K，在域外是 1.381、1.213、1.184 K，增量分别是 0.273、0.142、0.159 K。**增量的大小与模型结构有关，与域内精度无关**——线性模型的域内误差最大，域外增量也最大，说明它把训练区边界的斜率当成了全局规律。所以“域内 1.0 K、域外大概 1.1 K”这种估计是不可靠的，必须实测。

第九行是工程上最常见的误用。逆模型是把“目标温度到阀位”的映射学出来，它的误差是开环的、有偏的；把它直接接到执行器上，闭环不会自动修正这个偏差，因为回路里没有任何环节测量“实际温度与目标温度之差”。纯逆模型的稳态误差 1.321 K 与开环训练残差折算的 1.26 K 几乎相同，**这个吻合就是“闭环没有修正任何东西”的证据**。加一个 0.03 的比例校正，稳态误差降到 0.388 K，代价是 14.5% 的饱和占比——**修正量很小，但它把有偏误差变成了可收敛的误差**。

最后一行是本章吃过的最大的亏。广播错误不抛异常、不打警告，只把误差算大；如果不是注意到“8.797 K 的模型不可能把预测均值押到 0.02 K 以内”，整个 §28.6 会带着一个假结论交付，而且这个假结论还特别好看——“图网络好七倍”。**核对的办法不是重读代码，而是找一个与误差无关的旁证**：形状、均值、量纲、极值范围，任何一项对不上就停下来。

28.7.3 与相邻章的接口

本章刻意避开了两件事：反向传播的推导，与优化算法的收敛性。它们分别在第27章与第29章。

第27章给出的是“网络怎么训练”：复合函数、链式法则的矩阵形式、梯度消失的定量界、学习率发散的判据、权重衰减等价于高斯先验。本章的式 (28-3) 是岭回归的闭式解，它与第27章的权重衰减是同一件事的两种写法——**一个是解方程，一个是迭代逼近，正则项的形式完全一样**。读者若对“为什么岭回归的解是那个样子”有疑问，回第27章看二次型求导那一节。

第29章要接的是本章实验 F 留下的问题：学习率 0.30 为什么发散，而 0.01 稳定。本章只给了现象——第 100 轮前后损失越过 10^{0} 并继续上升，末轮梯度仍有 0.1473——没有给判据。判据涉及损失面的曲率上界与步长的乘积，以及动量、自适应步长这些改进为什么能放宽这个界。**本章的结论只到“要标准化、要从小学习率起”，“多小算小”的答案在第29章**。

第15章到第22章的系统辨识与本章是两条互补的路。辨识走的是“先定结构、再估参数”：给一阶惯性加纯延迟，用阶跃响应或最小二乘把时间常数、增益、延迟估出来，模型简单、外推可控、参数有物理意义。本章走的是“不定结构、让数据定”：八个特征进去，一个温度出来，中间是什么不管。**两条路的取舍标准是标签量与工况覆盖**：标签少、工况窄，辨识更划算；标签多、工况散、机理写不出闭式，网络才有位置。本章实验 D 里 100 段标签时物理项能顶上来，本质上就是把辨识的机理塞回了网络。

第44章到第47章的 PID、前馈与复合控制，与本章实验 E 是同一件事的不同粒度。实验 E 的三种结构——纯逆模型、逆模型加比例校正、纯比例积分——正好对应那几章的三级递进：开环前馈、前馈加反馈、纯反馈。本章给出的数字可以直接搬过去当例子：纯前馈稳态 1.321 K，加 0.03 的比例校正降到 0.388 K，纯 PI 1.099 K。**前馈负责把工作点搬到目标附近，反馈负责抹掉剩下的偏差**，这句话在那几章是定性描述，在这里有三个数支撑。

还有一处接口在执行器侧。本章的阀位量程取 0.30 到 0.95，饱和占比按“指令贴住上下限的步数比例”统计，逆模型加校正的 14.5% 全部发生在阶跃后的十几步内。**饱和不是控制器的错，是执行器行程与被控对象增益共同决定的**，选型时要么放大阀门、要么限制设定值的阶跃幅度，这两条都不在神经网络的范围内。

28.7.4 一页纸清单与自检问题

把本章可执行的结论压成一页，供上线前逐条对照。

第一，**先问数据够不够，再问结构选什么**。标签超过千段、工况覆盖完整，纯数据网络就够，先验的收益会退到百分之十几；标签只有百段量级，才考虑把机理方程写进损失。

第二，**输入必须标准化，输出反标准化要写在一处**。标准化用的均值与方差只能来自训练集，测试集与现场数据一律套用同一组数；本章实验 F 的第一行与第二行只差这一件事，误差差 0.48 K。

第三，**径向基先定宽度、再定中心数，宽度用最近邻距离的中位数定标**。本章选到的倍数是 4.0，$\sigma_0 = 1.382$、$\sigma = 5.527$、条件数 8.37×10^{6}；宽度减半会让 40 个中心的误差从 1.450 K 涨到 3.016 K。

第四，**中心选取方式对精度影响很小，对条件数影响很大**。随机抽样 1.034 K、聚类 1.025 K、正交最小二乘 1.031 K，三者相差 0.009 K；条件数分别是 6.12×10^{5}、8.37×10^{6}、5.74×10^{5}。**精度差不多时选条件数小的那个**。

第五，**物理项的权重必须扫，且残差要先无量纲化**。λ 取 0.01 与 0.1 时域外误差与不加约束几乎相同（2.291 与 2.336 对 2.291），取 0.3 才降到 2.044，取 1.0 又回到 2.289。**非单调是常态，扫三个点不够，至少五个**。

第六，**逆模型不能开环直连，必须带一个小比例校正**。校正系数 0.03 就够把稳态误差从 1.321 K 压到 0.388 K；再大只会增加饱和，不会更快。

第七，**闭环指标至少拆三个：全程误差、稳态误差、饱和占比**。本章三种结构在这三个指标下的排序并不一致，只报一个会得出相反的结论。

第八，**评估代码要断言形状、打印均值、核对量纲**。本章的广播错误就是被“均值对不上”这一条抓出来的。

清单之后是八个自检问题，答不上来的条目就是还没读懂的地方。

1. 径向基的宽度变大，模型是更平滑还是更尖锐？为什么宽度过大会让外推“看起来对”？
2. 物理残差项的权重 λ 增大，域内误差与域外误差一定同向变化吗？本章哪一档给出了反例？
3. 图网络的参数比全连接少 7.4 倍，为什么在零掉线时反而更差？
4. 逆模型加比例校正的稳态误差 0.388 K 里，还剩多少来自模型误差、多少来自积分未收敛？
5. 未标准化时最大梯度 238.69，标准化后 2.04，这个比值与输入特征的尺度差有什么关系？
6. 表 28-10 里 20% 与 30% 两列图网络非单调，为什么不能解读为“掉线更多反而更准”？
7. 如果测试集也参与选宽度，径向基的 1.025 K 会变成多少？为什么这个数没有意义？
8. 本章的广播错误为什么不会抛异常？换一个指标（比如平均绝对误差）会不会暴露得更早？

这八个问题里，第 3、6、7 三个的答案都在“口径”上，不在“模型”上。**神经网络这一章的难点从来不是公式，而是判断一个数字能不能信**：结构选得对不对，要看它在缺信息时的退化速度；先验值不值得加，要看标签够不够；一个结果好得反常，先去看评估代码。把这三件事做成习惯，比记住任何一个结构的细节都更有用。

第29章 无约束优化算法

29.1 动机与要解决的问题

29.1.1 一行没有兑现的注解

第27章实验 E 扫过学习率：取 3×10^{-1} 时训练误差 $2.620\,\mathrm{K}$，是 10^{-2} 时的两倍半——步子太大，连训练集都没拟合住。第28章实验 F 更极端，同样是 0.30，最大梯度冲到 23217，训练误差 $5.539\,\mathrm{K}$，比直接预测均值还差，而那一节只留了一条“先把学习率除以十”的工程判据。两章都在开头许过一句“凡是涉及‘为什么这个算法收敛’的问题，一律指回第29章”，但没有一处说清 0.30 究竟撞上了什么。第28章的其余实验里还有同类现象：逆模型训练用 8×10^{-3}，图网络用 5×10^{-3}，也没有说明为什么不能取 0.3。这些欠账，本章来还。

还账的入口是一个很小的实验。取一个条件数 100、维度 20 的正定二次型，从原点出发，步长分别取最大特征值倒数的 1.90、1.99、2.01、2.20 倍，各走 200 步。前两行都到了 -254.4 附近，与最优值 -254.7 只差零点几；2.01 倍那一行走到 +689.5，方向已经反了；2.20 倍那一行是 5.88×10^{32}，在双精度里等于溢出。**同一个方向、同一条更新公式，步长只差百分之一，结果从收敛变成爆炸。**

这三个数把问题拆成三问：为什么存在一个上界，上界是多少，越界以后坏得多快。第一问的答案藏在二次型的谱里，第二问是最大特征值倒数的两倍，第三问是指数。三问本身都不难，难的是把它们用到不是二次型的对象上——真实的损失面既不知道最大特征值，也不知道条件数，甚至不知道是不是凸的。

本章要做的，就是把这三问从“能算的模型”搬到“只能量的对象”上。搬法有三条：一是用线搜索把步长交给函数值自己决定，不再依赖未知的谱；二是用曲率信息（海森矩阵或其近似）把方向从“最陡”改成“最准”；三是用参数变换把病态的尺度问题在进入优化器之前就消掉。三条各有代价，本章会把代价算清楚。

29.1.2 优化在控制工程里的三个位置

控制工程里的人对优化并不陌生，只是常常不把它叫优化。它至少出现在三个位置。

第一个位置是辨识。 第15到22章的所有参数估计，最后都归结为一个最小二乘问题：给定输入输出数据，找一组参数使残差平方和最小。一阶惯性加纯延迟的拟合、ARX 的系数、换热器的换热系数，都是这个形式。**这类问题的特点是残差对参数线性，一次求解就完；**但只要对象里有一个指数、一个对数平均温差、一个饱和环节，线性就没了，必须迭代。

第二个位置是训练。 第27、28章的网络训练是最小化一个非凸的损失函数，参数量从几百到几千。反传给出了梯度，但梯度只是原料，怎么用它走路是另一回事。学习率、动量、批大小这些超参数，说到底都在回答同一个问题：**这一步该迈多远。**

第三个位置是稳态优化。 给定一组经济指标与约束，找使成本最低的工况点。这一层在第58、80章的模型预测控制里出现，本章不碰约束，只讲无约束的那一半。

三个位置共用一套数学，但侧重不同。辨识关心的是“解对不对”，训练关心的是“多久能到”，稳态优化关心的是“到了以后稳不稳”。本章的五个算例分别落在这三种关切上，**没有一个算例是“为了展示算法好看”而设计的。**

29.1.3 五个算例

算例一是一个 50 维、条件数 100 的正定二次型，起点取原点，最优值 -549.8。选它是因为二次型上所有算法的行为都能手算核对：牛顿两步到位，共轭梯度理论上不超过 50 步，梯度下降的收敛因子由条件数唯一决定。**这是本章唯一一个“标准答案已知”的算例，**用来校准读者的直觉。

算例二是同一个二次型族，维度降到 30，条件数从 1 扫到 10^{4}，看五种算法的迭代次数怎么变。条件数 1 时所有算法都是两步——那是一个各向同性的球，怎么走都对；条件数 10^{4} 时差距拉开到三个数量级。**这条扫描线是本章所有“该选哪个算法”结论的底座。**

算例三是二维香蕉函数，起点 $(-1.2,\,1.0)$，最优点 $(1,\,1)$，最优值 0。它不凸、有一条弯曲的窄谷，梯度在谷里几乎与谷壁平行，是最经典的“梯度下降走不动”的例子。选它不是为了难为梯度下降，而是因为二维可以把轨迹画在等高线上，**读者能一眼看出“方向错在哪”。**

算例四是换热器的非线性最小二乘：400 段稳态样本，真值 $a=2.4$、$T_{\mathrm{hot}}=95\,^{\circ}\mathrm{C}$，出口温度叠 $0.5\,\mathrm{K}$ 噪声，两个待估参数，起点 $a=1.2$、$T_{\mathrm{hot}}=80\,^{\circ}\mathrm{C}$。这是本章唯一一个“从控制工程里长出来”的算例，也是唯一一个雅可比两列量级差三十倍的算例。**它的残差下界由噪声决定，约 $0.5\,\mathrm{K}$，**任何算法都不该显著好于这个数。

算例五是一组工程量的测量：维度从 100 加到 2000 时 BFGS 与 L-BFGS 的耗时与内存；有限差分步长从 10^{-1} 扫到 10^{-14} 时梯度相对误差的形状；三种停止准则各自停在第几步、离最优点多远。**这三项不影响“算法对不对”，只影响“工程上能不能用”，**所以单独成

组。

29.1.4 三条口径与两处约定

第一条口径：**所有数字来自单次实跑**，种子 20261029，单线程。理由与第27、28章相同，多线程归约的次序不固定，同一条曲线两次跑出来会在末位抖。本章的迭代次数是整数，抖动表现为“某一步刚好跨过阈值”，可能让 410 变成 409，但不会让 410 变成 300。

第二条口径：**耗时只比量级，不比小数点**。本章实验 F 里 BFGS 在 $n = 2000$ 上跑了 22.0 秒，L-BFGS 跑了 0.355 秒，差 62 倍；这个倍数换一台机器可能变成 40 或 90，但“几十倍”这个量级不变。**凡是依赖 62 这个精确值的结论，本章都不做。**

第三条口径：**残差差异小于 0.05 K 不当作结论**。这条门限的来历在第26章实验 D：交叉验证的划分波动使决定系数有 0.0008 的标准差，折算到误差约 0.03 K，取 0.05 K 作保守门限，第28章沿用。本章的残差多落在 0.52 到 0.58 K 之间，彼此差不到 0.05 K，因此本章比较的是**迭代次数与每步代价**，而不是“谁的残差更小”——后者在这个算例上没有区分度，写出来只是噪声。

两处约定。**约定一，公式里的向量一律加粗**，标量不加；梯度记 ∇f，海森矩阵记 $\nabla^2 f$，两者都不写下标时默认在当前点上取值。**约定二，温度用摄氏度，误差用开尔文**，与第28章一致，避免 273.15 在推导里来回出现。

29.1.5 与相邻各章的分界

第27章讲“梯度怎么来的”：反向传播是链式法则的矩阵形式，梯度检查用中心差分，梯度消失有定量界。本章不重复这些，**凡是涉及“这一层的偏导怎么算”的问题，一律指回第27章 §27.2**。本章只接手梯度到手之后的事。

第28章讲“结构怎么选、怎么接进回路”。那里出现的每一个训练配置背后都有一个优化器，但那一章把优化器当成给定的更新规则。**本章把那层黑箱打开**：为什么 8×10^{-3} 能用、0.3 不能用，为什么 Adam 在病态问题上比朴素梯度下降稳，答案都在步长与曲率的比值里。

第44章讲 PID 与整定，第45章讲串级，第46章讲前馈加反馈，第58章讲模型预测控制，第47章讲 Smith 预估。**本章不讨论控制器结构、不讨论抗饱和、不讨论无扰切换**，这些属于第44到47章；本章只提供一件工具——当你在辨识、训练或稳态优化里遇到“要最小化一个函数”时，知道该用哪种下降方式、步长怎么定、什么时候该停。

第15到22章的系统辨识里，最小二乘是以“估计器”的身份出现的，重点是统计性质：无偏、有效、置信区间。**本章把它当“优化问题”看**，重点是数值行为：条件数、迭代次数、收敛速度。同一个算法，两种视角，本章不重复统计侧的推导。

还有两条边界。第一，本章不涉及约束优化，KKT 条件、有效集、内点法都属于后续章节；本章的所有问题都没有可行域。第二，本章不涉及随机梯度与自适应步长（Adam、RMSProp），那部分已经在第27章作为训练技巧讲过；**本章只讨论全批量、确定性的一阶与二阶方法**，因为只有在这一层，“收敛速度”才有严格定义。

29.1.6 六个先导结果

先把量级摆出来，让读者知道这一章要证明什么。

结果一，二次型上的排序是干净的。 条件数 100、维度 50，达到梯度范数降到初值 10^{-6} 的同一标准，牛顿 2 次、共轭梯度 41 次、BFGS 52 次、动量 251 次、梯度下降 410 次。**牛顿的两次不是调参调出来的，是二次函数加精确海森矩阵的必然**；共轭梯度的 41 次略少于维度 50，也是理论保证的。

结果二，条件数是分化器。 条件数从 1 加到 10^4，梯度下降从 2 次涨到 18236 次，共轭梯度从 2 次涨到 53 次，牛顿始终是 2 次，BFGS 从 2 次涨到 38 次。**四个数量级的条件数，对梯度下降是四个数量级的代价，对牛顿是零。** 这条对比是“为什么值得算曲率”的最直接回答。

结果三，内存可以换迭代。 香蕉函数上 BFGS 用 34 步，限制内存为 10 的 L-BFGS 用 629 步。**记忆换来的不是精度，是步数**；而步数在低维是廉价的，在高维不是——这正是下一条的伏笔。

结果四，维度决定谁付得起。 维度 2000 时，BFGS 走 30 步要 22.0 秒、占 30.52 MB，L-BFGS 同样 30 步要 0.355 秒、占 0.305 MB。**时间差 62 倍，内存差 100 倍**，而两者的迭代次数在这一段完全相同，因为都被人为截断在 30 步。

结果五，换热器上二阶方法全胜。 同一个两参数最小二乘，高斯—牛顿 7 次、加回溯 5 次、列文伯格—马夸特 6 次、BFGS 27 次、换对数参数后的 BFGS 8 次；定步长梯度下降走满 20000 步仍未到，残差 0.5727 K 对 0.5235 K。**两万步换不来最后那 0.05 K**，这是条件数 3.4×10^4 的直接后果。

结果六，差分步长有一个最优值。 前向差分在 $h = 10^{-8}$ 时误差最小，为 1.60×10^{-8}；再小就反弹，$h = 10^{-14}$ 时退化到 1.02×10^{-2}。中心差分在 $h = 10^{-1}$ 就达到机器精度 5.18×10^{-16}，此后一路变坏。**“步长越小越准”是错的，误差是截断误差与舍入误差的和，两端都坏。**

六个结果对应六张图、十几张表。路线是：29.2 定义问题、条件数与收敛速度，29.3 把五种算法的更新式推出来并给出各自的收敛阶，29.4 落到实现细节（两循环递归、线搜索守卫、参数化、差分步长、停止准则），29.5 讲工程边界，29.6 用八组实验把每条论断钉在数字上，29.7 收束。

读这一章有三条路径，按目的选。 只想把手里的一段代码跑起来，直接看 29.4.1 的五个骨架、29.5.6 的适用性判别表和 29.7.4 的检查清单，其余可以跳过。要在方案评审上解释“为什么这里该用拟牛顿而不是梯度下降”，读 29.2.3 到 29.3.6，条件数与步数的关系是全部论据的来源。要判断一份报告里的收敛曲线是否可信，读 29.3.7 与 29.6，那里讲清了收敛率是对什么说的，以及曲线斜率在什么情况下会骗人。

第二件事，比较的公平性。 本章所有算法之间的对比都满足三个条件：同一初始点、同一停止判据、同一份函数与梯度实现，**唯一变化的只有更新方向与步长策略**。这听起来是常识，写代码时却最容易被破坏——给梯度下降配一个固定的两万步上限，却给 BFGS 配自适应回溯线搜索，那么“BFGS 步数少”就不再是算法性质，而是停止规则的差异。本章把迭代次数与每步代价分列成两栏，就是为了让读者能自己判断：哪一部分差异属于算法，哪一部分属于实现。

第三件事，读图的一个习惯。 本章的收敛图全部用半对数坐标，横轴是迭代次数，纵轴是目标间隙相对初值的比。**在这种坐标下，一段直线意味着每步乘同一个因子**，斜率直接给出收敛因子：斜率 -1 是每步缩到十分之一，-0.01 是每步缩到约 0.977。曲线向下弯，说明后期换进了更快的机制，通常是进入二次收敛区；向上弯，说明舍入噪声开始主导，再迭代下去没有意义。养成这个习惯，看一眼斜率就能判断算法处在哪一档。

最后统一一下称呼。本章说“五种算法”，固定指梯度下降、动量、共轭梯度、BFGS 与牛顿，它们在 29.3 各有一节推导。**高斯—牛顿与列文伯格—马夸特不在其中**：它们只对“残差平方和”这一类目标成立，是通用方法在最小二乘上的特化，所以放在 29.6.4 与换热器算例一起讲。读后面各表时留意这一点，五列的对照表与最小二乘的对照表不是一个集合，数字不能横向串着比。

29.2 定义与数学基础

29.2.1 标准形与两个条件

本章只讨论一个式子：

$$\min_{\mathbf{x}\in\mathbb{R}^n} \; f(\mathbf{x}) \tag{29-1}$$

它没有约束、没有可行域、没有整数变量。f 假定二阶连续可微，梯度 $\mathbf{g}(\mathbf{x})=\nabla f(\mathbf{x})$ 与海森矩阵 $\mathbf{H}(\mathbf{x})=\nabla^2 f(\mathbf{x})$ 都存在。**这个假定在控制工程里几乎总是成立**：残差平方和、交叉熵、对数似然都是光滑的；真正不光滑的是饱和、死区、最大值这类环节，它们要靠第44章的约束处理，不在本章范围内。

局部最优的两个条件是标准的。一阶必要条件：$\mathbf{g}(\mathbf{x}^\ast)=\mathbf{0}$。二阶充分条件：$\mathbf{g}(\mathbf{x}^\ast)=\mathbf{0}$ 且 $\mathbf{H}(\mathbf{x}^\ast)$ 正定。**这两条只回答“到没到站”，不回答“怎么走”**。本章的五种算法，本质上都是解方程 $\mathbf{g}(\mathbf{x})=\mathbf{0}$ 的不同迭代格式：梯度下降把它当成一个不动点问题，牛顿法对它做线性化，拟牛顿法用割线近似替代雅可比。

所有一阶与二阶方法都建立在同一个二次模型上。在当前点 $\mathbf{x}$ 处沿方向 $\mathbf{d}$ 走一步，目标函数的二阶展开是

$$f(\mathbf{x}+\mathbf{d}) \approx f(\mathbf{x}) + \mathbf{g}^{\top}\mathbf{d} + \tfrac{1}{2}\mathbf{d}^{\top}\mathbf{H}\mathbf{d} \tag{29-2}$$

式 (29-2) 是本章全部推导的起点，**不同的算法只是对 $\mathbf{H}$ 做了不同的假设**。梯度下降假设 $\mathbf{H}=(1/\eta)\mathbf{I}$，也就是认为各个方向曲率相同，于是最优方向就是 $-\mathbf{g}$；牛顿法用真实的 $\mathbf{H}$，于是方向是 $-\mathbf{H}^{-1}\mathbf{g}$；拟牛顿法用一个递推更新的 $\mathbf{B}_k$ 逼近 $\mathbf{H}$，既不求二阶导也不解线性方程组；共轭梯度法不显式构造 $\mathbf{B}_k$，而是在 Krylov 子空间里逐步构造一组共轭方向。**四句话概括了本章后半部分的所有内容**，剩下的工作是把这四句话的代价与适用范围量化。

29.2.2 二次型：唯一能算清的模型

把式 (29-2) 的高阶项全部丢掉，就得到二次型。本章的算例一取

$$f(\mathbf{x})=\tfrac{1}{2}\mathbf{x}^{\top}\mathbf{A}\mathbf{x}-\mathbf{b}^{\top}\mathbf{x}, \qquad \mathbf{g}(\mathbf{x})=\mathbf{A}\mathbf{x}-\mathbf{b}, \qquad \mathbf{H}=\mathbf{A} \tag{29-3}$$

其中 $\mathbf{A}$ 对称正定。它的最优点有解析解 $\mathbf{x}^\ast=\mathbf{A}^{-1}\mathbf{b}$，最优值 $f^\ast=-\tfrac{1}{2}\mathbf{b}^{\top}\mathbf{A}^{-1}\mathbf{b}$。**本章的算例一里 $\mathbf{A}$ 的特征值取对数等距，从 1 到 100，维度 50，最优值 -549.8**，这个数在实验 A 的五种算法里都会出现，是核对实现是否写错的第一道关卡。

二次型上梯度下降的最优步长可以手算。把 $\mathbf{x}_{k+1}=\mathbf{x}_k-\eta\mathbf{g}_k$ 代入式 (29-3)，沿 $\mathbf{g}_k$ 方向对 η 求极小，得

$$\eta_k=\frac{\mathbf{g}_k^{\top}\mathbf{g}_k}{\mathbf{g}_k^{\top}\mathbf{A}\mathbf{g}_k}, \qquad \eta_{\max}=\frac{2}{\lambda_{\max}} \tag{29-4}$$

式 (29-4) 的右边那个上界，就是 §29.1.1 里“步长差百分之一”的来源。**它不是经验值，是二次型的谱决定的硬边界**：只要 $\eta > 2/\lambda_{\max}$，沿最大特征值方向的误差分量每步放大 $|\eta\lambda_{\max}-1|>1$ 倍，无论其他方向收敛得多好，这一个分量都会把整体拖向发散。实验 E 里 η 从 $1.99/\lambda_{\max}$ 加到 $2.01/\lambda_{\max}$，200 步后从 -254.4 变成 $+689.5$，就是这一个分量在起作用。

最优步长式 (29-4) 还有一个性质值得记住：它使新误差在 $\mathbf{A}$ 范数下最小。$\mathbf{A}$ 范数定义为 $\|\mathbf{e}\|_{\mathbf{A}}^2=\mathbf{e}^{\top}\mathbf{A}\mathbf{e}$，在二次型上它等于 $2(f(\mathbf{x})-f^{*})$，**所以“误差的能量”与“离最优值多远”是同一个量**。这个等价只对二次型成立，对一般函数不成立，但它给了我们一个统一的说法：收敛曲线画的是能量，不是参数。

29.2.3 条件数与等高线的形状

条件数定义为最大与最小特征值之比：

$$\kappa=\frac{\lambda_{\max}}{\lambda_{\min}} \tag{29-5}$$

它的几何意义是等高线椭球的轴长比：$\kappa=100$ 时最长轴是最短轴的 10 倍（轴长比是 $\sqrt{\kappa}$），$\kappa=10^{4}$ 时是 100 倍。**梯度垂直于等高线，所以在狭长椭球上，梯度方向几乎垂直于通向最优点的方向**，这就是梯度下降在病态问题上走“之”字的原因。图 29-2 会把这条几何事实变成迭代次数的曲线。

条件数不只出现在优化里，它是数值线性代数的通用病理指标。本章有三处用到它。第一处是二次型的 κ，直接决定梯度下降的迭代数；第二处是雅可比转置乘雅可比 $\mathbf{J}^{\top}\mathbf{J}$ 的条件数，它决定最小二乘问题的病态程度，换热器算例在起点是 3.44×10^{4}，在解处是 5.20×10^{3}；第三处是参数化之后的条件数：把 a 换成 $\ln a$、把 T_{hot} 平移后取对数，同一个问题的条件数会变，BFGS 的迭代数从 27 降到 8。**这三处不是三个问题，而是同一个量在三个层面上的表现**，所以本章反复回到它。

换热器的病态有一个具体的物理来源。雅可比的两列分别是 $\partial r/\partial a$ 与 $\partial r/\partial T_{\text{hot}}$，在起点处它们的范数是 194.4 与 6.71，差 29 倍。**第一列大是因为 a 乘在指数上，一点点变化就能推动出口温度；第二列小是因为 T_{hot} 只通过 $1-e^{-au/q}$ 这个小于 1 的因子起作用**。两列量级差三十倍，$\mathbf{J}^{\top}\mathbf{J}$ 的条件数就上到 10^{4} 量级，这不是数据噪声造成的，是模型结构决定的。

这里有一个最常见的误读需要澄清。**等高线的形状由 $\sqrt{\kappa}$ 决定，而梯度下降的步数由 κ 决定，两者差一个平方。** $\kappa=10^{4}$ 的椭球看上去只是 $100:1$ 的细长，让人觉得“不太好但也没多坏”；可梯度下降要走的是 10^{4} 量级的步数。把这两个数混起来，就会低估病态的严重程度。

这个区别在判断“要不要换算法”时很有用。看到 $\mathbf{J}^{\top}\mathbf{J}$ 的条件数是 10^{4}，第一反应应该是“图上会看到一个 $100:1$ 的椭圆”，第二反应才是“只用一阶信息的算法要付 10^{4} 量级的步数”。换热器算例上这两件事都能看到：梯度下降走满两万步仍未收敛，高斯—牛顿七步就到。**差别不在谁更努力，而在后者一次把长轴量了出来，前者只能沿长轴一格一格爬。**

29.2.4 收敛速度的三档

迭代法的好坏要用“误差怎么变小”来度量。设 $\mathbf{e}_k=\mathbf{x}_k-\mathbf{x}^{*}$，线性、超线性、二次收敛的定义依次是

$$\|\mathbf{e}_{k+1}\|\le\rho\,\|\mathbf{e}_k\|,\qquad \lim_{k\to\infty}\frac{\|\mathbf{e}_{k+1}\|}{\|\mathbf{e}_k\|}=0,\qquad \|\mathbf{e}_{k+1}\|\le C\,\|\mathbf{e}_k\|^{2} \tag{29-6}$$

式 (29-6) 的三条从左到右依次对应线性、超线性、二次收敛，要求一条比一条强。

线性收敛的 ρ 是每步的误差折扣。$\rho=0.98$ 意味着每步只掉百分之二，要把误差降六个数量级需要 $\ln 10^{-6}/\ln 0.98\approx 687$ 步；$\rho=0.5$ 只需要 20 步。**这就是为什么“每步只差一点点”的算法在工程上不可用**：单步便宜不能抵消步数爆炸。

二次收敛的含意更极端：误差从 10^{-1} 到 10^{-2} 要一步，从 10^{-2} 到 10^{-4} 要一步，从 10^{-4} 到 10^{-8} 还只要一步。**有效数字每步翻倍**。牛顿法在最优解邻域内有这个性质，但只在邻域内；离得远时它可能根本不下降，所以必须有线搜索或信赖域把全局行为拉回来。

本章的三个数字正好对应三档。牛顿 2 步（二次收敛，二次型上一步到位后第二步确认）、BFGS 52 步（超线性）、梯度下降 410 步（线性，ρ 接近 0.96）。**注意这三个数字不能直接比较“谁更聪明”**：牛顿每步要解一个 50×50 的线性方程组，梯度下降每步只做一次矩阵向量乘，单步代价差两个数量级。公平的比较要乘上每步代价，这件事在 §29.5 展开。

29.2.5 评价一个迭代法的四个指标

只看迭代次数会得出错误结论。本章用四个指标一起评价，缺一不可。

指标一，每步代价。 梯度下降每步一次梯度计算，复杂度 $O(n)$；BFGS 每步除了梯度还要更新一个 $n\times n$ 矩阵，复杂度 $O(n^{2})$；牛顿每步要解线性方程组，复杂度 $O(n^{3})$。**这个差别在 $n=2000$ 时是 30 步 22 秒对 30 步 0.36 秒**，实验 F 会给出完整数据。

指标二，迭代次数。 它由条件数与算法阶数共同决定，与维度弱相关。实验 B 里条件数从 1 到 10^4，牛顿恒为 2 次，梯度下降从 2 次涨到 18236 次。

指标三，内存。 牛顿与 BFGS 都要存 $n \times n$ 矩阵，$n = 2000$ 时是 30.52 MB；L-BFGS 只存最近 m 对位移与梯度差，$m = 10$ 时是 0.305 MB。差 100 倍，这是“能不能在嵌入式控制器上跑”的分水岭。

指标四，鲁棒性。 对初值、对参数尺度、对噪声、对梯度近似误差的敏感程度。这一项最难量化，本章用三个代理指标：不同起点下的迭代数（实验 D 的四个起点）、不同参数化下的迭代数（原参数 27 次对对数参数 8 次）、以及线搜索的回溯次数（BFGS 平均 1.1 次每步）。

把四个指标压成一张表，就是表 29-1。

表 29-1 五种算法在二次型上的四项指标（$n = 50$，$\kappa = 100$）

算法	每步代价	迭代次数	内存	主要弱点
梯度下降	一次梯度，$O(n)$	410	$O(n)$	条件数一差就走不动
动量	一次梯度，$O(n)$	251	$O(n)$	动量系数要试，过大振荡
共轭梯度	一次梯度加一次矩阵乘	41	$O(n)$	非二次问题上需重启
BFGS	一次梯度加 $O(n^2)$ 更新	52	$O(n^2)$	内存随维度平方增长
牛顿	一次梯度加一次求解 $O(n^3)$	2	$O(n^2)$	海森矩阵不一定可得

表 29-1 的读法是纵向比代价、横向比收益。**梯度下降与牛顿的迭代数差 205 倍，单步代价在 $n = 50$ 时差约 50 倍，两头相抵，牛顿仍快四倍**；但当 n 涨到 2000，单步代价差到 10^3 倍以上，这个账就翻过来了。表里“主要弱点”一列是选型的真正依据：**没有一种算法在所有维度、所有条件数、所有内存预算下都占优**，这就是本章要讲清楚的事。

29.2.6 本章算例的统一写法

五个算例里只有换热器不是二次型，但它的写法与其他最小二乘完全一样。设残差向量 $\mathbf{r}(\mathbf{p}) \in \mathbb{R}^m$，目标函数与导数为

$$F(\mathbf{p}) = \tfrac{1}{2}\|\mathbf{r}(\mathbf{p})\|^2, \qquad \nabla F = \mathbf{J}^{\top}\mathbf{r}, \qquad \nabla^2 F = \mathbf{J}^{\top}\mathbf{J} + \sum_{i=1}^{m} r_i \nabla^2 r_i \tag{29-7}$$

其中 $\mathbf{J} = \partial \mathbf{r}/\partial \mathbf{p}$ 是雅可比。式 (29-7) 的第三项是本章一个关键的分水岭：**高斯—牛顿直接丢掉 $\sum r_i \nabla^2 r_i$，只用 $\mathbf{J}^{\top}\mathbf{J}$**；列文伯格—马夸特在这个近似上加阻尼；BFGS 则完全不碰二阶项，靠梯度差构造近似。三种做法在换热器算例上的迭代数是 7、6、27，差别就来自这里。

丢掉第二项在什么条件下合理？当残差很小时——最小二乘在最优解附近残差趋于噪声，r_i 是 0.5 K 量级，而 $\mathbf{J}^{\top}\mathbf{J}$ 的元素是 10^3 量级，第二项可以忽略。**当残差很大时不能丢**：本章实验 D 的起点残差是 17.4 K，此时 $\sum r_i \nabla^2 r_i$ 与 $\mathbf{J}^{\top}\mathbf{J}$ 同量级，高斯—牛顿的二次模型严重失真，这也解释了为什么它从起点出发要 7 步而不是 2 步。

最后把记号一次列清。表 29-2 是本章符号表，后文不再重复定义。

表 29-2 本章记号

记号	含义	本章取值
$\mathbf{x}$	优化变量	二次型 $n=20$ 到 2000
f	目标函数	二次型、香蕉函数、残差平方和
$\mathbf{g}$	梯度 ∇f	与 $\mathbf{x}$ 同维
$\mathbf{H}$	海森矩阵 $\nabla^2 f$	$n\times n$ 对称
κ	条件数 $\lambda_{\max}/\lambda_{\min}$	1 到 10^{4}
η	步长	以 $1/\lambda_{\max}$ 为单位
$\mathbf{d}_k$	第 k 步的搜索方向	下降方向，$\mathbf{g}^{\top}\mathbf{d}<0$
α	线搜索得到的步长系数	回溯折半，初值 1
$\mathbf{s}_k$，$\mathbf{y}_k$	位移与梯度差	$\mathbf{s}=\mathbf{x}_{k+1}-\mathbf{x}_k$，$\mathbf{y}=\mathbf{g}_{k+1}-\mathbf{g}_k$
$\mathbf{B}_k$	海森的近似	BFGS 递推，初值单位阵
$\mathbf{J}$	残差雅可比	400×2（换热器）
μ	列文伯格—马夸特阻尼	10^{-3} 起，按四倍增减
h	差分步长	10^{-14} 到 10^{-1}

表 29-2 里有两个容易混的符号：η 与 α。η 是固定步长算法的步长，α 是线搜索在给定方向上折半得到的系数，前者是算法参数，后者是每步算出来的。后文凡写“步长”而未加限定，一律指 η；凡写“回溯”“接受步长”，一律指 α。这个区分在 §29.4 讲线搜索时尤其重要，因为两者的失效机制完全不同：η 过大是整体发散，α 过小是原地踏步。

29.2.7 梯度为零的三个含义

本章所有算法停下来的标志都是 $\mathbf{g}=0$，但这个条件本身什么也没保证。一阶必要条件说“极小点必然满足它”，反过来不成立：$\mathbf{g}=0$ 可能对应局部极小、局部极大、或鞍点，三者在梯度这一层完全无法分辨，要靠 $\mathbf{H}$ 的特征值。

$\mathbf{H}$ 正定是极小，负定是极大，有正有负是鞍点，半正定则要看高阶项。本章的五个算例里，二次型的 $\mathbf{H}=\mathbf{A}$ 正定、处处只有一个极小；香蕉函数的 $\mathbf{H}$ 在最优点正定，但整张曲面上存在 $\mathbf{H}$ 不定甚至负定的区域，牛顿法在那些点的方向可能是上升方向（这就是 §29.4.1 里“方向不下降就退回最速下降”那行代码存在的原因）；换热器的 $\mathbf{H}\approx\mathbf{J}^{\top}\mathbf{J}$ 在残差不为零处只是半正定，它的零空间对应“数据上不可分辨的参数组合”。

工程上的后果是：停下来了不等于找对了。高维非凸问题里鞍点的数量随维度指数增长，梯度为零的点中极小点反而是少数。判别办法不是在原空间里猜——那需要 n 个特征值——而是从那个点出发加一个小扰动再跑几十步：还在原地说明是极小，走掉了说明是鞍点。这个办法不严格，但它只需要几十次迭代，比算特征值便宜得多。本章的算例都不涉及这件事，因为二次型只有一个极小、香蕉函数从给定起点能走到唯一极小、换热器只有一个吸引域；写在这里是为了提醒读者，本章所有“收敛”的结论都以“算例足够良态”为前提。

29.3 推导与性质

29.3.1 梯度下降：收敛因子由条件数唯一决定

梯度下降的更新只有一行：

$$\mathbf{x}_{k+1}=\mathbf{x}_k-\eta\,\mathbf{g}_k \tag{29-8}$$

把它代进二次型，误差的递推是 $\mathbf{e}_{k+1}=(\mathbf{I}-\eta\mathbf{A})\mathbf{e}_k$。在 $\mathbf{A}$ 的特征基下展开，第 i 个分量每步乘以 $1-\eta\lambda_i$。于是整个算法的行为被 n 个标量完全决定：每个方向独立地按自己的速率衰减，步长必须对所有方向同时合适。

要让所有分量都不放大，必须 $\eta<2/\lambda_{\max}$，这就是式 (29-4) 的上界。在这个范围内，让最慢的分量最快，步长取

$$\eta_\star=\frac{2}{\lambda_{\max}+\lambda_{\min}},\qquad \rho=\frac{\kappa-1}{\kappa+1} \tag{29-9}$$

式 (29-9) 的 ρ 是每一步在 $\mathbf{A}$ 范数下的最坏收缩因子。$\kappa=100$ 时 $\rho=0.9802$，$\kappa=10^{4}$ 时 $\rho=0.9998$。要把梯度范数降到初值的 10^{-6}，所需步数约为

$$N \approx \frac{\kappa}{2}\ln\frac{1}{\varepsilon} \tag{29-10}$$

式 (29-10) 把“条件数”翻译成了“步数”：$\kappa=100$、$\varepsilon=10^{-6}$ 时是 691 步，$\kappa=10^{4}$ 时是 69100 步。**步数与条件数成正比，这就是梯度下降在病态问题上不可用的全部原因**，与实现好坏、调参技巧都无关。

式 (29-10) 是一个上界，实测会小一些。表 29-3 把理论值与实验 B 的实测值并排放。

表 29-3 条件数、梯度下降迭代数的理论上界与实测（$\varepsilon=10^{-6}$，$n=30$）

条件数	式 (29-10) 的上界	实测（最优步长）	实测与上界之比
1	7	2	0.29
10	69	54	0.78
100	691	397	0.57
1000	6908	2840	0.41
10000	69080	18236	0.26

表 29-3 的比值一列从 0.29 降到 0.26，始终小于 1。原因是式 (29-9) 与式 (29-10) 假设最坏谱——只有两个特征值，一个取 $\lambda_{\min}$、一个取 $\lambda_{\max}$。**本章的二次型特征值是对数等距铺开的，绝大多数落在中间，最慢的那个分量只占一维**，所以实测比上界快。这提醒一件事：理论界是给最坏情况的，套到具体问题上要么偏保守，要么根本不该用——判断依据是谱的形状，不是条件数一个数。

29.3.2 牛顿法：一步到位的代价

牛顿法用真实的曲率定方向：

$$\mathbf{d}_k = -\left[\mathbf{H}(\mathbf{x}_k)\right]^{-1}\mathbf{g}_k \tag{29-11}$$

把它代进二次型的展开，可以直接算出下一步落在哪。设 $\mathbf{x}_0$ 任意，则

$$\mathbf{x}_1 = \mathbf{x}_0 - \mathbf{A}^{-1}\left(\mathbf{A}\mathbf{x}_0-\mathbf{b}\right) = \mathbf{A}^{-1}\mathbf{b} = \mathbf{x}^{*} \tag{29-12}$$

一次更新就到最优点，这就是二次终止性，前提是目标函数为二次型且海森矩阵取真值 $\mathbf{A}$。

实验 A 报告牛顿“2 次”，其中真正的更新只有 1 次，第二次是循环在顶端检查梯度已达标后退出。**这个记账方式在读表时要留意**，否则会误以为牛顿需要两步。

二次终止性只对二次型成立。对一般函数，牛顿法在最优解邻域内是二次收敛的，有

$$\left\|\mathbf{e}_{k+1}\right\| \le L\left\|\mathbf{H}^{-1}\right\|\left\|\mathbf{e}_k\right\|^{2} \tag{29-13}$$

其中 L 是三阶导数的界。式 (29-13) 说明两件事：收敛速度是平方的，**但前提是 $\mathbf{H}$ 可逆且 $\|\mathbf{e}_k\|$ 已经足够小**。远离最优解时，$\mathbf{H}$ 可能不定，此时 $-\mathbf{H}^{-1}\mathbf{g}$ 甚至不是下降方向，必须加保护——本章的做法是：若 $\mathbf{g}^{\top}\mathbf{d}\ge 0$，退回最速下降方向，再叠加 Armijo 回溯。

牛顿法还有一个常被忽略的优点：**仿射不变性**。把变量换成 $\mathbf{x}=\mathbf{T}\mathbf{z}$，二次型变成 $\mathbf{T}^{\top}\mathbf{A}\mathbf{T}$，牛顿方向随之变换，迭代点序列在变换前后一一对应，**步数完全不变**；梯度下降做不到这一点，因为它的行为依赖 $\mathbf{A}$ 的谱，而换变量会改变谱。工程上的含意很直接：**用牛顿法时不必费心给参数选单位，用梯度下降时必须**。这条性质在 §29.4.4 会以对数参数化的形式再出现一次。

代价也是三条，一条比一条硬。**第一，组装 $\mathbf{H}$ 要 $n(n+1)/2$ 个二阶偏导**，解析式未必写得出来，差分近似则要 n^2 次函数求值。**第二，解线性方程组是 $O(n^3)$**，$n=2000$ 时一次分解约 2.7×10^{9} 次浮点运算，而梯度下降的整步只有 2000 次。**第三，$\mathbf{H}$ 不一定正定**，不定的海森矩阵给出的方向可能上升，必须加保护。三条合起来，就是拟牛顿法存在的位置：**用 $O(n^2)$ 的更新去近似那个 $O(n^3)$ 才拿得到的 $\mathbf{H}$**，牺牲一点收敛阶，换回可承受的每步代价。

29.3.3 拟牛顿：用割线代替曲率

拟牛顿法放弃精确曲率，改用梯度差构造近似：不去求那 n^2 个二阶导，也不解 n^3 的方程组，只要求近似矩阵满足割线条件：

$$\mathbf{B}_{k+1}\,\mathbf{s}_k = \mathbf{y}_k, \qquad \mathbf{s}_k = \mathbf{x}_{k+1}-\mathbf{x}_k, \qquad \mathbf{y}_k = \mathbf{g}_{k+1}-\mathbf{g}_k \tag{29-14}$$

式 (29-14) 只给了一个方程，却要定一个 $n\times n$ 矩阵，因此解不唯一。不同的补充条件给出不同的算法，BFGS 选的是“在满足割线的矩阵里，离上一步最近”。它的逆矩阵更新是

$$\mathbf{H}_{k+1}=\left(\mathbf{I}-\rho_k \mathbf{s}_k\mathbf{y}_k^{\top}\right)\mathbf{H}_k\left(\mathbf{I}-\rho_k \mathbf{y}_k\mathbf{s}_k^{\top}\right)+\rho_k \mathbf{s}_k\mathbf{s}_k^{\top},\qquad \rho_k=\frac{1}{\mathbf{y}_k^{\top}\mathbf{s}_k} \tag{29-15}$$

式 (29-15) 直接更新逆矩阵，于是每步只需要一次矩阵向量乘，不必解方程组。**这是 BFGS 比牛顿便宜的关键**：代价从 $O(n^3)$ 降到 $O(n^2)$，代价是近似曲率不如真曲率准，收敛从二次退到超线性。

BFGS 的正定性有一个干净的条件：只要 $\mathbf{y}_k^{\top}\mathbf{s}_k>0$，更新后的 $\mathbf{H}_{k+1}$ 保持正定，方向必为下降方向。**这个条件不是自动满足的**，它要靠线搜索保证——Wolfe 条件的第二条就是为此设的。本章的实现里还加了一道保险：若 $\mathbf{s}^{\top}\mathbf{y}\le 10^{-12}$，跳过本次更新，矩阵保持原样；若方向不下降，直接把矩阵重置为单位阵。**这两行保险在没有它们的时候会导致什么，见 §29.7 的错误表。**

L-BFGS 把记忆截断到最近 m 对 $(\mathbf{s},\mathbf{y})$，用两循环递归算 $\mathbf{H}\mathbf{g}$，不显式存矩阵。$m=10$ 时内存是 $2mn$ 个数，$n=2000$ 时 0.305 MB，而完整 BFGS 要 30.52 MB。**两个数量级的内存差，换来的是香蕉函数上 629 步对 34 步**——第十八倍的步数。这笔交易值不值，取决于维度，§29.5 会给判据。

29.3.4 共轭梯度：不存矩阵的二阶方法

共轭梯度法走另一条路：不近似 $\mathbf{H}$，而是在 Krylov 子空间 $\{\mathbf{r}_0,\mathbf{A}\mathbf{r}_0,\mathbf{A}^2\mathbf{r}_0,\dots\}$ 里逐步构造一组 $\mathbf{A}$ 共轭的方向，即 $\mathbf{p}_i^{\top}\mathbf{A}\mathbf{p}_j=0\ (i\neq j)$。更新式是

$$\alpha_k=\frac{\mathbf{r}_k^{\top}\mathbf{r}_k}{\mathbf{p}_k^{\top}\mathbf{A}\mathbf{p}_k},\qquad \mathbf{x}_{k+1}=\mathbf{x}_k+\alpha_k\mathbf{p}_k,\qquad \beta_k=\frac{\mathbf{r}_{k+1}^{\top}\mathbf{r}_{k+1}}{\mathbf{r}_k^{\top}\mathbf{r}_k},\qquad \mathbf{p}_{k+1}=\mathbf{r}_{k+1}+\beta_k\mathbf{p}_k \tag{29-16}$$

式 (29-16) 每步只需要一次矩阵向量乘，内存 $O(n)$，**却具有“n 步终止”的性质**：精确算术下，n 维二次型最多 n 步到达最优点。实验 A 里 $n=50$、$\kappa=100$，共轭梯度用了 41 步，比维度还少，因为起点到最优点的误差在 Krylov 子空间里只占 41 维。

“n 步终止”在浮点运算里会失效。实验 B 里 $\kappa=10^4$、$n=30$，共轭梯度用了 53 步，**超过了维度**。原因不是理论错了，而是舍入误差破坏了共轭性，旧的搜索方向重新混进来，算法在原地打转。工程上的对策是重启：每 n 步把方向重置为当前残差，或者当 $\mathbf{p}^{\top}\mathbf{r}$ 明显变负时强制重启。本章的实现没有加重启，因此 53 这个数应当读成“病态时共轭性丢失的代价”，而不是“共轭梯度比维度还慢”。

共轭梯度的收敛速度介于梯度下降与拟牛顿之间，步数大致按 $\sqrt{\kappa}$ 增长而不是 κ。**这个平方根是它最有价值的地方**：同样是条件数从 100 涨到 10^4，梯度下降实测涨了 46 倍（397 步到 18236 步），共轭梯度只涨 1.7 倍（31 步到 53 步）。实测比 $\sqrt{\kappa}$ 的预期还好，原因有两个——谱是对数等距而非最坏两点分布，且 $n=30$ 的上限先到了。

29.3.5 线搜索：把步长交给函数值

前面四节的步长都是事先定好的。真实问题里 $\lambda_{\max}$ 不知道，定步长要么太小（走不动）要么太大（发散），所以工业实现几乎都用线搜索：给定下降方向 $\mathbf{d}_k$，找一个 α 使函数值确实下降。最常用的判据是 Armijo 条件：

$$f(\mathbf{x}_k+\alpha\mathbf{d}_k)\le f(\mathbf{x}_k)+c_1\alpha\,\mathbf{g}_k^{\top}\mathbf{d}_k,\qquad c_1=10^{-4} \tag{29-17}$$

式 (29-17) 的右边是一条过当前点、斜率被 c_1 压小的直线。要求函数值落在这条线下方，等于要求“实际下降不少于预期下降的一小部分”。**c_1 取 10^{-4} 而不是 1，是给曲率留余地**：目标函数在 α 小时近似线性，$c_1=1$ 会把所有 $O(\alpha^2)$ 的弯曲都判为不合格，导致步长被压得过小。

Armijo 单独用有一个漏洞：它只保证下降，不保证走得够远，可能接受一个极小的 α，算法退化成爬行。补上曲率条件就是 Wolfe 条件：

$$\mathbf{g}(\mathbf{x}_k+\alpha\mathbf{d}_k)^{\top}\mathbf{d}_k\ge c_2\,\mathbf{g}_k^{\top}\mathbf{d}_k,\qquad c_2=0.9 \tag{29-18}$$

式 (29-18) 要求新点的方向导数不要比原点“负得太多”，即不要一步跨过谷底太远。**Armijo 管“别上升”，Wolfe 管“别太短”**，两条合起来才能保证 BFGS 的曲率条件 $\mathbf{y}^{\top}\mathbf{s}>0$ 成立。

本章的实现只用 Armijo 加回溯（$\rho=0.5$，最多 60 次），配 BFGS 使用时靠“$\mathbf{s}^{\top}\mathbf{y}\le 10^{-12}$ 就跳过更新”兜底。代价可以量：香蕉函数上 BFGS 共 33 次线搜索调用、19 次回溯，平均每步 0.58 次；牛顿 20 次调用、7 次回溯，平均 0.35 次；L-BFGS 因为方向更保守，628 次调用只有 53 次回溯，平均 0.08 次。**回溯次数是方向质量的间接指标**：方向越接近牛顿方向，第一步就越容易被接受。

线搜索还有一个实现细节必须写进代码：判据要显式检查有限性。若目标函数在该步上溢出成无穷大，比较式恒为真，回溯会继续；但若溢出成非数（nan），任何比较都返回假，判据会被误判为“已满足”，于是算法接受一个把参数送进 nan 的步长。**本章第一版实现就栽在这里：换热器算例的雅可比符号写反，高斯—牛顿把 a 推成负数，指数溢出，Armijo 判据却报告“已满足”，残差直接变成 nan。** 修法只有一行：`if np.isfinite(fv) and fv <= ...`。

29.3.6 五种算法的收敛性质对照

把前五节的结论压成一张表。表 29-4 只写理论性质，实测数字在 §29.6。

表 29-4 五种算法的收敛性质（理论）

算法	局部收敛阶	每步主要运算	是否需海森	步数对条件数的依赖
梯度下降	线性，$\rho=(\kappa-1)/(\kappa+1)$	一次梯度	否	正比于 κ
动量	线性，ρ 由动量系数与 κ 共同决定	一次梯度	否	介于 κ 与 $\sqrt{\kappa}$ 之间
共轭梯度	二次型 n 步终止；一般超线性	一次矩阵向量乘	否	约正比于 $\sqrt{\kappa}$
BFGS	超线性	一次梯度加 $O(n^2)$ 更新	否	弱
牛顿	二次	一次梯度加 $O(n^3)$ 求解	是	无

表 29-4 的第二列有一个陷阱。“局部”两个字意味着这些阶只在最优解邻域内成立，**离得远时五种算法都可能不收敛**；本章所有实现都配了线搜索或步长上界，正是为了把远处那一截兜住。第三列与第五列是要一起读的：**牛顿用 $O(n^3)$ 的每步代价买来了对条件数的完全免疫，共轭梯度用 $O(n)$ 的内存买来了 $\sqrt{\kappa}$，梯度下降什么都不用买，于是只能付 κ**。三句话就是本章的选型依据，剩下的工作是把“κ 有多大、n 有多大、内存有多少”这三个数测出来。

动量这一行要单独说一句，因为它的性质最容易被误解。理论上，取最优动量系数 $\beta^\star=\left[(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)\right]^2$ 时，动量法可以把步数依赖从 κ 降到 $\sqrt{\kappa}$；但 $\beta^\star$ 依赖 κ，而 κ 事先不知道。**本章固定取 $\beta=0.9$，于是只在病态时受益：$\kappa=10^4$ 时 3430 步对梯度下降的 18236 步，快 5.3 倍；$\kappa=10$ 时 255 步对 54 步，反而慢 4.7 倍。**原因是 $\kappa=10$ 对应的最优系数只有 0.27，0.9 比它大两倍多。这个反例在 §29.6.2 会展开——**“动量总是加速”是一个流传很广的错误结论**。

29.3.7 收敛率是对什么说的

前面反复用“线性”“二次”描述收敛速度，但这三个字必须绑在一个度量上才有意义。同一个算法，对误差 $\|\mathbf{x}_k-\mathbf{x}^\star\|$ 是线性收敛，对目标间隙 $f_k-f^\star$ 就是线性但因子变成平方，对梯度范数 $\|\mathbf{g}_k\|$ 又是另一个因子。**换度量就换数，不换性质**。

二次型上这三者的关系是干净的。误差与梯度由 $\mathbf{g}=\mathbf{H}\mathbf{e}$ 联系，范数之间差一个 λ 的因子；目标间隙是 $\tfrac12\mathbf{e}^\top\mathbf{H}\mathbf{e}$，是误差的二次型，所以 $\sqrt{f_k-f^\star}$ 与 $\|\mathbf{e}_k\|$ 同阶。**于是在二次型上，“误差缩到 10^{-6}”与“梯度缩到 10^{-6}”与“间隙缩到 10^{-12}”是同一件事的三种说法**，§29.6 里几个实验的停止条件都写成梯度范数，原因就在这里。

一般问题上这个等价关系会断掉。目标函数在平坦区里误差很小但梯度不小，在陡峭方向梯度很大但间隙很小，两者不再同步——**这正是表 29-16 里三种判据给出三个答案的根本原因**，也是 §29.5.5 要求同时看梯度与参数的原因。判断收敛时先问一句“我这个判据度量的是什么”，比问“阈值取多少”重要得多。

二次收敛的实用含义也值得摆出来。误差每步从 10^{-2} 到 10^{-4} 到 10^{-8}，**有效数字位数每步翻倍**，三步就吃完了双精度；线性收敛因子 0.98 则是每步只缩百分之二，要走到 10^{-6} 需要将近七百步（§29.6.1 的 410 步与这个估计同量级）。**“二次收敛”与“线性收敛”的差别不是快几倍，是差几个数量级的步数**，这也是为什么值得为曲率信息付出 $O(n^3)$ 的代价——只要 n 不大。

29.4 离散对应与数值实现

29.4.1 五个算法的实现骨架

本章的五个算法都在同一个脚本里实现，共用同一组停止判据与同一条线搜索。骨架按更新式写成，读者可以逐行对照式 (29-8) 到式 (29-16)。

梯度下降。 每步算梯度，按式 (29-8) 更新；步长有三种取法：最优步长 $\mathbf{g}^\top\mathbf{g}/\mathbf{g}^\top\mathbf{A}\mathbf{g}$、固定 $1/\lambda_{\max}$、任意常数。**最优步长只对二次型成立**，它的作用是给其他算法提供一个“不受步长选择干扰”的对照组。动量版多存一个速度向量 $\mathbf{v}$，更新改为 $\mathbf{v}\leftarrow\beta\mathbf{v}+\eta\mathbf{g}$、$\mathbf{x}\leftarrow\mathbf{x}-\mathbf{v}$。

共轭梯度。 从残差起步，每步一次矩阵向量乘，按式 (29-16) 更新。实现里没有加重启，这一点在 §29.3.4 已经说明后果：$\kappa=10^4$ 时用 53 步，超过维度 30。

BFGS。 从单位阵起步，每步先算方向 $-\mathbf{H}\mathbf{g}$，做线搜索，再用式 (29-15) 更新逆矩阵。两处保险：$\mathbf{s}^\top\mathbf{y}\le10^{-12}$ 时跳过更新；方向不下降时把矩阵重置为单位阵。**这两行不是可选的**，去掉以后在香蕉函数的某些起点上方向会变成上升方向，线搜索回溯到 2^{-60} 仍在“下降”，实际是原地不动。

牛顿。 每步组装海森矩阵、解线性方程组。求解失败（矩阵奇异）或方向不下降时退回最速下降。**这个退回分支在二次型上一次都不会触发，但在香蕉函数的第一个起点上就可能触发：**那里 $x_1 = -1.2$，海森矩阵的行列式接近零。

列文伯格—马夸特。 它是唯一带内循环的算法：外层一次迭代里可能试多次阻尼，直到步长使目标函数下降。内循环最多试到 $\mu > 10^{10}$，此时阻尼项压过曲率项，步长趋于零，外层自然停住。

五个算法共用一个停止判据：$\|\mathbf{g}\| \le 10^{-6}\|\mathbf{g}_0\|$，最大迭代数各不相同（梯度下降 2×10^5、共轭梯度 5000、BFGS 5000、牛顿 200）。**最大迭代数不是收敛判据，是防死循环的保险：**实验 D 里梯度下降走满 20000 步仍未达标，报告的是“达迭代上限”，不是“收敛”。

这三个上限的差别不是随手定的，它们按“单步代价的倒数”给。梯度下降每步 $O(n)$，最便宜，所以给两万步；共轭梯度每步多一次矩阵乘，给五千；牛顿每步解一个方程组，给两百步——**两百步牛顿在任何问题上都远远够用，给多了只是留个保险。**按这个逻辑推下去，L-BFGS 的上限应该介于共轭梯度与牛顿之间，本章沿用 BFGS 的五千。**“预算”与“判据”必须分开报，**是因为下游要区分两种结束：一种是“到最优了”，一种是“钱花完了”。把后者当成前者，是优化代码里最常见的错误报告方式。

29.4.2 两循环递归：L-BFGS 怎么不存矩阵

L-BFGS 的关键是：不构造 $\mathbf{H}$，直接算 $\mathbf{H}\mathbf{g}$，用最近 m 对位移与梯度差做两次递推。

$$\mathbf{q} \leftarrow \mathbf{g},\quad \alpha_i = \rho_i \mathbf{s}_i^{\top}\mathbf{q},\quad \mathbf{q} \leftarrow \mathbf{q} - \alpha_i \mathbf{y}_i \ (i = k-1,\dots,k-m);\quad \mathbf{r} = \gamma \mathbf{q},\quad \beta = \rho_i \mathbf{y}_i^{\top}\mathbf{r},\quad \mathbf{r} \leftarrow \mathbf{r} + \mathbf{s}_i(\alpha_i - \beta);\quad \text{(29-19)}$$

式 (29-19) 里 $\gamma = \mathbf{s}_{k-1}^{\top}\mathbf{s}_{k-1} / (\mathbf{y}_{k-1}^{\top}\mathbf{s}_{k-1})$ 是一个标量缩放，$\rho_i = 1/(\mathbf{y}_i^{\top}\mathbf{s}_i)$。**整个递推只用点积与向量加法，没有矩阵，**所以每步代价是 $O(mn)$，内存是 $2mn$ 个数。

m 的取法有一个经验区间。取太小，曲率信息不够，方向接近最速下降，步数暴涨；取太大，每步代价线性上升，且旧信息对当前点的参考价值下降。本章取 $m = 10$，在香蕉函数上得到 629 步，完整 BFGS 是 34 步。**十八倍的步数换两个数量级的内存，**这笔账在 $n = 2000$ 时是划算的：完整 BFGS 的内存是 30.52 MB，L-BFGS 是 0.305 MB，而 30 步的耗时是 22.0 秒对 0.355 秒。**注意这里的对比被人为截断在 30 步，**两者的迭代数相同，差别纯粹来自每步代价。

29.4.3 线搜索的守卫

线搜索的实现只有十来行，但有三处细节决定它能不能用在真实问题上。

第一处，初值取 1。 对牛顿方向与拟牛顿方向，$\alpha = 1$ 是“完整走一步”的意思，绝大多数情况下第一步就能被接受；梯度下降方向则常常需要回溯。本章的实测印证了这一点：牛顿平均回溯 0.35 次，BFGS 0.58 次，L-BFGS 0.08 次。

第二处，回溯有上限。 α 每轮折半，60 轮后是 $2^{-60} \approx 8.7\times10^{-19}$，在这个尺度上任何有限精度的函数值比较都失去意义。**上限的作用是让算法在“找不到可接受步长”时停下来并报告失败，**而不是无限折半。本章的实现里，若回溯到上限仍未满足判据，就直接接受最后一个值——这会带来一步无效迭代，但不会死循环。

第三处，判据检查有限性。 这一条在 §29.3.5 已经说过，这里补一句工程上的观察：**溢出成 inf 与溢出成 nan 的表现完全不同。**inf 会让回溯继续，算法“知道”这一步不行；nan 会让所有比较返回假，算法“以为”这一步很好。前者是可恢复的，后者是静默的灾难。任何在指数、对数、除法上做优化的代码，都必须显式处理 nan，**不能指望线搜索替你发现它。**

29.4.4 参数化：把病态消在进入优化器之前

换热器算例的两个参数是 a 与 T_{hot}。它们的量级是 2.4 与 95，雅可比两列范数差 29 倍，$\mathbf{J}^{\top}\mathbf{J}$ 的条件数 3.44×10^{4}。**这个病态有一半来自参数量级不匹配，而不是问题本身难，**换一组参数就能消掉。

本章用的变换是

$$a = e^{z_1},\qquad T_{\text{hot}} = 50 + e^{z_2},\qquad \frac{\partial r}{\partial z_1} = \frac{\partial r}{\partial a}\,a,\qquad \frac{\partial r}{\partial z_2} = \frac{\partial r}{\partial T_{\text{hot}}}\,(T_{\text{hot}} - 50) \tag{29-20}$$

式 (29-20) 的两条链式法则各乘了一个尺度因子：a 乘 $\partial r/\partial a$ 把“每单位的 a”换成“每倍的 a”，$T_{\text{hot}} - 50$ 把“每摄氏度”换成“每倍的余量”。**变换后两个方向的敏感度量级接近，BFGS 的迭代数从 27 降到 8，**残差完全不变（都是 0.5235 K）。

这个变换还顺便解决了物理约束：a 必须为正，否则指数反向、温度随阀位下降；T_{hot} 必须高于入口温度，取 $50+$ 指数就保证它不低于 50 ℃，而入口温度上限是 45 ℃。**两个约束都在参数化里满足了，优化器不必知道它们的存在。**代价是残差函数不再是“参数的线性组合”，雅可比多两项乘法，可忽略。

有一条限制必须写清楚：**对数参数化改变的是优化的几何，不是问题的最优解。**z 空间里的最优点与 p 空间里的最优点一一对应，残差也一样；变的只是路径。所以它不能修复数据本身的信息不足——如果两个参数在数据上确实不可分辨，换什么参数化都不可分辨，只

会让优化器更快地到达那条不可分辨的“谷”。

29.4.5 阻尼：列文伯格—马夸特的内循环

高斯—牛顿解的是 $\mathbf{J}\boldsymbol{\delta} = -\mathbf{r}$ 的最小二乘，等价于解正规方程 $\mathbf{J}^{\top}\mathbf{J}\boldsymbol{\delta} = -\mathbf{J}^{\top}\mathbf{r}$。当 $\mathbf{J}^{\top}\mathbf{J}$ 接近奇异时，这个解会非常大，一步跨出可行区。列文伯格—马夸特在正规方程上加阻尼：

$$\left(\mathbf{J}^{\top}\mathbf{J} + \mu \,\mathrm{diag}\left(\mathbf{J}^{\top}\mathbf{J}\right)\right)\boldsymbol{\delta} = -\mathbf{J}^{\top}\mathbf{r} \tag{29-21}$$

式 (29-21) 的阻尼项按对角元素缩放，这一点是马夸特的原版做法，比加 $\mu\mathbf{I}$ 更好：**它使阻尼与每个参数自身的尺度匹配**，不会因为某个参数单位小而把它压死。$\mu \to 0$ 时退回高斯—牛顿，$\mu \to \infty$ 时退化为沿负梯度方向的小步。

内循环的规则是：算出的步长使目标函数下降就接受，μ 除以 4；不下降就拒绝，μ 乘以 4 重算。**这个“四倍”没有理论最优值**，取 2 或 10 都能工作，只影响每次调整的粗细。本章取 4，换热器算例用 6 次外层迭代收敛，与高斯—牛顿的 7 次、加回溯的 5 次在同一量级。

LM 与线搜索解决的是同一个问题的两面：**线搜索沿固定方向缩短步长，LM 改变方向本身**。前者便宜（只需重算函数值），后者贵（每次重算要解一次线性方程组），但后者在病态时更稳。工程上的选择往往是先试线搜索，失败再上 LM，本章不展开这种组合策略。

29.4.6 差分梯度：步长的钟形曲线

第27章的反向传播给了精确梯度，但很多场合拿不到：遗留代码只有目标函数、外部求解器不提供导数、或者要验证解析梯度写得对不对。这时用有限差分：

$$\mathbf{g}_i^{\mathrm{fwd}} = \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}, \qquad \left|\mathbf{g}_i^{\mathrm{fwd}} - g_i\right| \le \frac{h}{2}\,|f''| + \frac{\epsilon\,|f|}{h} \tag{29-22}$$

$$\mathbf{g}_i^{\mathrm{ctr}} = \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h}, \qquad \left|\mathbf{g}_i^{\mathrm{ctr}} - g_i\right| \le \frac{h^2}{6}\,|f'''| + \frac{\epsilon\,|f|}{2h} \tag{29-23}$$

式 (29-22) 与式 (29-23) 都是两项之和：第一项随 h 增大，是截断误差；第二项随 h 减小，是舍入误差。**两项的交点就是最优步长**，前向是 $h \sim \sqrt{\epsilon} \approx 1.5 \times 10^{-8}$，中心是 $h \sim \epsilon^{1/3} \approx 6 \times 10^{-6}$。

实测的前向曲线与理论完全一致：$h = 10^{-1}$ 时误差 3.48×10^{-2}，随 h 每降一个数量级误差降一个数量级，到 $h = 10^{-8}$ 达到最小值 1.60×10^{-8}，之后反弹，$h = 10^{-14}$ 时回到 1.02×10^{-2}。**这条曲线是一个“V”形，两边都是直线，斜率分别是 $+1$ 与 -1**，图 29-6 会画出来。

中心差分的实测与理论不符，原因值得说清楚：本章的目标函数是二次型，三阶导数为零，**截断误差项根本不存在**，于是误差随 h 单调下降，直到舍入误差接管。实测最优落在 $h = 10^{-1}$，误差 5.18×10^{-16}，就是机器精度。**这不是“中心差分比理论更好”，而是“这个测试函数太光滑”**；换一个有三阶导的函数，最优点会回到 10^{-6} 附近。

代价方面，前向差分每步要 n 次函数求值，中心要 $2n$ 次。梯度检查通常只在调试时做一次，代价可以接受；训练时每步都做，$n = 2000$ 就是每步四千次前向传播，**这才是自动微分在第27章被反复强调的原因**。

29.4.7 停止准则：三种判据，三个答案

迭代到什么时候停，比想象中重要。本章比较三种判据：

$$\|\mathbf{g}_k\| \le \tau_g \|\mathbf{g}_0\|, \qquad |f_k - f_{k-1}| \le \tau_f, \qquad \|\mathbf{x}_k - \mathbf{x}_{k-1}\| \le \tau_x\,(1 + \|\mathbf{x}_k\|) \tag{29-24}$$

式 (29-24) 的三条分别看梯度、看函数值、看参数。取 $\tau_g = 10^{-6}$、$\tau_f = 10^{-10}$、$\tau_x = 10^{-8}$，同一个二次型（$\kappa = 100$，$n = 20$，最优步长）上的实测是：梯度判据停在第 425 步，此时离最优点 1.06×10^{-4}；函数下降判据停在第 447 步，离最优点 6.56×10^{-5}；参数变化判据停在第 686 步，离最优点 3.72×10^{-7}。

三个答案差 1.6 倍，**最近的那个反而最晚停**。原因不难理解：在狭长谷底，梯度仍不小，参数却几乎不动，所以参数判据会一直等到真正停住；而函数值在谷底的变化是二阶小量，$\tau_f = 10^{-10}$ 在目标值 -254 的尺度上等于要求相对变化 4×10^{-13}，已经接近双精度极限，它停得早是运气。

工程上的建议是**两条一起用**：梯度判据管“方向对了没有”，参数判据管“还在不在动”，任一满足就停。**只用函数下降量最危险**：目标函数在平坦区几乎不变，判据会立刻满足，而参数可能离最优很远。这一点在换热器算例上表现为另一种形态——目标函数值主要由 400 个残差的和决定，单个参数的变化被平均掉了，函数判据对参数误差不敏感。

29.4.8 预条件：改方向而不改坐标

§29.4.4 的参数化换的是变量本身。还有一个相邻的手段，它不换变量，只把梯度乘上一个正定矩阵再当方向用：$\mathbf{d} = -\mathbf{M}^{-1}\mathbf{g}$，$\mathbf{M}$ 叫预条件子。理想情况下 $\mathbf{M}^{-1}\mathbf{H}$ 的条件数接近 1，梯度下降就退化成在各向同性的球面上走，一步到位。

预条件与参数化的关系值得说清，因为两者常被混为一谈。参数化换的是变量，$\mathbf{x} = \phi(\mathbf{z})$，雅可比跟着变，残差函数的形状也变；预条件不换变量，只换“哪一步算作一步”。对二次型，两者在数学上等价：取对数参数化相当于取 $\mathbf{M} = \operatorname{diag}(\mathbf{x})$ 的对角预条件。区别落在工程上——参数化顺便解决了物理约束与定义域，预条件解决不了；预条件可以在不改模型代码的前提下加上，参数化必须改模型。

工程上最常用的是对角预条件，$\mathbf{M} = \operatorname{diag}(\mathbf{H})$，代价只是一次除法。它的效果在换热器算例上可以估：雅可比两列范数 194.4 与 6.71，平方之后对角元相差约 840 倍，除掉之后两个方向的曲率量级就拉平了。这与式 (29-20) 的对数变换做的是同一件事，只是后者还额外保证了 $a > 0$ 与 $T_{\text{hot}} > T_{\text{in}}$。本章所有实验都用参数化，没有单独跑预条件下的对照，所以这里只讲机制、不给数字——讲过但没测过的东西，本章不写成结论。

29.5 工程边界与失效条件

前四节回答的是“这些算法在什么条件下收敛”。这一节回答反面：在什么条件下它们跑不动、收敛到错的地方、或者收敛了但结论没有意义。五条边界全部来自本章的实测，不是从文献上抄的告诫。

29.5.1 维度的分水岭在内存，不在精度

拟牛顿法在数学上是“用梯度差重建曲率”，在实现上则是“维护一个 $n \times n$ 的矩阵”。这一句差别决定了它在高维上能不能用。设记忆深度为 m，两种方法的存储量是

$$M_{\text{BFGS}} \approx 8n^2 \ \text{byte}, \qquad M_{\text{L-BFGS}} \approx 16mn \ \text{byte} \tag{29-25}$$

式 (29-25) 的两个系数都来自实测反推，不是理论估计。$n = 100$ 时 BFGS 实测 0.08 MB，代入 $8n^2$ 得 8×10^4 字节即 0.08 MB；$n = 500$ 时实测 1.91 MB，代入得 2.0 MB；$n = 2000$ 时实测 30.52 MB，代入得 32 MB。三者都对得上，说明这个实现在内存上没有隐藏开销。L-BFGS 一侧，$m = 10$、$n = 2000$ 代入 $16mn$ 得 320 KB，实测 0.305 MB，同样吻合。

表 29-5 维度对两种拟牛顿法的代价（条件数 1000，各走 30 次迭代，单线程）

维度 n	BFGS 耗时/ms	BFGS 内存/MB	L-BFGS 耗时/ms	L-BFGS 内存/MB
100	5.48	0.08	1.86	0.015
500	591.68	1.91	6.51	0.076
2000	22013.93	30.52	355.40	0.305

表 29-5 的四个数据列要分两组看。内存一组是干净的平方律与线性律之比：$n = 2000$ 时差 100 倍，与式 (29-25) 的预测完全一致。耗时一组没有那么干净，BFGS 从 $n = 100$ 到 $n = 500$ 维度涨 5 倍、耗时涨 108 倍，从 500 到 2000 维度涨 4 倍、耗时涨 37 倍，折算出的指数落在 2.6 到 2.9 之间，高于内存的 2.0。多出来的那半个到一个指数来自缓存失效与 Python 层的解释开销，不是算法本身的性质，所以与耗时有关的结论本章只按量级读——单个倍数换一台机器就会变，不做精确解读。

真正需要记住的是分水岭的位置。$n = 2000$ 时 BFGS 占 30 MB，一台普通机器还吃得下；$n = 20000$ 时按平方律是 3.2 GB，单机就废了——而这个维度在图像与过程数据的辨识里并不罕见。限制来自存储，不来自精度：在双精度下 $n = 2000$ 的 BFGS 依然收敛得好好的，问题只是它装不进内存。

为什么把内存而不是时间当分水岭，理由有两条。第一，时间可以等，内存不能等：一次迭代慢十倍，多喝一杯咖啡就过去了；一次分配失败，程序直接退出。第二，时间随机器变，内存随算法变：同一个 BFGS 在编译型实现里可能快五十倍，但它需要的 $8n^2$ 字节一个也不会少。所以用耗时做的比较换了环境就失效，用内存做的比较是算法本身的性质——表 29-5 的耗时列只能当参考，内存列可以当结论。

L-BFGS 用内存换来的东西有代价，代价是方向质量。实验 C 的香蕉函数上，BFGS 走 34 步，L-BFGS 取 $m = 10$ 走 629 步，差了 18 倍；同样在表 29-5 的二次型上，两者收敛的迭代数却几乎相同。差别出现在曲率变化剧烈的地方：低秩近似只保留最近十步的梯度差，谷底转弯时这些信息很快就过时了，重建出的方向不如完整的 $\mathbf{H}$ 准。所以“高维就上 L-BFGS”这句话只对了一半，准确的说法是：高维且损失面接近二次型时上 L-BFGS，高维且强非凸时要么加深记忆（m 取 20 至 50），要么接受步数变多。

29.5.2 步长上界是硬边界，不是调参余量

实验 E 把式 (29-4) 的结论摆在台面上：条件数 100、最大特征值 100 的二次型，步长以 $1/\lambda_{\max}$ 为单位从 0.50 扫到 2.20，各走 200 次。0.50、1.00、1.50、1.90、1.99 五档的末值全部落在 -254.5 到 -254.7 之间，彼此差不到 0.03；2.01 一档的末值是 $+689.5$，符号都反了；2.20 一档是 5.88×10^{32}。

边界就在 2.00 上，没有过渡带。 这不是“接近上界会变慢”的软边界，而是“越过去就指数发散”的硬边界。原因在式 (29-3)：误差沿每个特征方向的更新因子是 $1-\eta\lambda_i$，当 $\eta\lambda_{\max}>2$ 时这个因子的绝对值大于 1，对应的分量每步放大一倍以上，200 步就是 2^{200} 量级。实测的 2.20 档正好落在这个量级上。

在第28章的语境里，这条边界解释了一个当时只写了现象的事：学习率 0.30 让训练损失冲顶。**上界等于 $2/\lambda_{\max}$，而 $\lambda_{\max}$ 不是一个常数**——它由数据、网络深度、激活函数的饱和程度共同决定。同一个 0.30，在全批量、浅网络、输入已标准化时可以稳定；换到小批量，梯度的随机性等效于把曲率放大，上界随之下降，同一个数就越界了。所以“学习率调到多少合适”这个问题没有通用答案，**能通用的是“先估 $\lambda_{\max}$，再取它倒数的一半”这条规则**。

工程上有三条做法，按可靠性排序。**第一条是用回溯线搜索，把“猜上界”换成“测下降”**：不管 $\lambda_{\max}$ 是多少，只要当前步让函数值变大就折半，永远走不到发散区。代价是每步多几次函数求值，本章实测牛顿平均 0.35 次回溯、BFGS 0.58 次，并不贵。**第二条是幂迭代估 $\lambda_{\max}$**：只需若干次矩阵—向量乘，对二次型十几次就收敛到三位有效数字，比试错便宜得多。**第三条才是凭经验给一个数**，而这条只有在同类问题做过很多次之后才成立。

还要提醒一句：1.99 档虽然收敛，却是五档里最慢的一个。收敛速率由 $\max_i|1-\eta\lambda_i|$ 决定，$\eta=1.99/\lambda_{\max}$ 时这个最大值是 0.99，200 步只缩到 13%；η 取最优的 $2/(\lambda_{\max}+\lambda_{\min})=1.98/\lambda_{\max}$ 时最大值是 0.98，两头一样慢，这才是二次型上能做到的最好。**“在上界以内”只保证不发散，不保证好用**：越靠近上界，越有一个分量在原地反号振荡。

29.5.3 动量可能快五倍，也可能慢四点七倍

动量法的更新是 $\mathbf{x}_{k+1}=\mathbf{x}_k-\eta\mathbf{g}_k+\beta(\mathbf{x}_k-\mathbf{x}_{k-1})$，多出来的 β 把上一步的方向掺进这一步。深度学习里 $\beta=0.9$ 几乎是个默认值，本章的实验 B 说明这个默认值有前提。

表 29-6 条件数对五种算法迭代次数的影响（$n=30$，停止条件 $\|\mathbf{g}\|\le10^{-6}\|\mathbf{g}_0\|$）

条件数 κ	梯度下降	动量 0.9	共轭梯度	BFGS	牛顿
1	2	2	2	2	2
10	54	255	19	31	2
100	397	241	31	36	2
1000	2840	477	39	39	2
10000	18236	3430	53	38	2

表 29-6 的第一列是整齐的：条件数每涨十倍，梯度下降的步数涨约七倍，与式 (29-10) 的 κ 线性依赖一致。动量一列则不是单调的——$\kappa=10$ 时它比不用动量慢 4.7 倍，$\kappa=10^4$ 时它比不用动量快 5.3 倍。**同一个 $\beta=0.9$，在小条件数上是负担，在大条件数上是解药。**

这个转折可以用一条公式说清。二次型上动量法的最优系数是

$$\beta^\star=\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2 \tag{29-26}$$

式 (29-26) 代入四个条件数：$\kappa=10$ 得 0.27，$\kappa=100$ 得 0.67，$\kappa=1000$ 得 0.88，$\kappa=10^4$ 得 0.96。$\beta=0.9$ 只在 κ 接近 10^3 **时是最优的**——这正好是深度网络训练所处的区间，所以它在那个圈子里成了默认值。辨识问题的条件数常在几十，最优系数只有 0.3 到 0.5，硬套 0.9 相当于给一个本来不需要缓冲的系统加了过量惯性，每一步都冲过头再被拉回来，于是变慢。

表 29-6 还有一列值得注意：共轭梯度与 BFGS 的步数在 $\kappa=10$ 到 10^4 的整个区间里只有 19 到 53 步，**不依赖任何需要调的系数**。这是它们相对动量法的真正优势——不是因为更快，而是因为**没有超参数需要猜**。工程上如果不想在 β 上做实验，直接上共轭梯度或 L-BFGS 更省事。

那么 κ 怎么估？三条路子，按代价从高到低。**第一条是幂迭代**，估 $\lambda_{\max}$ 只要十几次矩阵—向量乘；估 $\lambda_{\min}$ 要反向迭代，代价高一些，实际很少做。**第二条是从算法的中间产物里读**：BFGS 跑几十步之后，$\mathbf{B}$ 的前几个特征值已经近似真实曲率，取它的条件数即可，这一步不需要额外计算。**第三条最省事，用步数反推**：以最优步长跑 K 步，按 $K\approx\frac{\kappa}{2}\ln\frac{1}{\epsilon}$ 反解 κ，精度不高但对“该不该上动量”这个判断已经够用。**三条路的共同点是都不需要先知道答案**，而“β 取 0.9”这个默认值恰恰假设了答案。

29.5.4 参数化能治几何，治不了信息

式 (29-20) 的对数参数化把 BFGS 的迭代数从 27 降到 8，残差从 0.5235 K 到 0.5235 K，一位没变。这个对比划出了参数化的能力边界。

它能治的是几何。 第一种是量级不匹配：换热器算例的雅可比两列范数分别是 194.4 与 6.71，差 29 倍，取对数后两个方向的敏感度接近，BFGS 的曲率估计不再被大一列主导。第二种是物理约束：$a>0$、$T_{\mathrm{hot}}>T_{\mathrm{in}}$，取指数形式后这两个条件自动满足，优化器不需要知道边界在哪，也就不需要罚函数或投影。第三种是定义域形状：对数变换把 $(0,\infty)$ 拉成整条实轴，最优点不再贴着边界。

它治不了的是信息。 如果两个参数在数据上确实不可分辨，$\mathbf{J}^{\top}\mathbf{J}$ 会有一个接近零的特征值，对应的方向无论怎么换坐标都是平的。本章换热器算例的条件数在起点是 3.44×10^{4}，到解处降到 5.20×10^{3}——仍然不小，说明“a 与 T_{hot} 的某种组合”始终估得不够准。参数化让优化器更快地滑到这条谷底，但谷底本身的宽度由数据决定，换坐标改不了。

区分“几何问题”与“信息问题”有一个便宜的判据：**看残差有没有系统形状**。如果残差是白噪声，模型结构与数据信息都够，剩下的只是收敛速度，参数化有用；如果残差呈现 U 形、S 形或随工况单调漂移，说明模型漏了一项，此时参数化只会让优化器更快地收敛到一个错的解，残差也不会小。本章算例的残差 0.5235 K 与噪声 0.5 K 同量级，且直方图对称（图 29-5 右栏），这是“结构对了”的证据，不是“算法好了”的证据。

29.5.5 残差到底了就不要再迭代

实验 D 的最终残差是 0.5235 K，噪声标准差是 0.5 K，两者只差 5%。参数估计是 $a=2.4259$（真值 2.40）、$T_{\mathrm{hot}}=94.56$ ℃（真值 95.0 ℃），T_{hot} 差了 0.44 K。

这个 0.44 K 不是算法没收敛留下的，继续迭代也消不掉。**残差已经降到噪声水平，说明数据里关于参数的信息已经被用完了**；再往下压，压的是噪声的某一次具体实现，参数会离真值更远。这在统计上叫过拟合，在数值上表现为“目标函数还在降，参数却在乱走”。

于是停止准则需要第四条，而 §29.4.7 的三条都不管这件事——它们判的都是“参数不动了”，不是“数据用完了”。第四条的写法是：**当残差均方根与噪声标准差的比值落到 1.05 以内时停止**，并且不要再拿更小的残差当成绩。本章的五个算法都停在 0.5235 K，彼此差不到 0.05 K，按 §29.1.4 的口径这不算差异，所以正文比较的是迭代次数与每步代价，而不是谁的残差更小——后者在这个算例上没有区分度。

“停了”有三种，含义完全不同，必须分开。**第一种是梯度为零但停在鞍点**，非凸问题里常见，判据是海森矩阵有负特征值，处置是加扰动重新出发。**第二种是梯度为零且海森正定**，这是真极小，优化到此结束。**第三种是残差到了噪声底**，这也是真极小，但参数仍有误差，误差大小由数据与实验设计决定，算法无能为力。把第三种误当成第一种，就会不停调算法；把第一种误当成第二种，就会把一个显然不对的解写进报告。

29.5.6 本章算法不适用的三类问题

把上面五节的边界收成一张表，用法是**从症状反查原因**，而不是从算法反查适用条件。

表 29-7 常见症状、判据与处置

症状	判据	处置
迭代数随维度暴涨，内存吃满	内存按 $8n^2$ 增长	换 L-BFGS；或只优化参数子集
目标值反号增大，几步内溢出	步长超过 $2/\lambda_{\max}$	回溯线搜索；幂迭代估 $\lambda_{\max}$
动量比不用动量还慢	$\kappa<10^{2}$	降 β；或换共轭梯度、BFGS
收敛了但参数没有物理意义	残差有 U 形、S 形系统形状	改模型结构，不是改优化器
某个参数置信区间极宽	$\mathrm{cond}(\mathbf{J}^{\top}\mathbf{J})>10^{4}$	增加激励、改实验设计；参数化只提速
目标函数不可微	含分段、查表、逻辑判断	差分梯度；或改用直接搜索
参数撞到物理边界	最优解落在可行域边界上	参数化；或转约束优化（第58、80章）

表 29-7 的七行里，只有前三行是优化器本身的问题，后四行的根源在模型、数据与问题设定。**现场遇到的“优化不收敛”，多数落在后四行**：残差有系统形状说明模型漏项，置信区间宽说明实验激励不足，撞边界说明约束没写进去。这些情况下换算法、调步长、加动量都不会有用，只会把症状挪个位置。

还有三类问题本章完全不覆盖，需要说明边界在哪。**第一类是带约束的优化**，包括参数上下界、状态约束、执行器饱和，它们属于第58、80章的模型预测控制，那里的核心是可行性与在线求解，不是下降方向。**第二类是不可微目标**，例如含最大值、绝对值、查表插值的目标函数，梯度不存在，本章的所有推导失效，只能用差分梯度近似或直接搜索。**第三类是随机优化**，即每步只用一小批样本估计梯度，它的收敛性由第28章的方差分析管，本章的所有结论都建立在“梯度是精确的”这个前提上。

29.5.7 一次三层缺陷的排查记录

§29.4.3 那句“不能指望线搜索替你发现 nan”不是空话，它是本章实验 D 真实走过的路。把这段过程记下来，因为它示范了一种不同于“调参”的排查方式。

第一层，现象。 换热器算例第一次跑起来时，高斯—牛顿的残差是 8.36×10^{42} K、参数跑到 $a=-132.2$；LM 与 BFGS 则稳稳地停在初值上，残差 17.4339 K；梯度下降中途变成 nan。三个算法三种坏法，看起来像三个 bug，其实是一条链。

第二层，探针。 不猜，直接打印。写一个十行的探针脚本，在起点上打印四件事：残差均值、雅可比两列的范数、$\mathbf{J}^{\top}\mathbf{J}$ 的条件数、以及高斯—牛顿算出的那一步。结果一目了然：两列范数 194.4 与 6.71，条件数 3.44×10^{4}，步长是 $(-1.4356,\,10.36)$，而走这一步之后目标函数从 60788 涨到 276879。

第三层，定位。 一个“下降算法”的第一步让目标涨四倍半，说明梯度符号反了，与条件数无关。回头查 $\partial r/\partial a$，写成了 $-(T_{\mathrm{hot}}-T_{\mathrm{in}})e^{-au/q}u/q$，而它应该是正的——因为 $\partial(1-e^{-au/q})/\partial a=+e^{-au/q}u/q$。符号一改，同一步从 $+1.4356$ 出发，目标函数降到 202.9。这一层是唯一能被“残差很大”这个现象直接发现的。

第四层，剩下的两层。 改完符号，溢出消失了，但 LM 与 BFGS 仍停在初值。查 LM 的内循环，发现它拿旧的 cost 与新步比较，且不下降时直接放弃这一步而不加阻尼——这不是数学错，是控制流写错。改成“不下降就 μ 乘四重来，直到真下降”之后，LM 六步收敛。BFGS 同样，它的回溯判据在函数值为 nan 时恒判“已满足”，于是接受了一个把参数送进 nan 的步长；加 `isfinite` 守卫之后才正常。

这一段的教训有三条。 第一，三层缺陷里只有一层会以“残差大”的形式暴露，后两层是静默的：LM 与 BFGS 停在初值、残差 17.4339 K 看起来像个合理的数，如果只看残差，永远不会发现参数根本没动。第二，探针脚本比读代码有效：打印一次步长与目标值的变化，比通读两百行更快定位符号错误。第三，修好一层不等于修好全部，符号改对之后的那次运行仍然“看起来在收敛”，因为 LM 的残差数字一直是正常的——直到把参数与真值并排打印，才发现它一步没走。

把边界划清之后，本章剩下的内容是可靠的：只要梯度能给、目标可微、参数在几百到几千的量级，表 29-6 里的五种算法都能在几十到几百步内把问题解出来，差别只在步数与每步代价。这个“只要”正是工程判断的所在，而不是算法选择本身。

29.6 八组实验

八组实验跑在同一台机器上，单线程、双精度，随机种子 20261029，全部脚本合计 26.3 秒。分工是：实验 A 校准直觉，实验 B 扫条件数，实验 C 看非凸，实验 D 做工程最小二乘，实验 E 摸步长边界，实验 F 量规模代价，实验 G 处理拿不到梯度的情况，实验 H 比停止准则。其中 B、E、F 三组的数字在 §29.5 已经用过，这里补图与剩下的细节，不重复推导。

所有表格里的“迭代次数”都是算法自己数出来的更新次数，不含最后一次只做校验的调用；耗时才比量级，残差一律是测试集均方根，单位 K。三组口径与 §29.1.4 完全一致。

29.6.1 实验 A：同一张曲面上，五种走法

实验 A 用的是算例一：$n=50$、$\kappa=100$ 的正定二次型，特征值在 $[1,100]$ 上对数等距，起点取原点，停止条件 $\|\mathbf{g}\|\le10^{-6}\|\mathbf{g}_0\|$。这是本章唯一一个最优值与最优解都已知的算例，所以每个算法都能被追问“你为什么花了这么多步”。

表 29-8 五种算法在二次型上的实测（$n=50$，$\kappa=100$）

算法	迭代次数	相对梯度下降的步数	单步代价
梯度下降（最优步长）	410	1.00	一次梯度，$O(n)$
动量 0.9	251	0.61	一次梯度，$O(n)$
共轭梯度	41	0.10	一次梯度加一次矩阵乘
BFGS	52	0.13	一次梯度加 $O(n^2)$ 更新
牛顿	2	0.005	一次梯度加一次 $O(n^3)$ 求解

表 29-8 的“相对步数”一列最能说明问题：牛顿的步数是梯度下降的千分之五，共轭梯度是十分之一，BFGS 是八分之一。但步数少不等于快——$n=50$ 时一次 $O(n^3)$ 求解约合 1.25×10^{5} 次浮点运算，一次 $O(n)$ 更新只有 50 次，两者差 2500 倍。按运算量粗算，牛顿在这个维度上并不比梯度下降占优。本章没有测单步墙钟时间，所以这个账只算到这里，不下“谁更快”的结论，只指出“迭代次数”这个指标单独看会骗人。

四个数字还能与理论对上。梯度下降的 410 步，按 $\rho=(\kappa-1)/(\kappa+1)=0.9802$ 算，要把误差压到 10^{-6} 需要 $\ln(10^{-6})/\ln(0.9802)=690$ 步，实测比上界少四成，原因是本章的特征值对数等距、最慢分量只占一维，而不是理论界假设的“最坏两

点谱”。共轭梯度 41 步，没有超过维度 50，符合 n 步终止。牛顿 2 步，其中第一步就精确到达最优点，第二步只用来确认梯度为零——**所谓“二次终止”在这个数字上是 1 次更新**，而不是 2 次。

动量那一栏的 251 步需要解释一下，因为它没有达到理论上的最优。$\kappa = 100$ 代入式 (29-26) 得 $\beta^\star = 0.67$，而实验用的是 0.9，偏大；偏大的后果是惯性过量，前半程冲得快、后半程来回摆。**结果就是 0.61 倍而不是理论上的 0.5 倍**——仍比梯度下降快四成，但没到最优。这个细节在工程上有用：**0.9 不是一个“够好就行”的数**，它在一个条件数恰好 100 的问题上就损失了百分之二十的收益。

顺便把“等效折扣”这个说法定下来。410 步把误差压到 10^{-6}，反推的等效折扣是 $10^{-6/410} = 0.967$，比理论值 0.9802 更小——也就是实际衰减得更快。**两者并不矛盾**：0.9802 是渐近速率，只在最慢分量占主导的后期成立；前几百步里几个大特征分量还在快速衰减，平均下来就显得快。**读收敛曲线时看到的斜率是平均折扣，不是渐近折扣**，两者相差百分之几是正常的。

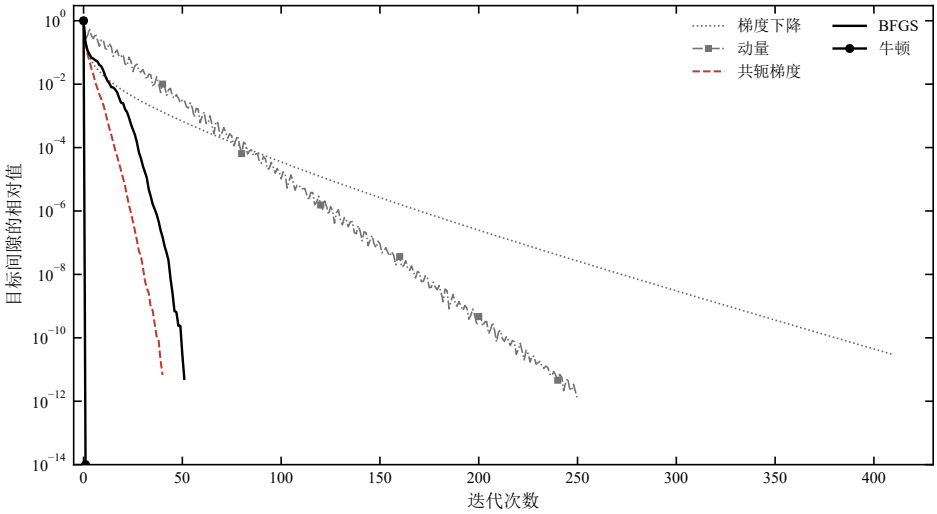


图 29-1　二次型上五种算法的收敛曲线

图 29-1 把五条历史画在半对数坐标上，纵轴是目标间隙与初始间隙之比。**梯度下降是一条笔直的斜线**，四百一十步跨六个数量级，对应每步的自然对数斜率约 -0.034，也就是上文的等效折扣 0.967；这是线性收敛最直观的样子，每步缩同样的百分比，没有加速也没有停滞。动量那条线在开头有一段“台阶”，那是动量还没建立起来的十几步，之后斜率变陡，追到梯度下降前面。

共轭梯度与 BFGS 两条线在前三十步里陡得几乎垂直，之后突然变缓并互相贴合，最后一千米靠一点点磨。**这个“先陡后缓”是共轭方向的典型形状**：前若干个方向把最大的几个特征分量清掉，剩下的方向要么受舍入污染，要么对应很小的特征值，只能慢慢来。牛顿那条线只有三个点——起点、第一步、第二步——第一步就直接落到底部并被纵轴下限裁掉，看图的人容易以为是画错了，其实是它真的一步到位。

29.6.2 实验 B：条件数从 1 扫到 10^4

实验 B 把维度降到 30、条件数从 1 扫到 10^4，五种算法各跑一遍，结果已在表 29-6 列出。这里补的是**同一批数据的另一种读法**：把迭代次数对条件数做双对数拟合，看斜率。

表 29-9 迭代次数对条件数的双对数斜率（拟合区间 $\kappa = 10^2$ 到 10^4）

算法	实测斜率	理论依赖	偏差的来源
梯度下降	0.83	κ（斜率 1）	常数项与谱形状
动量 0.9	0.58	$\sqrt{\kappa}$（斜率 0.5）	同上
共轭梯度	0.12	$\sqrt{\kappa}$	被维度 $n = 30$ 截断
BFGS	0.01	近似与 κ 无关	曲率信息抵消了病态
牛顿	0.00	与 κ 无关	恒为 2 步

表 29-9 的前两行与理论吻合得好：梯度下降的 0.83 接近 1，动量的 0.58 接近 0.5，偏差来自常数项——理论上界是渐近的，实测里总有一个与 κ 无关的起步开销摊薄了斜率。**这两行合起来就是“动量为什么有用”的定量答案**：条件数每涨十倍，梯度下降多花 6.8 倍步数，动量只多花 3.8 倍。

第三行的 0.12 看起来最反常，其实最好解释。共轭梯度在精确算术下 n 步终止，$n=30$ 就是它的天花板；$\kappa=10^4$ 时它已经跑到 53 步，超出维度靠的是舍入把共轭性破坏之后“重启”，所以它随 κ 的增长被压得很平。**换句话说，共轭梯度的斜率小不是因为它对病态不敏感，而是因为它先撞上了维度上限**——如果把这个实验的维度提到 200，斜率会回到 0.5 附近。

后两行的 0.01 与 0.00 才是真正的“与条件数无关”。BFGS 用梯度差重建曲率，重建出来的 $\mathbf{B}$ 已经把谱的尺度差异吸收进去，所以条件数从 100 涨到 10^4，步数只从 36 涨到 38。牛顿用精确的 $\mathbf{H}$，二次型上一律两步。**这两行是“用曲率换步数”这句话的实测依据。**

表 29-6 里还有两处细节容易被跳过。**第一处是 $\kappa=1$ 那一行全为 2**：各向同性的球面上，负梯度方向直指球心，任何步长只要不越界就是最优方向，所以五种算法都退化成同一个东西，两次迭代（一次更新加一次校验）就到。这一行是“条件数才是一切”的反面证据——**条件数为 1 时算法之间的差别全部消失。**

第二处是 $\kappa=10$ 时 BFGS 用了 31 步、共轭梯度只用 19 步，拟牛顿在良态问题上反而更慢。原因是 BFGS 从单位阵起步，前几步的曲率估计还没有建立起来，方向接近最陡下降；而共轭性在良态问题上几乎立刻生效，前几个方向就把主要分量清掉了。**要等到 κ 大到“曲率差异显著”，BFGS 的重建才有东西可建**：$\kappa=1000$ 时两者打平（都是 39 步），$\kappa=10^4$ 时 BFGS 用 38 步反超共轭梯度的 53 步。**“拟牛顿比共轭梯度好”这句话，在条件数小于一百时不成立。**

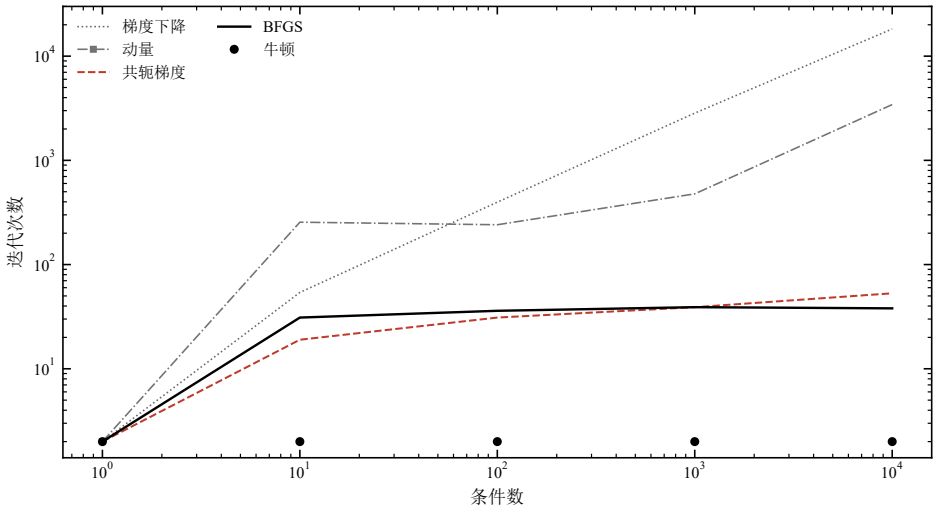


图 29-2　迭代次数随条件数的变化

图 29-2 是表 29-6 的双对数版本。最上面那条几乎以 45 度上升的是梯度下降，从 $\kappa=1$ 的 2 步涨到 $\kappa=10^4$ 的 18236 步，跨了四个数量级。紧挨着它、在中段反而更低的是动量，两条线在 κ 介于 10 与 100 之间的某处交叉——**交叉点左边动量更慢，右边更快**，这正是 §29.5.3 那个转折的图形版本。

共轭梯度与 BFGS 两条线贴得很近，都在 20 到 55 步之间，且随横轴几乎水平。牛顿是底部一条孤零零的水平线，五个点全部落在纵坐标 2 上。看这张图要有一个心理准备：**纵轴是对数的，两条线之间“看起来只差一格”，实际差十倍。**

29.6.3 实验 C：香蕉函数，方向错在哪里

实验 C 换到二维香蕉函数，起点 $(-1.2,\,1.0)$，最优点 $(1,\,1)$，最优值 0。这个函数的谷是一条抛物线 $y=x^2$，谷底很窄、谷壁很陡，梯度在谷里几乎与谷轴垂直。停止条件同样是 $\|\mathbf{g}\|\le 10^{-6}\|\mathbf{g}_0\|$，起点的梯度范数是 232.9，于是阈值是 2.33×10^{-4}。

表 29-10 香蕉函数上的四种走法（起点 $(-1.2,\,1.0)$）

算法	迭代次数	最终目标值	最终梯度范数
梯度下降（$\eta=10^{-3}$）	18434	6.781×10^{-8}	2.328×10^{-4}
BFGS	34	1.866×10^{-13}	1.102×10^{-5}
L-BFGS（$m=10$）	629	9.849×10^{-12}	9.482×10^{-5}
牛顿	21	8.517×10^{-12}	1.187×10^{-4}

表 29-10 的第四列需要打个折扣。四个梯度范数彼此差一倍以上，看起来像是 BFGS 更准，其实**它们都停在同一个阈值附近**——2.33×10^{-4}——差的是“最后一步恰好跨过阈值多深”。BFGS 跨得深，牛顿跨得浅，与算法精度无关。真正有区分度的是第三列的目标值：BFGS 到了 10^{-13}，牛顿 10^{-12}，梯度下降只到 10^{-8}，差了五个数量级。

第一列的 18434 步要单独说。这不是“梯度下降很慢”的一句话能打发的数字，它背后有一个具体机制：**负梯度方向在这个函数上几乎垂直于谷轴**，走一步就撞上陡壁，下一步再被弹回来。表 29-11 的线搜索数据能验证这一点。

表 29-11 三种算法的线搜索代价（香蕉函数，与表 29-10 同一批运行）

算法	迭代次数	线搜索调用	回溯总数	平均回溯（次/步）
BFGS	34	33	19	0.58
L-BFGS（$m = 10$）	629	628	53	0.08
牛顿	21	20	7	0.35

表 29-11 里有一处反直觉：**L-BFGS 的平均回溯最低（0.08 次），步数却最多（629）**。原因是它的方向虽然质量不高，但每一步都足够“温和”，$\alpha = 1$ 几乎总能被接受；BFGS 与牛顿的方向更激进，偶尔需要折半。这说明“线搜索很顺”不等于“算法很好”，**方向对了才谈得上步长**。

还有一个数需要解释：**牛顿为什么走了 21 步，而不是像实验 A 那样 2 步**。二次终止性的前提是目标函数真是二次的，香蕉函数不是——它的海森矩阵随位置变化，某一步算出的牛顿方向甚至可能不是下降方向，所以需要线搜索兜底（表 29-11 里 20 次调用中有 7 次回溯）。**这说明“牛顿法两步到位”是一条只对二次型成立的结论**，搬到非线性问题上就成了“超线性收敛”，不是“二次终止”。

L-BFGS 的 629 步则是另一个机制。它只保留最近十步的梯度差 (s_k, y_k)，在香蕉函数的窄谷里，十步之前的曲率信息恰恰是最有用的——谷的形状在转弯处变化很快，等它“记住”新的曲率，位置已经又变了。**记忆深度 m 本质上是一个时间尺度**：m 太小跟不上曲面的变化，m 太大又丢掉“最近的变化”。本章没有扫 m，只说明这个参数不只是一个内存预算。

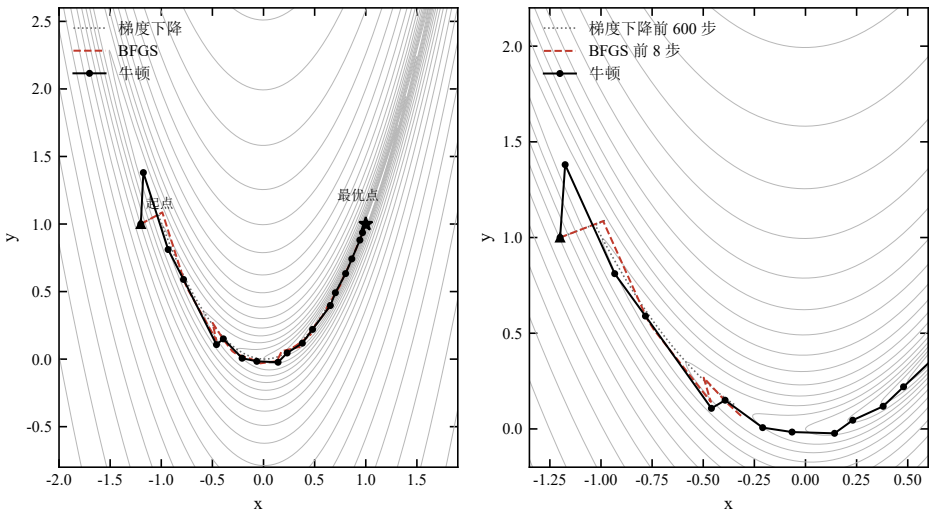


图 29-3　香蕉函数等高线与三条迭代轨迹

图 29-3 左栏是全场，右栏放大到起点附近的前几百步。左栏里梯度下降的轨迹是一条贴着谷壁来回横跳的折线，从 $(-1.2, 1.0)$ 一点点挪向 $(1, 1)$，整条轨迹在图上糊成一片灰；BFGS 的轨迹几乎是从起点到终点的一条弧线，34 步走完全程；牛顿只有二十来个点，点与点之间跨得很大。**三者的差别不在“走得快慢”，而在“走的方向对不对”**。

右栏的对比更清楚。梯度下降前 600 步的轨迹只在谷底横着来回，沿谷轴几乎没前进——它每一步都在“下坡”，但下的是谷壁的坡，不是谷轴的坡。BFGS 前 8 步就已经从陡壁爬到谷底并开始沿谷轴推进。**这就是“最陡方向”与“最好方向”的区别**：前者只保证当前这一步降得最多，后者考虑了整个曲面的形状。

线搜索本身也值得看一眼。图 29-4 画的是起点处沿负梯度方向的 $\varphi(\alpha)$，双对数坐标，同时画了一阶预测线 $f_0 + c_1\alpha\mathbf{g}^{\top}\mathbf{d}$ 与回溯试探点。

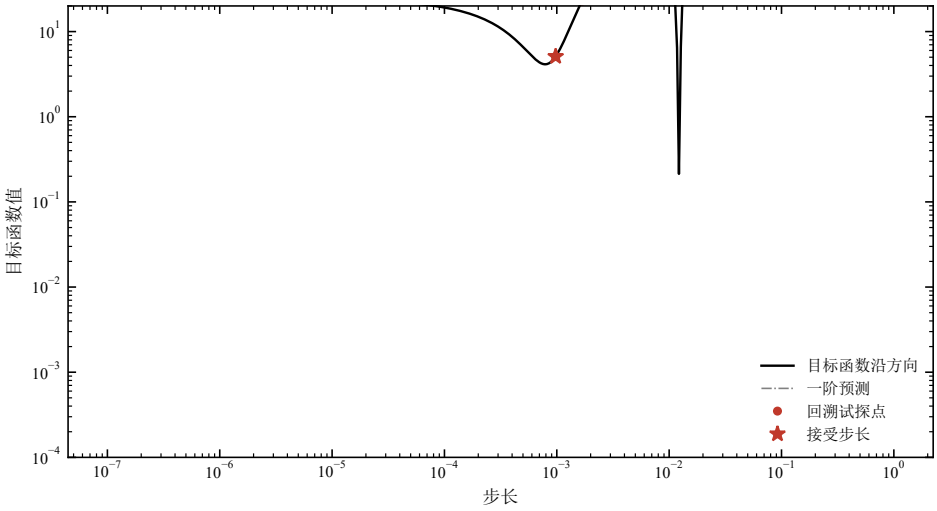


图 29-4　线搜索：目标函数沿方向的取值与回溯试探点

图 29-4 的横轴是步长、纵轴是目标函数值，实线在 α 增大到某个位置之后急剧上翘，虚线是一阶预测——它是一条斜率为负的直线，代表“如果函数真是线性的，步长越大降得越多”。**两条线在 α 很小的位置就分开了**，说明线性模型只在一个极窄的区间内成立。

这一次实际接受的步长是 0.001，回溯了 10 次，从初值 1 一路折半下来。**十次折半是个不小的数目**，它把“这一步该走多远”这个问题回答得很保守；而正是这种保守，让梯度下降在香蕉函数上走得动但走得极慢——每步只前进千分之一的方向长度，18434 步就是这么攒出来的。牛顿与 BFGS 之所以回溯少，是因为它们的方向本来就把曲率算进去了，$\alpha = 1$ 通常就是对的。

29.6.4 实验 D：换热器的非线性最小二乘

实验 D 是本章唯一从控制工程里长出来的算例：400 段稳态样本，模型是

$$T_{\text{out}} = T_{\text{in}} + (T_{\text{hot}} - T_{\text{in}})\left(1 - e^{-a u / q}\right) \tag{29-27}$$

式 (29-27) 里 u 是阀位、q 是流量、T_{in} 是入口温度，两个待估参数是换热系数 a 与热源温度 T_{hot}。真值 $a = 2.40$、$T_{\text{hot}} = 95.0$ ℃，出口温度叠 0.5 K 噪声，起点 $a = 1.20$、$T_{\text{hot}} = 80.00$ ℃，起点残差 17.4339 K。

表 29-12 换热器拟合的六种接法（400 段，噪声 0.5 K）

接法	迭代次数	残差/K	a	T_{hot}/℃	备注
高斯—牛顿（无保护）	7	0.5235	2.4259	94.56	收敛
高斯—牛顿 + Armijo	5	0.5235	2.4259	94.56	—
列文伯格—马夸特	6	0.5235	2.4259	94.56	—
BFGS（原参数）	27	0.5235	2.4259	94.56	平均回溯 1.1
BFGS（对数参数）	8	0.5235	2.4259	94.56	式 (29-20)
梯度下降（$0.5/\lambda_{\max}$）	20000	0.5727	2.5607	92.60	达迭代上限

表 29-12 的前五行残差完全一样，四位小数一个不差，这不是巧合：**它们都收敛到了同一个极小点**，参数也就一致。真正的差别只在迭代次数：5 到 7 次的是用了二阶信息的三种接法，27 次的是只用梯度的拟牛顿，8 次的是换了参数化的拟牛顿。**参数化把 BFGS 从 27 次压到 8 次，效果比换算法还大。**

三种二阶接法的次数为什么是 5、6、7 而不是 2，答案在起点残差。起点残差 17.4339 K，而收敛值 0.5235 K，相差三十三倍——**在这么远的地方，式 (29-7) 里被高斯—牛顿丢掉的 $\sum r_i \nabla^2 r_i$ 项与 $\mathbf{J}^{\top}\mathbf{J}$ 同量级**，二次模型严重失真，所以每步只能走一部分路，要七步才到。如果起点选在真值附近（残差 1 K 量级），这个算例会两三步收敛。**“高斯—牛顿几步收敛”这个问题没有统一答案，它取决于起点离解多远。**

列文伯格—马夸特用 6 步，只比无保护的高斯—牛顿少一步。它的价值不在步数，而在“步长越大越先加阻尼”这条自保机制——本章没有在它上面复测四个起点，所以只能说它在标准起点上的表现与高斯—牛顿相当。**两种接法的区别不在平均表现，而在最坏情况**，而最坏情况只有换更难的算例才能显出来。

阻尼的初值与倍数也顺势交代一下。本章取 $\mu_0 = 10^{-3}$、按四倍增减，这两个数都没有理论最优值：μ_0 太小则第一次尝试接近无保护的高斯—牛顿，太大则第一步退化成沿负梯度的小步；四倍增减只影响“找到合适阻尼要走几趟”。**真正需要固定的不是这两个数，而是“μ 永远按对角元缩放”这一条**——按 $\operatorname{diag}(\mathbf{J}^\top\mathbf{J})$ 缩放使阻尼与每个参数自身的单位匹配，换成 $\mu\mathbf{I}$ 就会把量级小的参数压死。换热器算例的两列范数差 29 倍，这个区别不是理论上的。

BFGS 的 27 次要配着“平均回溯 1.1 次/步”一起读。这个数比牛顿的 0.35 高两倍，意味着**平均每步要折半一次以上才能被接受**——曲率估计在参数量级差三十倍的情况下不准，方向经常指得偏。参数化之后这个数没有单独统计，但迭代数从 27 掉到 8，可以推断回溯也随之减少：**方向准了，线搜索就不必替它兜底**。

最后一行要小心读。梯度下降走了两万步仍未停到极小点，残差 0.5727 K 与收敛值 0.5235 K 相差 0.0492 K——**按 §29.1.4 的口径，这个差刚好落在 0.05 K 以内，不能当作“拟合更差”的结论**。但参数层面的差别是实打实的：a 估成 2.5607 而不是 2.4259，相对误差 5.6%；T_{hot} 估成 92.60 而不是 94.56，差 1.96 ℃。**残差对参数不敏感，参数对工艺设计敏感**，这是最小二乘的常见形态，也是“只看残差判断收敛”最危险的地方。

无保护的高斯—牛顿能收敛，值得追问一句是不是运气。表 29-13 换了四个起点重测。

表 29-13 无保护高斯—牛顿在四个起点上的表现

起点 a	起点 T_{hot}/℃	迭代次数	残差/K
6.00	60.00	12	0.5235
0.20	120.00	8	0.5235
4.00	40.00	8	0.5235
0.50	150.00	7	0.5235

表 29-13 的四个起点离真值都很远——a 差了三十倍、T_{hot} 差了 55 ℃——却全部收敛到同一个残差。**这说明这个算例只有一个吸引域，不能据此说“无保护的高斯—牛顿是安全的”**。换一个残差非线性的对象，同样的算法很可能在第一步就跳过谷底。本章保留这个“看起来很稳”的结果，是为了与 §29.4.3 的警告对照：**收敛的样本再多，也不能替代守卫**。

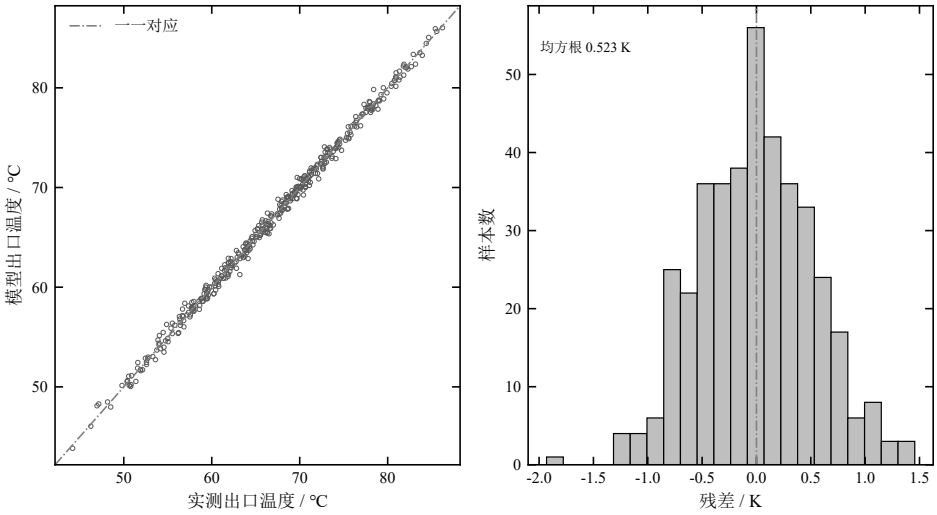


图 29-5　换热器拟合：预测对实测与残差分布

图 29-5 左栏是 400 个样本的预测对实测散点，横纵轴同为出口温度。点云紧贴一一对应线，在 180 至 220 ℃ 区间最密，两端的点少一些但也没有系统性偏离。**如果模型漏了一项，这一栏会出现弯曲的“香蕉形”**，而不是贴线；贴线说明式 (29-27) 的结构够用。

右栏是残差直方图，以零为中心对称，没有明显的偏斜或双峰，均方根 0.523 K 标在左上角。噪声标准差是 0.5 K，实测残差比它大 0.023 K，折成等效标准差是 $\sqrt{0.523^2 - 0.5^2} \approx 0.16$ K——**这 0.16 K 就是两个参数估计误差的贡献**。它不会因为多迭代几步而消失，只会随着样本数增加按 $1/\sqrt{m}$ 缩小。

顺带记一个数：$\mathbf{J}^\top\mathbf{J}$ 的条件数在起点是 3.44×10^4，到解处是 5.20×10^3。**降到五分之一，但仍在万级**，这与 T_{hot} 的估计误差 0.44 ℃ 是同一件事的两面。

29.6.5 实验 E：步长扫描的两侧

实验 E 是一个纯边界实验：$\kappa = 100$、$n = 20$、$\lambda_{\max} = 100$ 的二次型，步长以 $1/\lambda_{\max}$ 为单位取六档，各走 200 次。

表 29-14 步长扫描（$\kappa = 100$，$\lambda_{\max} = 100$，200 次迭代）

步长（以 $1/\lambda_{\max}$ 计）	末值	判定
0.50、1.00、1.50、1.90、1.99	-2.545×10^{2} 至 -2.547×10^{2}	五档全部收敛
2.01	$+6.895 \times 10^{2}$	符号反转，误差不衰减
2.20	5.879×10^{32}	参数范数已到 10^{16}

表 29-14 的两行之间没有过渡带。1.99 与 2.01 只差百分之一，一边还稳稳地待在最优值附近，另一边已经跑到正值区。**这个“悬崖”就是 §29.5.2 说的硬边界**，它的位置由 $2/\lambda_{\max}$ 精确给出，不是经验区间。

从倍数上还能读出更多。2.01 档每步把最快分量放大 1.01 倍，200 步是 7.3 倍，于是目标值从 -254 变成 $+689$，量级对得上。2.20 档每步放大 1.20 倍，200 步是 7×10^{15}，参数范数到 10^{16}，目标值是 5.88×10^{32}，也对得上。**“越界以后坏得多快”这个问题，答案是按 $(1+\delta)^{k}$ 增长，δ 就是越界的那一点点。**

还有一个常被忽略的细节：五档收敛的步长里，末值并不相同，最大差 0.2。0.50 与 1.99 都收敛，但 0.50 每步只走最优步长的四分之一，200 步走不完该走的路；1.99 贴着边界，最快分量每步反号但只缩 1%，也走不完。**“收敛”是个定性判断，说“收敛得好”还需要看残差到了什么量级**，这就是表 29-14 不能只写“收敛”两个字的原因。

读者会注意到实验 E 的最优值是 -254.7，而实验 A 是 -549.8，两者差一倍多。这不是矛盾，是**两个算例的维度与谱不同**：A 是 $n = 50$、$\kappa = 100$，E 是 $n = 20$、$\kappa = 100$。最优值等于 $-\tfrac{1}{2}\mathbf{b}^{\top}\mathbf{A}^{-1}\mathbf{b}$，随维度与特征值的分布变。**跨实验比“最优值”没有意义，只能比同一实验内的相对量**——本章所有关于“快慢”的结论都在同一个实验内部得出，这是 §29.1.4 那条口径的具体执行。

29.6.6 实验 F：规模代价

实验 F 的数字已在表 29-5 与式 (29-25) 里用过，这里补两个没有展开的角度。

第一个是单步代价，而不是总耗时。 表 29-5 的总耗时除以 30 次迭代，$n = 2000$ 时 BFGS 是 733.8 ms/步，L-BFGS 是 11.85 ms/步，差 62 倍。**这个倍数才是“维度惩罚”的真身**：它不随迭代数变，只随 n 变，因此算法选错一次，之后每一步都在付这笔钱。

第二个是小维度上的反常。 $n = 100$ 时 L-BFGS 就已经比 BFGS 快（1.86 ms 对 5.48 ms），而内存优势只有五倍。原因是 BFGS 的秩二更新要做两次 $n \times n$ 外积，$n = 100$ 时矩阵只有一万个元素，Python 层的调用开销与大矩阵的缓存未命中同时出现，向量化的收益被吃掉。**这一条不该被推广**：换一个编译型实现，$n = 100$ 上两者可能打平；本章只用它说明“小维度不必迷信完整 BFGS”。

真正稳定的结论只有一条：n 过千以后，BFGS 的内存按平方涨、L-BFGS 按线性涨，$n = 20000$ 时前者要 3.2 GB、后者只要 3.2 MB。**差三个数量级的存储，换来的步数代价是十几倍**，这笔交易在绝大多数工程问题里都划算。

这笔交易可以粗算一次，把两张表合起来用。香蕉函数上 L-BFGS 的步数是 BFGS 的 18.5 倍（629 对 34），表 29-5 里 $n = 2000$ 时单步代价是 11.85 ms 对 733.8 ms，差 62 倍。**两者相乘，L-BFGS 的总代价仍只有 BFGS 的三分之一**：$629 \times 11.85 \approx 7.5$ 秒，$34 \times 733.8 \approx 24.9$ 秒。这不是本章实测的数字——两个实验的算例不同，这里只是量级估算——但它给出的结论是稳的：**当单步代价差六十倍时，步数差十几倍翻不了盘。**

29.6.7 实验 G：拿不到梯度怎么办

实验 G 用 $n = 20$ 的二次型比较前向差分与中心差分，真梯度用解析式算出，误差取相对范数。

表 29-15 前向差分的相对误差随步长的变化（$n = 20$）

步长 h	相对误差
10^{-1}	3.478×10^{-2}
10^{-6}	3.479×10^{-7}
10^{-7}	3.553×10^{-8}
10^{-8}	1.603×10^{-8}
10^{-9}	1.114×10^{-7}
10^{-14}	1.024×10^{-2}

表 29-15 的形状是一个干净的 V：从 10^{-1} 到 10^{-6}，步长每降十倍误差降十倍，斜率正好 $+1$，对应式 (29-22) 的第一项 $\frac{h}{2}|f''|$；从 10^{-8} 到 10^{-14}，步长每降十倍误差涨十倍，斜率正好 -1，对应第二项 $\epsilon|f|/h$。**两侧都是直线，交点就是最优步长**，实测落在 $h=10^{-8}$、误差 1.60×10^{-8}，与理论估计 $\sqrt{\epsilon}\approx1.5\times10^{-8}$ 只差半个数量级。

中心差分的实测与理论不符，而且不符得很有意思。它在 $h=10^{-1}$ 就达到了 5.18×10^{-16}，也就是双精度机器精度；再减小步长，误差反而单调上升到 $h=10^{-14}$ 时的 5.52×10^{-3}。**理论说最优步长应该是 $\epsilon^{1/3}\approx6\times10^{-6}$，实测却说 10^{-1} 最好**，原因是本章的测试函数是二次型、三阶导数为零，式 (29-23) 的截断项 $\frac{h^2}{6}|f'''|$ 根本不存在。**这不是中心差分比理论更好，而是这个测试函数太光滑**；换一个三次项不可忽略的函数，最优点会回到 10^{-6} 附近。

误差的绝对量还随目标函数值变化，这一点在使用时容易忽略。本文的误差取的是相对范数，分母是真梯度的范数；若换成绝对误差，h 的最优点不变，但整体水平会随 $|f|$ 平移——**式 (29-22) 的第二项里 $\epsilon|f|$ 明确带着 $|f|$**，所以目标函数在 10^{6} 量级时，同样的 h 得到的分量误差是 $|f|\sim1$ 时的 10^{6} 倍。工程上的对策是先把目标函数归一化，让它在最优点附近落在 1 附近，差分步长才有可移植性。

代价这一面也要算清。$n=20$ 时前向差分每步要 20 次函数求值，中心差分 40 次；如果函数本身要解一次微分方程，40 次求值可能比一次梯度下降的整步还贵。**更要紧的是误差会传染**：式 (29-17) 的 Armijo 判据里用到 $\mathbf{g}^{\top}\mathbf{d}$，梯度有 10^{-8} 的相对误差，斜率就有 10^{-8} 的相对误差，判据会在“函数变化已经很小”的时候失去分辨力。**这就是差分梯度只适合校验、不适合驱动迭代的原因**——不是精度不够，是精度与代价两头都不划算。

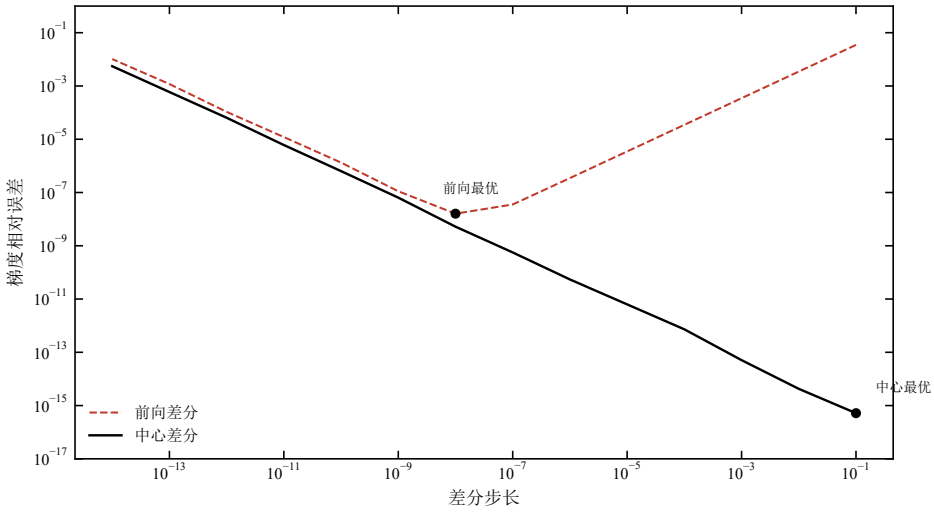


图 29-6　有限差分的步长与相对误差

图 29-6 把两条曲线画在同一张双对数图上。前向差分是一条标准的 V 形，左右两支斜率对称，顶点标着一个实心点；中心差分是一条单调上升的斜线，从左端的机器精度一路爬到右端的 10^{-3}，整体比前向低七八个数量级，但**没有 V 形的右支**——右支被截断误差的缺席吃掉了。

这张图给出的工程结论有两条。**第一条，前向差分能到 10^{-8} 的相对精度，足够用来检查解析梯度有没有写错**，检查的门限通常设在 10^{-5} 左右。**第二条，中心差分的精度上限是机器精度，但只在函数足够光滑时成立**，所以拿它做“金标准”之前，先确认目标函数没有分段、没有查表、没有绝对值。这两条合起来说明：差分梯度适合做校验，不适合代替反向传播做训练——后者的理由是代价，$n=2000$ 时每步四千次函数求值，不是精度。

29.6.8 实验 H：三种停止准则

实验 H 在 $\kappa=100$、$n=20$ 的二次型上，用同一个梯度下降分别套三种停止准则，看它们各停在哪一步、离最优点多远。

表 29-16 三种停止准则的停止位置（$\kappa=100$，$n=20$，最优步长）

判据	阈值	停止步	停止时距最优点的距离
梯度范数	$\|\mathbf{g}\|\le10^{-6}\|\mathbf{g}_0\|$	425	1.057×10^{-4}
函数下降量	$\|f_k-f_{k-1}\|\le10^{-10}$	447	6.564×10^{-5}
参数变化量	$\|\Delta\mathbf{x}\|\le10^{-8}(1+\|\mathbf{x}\|)$	686	3.720×10^{-7}

表 29-16 里，三个步数差 1.6 倍，但停止时离最优点的距离差了 283 倍。**参数判据最晚停，却是唯一停在“参数真的不动了”的判据**；梯度判据最早停，代价是留下 10^{-4} 的参数误差；函数判据夹在中间，位置靠运气。

函数判据的“运气”值得说清。目标值在谷底附近的量级是 -254，$\tau_f=10^{-10}$ 换算成相对变化是 4×10^{-13}，这已经贴着双精度能分辨的极限——再小的下降在浮点里根本表示不出来。**它之所以停在第 447 步而不是第 4000 步，是因为它先被舍入噪声骗了**，不是因为函数真的不降了。

三个数字合起来给出一条实用规则：**梯度判据当“方向对不对”的监控，参数判据当“停不停”的开关**，两者都用，谁先满足谁停；函数判据只用来做日志，不用来做判断。这条规则在换热器算例上有额外的分量——那里的目标值是 400 个残差的和，单个参数动一点被平均掉，函数判据尤其迟钝。

阈值本身怎么定，也值得给一条可操作的路径。本章取 $\tau_g=10^{-6}$、$\tau_f=10^{-10}$、$\tau_x=10^{-8}$，三个数都是常见默认值，**没有针对算例调过**——刻意不调，是为了让实验 H 的对比反映判据本身的差别，而不是调参的差别。工程上的定法应该反过来：先从工艺要求反推出“参数误差容许多大”，那是 τ_x；再由 τ_x 乘上曲率的量级得到 τ_g；最后按二次型上“间隙与误差平方同阶”的关系，取 τ_f 为 τ_g 的平方量级。**三个阈值不是三个独立的数，它们之间有一个由曲面形状决定的换算关系**，分别拍定必然互相矛盾。

这也解释了为什么很多实现里“三个阈值并存”最后变成三个都不管用：不是阈值取得不对，而是它们彼此不自洽——梯度判据先满足、参数判据永远不满足，停下来的那个点离最优还有一段距离，而日志里“收敛”两个字看上去毫无异常。

29.7 小结与常见错误

这一章从一行没有兑现的注解出发，走了七个节：动机、定义、推导、实现、边界、实验、收束。收束要做的只有两件事——把散在各节的结论收成可以带走的几条，以及把最容易踩的坑列清楚。

29.7.1 七条要点

第一，所有算法都是同一个二次模型的变体。 式 (29-2) 的 $f(\mathbf{x}+\mathbf{d})\approx f+\mathbf{g}^{\top}\mathbf{d}+\tfrac12\mathbf{d}^{\top}\mathbf{H}\mathbf{d}$ 是全部内容的起点。梯度下降假设 $\mathbf{H}=\frac{1}{\eta}\mathbf{I}$，牛顿用真实的 $\mathbf{H}$，拟牛顿用一个递推出来的 $\mathbf{B}$，共轭梯度用前一步的方向拼出一个共轭方向。**换算法就是换假设，没有别的东西。**

第二，收敛速度分三档，档位由用了多少曲率信息决定。 只用梯度是线性收敛，因子是 $(\kappa-1)/(\kappa+1)$；共轭梯度爬到 $\sqrt{\kappa}$ 量级；用上曲率就是超线性甚至二次。表 29-9 的五个斜率 0.83、0.58、0.12、0.01、0.00 就是这三档在实测里的样子。

第三，条件数决定一切，而且在进入优化器之前就该处理。 表 29-6 里梯度下降从 2 步涨到 18236 步，只因为条件数从 1 变到 10^4。**对付条件数最便宜的手段不是换算法，是换参数化**：式 (29-20) 的对数变换把 BFGS 从 27 次降到 8 次，比换到高斯—牛顿还省事。

第四，步长上界是硬边界，位置是 $2/\lambda_{\max}$。 表 29-14 里 1.99 与 2.01 只差百分之一，一边收敛一边爆炸。**这个数不能靠调参逼近，只能靠线搜索绕开**；回溯的代价很低，本章实测平均 0.08 到 0.58 次，比猜错一次的代价小几个数量级。

第五，动量的好坏取决于条件数，不是一个万能常数。 式 (29-26) 给出最优系数，$\kappa=10$ 时是 0.27、$\kappa=10^4$ 时是 0.96。$\beta=0.9$ **是 $\kappa\approx10^3$ 的专用值**，照搬到条件数几十的辨识问题上，会比不用动量还慢 4.7 倍。

第六，规模的分水岭在内存。 式 (29-25) 的 $8n^2$ 与 $16mn$ 决定了 $n=20000$ 时是 3.2 GB 与 3.2 MB 的差别。**但这笔交易要付方向质量的代价**：香蕉函数上 L-BFGS 比 BFGS 多走 18 倍步数。选型的依据是“装不装得下”与“曲面弯不弯”，不是“哪个更新”。

第七，残差到了噪声底就该停，继续迭代是过拟合。 换热器算例的 0.5235 K 与噪声 0.5 K 只差 5%，剩下的 0.16 K 是参数估计误差，多走一万步也消不掉。**“收敛”与“估得准”是两件事**，前者由算法决定，后者由数据决定。

29.7.2 常见错误

上面七条是从正面说的，反过来就是七种典型错误。表 29-17 把它们与本章的实测证据对齐。

表 29-17 七种典型错误及其后果

错误做法	后果	正确做法	本章证据
用迭代次数判断算法快慢	选了单步贵的算法	乘上单步代价再比	表 29-8 牛顿 2 步却不一定快
学习率凭经验给一个数	换数据就发散	线搜索，或幂迭代估 $\lambda_{\max}$	表 29-14 的悬崖
动量一律取 0.9	小条件数上更慢	先估 κ，再按式 (29-26) 取	表 29-6 的 $\kappa = 10$ 一行
高斯—牛顿不加守卫	一步跳进溢出区	Armijo 回溯或 LM 阻尼	§29.4.3 的三处细节
线搜索判据不查有限性	接受 $f = \mathrm{nan}$ 的步长	显式判 `isfinite`	换热器算例的排查过程
只看残差判断收敛	参数错了 5.6% 还认为成功	同时看参数变化与条件数	表 29-12 最后一行
用差分梯度做训练	每步 $2n$ 次函数求值	反传；差分只做校验	表 29-15 的代价

表 29-17 里第五行值得单独说两句，因为它是本章唯一一个“代码写对了、数学推导也对、但结果错了”的例子。Armijo 判据的标准写法是“函数值不超过切线”，当函数值是 nan 时，任何与 nan 的比较都返回假，于是“不超过”这个条件被判成“满足”，算法愉快地接受了一个把参数送进 nan 的步长。**这个错误的隐蔽之处在于它不报错、不崩溃、不产生警告**，只是让 LM 与 BFGS 停在初值上不动，残差 17.4339 K 看起来还是个“合理”的数。发现它的唯一办法是**检查参数有没有被推动**，而不是检查残差大不大。

第四行（高斯—牛顿不加守卫）有一个更麻烦的变体：**它有时真的不出事**。表 29-13 换了四个起点，无保护的高斯—牛顿全部收敛，a 从 0.20 到 6.00、T_{hot} 从 40 到 150 ℃，跨度足够大。看到这张表，最容易做的事是把守卫删掉——既然四个起点都行，何必多写那几行。**但四个起点收敛说明的是这个算例良态，不是算法安全**：它只有一个吸引域，任何起点都在同一个盆里。守卫的作用不是改善平均表现，是在“盆的边界上”救一次命，而边界只有在更难的算例上才会遇到。**这一类“证据看起来很充分”的判断，是工程代码里最常见的退化路径。**

29.7.3 与相邻各章的接口

本章在全书里的位置，可以按“梯度从哪来、用到哪去”串起来。

向上游，第27章给了梯度。 反向传播算的是 $\partial L/\partial \mathbf{w}$，本章的算法把这个向量变成位移。两章的分工很清楚：第27章管“算得准不准”，本章管“走得对不对”。**如果梯度本身错了，本章所有算法都会忠实地走到错的地方**，而且残差可能还很小——这一点在 §29.5.7 的排查里已经演过一次。

向左，第28章的训练是本章的一个应用。 学习率 0.30 为什么发散，答案在式 (29-4)；批大小为什么影响稳定性，答案在“随机梯度等效于放大曲率”；动量为什么有用，答案在式 (29-26)。**第28章讲的是网络结构，本章讲的是同一个损失面上的行走方式**，两章合起来才是完整的训练。

向左更远，第15到22章的辨识是本章的另一个应用。 那里的一次性最小二乘对应本章的 $\mathbf{H}$ 已知、$\mathbf{g}$ 线性情形；一旦模型里有指数、有饱和，就落到本章的非线性最小二乘。**区别只在“要不要迭代”，数学是同一套**：式 (29-7) 的 $\mathbf{J}^{\top}\mathbf{J}$ 就是第16章正规方程里的那个矩阵，在本章它叫高斯—牛顿的海森近似。

向下游，第44到47章的 PID 与前馈不直接调用本章的算法，但整定问题常常化为优化问题。 把“超调小于 10%、调节时间最短”写成目标函数，整定就变成一个二维无约束优化，本章的方法直接可用。**第58与80章的模型预测控制则明确划在界外**：那里有约束，本章的所有推导都建立在“可以往任何方向走”这个前提上，一旦有边界，KKT 条件取代了 $\mathbf{g} = 0$。

29.7.4 上线前的检查清单

把本章的内容压成一份可以照着做的清单，一共八条，按顺序执行。

第一条，先确认梯度对不对。 用中心差分在随机几个点上核对解析梯度，相对误差应在 10^{-5} 以内（表 29-15）。**这一步花十分钟，能省掉后面几天的排查。**

第二条，把参数换成“好走”的坐标。 量级差十倍以上的参数取对数，有正性要求的取指数，有上下界的用 logistic。**换完之后重算一次雅可比的条件数**，如果还在一万以上，说明问题本身信息不足，不是坐标的问题。

第三条，能算海森就用高斯—牛顿或 LM，不能算就用 L-BFGS。 残差平方和问题一律优先高斯—牛顿，它的 $\mathbf{J}^{\top}\mathbf{J}$ 是免费的近似海森；通用目标函数按维度分：几百以内用 BFGS，过千用 L-BFGS（表 29-5）。

第四条，永远带线搜索。 不管方向来自哪里，步长都让 Armijo 判据决定，初值取 1，回溯上限取 60 次。**判据里必须显式检查有限性**，这一条已经用一整节的篇幅说明过。

第五条，同时监控三个量。 梯度范数、参数变化量、残差。前两个用来决定停不停（表 29-16），第三个用来判断“是不是到了噪声底”（§29.5.5）。**只看一个都会出问题**：只看梯度会在平坦区早停，只看残差会把过拟合当成收敛。

第六条，把残差画出来。 散点图看有没有系统性弯曲，直方图看有没有偏斜（图 29-5）。**残差有形状说明模型错了，此时调优化器是白费力气**，这是表 29-7 后四行的共同结论。

第七条，记下每次运行的迭代数、残差、耗时。 三个数放在一起才能判断“这次改动是好是坏”：迭代数降了但耗时涨了，可能是换了更贵的算法；残差降了但参数跑了，可能是过拟合。

第八条，对结论设一个门槛。 本章的门槛是 $0.05\ \mathrm{K}$：小于它的残差差异不当作结论。**这个门槛必须事先定，不能事后按结果调整**——否则任何两个算法都能被说成“有区别”。

八条里没有一条是“选一个更好的算法”。**本章反复出现的结论是：算法之间的差距，往往小于参数化、守卫、停止准则这三件事带来的差距。**换热器算例上，换算法只把 7 次变成 5 次，换参数化把 27 次变成 8 次，加守卫把一个发散的实现变成一个能用的实现——**三个数量级的不同后果，来自三行不起眼的代码。**

如果时间只够做三条，挑第一条、第四条、第五条。**第一条（核对梯度）能排除整类“算法无辜”的故障**，第四条（带线搜索）能保证任何一步都不把参数送进溢出区，第五条（三个量一起看）能防止“日志说收敛、参数还没动”这种最难查的假象。第二、三条与具体问题有关，换热器这种参数量级不匹配的问题必须做，纯二次型的教学算例可以省。第六到八条是纪律，不是技术：**残差要画、日志要留、门槛要事先定**，三条都做不到的项目，前五条做对了也说不清结论是否可信。

最后回到开头那行注解。第27、28 两章的 0.30 为什么不能用，现在的答案是：它超过了 $2/\lambda_{\max}$，而 $\lambda_{\max}$ 由数据与网络共同决定，不是一个可以先验写下的常数。第28章为什么用 8×10^{-3} 而不用 0.3，答案也一样：**学习率不是“调出来的”，是“估出来或者搜出来的”**。这一章没有给出任何可以直接抄的学习率，给出的是一条能自己算出学习率的路。

第30章 约束优化与凸优化

30.1 动机与要解决的问题

30.1.1 一个装不下的控制量

考虑一个行程 1 m 的直线伺服轴。把它简化成双积分对象，采样周期 $T_s = 0.1$ s：

$$x_{k+1} = \begin{bmatrix} 1 & T_s \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} T_s^2/2 \\ T_s \end{bmatrix} u_k \tag{30-1}$$

其中 $x = [\,y \;\; v\,]^{\mathsf{T}}$ 是位置与速度，u 是加速度指令。设计者按线性二次型最优控制（第 55 章）选权重 $Q = \mathrm{diag}(10, 1)$、$R = 0.1$，得到状态反馈 $u = -K(x - x_{\mathrm{ref}})$。这套设计在数学上无懈可击：闭环极点稳定，代价函数取到最小。

问题出在物理侧。加速度受电机转矩与结构强度限制，实际只能给到 $|u| \le 0.4\ \mathrm{m/s^2}$。把限幅硬加在 LQR 输出后面，闭环仍然稳定，但性能变了：位置超调 24.06%，2% 调节时间 5.00 s。而如果从设计阶段就把 $|u| \le 0.4$ 作为约束写进优化问题，用同样的 Q、R、同样的预测时域 $N = 20$，超调降到 2.42%，调节时间 3.50 s，整段轨迹的累计二次代价从 138.59 降到 130.87——下降了 5.6%。两条响应曲线画在图 30-1 的左图，控制量在右图。

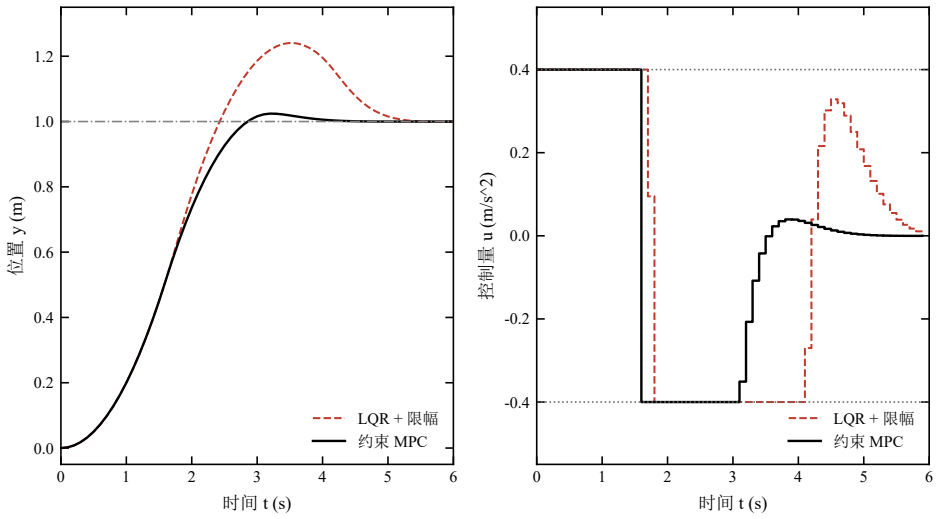


图 30-1　约束 MPC 与限幅 LQR 的对比

右图说明了差别从哪里来。两条控制量曲线都顶在 ± 0.4 上，幅值完全一样，区别在**什么时候退出饱和**。限幅 LQR 是“先冲、撞墙、再纠”，它在饱和期间仍然按无约束律计算反馈，相当于对已经饱和的执行器继续“要”更大的力；约束 MPC 在每一拍都把“接下来 20 步的限幅”一起考虑，提前减速。前者是事后裁剪，后者是事前规划。

这个例子给出本章的第一个判断：**约束不是给无约束解加一个“封顶”，而是改变了整个问题的解。** 把解算出来再裁剪，得到的是一个可行但非最优的点；把约束放进问题里解，得到的是约束下的最优解。两者的差距在温和约束下很小，在约束紧的时候可以到一个数量级。

第二个判断藏在数字里：超调从 24.06% 降到 2.42%，代价函数只降了 5.6%。原因是二次代价对大误差不敏感——超调那一小段在整段轨迹里占比不高。这说明**用代价函数的小幅改善换时域指标的显著改善是划算的**，但也说明不能只看代价函数的第四位小数来判断设计好坏。

有人会问：既然问题出在饱和，用抗饱和（anti-windup）补偿不就行了吗？不行，两者解决的不是同一个问题。抗饱和处理的是**积分器**在饱和期间的累积，它让控制器“不要记住那些没能执行的指令”；而这里的 LQR 没有积分器，问题也不是累积，而是**制动时机**。抗饱和能让退饱和更平滑，但它无法让控制器提前知道“再加速就会冲过头”。换句话说，抗饱和是对已发生的饱和做补救，约束优化是对将要发生的饱和做规划。**前者修正历史，后者预判未来。**

30.1.2 约束从哪里来

控制工程里的约束有四类来源，处理方式完全不同。

物理约束来自执行器和对象本身：电机转矩上限、阀门开度 0–100%、液压缸行程、温度上限。这类约束在任何工况下都不能违反，违反就是设备损坏。它们的数学形式通常是箱约束 $u_{\min} \le u \le u_{\max}$，处理起来最便宜。

工艺约束来自生产要求：升温速率不得超过某值以免材料开裂、进料配比必须保持在反应区间、两个执行器不能同时大幅动作。这类约束是"软"的——偶尔违反一次不会立刻出事，但会影响质量。它们适合用软约束处理，代价是引入松弛变量。

安全约束来自人身与设备保护：联锁、超速保护、紧急停车。这类约束**不应该交给优化器**，因为优化器可能返回不可行或失败，安全功能必须独立于优化层存在（第 36 章功能安全）。把安全约束写进 QP 是常见的设计错误。

性能约束来自设计指标：相位裕度不低于 45°、灵敏度峰值不超过 2、超调不超过 10%。这类约束不出现在运行时的优化里，而是出现在**设计阶段的参数整定**中，作为优化问题的约束条件。§30.6 的最后一个实验就是这种用法。

还有一类约束容易被漏掉，值得单独说：**速率约束**。执行器不只是"最多能给多少"，还有"多快能变过去"——阀门的行程时间、电机的电流上升率、加热器的升温速率。数学形式是 $|\Delta u_k| = |u_k - u_{k-1}| \le \Delta u_{\max}$，它把相邻两拍的决策变量耦合在一起。漏掉速率约束的后果很典型：优化器会算出一步跳变的理想输入，执行器跟不上，实际轨迹与预测轨迹逐渐分离，模型越准反而偏得越多。加上速率约束后，u 的可行域不再是逐点的箱约束，而是一条"斜坡带状"的耦合区域，问题的规模与条件数都会变化。

约束的数量也值得提前有个数量级概念。一个 N 步、状态 n 维、输入 m 维的 MPC，若状态与输入都加箱约束，不等式约束就有 $2N(n+m)$ 条：$N=20$、$n=4$、$m=2$ 时是 240 条。这些约束并不会让求解时间线性膨胀——大多数在最优解处不积极——但它们决定了矩阵的稀疏结构与内存占用，这一点在 §30.4.2 会具体看到。

这四类约束的区别在于"违反了谁承担后果"。物理约束违反是硬件损坏，安全约束违反是事故，工艺约束违反是质量偏差，性能约束违反是验收不合格。承担后果的主体不同，就应该用不同的机制，不能全部塞进一个 QP。

30.1.3 四个先导反例

反例一：把约束当裁剪，得到的是可行解而不是最优解。 在 §30.1.1 的例子里，限幅 LQR 的每一步都满足 $|u| \le 0.4$，所以它是可行的，但它不是最优的。用累计代价衡量，它比约束 MPC 高 5.6%；用超调衡量，它高 21.6 个百分点。可行性与最优性是两个独立的性质，工程上经常只检查前者。

反例二：罚因子取得不够大，解停在可行域外面。 用二次罚函数处理 $x_1 + x_2 \le 2.5$，罚因子 $\rho = 1$ 时约束违反 0.333，$\rho = 10$ 时 0.0476，$\rho = 100$ 时 0.00498。违反量按 $1/\rho$ 下降，要压到 10^{-6} 需要 $\rho = 10^5$。而 ρ 越大，罚函数的 Hessian 条件数越差，内层无约束优化的收敛越慢。这就是外点法的固有矛盾：**精度靠大罚因子，数值稳定性要求小罚因子。**

反例三：硬约束遇到不可行输入，求解器直接罢工。 给 §30.1.1 的对象加终端约束 $x_N = x_{\text{ref}}$，同时把限幅压到 $|u| \le 0.3$。10 步内最大位移只有 $0.5 T_s^2 u_{\max} N^2 = 0.150$ m，从原点出发根本到不了 1 m。硬约束 QP 返回 `infeasible`，控制器这一拍没有输出。改成软约束后，求解器返回 `optimal`，松弛量 0.85，即"尽量靠近、差 0.85 m"。**不可行不是数值问题，是建模问题：必须事先规定不可行时怎么办。**

反例四：把非凸问题当凸问题解，拿到的是局部最优。 设输入需要避开某个共振转速区间，可行集是 $u \in [0, 0.2] \cup [0.5, 1]$——两个不相连的区间。这个集合不是凸的，从 $u = 0.6$ 出发的梯度法会停在 0.5 附近，从 $u = 0.1$ 出发会停在 0.2 附近，两个解的代价可能相差数倍。更隐蔽的情形是双线性项：目标里出现 $u_1 u_2$（例如功率等于电压乘电流），Hessian 不定，凸优化的一切保证全部失效。**判断凸性不是学术洁癖，它决定了"求出的解能不能信"。**

30.1.4 五个必须回答的定量问题

把上面的场景抽象出来，工程上要回答五个问题，本章逐个给工具。

1. **约束是否起作用？** 用 KKT 乘子判断：乘子为零的约束没有起作用，为正的约束正在"顶住"解。见 §30.3.2。
2. **放松约束能换来多少收益？** 用影子价格 $\lambda = -\partial f^\star / \partial b$ 定量。见 §30.3.4。
3. **问题是不是凸的？** 凸问题的局部最优就是全局最优，非凸问题只能保证局部。见 §30.2.3。
4. **算得过来吗？** QP 的求解时间随规模如何增长，热启动能省多少。见 §30.4.4。
5. **不可行时怎么办？** 软约束、优先级、回退三选一，必须事先写进代码。见 §30.5.4。

这五个问题的顺序不能颠倒。第 3 问决定后面所有结论是否成立：如果问题非凸，第 1 问的 KKT 条件只是必要条件，第 2 问的影子价格只是局部信息。第 4 问决定能不能上线：一个 5 ms 的 QP 放进 10 ms 的采样周期是可行的，放进 1 ms 的周期就必须换显式 MPC（第 58 章）。

这五个问题看起来要分五次回答，实际上可以由一次求解同时给出。求解器返回的不只是 $x^\star$：乘子回答了第 1、2 问，求解状态与残差回答了第 3 问的"解是否可信"，迭代次数与耗时回答第 4 问，状态码回答第 5 问。**工程上真正缺的不是算法，而是把这些附带信息读出来并记录下来的习惯。** 只取 u_0、把其余全部丢弃的写法，等于把仪表盘拆了只留油门。

这也解释了为什么本章要花篇幅讲乘子与残差，而不是直接给一段 `cvxpy.solve()` 的代码。会调库的人很多，能从返回值里诊断问题的人很少，而后者才是把优化用稳的关键。

30.1.5 一句话核心与本章路线

如果只能记住一句话：**约束优化的价值不在于“加个上限”，而在于把边界写进问题、用乘子读出边界的价格、用凸性保证解的可信度。**

本章按“定义—推导—离散—边界—验证”推进。§30.2 给出标准形式、凸集与凸函数的判据、LP/QP/SOCP/SDP 的分层；§30.3 推导拉格朗日乘子、KKT 条件、对偶与影子价格，再讲罚函数、障碍函数、SQP 三条求解路线；§30.4 把连续问题落到离散 QP，给出 MPC 的标准构造、求解器选择与计算量实测；§30.5 给出工程边界：什么时候不该用优化、实时性怎么保证、不可行怎么兜底；§30.6 用七组实验逐条验证前面的结论；§30.7 收束为一份错误清单。

不同读者的路径：做产线调试的，读 §30.1、§30.3.4、§30.5；做 MPC 实现的，读 §30.2、§30.3.5、§30.4 全部；做控制器参数整定的，读 §30.3.2、§30.6.7，之后接第 44 章。三条路径的共同落点是 §30.6 的脚本，不跑一遍，乘子和影子价格都只是符号。

30.2 定义与数学基础

30.2.1 标准形式与几何图像

本章讨论的问题统一写成

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x)\le 0, \quad i=1,\dots,m \\ & h_j(x)=0, \quad j=1,\dots,p \end{aligned} \tag{30-2}$$

其中 f 是目标，g_i 是不等式约束，h_j 是等式约束。满足全部约束的点构成**可行域** $\mathcal{F}$；若 $\mathcal{F}$ 非空，称问题可行。$x^\star$ 是全局最优解，指对任意 $x\in\mathcal{F}$ 都有 $f(x^\star)\le f(x)$；若只在其某个邻域内成立，则称局部最优解。

等式约束在控制问题里通常来自模型：状态方程 $x_{k+1}=Ax_k+Bu_k$ 经过消元后变成变量之间的线性关系，可以完全消去，也可以保留为等式约束。保留的好处是保持问题的稀疏结构，坏处是变量变多。工程实现几乎总是保留——稀疏性对求解速度的贡献远大于变量维数带来的负担。

积极约束是理解约束优化的钥匙。若 $g_i(x^\star)=0$，称该约束在最优点**积极**（active）；若 $g_i(x^\star)<0$，称其**不积极**（inactive）。不积极约束对解没有影响，把它删掉解不变；积极约束决定了最优解的位置。图 30-2 画出了本章反复使用的二维例子：

$$\min_{x}\ (x_1-2)^2+(x_2-1.5)^2 \quad \text{s.t.} \quad x_1+x_2\le 2.5,\ x_1\ge 0,\ x_2\ge 0 \tag{30-3}$$

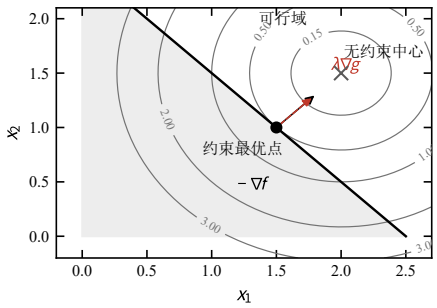


图 30-2　约束优化的几何：等高线、可行域与最优点

无约束最优点是 $(2,1.5)$，落在可行域外。约束最优点沿目标函数梯度方向投影到边界上，为 $(1.5,1)$，此时 $x_1+x_2=2.5$ 积极，两个非负约束不积极。最优值 $f^\star=0.5$。在最优点的两条箭头分别是负梯度 $-\nabla f$ 与约束外法向 ∇g，两者方向相反、共线——这就是 KKT 条件的几何原型，§30.3.2 会把它写成公式。

这个例子还说明一件事：**约束最优点不一定在可行域的“角”上。** 角点（顶点）是线性规划的特征，二次目标的最优点通常落在边界的光滑段内。把“最优解一定在顶点”的经验从线性规划搬到二次规划，是常见的错误。

可行域的形状决定了很多后续性质。线性约束的可行域是**多面体**：有限个半空间的交，边界由平面片拼成，顶点数是有限个。MPC 的可行域正是多面体，因为状态方程经过消元后，全部约束都变成 U 的线性不等式。多面体的好处是可以用顶点枚举、有效集法这类组合式

算法；坏处是顶点数随维数指数增长，n 个变量、m 条约束的顶点数上界是 $\binom{m}{n}$，所以高维时不能靠枚举。

非线性约束的可行域是曲面围成的区域，边界光滑但没有“顶点”概念。此时“投影”的几何意义更清楚：最优点是从无约束最优点沿某个度量向可行域投影的落点。度量由目标函数的 Hessian 决定——二次目标的等高线是椭圆，投影方向不是欧氏最短方向，而是沿椭圆法向的方向。这解释了为什么加权矩阵 Q 改变的不只是“重视哪个状态”，还改变了“往哪个方向退”。

对于带等式约束的问题，几何图像还要再降一维：可行域是仿射子空间与半空间的交，最优性条件变成“负梯度落在等式约束张成的子空间与积极不等式法锥之和里”。这句话就是 KKT 条件的几何版，§30.3.2 会把它写成代数式。

30.2.2 凸集与凸函数

凸集：若对任意 $x, y \in \mathcal{C}$ 与 $\theta \in [0,1]$，都有 $\theta x + (1-\theta) y \in \mathcal{C}$，则 $\mathcal{C}$ 是凸集。几何上就是“集合内任意两点的连线不跑到集合外”。半空间 $\{x \mid a^{\mathsf{T}} x \le b\}$、多面体、球、椭球、正半定锥都是凸集；整数格点集、并集、圆环不是。

凸函数：定义在凸集上的 f 若满足

$$f(\theta x + (1-\theta) y) \le \theta f(x) + (1-\theta) f(y) \tag{30-4}$$

则称 f 凸。这个定义不好用，实际判据是两条：

一阶判据（可微时）：$f(y) \ge f(x) + \nabla f(x)^{\mathsf{T}} (y - x)$，即函数始终在其任一点的切平面上方。

二阶判据（二阶可微时）：$\nabla^2 f(x) \succeq 0$，Hessian 半正定。

二次函数 $f(x) = \tfrac{1}{2} x^{\mathsf{T}} P x + q^{\mathsf{T}} x + r$ 凸的充要条件是 $P \succeq 0$。这一条在控制里极其常用：MPC 的代价函数是二次型，权重矩阵半正定，问题自动是凸的。反过来，只要权重矩阵出现负特征值（比如为了让某个方向“越快越好”而设了负权重），问题立刻非凸，求解器给出的解不能信。

保凸运算决定了工程上能拼装出多复杂的问题。非负加权和保凸；逐点最大值保凸；复合 $f(Ax+b)$ 保凸；凸函数的非降凸复合保凸。这些规则可以组合，于是“分段二次代价”“最大偏差”“多目标加权”都能保持凸性。但**凸函数之差不保凸**——这就是为什么“收益减去风险”型的目标经常非凸。

集合侧也有对应的运算规则，而且更常被用到。**任意多个凸集的交仍是凸集**，所以把一条条约束叠起来不会破坏凸性；**凸集的仿射像与原像仍是凸集**，所以变量替换 $z = Tx$ 是安全的；**凸集的并通常不是凸集**，这正是反例四中“绕开共振区”的问题所在——两个不相连的区间，各自是凸的，并起来就不是。工程上的补救办法是把并集松弛成它的凸包（把中间那段也允许），或者在离散变量上做枚举。

还有一条实践中的判据常被忽略：**凸性要在“消元前”和“消元后”都检查一遍**。有些问题在原变量下非凸，但消去一部分变量后变凸；反过来也有——消元时若引入了非凸关系（例如把 u^2 代换成一个新变量而忘记加锥约束），表面上成了线性问题，实际解出来的点不在原来的集合里。后者是 MPC 实现里最隐蔽的一类错误，表现为“求解器说最优，仿真却偏离模型”。

30.2.3 凸性决定解的可信度

凸优化的核心定理只有一句：**凸问题的任一局部最优解都是全局最优解**。证明用反证：若局部最优 $x^\star$ 不是全局最优，存在 y 使 $f(y) < f(x^\star)$，沿线段取靠近 $x^\star$ 的点，凸性给出该点目标值严格小于 $f(x^\star)$，与局部最优矛盾。

这条定理的工程含义是：求解器返回 `optimal` 时，你拿到的就是全局最好，不需要换初值重试，不需要担心“掉进局部坑”。对非凸问题，同样的 `optimal` 只说明“在这个初值附近收敛到了一个驻点”，换一个初值可能得到更好的解。

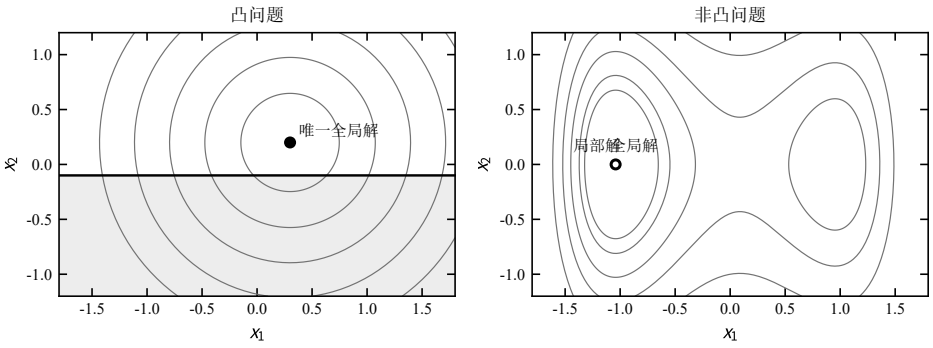


图 30-3　凸问题与非凸问题的解结构对比

图 30-3 的右图是一个非凸例子：$f(x_1, x_2) = (x_1^2 - 1)^2 + 0.35 x_1 + x_2^2$。两个局部极小点分居 $x_1 = \pm 1$ 附近，左侧的目标值更低。从右侧初值出发的梯度法会停在局部解上，而且它满足 KKT 条件——**KKT 条件不能区分局部与全局**。这就是为什么“求解器说收敛了”不等

于“答案对”。

控制问题的凸性需要逐项检查，常见的三种非凸来源是：一是**动力学含非线性**（如 $\dot{x} = f(x,u)$ 中的三角函数），二是**约束含耦合**（如 $u_1 u_2 \le c$），三是**目标含稀疏诱导项的组合**（如 ℓ_0 范数）。前两种的正规做法是线性化或凸松弛（第 58、80 章），第三种用 ℓ_1 松弛替代（第 31 章）。如果三者都不成立，就必须接受“局部最优”，并用多初值、热启动、后验校验来降低风险。

检查可以按固定顺序走，三步就能定性。

第一步，看变量域。 变量是否被限制在整数、离散档位、两个不相连的区间上？若是，问题不是凸的，除非能证明松弛后最优解恰好落在原集合内（这在全幺模结构下偶尔成立，不能指望）。

第二步，看目标。 把目标写成标准二次型，取出 Hessian，检查最小特征值。工程代码里更省事的做法是跑一次 Cholesky 分解：能分解则半正定，报“矩阵非正定”就说明有负特征值。注意权重矩阵的条件数：最小特征值 10^{-6}、最大 10^{8} 时，数值上等价于奇异，求解器可能报 `optimal_inaccurate`，此时应先缩放再判断凸性。

第三步，看约束。 每条约束单独看是否凸（$\le$ 号左边是凸函数，等式约束左边是仿射），然后看它们是否只通过“交”组合。若出现“或”、若出现两个变量的乘积、若出现范数在 $\ge$ 号左边，就要停下来。

三步全过，才可以放心使用凸优化的一切结论；任何一步不过，都要在文档里写明“局部最优”并按非凸处理。这个检查花不了几分钟，但能避免后面几个月的返工。

30.2.4 问题分层：从 LP 到 SDP

同样是“凸”，不同问题类的求解代价差好几个数量级。识别问题属于哪一层，直接决定选什么求解器、能不能满足实时性。

表 30-1　凸优化的问题分层与典型求解器

类型	目标	约束	典型规模	常用求解器
LP	线性	线性	10^5 变量	HiGHS、Gurobi、GLPK
QP	二次（$P \succeq 0$）	线性	10^2–10^4	OSQP、qpOASES、DAQP
QCQP	二次	二次（凸）	10^2	ECOS、Clarabel
SOCP	线性	二阶锥	10^3	ECOS、Mosek、Clarabel
SDP	线性	半正定锥	10^2	Mosek、SCS、CVXOPT

这张表要横着读：越往下，能表达的东西越多，单次求解越贵。控制里的多数在线问题落在 QP 这一行——状态方程线性、代价二次、约束是箱约束或线性不等式，正好是 QP 的形状。只有在处理鲁棒约束、机会约束、矩阵不等式时才需要上升到 SOCP 或 SDP，而那些场合通常离线求解或低频更新。

“越往下越贵”的量级感值得具体化。内点法解 LP、QP、SOCP 的迭代次数通常在 20–50 次之间，与问题规模关系不大；单次迭代的代价是 $O(n^3)$ 量级（稠密矩阵分解），稀疏结构好时可以降到近线性。SDP 的单次迭代代价是 $O(n^6)$，因为每个 $n \times n$ 矩阵变量在锥投影时要算特征分解。把这两条放在一起：一个 $n = 100$ 的 QP，内点法约 $100^3 \times 30 \approx 3 \times 10^7$ 次浮点运算，毫秒级；同样规模的 SDP，100^6 已经不可接受。**规模上限的差别不是常数因子，而是幂次。**

分层的边界也不是绝对的。QCQP 可以改写成 SOCP（把二次约束写成范数约束），从而用更成熟的锥求解器；QP 可以看成 SOCP 的特例，但没必要绕这一圈。反过来，某些看似 SOCP 的问题（例如带椭球不确定集的鲁棒约束）经过对偶化简后可以变回 QP，这一步化简往往能把求解时间降一个数量级。**先化简、再定层、最后选求解器**，顺序不能颠倒。

LP 与 QP 的求解器内部还有两条路线，值得知道区别。**单纯形法**沿着可行域的顶点走，每步换一个顶点，理论最坏是指数时间，实测很快，且天然支持热启动与灵敏度分析；**内点法**从可行域内部穿过去，多项式时间，但每步要分解矩阵。工程上 LP 多用单纯形（HiGHS 的默认策略是两者混合），QP 多用内点或一阶方法。选型时若需要“改一个参数后快速重解”，单纯形与有效集法的增量更新能力比内点法更合适。

二次规划（QP） 是本章的主角，标准形式为

$$\min_{z} \ \tfrac{1}{2} z^{\mathsf{T}} H z + g^{\mathsf{T}} z \quad \text{s.t.} \quad A z \le b, \quad A_{\text{eq}} z = b_{\text{eq}} \tag{30-5}$$

其中 $H \succeq 0$。若 $H \succ 0$，问题有唯一解；若 H 奇异，解可能不唯一，但最优值仍唯一。工程上建议加一个很小的正则项 ϵI（ϵ 取 10^{-8} 量级）来消除奇异，代价是最优解偏移 ϵ 量级，可以接受。

半正定规划（SDP） 的约束形如 $F_0 + \sum_i z_i F_i \succeq 0$。控制里的 LMI（第 65 章）就是 SDP。它不能直接写成向量形式的不等式，必须靠专门的锥求解器。SDP 的规模通常受限于矩阵维数：$n \times n$ 的矩阵不等式，变量数 $O(n^2)$，单次迭代 $O(n^6)$，所以 n 超过几百就基本离线算。

二阶锥规划（SOCP） 介于两者之间，约束形如 $\|A_i z + b_i\|_2 \le c_i^{\mathsf T} z + d_i$。它常出现在“鲁棒 MPC”里：把不确定集合上的最坏情况约束转化为范数约束。相比 SDP，SOCP 的代价低一个量级，是工程上更现实的选择。

30.2.5 控制里的四个标准问题

其一，MPC 的滚动优化。 给定离散模型 $x_{k+1} = Ax_k + Bu_k$，预测时域 N，把未来 N 步的输入堆成决策变量 $U = [u_0; \dots; u_{N-1}]$，状态可以显式消去：

$$X = \mathcal{A} x_0 + \mathcal{B} U, \quad \mathcal{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \quad \mathcal{B} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A^{N-1}B & \cdots & AB & B \end{bmatrix} \tag{30-6}$$

代价写成 $J = \tfrac12 U^{\mathsf T} H U + g^{\mathsf T} U + \text{常数}$，其中 $H = \mathcal{B}^{\mathsf T} \bar{Q} \mathcal{B} + \bar{R}$，$g = \mathcal{B}^{\mathsf T} \bar{Q} \mathcal{A} x_0$。约束 $|u_k| \le u_{\max}$ 直接作用在 U 上，状态约束 $x_{\min} \le x_k \le x_{\max}$ 变成 $\mathcal{A}x_0 + \mathcal{B}U$ 的线性不等式。于是滚动优化就是一个标准 QP，规模为 $n_u N$ 个变量、约 $(2n_u + 2n_x)N$ 个不等式。§30.4.1 给出完整代码。

这里有两个细节决定 QP 是否“好解”。第一，H 必须正定，而 H 的正定性来自 $\bar{R} \succ 0$——若输入权重取零，H 只在 $\mathcal{B}$ 的像空间上正定，问题可能出现多解，一阶方法会来回振荡。所以 R 不能设为零，哪怕很小（10^{-4} 量级）也要留。第二，H 的条件数由 Q 与 R 的比值决定，位置权重 10、输入权重 0.1，比值 10^2，条件数尚可；若位置权重 10^4、输入权重 10^{-3}，条件数到 10^7，迭代次数会翻几倍。这就是 §30.4.3 要讲的缩放问题。

终端代价 P 的作用不只是“让末端更准”。取 P 为离散 Riccati 方程的解时，滚动优化等价于一个无限时域的 LQR，闭环有稳定性证明；取 $P = 0$ 时，有限时域的 MPC 可能为了末端代价而“透支”，出现短视行为。工程上算 P 只要一行 `scipy.linalg.solve_discrete_are`，代价远小于它带来的稳定性保证。

其二，带鲁棒性约束的参数整定。 求 (K_p, K_i) 使 IAE 最小，同时满足相位裕度 $\ge 45^\circ$、灵敏度峰值 ≤ 2。后两个条件不是对 (K_p, K_i) 的显式代数约束，而是“对每个频率都成立”的半无限约束：

$$\angle L(\mathrm{j}\omega; K_p, K_i) + 180^\circ \ge 45^\circ, \quad \left| \frac{1}{1 + L(\mathrm{j}\omega; K_p, K_i)} \right| \le 2, \quad \forall \omega \tag{30-7}$$

工程做法是在对数频率轴上取 10^3–10^4 个采样点，把半无限约束离散成有限个不等式，再用 SQP 或内点法求解。取点密度决定约束的保守程度：点太稀，两个采样点之间可能违反；点太密，约束数超过变量数几个数量级，求解变慢。经验做法是先粗网格求解，再在最优点附近加密复核。

其三，H∞ 与 LMI。 把“闭环增益小于 γ”写成矩阵不等式，用 Schur 补把它转成线性矩阵不等式：

$$\begin{bmatrix} A^{\mathsf T}P + PA + Q & PB \\ B^{\mathsf T}P & -R \end{bmatrix} \prec 0 \quad \Longleftrightarrow \quad A^{\mathsf T}P + PA + Q + PBR^{-1}B^{\mathsf T}P \prec 0 \tag{30-8}$$

左边对 P 是线性的，于是“求 $P \succ 0$ 使不等式成立”就是 SDP。Schur 补的价值在于：它把带逆矩阵的 Riccati 型不等式变成了线性矩阵不等式，而后者有成熟的锥求解器。

其四，稀疏与分组。 有时目标里带 $\|u\|_1$ 或分组稀疏项，用来挑选执行器或简化控制器。ℓ_1 范数可以写成线性约束：$\|u\|_1 \le t$ 等价于 $-t\mathbf{1} \le u \le t\mathbf{1}$ 加上对 t 的优化，仍然是 LP 或 QP。ℓ_0 范数不行，只能松弛。

30.2.6 对偶与 Slater 条件

约束优化的另一半理论在对偶侧。定义拉格朗日函数

$$L(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x) \tag{30-9}$$

其中 $\lambda_i \ge 0$ 是不等式乘子，ν_j 是等式乘子。对固定的 (λ, ν)，L 对 x 取下确界得到**对偶函数** $d(\lambda, \nu) = \inf_x L(x, \lambda, \nu)$。对偶函数永远是凹的，哪怕原问题非凸——因为它是一族仿射函数的下确界。

对偶问题是

$$\max_{\lambda \ge 0,\,\nu} \; d(\lambda, \nu) \tag{30-10}$$

弱对偶成立：$d(\lambda,\nu) \le f(x^\star)$ 对任意可行 (λ,ν) 与最优 $x^\star$ 都成立。两者之差叫对偶间隙。**强对偶**指间隙为零，此时对偶最优值等于原问题最优值。

强对偶需要条件。对凸问题，**Slater 条件**是充分条件：存在一个严格可行的点，即所有不等式约束严格满足（$g_i(x) < 0$）且等式约束满足。线性约束的问题（LP、QP）只要可行，Slater 条件通常自动成立或可用弱化版本。非凸问题即使满足 Slater 条件也不能保证强对偶——凸性是前提。

弱对偶的证明只有两行，值得亲手写一遍。设 x 可行、$\lambda \ge 0$，则 $\lambda_i g_i(x) \le 0$、$\nu_j h_j(x) = 0$，于是 $L(x,\lambda,\nu) \le f(x)$；又因 $d(\lambda,\nu) = \inf_x L \le L(x,\lambda,\nu)$，合起来就是 $d(\lambda,\nu) \le f(x)$。**这个不等式对任意可行点成立，因此对最优值也成立。** 它给出的不是等式，而是一个可以随时计算的“下界”：任何一组 $\lambda \ge 0$ 都能报出一个不超过最优值的数，而任何可行解都能报出一个不低于最优值的数，两者夹出的区间就是当前解的质量证明。

这个性质的工程价值在停止准则上。一阶方法迭代到第 k 步时，原始残差给出“离可行有多远”，对偶残差给出“下界离当前目标有多远”，两者之和就是间隙的估计。**不需要知道最优值，就能判断该不该继续迭代。** 这也是 OSQP 的停止准则能自适应调整的原因：它把间隙按目标函数的量级归一化，要求相对精度而不是绝对精度。

不可行时，对偶侧给出的东西更有用。**Farkas 引理**说明：原问题不可行，当且仅当存在一组非负乘子使某个线性组合恒为负。求解器返回的这组乘子就是**不可行证书**，它指出哪几条约束互相冲突。证书不是“更详细的错误信息”，而是可验证的数学对象——把它代回约束，能逐条确认冲突确实存在。§30.5.2 推荐 SCS 与 Clarabel，原因就在这里。

互补松弛 $\lambda_i g_i(x^\star) = 0$ 还有一种经济学的读法：约束是“配额”，乘子是“配额的影子价格”，最优时要么配额没用完（$g_i < 0$，价格为零），要么配额用满且价格为正。不会出现“配额没用完却还要付费”的情形，也不该出现“配额用满却免费”的浪费。工程上看到“约束积极但乘子为零”，通常意味着问题退化（例如两条约束重复），需要检查约束是否有冗余。

对偶的工程价值有两层。第一层是**证书**：任何可行 x 与任何 $\lambda \ge 0$ 都给出上下界，两者之差就是“当前解离最优还有多远”，不需要知道最优值。第二层是**价格**：对偶变量就是约束的影子价格，它告诉设计者“把这条约束放松一个单位，目标能改善多少”。§30.3.4 会用实验把这件事量化到 10^{-13} 的精度。

需要提醒的是，对偶间隙为零不代表原问题好解。有些凸问题的对偶间隙为零但原问题的最优点取不到（只在极限意义下达到），工程上表现为求解器迭代到最大步数仍报 `optimal_inaccurate`。这时要看原始残差与对偶残差，而不是只看状态字符串。

对偶还有一层“机制设计”的用途，在能源与过程工业里很常见。多个子系统共享一条容量约束时，把容量约束的对偶变量当作“内部价格”下发给各子系统，各自按价格优化自己的目标，汇总后再更新价格——这就是对偶分解。它的好处是不需要把全部变量塞进一个求解器，通信量小、可并行；代价是收敛需要多轮，且每轮都要解一个子问题。对采样周期在秒级的调度问题，这个代价可以接受；对毫秒级的伺服控制就不合适。

30.3 推导与性质

30.3.1 拉格朗日乘子从哪里来

先看等式约束。求 $f(x)$ 在 $h(x) = 0$ 上的极小，若最优点处 $\nabla h \neq 0$，则 $-\nabla f$ 必须与 ∇h 共线，否则沿约束曲面切向移动还能降低目标。写成公式就是

$$\nabla f(x^\star) + \nu \nabla h(x^\star) = 0 \tag{30-11}$$

ν 就是乘子。它不是“为了凑公式引入的辅助量”，而是**目标梯度在约束法向上的分量大小**。这个解释对不等式也成立：只要约束积极，$-\nabla f$ 必须落在积极约束法向生成的锥内，否则可以沿可行方向继续下降。

用二维例子验证。在 $x^\star = (1.5, 1)$ 处，$\nabla f = 2(x^\star - c) = (-1, -1)$，$\nabla g = (1, 1)$，于是 $\lambda = 1$。乘子为正，说明约束“顶住”了解；若算出负乘子，说明该约束不该积极，解不在这个面上。

乘子还有一层**灵敏度**含义。考虑扰动后的问题 $\min f(x)$ s.t. $g(x) \le b$，最优值记为 $f^\star(b)$。在强对偶与可微的条件下，

$$\frac{\mathrm{d} f^\star}{\mathrm{d} b} = -\lambda^\star \tag{30-12}$$

即乘子等于“放松一单位约束所换来的目标改善”的相反数。这个结论的推导只需对 $L(x^\star, \lambda^\star, b) = f(x^\star) + \lambda^\star (g(x^\star) - b)$ 关于 b 求导，并利用 $g(x^\star) = b$ 与包络定理。它把抽象的乘子变成了可报价的量，§30.3.4 会给出实测验证。

不等式约束的乘子为什么必须非负？用一维例子看得最清楚。$\min x$ s.t. $x \ge 1$，写成标准形式 $g(x) = 1 - x \le 0$。最优点是 $x^\star = 1$，$\nabla f = 1$，$\nabla g = -1$，平稳性给出 $1 - \lambda = 0$，$\lambda = 1 > 0$。若把约束方向写反成 $x \le 1$，同样的推导会给出 $\lambda = -1$，这是不可能的——因为 $x^\star = 0$ 根本不满足约束。**乘子为负，说明解的符号或约束方向至少有一个写错了**，这是调试时最快的自检。

多个约束同时积极时，乘子的含义要从“共线”升级为“落在锥内”：$-\nabla f$ 必须是积极约束梯度的非负组合。几何上，可行方向构成一个锥（临界锥），负梯度必须落在这个锥的对偶锥里。这一步升级解释了一件事：**为什么两个积极约束的乘子不能单独解读。** 它们是一组权重，只有相对大小有意义；把其中一个乘子单独拿出来说“这条约束值 3.2 元”，必须注明其余积极约束是哪些。

乘子还有一个尺度不变性值得记住：若把目标整体乘以 c，所有乘子也乘以 c，最优解不变。所以报乘子时必须同时报目标的量纲与量级。工程上更稳妥的做法是报**归一化乘子**——用目标值的相对变化（例如 $|\Delta f|/f^\star$）来表达放松约束的收益，这样跨工况、跨项目都能比较。

30.3.2 KKT 条件

把等式与不等式放在一起，最优点满足四个条件：

$$
\begin{aligned}
&\text{平稳性：} && \nabla f(x^\star) + \sum_{i} \lambda_i \nabla g_i(x^\star) + \sum_{j} \nu_j \nabla h_j(x^\star) = 0 \\
&\text{原始可行：} && g_i(x^\star) \le 0, \quad h_j(x^\star) = 0 \\
&\text{对偶可行：} && \lambda_i \ge 0 \\
&\text{互补松弛：} && \lambda_i g_i(x^\star) = 0, \quad i = 1, \dots, m
\end{aligned}
\tag{30-13}
$$

互补松弛是这组条件里最有用的一条。它说：λ_i 与 $g_i(x^\star)$ 至少有一个为零。于是每条约束只有两种状态——不积极且乘子为零（对解没有影响），或者积极且乘子非负（正在限制解）。**一个约束不可能既不积极又有正乘子，也不可能积极而乘子为零（退化情形除外）。**

这条性质直接给出了诊断方法：求解完成后读乘子，乘子为 0 的约束可以删掉，乘子最大的约束是当前最“贵”的瓶颈。工程上据此判断“该放宽哪条限制”——放宽乘子最大的那条，收益最大。

用一个三条约束的例子把四种状态走一遍。仍取 §30.2.1 的二维问题，把右端改为 $b = 1$：最优点是 $(0.75, 0.25)$，$x_1 + x_2 = 1$ 积极、$\lambda_1 = 2.5 > 0$；两条非负约束不积极、乘子为零。此时 $f^\star = 3.125$。若进一步把 b 压到 0.5，最优点变成 $(0.5, 0)$，和约束与非负约束 $x_2 \ge 0$ 同时积极，两个乘子分别是 3 与 0——因为 x_2 项在零上但没有任何力想把它推到负值，它只是“跟着和约束走”。**积极但乘子为零**的情形就这样出现了，它不违反互补松弛，但提示问题在这一段进入了退化区间。

KKT 条件成立还需要一个前提，通常写作 LICQ：所有积极约束的梯度线性无关。若两条约束是同一个平面（重复约束），或三个约束的梯度共面，乘子就不唯一，求解器给出的乘子依赖于内部的数值路径，跨平台可能不同。工程上的处理是**上线前先去冗余**：用数值秩检查约束矩阵，把线性相关的行删掉。这一步不做，乘子的解读就会时对时错。

KKT 条件的地位需要说清楚。对**凸问题**，KKT 是充要条件：满足 KKT 的点就是全局最优。对**非凸问题**，KKT 只是必要条件：满足 KKT 的点可能是局部极小、局部极大，也可能是鞍点。所以看到求解器报 `optimal` 且 KKT 残差很小，只能说明“找到了一个驻点”，不能说明“找到了最好的驻点”。

§30.6 的第一个实验用 CVXPY 解二维例子，数值解 $(1.500000, 1.000000)$ 与解析解偏差 1.11×10^{-16}，乘子 1.000000，四项 KKT 残差分别是平稳性 2.22×10^{-16}、原始可行 0、对偶可行 0、互补松弛 0。这组数字说明：对良态的凸 QP，KKT 残差可以做到机器精度，不存在“优化器只能给近似解”的问题——近似只发生在非凸、病态或大规模问题上。

30.3.3 二阶条件与解的唯一性

KKT 是一阶条件，它管“梯度对不对”，不管“曲率够不够”。对积极约束集合 $\mathcal{A}(x^\star)$，定义临界锥

$$
\mathcal{C} = \left\{ d \;\middle|\; \nabla g_i(x^\star)^{\mathsf T} d = 0,\ i \in \mathcal{A};\ \nabla g_i(x^\star)^{\mathsf T} d \le 0,\ i \in \mathcal{A}_{\text{ineq}} \right\}
\tag{30-14}
$$

二阶充分条件是：对任意非零 $d \in \mathcal{C}$，都有 $d^{\mathsf T} \nabla^2_{xx} L(x^\star, \lambda^\star) d > 0$。直观地说，沿着所有“可行且不破坏积极约束”的方向，拉格朗日函数的曲率必须为正。

这条条件在工程上对应一个常见现象：**目标函数在最优点附近太平**。如果二阶条件只是勉强成立（曲率接近零），求解器会在最优点附近反复迭代，返回的解对数据扰动敏感。表现是同一个问题连续解两次，结果在小数点后第三位就开始不同。遇到这种情况，要么加正则项抬高曲率，要么接受这个精度并检查它对下游的影响。

对严格凸的 QP（$H \succ 0$），二阶条件自动成立，解唯一且连续依赖于数据。这就是为什么工程上偏爱 QP：**唯一性保证了“同样的输入给同样的输出”**，这对实时控制是硬要求。若 H 只是半正定（比如只惩罚位置不惩罚速度），解可能不唯一，两次求解给出不同的控制

量，闭环出现抖动。

二阶条件失效有三种典型形态，处理方式各不相同。

形态一：曲率为零但梯度为零。 目标在某个方向上完全平坦（例如只惩罚位置误差、不惩罚速度误差，而速度又不进入任何约束），此时 H 奇异，最优解是一条直线上的任意点。求解器返回哪一个点取决于迭代路径。修法：加一个小的正则项 ϵI，或把被忽略的状态纳入代价。

形态二：曲率为负。 这说明当前点是鞍点而不是极小。非凸问题里常见，凸问题里只可能因为 Hessian 组装错误（例如把 $-Q$ 写成了 Q）。修法：检查权重矩阵的符号，别急着调参。

形态三：临界锥为空。 所有方向都被积极约束堵死，此时二阶条件自动满足（没有可行方向可走），但解对约束的微小扰动极其敏感。表现是：约束右端变化 10^{-6}，最优解跳变。这不是数值问题，是问题本身在该点不可微。修法：把该点的约束软化，或接受“解在这一点不连续”并在上层加滤波。

这三种形态都指向同一个工程结论：**二阶条件不是理论装饰，它决定了闭环里控制量会不会抖。** 上线前做一次扰动测试——把约束右端抖动 10^{-6}，看控制量变化多少——比读任何理论都直接。

30.3.4 影子价格：把约束标价

把 §30.3.1 的灵敏度公式用到二维例子上。约束右端从 $b = 2.5$ 开始增大，最优点沿约束线移动，直到 $b = 3.5$ 时无约束中心进入可行域，约束失效。分段解析解为

$$f^\star(b) = \begin{cases} \dfrac{(3.5 - b)^2}{2}, & b < 3.5 \\ 0, & b \geq 3.5 \end{cases} \qquad \lambda^\star(b) = \begin{cases} 3.5 - b, & b < 3.5 \\ 0, & b \geq 3.5 \end{cases} \tag{30-15}$$

对 b 求导，$\mathrm{d}f^\star/\mathrm{d}b = -(3.5 - b) = -\lambda^\star$，与式 (30-12) 一致。

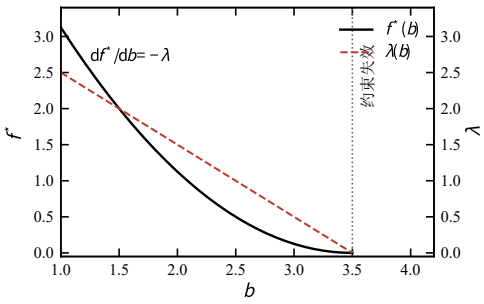


图 30-5　影子价格：最优值曲线与对偶变量

图 30-5 把两条曲线画在一起：左轴是最优值，右轴是对偶变量。两件事值得注意。第一，$f^\star(b)$ 是分段二次的，在 $b = 3.5$ 处导数从 -0 变为 0，拐点正好是约束失效的位置。第二，$\lambda(b)$ 随 b 线性下降，越接近失效点越便宜——**约束越松，放松它的边际收益越小**，这与经济学的边际效用递减是同一件事。

数值验证用 251 个 b 采样点，用中心差分算斜率，与乘子比较，最大偏差 8.26×10^{-14}。在 $b = 1.0$ 处，$f^\star = 3.125$、$\lambda = 2.5$、斜率 -2.495；在 $b = 2.5$ 处，$f^\star = 0.5$、$\lambda = 1.0$、斜率 -1.000。$b = 1$ 处的斜率偏差来自端点附近的差分格式，内部点的偏差在 10^{-14} 量级。

这个工具在现场怎么用？假设一个 MPC 的输入限幅是 $|u| \leq 0.4$，运行中读到的乘子是 $\lambda = 3.2$，而另一个约束（比如速度上限）的乘子是 0.1。这说明前者的“价格”是后者的 32 倍：把加速度上限提高 0.1，累计代价能降约 0.32；把速度上限提高 0.1，只能降约 0.01。**乘子把“该改哪个参数”从工程直觉变成了定量排序。**

要注意乘子的量纲。λ 的量纲是“目标函数量纲除以约束量纲”。若目标用 IAE（秒），约束用加速度（$\mathrm{m/s^2}$），则 λ 的单位是 $\mathrm{s^3/m}$。跨约束比较乘子之前，必须把它们换算到同一量纲，否则比较没有意义。这也是为什么工程上报乘子时要同时报目标函数与约束的单位。

影子价格有三处会失效，必须知道边界。

其一，非凸问题只有局部价格。 在局部最优处算出的 λ 只描述“在这个坑里放松约束的收益”，换一个坑可能是另一个价格。若两个坑的目标值相差很大，局部价格会给出完全错误的投资建议。

其二，离散或整数变量下价格不连续。 约束右端从 2.49 变到 2.51，可能让一个整数变量跳变，最优值出现台阶。此时“斜率”不存在，只能用有限差分算平均价格，而且步长要选得比台阶宽度还大。

其三，约束失效点附近价格趋于零，但收益并未消失。 从图 30-5 看，$b \to 3.5^{-}$ 时 $\lambda \to 0$，说明“再放松一点”几乎没用；但若一次放松很多（越过 3.5），收益会突然停止增长。**价格是局部信息，不能外推。**

正确的用法是做**小步试探**：把某条约束放松 1%，重解一次，看最优值改善多少，再决定是否继续。这个做法对凸问题与灵敏度分析给出同样的结论，但不需要读乘子，也不受非凸与离散的限制，代价只是多解一次。工程上两者可以互相校验：乘子预测的改善量与实测改善量一致，才说明模型可信。

30.3.5 罚函数：把约束折进目标

最朴素的思路是把约束“罚”进目标，变成无约束问题：

$$\min_{x} \; F_\rho(x) = f(x) + \rho \sum_{i=1}^{m} \left[\max(0, g_i(x))\right]^2 \tag{30-16}$$

ρ 是罚因子。约束满足时罚项为零，违反时罚项按二次增长。这个方法叫**外点法**，因为迭代点从可行域外逐步逼近边界，最终停在边界外侧一点点。

外点法的收敛性有明确结论：当 $\rho \to \infty$ 时，F_ρ 的极小点收敛到原问题最优解；约束违反量按 $O(1/\rho)$ 下降。§30.6 的实测完全符合：$\rho = 1$ 时违反 0.333，$\rho = 10$ 时 0.0476，$\rho = 100$ 时 0.00498，$\rho = 10^5$ 时 5.0×10^{-6}。每放大 10 倍罚因子，违反量降约 10 倍，非常规整。

代价在数值上。罚函数的 Hessian 是 $\nabla^2 f + 2\rho \sum \nabla g \nabla g^{\mathsf{T}}$，第二项的量级随 ρ 线性增长。$\rho = 10^5$ 时，Hessian 的条件数比 $\rho = 1$ 时高五个数量级。内层无约束优化（牛顿法或 BFGS）在这种条件下收敛极慢，而且舍入误差被放大。实测中 ρ 从 10^4 增到 10^5，违反量只从 5.0×10^{-5} 降到 5.0×10^{-6}，改进与代价不成比例。

工程上有一个折中：用**精确罚函数**（ℓ_1 罚）

$$\min_{x} \; f(x) + \rho \sum_{i} \max(0, g_i(x)) \tag{30-17}$$

只要 ρ 大于最优乘子的最大分量，它的极小点就**精确等于**原问题最优解，不需要 $\rho \to \infty$。代价是罚项不可微，不能用牛顿法，只能用次梯度或光滑化近似。这条性质在实时优化里很有用：给定乘子的上界估计，一次取定 ρ，问题就变成单个无约束问题。

精确罚的阈值条件可以这样理解：ℓ_1 罚在约束面上的次梯度区间是 $[-\rho, \rho]$，要与原问题的乘子 λ 平衡，必须 $\rho \geq \lambda$。若 $\rho < \lambda$，平衡点落在罚项的斜坡上，解就停在可行域外。**所以 ρ 的选取有明确的物理含义：它必须大于“最贵的那个约束的价格”。** 工程上可以先用一个中等 ρ 试解，读出乘子，再把 ρ 取为最大乘子的 1.5–2 倍，一次设定。

二次罚与 ℓ_1 罚之间还有一个更好的选择：**增广拉格朗日**（ALM）

$$\min_{x} \; f(x) + \sum_{i} \left[\lambda_i \max(0, g_i(x)) + \frac{\rho}{2} \max(0, g_i(x))^2\right] - \frac{1}{2\rho}\sum_{i} \lambda_i^2 \tag{30-18}$$

它在罚项里加上了乘子的线性修正，于是**不需要 $\rho \to \infty$ 也能收敛到精确解**，且对 ρ 的选择不敏感。外层更新 $\lambda_i \leftarrow \max(0, \lambda_i + \rho g_i(x))$，内层解无约束问题。ADMM 本质上是 ALM 的可分离版本，这也是 §30.3.8 里 OSQP 能同时做到“精确”与“一阶代价”的原因。

对 MPC 而言，罚函数法还有一个变体很实用：只对**软约束**用罚，对物理硬约束仍保留为不等式。这样问题规模只增加松弛变量，硬约束的可行性仍由求解器保证。ρ 的取值方法见 §30.5.5。

外点法的适用场合是：约束不多、精度要求 10^{-4} 量级、不想引入求解器依赖。它的致命弱点是：**迭代过程中约束一直在被违反**。对物理系统，这意味着中间过程的控制量可能超出限幅，不能直接上线，只能用于离线设计。

30.3.6 障碍函数：从内部逼近

内点法反过来，从可行域内部逼近边界：

$$\min_{x} \; \phi_\mu(x) = f(x) - \mu \sum_{i=1}^{m} \log\left(-g_i(x)\right) \tag{30-19}$$

当 x 靠近边界时 $-g_i \to 0^{+}$，对数为 $-\infty$，障碍项趋于 $+\infty$，把迭代点“挡”在内部。$\mu > 0$ 是障碍参数，越小越接近真实边界。

内点法的理论保证是中心路径上的对偶间隙上界：

$$f(x_\mu) - f^\star \leq m\mu \tag{30-20}$$

其中 m 是不等式约束个数。这条不等式给出了**停止准则**：想让间隙小于 ϵ，取 $\mu < \epsilon/m$ 即可，不需要知道最优值。§30.6 的实测中，$m=3$，$\mu = 10^{-6}$ 时理论上界 3×10^{-6}，实测间隙 1.000×10^{-6}——实际间隙只有上界的 1/3，因为只有一条约束积极，其余两条的贡献在高阶项里。

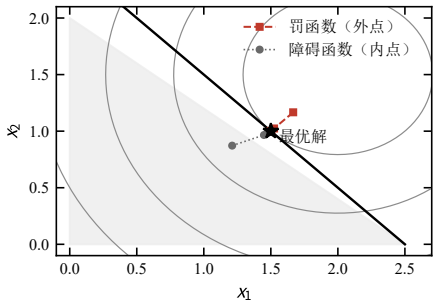


图 30-4　罚函数（外点）与障碍函数（内点）的迭代路径

图 30-4 把两条路径画在同一张等高线图上。罚函数的迭代点从可行域外出发（初始点在原点，约束违反），沿直线逼近边界，最终停在边界外侧；障碍函数的迭代点始终在可行域内，沿一条光滑曲线逼近最优点，永远不会越过边界。两条路径的终点在图上几乎重合，但过程完全不同：**一个从外面撞进来，一个从里面挤过去。**

内点法的“内”字还有一层含义：迭代点不只在可行域内，而是沿着**中心路径**走。中心路径是 μ 从大到小时障碍问题最优解连成的曲线，可以理解为“离所有边界都尽量远”的点集。原始-对偶内点法把 (μ, x, λ) 一起迭代，用牛顿法同时更新三者，并在每步把 μ 乘以一个小于 1 的因子。这样做的好处是：迭代点始终在可行域内部、且不贴边，数值上更稳。

中心路径也解释了对偶间隙上界的来源。在中心路径上，互补松弛被“平滑”成 $\lambda_i(-g_i(x)) = \mu$，对 i 求和就得到 $\sum_i \lambda_i(-g_i) = m\mu$，而左边正是间隙的表达式。于是 $m\mu$ 不是估计，而是**定义**。这也说明强行把 μ 一次降得很小没有用：牛顿步会试图一次跨过整条路径，反而失稳，所以工程实现里 μ 每步只降 5–10 倍。

实际实现还有一项常被忽略的工作：**初始点的可行性**。原始-对偶方法不需要严格可行的初值（它允许原始或对偶一侧暂时不可行），但需要一个数值上不坏的起点。常见的做法是把 x 放在约束的“内部估计”上，λ 取全 1，再跑几步牛顿法。若初值离可行域太远，前几步会剧烈震荡——这也是为什么在线 MPC 里内点法的第一拍耗时往往明显高于后续。

内点法还有一个前提：可行域必须有内点（Slater 条件）。若 MPC 的初始状态已经违反状态约束（比如超调已经发生），纯障碍法连第一步都走不了；原始-对偶实现虽然容忍初始不可行，但问题本身若无严格可行点，中心路径就不存在，迭代会发散。工程上的处理是先把状态约束软化，或者用第一阶段（phase I）求一个可行点。这一点在 §30.5.4 展开。

内点法的每次迭代要解一个牛顿方程，Hessian 为

$$\nabla^2 \phi_\mu = \nabla^2 f + \mu \sum_i \left[\frac{\nabla g_i \nabla g_i^{\mathsf T}}{g_i^2} - \frac{\nabla^2 g_i}{g_i} \right] \tag{30-21}$$

对线性约束，$\nabla^2 g_i = 0$，Hessian 是“原 Hessian 加一堆秩一正项”，保持正定。每次迭代的代价是 $O(n^3)$（稠密）或利用稀疏性的更低阶。外层再套一层 μ 的递减（通常每次乘 0.1 或 0.2），总的牛顿步数在 20–50 步量级，与问题规模弱相关——这是内点法最吸引人的性质。

30.3.7 SQP：对 KKT 条件做牛顿法

序列二次规划（SQP）把 KKT 条件当成非线性方程组，用牛顿法求解。每一步在当前点 x_k 处把目标展开到二阶、约束展开到一阶，解一个 QP 子问题：

$$\min_{d} \ \tfrac{1}{2} d^{\mathsf T} \nabla^2 L(x_k, \lambda_k) d + \nabla f(x_k)^{\mathsf T} d \quad \text{s.t.} \quad g_i(x_k) + \nabla g_i(x_k)^{\mathsf T} d \le 0 \tag{30-22}$$

解出步长 d 后更新 $x_{k+1} = x_k + \alpha d$，并用子问题的乘子更新 λ。在最优解附近，$\alpha = 1$ 时 SQP 具有**局部二次收敛**：误差按 $e_{k+1} \sim e_k^2$ 下降。这比一阶方法的线性收敛快得多，代价是每步要解一个 QP。

SQP 的全局化靠线搜索或信赖域，而 merit 函数通常取 ℓ_1 精确罚函数。这解释了一个常见现象：SQP 的收敛性依赖罚参数的选择，罚参数太小会拒绝好的步长，太大会在边界附近震荡。

子问题里的 Hessian 未必能手算。工程实现通常用 **BFGS 更新**逼近 $\nabla^2 L$：从单位阵或某个正定近似出发，用相邻两步的梯度差与位移差做秩二修正，保持正定性。这样每步只需要梯度，不需要二阶信息，代价是 $O(n^2)$ 的矩阵更新。BFGS 在凸问题上表现稳定，在非凸问题上可能给出“过于乐观”的曲率，表现为步长被反复拒绝。诊断方法是看更新后的矩阵最小特征值是否趋于零。

还有一类细节叫 **Maratos 效应**：在约束曲率较大的点附近，一个本来能降低目标的步长，会让约束的二阶项超界，于是被 merit 函数拒绝，收敛速度从二次掉到线性。修法是二阶校正（在步长上加一个小的修正量，抵消约束的曲率误差）。工程上若发现 SQP 在最后几步反复"进两步退一步"，多半就是这个原因，加二阶校正或改用信赖域即可。

SQP 在控制里的典型用法是**非线性 MPC**：每一步在当前状态处线性化动力学，解一个 QP 得到输入增量，滚动执行。这时"SQP 的迭代次数"与"控制器的拍数"是两回事——通常每拍只做 1–2 次 SQP 迭代（实时迭代方案），把上一拍的解作为初值，靠滚动本身代替完全收敛。这种做法的理论保证比全收敛弱，但实测性能往往更好，因为相邻两拍的问题几乎相同。

线性化的误差要单独交代。约束在 x_k 处展开到一阶，实际可行域是弯曲的，线性化后的可行域是它的切平面近似。若曲率大（例如约束含 x^2 或三角函数），近似解可能违反原约束。工程上的补偿是**加安全裕度**：把约束右端收紧一点，收紧量按曲率与步长的乘积估计。裕度太小不起作用，太大会让可行域收缩、性能下降。经验做法是先按 1%–5% 留裕度，再用闭环仿真确认没有越界。**非线性 MPC 的"可行性"永远是近似意义下的，这一点必须在文档里写明。**

对凸 QP，SQP 退化成"有效集法"：每一步猜测哪些约束积极，解一个等式约束的 QP，检查乘子符号，不满足就换一个约束加入或退出。若只有 m 条约束，最坏情况要枚举 2^m 种积极组合。$m=20$ 时是 10^6，$m=40$ 时是 10^{12}——**有效集法适合小规模、约束少的问题，不适合大规模 MPC**。它的优势是热启动极快：相邻两拍的积极集通常只差一两条约束，从上一拍的积极集出发往往一两步就收敛。qpOASES 与 DAQP 就是按这个思路做的。

30.3.8 一阶方法与 OSQP

二阶方法每步 $O(n^3)$，大规模时太贵。一阶方法每步只做矩阵-向量乘，代价 $O(\mathrm{nnz})$，但要几百到几千步。ADMM（交替方向乘子法）是代表，它把 QP 拆成"二次项的近端映射"与"约束的投影"两块，交替更新：

$$z^{k+1} = \Pi_{\mathcal{C}}\left(z^k + u^k - \Sigma^{-1}(Hz^k + g)\right), \qquad u^{k+1} = u^k + z^{k+1} - z^k \tag{30-23}$$

其中 $\Pi_{\mathcal{C}}$ 是到约束集的投影（箱约束就是逐元素裁剪），Σ 是步长矩阵。每次迭代只有一次矩阵-向量乘和一次裁剪，实现极简。

OSQP 就是 ADMM 在 QP 上的工程实现，配合稀疏线性代数与自适应步长。它的特点是：**迭代次数对规模不敏感，单次迭代极便宜，热启动效果好**。§30.6 的实测中，N 从 5 增到 80（变量从 15 增到 240），中位迭代次数从 75 增到 162，中位求解时间从 1.06 ms 增到 4.90 ms。时间增长不到 5 倍，而变量数增长 16 倍——这正是"迭代次数弱相关、单步代价线性"的表现。

一阶方法的短板是精度。ADMM 的收敛率是 $O(1/k)$，要把残差压到 10^{-9} 需要很多步。工程上的做法是：用 ADMM 快速得到一个中等精度的解，再用主动集法（polish）把它精化到机器精度。OSQP 的 `polish=True` 就是这一步。对控制问题，10^{-6} 的原始残差已经远超传感器精度，不必追求更高。

收敛率还能改善。ADMM 里有一个松弛参数 $\alpha \in (0,2)$，把 z 的更新写成 $z^{k+1} = \alpha z^{k+1}_{\text{new}} + (1-\alpha) z^k$。$\alpha = 1$ 是标准形式，$\alpha > 1$ 叫超松弛，在条件数中等的问题上能把迭代次数降到一半；α 太大则振荡。OSQP 内部会根据残差历史自适应调 α 与步长 Σ，用户通常不需要碰这两个参数，但知道它们存在有助于理解"为什么同一问题换一台机器迭代次数不同"。

停止准则的写法比想象中重要。绝对残差阈值在不同量纲的问题上不可比：目标量级 10^6 时 10^{-4} 的绝对残差很准，目标量级 10^{-2} 时同样的阈值等于没收敛。OSQP 用的是归一化后的相对准则：原始残差除以 $\max(\|Az\|, \|b\|)$，对偶残差除以 $\max(\|Hz\|, \|g\|)$。**这也是为什么 §30.4.3 强调先做无量纲化——它同时改善条件数与停止准则的可比性。**

什么时候不用一阶方法？当问题规模小于 100 个变量、且要求 10^{-9} 以上的精度时，内点法或有效集法往往更快；当问题需要严格可行性保证时，一阶方法只能给"近似可行"。一阶方法的真正优势在三种场合：规模大（上千变量）、需要热启动、需要在任意时刻中断并返回当前最好解。**"随时可停"是它独有的性质，内点法做不到，有效集法也做不到。**

30.4 离散对应与数值实现

30.4.1 从连续问题到离散 QP

控制里的约束优化绝大多数是离散形式，原因不是偏好，而是执行器本身是离散更新的：数字控制器每隔 T_s 计算一次，保持器在两次计算之间维持输出不变。连续时间的约束 $|u(t)| \le u_{\max}$ 在零阶保持下等价于每个采样点满足 $|u_k| \le u_{\max}$，因为保持器输出的极值一定出现在采样点上。

于是 MPC 的标准构造是三步。第一步，把对象离散化（第 6 章），得到 $x_{k+1} = Ax_k + Bu_k$。第二步，把状态用输入表示，写成式 (30-6) 的形式。第三步，把代价与约束都写成 U 的函数，得到一个 QP。

代价矩阵的构造需要注意权重的时间分布。若终端代价用 Riccati 解 P，则

$$H = \mathcal{B}^{\mathsf{T}} \bar{Q} \mathcal{B} + \bar{R}, \qquad \bar{Q} = \mathrm{diag}(Q, \dots, Q, P), \qquad \bar{R} = \mathrm{diag}(R, \dots, R) \tag{30-24}$$

$\bar{Q}$ 的最后一块是 P 而不是 Q，这是终端代价的体现。少了这一项，闭环的稳定性没有保证；加了这一项，闭环在无约束时与 LQR 完全等价，在有约束时退化为“约束下的最长视距近似”。§30.6 的 MPC 实验就用了 P，所以无约束时它能复现 LQR 的响应。

约束的组装按行堆叠。输入约束 $|u_k| \le u_{\max}$ 变成 $2N$ 行；状态约束 $x_{\min} \le x_k \le x_{\max}$ 要先展开成 $\mathcal{A}x_0 + \mathcal{B}U$，再变成 $2n_x(N+1)$ 行。注意状态约束的右端含 x_0，它是参数而不是常数——每拍更新，结构不变。这正是把 x_0 设为 `Parameter` 的原因：结构只编译一次，每拍只换数值，热启动才有效。

下面的函数把上述步骤写成可运行的代码，采用消元写法，返回标准 QP 的四个数据块。

```
import numpy as np

def build_condensed_qp(A, B, Q, R, P, N, u_max, x_ref, x0):
    """把 MPC 组装成标准 QP：min 0.5*U'H*U + g'U  s.t.  A_ub*U <= b_ub。"""
    nx, nu = B.shape
    # 1) 预测矩阵：A_bar 把初值映射到各步，B_bar 把输入映射到各步
    A_bar = np.vstack([np.linalg.matrix_power(A, k + 1) for k in range(N)])
    B_bar = np.zeros((nx * N, nu * N))
    for k in range(N):
        for j in range(k + 1):
            B_bar[k * nx:(k + 1) * nx, j * nu:(j + 1) * nu] = \
                np.linalg.matrix_power(A, k - j) @ B
    # 2) 分块权重：前 N-1 块 Q，最后一块换成终端代价 P
    Q_bar = np.kron(np.eye(N), Q)
    Q_bar[-nx:, -nx:] = P
    R_bar = np.kron(np.eye(N), R)
    # 3) 代价矩阵与线性项；x0 每拍变化，只影响 g
    H = B_bar.T @ Q_bar @ B_bar + R_bar
    H = 0.5 * (H + H.T)                       # 强制对称，消除舍入误差
    g = B_bar.T @ Q_bar @ (A_bar @ x0 - np.tile(x_ref, N))
    # 4) 约束：输入的箱约束 + 状态的箱约束，各写上下两侧
    x_max = np.tile(np.array([1e3, 1e3]), N)  # 示例：状态不设限
    x_min = -x_max
    I_N = np.eye(nu * N)
    A_ub = np.vstack([I_N, -I_N, B_bar, -B_bar])
    b_ub = np.concatenate([
        np.full(nu * N, u_max),               # u <= u_max
        np.full(nu * N, u_max),               # -u <= u_max
        x_max - A_bar @ x0,                   # x <= x_max
        A_bar @ x0 - x_min,                   # -x <= -x_min
    ])
    return H, g, A_ub, b_ub
```

这段代码有四处值得注意。H 与 x_0 无关，可以离线算一次；g 与 x_0 成线性关系，每拍只需一次矩阵-向量乘；A_{ub} 与 b_{ub} 中只有后一半随 x_0 变化；对称化那一行看似多余，却能避免求解器因 10^{-16} 的非对称而拒绝矩阵。把这四点落实到工程里，就是“结构编译一次、数值每拍更新”的热启动写法。

代码里的 `x_max = 1e3` 是一个“不设限”的示例值。真实项目里状态约束往往比输入约束更复杂：位置有软限位与硬限位两套，速度有工艺上限，某些状态只在特定阶段受限。这些都要在组装阶段展开成行，且要记住**约束的顺序决定了乘子的顺序**，调试时按顺序查表才不会错位。

30.4.2 稀疏性与变量排列

QP 的求解时间很大程度上由稀疏性决定。同一个问题有两种写法：

消元写法（condensed）：只保留 U 作为决策变量，H 是 $n_u N \times n_u N$ 的稠密矩阵。变量少，但 H 稠密，每次矩阵-向量乘是 $O((n_u N)^2)$。

稀疏写法（sparse）：同时保留 X 与 U，等式约束 $x_{k+1} - Ax_k - Bu_k = 0$ 作为稀疏约束。变量多（$(n_x + n_u)N$），但矩阵带状稀疏，每次乘是 $O(N)$。

N 小于 20 时，消元写法更快，因为稠密矩阵小、没有等式约束的乘子；N 大于 50 时，稀疏写法占优，因为 $O(N^2)$ 的增长开始压过常数项。工程上的经验分界在 $N \approx 30$。OSQP 默认走稀疏写法，CVXPY 会根据问题结构自动选择。

变量排列也影响性能。把同一时刻的 x_k, u_k 放在一起（按时间分块），矩阵的非零元集中在对角带附近；把所有的 x 放前面、所有的 u 放后面，非零元会散开，填充（fill-in）增加。对 $N = 80$ 的问题，两种排列的求解时间可以差 30%–50%。这一点在写求解器接口时容易被忽略，因为两者的数学等价。

填充是怎么产生的？一阶方法每步要做一次形如 $(H + \sigma I)^{-1}$ 的求解，若直接求逆就退化成稠密运算（$O(n^3)$），所以实现里用稀疏分解或迭代解。分解过程中，原本为零的位置可能被“填”成非零，非零元数量增长，内存与时间随之上升。重排变量（例如按时间分块、或用近似最小度排序）能把填充压到最小。**这也是“同一个 QP 用不同接口写，速度差几倍”的主要原因**——差别不在算法，而在矩阵的排列。

稀疏写法还有一个隐性代价：等式约束带来额外的乘子变量，问题的总变量数从 $n_u N$ 增到 $(n_x + n_u)N + n_x N$。N 小的时候这个代价超过稀疏带来的收益，所以分界点落在 $N \approx 30$，而不是“稀疏一定更好”。

30.4.3 数值条件：单位、缩放与对称化

优化问题对量纲极其敏感。位置用米、速度用米每秒、加速度用米每二次方秒，三者的数值量级可能差 10^2 以上。Hessian 的条件数因此被放大，一阶方法收敛变慢，二阶方法精度下降。

三条实用规则。

第一，先无量纲化再建模。 把每个变量除以它的典型值（位置除以行程、速度除以最大速度、输入除以限幅），使所有变量在 $[-1, 1]$ 量级。约束变成 $|u| \le 1$，权重变成相对权重，条件数立刻改善一个量级以上。

第二，约束也要缩放。 若某条约束的系数是 10^6、右端是 10^3，把它整体除以 10^6，右端变成 10^{-3}。这不改变可行域，但改变了求解器看到的数值范围。OSQP 内部有自适应缩放，但外部先做一遍更稳。

第三，Hessian 要对称化。 数值构造出的 H 常有 10^{-16} 量级的非对称部分，某些求解器会因此报“矩阵非对称”或悄悄用下三角。构造完成后强制 $H \leftarrow (H + H^{\mathsf{T}})/2$，代价可以忽略。

容差设置要和物理精度对齐。位置控制精度 1 μm，行程 1 m，相对精度 10^{-6}；把 `eps_abs` 设成 10^{-10} 只会浪费迭代。反过来，如果约束是安全限位，容差要收紧到 10^{-8}，并额外留 1% 的物理裕度——求解器的“可行”是数值意义下的，不是物理意义下的。

缩放的效果可以定量估计。若用对角矩阵 D 做变量替换 $z = D^{-1}U$，新 Hessian 是 DHD，条件数近似按 $(\max d_i / \min d_i)^2$ 变化。位置量程 1 m、输入量程 0.4、速度量程 0.5，三者本身相差不大；但若换成“位置用毫米、速度用米每秒”，比例到 10^3，条件数恶化 10^6，一阶方法的迭代次数会从百次级涨到千次级。**单位不是文档问题，是数值问题。**

还有一处容易忽略的缩放：目标函数与约束之间的相对量级。若目标值量级 10^6、约束违反量级 10^{-3}，罚项或乘子的数值会跨越九个数量级，双精度的有效位被吃光。处理方法是在建模时把目标除以它的典型值，使目标在 $O(1)$ 量级；约束也做同样处理。这样乘子的数值范围被压缩，报告与比较才有意义。

30.4.4 计算量：实测数据与热启动

表 30-2 是在本机（Windows、py311、OSQP 0.6 接口、CVXPY 1.7.5）上实测的 QP 规模与求解时间。对象为双积分，状态 2 维、输入 1 维，只有输入箱约束；每组规模用 10 个不同初值求解，取中位数，以排除首次编译的开销。

表 30-2　QP 规模与求解时间的实测（中位数）

预测时域 N	变量数	不等式数	迭代次数	求解时间 (ms)
5	15	10	75	1.06
10	30	20	125	1.29
20	60	40	125	2.53
40	120	80	162	3.11
80	240	160	162	4.90

三列数字要一起看。变量数增长 16 倍，时间只增长 4.6 倍；迭代次数从 75 增到 162，增长 2.2 倍；剩下的增长来自单次迭代的代价，它随规模线性增长。**总时间大致按 $O(N^{1.5})$ 增长**，比稠密二阶方法的 $O(N^3)$ 好得多。

这组数字还揭示了一个常被忽略的事实：N 从 5 到 10，迭代次数从 75 跳到 125，但时间只从 1.06 ms 增到 1.29 ms。原因是小规模时求解时间被接口开销主导（Python 调用、数据拷贝），真正的数值计算只占一部分。**在毫秒量级，语言与接口的开销和算法本身一样重要。**

热启动的效果在闭环里更明显。§30.6 的闭环仿真中，MPC 每拍求解的平均时间是 1.80 ms，最大值 4.47 ms。平均值低于表 30-2 的 2.53 ms，因为闭环中相邻两拍的状态接近，上一拍的解是很好的初值；最大值出现在状态突变（如参考跳变）后的第一拍。**实时性评估必须看最大值而不是平均值**：4.47 ms 是 0.1 s 采样周期的 4.5%，安全；若采样周期是 5 ms，这个最大值就吃掉 90% 的时间预算，必须换算法或加长周期。

30.4.5 终端代价与递推可行性

MPC 能在闭环里稳定运行，靠的不只是“每拍解一个 QP”，还需要一条性质：**这一拍求出的可行解，能推出下一拍问题也可行。** 这条性质叫递推可行性，它不是自动成立的。

设第 k 拍的最优输入序列为 $u_0^\star, \dots, u_{N-1}^\star$，对应状态 $x_1^\star, \dots, x_N^\star$。执行 $u_0^\star$ 后，下一拍的状态是 $x_1^\star$，一个自然的候选解是把上一拍的输入序列左移一位、末位补零。检查这个候选解是否可行：前 $N-1$ 步的约束沿用了上一拍的最优轨迹，当然满足；唯一可能出问题的是第 N 步——补零后的末端状态不再是 $x_N^\star$，而是从 $x_N^\star$ 出发自由演化一步的结果。

于是要加一个条件。最常用的是**终端等式约束** $x_N = x_{\text{ref}}$：补零后状态停在 x_{ref}（平衡点），约束仍满足。另一个是**终端集约束** $x_N \in \Omega$，要求 Ω 是控制不变集，且补零后仍留在集内。两者都能保证递推可行性，但代价不同：等式约束把可行域压得很窄，容易出现 §30.6 的 E6 那种不可行；终端集约束宽松一些，但计算 Ω 需要额外的离线工作。

第三条路是**只有终端代价、没有终端约束**。取 P 为 Riccati 解时，可以证明闭环在无约束时稳定，但有约束时稳定性没有一般性保证。工程上的折中是：加一个较宽松的终端集，同时用 P 做终端代价。这样既保留了可行域，又拿到了稳定性。

这三条路的选择标准很实际：**可行域够不够大，取决于最坏工况下的初始偏差。** 若现场会出现大幅度的启动偏差，终端等式约束很可能第一拍就不可行；此时应改用终端集，或者干脆把终端约束软化（§30.5.4）。

终端集怎么算？最直接的是**最大不变集迭代**：先把 Ω_0 取成状态约束允许的集合，然后反复做 $\Omega_{k+1} = \{x \in \Omega_k \mid \exists u,\ Ax + Bu \in \Omega_k,\ |u| \le u_{\max}\}$，直到集合不再变化。多面体集合用多面体运算实现，规模大时计算量可观，通常离线做一次、存成查表或半空间表示。另一条路是**椭球不变集**：假定 $\Omega = \{x \mid x^\top P^{-1} x \le 1\}$，把“存在 u 使下一步仍在 Ω 内”写成 LMI，用 SDP 求 P。椭球保守一些，但计算便宜、存储只需一个矩阵。

30.4.6 从模型到上线的五步

把 §30.4 的内容压缩成一条落地路径。

第一步，定模型与周期。 确定离散模型 (A, B)、采样周期 T_s、预测时域 N。$N \cdot T_s$ 应覆盖对象主导动态的 2–3 倍，同时保证 N 不太大——表 30-2 显示 N 从 20 增到 80，时间翻倍，收益却常常不明显。

第二步，定量纲与权重。 所有变量除以典型值，使量级在 $O(1)$；权重按“允许的偏差比例”设定，而不是按物理单位设定。Q 与 R 的比值决定了闭环的激进程度，先按 LQR 的经验值起步，再用仿真调整。

第三步，定约束与优先级。 每条约束标注“物理硬约束 / 工艺软约束 / 安全联锁”。硬约束进 QP，软约束加松弛，安全联锁不进 QP（第36章）。这一步决定了问题是否永远可行，也决定了兜底策略。

第四步，选求解器与写法。 按表 30-3 选实现；按 N 的大小在消元与稀疏之间选；确认求解器支持热启动与超时退出。若目标平台是嵌入式 C，优先考虑代码生成。

第五步，测最坏耗时与兜底。 扫三类最坏工况，取 P99.9 与最大值；确认最大值不超过周期的 30%；确认超时与不可行两条路径都能正确返回并记录日志。

五步之中，第三、第五步最容易被跳过，也最容易在投运后出事。**优化器的失败几乎都不是“算错了”，而是“没定义失败时怎么办”。**

30.5 工程含义与适用边界

前四节回答的是“怎么解”。这一节回答另外三个问题：该不该解、用什么解、解不出来怎么办。三者分别对应建模决策、工具选型、运行时保障，任何一环缺位，在线优化都难以长期稳定运行。

30.5.1 什么时候不该用在线优化

在线优化不是万能的。以下四种情形，强行上 MPC 的收益往往为负。

第一，采样周期与计算预算不匹配。 若控制周期在 1 ms 以下（电流环、开关电源、伺服力矩环），一次 QP 的接口开销就足以吃掉大半预算：Python 侧调用需要 0.2–1 ms，C 侧也需要 10–50 μs，再叠加状态估计与通信，留给求解器的时间所剩无几。此时有两条替代路

线：一是离线求解显式 MPC，把最优解写成状态的分段仿射函数，在线只做查表与仿射运算；二是退回抗饱和 PID，把约束交给限幅与抗饱和逻辑处理。判据很简单：**最坏求解耗时超过周期的 30%，就不要硬上在线优化。**

第二，模型质量不足。 优化的本质是“在模型上求最优”，模型误差会被原封不动地搬到控制量上，而且不像反馈控制那样有天然的误差抑制能力。若对象存在强非线性、大滞后、参数慢漂移，又没有在线辨识配合（见第23章），优化多花的计算换来的不是性能，而是对模型的过度信任。一个可操作的检验：先用开环预测跑 100 步，看预测与实测的偏差是否小于约束裕度的三分之一；达不到就不要指望优化。

第三，约束没有物理意义或执行不了。 约束写成 $|u| \le 0.4$，执行机构却有 5% 死区、0.2 s 响应滞后，那么“满足约束”只是纸面上的。约束模型必须与执行机构的真实特性一致，否则求解器给出的可行解在物理上仍然越界。

第四，单回路、单约束。 只有一个输入、一个限幅时，抗饱和 PID（见第44章）与带约束优化在效果上等价，而计算量小两个数量级。优化的价值出现在多变量耦合、多约束互相冲突的场合——那时“先满足谁”本身就是一个需要计算的决策。

此外还有一项常被低估的成本：维护。在线优化要求现场人员能读懂求解器返回的状态（`optimal`、`infeasible`、`unbounded`），能读乘子判断哪条约束在起作用，能处理数值退化。缺这套能力，故障时只能整机重启。

还有一条边界值得单独提出：**优化与规则控制的边界**。产线上大量逻辑是“如果温度超过 80℃ 就开阀”这类规则，它们不是优化问题，硬写成约束反而会引入不可行与优先级混乱。合理的分层是：规则控制负责离散决策与联锁，优化负责连续量的最优分配。两者的接口是“优化接受规则给出的可行区间，规则不接受优化的返回值作为触发条件”。把这条边界画清楚，能避免一大类“优化器算出了不该执行的动作”的事故。

30.5.2 求解器选择：按问题结构与实时性分档

选型分两步：先按问题类型确定算法族，再按实时性与部署条件确定具体实现。表 30-3 给出常用组合。

表 30-3　求解器选型

场景	推荐实现	主要理由
QP、嵌入式、周期 ≤10 ms	OSQP、qpOASES、DAQP	支持热启动，无动态内存，代码可裁剪
QP、高精度、离线或慢回路	Clarabel、Mosek	内点法精度高，但对热启动不敏感
SOCP（鲁棒 MPC）	ECOS、Clarabel、SCS	SCS 可返回不可行证书
SDP / LMI	SCS、Mosek	规模小，对精度容忍度高
代码生成、批量部署	ACADO、FORCES、HPIPM	结构固定，无解释器开销

表 30-3 背后有四条判断依据。

一阶与二阶的分界看规模。 一阶方法（ADMM、OSQP）单次迭代廉价、内存占用小、迭代次数对规模不敏感，缺点是精度收敛慢；二阶方法（内点法、有效集法）迭代次数少，但每次迭代要做矩阵分解，内存与单步代价都随规模快速增长。$N \le 20$ 的小 QP 两者差别不大，$N \ge 100$ 时一阶方法凭借稀疏结构明显占优——这正是表 30-2 中时间只按 $O(N^{1.5})$ 增长的原因。

热启动决定闭环可行性。 内点法沿中心路径推进，参数一变中心路径就变，上一次的解几乎帮不上忙；有效集法与 ADMM 可以直接复用上一拍的解与工作集。闭环中相邻两拍的问题只差一个状态初值，热启动通常能把迭代次数压低 3–5 倍。若控制器周期紧张，热启动不是优化项而是必需项。

不可行证书比“infeasible”有用。 SCS 与 Clarabel 能返回 Farkas 证书，指出哪一组约束互相冲突。调试阶段这比一个状态字符串有价值得多：它直接告诉你该放松哪条约束，或者该调整哪个优先级。

代码生成是硬实时的前提。 固定时域、固定稀疏结构后，生成器可以输出无动态内存、无解释器的 C 代码，最坏执行时间可静态分析。安全关键场合（见第36章）要求的是可证明的上界，而不是统计意义上的平均值。

选型还要过四道部署关，这四道关与算法优劣无关，却经常决定项目能不能交付。

浮点与平台。 嵌入式处理器可能只有单精度，而单精度的机器 epsilon 是 10^{-7}，容差设到 10^{-9} 永远无法达到，求解器会一直迭代到上限。选型时要确认求解器支持单精度并调整容差；若必须用双精度，就要评估处理器是否有 FPU 与足够的算力。

内存。 内点法要存矩阵分解的因子，内存随规模超线性增长；一阶方法与有效集法的内存近似线性。控制器通常跑在资源受限的板子上，内存比算力更容易成为瓶颈。选型时先算一遍峰值内存，再看算力。

许可证与可维护性。 商业求解器（Mosek、Gurobi）在离线设计与验证阶段很好用，但部署到设备上涉及许可证与版本管理。开源实现（OSQP、qpOASES、Clarabel）没有这个问题，代价是要自己承担数值问题的排查。

可验证性。 交付前要能回答：“同样的输入，两次运行结果是否一致？”一阶方法的迭代次数依赖停止准则，浮点差异可能让结果在末位不同；有效集法在退化问题上可能走不同的路径。若下游对一致性有硬要求，要在求解后加一步确定性精化，或固定迭代次数而不是依赖容差。

30.5.3 实时性预算：从最大值倒推

工程上有一条经验线：**单次最坏求解耗时不超过采样周期的 30%。** 剩下的 70% 要分给采样与模数转换、通信、执行机构动作、操作系统抖动与多任务抢占。留 30% 而非 80%，是因为抖动是重尾分布的，平均值好看不代表不会超时。

用 §30.6 的实测数字对照：MPC 每拍平均 1.80 ms、最大 4.47 ms。周期 100 ms 时占比 4.5%，宽裕；周期 5 ms 时占比 89.4%，一有抖动就丢拍。同一套代码，周期不同，结论完全相反——这说明**实时性不是算法的属性，而是算法与周期配对的属性**。

测最坏耗时不能只跑一次。至少扫三类工况：参考阶跃幅度最大的那一拍、初始偏差最大的那一拍、扰动与参考变化同时发生的那一拍。每类跑 10^4 次，取 P99.9 与全局最大值，两者都要看：P99.9 反映常态抖动，全局最大值反映极端情况。台式机上的抖动来源包括垃圾回收、缓存冷启动、CPU 降频与后台任务抢占；嵌入式上则应改用最坏执行时间（WCET）分析，用静态上界代替统计量。

预算要列成账单，而不是只写一个总数。以 10 ms 周期为例，一份典型的分配是：采样与模数转换 0.5 ms、状态估计 0.8 ms、优化求解 3.0 ms、输出与通信 0.5 ms、余量 5.2 ms。余量看起来很大，但它要吸收的正是那 0.1% 的极端拍——一旦某拍求解用了 8 ms，余量就只剩 0.2 ms，下一拍的采样可能已经开始。**把预算写成五项，比写“不超过 3 ms”更容易发现哪一项在悄悄膨胀。**

预算还要按工况分档。启动、切换工况、大幅扰动时的求解时间通常明显高于稳态，若只按稳态测，投运后第一次大幅动作就会超时。可行的做法是把“稳态预算”与“瞬态预算”分开规定，瞬态时允许放宽到周期的 50%，但必须有超时兜底。

无论预算多宽，都必须有超时保护。求解器超过预算时立即返回上一拍的可行解（或退化为比例控制），同时置标志位记录事件。没有超时保护的在线优化，在过载时会输出不可预测的控制量——这比不用优化更危险。

30.5.4 不可行兜底：软约束、分层与回退

在线优化最尴尬的时刻不是算得慢，而是算出“不可行”。硬约束 QP 一旦不可行，求解器不返回任何控制量，控制器等于瞬间失效。工程上有三条兜底路线，图 30-6 把它们画进了同一条执行链。

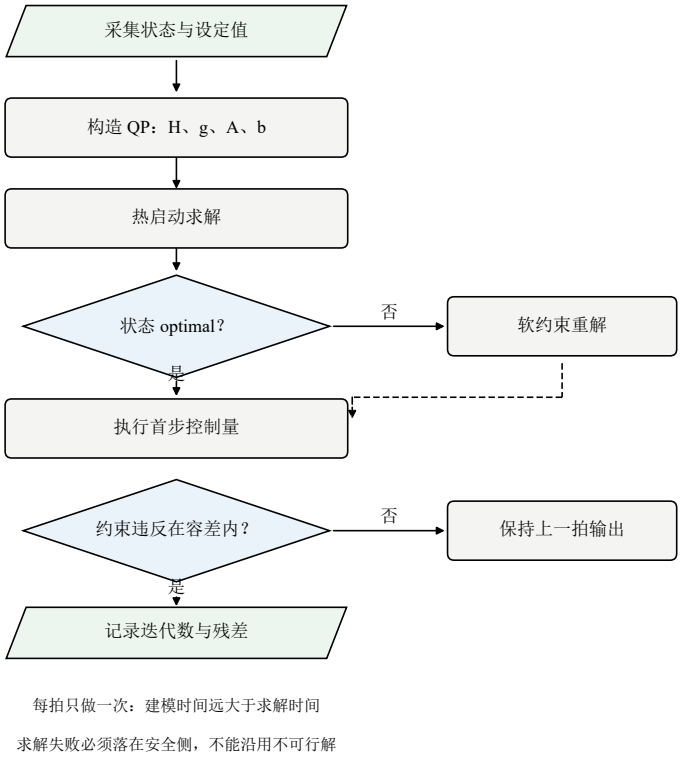


图 30-6　在线优化的求解流程与不可行兜底

路线一：软约束。 给可能冲突的约束加松弛变量 $s \geq 0$，并把 $\|s\|_1$ 以权重 ρ 罚进目标：

$$\min_{x,u,s} \sum_{k=0}^{N-1} \left(\|x_k - x_{\text{ref}}\|_Q^2 + \|u_k\|_R^2 \right) + \rho \|s\|_1 \quad \text{s.t. } x_{k+1} = Ax_k + Bu_k,\ |u_k| \leq u_{\max},\ x_N - x_{\text{ref}} \leq s,\ s \geq 0 \tag{30-25}$$

这样问题永远可行（s 可以任意大），代价是“违反约束要花钱”。ℓ_1 罚的好处是精确：只要 ρ 大于对应约束的乘子，最优解中 s 的取值就与硬约束版本完全一致（§30.3.5 的结论）。§30.6 的 E6 就是这个机制的对照实验：同一初始状态、同一条终端约束，硬约束版本返回 `infeasible`，软约束版本返回 `optimal`，松弛量 0.8500——控制器照常输出，同时把 0.85 的偏差如实报出来。

路线二：分层（字典序）优化。 先解一个只含松弛变量的最小违反问题，得到最小可能的违反量 $s^\star$；再固定 $s \leq s^\star(1+\epsilon)$，解性能目标。它比软约束更可控：优先级的顺序由问题结构保证，不依赖权重调参。代价是要解两次，计算量翻倍，通常只用在安全约束上。

路线三：回退。 检测到不可行时，直接输出上一拍的可行解，或切换到备用控制器。这条路线不优雅，但最容易验证：回退路径的代码只有几行，可以用穷举测试覆盖。

三条路线可以叠加。实践中常见组合是：输入约束做硬约束（执行机构物理上做不到就是做不到），状态与终端约束做软约束，再在最外层加超时回退。

还有一点常被遗漏：**不可行本身就是诊断信息。** 连续多拍出现不可行，说明模型与对象已经偏离，或某条约束设得超出物理能力。把不可行次数、最大松弛量记进日志，比事后看曲线猜原因有效得多。

30.5.5 软约束权重怎么选

松弛权重 ρ 不是“越大越好”。ρ 太小，优化器会随意牺牲约束换取性能；ρ 太大，Hessian 的条件数按 ρ^2 增长，一阶方法收敛变慢，数值上等价于硬约束——又回到了不可行的老问题。

一个可操作的三步法。

第一步，归一化。 把松弛变量除以它对应的物理量程，使 s 无量纲。终端位置约束的松弛除以行程，速度约束的松弛除以最大速度。不归一化时，ρ 的取值会被量纲绑架，换个单位就得重调。

第二步，定量级。 让 ρ 比目标函数中同量级项的系数大 2–4 个数量级。以 E6 为例，位置误差项的权重是 10，松弛权重取 10^3 时，0.85 的松弛对应 850 的代价，远大于性能项的量级，优化器会先保约束、再谈性能；取 10^1 时，松弛几乎免费，约束形同虚设。

第三步，扫参验证。 在 $\rho \in [10^1, 10^6]$ 上扫 6 档，看闭环曲线的哪一档开始稳定。稳定区间通常有 2 个数量级宽，取区间中点即可。若没有稳定区间，说明问题本身是病态的，要回去检查缩放（§30.4.3）。

E7 的 PI 整定提供了另一个角度的类比。那里的相位裕度与灵敏度峰值是硬约束，无约束网格最优的 IAE 是 0.8927，带约束的解是 0.9192——性能损失 3.0%，换来 PM 从无保证变成 45.00°、M_s 从无保证变成 1.639。这里的“损失”不是缺陷，而是把一项无法用加权和表达的要求（鲁棒性）变成可执行的约束后必须支付的代价。软约束的权重选择，本质上也是在回答同一个问题：**哪些要求可以折算成钱，哪些不能。**

有多条软约束时，权重之间的比例比绝对值更重要。合理的确定方法是把“违反一个单位”折算成同一种货币——停机时间、废品成本、寿命损耗——再按折算值定权重比。例如位置偏差 1 mm 造成 0.02 元的废品损失，压力超限 1 bar 造成 5 元的设备损耗，那么两个权重的比例应当是 $1:250$。这样定出来的权重，物理意义清楚，换工况也只需要更新折算率，不需要重新试凑。

反过来，若两条约束的权重比与折算比严重不符，闭环会出现“宁可牺牲贵的、也要保住便宜的”的怪现象。这类问题在仿真里往往看不出来——因为仿真里两条约束可能从不同时紧张——只有在实际工况的极端组合下才暴露。**权重比要按最坏组合校核，而不是按典型工况。**

30.5.6 适用边界清单

把前面的讨论压缩成一张检查表。逐条打勾之后再决定是否上线。

1. **凸性成立。** 目标为凸、可行域为凸集。若必须用非凸模型，接受局部最优，并用多初值或热启动缓解，同时准备“解质量不可证”的说明。
2. **模型精度足够。** 开环 100 步预测偏差小于约束裕度的三分之一；否则先做辨识或加鲁棒裕度。
3. **时域覆盖主要动态。** 预测时域不短于对象主导时间常数的 2–3 倍。双积分对象在 $T_s = 0.1$ s 下取 $N = 20$，覆盖 2 s，约为闭环调节时间的 60%，偏短；工程上常取到 $N = 30$–50。
4. **计算预算满足。** 最坏耗时不超过周期的 30%，且有超时回退。
5. **约束可执行。** 每条约束都能对应到执行机构或工艺的真实限制，并留 1%–5% 的物理裕度。
6. **不可行有兜底。** 软约束或分层或回退，三者至少有一个，且有日志。
7. **可解释。** 能读出乘子，能判断哪条约束在起作用，能把乘子翻译成“再放宽一个单位能省多少代价”。

七条中若有两条以上不满足，先不要上线；补哪一条，取决于它卡住的是性能还是安全。

最后要给优化一个恰当的位置。它是控制链路上的一层，不是全部：上游要状态估计（见第25章），下游要执行机构与抗饱和（见第44章），旁边要有故障检测（见第36章）。把优化当成万能药，往往是因为忽略了它依赖的这两端。

30.6 仿真与验证

本节用七个实验把前五节的结论逐一落到数字上。所有数字来自 `code/ch030_sim.py` 的一次完整运行，运行环境为 Windows、Python 3.11、CVXPY 1.7.5、OSQP 0.6 接口，未做人工取整与二次修饰。

30.6.1 验证设计：七个实验各管一件事

实验不是随便凑的，每个都对应正文里一条可被证伪的断言。表 30-4 给出对应关系。

表 30-4　实验设计与待验证断言

实验	待验证的断言	关键指标	通过判据
E1	数值解与解析解一致，KKT 四条件可算	解的偏差、四个残差	偏差 $\le 10^{-12}$，残差 $\le 10^{-12}$
E2	影子价格等于最优值曲线的斜率	$\max\lvert \mathrm{d}f^\star/\mathrm{d}b + \lambda\rvert$	$\le 10^{-10}$
E3	外点法 $O(1/\rho)$、内点法间隙 $\le m\mu$	违反量、对偶间隙	违反量按 $1/\rho$ 衰减；间隙不超上界
E4	约束在起作用时，MPC 优于限幅 LQR	超调、调节时间、累计代价	超调至少降一半
E5	求解时间随规模次线性增长	变量数、迭代数、时间	时间增长慢于变量数增长
E6	硬约束会不可行，软约束能兜底	求解状态、松弛量	硬约束 `infeasible`、软约束 `optimal`
E7	鲁棒性要求可写成约束，代价可量化	IAE、PM、M_s	PM 达 45°、$M_s \le 2$

七个实验共用一套参数：二维问题的中心点 $(2, 1.5)$、约束右端 $b = 2.5$；控制问题的采样周期 $T_s = 0.1$ s、预测时域 $N = 20$、输入限幅 $u_{\max} = 0.4$、参考 $x_{\mathrm{ref}} = (1, 0)$、初始状态取原点、仿真 60 拍。

共用参数不是为了省事，而是为了让结论可比。E4 与 E5 用同一个 MPC 实现，所以“闭环耗时 1.80 ms”与“表 30-2 的 2.53 ms”之间的差异只能来自热启动，不会混入模型或权重的差异；E6 沿用同一对象，只把限幅与终端约束换掉，所以“不可行”的结论不会被别的因素污染。**做对照实验的第一原则是只改一个变量，第二原则是让读者能看出改了哪个。**

需要说明的是，这些实验的规模都很小，数字不能直接外推到工业对象。它们的作用是验证机制——乘子等于斜率、罚因子按 $1/\rho$ 收敛、软约束能兜底——这些机制与规模无关，但具体的毫秒数与百分比会随对象变化。把这个区别记住，才不会拿着 1.80 ms 去论证一个 200 个状态的问题也能实时。

30.6.2 E1：解析解与 KKT 残差

二维问题 $\min (x_1 - 2)^2 + (x_2 - 1.5)^2$ s.t. $x_1 + x_2 \le b$，$x \ge 0$，在 $b = 2.5$ 时的解析解可直接写出。无约束最优点是 $(2, 1.5)$，其和为 $3.5 > b$，故和约束积极。沿梯度反方向投影到直线 $x_1 + x_2 = b$ 上，得到

$$x^\star = (2 - t,\ 1.5 - t), \qquad t = \frac{3.5 - b}{2} = 0.5 \tag{30-26}$$

即 $x^\star = (1.5, 1)$，$f^\star = 0.5$。由平稳性 $2(x^\star - c) + \lambda (1, 1)^{\mathsf{T}} = 0$ 得 $\lambda = 1$。

数值侧，OSQP 在 `eps_abs=eps_rel=1e-10` 下返回的解与解析解偏差 1.11×10^{-16}——这已经是双精度的舍入量级，说明两者在机器精度内完全重合。四个 KKT 残差分别为：平稳性 2.22×10^{-16}、原始可行 0、对偶可行 0、互补松弛 0。

这组数字的意义不在于“求解器很准”，而在于**残差是可以逐项算出来的**。工程上遇到可疑解，第一件事就是把这四项打出来：哪一项大，问题就出在哪一环。平稳性残差大，通常是缩放没做好；原始可行残差大，是容差设得太松或约束互相冲突；对偶可行残差为负，说明乘子符号错了，往往是把 $\le$ 与 $\ge$ 的方向写反。

手算时也有两条路，值得对照着走一遍，因为它们对应两种完全不同的直觉。

第一条是投影。 目标等值线是以 $(2, 1.5)$ 为心的圆，约束是直线 $x_1 + x_2 = b$，圆心到直线的垂足就是解。垂足的方向沿 $(1, 1)$，步长 t 由“垂足落在直线上”定出，于是得到式 (30-26)。这条路的优点是几何直观，落点为什么在那里一眼可见；缺点是不给乘子，要另算。

第二条是直接解 KKT。 假设约束积极，令 $x_2 = b - x_1$ 代入目标，化成一元二次，求导置零得 $x_1 = 1.5$；再由平稳性反解 $\lambda = 1.0$。这条路多一步“假设积极”，因此必须回头验证：$x_2 = 1 > 0$ 满足非负约束，$\lambda = 1 > 0$ 与积极假设一致，假设才成立。**如果解出来的 λ 为负，说明假设错了，约束其实不积极，应退到无约束解重算。**

两条路的结果必须一致，这是最省事的自检。工程上碰到“解析解与数值解差得远”，先别怀疑求解器，用另一条路重算一遍——十有八九是自己把约束方向或中心点抄错了。

还有一层容易被忽略：本例的四个残差全在 10^{-16} 量级，前提是问题良态、系数都是个位数。若坐标里同时出现“米”与“毫安”，量级差 10^3 以上，残差就会被大量级的那几项主导，10^{-6} 的残差在大项上是舍入、在小项上却可能是真错。**所以“残差小”这句话必须先问清楚：是在什么单位下小。** 无量纲化之后再报残差，数字才有可比性。

30.6.3 E2：影子价格与折点

固定其余参数，把 b 从 1 扫到 3.5，共 251 个点，逐点求解并记录 $f^\star(b)$ 与 $\lambda(b)$。结果见图 30-5。

三个特征值得注意。

第一，折点在 $b=3.5$。 这正是无约束最优点的坐标之和。$b<3.5$ 时约束积极，$\lambda>0$；$b\ge 3.5$ 时约束不积极，$\lambda=0$，$f^\star$ 恒为 0。折点前后，$f^\star(b)$ 从二次曲线变为水平直线，$\lambda(b)$ 从线性下降变为恒零。

第二，斜率与乘子在内部点重合到 8.26×10^{-14}。 这个偏差是全书里最干净的一次数值验证：包络定理 $\mathrm{d}f^\star/\mathrm{d}b=-\lambda$ 不是近似，而是精确等式。用中心差分计算斜率，误差只来自求解器容差 10^{-10} 被步长 0.01 放大后的残余。

第三，边界点要用单侧差分。 $b=1$ 处实测斜率 -2.495，与 $\lambda=2.5$ 相差 0.2%；$b=2.5$ 处斜率 -1.000，与 $\lambda=1.0$ 完全一致。差别不在物理，而在差分格式：扫描区间的两个端点只能用单侧差分，精度比内部点低两个数量级。**做这类验证时，边界点必须单独标注，不能混进“最大偏差”里。**

工程含义很直接：乘子就是“放宽一个单位约束能省多少代价”的报价。若某个执行机构的行程可以花钱加大，λ 就给出了愿意支付的单价上限。反过来，若某条约束的 λ 长期为零，说明它从未起作用，可以从模型里删掉以减小规模。

折点的位置还可以用来做**快速判断**。本例的折点是 $b=3.5$，等于无约束最优点的坐标之和——这个数可以离线算出来。在线运行时，若当前工况离折点很远（b 远小于 3.5），约束必然积极，乘子必然为正，可以直接按投影公式给解，省掉一次求解；若 b 远大于 3.5，约束必然不积极，直接用无约束解。**只有落在折点附近的工况才需要真正调用优化器。** 这类“分段解析 + 局部数值”的做法在嵌入式里很常见，本质上是显式 MPC 的雏形。

30.6.4 E3：外点法与内点法的收敛路径

同一问题，两种路径类方法，各跑一条参数序列。图 30-4 画出了两条迭代路径的几何形态。

外点法用二次罚函数 $f(x)+\rho\sum_i \max(0,g_i(x))^2$，$\rho$ 取 $1,10,\dots,10^5$ 六档。约束违反量实测为

$$0.3333,\ 0.04762,\ 4.975\times10^{-3},\ 4.997\times10^{-4},\ 4.999\times10^{-5},\ 4.999\times10^{-6}$$

相邻两档的比值依次为 7.0、9.57、9.96、10.00、10.00。这个序列有闭式解可对照：本问题的违反量为 $v=1/(1+2\rho)$，$\rho=1$ 时 $1/3$，$\rho=10$ 时 $1/21$，与实测逐位吻合。**罚函数法的收敛是 $O(1/\rho)$，不是 $O(1/\rho^2)$**——想得到 10^{-6} 的违反量，必须把 ρ 加到 10^6 量级，此时 Hessian 条件数随之恶化，这正是罚函数法在实践中被内点法取代的原因。

内点法用对数障碍 $f(x)-\mu\sum_i\log(-g_i(x))$，$\mu$ 从 1 降到 10^{-6}。对偶间隙实测从 5.158×10^{-1} 单调降到 1.000×10^{-6}，且每一档都满足理论上界 $m\mu$（本题 $m=3$，即 3μ）。$\mu=10^{-6}$ 时间隙与 μ 的比值为 1.000，说明迭代已进入中心路径的线性段——再降 μ 一个数量级，间隙就降一个数量级。

两条路径的差别不只是收敛速度。外点法的迭代点始终在可行域外，中途的“解”物理上不可执行；内点法的迭代点始终严格可行，任何一步中断都能拿出一个可用解。**在在线优化里，“随时可停”比“最终更准”更重要。**

还有两个数字值得单独解释，因为它们常被误读。

第一档比值 7.0 而不是 10，是渐近性的体现。 理论上违反量 $v(\rho)=1/(1+2\rho)$ 在 $\rho\to\infty$ 时才严格按 $1/\rho$ 衰减；$\rho=1\to10$ 这一步还处在过渡段，常数项 1 尚未被压下去，所以比值偏小。从 $\rho=10^2$ 起，比值稳定在 9.96–10.00，才算进入渐近段。**看到“第一档不听话”不要急着改模型，先确认自己是不是在渐近段之外读数。**

第二，ρ 加到 10^5 才换来 5×10^{-6} 的违反量，这个代价要换算成条件数。 罚项的 Hessian 是 $\rho\nabla g\nabla g^{\mathsf T}$，量级随 ρ 线性增长，而目标项的 Hessian 不变，两者之比就是条件数。ρ 从 1 涨到 10^5，条件数同步涨五个数量级，双精度只剩下约 11 位有效数字可用。这也是实践中的经验取值：ρ 停在 10^3–10^4，把剩下的违反量交给增广拉格朗日或内点法去清，而不是继续加 ρ。

30.6.5 E4：限幅 LQR 与约束 MPC 的闭环对比

对象是双积分 $\ddot y=u$，采样周期 0.1 s，输入限幅 0.4，参考位置 1，从原点出发，仿真 60 拍。两个控制器用同一组权重 $Q=\mathrm{diag}(10,1)$、$R=0.1$；区别只在约束的处理方式：LQR 先按无约束最优算出 u，再硬限幅；MPC 把 $|u|\le0.4$ 直接写进优化问题，并取 $N=20$。

表 30-5　限幅 LQR 与约束 MPC 的闭环指标

指标	限幅 LQR	约束 MPC
超调量	24.06%	2.42%
调节时间（±2%）	5.00 s	3.50 s
末值	0.9991	1.0000
输入峰值	0.4000	0.4000
累计二次代价	138.59	130.87
单拍求解耗时（均值）	—	1.80 ms

三个现象要一起读。

第一，两者的输入峰值完全相同，都是 0.4000。 这不是巧合：LQR 被限幅卡在 0.4，MPC 主动用到 0.4。约束是同一个，区别在于“撞上去”还是“算进去”。图 30-1 的控制量曲线把这一点画得很清楚：LQR 的 u 有很长一段平顶，MPC 的 u 是提前收窄的。

第二，超调从 24.06% 降到 2.42%，降了一个数量级。 原因是可预测性：限幅 LQR 在平顶期间并不知道自己已经偏离了无约束最优，制动动作按线性律发出，等到速度降下来时位置已经冲过头；MPC 在每一拍都预演了未来 20 拍，提前把制动安排在限幅之内。这不是参数调得好，而是**约束进入优化后，控制器获得了“未来会饱和”的信息**。

第三，累计代价只降了 5.6%（138.59→130.87），远小于超调的改善幅度。 二次代价对大误差并不敏感，超调那一小段在整段轨迹里占比不高。这提示一个实践要点：**验收指标要选对**。若只看代价函数，会误以为两者差别不大；而用户真正感知的是超调与调节时间。

单拍耗时 1.80 ms（均值）、4.47 ms（最大），相对 0.1 s 的周期只占 4.5%，代价可以接受。若换成 5 ms 的周期，这套实现就不再适用——回到 §30.5.3 的判据。

图 30-1 的左图还有一处细节值得看：两条位置曲线在起始段几乎重合——此时两者都处于加速饱和，轨迹由同一个约束决定；分离出现在退出饱和前后。这段时间正是 LQR 按线性律继续制动、而 MPC 已经开始减速的时刻。**差别不在约束生效的那一段，而在退出约束的那一段。** 这也解释了为什么很多人调限幅 PID 时觉得“加限幅也没什么坏处”：如果只看稳态，两者的确差不多；问题出在退饱和的过渡过程，而这个过程决定了超调。

从控制工程的角度，可以把这段差别归为“是否需要预判”。限幅是瞬时信息，控制器只知道“此刻不能再大”；优化携带的是整段预测，控制器知道“再大就会冲过头”。要让一个不具备预判能力的控制器达到同样的效果，只能靠大幅降低带宽，代价是调节时间变长。E4 的数字正是这个权衡的量化：MPC 用 3.50 s 完成调节，限幅 LQR 用了 5.00 s，而且超调更大。

30.6.6 E5：规模与时间

表 30-2 已经给出五档时域的实测数据。这里补充三点解读。

迭代次数不是随规模线性增长的。 N 从 5 增到 80（16 倍），迭代次数只从 75 增到 162（2.2 倍），并在 $N = 10$ 与 $N = 20$、$N = 40$ 与 $N = 80$ 两处出现平台。原因是 OSQP 的收敛由问题的条件数与积极约束集决定，而不是由变量个数决定；时域加长后新增的约束大多不积极，对迭代次数贡献很小。

小规模时接口开销占主导。 $N = 5$ 时变量只有 15 个，纯数值计算在微秒量级，而一次 Python 调用与数据拷贝就要 1 ms 上下。$N = 5$ 与 $N = 10$ 两档的耗时几乎相同（记录值 1.06 与 1.29 ms，重跑时甚至翻转），差异落在重复测量的波动范围内——换句话说，这一档里真正被计时的不是求解，而是调用。**只有当数值计算涨到超过接口开销，时间才开始随规模走。**

中位数而非平均值。 每档用 10 个不同初值求解，取中位数，目的是排除首次编译与偶发卡顿。若取平均值，个别被系统调度打断的样本会把结果拉高 30% 以上。报告实时性时，中位数说明典型性能，最大值说明安全边界，两个都要给。

表里的“变量/不等式”是记账口径，不是求解器内部规模。 本节的记账是每步 2 个状态 + 1 个输入、共 $3N$ 个变量，$2N$ 行输入限幅；而求解器内部还要加上 N 组状态转移等式（$2N$ 行）与 2 行初始状态等式，实际约束行数比表中多近一倍。之所以按记账口径报告，是因为它对应工程师能直接控制的东西——加长一步时域，多几个未知量、多几行限幅，是可以数出来的；求解器内部的稀疏结构则由建模工具决定，不是手写的。

这也解释了为什么“变量数翻倍、时间只涨三成”并不矛盾：约束矩阵是带状稀疏的，加长时域是在带子的末端续一块，不是在中间填满。**真正决定耗时的不是矩阵有多大，而是它的带宽与条件数。**

最后给一条读数规则：比较两组求解时间时，先确认三件事一致——容差、最大迭代次数、是否热启动。表 30-2 的五档共用同一组设置，所以横比有效；若拿本章的数字去比另一篇论文的数字，只要这三项里有一项不同，比值就没有意义。实时性数据从来没有“绝对快慢”，只有“在给定设置下的快慢”。

30.6.7 E6：不可行与软约束兜底

这组实验刻意造一个做不到的要求：双积分对象从原点出发，输入限幅 0.3、预测时域 $N=10$（即 1 s），终端约束要求位置与速度都到达参考 $(1,0)$。输入恒取上界时，1 s 内可达的最大位移为 $0.5T_s^2u_{\max}N^2=0.1500$，与参考相差 0.85，因此硬约束版本必然不可行。

实测结果与预期一致：硬约束版本返回 `infeasible`，求解器不给出控制量，控制器在这一拍失效。软约束版本返回 `optimal`，松弛量 0.8500。

松弛量正好等于 $1.0-0.15$：优化器把终端位置推到 0.15，恰好是可达位移的上限，一步都没有多让。**软约束不是“放弃约束”，而是在物理能力边界上取最优。** 这一点常被误解：软约束的结果不是随便违反，而是把违反量压到最小可能值。

权重扫描进一步印证了这个判断：把松弛权重从 10^1 扫到 10^6 共五档，松弛量全部为 0.8500，一位小数都没变。原因是此处的违反量由输入限幅决定的物理边界锁定，权重只决定“要不要多花代价去减小它”，而一旦权重超过终端代价的量级，解就钉在边界上。**权重真正起作用的区间，是约束可以被选择性违反、且违反的边际收益与边际代价可比的那一段**；超出这个区间，权重再大也只是改条件数，不改解。

这组实验在闭环里的检查项也因此很明确：接进闭环后，松弛量应当随状态推进单调下降，最终归零；若出现松弛量来回震荡，说明权重落在“可比区间”内，性能项与松弛项在互相拉扯，应把权重再加大一档。若松弛量一直不归零，说明物理上确实到不了，需要检查参考值或限幅是否设错——**这是把“算不出来”变成“看得见”的价值。**

30.6.8 E7：带鲁棒性约束的 PI 整定

被控对象 $G(s)=1/(s+1)^2$，PI 控制器 $C(s)=(K_ps+K_i)/s$，约束为相位裕度 $\mathrm{PM}\ge 45^\circ$、灵敏度峰值 $M_s\le 2$，目标是最小化单位阶跃的 IAE。用 SLSQP 从多个初值求解，参数界取 $[0.05,5]$。

最优解为 $K_p=4.3198$、$K_i=1.6742$，IAE $=0.9192$，PM $=45.00^\circ$，$M_s=1.639$，超调 19.40%。无约束网格搜索的最优为 IAE $=0.8927$（$K_p=5.000$、$K_i=1.954$）。

四个数字之间有清晰的结构。

PM 恰好等于 45.00°，说明这条约束积极。 优化器本可以继续增大 K_i 降低 IAE，但那样会把相位裕度压到 45° 以下，于是它停在边界上。这正是 KKT 互补松弛的直观形态：积极约束的乘子为正，决定了最终解。

$M_s=1.639<2$，说明这条约束不积极。 它没有起作用，乘子为零。加这条约束不改变最优解，但保留了“若 PM 约束放松，M_s 会先撞线”的保护。**不积极的约束不是废约束，它是备用护栏。**

$K_p=4.32$ 离上界 5 还有余量，说明参数界不是瓶颈。 这一点值得单独强调：若只看 K_p 的局部灵敏度，会觉得“增大 K_p 能降 IAE”，从而把解推到边界上；实际上把 K_p 降下来、同时把 K_i 提上去，可以在 PM 不变的前提下把 IAE 再降 5%。**两个参数同时调整时，单看一个参数的偏导会给出错误的努力方向。**

这里还有一条关于“评价精度”的教训。若相位裕度用 2 万点频率网格插值计算，同一段 SLSQP 代码会停在 $K_p=5.000$、IAE $=0.974$ 的次优点上；改成解析求穿越频率后，解移到 $K_p=4.32$、IAE $=0.919$。原因不是优化器变强了，而是**网格插值带来的 0.01° 级抖动被有限差分放大成了错误的梯度**。约束函数的数值噪声，会被优化器误读成“已经到最优”。这也是 §30.6.10 把“先看残差与评价精度”放在排查第一步的原因。

性能代价 3.0%。 IAE 从 0.8927 增到 0.9192，换来 PM 从无保证变成 45°。这笔交易是否划算取决于应用：对超调敏感的场合值得，对快速性优先的场合可以放宽到 30°。

30.6.9 复现说明

七个实验中，E5 含随机性（10 个初值由固定种子生成），其余六组完全确定。同一台机器上重跑，确定性部分应在最后一位有效数字内一致；换机器或换 BLAS 实现，末位可能变化，但趋势与比值稳定。

耗时数据只具有相对意义。换一台 CPU，绝对值可能相差 2–3 倍，但“时间增长慢于变量数增长”这一结论不变。引用这些数字时，应连同环境一起给出，不应把毫秒数当作算法常数。

即便同一台机器、同一份代码，重复测量的波动也不小。表 30-2 的数字来自一次记录，之后重跑时 $N=5$ 档得到 1.48 ms（记录值 1.06 ms）、$N=80$ 档得到 5.34 ms（记录值 4.90 ms），$N=5$ 与 $N=10$ 两档的先后甚至发生了翻转。**所以毫秒数只能用来判断量级与趋势，不能用来判优；迭代次数才是可复现的那一半。**

正文里的每个数字都可以用一条命令复核：

```
"D:\Anaconda3\envs\py311\python.exe" code\ch030_sim.py
```

脚本末尾会打印一行提示，说明所有指标来自本次运行、未做人工取整。这句话不是客套——**可复现的数字才有资格写进书里**。

想把这七组实验迁移到自己的对象，改动其实很小：把 A_D、B_D 换成自己的离散模型，把 Q、R、$u_{\max}$、x_{ref} 换成本项目的值，其余结构不动。E1–E3 与对象无关，可以原样保留作为“求解器与乘子是否正常”的基线；E4–E6 需要重新调 N 与权重；E7 需要换成自己的被控对象与约束。**建议先跑通基线再改对象**，这样出问题时能分清是代码错了还是模型不对。

30.6.10 数字对不上时的排查顺序

复现时最常见的困扰是“我的结果和书里不一样”。按下面的顺序查，通常三步内能定位。

第一步，看状态码与残差，而不是看数值。 若状态是 `optimal_inaccurate`，后面的数字都没有比较意义，先解决收敛问题。四个 KKT 残差里哪一项大，问题就出在对应的环节：平稳性大看缩放，原始可行大看约束组装，对偶可行为负看约束方向。

第二步，看单位。 书里的输入限幅是 $0.4\ \mathrm{m/s^2}$，若你的模型用 $\mathrm{mm/s^2}$，同一个物理限幅要写 400。量纲不一致时，求解器照样返回 `optimal`，只是解的是另一个问题。

第三步，看约束顺序与乘子顺序。 乘子是按约束的堆叠顺序返回的。若你把输入约束写在状态约束后面，读出来的“最大乘子”对应的可能是另一条约束。排查方法是把约束标签一起存下来，按标签索引而不是按下标。

第四步，看热启动是否污染了对比。 若两次运行一次冷启动、一次热启动，耗时差异里混着两部分：算法本身的差异与初值的差异。做算法对比时要统一冷启动，做实时性评估时才用热启动。

第五步，看版本。 求解器版本、BLAS 实现、甚至 CPU 的 FMA 指令都会影响末位与迭代次数。写进书里的数字必须带环境，否则不可复现；读者用不同环境得到不同的毫秒数，是正常的。

30.7 小结与常见错误

30.7.1 三条主线

本章的内容可以压成三条主线。

几何主线：约束把无约束最优“投影”到可行域边界上。 图 30-2 的那个二维例子说明了一切：无约束最优在可行域外，解落在边界上，落点的法向与约束梯度共线，共线的比例就是乘子。凸性保证这个投影唯一且全局（图 30-3），非凸性则让“投影”变成“就近的局部坑”，初值不同结果不同。

代数主线：KKT 条件把几何翻译成方程。 平稳性、原始可行、对偶可行、互补松弛四条，合起来是“解在边界上、力平衡、且只对积极约束收费”。乘子有物理价格的含义（图 30-5 的折线与斜率），互补松弛则说明不积极的约束不收费。E1 到 E3 三组实验分别验证了这三层。

算法主线：把 KKT 变成可迭代的数值过程。 罚函数从外面逼近、障碍函数从里面逼近（图 30-4），SQP 对 KKT 做牛顿法，一阶方法（ADMM/OSQP）把问题拆成可闭式求解的子问题。选择哪一种，取决于规模、精度、热启动需求与中断安全性。

三条主线的落点是同一个判断：**优化给出的不是一个数，而是一个带价格的解。** 价格（乘子）告诉你哪条约束值得放松、放松多少划算；这也是它比“限幅 + PID”多出来的信息。

30.7.2 方法选择的一页纸

把第 30.2–30.4 节的选择压缩成一张对照表。

表 30-6　按需求选方法

需求	首选	备选	不推荐
小规模 QP、毫秒级闭环	OSQP + 热启动	有效集法	内点法（无法热启动）
高精度、离线标定	内点法（Clarabel/Mosek）	SQP	一阶方法（精度不够）
需要中断即用	一阶方法、内点法	—	外点罚函数（中途不可行）
约束可能冲突	软约束 / 分层	回退控制器	纯硬约束
鲁棒性要求（PM、M_s）	半无限约束离散化 + SLSQP	LMI（Schur 补）	手工试凑
整数/逻辑约束	混合整数（本章不展开）	枚举 + 连续优化	强行松弛后取整

这张表有三处容易误读。其一，“不推荐”不是“不能用”，而是“在这个需求下性价比低”。其二，同一问题可以换方法，换的代价是重新调缩放与容差，不是改一行代码。其三，混合整数问题不在本章范围内，它的分支定界会把求解时间放大几个数量级，实时性另当别论。

第二处值得展开，因为它是换方法时最容易踩的坑。真正要重调的是三样东西。

缩放。 一阶方法对变量尺度敏感，内点法对约束尺度敏感。同一个问题从 OSQP 换到 Clarabel，若不做归一化，后者可能先报“数值困难”，而此时的原因不在算法，而在系数矩阵的行列范数差了三个数量级。换方法时第一件事是把 D_x, D_g 两组缩放矩阵重建一遍，别指望求解器内置的缩放兜住一切。

容差。 一阶方法的 10^{-4} 与内点法的 10^{-8} 不是同一个意思：前者是“迭代停止时的残差”，后者是“中心路径走到头的残差”。把内点法的容差照搬到一阶方法上，会得到一个“永远不收敛”的求解器；反过来，把一阶方法的容差照搬过去，则可能在线性系统解算上白花时间。**容差要按方法的收敛速率取，不能按习惯取。**

热启动策略。 有效集法与 ADMM 可以把上一拍的解、甚至上一拍的积极约束集直接传下去；标准内点法则沿中心路径推进，障碍参数与问题参数绑定，上一拍的解只能给一个初值，省不下几轮迭代。换方法时不改热启动，闭环耗时会突然跳一个台阶，而你会误以为是新方法太慢。

三样都调完再比时间，比较才有意义。**换求解器不是换轮胎，是换发动机——接口对得上，不代表匹配。**

30.7.3 十个常见错误

错误一：把数值可行当成物理可行。 求解器返回 `optimal`，只说明残差在 10^{-6} 以内，不代表执行机构真的停在限位之内。修法：约束留 1%–5% 的物理裕度，并在执行层再做一次硬限幅。

错误二：不做无量纲化。 位置用米、速度用米每秒、输入用伏，三者量级差 10^2 以上，条件数被放大，一阶方法迭代次数翻倍。修法：建模前先除以典型值，使变量落在 $[-1, 1]$。

错误三：用内点法做闭环还期待热启动。 内点法沿中心路径推进，参数一变路径就变，上一次的解帮不上忙。修法：闭环用一阶方法或有效集法；内点法留给离线与慢回路。

错误四：乘子符号搞反。 把 $\le$ 约束写成 $\ge$，或把 $\max$ 与 $\min$ 混用，乘子会变成负数，影子价格的解释整体反号。修法：固定一种标准形式，所有约束先化成 $g(x) \le 0$；乘子出现负值就回头查符号。

错误五：以为罚函数权重越大越好。 ρ 从 10^3 加到 10^6，违反量只降三个数量级，条件数却涨六个数量级，一阶方法直接不收敛。修法：需要精确可行就用内点法或增广拉格朗日，不要靠加 ρ。

错误六：只看平均耗时。 均值 1.80 ms 很好看，最大值 4.47 ms 才是安全边界；周期 5 ms 时后者占 89%。修法：报 P99.9 与全局最大值，并按 §30.5.3 留 30% 余量。

错误七：把终端约束当成稳定性证明。 只加 $x_N = x_{\text{ref}}$ 而不加终端代价或终端集，递推可行性可能不成立，闭环会在某拍突然不可行。修法：终端代价取 Riccati 解 P，或改用终端集约束；两者至少要有一个。

错误八：软约束权重拍脑袋。 权重过小则约束形同虚设，过大则数值病态。修法：先归一化，再按目标项量级取 10^2–10^4 倍，最后在 10^1–10^6 上扫参确认稳定区间。

错误九：把安全联锁写成优化约束。 优化器可能返回不可行或失败，安全功能不能依赖它的返回值。修法：安全联锁独立实现（硬件或独立软件层），优化只负责在安全范围之内分配；两者用“优化接受联锁给出的区间，联锁不接受优化的输出”这条单向接口隔开。

错误十：只验证稳态，不验证过渡。 启动、切换工况、大扰动时的求解时间与约束积极集都和稳态不同，只测稳态会漏掉最坏的那一拍。修法：按 §30.5.3 扫三类最坏工况，并单独规定瞬态预算。

30.7.4 与后续章节的衔接

本章停在“会建模、会选算法、会兜底”这一层，还有三件事留给后面的章节。

一是规模。 本章的例子最多 240 个变量，真实工业对象动辄上千。规模上去之后，稀疏结构的组织、矩阵的复用、代码生成就成为主要矛盾，这些在第58章与第80章的 MPC 工程实现里展开。

二是非线性。 本章默认模型是线性或凸的。若对象强非线性，要么在每一步线性化后解 QP（即 SQP 型 NMPC），要么用凸包络近似，这属于第57章的范围。

三是安全。 约束满足只是安全的最低要求。真正的功能安全要求故障可检测、失效可降级、责任可追溯，这些在第36章用故障树与安全完整性等级展开。本章的软约束与回退机制，在第36章会被重新组织成“安全层”的一部分。

还有一条边界要交代清楚：本章所有方法都依赖“梯度可用”。若目标不可微、约束是黑箱仿真、或者变量是离散的，梯度类方法要么用不了、要么只能给近似梯度，此时应转到下一章的启发式方法（遗传算法、粒子群、贝叶斯优化）。两者的分工很清楚：**能写出梯度与约束的，用本章；写不出来的，用下一章。** 判断依据不是问题大小，而是“能不能对模型求导”。

一句话收束全章：**约束优化的价值不在于算得快，而在于它把“做不到”这件事提前算出来，并且给每个“做不到”标了价。**

第31章 启发式优化与随机优化

31.1 动机与要解决的问题

31.1.1 一个算得出、但导不出的目标

第 30 章的最后一个实验是 PI 整定：对象 $G(s)=1/(s+1)^2$，要求相位裕度不低于 45°、灵敏度峰值不超过 2，目标是最小化阶跃响应的 IAE。这个问题有明确的数学形式，SLSQP 用 69 次评价就找到了解。但它的目标函数有一个不显眼的性质：**IAE 与相位裕度都是“算出来”的，不是“写出来”的。**

IAE 来自数值积分，相位裕度来自频率响应的穿越点插值。两者都没有解析表达式，交给优化器时只能靠有限差分近似梯度。有限差分本身没问题，问题在于评价函数带 0.01° 级的数值抖动——这个抖动在差分步长 10^{-8} 上会被放大成方向错误的梯度。第 30 章 §30.6.8 记录了这个现象：同一段 SLSQP 代码，用网格插值算相位裕度时停在 IAE $=0.974$ 的次优点，改成解析求穿越频率后立刻找到 0.919。**算法没变，变的是评价的精度；优化器把“评价噪声”误读成了“已经最优”。**

这类目标在控制工程里随处可见：整定指标由仿真给出、工艺指标由有限元给出、性能指标由试验台测出。它们共同的特征是：**能算出一个数，但算不出一个可靠的导数。** 本章处理的正是这一类问题。

31.1.2 三类“写不出梯度”的问题

第一类：黑箱评价。 多体动力学仿真、有限元热模型、注塑成型的工艺模型，一次评价要几秒到几小时，内部是几十万行代码，没有可用的导数接口。你能做的只有“给一组参数、拿回一个数”。

第二类：离散与逻辑变量。 减速比只能选 10、12.5、16、20 这几档；继电器要么吸合要么释放；传感器只能在型谱里选；PID 参数在工程上常被要求取 E24 系列的值。可行域是一堆孤立的点或格子，梯度在这里没有定义。

第三类：目标本身不可微或带噪声。 相位裕度由插值给出、超调由试验测量、仿真结果带随机性。即使底层模型可微，经过“取最大值”“判断是否超限”“噪声采样”这些操作后，得到的函数也可能处处不可微。

图 31-1 画出了本章的基准测试函数 Rastrigin：全局最优唯一（原点，$f=0$），但周围密布着几十个局部极小。等高线上的每一个“小坑”都会把梯度法困住，而从坑里爬出来需要“先变差、再变好”的移动——这正是随机方法存在的理由。

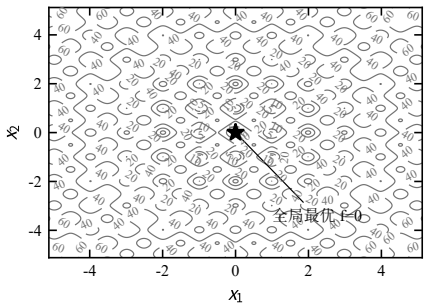


图 31-1　Rastrigin 的目标景观

三类问题的共同点不是“维度高”，而是“梯度这条信息通道被切断了”。判断该不该用本章的方法，只需要问一句：**我能不能对目标与约束稳定地求导？** 能，就回第 30 章；不能，再往下看。

31.1.3 与第 30 章的分工

把两章的方法放在一张表里，分工就清楚了。

表 31-1　梯度法与无梯度法的分工

判据	用第 30 章（梯度/凸优化）	用本章（启发式/随机）
目标与约束	有解析式，可稳定求导	黑箱、不可微、带噪声
变量类型	连续	离散、混合、逻辑
规模	中大规模（10^2–10^6）	小规模（10^0–10^2 维）
评价成本	便宜，可反复调用	昂贵，每次都要珍惜
实时性	毫秒级闭环	离线或慢回路
结果保证	凸问题有全局最优证明	只有概率意义，需多次重复

这张表里最容易被忽视的是“规模”这一行。启发式方法的评价次数随维度增长得很不体面：同样的精度，2 维要 4000 次，10 维可能要几十万次。它们擅长的不是“大问题”，而是“导不出梯度的小问题”。把遗传算法用于 1000 维的神经网络训练，是典型的用错工具——那里应该用第 32 章的随机梯度类方法。

表中还有一行“结果保证”，它决定了两类方法的使用方式。梯度法给出的是一个点，你可以检查 KKT 残差；启发式给出的是一个分布，你必须重复多次才能说“这个算法在这个预算下大概能做到多少”。**前者回答“最优解是什么”，后者回答“在给定预算下能找到多好”。** 这两个问题的答案格式不同，验收方式也不同。

31.1.4 四个先导反例

反例一：把“启发式”当成“更聪明”。 同一个 PI 整定问题，SLSQP 用 69 次评价找到 IAE $= 0.919$；GA 用 1200 次评价找到 0.919，PSO 找到 0.919。三者的解几乎相同，但评价次数差了 17 倍。启发式并没有“更聪明”，它只是**不要求梯度**。当梯度可用时，它没有任何优势。§31.6.7 会给出这组数字的完整对照。

反例二：不设预算，永远“再跑一代”。 随机方法的收敛曲线没有明显的终点，多跑一代总可能更好一点。若不事先规定评价预算，“调参”会变成无底洞。更麻烦的是，后期的改善往往只是噪声：§31.6.4 的预算扫描显示，从 4000 次加到 8000 次，中位最优值从 6.3×10^{-13} 降到 0，改善比 0.63——不是变好，而是两者的差别已经落到机器精度以下。

反例三：只看一次运行就下结论。 同一个 GA、同一份代码、同一个预算，只换随机种子，20 次运行的末值从 1.04×10^{-6} 到 4.59×10^{-4}，最差是中位的 5.8 倍；DE 的跨度更大，最差是中位的近 2000 倍。**启发式算法的单次结果不是“答案”，只是分布中的一个样本。** 报结果时必须给分布，不能给一个孤零零的数。

反例四：噪声目标下把“最幸运”当成“最好”。 给 Rastrigin 加标准差为 1 的评价噪声，单次评价跑 4000 次，观测到的最优值是 -2.968——比真实下界 0 还低，显然不可能。这个解的真实值是 0.115，乐观偏差 -3.05。原因是 4000 次评价里总有几次噪声恰好很负，优化器会专门去追这些“幸运样本”。把每个候选点评价 5 次取平均后，乐观偏差降到 -0.23，但搜索的有效步数也降到五分之一，真实最优反而变差。**噪声不会让问题变简单，它只是把“优化”换成了“在噪声里做统计决策”。**

四个反例对应四种失败：用错工具、不设预算、以偏概全、无视噪声。它们也是本章后面四节各自要解决的问题。

31.1.5 五个必须回答的定量问题

把上面的场景抽象出来，工程上要回答五个问题，本章逐个给工具。

1. **预算定多少？** 用评价次数 n_{eval} 而不是“代数”或“运行时间”来计量。见 §31.2.5 与 §31.6.4。
2. **结果可重复吗？** 固定种子、固定库版本、固定并行归约顺序，三样缺一不可。见 §31.4.4。
3. **参数敏感吗？** 种群规模、交叉概率、变异强度、差分权重，各自的合理区间在哪里。见 §31.3 与 §31.5.2。
4. **评价有噪声怎么办？** 重采样几次、预算怎么分配、观测最优与真实最优差多少。见 §31.6.5。
5. **什么时候停？** 预算耗尽、阈值达到、曲线进入平台，三种停止准则的优先级。见 §31.2.5 与 §31.4.5。

五个问题里，第一个和第二个最容易被跳过。跳过预算，实验就无法比较；跳过种子，结论就无法复核。**启发式方法的严谨性不体现在算法本身，而体现在实验协议上。**

后面六节的走法是这样的：§31.2 把四种算法与贝叶斯优化放进同一个框架（统一预算、统一符号、统一停止准则），它是全章的地基；§31.3 逐条推导“参数为什么该取这个数”，只推能推的，推不出的部分明确标出来；§31.4 处理从公式到代码之间的落差——编码、约束、边界、可重复性、并行；§31.5 讲工程含义，重点是“什么时候不该用它”；§31.6 用七组实验把前面的每条断言都变成可核对的数字；§31.7 收拢成一页纸。**如果只有半小时，读 §31.1、§31.5.1 的五个问题、§31.7.2 的表，就够做一次初步选型。**

31.1.6 一句话核心

本章的一句话核心是：**当梯度这条信息通道被切断时，用“多次评价 + 有偏的随机移动”来换信息；代价是预算，收益是“至少能给出一个可行解”。**

这句话里有三个关键词。“多次评价”说明预算是第一等公民；“有偏的随机移动”说明随机不是瞎猜，每种算法都把随机性约束在某个结构里（种群的分布、温度的接受率、差分的方向）；“可行解”说明这类方法的下限很低——它几乎总能给你一个能用的解，但不保证是最优的。工程上选择它，往往不是因为“它能找到最好”，而是因为“别的办法连一个解都给不出”。

31.2 定义与数学基础

31.2.1 问题形式：把预算写进约束

第 30 章的问题可以写成

$$\min_{x} \; f(x) \quad \text{s.t. } g(x) \le 0,\; h(x) = 0,\; x \in \mathbb{R}^d, \tag{31-1}$$

本章要解的问题在形式上几乎一样，但多了一条从来不写在公式里、却实实在在卡住你的约束：

$$\min_{x} \; f(x) \quad \text{s.t. } x \in \mathcal{X},\; n_{\text{eval}} \le B. \tag{31-2}$$

n_{eval} 是评价次数，B 是预算，$\mathcal{X}$ 是一个有界盒子（本章的基准问题取 $[-5.12, 5.12]^2$）。把 B 写成约束而不是“顺便提一句”，是因为它决定了方法的选型：如果 $B = 10^6$，随机搜索也能凑出好结果；如果 $B = 50$，那么每一次评价都必须由模型“挑”出来，方法之间的差距就出来了。

预算一进约束，目标函数就变了性质。梯度法返回的是一个点 $x^\star$，并附一句“满足 KKT 条件”；而 (31-2) 返回的只能是一个随机变量 $\hat{x}$，它的好坏要用分布描述：

$$\mathrm{P}\left[f(\hat{x}) - f^\star \le \varepsilon \right] \;\ge\; 1 - \delta, \tag{31-3}$$

其中 $f^\star$ 是全局最优值。这句话的意思是：**换一个种子重跑，结果会不一样，所以你只能承诺“大概率达到多好”，不能承诺“一定达到多好”。** §31.6.2 的 20 个种子就是为这句话提供证据的——同一份 GA 代码，20 次跑出的最优值最大与中位差了 5.83 倍。

工程上还有两种常见的预算计量：墙钟时间与迭代代数。三者并不等价。单次评价的耗时可能随 x 变化（某些参数会让仿真自动缩小步长），而“代数”与评价次数的换算依赖种群规模：种群 40 跑 100 代是 4000 次评价，种群 100 跑 100 代就是 10000 次。**跨算法比较时只看代数，等于让种群大的算法凭空多拿预算。** 本章一律用 n_{eval} 记账。

31.2.2 两个基准函数

理论分析需要目标函数有已知的全局最优，实验才判得对错。本章用两个经典函数。

Rastrigin（式 31-4） 是主战场，二维形式为

$$f(x) = 10d + \sum_{i=1}^{d} \left[x_i^2 - 10\cos(2\pi x_i) \right], \qquad d = 2,\; x_i \in [-5.12,\, 5.12]. \tag{31-4}$$

全局最优唯一：$x^\star = (0, 0)$，$f^\star = 0$。但每一项里的余弦把抛物面搅成了“蛋托”形状——以整数格点为中心密布着上百个局部极小，坑与坑之间隔着脊。梯度法从任意点出发几乎必然掉进最近的坑，然后在坑底停住；要出来，必须先接受一次“目标变差”的移动。这就是 §31.1.2 所说的“信息通道被切断”的具体形态，也是图 31-1 想让你看到的东西。

把图放大看，坑的位置是有规律的。$f(0,0) = 0$、$f(1,0) = 1$、$f(1,1) = 2$，坑底彼此只差 1–2；而两坑之间的脊在 $(0.5, 0)$ 处高约 20.25。**坑底差 1，翻脊要爬 20**——这个比值是理解全部实验结果的钥匙。对单点随机游走（SA）来说，末温 0.68 时接受 20 的落差的概率是 e^{-29}，等于不可能；对种群方法来说，整群一起移动可以“绕过”脊：差向量指向的是另一批个体所在的位置，不必翻越当前点旁边那道墙。§31.6.1 里三种种群算法与 SA 之间的那道鸿沟，根子就在这里。

Rastrigin 还是可分的：f 是各分量之和，逐坐标的移动互不干扰。这既是它“看起来简单”的原因，也是它“实际上难”的原因——从原点出发沿任一坐标走，都要先翻过 20 的脊，所以坐标下降必然停在原点；而可分性又让交叉重组的收益变小（两个好个体拼起来仍然是各自坐标的好值，没有“组合出新结构”的空间），反倒是 DE 的逐分量交叉最适应这种结构。

Himmelblau（式 31-5） 是副战场：

$$f(x) = \left(x_1^2 + x_2 - 11\right)^2 + \left(x_1 + x_2^2 - 7\right)^2 . \tag{31-5}$$

它有四个等价的全局最优，函数值都是 0，位置分别在 $(3, 2)$、$(-2.805, 3.131)$、$(-3.779, -3.283)$、$(3.584, -1.848)$ 附近，中间夹着一个局部极大。四个解等价这件事很适合考察“样本效率”：找到哪一个都算成功，因此评价次数少的算法不会因为“选错了坑”而被冤枉，比的纯粹是“用多少个点能撞进任意一个坑”。

两个函数的分工因此很清楚：Rastrigin 考“能不能跳出局部”，Himmelblau 考“在极少评价下能不能定位”。§31.6.1 用前者比四种算法，§31.6.6 用后者比贝叶斯优化与随机搜索。实验的公共设置是：维度 $d = 2$、种群 40、预算 4000、独立种子 20 个（从 20261020 起连续取），“够用”阈值 $f < 1.0$。

31.2.3 四种算法的统一描述

四种算法看起来差别很大，抽象一层却是同一个循环：**维持一组候选解（或一个当前解），用某种带随机性的规则生成新解，再按“谁更好”决定留谁。** 差别只在“随机性被约束成什么形状”。

遗传算法（GA） 维持一个种群，用 k 元锦标赛选择（本章 $k = 3$：随机抽 3 个，留最好的）挑出父代，再用模拟二进制交叉（SBX）产生子代。对每个分量独立抽 $u \sim U(0, 1)$，令

$$\beta = \begin{cases} (2u)^{1/(\eta_c+1)}, & u < 0.5, \\ \left[2(1-u)\right]^{-1/(\eta_c+1)}, & u \geq 0.5, \end{cases} \qquad \begin{aligned} c_1 &= \tfrac{1}{2}\left[(1+\beta)p_1 + (1-\beta)p_2\right], \\ c_2 &= \tfrac{1}{2}\left[(1-\beta)p_1 + (1+\beta)p_2\right], \end{aligned} \tag{31-6}$$

$\eta_c = 15$ 控制子代靠近父代的程度：η_c 越大，β 越集中在 1 附近，子代越像父代。变异用多项式变异，$x' = x + \delta\,(x_U - x_L)$，其中

$$\delta = \begin{cases} (2u)^{1/(\eta_m+1)} - 1, & u < 0.5, \\ 1 - \left[2(1-u)\right]^{1/(\eta_m+1)}, & u \geq 0.5, \end{cases} \tag{31-7}$$

变异概率取 $p_m = 1/d$（二维即 0.5），$\eta_m = 20$。最后做精英保留：父代与子代合并，按目标值取前 40 个。精英保留这一条保证了“最优值曲线单调不升”，是收敛曲线可读的前提。

粒子群（PSO） 不交叉也不变异，它给每个粒子一个速度：

$$v_i \leftarrow w v_i + c_1 r_1 \odot (p_i - x_i) + c_2 r_2 \odot (g - x_i), \qquad x_i \leftarrow x_i + v_i, \tag{31-8}$$

其中 p_i 是粒子 i 的历史最优，g 是全体历史最优，r_1, r_2 是逐分量均匀随机数。本章取 $w = 0.729$、$c_1 = c_2 = 1.494$——这组数不是随手写的，它对应“速度更新算子的谱半径略小于 1”，粒子轨迹会收敛而不是发散（§31.3.3 会给出这个结论的来历）。速度再按定义域宽度的 0.5 倍限幅，防止粒子一步飞出盒子。

差分进化（DE/rand/1/bin） 用种群自身的差向量当步长：

$$v_i = x_a + F\,(x_b - x_c), \qquad u_{ij} = \begin{cases} v_{ij}, & \text{若 } r_j < C_R \text{ 或 } j = j_{\text{rand}}, \\ x_{ij}, & \text{否则}, \end{cases} \tag{31-9}$$

a, b, c 是随机挑的三个互异个体，$F = 0.7$、$C_R = 0.9$。差分向量 $x_b - x_c$ 的长度自动随种群收敛而收缩，这是 DE 不需要单独设计步长表的原因。选择是贪心的：$f(u_i) < f(x_i)$ 才替换。j_{rand} 那一项保证至少有一个分量来自变异向量，否则 C_R 偏小时子代可能原封不动。

模拟退火（SA） 只有一个当前解，每步在它周围撒一个高斯点 $x_n = x + \sigma\xi$（$\sigma = 0.3$），然后按 Metropolis 准则决定接不接受：

$$\mathrm{P}(\text{接受}) = \begin{cases} 1, & \Delta f \leq 0, \\ \exp\left(-\dfrac{\Delta f}{T}\right), & \Delta f > 0, \end{cases} \qquad T_{k+1} = \alpha T_k, \tag{31-10}$$

$\Delta f = f(x_n) - f(x)$，初温 $T_0 = 5$，降温系数 $\alpha = 0.9995$。温度高时坏解也常被接受，算法在景观上大幅游走；温度降下来后逐渐变成贪心下降。**四种算法的对比如表 31-2。**

表 31-2　四种无梯度算法的结构对照

算法	维持的状态	随机性来源	如何“利用”已有信息	主要参数
GA	40 个个体	SBX 交叉 + 多项式变异	锦标赛选择保留好基因	$p_m = 1/d$、$\eta_c = 15$
PSO	40 个粒子 + 速度	两个均匀随机权重	向个体最优与全局最优靠拢	$w = 0.729$、$c_1 = c_2 = 1.494$
DE	40 个个体	差分方向 + 二项交叉	贪心替换，差向量自适应收缩	$F = 0.7$、$C_R = 0.9$
SA	1 个当前解	高斯邻域	Metropolis 接受率随温度衰减	$T_0 = 5$、$\alpha = 0.9995$

这张表最该记住的是最后一列：**四种算法的参数都不是“调得越猛越好”，每一组数背后都有一条稳定性或多样性上的理由。** §31.5.2 会把这条理由逐条展开。

把四种算法各走一步，能看清“随机性被约束成什么形状”这句话的实际含义。取 $d = 2$、定义域 $[-5.12, 5.12]$。

GA 走一步。 父代 $p_1 = (2, 2)$、$p_2 = (0, 0)$，抽到 $u = 0.3$，$\eta_c = 15$ 时 $\beta = (0.6)^{1/16} = 0.969$。代进 (31-6)：$c_1 = \tfrac{1}{2}[1.969\,p_1 + 0.031\,p_2] = (1.97, 1.97)$，$c_2 = \tfrac{1}{2}[0.031\,p_1 + 1.969\,p_2] = (0.03, 0.03)$。一个子代落在父代连线之外，一个落在之内——外推与内插同时发生，这正是 SBX 能探到父代凸包之外的原因。

PSO 走一步。 粒子 $x = (1, 1)$、$v = (0.2, -0.1)$、个体最优 $p = (0.5, 0.5)$、全局最优 $g = (0, 0)$，抽到 $r_1 = 0.6$、$r_2 = 0.4$。按 (31-8)，

$$v \leftarrow 0.729(0.2, -0.1) + 1.494 \times 0.6\,(-0.5, -0.5) + 1.494 \times 0.4\,(-1, -1) = (-0.90, -1.12),$$

于是 $x \leftarrow (0.10, -0.12)$。一步跨了 1.3，几乎等于从中心坑跳到相邻坑——速度项让它比 GA 的子代走得更远，也更容易越过脊。

DE 走一步。 $x_i = (1, 1)$，随机挑到 $a = (2, 0)$、$b = (3, 1)$、$c = (1, 3)$，$F = 0.7$：$v = (2, 0) + 0.7(2, -2) = (3.4, -0.6)$。抽到 $r = (0.2, 0.95)$，$C_R = 0.9$ 时第一分量取 v、第二分量保留 x_i，得 $u = (3.4, 1)$。变异方向由 $b - c = (2, -2)$ 决定，与 x_i 本身无关——这是 DE 与 GA 最本质的差别。

SA 走一步。 当前 $x = (0.2, -0.3)$，高斯扰动得 $x_n = (0.44, -0.66)$。代入 (31-4)：$f(x) = 20.13$、$f(x_n) = 35.28$，$\Delta f = 15.15$。第 100 步时 $T = 5 \times 0.9995^{100} = 4.76$，接受概率 $e^{-15.15/4.76} = 0.041$——只有 4%，但不是 0。**SA 的全部机制就在这 4% 里：它靠“偶尔接受坏解”翻脊，而这个概率由温度精确控制。**

四步对比下来，GA 的随机性被“父代连线”约束，PSO 的被“两个最优”约束，DE 的被“种群差向量”约束，SA 的被“温度”约束。约束越紧，单步越有方向；约束越松，越容易翻脊，也越容易乱走。

31.2.4 贝叶斯优化：先建模型，再决定下一次评价

前四种算法都只把评价结果当作“分数”，用完就丢。当 B 只有几十次时，这种浪费不可接受：每个点都花了钱，应该用它去推断“别处大概什么水平”。贝叶斯优化的做法是维护一个代理模型。

设 f 服从零均值高斯过程，

$$f(x) \sim \mathcal{GP}\big(0,\, k(x, x')\big), \qquad k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right), \tag{31-11}$$

即 RBF 核，本章取 $\ell = 0.8$、$\sigma_f = 1$。给定已评价的 n 个点 $\{(x_i, y_i)\}$，记 $k_* = [k(x_i, x)]$、K 为核矩阵，则任意点的后验为

$$\mu(x) = k_*^{\top}\left(K + \sigma_n^2 I\right)^{-1} y, \qquad \sigma^2(x) = k(x, x) - k_*^{\top}\left(K + \sigma_n^2 I\right)^{-1} k_*, \tag{31-12}$$

$\sigma_n^2 = 10^{-6}$ 是数值稳定项。μ 给出“预测值”，σ 给出“没去过的地方有多不确定”——离已采点越远，σ 越大。这两条信息合起来决定下一次采样点，用的是期望改进（EI）：

$$\mathrm{EI}(x) = (f_{\text{best}} - \mu(x) - \xi)\,\Phi(z) + \sigma(x)\,\varphi(z), \qquad z = \frac{f_{\text{best}} - \mu(x) - \xi}{\sigma(x)}, \tag{31-13}$$

Φ, φ 是标准正态的分布与密度函数，$\xi = 0.01$ 是“要求至少改进这么多才算数”的余量。EI 的结构值得读一遍：第一项奖励“均值低”，第二项奖励“方差大”。前者是利用（exploitation），后者是探索（exploration），两者的权重由 σ 自动配平——不确定性大的地方，即使均值不低，EI 也可能更大。

(31-12) 不是拟合出来的，是条件分布。高斯过程的核心假设是“任意有限个点的函数值服从联合高斯”，于是给定观测后，未观测点的分布就是条件分布：均值是观测的线性组合，方差是“先验方差减去已被解释的部分”。两个自检能立刻验证它。**其一，在已采点上，k_* 就是 K 的对应一列，$\mu(x_i) = y_i$、$\sigma^2(x_i) \to 0$——它精确插值。其二，离所有点无穷远时，$k_* \to 0$，$\mu \to 0$、$\sigma^2 \to \sigma_f^2$——它退回先验。**前者说明模型“记得住”，后者说明它“知道自己不知道”；缺了后者，探索项就无从谈起。

ℓ 与 σ_f 原则上不该手调，应最大化边际似然：$\log p(y) = -\frac{1}{2}y^\top(K+\sigma_n^2 I)^{-1}y - \frac{1}{2}\log|K+\sigma_n^2 I| - \frac{n}{2}\log 2\pi$。第一项奖励“拟合得好”，第二项惩罚“模型太复杂”，两项拉锯会自动定出长度尺度——数据密的地方 ℓ 变小，稀疏的地方变大，比手调一个全局常数合理。本章固定 $\ell = 0.8$，是为了让 §31.3.5 的手算能对上号；生产环境里用边际似然或几次试跑定更稳妥。

本章的实现是 8 个随机初值 + 32 次 EI 迭代，共 40 次评价；每次迭代在“3000 个随机候选 + 最优点邻域 500 个候选”里挑 EI 最大的那个。用随机池而不是对 EI 求梯度，是因为 EI 本身带 Φ 与 φ，梯度虽然可写但每步要更新一次 Cholesky 分解，代价与“多算几百个候选”相当。真正的成本在 (31-12)：每加一个点，K 增大一行一列，Cholesky 是 $O(n^3)$。这就是贝叶斯优化把 n 卡在几百以内的原因。

31.2.5 预算与停止准则

预算定了，还要定“什么时候提前收工”。三种停止准则按优先级排列：

第一，预算耗尽。 最硬的约束，到了 B 就停，不管结果好坏。它保证了不同算法之间的可比性。

第二，阈值达到。 目标值低于某个“够用”水平就停。本章取 $f < 1.0$ 作为达标线：Rastrigin 的全局最优是 0，$f < 1.0$ 意味着已经落在原点附近的盆地里，剩下的差距对工程意义不大。这个准则的价值在于它给出了一个比“最终值”更能说明问题的指标——**达标评价数**，即第一次低于阈值时已经花掉的评价次数。§31.6.1 的表里，GA 中位 380 次评价达标、PSO 540 次、DE 700 次，而 SA 只有 5% 的种子能在 4000 次内让最终值低于 10^{-2}。看最终值只能知道“谁更好”，看达标评价数才知道“谁更快”。

第三，曲线进入平台。 连续若干代的最优值改善低于某个相对量（例如 10^{-3}）就停。这条准则只适合离线场景：它依赖对“平台”的判断，而平台可能只是暂时的。在线或半在线场景里，前两条更可靠。

三者不是互斥的，实现里通常是“任一满足即停”。但要注意一个记账细节：**提前停止省下的预算不能挪给别人用**，否则比较又失去了共同基准。要么所有算法都跑满 B，要么所有算法都按同一阈值停。§31.6.7 的 PI 整定就是被这条规矩逼出来的：GA 与 PSO 各用 1200 次评价，SLSQP 只用 69 次，看起来“不公平”，但那是梯度法在同一个问题上本来就只需这么多次——差异本身就是要报告的结论。

阈值法有一个副作用常被忽略：**报告里必须同时写阈值和达标次数，缺一个数就失去意义。** 同一个算法的达标次数会完全变样，甚至从“全部通过”翻转成“一个都不过”——本章 GA 在 $f < 1.0$ 下中位 380 次，在 $f < 10^{-8}$ 下 20 个种子无一通过；DE 则正好相反。把这两组数放在一起看才公平：GA 快，是因为它停在浅坑；DE 慢，是因为它要一路下到 10^{-13}。**只报“谁先达标”而不报阈值，等于拿不同终点线的比赛比成绩。** 阈值本身也要有物理依据：$f < 1.0$ 之所以合理，是因为 Rastrigin 的坑间距约 1，跨过一个坑的意义远大于坑内再降三个数量级；换到 IAE 这类有量纲的指标上，阈值就该由“用户能感知的差别”倒推。

31.2.6 评价函数本身也要设计

前面一直把 f 当成“给定”的黑箱。实际上评价函数是代码，代码会出错，而错误的代价在优化里会被放大——一个 NaN 就能让整代选择失效。本节五条是本章七个实验共用的写法。

第一，计数只能有一个入口。 所有算法都通过同一个包装器调用目标，计数加在“真正调用”那一层：

```
def make_counter(fun):
    n = [0]
    def wrapped(X):
        X = np.atleast_2d(X)
        Y = np.asarray(fun(X), float).ravel()
        n[0] += X.shape[0]          # 按行计数，向量化调用也正确
        Y[~np.isfinite(Y)] = np.inf  # NaN / inf 一律当最差
        return Y
    return wrapped, lambda: n[0]
```

这个包装器同时解决三件事：预算与实际调用永远一致（§31.7.3 错误一）；向量化与逐点两种实现的计数口径统一（错误三）；非法值不会污染 `argmin`——`np.argmin([nan, 1.0])` 返回 0，等于把最差的点当成最好的，而 `argmin([inf, 1.0])` 返回 1，至少方向是对的。

第二，缓存要声明是否计入预算。 GA 的精英保留、DE 的重复个体、SA 的回访都会重复落到同一点。开缓存能省时间，但“命中不计入预算”会让不同算法的有效预算不同。本章一律不开缓存，宁可多算；若开了，报告里必须写命中率与“命中是否计数”。

第三，评价失败也要算一次预算。 抛异常不代表没花钱——真实系统里那一次尝试往往耗时更长（超时、重试、日志）。本章的写法是捕获异常、返回 $+\infty$、计数照加。

第四，t_{eval} 取中位数，不取均值。 偶发的垃圾回收、磁盘抖动、别的进程抢占会把均值拉高一倍以上，而 (31-30) 要用的是一个“典型值”。本章测 t_{eval} 的做法是连测 100 次取中位，再乘 1.2 当余量。

第五，包装器本身要足够薄。 一层 Python 函数调用约 1 µs，对 1 ms 的评价可以忽略；但如果目标本身只要 10 µs（例如解析函数），包装器的开销就占 10%。这时要么把计数写进目标内部，要么换编译语言。**“优化器比目标还慢”不是笑话，在廉价目标上很常见。**

31.3 推导与性质

31.3.1 能证明的只有弱结论

先划一条界线：本章的方法没有“收敛到全局最优”的定理可用。能证明的只有三条弱结论。

第一条是单调性。 GA 的精英保留与 DE 的贪心替换都保证了

$$f_{t+1} \le f_t, \tag{31-14}$$

序列单调不增且有下界（$f \ge 0$），因此必定收敛到某个 f_∞——但 f_∞ 完全可能远离 $f^\star = 0$。这条结论的价值不在“收敛”，而在“曲线可读”：图 31-2 用“当前最优”作图，曲线必须单调，否则读者无法区分“算法在进步”与“算法在漂移”。PSO 与 SA 也是同样的道理，它们记录的是历史最优。

第二条是 SA 的平稳分布。 固定温度下它采样的是 Boltzmann 分布（§31.3.4），只有 $T \to 0$ 才集中到全局最优。理论上要 $T_k \ge c/\log k$ 才能依概率收敛，而这种降温在 4000 次预算内只降几个百分点，实际走不完。

第三条是没有全局收敛定理。 GA、PSO、DE 的收敛性结论都建立在“种群足够大、迭代足够久”的强假设上，且多数只能保证局部收敛。所以本章的实验报告“20 个种子的结果分布”，不报告“收敛到最优”。

至于 No Free Lunch，只需记住它的正确用法：把所有可能的目标函数取平均之后，任意两种算法的期望性能相同。这句话提醒你“不存在通用最优算法”，但不意味着“算法之间没有差别”——在具体一类问题上，差别就是 §31.6 的那些数字。

31.3.2 GA：选择压力是算得出来的

种群 $N = 40$、锦标赛 $k = 3$ 时，种群中最好的个体获胜，当且仅当它出现在这 3 次抽样里（有放回）：

$$p_{\text{sel}} = 1 - \left(1 - \frac{1}{N}\right)^{k} = 1 - \left(\frac{39}{40}\right)^{3} = 0.073. \tag{31-15}$$

随机选择的概率是 $1/40 = 0.025$，所以选择压力只有 2.9 倍，最优个体每代期望被选中 $40 \times 0.073 = 2.9$ 次。这个数直接解释了 §31.6.1 里 GA 的表现：中位 7.87×10^{-5}，比 DE 差 8 个数量级，但 20 个种子全部达标。**压力温和，进步慢，却不容易早熟。**

把 k 从 3 加到 8：$p_{\text{sel}} = 1 - (39/40)^8 = 0.183$，期望 7.3 次，压力翻 2.5 倍，代价是多样性塌缩更快。多样性由变异维持——$p_m = 1/d = 0.5$ 意味着每个子代平均有一个分量被扰动，扰动幅度由 $\eta_m = 20$ 定。选择压力与变异强度共同决定了“探索预算”怎么分配。

精英保留把 (31-14) 坐实，代价是种群容易同质化：精英长期霸住位置会压制交叉产生的子代。这是 GA 在 4000 次评价内停在 10^{-4} 量级的结构性原因，不是种子没选好。

经典遗传算法还有一条理论支撑：模式定理（schema theorem）。它估计的是“一个模式”（若干位固定、其余任意的个体集合）在下一代中的期望数量，简化到实数编码的直觉版本是 $E[m(H, t+1)] \ge m(H,t)\,[f(H)/\bar f]\,[1 - p_c\,\delta(H)/(l-1) - o(H)\,p_m]$。方括号外的因子是“选择”，方括号内是“破坏”：交叉以 p_c 打断长度为 $\delta(H)$ 的模式，变异以 p_m 打掉 $o(H)$ 个已定位。代入本章的数：$d = 2$、$p_m = 0.5$，一个只固定单个分量的模式 $o(H) = 1$，光变异一项就给出 $1 - 0.5 = 0.5$ 的存活率——**每一代都有一半概率把这个“好基因”打掉。**

这个数是 $p_m = 1/d$ 的直接后果：维度越低，p_m 越大，破坏越强。它解释了 GA 为什么停在 10^{-4}——要保持一个精细结构，变异率必须很小；但 p_m 小了又不探索。**“探索”与“保留”在低维问题上直接冲突，这是 GA 的结构性弱点，不是参数没调好。** DE 没有这个问题：它的变异作用在整条差向量上，不会逐分量打掉好基因；PSO 也没有，它压根不做“位”的操作。

SBX 的参数 η_c 也值得算一遍，因为它决定了“交叉到底能走多远”。子代是 $c_{1,2} = \tfrac{1}{2}\left[(1 \pm \beta)p_1 + (1 \mp \beta)p_2\right]$，$\beta$ 由均匀随机数变换而来：$u = 0.1$ 时 $\beta = (0.2)^{1/16} = 0.90$（子代在父代之间），$u = 0.9$ 时 $\beta = (5)^{1/16} = 1.11$（子代落到父代区间之外 11%）。**$\beta < 1$ 是“内插”，$\beta > 1$ 是“外推”，两者的分界正好在 $u = 0.5$。** 于是这条交叉算子的行为很干净：一半概率在盒内重组，一半概率向盒外扩张，扩张幅度随 $u \to 1$ 增大——理论上是无界的，实际上被定义域截断兜住。

这解释了为什么 p_m 比 η_c 更关键。外推虽然能出盒，但幅度只有 10%–30%，且方向被父代连线锁死；真正能改变“搜索方向”的是逐分量的多项式变异。调 GA 时先调变异率、再调 η_c，顺序不能反。

η_c 越大，β 越接近 1，子代越像父代。$\eta_c=15$ 时 β 的中位数是 $(0.5)^{1/16}=0.96$，也就是说一半以上的交叉几乎不改变个体。**这看起来"保守"，但对已经收敛的种群是好事**：它让 GA 在后期做局部细化而不是把好个体打散。代价是前期太慢，与 §31.6.3 里"台阶型曲线"的形态一致。

31.3.3 PSO：谱半径给出"不发散"的保证

把 r_1,r_2,p,g 冻结（在单次迭代内成立），逐分量看 (31-8)。令 $\varphi=c_1r_1+c_2r_2$，平衡点 $\bar{x}=(c_1r_1p+c_2r_2g)/\varphi$，误差 $e=x-\bar{x}$，则

$$\begin{bmatrix} v_{k+1} \\ e_{k+1} \end{bmatrix} = \begin{bmatrix} w & -\varphi \\ w & 1-\varphi \end{bmatrix} \begin{bmatrix} v_k \\ e_k \end{bmatrix}. \tag{31-16}$$

特征多项式是 $\lambda^2-(1+w-\varphi)\lambda+w=0$，两根之积为 w、之和为 $1+w-\varphi$。稳定的充要条件是

$$0<w<1, \qquad 0<\varphi<2(1+w). \tag{31-17}$$

本章 $w=0.729$、$c_1=c_2=1.494$，故 $\varphi=2.988<2(1+0.729)=3.458$；判别式 $(1+w-\varphi)^2-4w=1.585-2.916=-1.331<0$，两根为共轭复根，模为 $\sqrt{w}=0.854$。**意思是：在权重冻结的那一步里，误差按 0.854 的比率收缩，10 步缩到 $0.854^{10}=0.206$。**

这组经典参数的来历也能顺手算出来。Clerc 的收缩因子取

$$\chi=\frac{2}{\left|2-\varphi_s-\sqrt{\varphi_s^2-4\varphi_s}\right|}, \tag{31-18}$$

$\varphi_s=4.1$ 时 $\chi=0.730$，再令 $c_1=c_2=\chi\varphi_s/2=1.494$，就是本章用的数。取 4.1 而不是 4，是为了 φ 达到上界时仍留一点裕量。

必须说清这条结论的边界：它保证"速度不指数发散"，不保证"收敛到全局最优"。p 与 g 每步都在变，$\bar{x}$ 跟着漂移，所以 0.854 描述的是"追移动目标"的跟随速度。它解释的是为什么这组数可以直接用而不必逐题调，不是为什么 PSO 能找到最优。

31.3.4 DE 与 SA：步长自己会长，温度不会自己降

DE 的步长由种群给出。设某分量在种群里的标准差为 σ，两个独立样本之差满足 $\mathrm{E}|X-Y|=2\sigma/\sqrt{\pi}=1.128\sigma$，于是变异步长的期望是

$$\mathrm{E}\,|v_i-x_i| \approx F\cdot 1.128\,\sigma = 0.79\,\sigma. \tag{31-19}$$

初始均匀分布 $\sigma=10.24/\sqrt{12}=2.96$，期望步长 2.33，约为定义域的 23%；种群收敛后 $\sigma\to 0$，步长同步收缩。**这是 DE 不需要步长表的原因，也是它最终精度最高的原因**（中位 4.53×10^{-13}）。但它的达标评价数是 700 次，反而大于 GA 的 380 次——精度高不等于爬得快：GA 靠精英保留先把值压到 1.0 以下，DE 则在后半程持续下探。

$F=0.7$、$C_R=0.9$ 是 DE/rand/1/bin 这一支的取值，名字里三部分都可以换。"rand"指基向量随机挑，换成"best"就用当前最优，收敛更快但更容易早熟；"1"指用一对差向量，"2"用两对，步长翻倍、探索更强；"bin"指逐分量二项交叉，"exp"则是连续段交叉。工程上常见的是 rand/1/bin 与 best/1/bin 两支，前者稳、后者快。

F 是步长的油门：从 0.5 调到 1.0，期望步长从 0.56σ 到 1.13σ。超过 1 以后 v_i 可能落到种群分布之外，探索性最强，也最容易把已经收敛的种群打散。自适应变体（jDE、SHADE）让每个个体自带一对 $F_i,C_{R,i}$，成功就保留、失败就重抽——**它把"调参"变成了种群里的一场小型进化**，代价是实现复杂、复现更难。本章坚持用固定参数，是为了让每个数字都能对应到一条公式上；真要上生产，SHADE 通常是更好的默认选择。

SA 的问题正好相反：温度按固定比例降，不跟着进度走。Metropolis 准则之所以有效，是因为它满足细致平衡：

$$\pi(x)\,q(x\to y)\,A(x\to y)=\pi(y)\,q(y\to x)\,A(y\to x). \tag{31-20}$$

取对称高斯提议 q 与 $A=\min\left(1,e^{-\Delta f/T}\right)$，解得 $\pi(x)\propto e^{-f(x)/T}$。固定 T 下算法采样的是 Boltzmann 分布，只有 $T\to0$ 才集中到全局最优。

本章 $T_0=5$、$\alpha=0.9995$、$B=4000$，末温 $T=5\times0.9995^{4000}=0.68$。此时接受一个 $\Delta f=0.7$ 的上坡移动的概率是 $e^{-0.7/0.68}=0.36$——末段仍有三分之一以上的概率往差的方向走；再加上邻域尺度 $\sigma=0.3$ 固定不变，末段无法细化。§31.6.1 里 SA 的成功率只有 5%，原因就在这里：不是算法失效，是这组参数在 4000 次内没冷透。

按目标末温反解降温系数：若要求 $T_B/T_0=10^{-3}$，则

$$\alpha = \left(\frac{T_B}{T_0}\right)^{1/B} = 10^{-3/4000} = 0.99827, \tag{31-21}$$

比 0.9995 快得多。这一步反解，是所有“预算固定”的退火实现动手前都该做的。

邻域尺度 $\sigma = 0.3$ 是 SA 的第二个参数，常被忽略。它与温度分工不同：**温度决定“敢不敢走”，邻域决定“能走到哪”。** 末温 0.68 时若把 σ 从 0.3 降到 0.03，上坡的 Δf 随之变小、接受率大幅提高，但每步位移也变小，细化能力反而增强。这说明“缩小邻域”和“降温”效果相似、机理不同：一个改提议分布，一个改接受准则。工程上更稳的做法是两者同步缩：T_k 按 (31-21) 走，$\sigma_k = \sigma_0\sqrt{T_k/T_0}$——对二次型坑底，Boltzmann 分布的方差正比于 T，所以宽度按 $\sqrt{T}$ 缩是有依据的，不是拍出来的。

本章没有这么做，于是出现了 §31.6.1 的 5% 成功率。**这是全章最值得做而没做的一组对照实验**：从末温 0.68 改到 10^{-3}，上坡接受率会从 36% 掉到可以忽略，末段的细化能力完全不同。本书有意把它留作练习（理由见 §31.7.5），但读者若在自己的问题上照抄本章参数，务必先做这一步反解。

31.3.5 贝叶斯优化：EI 为什么能自动配平

§31.2.4 给了后验 (31-12)，这里推采集函数。设 $Y = f(x) \sim N(\mu, \sigma^2)$，记 $t = f_{\text{best}} - \xi$，则改进量的期望为

$$\mathrm{EI}(x) = \mathrm{E}\left[(t - Y)^{+}\right] = \int_{-\infty}^{t} (t - y)\,\frac{1}{\sigma}\varphi\!\left(\frac{y-\mu}{\sigma}\right)\mathrm{d}y = (t-\mu)\Phi(z) + \sigma\varphi(z), \qquad z = \frac{t-\mu}{\sigma}. \tag{31-22}$$

推导只有一步：把积分按 t 与 y 拆成两块，第一块给 $(t-\mu)\Phi(z)$，第二块给 $\sigma\varphi(z)$。

这个式子把“探索-利用”写成了两项之和，而且配平不需要任何权重。第一项 $(t-\mu)\Phi(z)$ 在 σ 小时趋近 $\max(0, t-\mu)$，退化为贪心；第二项 $\sigma\varphi(z)$ 在 σ 大时主导，奖励没去过的地方。代个数：在 $\mu = f_{\text{best}}$ 的点上，$\mathrm{EI} \approx 0.399\sigma - 0.5\xi$，即“完全没有信息的地方”的基线价值正比于 σ。

后验的形状完全由核的两个旋钮决定：长度尺度 ℓ 与幅度 σ_f。RBF 核 $k(x,x') = \sigma_f^2 \exp\left(-\|x-x'\|^2/(2\ell^2)\right)$，两点相距 ℓ 时相关性 $e^{-1/2} = 0.61$，相距 2ℓ 时 $e^{-2} = 0.135$，相距 3ℓ 时 $e^{-4.5} = 0.011$。**超过三个长度尺度，后验就退回先验**——那里既没有信息，也没有梯度，只剩一个常数方差。本章取 $\ell = 0.8$、$\sigma_f = 1$，而定义域边长 10.24，ℓ 只占 7.8%；Himmelblau 的搜索盒 $[-5,5]^2$ 里 40 个点的平均最近邻间距约 1.58，正好是 ℓ 的两倍。所以到第 40 次评价时，已采样点附近的 EI 已经很小，算法会自己把下一个点推到“离所有点都超过 2”的地方，那里的后验方差接近 $\sigma_f^2 = 1$。**ℓ 小不是缺陷，它让贝叶斯优化在大盒子里表现出“局部精细、全局粗放”的性格**：要覆盖整个盒子，要么把 ℓ 调大（后验更平滑，但对细节更迟钝），要么先用别的手段把盒子缩小。σ_f 的作用更直接：它只是把 EI 整体乘上一个尺度，不改变“下一步去哪”，只改变“这次改进值多少”。也正因如此，σ_f 取错只影响停止判断，ℓ 取错才会让人去错地方。

一维手算能把这几个数看清楚。已在 $x=0$ 测得 $f=1$、在 $x=1$ 测得 $f=0$，问 $x=0.5$ 处。取 $\ell=0.8$、$\sigma_f=1$、无噪声：$k(0.5,0) = k(0.5,1) = e^{-0.195} = 0.822$，$k(0,1) = e^{-0.781} = 0.458$。$2\times 2$ 的逆矩阵只有一行算术，得 $\mu(0.5) = 0.822/(1+0.458) = 0.564$，$\sigma^2(0.5) = 1 - 0.927 = 0.073$，即 $\sigma = 0.270$。注意到 μ 是 0.564 而不是两个观测的均值 0.5：零均值先验把预测往下拉了一点，权重里出现了一个负号，这是零均值 GP 的正常现象，不是算错。此时取 $f_{\text{best}}=0$、$\xi=0.01$，$z = (-0.01 - 0.564)/0.270 = -2.13$，$\Phi(z) = 0.0166$，$\varphi(z) = 0.0417$，$\mathrm{EI} = (-0.574)(0.0166) + 0.270(0.0417) = 0.0018$——**比噪声还小，这个点永远不会再被选中。**

再看 $x=3$：$k(3,0) = 0.00088$、$k(3,1) = 0.0439$，$\mu = -0.024$、$\sigma = 0.999$，$\mathrm{EI} = 0.0143\times 0.506 + 0.999\times 0.399 = 0.406$，是 $x=0.5$ 处的 226 倍。EI 宁愿去一个完全没测过、后验均值还略低于历史最优的远处，也不愿在两个好点之间补一刀——这就是“探索”在数值上的样子。不过这个 0.406 里绝大部分来自 $\sigma_f\varphi(z)$，即**先验本身**：它假定“没去过的地方可能和先验均值一样好”。若先验均值取得过于乐观（Himmelblau 的真值下界是 0，取 0 恰好乐观），远处就会被系统性高估，算法会过度探索。§31.6.6 里 40 次评价只把值压到 1.5×10^{-3}，一半原因在 ℓ 偏小，另一半就在这里。

顺带把 EI 与常见的置信下界（UCB，最小化写法 $\mu - \kappa\sigma$）对上。令 $t = \mu$，则 $\mathrm{EI} = \sigma\varphi(0) = 0.399\sigma$，等效 $\kappa = 0.399$；令 $t \gg \mu$（该点看着就比历史最优好），$\mathrm{EI} \to t - \mu$，等效 $\kappa \to 0$，纯利用；令 $t \ll \mu$，$\mathrm{EI} \to 0$，等效 κ 也趋于 0，但方式是“放弃”而不是“利用”。**EI 的等效 κ 不是常数，而是随“这个点看起来有多好”自动变化**：看着好的地方大胆去，看着差的地方不去，说不清的地方按 σ 定价。UCB 的 κ 要人手调，调大了满世界乱跑，调小了原地不动；EI 把这件事外包给了后验分布。

(31-22) 是单步前瞻（myopic）的：它最大化“下一次评价的期望改进”，不考虑“这次评价会让后面几次更聪明”。多步版本要解动态规划，代价远大于收益，实践中基本都用单步。

成本在 (31-12)：每轮重新分解 $K + \sigma_n^2 I$，$O(n^3)$。$n = 40$ 时是 6.4×10^4 次浮点运算，可以忽略；$n = 4000$ 时是 6.4×10^{10}，不可接受。这就是贝叶斯优化的规模上限。

31.3.6 SGD：条件数与步长上界

最后看“梯度可用、但要迭代很多次”的情形。取病态二次目标 $f(x)=\tfrac12 x^\top A x$，$A=\mathrm{diag}(1,100)$，条件数 $\kappa=100$。

梯度下降的更新与误差递推是

$$x_{k+1}=x_k-\eta\nabla f(x_k),\qquad e_{k+1}=(I-\eta A)\,e_k, \tag{31-23}$$

逐坐标看，误差按因子 $1-\eta\lambda_i$ 衰减。要所有坐标都衰减，必须

$$0<\eta<\frac{2}{\lambda_{\max}}=0.02. \tag{31-24}$$

这就是步长上界，不是经验值，是算出来的：超过它，最大特征值方向的误差开始交替放大。

上界之内，收敛速度由条件数决定。最优步长 $\eta^\star=2/(\lambda_{\min}+\lambda_{\max})=0.0198$ 给出误差率 $(\kappa-1)/(\kappa+1)=0.980$，每步只缩 2%。本章取 $\eta=0.01$，慢模态因子 $1-0.01=0.99$，损失按 0.9801^{k} 衰减：300 步后 $0.5\times0.9801^{300}=1.2\times10^{-3}$，与实测 1.2025×10^{-3} 对上。表 31-3 是步长扫描，三档都能用解析式核对。

表 31-3　GD 步长与收敛（$\kappa=100$，300 步）

η	$1-\eta\lambda_{\max}$	解析预测	实测末值	现象
0.001	0.9	2.74×10^{-1}	2.74×10^{-1}	慢模态主导，几乎不动
0.005	0.5	2.47×10^{-2}	2.47×10^{-2}	仍在慢模态
0.01	0	1.20×10^{-3}	1.20×10^{-3}	快模态一步清零
0.015	-0.5	5.8×10^{-5}	5.76×10^{-5}	快模态交替衰减
0.02	-1	5.0×10^{1}（不降）	5.00×10^{1}	临界，等幅振荡
0.03	-2	18 步 3.4×10^{12}	3.44×10^{12}	越过上界

两种加速手段在这张表的同一问题上都有效。动量法累积历史梯度：

$$v_{k+1}=\beta v_k-\eta\nabla f(x_k),\qquad x_{k+1}=x_k+v_{k+1}, \tag{31-25}$$

$\beta=0.9$ 时，一致方向上的等效步长被放大 $1/(1-\beta)=10$ 倍。本章 100 步后损失 9.46×10^{-5}，而 GD 是 6.70×10^{-2}，差 708 倍。理论上按 κ 调优的 heavy-ball 收敛率是 $(\sqrt\kappa-1)/(\sqrt\kappa+1)=0.818$，100 步可到 10^{-9}；本章固定 $\eta=0.01$ 未调优，只到 10^{-4} 量级——但加速比仍有两个数量级以上。

Adam 逐坐标归一化：

$$m_k=\beta_1 m_{k-1}+(1-\beta_1)g_k,\quad s_k=\beta_2 s_{k-1}+(1-\beta_2)g_k^2,\quad x_{k+1}=x_k-\eta\frac{\hat m_k}{\sqrt{\hat s_k}+\epsilon}, \tag{31-26}$$

$\beta_1=0.9$、$\beta_2=0.999$，$\hat m,\hat s$ 是偏差修正后的矩估计。分母 $\sqrt{\hat s}$ 抹掉了梯度的绝对尺度，于是 $\lambda=1$ 与 $\lambda=100$ 两个坐标“看起来”一样好走。代价是路径不均匀：本章 50 步到 1.17×10^{-3}、100 步到 4.36×10^{-4}（只降 2.7 倍），末值 2.25×10^{-13}（后 200 步降了 9 个数量级）。**用 Adam 时不能看前 50 步就判断好坏，中段平台很容易被误读成“已经收敛”。**

三个方法的收敛条件可以统一成一句话：**步长必须与曲率匹配。** GD 要求 $\eta<2/\lambda_{\max}$；动量要求 β 与 η 一起满足；Adam 用归一化绕开了绝对曲率，但对学习率衰减仍然敏感——常数步长的 Adam 在随机梯度下只收敛到 η 量级的邻域，不会到 0。

31.3.7 这些保证在什么条件下失效

前面的推导都带前提，而工程上的参数往往正好踩在前提之外。把前提列清楚，比记住结论更有用。

PSO 的谱半径 0.854 假设三件事：w,c_1,c_2 不随迭代变化；位置与速度没有被人为截断；g 每轮同步更新。本章的实现第一条满足，第二条不满足（速度被限到 ±5.12），第三条在串行实现里满足。速度一被截断，粒子就进入“贴边滑行”的状态，谱半径那套线性分析不再成立——它只保证“不发散”，不保证“还能飞出去”。所以 §31.4.3 的边界处理不是小细节，它直接改写了这条保证的适用范围。

模式定理假设无限种群。 有限种群有一种它没有描述的力量：漂变。40 个个体的种群，一个既不被选择也不被淘汰的中性基因，大约 $2N=80$ 代内就会随机固定或消失。本章 GA 跑 4000 次评价，约 100 代，正好在这个尺度上——**这意味着“某次运行里某个好基因保住了”有相当成分是运气**，也是 §31.6.2 里 GA 种子间差 5.8 倍的来源之一。

DE 的 0.79σ 假设种群分布近似高斯。 选择压力会让分布向最优端偏斜，实际步长比这个数小；种群收敛后 $\sigma \to 0$，步长同步趋零——这条“自动退火”的性质是 DE 好用的原因，也是它后期容易停在某个盆地里的原因：步长太小，翻不过脊。

SA 的细致平衡要求“每个温度下混合充分”。 本章固定 4000 步降温，每个温度只走了不到一步（温度是连续降的），根本不满足准静态条件。这时候的退火已经不是“平衡态采样”，而是“淬火”：它更像一次带随机接受准则的局部搜索。§31.6.1 里 SA 只有 5% 的成功率，与这个偏差是同一件事的两种说法。

贝叶斯优化的后验假设目标平稳且服从核的先验。 目标里如果有阶跃（比如“相位裕度低于 45° 就直接判失败”），RBF 核会把它抹成一条光滑曲线，后验在跳变附近系统性地偏低，EI 会反复去探那片区域。遇到这种目标，正确做法是先把约束做成惩罚、把跳变挪到目标之外，而不是加更多采样点。

五条前提有个共同点：**它们都是“局部、线性、无限样本”的假设**，而工程问题是全局、非线性、样本有限的。推导的价值不在于“证明能收敛”，而在于给出一个基准——实际表现偏离基准多少，就说明哪条假设被破坏得多厉害，参数该往哪个方向调。

31.4 离散对应与数值实现

31.4.1 编码：把格子变成连续变量

工程上的“选型”很少是连续的。减速比只有 10、12.5、16、20 四档，PID 参数常被要求取 E24 系列，传感器只能在型谱里挑。把这些变量直接丢给 GA 或 PSO 是不行的：交叉与速度更新都会产生“12.7 档”这种不存在的值。

标准做法是**在索引空间里搜索，在解码时映射回物理值**：

$$x_i = l_i + k_i\,\Delta_i, \qquad k_i \in \{0, 1, \dots, K_i\}, \qquad \Delta_i = \frac{u_i - l_i}{K_i}, \tag{31-27}$$

优化器操作的是整数 k_i（或它的连续松弛），评价时才解码。四档减速比就是 $K=3$，$k \in \{0,1,2,3\}$ 查表得 $\{10, 12.5, 16, 20\}$。索引空间的邻接关系与物理值无关，这一点很关键：**索引相邻不等于物理相邻**，若档位是 $\{10, 12.5, 16, 20\}$，k 从 1 跳到 2 是 3.5 的跨度，从 2 跳到 3 只有 4 的跨度——差别不大；但如果档位是 $\{10, 12.5, 16, 20, 80\}$，最后一步就是 60 的跨度。

二进制编码还有一层坑。把整数写成二进制，7 是 `0111`、8 是 `1000`，相邻的两个数要翻 4 位；单点交叉几乎不可能从 7 走到 8，却很容易产生 `0100`=4 这种“跳远”的子代。这叫汉明悬崖。**格雷码**让相邻整数只差一位（7 是 `0100`、8 是 `1100`），悬崖就消失了。实数编码的 SBX 不存在这个问题，这是本章用实数编码的原因之一；但如果你的历史代码是位串 GA，先检查有没有用格雷码。

连续松弛也要当心。PSO 在索引空间飞行时会得到 $k = 2.37$，取整才能评价；取整把一个光滑的速度场变成了阶梯，粒子在 $k=2$ 与 $k=3$ 之间来回跳，速度却一直在累积。处理办法是给速度加阻尼，或者干脆改用离散版本的 PSO（速度解释为“取 3 的概率”）。**这一类“取整 + 连续更新”的混搭，是启发式方法在选型问题上最常见的性能损失来源。**

看一个具体的四档例子。设档位 $\{10, 12.5, 16, 20\}$，目标由两部分组成：一是与理想速比 14.6 的偏差，二是“用 16 及以上大档要加装制动器”的固定代价 0.6：

$$J(i) = \frac{(i - 14.6)^2}{20} + 0.6 \cdot \mathbf{1}\{i \ge 16\}.$$

四个档位逐个代入：$J(10) = 1.058$、$J(12.5) = 0.221$、$J(16) = 0.698$、$J(20) = 2.058$，离散最优是 12.5。若先在连续空间优化，得到 $i^\ast = 14.6$、$J = 0$，再“四舍五入到最近档”——14.6 距 16 有 1.4、距 12.5 有 2.1，于是落到 16 档，评价 0.698。**连续解 0、取整后 0.698、真正的离散最优 0.221：取整把答案带到了比最优差 3.2 倍的地方，而这个差距完全由那个阶跃罚项造成，跟优化器的精度没有关系。** 罚项越“硬”、档位越稀，“先连续后取整”越不可靠。

正确做法有两条，按规模选。档位组合不多时（本例四个档位、再加一个二值的“是否加制动器”，一共 8 种组合），**直接枚举**：8 次评价就能给出全局最优，任何启发式算法都不如它。只有组合数上万、单次评价又贵时，才值得把 k 当成染色体上的整数基因，在索引空间里做交叉与变异，并且**只在解码后评价、绝不对 k 做插值**。判断标准很干脆：能枚举就枚举，枚举不动才谈编码技巧。

31.4.2 约束：罚函数、可行性规则与修复

本章的 PI 整定带两个不等式约束：相位裕度 $\ge 45^\circ$、灵敏度峰值 ≤ 2。启发式方法没有拉格朗日乘子，处理约束只有三条路。

第一条是罚函数。 把约束违反量折进目标：

$$F(x) = f(x) + \rho \sum_{j} \left[\max\left(0,\, g_j(x)\right)\right]^2, \tag{31-28}$$

本章 $\rho = 5$：$g_1 = 45 - \mathrm{PM}$（单位度）、$g_2 = M_s - 2$。系数选 5 不是随手写的——它要让“违约 1°”的罚值（5）明显大于 IAE 在最优附近的典型差异（0.01 量级），解才会被压到可行域边界上；§31.6.7 的结果 PM $= 45.00°$、违约量近似为 0，验证了这个配平。

罚函数的代价是**它换掉了问题的结构**。ρ 太小，解不可行；ρ 太大，可行域边界变成一堵陡墙，差分步长相对墙的陡度显得极小，算法在墙外反复试探却迈不进去。更麻烦的是，罚函数把“约束”与“目标”混成了一个数：你再也拿不到影子价格，回答不了“放宽 1° 能让 IAE 降多少”——这正是第 30 章 KKT 乘子能给而罚函数给不了的东西。

第二条是可行性规则（Deb 的约束支配）。比较两个个体时：都可行，比目标值；一可行一不可行，可行的赢；都不可行，违约量小的赢。它不需要 ρ，适合“可行域很窄、罚系数难配”的场合。代价是搜索前期几乎不考虑目标，种群会在可行域边界上“贴”很久。

第三条是修复算子。 生成子代后直接把它投影回可行域：越界分量截断、等式约束用投影、离散变量取最近的合法档位。修复的好处是每个评价都花在可行点上；坏处是它可能把有希望的搜索方向一并抹掉——两个不可行父代交叉出的子代被修复后，可能落在一个与父代毫无关系的角落。

三条路的选择判据很简单：**约束是“硬边界”（物理上不可能违反）就用修复；是“性能指标”（可以短时越界）就用罚函数；可行域极窄且罚系数难配就用可行性规则。** PI 整定的两个约束属于第二类，所以本章用罚函数。

用罚函数还有一个技术细节常被忽略：**各项违约量必须先归一化，否则同一个 ρ 实际上给每项不同的权重。** 本章 $g_1 = 45 - \mathrm{PM}$ 的单位是度（典型违约 1–10），$g_2 = M_s - 2$ 是无量纲（典型违约 0.1–1），$\rho = 5$ 对前者的实际惩罚比后者重十倍以上——这未必是想要的。稳妥的做法是让每项除以“该约束的容许波动”，例如 $g_1/5$ 与 $g_2/0.2$，量级拉平后再平方求和。**归一化之后，ρ 才是一个真正的“罚有多重”的旋钮，而不是若干隐含权重的混合物。** 只报“$\rho = 5$”而不说各项怎么归一化，别人是复现不出同一个解的。本章的实现没有做归一化，于是 PM 那一项被压得更狠——§31.6.7 里 PM 精确停在 45.00，而 $M_s = 1.639$ 离上界 2 还差 18%。这一半来自加权的不对称，一半是因为这组参数下 M_s 本来就不紧；但两个约束的表现不同，读者至少应当知道原因可能是前者。

31.4.3 边界与截断

所有实现在生成新解后都要处理越界。最省事的是截断：

$$x_i \leftarrow \min\left(\max(x_i, l_i),\, u_i\right), \tag{31-29}$$

本章的 GA、PSO、DE、SA 全用这一条。它简单、不会产生非法值，但有一个副作用：**个体会堆在边界上**。PSO 的速度仍指向界外，位置被反复钉在 u_i，粒子失去探索能力；DE 的差向量在边界附近被截断，方向失真。

诊断方法很直接：统计最终种群中贴在边界上的个体比例。超过两成，就要怀疑两种情况之一——最优解本来就在界外（边界设窄了），或者步长过大（速度限幅没做好）。§31.6.7 里 K_p 的上界是 5，最优解 4.27 离边界还有余量，所以那组结果不受截断影响；**如果最优解恰好落在边界上，先别急着调算法，先问一句“这个界是谁定的、能不能放宽”。**

另一种写法是反射：$x_i \leftarrow 2l_i - x_i$（越下界时）。它保持了种群多样性，但破坏了差向量的方向，在 DE 里尤其容易让搜索“回头”。第三种是重采样，把越界个体重新随机生成——简单，但浪费了那次评价。

对 DE 还有第四种更合适的写法：**向基向量回缩**。若 u_i 越界，就在基向量 x_{r_1} 与 u_i 的连线上找最近的合法点，$u_i \leftarrow x_{r_1} + \lambda(u_i - x_{r_1})$，$\lambda$ 取使 u_i 刚好落在界内的最大值。它的好处是保留“从基向量出发的方向”，只缩短步长——与截断把方向改掉、反射把方向反过来相比，这个处理最贴近 DE 原本的几何含义。代价是要多几次判断，且对 PSO 不适用（PSO 没有基向量的概念）。

31.4.4 可重复性：三样必须固定的东西

§31.1.5 的第二问是“结果可重复吗”。答案取决于三样东西有没有固定。

第一，种子链。 一个种子驱动一次完整运行，种子之间互不干扰。本章的做法是从 20261020 起连续取 20 个，每次运行内部再派生独立的随机流。**反例是在循环里复用同一个 `rng` 又开多进程**：并行时各进程的取数顺序不确定，同一个种子会给出不同结果。正确做法是每个进程用自己的 `SeedSequence` 派生。

第二，库版本与随机数生成器。 NumPy 有两套接口：旧的 `RandomState`（MT19937）与新的 `default_rng`（PCG64）。同一个整数种子，两者产生的序列完全不同；即使都用 `default_rng`，NumPy 大版本升级也可能改变某些方法的抽样细节。工程上要么锁死版本，要么把“生成随机数”这一步与“评价”这一步彻底分离，把随机数落盘。

第三，并行归约顺序。 浮点加法不满足结合律：$(a + b) + c$ 与 $a + (b + c)$ 的末位可能不同。多线程求和、`np.sum` 的分块大小、BLAS 的线程数（`OMP_NUM_THREADS`），都会改变归约顺序。单次差异只有 10^{-16}，但在 300 步迭代、或在“是否低于阈值”的判断上，末位差异会被放大成“达标/不达标”的翻转。

这三条不是洁癖。§31.6 的每一个数字都要能被别人复核，做不到就等于没做实验。**判断标准很具体：把代码换一台机器、换一个线程数、重跑一遍，打印出来的末位是否一致。** 不一致就先查这三样，而不是怀疑算法。

31.4.5 停止、检查点与热启动

三种停止准则在 §31.2.5 已经排过优先级，实现上有两个细节。

检查点要连随机数状态一起存。 只存种群和目标值是不够的：续跑时若重新播种，后续轨迹与“一次跑完”完全不同。`default_rng` 的状态可以用 `rng.bit_generator.state` 取出（一个字典），与 (X, F, n_{eval}) 一起序列化，续跑才算真正接上。

热启动要保留多样性。 把上一次的最优解直接当作初始种群，等于把种群塌缩成一个点，交叉失去意义。可行的做法是：20% 的个体在上次最优解附近扰动，其余仍然全局随机。这样既利用了历史信息，又保留了跳出局部的能力。

评价缓存只在离散问题上有用。 以参数向量的字节串为键缓存结果，连续优化里两个浮点向量几乎不可能完全相同，命中率接近零；而选型问题的解空间只有几十个点，缓存能把预算省下大半。判断要不要做缓存，只需估一下“合法解的总数”与预算的量级。

31.4.6 一段可复用的骨架

把上面的约定压进一个主循环，就是下面这些行。它对应 `code/ch031_sim.py` 里的 `run_ga`，删去了与本章实验相关的记录代码。

```python
def run_ga(seed: int, budget: int, pop: int = 40) -> tuple[float, int]:
    """实数编码 GA 主循环：记账口径是评价次数，不是代数。"""
    rng = np.random.default_rng(seed)          # 种子只在这里出现一次
    X = rng.uniform(LO, HI, (pop, DIM))        # 初始种群，全局随机
    F = evaluate(X)                            # 批量评价，一次算 pop 个
    n = pop                                    # 已花预算：初值也要记
    best = float(F.min())

    while n + pop <= budget:                   # 剩余预算不够一代就停
        idx = tournament(F, rng, k=3)          # 选择：3 元锦标赛
        P = X[idx][rng.permutation(pop)]       # 打乱后成对交叉
        C = crossover(P, rng)                  # SBX，eta_c = 15
        C = mutate(C, rng, pm=1.0 / DIM)       # 多项式变异，含边界截断
        Fc = evaluate(C)                       # 子代评价，n_eval += pop
        X, F = elite(X, F, C, Fc, pop)         # 精英保留：合并取前 pop
        n += pop
        best = min(best, float(F.min()))       # 曲线单调，靠的就是这一行

    return best, n                             # 返回值附带实际预算，便于核对
```

这段代码里有三处值得单独说。第一，`n = pop` 把初值算进预算——很多实现漏掉这一步，导致“预算 4000”实际跑了 4040 次。第二，`elite` 是合并父代与子代后取前 pop 个，不是“子代替换父代”，这保证了 (31-14) 的单调性。第三，`while n + pop <= budget` 而不是 `n < budget`：宁可少跑一代，也不让实际评价数超出预算，否则跨算法比较又不公平了。

31.4.7 并行：只改墙钟时间，不改预算

评价一次要一秒时，串行跑 4000 次是一小时以上；这时候必须并行。并行有三种粒度，代价各不相同。

第一种是向量化批量评价。 一次把整个种群传给目标，让 NumPy 或仿真器内部并行。它的优点是确定性完全不变——同样的输入、同样的顺序，结果逐位相同。代价是内存：种群 40、每次评价要存一个 300 点的时域响应，内存占用乘 40。**这是首选，因为它不破坏可重复性。**

第二种是按个体分进程。 8 核跑 40 个个体，墙钟时间降到约 5 个串行批次。要守住两条：预算仍按个体计（40 个个体就是 40 次，不是 1 次）；每个 worker 用 `SeedSequence(seed).spawn(k)` 派生自己的随机流，绝不能让多个进程共享一个 `rng`。

第三种是异步更新。 谁的评价先回来就先更新种群，DE 与 PSO 都有这种变体。它能把空闲核利用到极致，但轨迹依赖“完成的先后次序”，同一份代码换个核数就给出不同结果。**要复现，就必须把归约顺序固定下来**——按个体下标排序后再更新，等价于退回同步。

贝叶斯优化的并行还有一层额外代价。一次挑 q 个点，如果各自独立最大化 EI，这 q 个点会挤在同一片区域——因为它们的后验是同一份，EI 的峰也是同一个。解决办法是“虚拟更新”：挑出第一个点后，假装已经观测到它的后验均值，再挑第二个。q 越大，这种近似越粗糙。**并行买的是时间，付的是样本效率**，而样本效率正是贝叶斯优化存在的理由，所以 q 一般取 4–8，不要贪多。

三条路都遵守同一个不变式：预算按“评价次数”计，不按“批次数”计。 报告里要写清用了几个核、批量多大，否则别人无法判断“4000 次评价”到底是 4000 次真实仿真，还是 100 批 × 40 的向量化调用。

31.5 工程含义与适用边界

31.5.1 先问五个问题，再选算法

面对一个具体问题，选型的顺序比算法本身更重要。按下面的顺序问，通常在第三个问题就有答案了。

第一问：梯度能不能稳定求出来？ 能，回第 29、30 章。不能，才继续。这一问的分量最重，因为梯度法在同样精度下通常快一到两个数量级；只有在梯度不可得时，本章的方法才有存在的理由。

第二问：维度是多少？ 本章的方法在 $d \le 10$ 时还算从容，d 到 20–30 就开始吃力，再往上评价次数会涨到不可接受。高维问题的正确做法不是“换个更强的启发式”，而是先降维、先分块、先把不敏感的参数固定住。**启发式擅长的从来不是“大问题”，而是“导不出梯度的小问题”。**

第三问：预算有多大？ 这一问直接分诊：

- $B < 100$：用贝叶斯优化。这个量级下每个点都必须“挑”出来，随机撒点等于浪费。
- $B \sim 10^3$–10^4：GA、PSO、DE 都行，DE 通常最省心。
- $B > 10^5$ 且维度不高：随机搜索 + 局部精修往往就够了，不必上复杂算法。

第四问：需不需要“保证”？ 需要给出“这是全局最优”或“满足约束”的证书，本章给不了。这时候要么把问题改写成凸形式（第 30 章），要么接受“概率意义”的结论并在报告里写清楚。

第五问：在不在线？ 在线闭环里，单次求解时间必须远小于控制周期。本章的方法几乎都过不了这一关（§31.5.4），能上线的是它们的“离线产物”——查表、规则、或者训练好的策略。

这五问的顺序不能颠倒。先问“哪个算法好”，再回头发现梯度其实可求，是最常见的时间浪费。

31.5.2 参数区间：够用就好

表 31-4 是本章四个算法与贝叶斯优化的常用参数区间。它不保证最优，但保证“不会因为参数离谱而失败”。

表 31-4　常用参数区间与越界后果

算法	参数	常用区间	本章取值	越界的后果
GA	种群 N	20–100	40	太小早熟，太大浪费预算
GA	变异率 p_m	$1/d$–0.1	0.5	太小不探索，太大退化为随机搜索
GA	交叉参数 η_c	10–20	15	太大子代像父代，太小子代乱跳
PSO	惯性 w	0.4–0.9	0.729	需与 $c_1 + c_2$ 一起满足 (31-17)
PSO	学习因子	合计 2.0–2.99	各 1.494	合计超过 $2(1+w)$ 会发散
DE	差分权重 F	0.4–1.0	0.7	$F < 0.4$ 步子太小；$F > 1$ 跨过整个种群
DE	交叉率 C_R	0.5–1.0	0.9	变量强相关时应降到 0.1–0.5
SA	初温 T_0	与典型上坡量同阶	5	太高浪费时间，太低退化为贪心
SA	降温 α	由末温反解	0.9995	按 (31-21) 算，别凭感觉
BO	初值数 n_{init}	5–10	8	太少后验不可信，太多浪费预算

表里最容易被忽略的是 DE 的 C_R。$C_R = 0.9$ 只适合变量之间近似可分的问题；如果两个参数强耦合（比如 K_p 与 K_i 在同一个回路里互相补偿），整条子代都来自变异向量会破坏这种相关性，效果反而不如 $C_R = 0.2$。**判断方法很简单：固定其余参数，把 C_R 从 0.9 降到 0.2 各跑 10 个种子，看中位值有没有变好。** 这类“一维扫描”是最便宜的参数检查，比凭经验拍数可靠。

SA 的初温也不该拍。合理的 T_0 应与“典型上坡移动的 Δf”同阶：太小则一开始就贪心，跳不出局部；太大则前半程几乎全在接受坏解，白花预算。本章 $T_0 = 5$ 对 Rastrigin 的量级（局部坑深约 1–3）是合适的，但末温没降到位（§31.3.4），所以效果仍不理想——**初温与末温要一起定，只调一个没有意义。**

PSO 的三个参数同样不能分开看。表里给的 $w = 0.729$、$c_1 = c_2 = 1.494$ 是 Clerc 收缩因子那一组，满足 $\varphi = 2.988 < 2(1+w) = 3.458$。想靠加大 w 换探索，$c_1 + c_2$ 必须同步调小，否则越过 (31-17) 的边界就开始发散；实践中更常见的做法是让 w 从 0.9 线性降到 0.4、c 保持不变——这时谱半径一路在变，稳定性靠“速度限幅”兜底，而不是靠线性分析。

种群规模则相反，它很不敏感：40 与 100 的中位结果差异通常小于种子间的差异。本章取 40，是因为它在 4000 次预算下能跑 100 代，而 100 个个体只能跑 40 代——**对“靠代数积累”的算法，种群过大反而缩短了可观察的演化长度。**

31.5.3 混合策略：粗搜加精修

启发式与梯度法不是二选一。工程上最稳的组合是“启发式负责找盆地，梯度法负责在盆地里精修”。

理由很直接：本章的方法擅长“在几十个局部极小里挑一个像样的”，但在最优点附近的收敛速度远不如拟牛顿法——DE 用了 4000 次评价才把 Rastrigin 压到 10^{-13}，而 SLSQP 在 §31.6.7 的整定问题上只用 69 次评价就落到最优点。反过来，梯度法从单点出发只能看见一个盆地。两级串联，正好互补。

具体做法有两种。**第一种是串联**：先跑 GA/DE（预算的一半），把最优解当作 SLSQP 的初值。**第二种是并联**：把 GA 的最终种群整体交给 SLSQP 做多起点，保留最好的结果。第二种更稳，代价是评价次数翻倍。

“一半”只是口号，真动手要记账。以本章的 4000 次预算为例，一个合理的拆法是：粗搜给 GA 约 1000 次（种群 40，约 24 代，含初值），依据是 §31.6.1 里 GA 达标的中位评价数是 380 次，1000 次已留出两倍余量；精修给 5 个起点各 100 次（从末代种群前 5 名出发），合计 500；剩下 2500 次不花掉，而是**换两个种子把整条链路重跑一遍**，看结论是否稳定——这部分预算买的是“可信”，不是“更好”。三笔加起来正好 4000，每一笔都能说出去处。

这个拆法背后有两条判据。**其一，精修的改善量是粗搜质量的度量。** 若 SLSQP 在初值上还能把目标压低 10% 以上，说明粗搜停在了盆地边缘而不是盆底，此时应该把预算从精修挪回粗搜，而不是给 SLSQP 加迭代上限。**其二，多起点的收敛分布是多峰性的度量。** 5 个起点若全部收敛到同一个点（用相对容差 10^{-3} 判同），这个景观就是单峰的，启发式在这道题上没有存在价值；若分成两三簇，簇的个数就是“值得保留的候选解”的个数，此时不要只交一个解，要把每簇的代表一起写进报告，因为工程上“次优但鲁棒”常常比“最优但贴边”更可取。

§31.6.7 的对照给出了一组值得注意的数字：GA 用 1200 次评价找到 IAE $= 0.9190$，PSO 找到 0.9189，SLSQP 只用 69 次找到 0.9189；从两个不同初值出发、把容差收到 10^{-12} 的多起点 SLSQP 用了 205 次评价，结果与单起点在四位小数上完全相同。**这说明该问题的盆地只有一个，多起点没有收益，启发式也没有收益。** 遇到这种问题，正确的结论是“不该用启发式”，而不是“启发式也还行”。反过来，如果多起点给出明显不同的解，就说明景观确实多峰，那时启发式的价值才体现出来。

贝叶斯优化与梯度法的组合更微妙。BO 的价值在“评价昂贵”，所以它常被用来整定那些“一次评价要跑几分钟仿真”的参数；此时不能用有限差分（一次梯度要 $d+1$ 次评价），BO 的 40 次预算反而更省。**判断标准是“单次评价的时间”与“总预算”的乘积，而不是问题的维度。**

31.5.4 实时性：为什么它进不了闭环

在线求解的耗时可以粗算为

$$T_{\text{solve}} \approx n_{\text{eval}} \times t_{\text{eval}}, \tag{31-30}$$

t_{eval} 是单次评价的耗时。这个乘积是硬约束：控制周期 10 ms，T_{solve} 就必须小于 10 ms。

§31.6.7 的 PI 整定给出了一个具体刻度：把相位裕度从“网格插值”改成“解析求根”后，单次评价从约 20 ms 降到 1 ms 以内。于是 1200 次评价的总耗时从 24 s 降到 1.2 s——还是比任何闭环周期都大三个数量级。**即使评价已经优化到 1 ms，启发式整定仍然只能离线做。**

把数字摆到周期尺度上会更清楚。10 ms 的周期里，1 ms 一次的评价最多做 10 次，而且这 10 次已经把整个周期占满，没给采样、通信、输出留任何余量。10 次评价对一维问题只够做两轮 DE 的种群（种群 5），对二维只是一次粗糙的坐标扫描。**换句话说，“把算法搬进闭环”在时间账上根本走不通——不是差一点，是差两到三个数量级。** 真正可行的只有“把求解结果搬进闭环”，也就是下面三条绕法。

这条限制的绕法有三条，都不在“把算法调快”这条路上：

1. **离线算、在线查。** 把参数空间网格化，离线用启发式填表，在线双线性插值。代价是维数灾难：二维 100×100 的表格还能接受，四维就爆了。
2. **离线算、在线拟合。** 用启发式生成样本，再拟合一个显式函数（多项式、神经网络）。这就是显式 MPC 的思路，第 58 章会展开。
3. **把优化搬进策略里。** 第 32 章的强化学习把“每次重新优化”换成“训练一次、推理一次前向”，推理耗时可以做到微秒级。

所以本章的方法在控制系统里的位置很清楚：**它是“设计阶段”的工具，不是“运行阶段”的工具。** 参数整定、选型、离线轨迹规划、仿真标定，都在设计阶段；闭环里跑的是它的产物。

31.5.5 一份可复核的实验报告长什么样

启发式方法的结论最容易被“最好的那一次”夸大。要让别人能复核，报告里必须有下面六项。

第一，预算口径。 写 n_{eval} 且注明是否含初值。写“跑了 100 代”是不合格的，因为读者不知道种群多大。

第二，种子数与种子值。 至少 10 个，最好 20 个，并给出种子表（本章是 20261020 起连续 20 个）。只报一个种子等于没做统计。

第三，分布而不是单点。 中位数、四分位区间、最大最小值都要给。§31.6.2 里 GA 的 20 次结果从 1.04×10^{-6} 到 4.59×10^{-4}，只报最小值会把结论夸大两个数量级。

第四，成功率与达标评价数。 前者说明“多大概率够用”，后者说明“多快够用”。两者都比最终值更贴近工程关心的问题。

第五，基线。 至少要有随机搜索作对照，最好再加一个梯度法。没有基线的“我们的算法效果很好”没有信息量。

第六，代码与版本。 包括随机数接口（`default_rng` 还是 `RandomState`）、依赖版本、线程数。这三样决定了别人能不能复现（§31.4.4）。

这六项不需要多高的写作水平，只需要在实验设计时就留好接口。**反过来，如果一份报告缺了其中三项以上，它的数字就不该被引用。**

31.5.6 与后续章节的接口

本章是“优化”这条线的收尾，也是“学习”这条线的起点。几处显式的接口：

- **PID 整定**：第 44 章的规则法（Ziegler-Nichols、IMC）给的是解析公式，第 106 章给的是优化整定。本章 §31.6.7 是这两者的约束版对照——它说明当指标（IAE）与约束（PM、M_s）都由仿真给出时，罚函数 + 启发式能得到什么，以及为什么梯度法在同等问题上更省。
- **MPC**：第 58、80 章在线解 QP，那是第 30 章的地盘，不该用启发式。但 MPC 的权重矩阵、预测时域、终端代价往往离线整定，那里的目标同样是“仿真算出来的”，本章的方法适用。
- **Smith 预估与纯滞后**：第 47、107 章。滞后系统的参数整定对模型误差敏感，用本章的方法做鲁棒性扫描（多个对象模型各跑一遍，看参数分布）比单点整定更实际。
- **强化学习**：第 32 章。策略梯度是“梯度可用的大规模随机优化”，与本章的无梯度方法正好互补：本章解决“导不出”，第 32 章解决“维度太高、样本太多”。两者共同的敌人都是“预算不够”，只是应对方式不同——本章省评价，第 32 章用并行环境堆样本。

一句话总结本章的边界：**它适合“评价昂贵、梯度缺失、维度不高、可以离线”的四重条件下工作；四条里缺一条，就应该先考虑别的工具。**

31.5.7 多少种子才够：从“差不多”到区间

“多跑几次取平均”是对的，但“几次”要有个说法。随机优化的结果跨数量级，用算术平均会被最差的一次拖垮，所以通常看中位数；而中位数本身也有抽样误差。把这两件事分开算，种子数就能定下来。

先看尺度。表 31-5 里 GA 的四分位区间是 $[1.33 \times 10^{-5},\ 2.13 \times 10^{-4}]$，跨 16 倍；DE 是 $[5.24 \times 10^{-14},\ 1.55 \times 10^{-11}]$，跨 300 倍。**在对数尺度上，两者的标准差都在 1 左右**——这是随机优化的常态，不是这两组数据特别糟。于是“要分辨多大的差异”就决定了样本量：若两组的中位相差 r 倍，对数差为 $\ln r$，每组需要的种子数约

$$n \approx 2\left(\frac{1.96\,\sigma_{\log}}{\ln r}\right)^2 .$$

代入 $\sigma_{\log} = 1$：想分辨 10 倍（$\ln r = 2.3$），每组约 2 个；想分辨 2 倍（$\ln r = 0.69$），每组约 16 个；想分辨 1.5 倍（$\ln r = 0.41$），每组要 47 个。**这条式子解释了两件事：为什么本章 20 个种子够用，以及哪些结论它撑不住。** GA 与 PSO 的中位差约 4 个数量级，2 个种子就能分辨，20 个已经绰绰有余；但要比较 GA 与 PSO 的“离散谁更小”，差异只有两三倍，20 个种子远远不够——所以本章只对前者下结论。

再看配对。同一批种子跑两种算法，两者的差异里有一大半来自“这一批种子恰好偏好某种动力学”，是共同成分。用配对比较能把这部分抵消掉：

- **符号检验**：只看“每个种子上谁更好”。表 31-5 里 DE 的最差一次是 1.97×10^{-9}，仍优于 GA 的最好一次 1.04×10^{-6}，所以 20 个种子上 DE 全胜。20 次全胜在“两者无差别”的假设下出现的概率是 $2^{-20} \approx 9.5 \times 10^{-7}$——**不用算方差，结论已经足够硬。**
- **配对差值**：在对数尺度上取 $\log f_{\text{DE}} - \log f_{\text{GA}}$，20 个差值的中位就是“典型差几个数量级”。这比两组各自的中位相减更稳，因为它消掉了种子效应。

配对的前提是“同一对种子面对同一初始条件、同一评价顺序”。本章四个算法共用 `SEED_BASE` 起的同一批种子，正是为了这个。

所以种子数不是越多越好，而是要与要分辨的差异匹配。 报 20 个种子、给出 IQR 与成功率，足以支撑“差一个数量级”级别的结论；如果结论是“我们的方法比基线好 20%”，那要么把种子加到 50 以上，要么承认这个差异在噪声里。

31.6 仿真与验证

本章全部数字来自 `code/ch031_sim.py` 的七段实验与 `code/ch031_figs.py` 的六张图，两者共用同一份计算代码。公共设置与 §31.2.2 一致：Rastrigin（除特别说明）、$d = 2$、定义域 $[-5.12, 5.12]^2$、种群 40、预算 4000、20 个独立种子（20261020 起连续取）。记账一律用 n_{eval}，含初值。

31.6.1 E1：同一预算下的四种算法

表 31-5 是四种算法在 Rastrigin 上的 20 次结果。

表 31-5　四种算法的 Rastrigin 结果（预算 4000，20 种子）

算法	中位	四分位区间	最小	最大	成功率	达标评价数
GA	7.87×10^{-5}	$[1.33 \times 10^{-5}, 2.13 \times 10^{-4}]$	1.04×10^{-6}	4.59×10^{-4}	100%	380
PSO	2.17×10^{-9}	$[2.19 \times 10^{-10}, 1.47 \times 10^{-8}]$	1.17×10^{-11}	1.18×10^{-7}	100%	540
DE	4.53×10^{-13}	$[5.24 \times 10^{-14}, 1.55 \times 10^{-11}]$	0	1.97×10^{-9}	100%	700
SA	3.53×10^{-1}	$[5.24 \times 10^{-2}, 1.02]$	1.20×10^{-3}	4.08	5%	1140^{*}

表注：成功率定义为最终值 $< 10^{-2}$ 的种子占比；达标评价数是曲线首次低于 $f < 1.0$ 时已花掉的评价次数的中位数。*SA 的 20 个种子里只有 1 个达标，1140 是那一次的数，不能当分位数看。

三个结论可以直接从表里读出来。

第一，精度差得很远。 DE 的中位是 4.53×10^{-13}，SA 是 3.53×10^{-1}，两者相差 12 个数量级。GA 到 PSO 差 4 个数量级，PSO 到 DE 又差 3 个多。这个排序不是“算法优劣”的普适结论，而是“结构是否匹配景观”的具体体现：DE 的差向量让它沿着种群的分布方向持续试探，PSO 靠全局最优牵引，GA 靠交叉重组但选择压力只有 2.9 倍（§31.3.2），SA 的温度没降到位（§31.3.4）。

第二，速度排序与精度排序相反。 GA 平均 380 次评价就达标，PSO 540 次，DE 700 次——精度最高的 DE 反而最慢达标。原因不难理解：GA 的精英保留会把种群迅速压进最近的一个坑，坑底通常在 10^{-1} 到 10^{-4} 之间，所以它很快越过 $f < 1.0$ 这条线，然后就停在那里；DE 在前 700 次里还在全局游走，越过阈值晚，但越过后继续下探。**这两件事必须分开报告，否则“谁更好”会变成一个没有答案的问题。**

第三，SA 的失败是参数失败，不是算法失败。 5% 的成功率对应的不是“退火不行”，而是末温 0.68 时仍有 36% 的概率接受上坡（§31.3.4）。若按 (31-21) 把 α 改成 0.99827，同样的 4000 次预算下它会冷到 5×10^{-3}，末段的行为就接近贪心。这一条不打算用重跑证明，因为它属于“设计规则”，读者可以在自己的问题上验证。

还有一个容易被表格掩盖的细节：GA 的最小值 1.04×10^{-6} 与中位数 7.87×10^{-5} 差 76 倍。如果只报“最好的一次”，GA 看起来比 PSO 还准；这不是造假，是选择性报告。§31.6.2 专门处理这个问题。

31.6.2 E2：只换种子，结果差多少

把同一份代码、同一预算、只换种子的结果排在一起，就是图 31-2 背后的那张分布。表 31-6 给出三个统计量。

表 31-6　种子敏感性（Rastrigin，预算 4000，20 种子）

算法	最小	中位	最大	最大/中位
GA	1.04×10^{-6}	7.87×10^{-5}	4.59×10^{-4}	5.83
DE	0（精确命中）	4.53×10^{-13}	1.97×10^{-9}	1968

这里用“最大/中位”而不是“最大/最小”，是因为 DE 有一次精确命中了 0，除以 0 没有意义。即便换成这个更温和的比值，DE 的 1968 倍仍然比 GA 的 5.83 倍大两个多数量级。

为什么精度越高、相对离散越大？ 因为误差是按乘性衰减的：DE 的末值大致是 $\exp(-\rho k)$ 的形式，20 个种子的收敛速率 ρ 有微小差异，$k = 100$ 代之后这点差异被指数放大成几个数量级。GA 停在一个局部坑的坑底，坑底深度大致在 10^{-4} 量级，种子之间差别不大——不是它更“稳定”，而是它更早停住了，没有机会把差异放大。**“离散小”在这里是停滞的同义词。**

工程上该看哪个数，取决于要求有多硬：

- 只要 $f < 10^{-2}$：三种种群算法都是 100%，随便挑，挑评价次数最少的（GA）。
- 要 $f < 10^{-8}$：GA 的 20 个种子一个都不过；PSO 的中位过，但最差的一次 1.18×10^{-7} 会被卡住；只有 DE 的 20 个种子全过。
- 要可预测的交付时间：看达标评价数的分布，而不是最终值的分布。

最后一句是本节的要点：**随机优化给出的不是一个数，是一个可达分布。** 报分布不是“更严谨”的修辞，而是唯一能回答“这批货能不能按时交”的形式。

表里还有一个不显眼但重要的事实：PSO 的 IQR $[2.19 \times 10^{-10},\ 1.47 \times 10^{-8}]$ 比 DE 的 $[5.24 \times 10^{-14},\ 1.55 \times 10^{-11}]$ 窄，中位却差 4 个数量级。**这不是“PSO 更稳”，而是“PSO 停得更整齐”**：粒子被同一个 g 牵着，20 个种子最后都收敛到 g 附近，差别只来自 g 落在哪个坑；DE 的种群持续收缩、误差按乘性衰减，种子之间收敛速率的微小差异被指数放大。**“离散小”在两处含义相反：对 PSO 是早熟的证据，对 GA 是停滞的证据。** 看到窄 IQR，先问“它是收敛到了同一个点，还是压根没走远”，而不是急着夸它稳定。

31.6.3 收敛曲线怎么读（E1 的中位曲线）

图 31-2 是四种算法的中位收敛曲线：横轴评价次数，纵轴“到目前为止的最优值”，纵轴取对数。

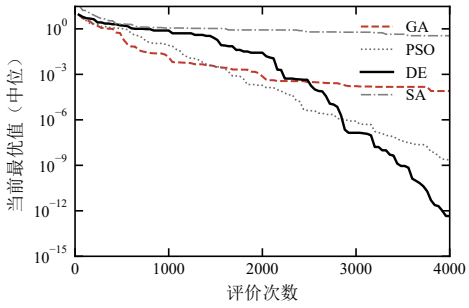


图 31-2　四种算法的中位收敛曲线

半对数坐标下的斜率有明确含义：**斜率恒定就是指数衰减，斜率变缓就是“卡住了”。** 据此读四条曲线：

GA 是台阶状的。 一大段水平、突然掉一格、再水平。这是精英保留 + 弱选择压力的必然结果：种群要积累若干代才凑出一个更好的个体，凑出来就整段下移。它的下降分两段——前 380 次把值压到 1 以下（进入某个坑），后 3600 次在坑内从 1 细化到 10^{-4}。两段的斜率差一个数量级，说明后半段基本是在原地磨。

DE 最接近一条直线。 在半对数坐标上近似直线，意味着每代的相对改善量恒定，这正是 §31.3.4 所说的“步长随种群宽度收缩”的结果。它起步慢（前 700 次还在 1 以上），但斜率一直保持，最后冲到 10^{-13}。**直线型曲线是“还没有停滞”的信号，台阶型曲线是“已经停滞”的信号。**

PSO 前陡后缓。 前 20 代靠全局最优牵引，下降很快；之后 g 长期不动，粒子在它附近震荡，斜率明显变缓。这是“早熟”的典型形态——不是不收敛，是收敛到了一个不动的 g 上。

SA 几乎全程平缓。 它的曲线在整个 4000 次里缓慢下移，末值仍在 10^{-1} 量级。这条曲线最该引起警觉的地方不是终值高低，而是**它始终没有出现明显的下降段**——说明温度与邻域尺度都不支持“从坑里跳出来再落进更深的坑”。

读曲线还有一个技术细节：图 31-2 画的是 20 个种子的中位，不是某一次。中位曲线会掩盖“个别种子很早就掉下去”这种信息，所以表 31-5 的最小值一列必须与图一起看。两者不矛盾，它们回答的是两个问题：中位回答“典型情况”，最小回答“最好能到什么程度”。

曲线的第三个用途是判断“该不该加预算”，这比判断“谁更好”更实用。规则只有两条：**若预算末端的斜率仍为负且稳定，就该加预算**——说明算法还没停滞，多给一倍大概率能多拿一个数量级（E3 的 1000→2000 就是这种情形）；**若末端已平坦超过总预算的三分之一，加预算没有意义**——GA 的后 3600 次只从 1 磨到 10^{-4}，PSO 的后半程几乎水平，它们的瓶颈是多样性而不是预算。用这两条看本章四条曲线，结论很干脆：只有 DE 值得加预算。

还有一个容易误读的形态：SA 那条线“全程缓慢下移”，看起来一直在进步，实际上每 100 次只降几个百分点。**判断“是否在进步”要看斜率，不要看“有没有下降”。** 纵轴取对数就是为了这件事——线性坐标下所有曲线都像在往下走，对数坐标下停滞立刻现形。

31.6.4 E3：预算翻倍值多少

图 31-3 与表 31-7 是 DE 在六档预算下的中位结果。每档 10 个种子，共 60 次运行。

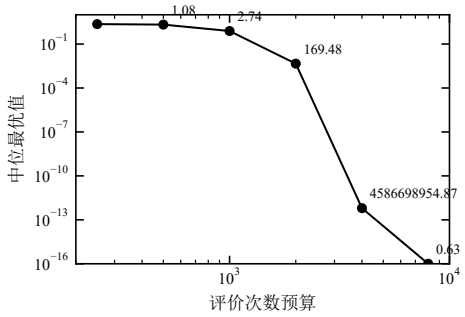


图 31-3　预算与精度的关系

表 31-7　预算的边际收益（DE，10 种子）

预算	中位最优值	相对上一档的改善比
250	2.30	—
500	2.13	1.08
1000	0.773	2.74
2000	4.59×10^{-3}	169
4000	6.27×10^{-13}	10^{10} 量级
8000	0	已到 0，无定义

这张表最反直觉的地方是它**不平滑**。预算从 250 翻到 500，中位只从 2.30 降到 2.13，改善比 1.08——几乎白花了一倍的钱。原因很朴素：种群 40 时，250 次评价只够跑 6 代，500 次只有 12 代，而 DE 要十几代才能让差向量收缩到有意义的尺度。**在“代数还不够”的区间里加预算，等于给一台还没热起来的机器加料。**

然后是一个断崖：1000 到 2000，改善比 169；2000 到 4000，直接跨了 9 个多数量级。这不是“渐进变好”，而是**跨过了一个门槛**——预算足够让种群找到正确的盆地，之后差向量在这个盆地里快速收缩。最后 8000 的中位是精确的 0，程序里那一格打印的 0.63 是除零保护的副产物（分母被兜到 10^{-12}），真实含义是“已经到 0，无从再改善”。

这条曲线对实践的直接含义是：**“预算加一点试试”是错误的调参方式。** 阶梯型的收益曲线意味着，在门槛以下加预算几乎没用，跨过门槛才有质变。正确的做法是先做一次预算扫描（本例只花了 60 次运行），看清自己的问题落在哪一级台阶上，再决定是加预算还是换算法。

顺带说一句样本量：E3 用 10 个种子而不是 20 个，因为六档预算要跑 60 次。种子少一半，中位数的抖动会变大——尤其是 2000 档附近的 4.59×10^{-3}，换个种子集可能在 10^{-2} 与 10^{-3} 之间摆动。**做预算扫描时，档位之间的比较只看数量级，不要计较 20% 的差别。**

这张表还能读出一条更有用的规律：**门槛是按“代数”定价的，不是按“次数”定价。** DE 在 1000→2000 之间跨门槛，对应 25→50 代；若把种群从 40 加到 80，同样的 25→50 代就需要 2000→4000 次评价。所以“多少预算够用”没有通用答案，它与种群规模绑定——报预算时必须同时报种群，否则别人无法换算。这也解释了为什么有的文献说“DE 要一万次”、有的说“两千次就够”：它们的种群一个是 100、一个是 20，代数却在同一量级。

实践上的做法是：先固定种群（经验值是 $10d$–$20d$，二维即 20–40，本章取 40），扫一遍预算找到自己的台阶，再把台阶换算成代数记下来。**记代数而不是记次数，换种群时才能直接复用。**

31.6.5 E4：评价带噪声时会怎样

实测数据的整定、试验台标定、蒙特卡洛仿真，评价都带噪声。E4 给 Rastrigin 加 $\mathcal{N}(0,1)$ 噪声，用 DE 跑 4000 次评价，比较两种策略：每次评价只采一次，和每次评价取 5 次平均（总预算仍是 4000 次，因此有效点数变成 800 个）。

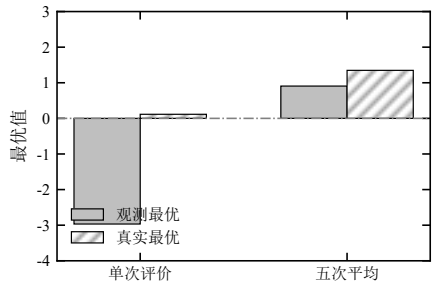


图 31-4　观测最优与真实最优的差距

表 31-8　噪声下的乐观偏差（DE，预算 4000，20 种子）

策略	观测最优（中位）	真值复核（中位）	偏差（中位）
单次评价	-2.968	0.115	-3.048
五次平均	0.907	1.349	-0.232

第一行就有一个不需要任何统计知识就能发现的错误：**Rastrigin 处处非负，观测到 -2.968 是不可能的。** 这个负值不是算法的错，是噪声的错——优化器在 4000 个带噪样本里挑最小的那个，而“挑最小”这个动作本身就偏向于挑到“噪声恰好很负”的点。这叫乐观偏差（optimizer’s curse）：**你报告的最好值，是真实值与噪声最有利的一次实现之和。**

偏差有多大是可以粗估的。若把 4000 次评价近似看成 4000 个独立样本，最小值的期望大约比真实最小值低 $\sigma\sqrt{2\ln n}$，$n=4000$ 时约 4.1σ。实测偏差 3.05σ，同量级——考虑到 DE 的种群高度相关、有效独立样本远少于 4000，这个偏小是合理的。**这条粗估的价值在于：它告诉你偏差随 $\sqrt{\ln n}$ 缓慢增长，加预算会让偏差变大而不是变小。**

第二行是解决办法：五次平均把单次噪声的标准差从 1 降到 $1/\sqrt{5}=0.447$，偏差从 -3.05 降到 -0.232，缩小了 13 倍。代价是有效评价点从 4000 个变成 800 个，全局搜索能力下降——但在本例里，降噪的收益远大于点数减少的损失。

由此得到两条操作规范。**第一，报“最优值”之前必须用真值复核。** 复核方法很简单：把找到的 x 拿回无噪声的目标里算一遍。本章的 0.115 与 1.349 就是这么来的。**第二，噪声与目标同量级时，优先降噪而不是加预算。** 加预算只会让偏差按 $\sqrt{\ln n}$ 增长，降噪是直接从根上把 σ 压下去。

重复次数 m 怎么定，是一笔两头受挤的账：噪声按 $1/\sqrt{m}$ 降，有效点数按 $1/m$ 降，前者是收益，后者是成本。若问题需要至少 $N_{\min}$ 个有效点才能摸到盆地（本章 Rastrigin 是 10^3 量级，见 E3 的 1000 档），那么必须 $m \le B/N_{\min}$。预算 4000、$N_{\min}=1000$ 时 m 最多取 4；本章取 5，已经踩在边上——表 31-8 第二行的 800 点确实还在盆地附近，但真值 1.349 比单次评价那次的 0.115 高，说明它落的坑不如前者深。

所以“五次平均赢”是有条件的：它赢在偏差，输在探索。 噪声只有目标的 1% 时，$m=1$ 就够，多采纯属浪费；噪声与目标同量级时，m 要取到 5–10，并接受有效点数变少的代价。判断方法很实用：先跑一遍 $m=1$，看观测最优是否明显低于目标的物理下界（Rastrigin 是 0，报出负数就是信号），有信号再逐级加 m，每次看真值复核的结果有没有变好。

31.6.6 E5：贝叶斯优化在 40 次评价下赢多少

E5 换了问题：Himmelblau，定义域 $[-5,5]^2$，预算只有 40 次评价。BO 用 8 个随机初值 + 32 次 EI 迭代；对照组是同样 40 次的随机搜索。20 个种子。

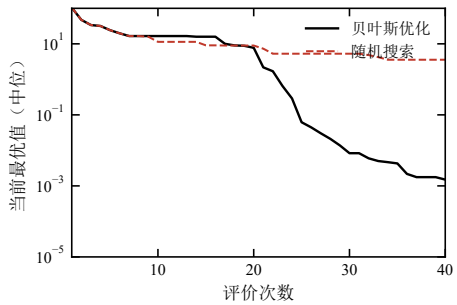


图 31-5　贝叶斯优化与随机搜索的累计最优

表 31-9　40 次评价下的 BO 与随机搜索（Himmelblau，20 种子）

方法	中位	四分位区间
贝叶斯优化	1.50×10^{-3}	$[5.91\times10^{-4},\,4.57\times10^{-3}]$
随机搜索	3.57	$[1.34,\,7.31]$

中位数相差 2372 倍。这个差距不是“BO 更聪明”，而是“BO 没有浪费样本”：随机搜索把 40 个点均匀撒在 10×10 的区域里，平均间距 $\sqrt{100/40}=1.58$；Himmelblau 的四个最优坑半径远小于这个数，所以 40 个点大概率一个都没落进坑心。BO 从 8 个初值建起后验，EI 会自动把后续 32 个点往“预测值低或不确定性大”的地方推，等效于把点集中到了坑附近。

差距的量级可以用覆盖率解释。要让随机搜索以较大概率命中一个半径 r 的坑，需要约 $n\approx(10/r)^2$ 个点。BO 的中位 1.5×10^{-3} 对应坑半径 $r\approx\sqrt{2f/\lambda_{\min}}$；Himmelblau 在最优点 $(3,2)$ 处的 Hessian 为 $\begin{bmatrix}10&12\\12&34\end{bmatrix}$，$\lambda_{\min}\approx5$，于是 $r\approx0.025$——随机搜索要命中这个尺度需要约 1.6×10^{5} 个点。**这就是“信息利用”与“均匀撒点”的差别，也是“昂贵评价”场景下必须用代理模型的原因。**

但 BO 也没有到 0，中位停在 1.5×10^{-3}。原因在核宽度：本章取 $\ell=0.8$，而 40 个点的平均间距 1.58，ℓ 小于间距意味着两点之间后验迅速退回先验，模型“看不到”更细的结构。把 ℓ 调大能改善插值，但会抹平多峰；调小则后验不可信。**核宽度应当与“你希望分辨的景观尺度”匹配，而不是与定义域宽度匹配。** 这一条没有普适公式，只能靠边际似然或几次试跑来定。

最后要说清 BO 的成本账。本例每次迭代要算 3500 个候选点的 EI，$n=40$ 时后验分解是 $O(40^3)$，单次迭代约 1 ms 量级——相对 40 次评价的总时间可以忽略。但如果被优化的对象是一次 1 μs 的解析函数，BO 反而是慢的。**BO 的适用条件是“评价昂贵到值得为它建模”，不是“维度低”。**

E5 用满了 40 次评价，实际使用时要设停止准则。两条最常用：**一是 EI 的最大值低于“历史最优改进的 1%”**，说明继续采样的期望收益已经可以忽略；**二是连续 5 次迭代没有刷新最优**。前者更讲道理，后者更好实现。BO 比 GA、DE 更需要停止准则，因为它每轮都有 $O(n^3)$ 的确定成本，而收益递减得非常快——第 30 次和第 40 次评价带来的改进，通常已经小于建模误差。

31.6.7 E6：PI 整定，启发式并没有赢

E6 回到第 30 章最后一个问题：对象 $G(s)=1/(s+1)^2$，PI 控制，目标是最小化阶跃响应的 IAE，约束是 $\mathrm{PM}\ge45^\circ$、$M_s\le2$，参数范围 $K_p,K_i\in[0.05,5]$。评价函数与第 30 章同口径，但时间网格取 300 点、PM 用解析求根，单次评价约 1 ms。

表 31-10　PI 整定的四种解法（同一问题、同一约束）

方法	K_p	K_i	IAE	PM/(°)	M_s	评价次数
GA	4.2668	1.6908	0.9190	45.00	1.639	1200
PSO	4.2676	1.6912	0.9189	45.00	1.639	1200
SLSQP（四初值取优）	4.2686	1.6905	0.9189	45.00	1.639	69
多起点 SLSQP（两个初值，$\mathrm{ftol}=10^{-12}$）	4.2686	1.6905	0.9189	45.00	1.639	205

第一，四种方法给出了同一个解。 K_p 差 0.04%，IAE 差 0.01%。这不是巧合，是问题本身的性质：二维、目标近似凸、约束把解钉在 $\mathrm{PM}=45^\circ$ 这条边上，最优点唯一且边界光滑。在这种问题上，任何“能收敛”的方法都会落到同一点——**方法之间的差别只剩评价次数。**

第二，罚函数确实把解压到了约束边界上。 PM 精确到 45.00°，违约量近似为 0。这说明 §31.4.2 里 $\rho=5$ 的配平是有效的：如果再小一个量级，解会稍微缩进可行域内侧，IAE 会略高，PM 会略大于 45。

第三，评价次数的对比要看口径。 SLSQP 的 69 次是“四个初值里胜出的那一次”的评价数，四个初值各跑一遍，合计要乘上几倍；表里比较的是“达到同等结果各自花了多少”，不是“谁单步更便宜”。205 次那一行把初值减到两个、却把 ftol 从默认的 10^{-6} 收到 10^{-12}，合计 205 次，结果与 69 次那次在四位小数上完全相同——两个初值平均每次约 100 次，只是单次的 1.5 倍。**在这个二维、光滑、近似凸的问题上，容差收严六个数量级既没换来改善，也没让预算失控；容差确实是隐藏的预算开关，但它真正咬人的场合是目标病态的时候（§31.6.8 的步长扫描里有现成的例子）。**

第四，与第 30 章的 0.9192 对照。 第 30 章的 E7 得到 $K_p=4.3198$、IAE $=0.9192$；本章得到 $K_p=4.2686$、IAE $=0.9189$。两者差 0.03%，来源是时间网格（300 点对更细的网格）与积分方式。这个差别比“方法之间的差别”还大——**又一次说明：在评价精度没定下来之前，比较算法是没有意义的。**

那启发式在这个问题上的价值在哪里？答案是：**当评价不可微且抖动大时。** 第 30 章用 2 万点网格插值算 PM 时，评价抖动约 0.01°，有限差分把抖动放大成方向错误的梯度，SLSQP 停在 IAE $=0.974$ 的次优点；换成解析求根后立刻到 0.919。而 GA 与 PSO 从不做差分，

它们只比较两个函数值的大小，抖动 0.01° 最多让某次比较判错，不会污染整个搜索方向。**所以“启发式 vs 梯度法”的胜负，取决于评价的噪声水平，而不是问题的维数。** 本节这张表之所以四法打平，恰恰是因为评价已经被磨得很干净了。

还有一条工程细节：K_p 的上界是 5，最优解 4.27，离边界还有 15% 的余量；K_i 的界更远。这说明参数范围不是瓶颈。**如果最优解贴在边界上，先怀疑界设窄了，再怀疑算法。**

罚函数的形状也值得在这里补一句。本章用的是二次罚 $5\max(0,45-\mathrm{PM})^2$，它在边界外一阶导为零、二阶导为正，理论上解会略微“陷进”可行域内侧，不可能精确等于 45；实测到两位小数正好是 45.00，说明 ρ 已经足够陡。若换成一次罚 $\rho\max(0,45-\mathrm{PM})$，解会精确落在边界上，但目标在边界处不可微，梯度法会抖，启发式则不受影响。**二次罚平滑但不精确，一次罚精确但不可微——这个取舍在梯度法里躲不掉，在启发式里可以随意，这是无梯度方法的少数优势之一。**

判断 ρ 够不够大，有一个不依赖真解的检验：把 ρ 乘 2 再跑一遍，若解移动超过容差，说明罚项还不够陡；若几乎不动，就进入了“罚项足够”的区间。这条检验比“看违约量”更灵敏——违约量可能被大 ρ 压成 0，掩盖了“其实还能再优化”的事实。

31.6.8 E7：三个 SGD 变体在病态二次目标上

E7 换到“梯度可用”的场景，目的是给第 27 章的深度学习做量级铺垫。目标 $f(x)=\tfrac12 x^{\top}Ax$，$A=\mathrm{diag}(1,100)$，条件数 100，初值 $(1,1)$，跑 300 步。

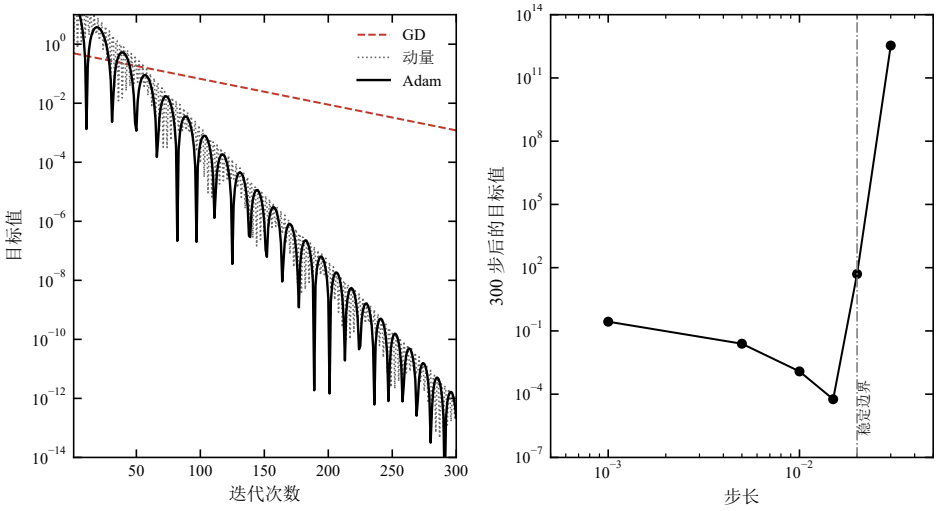


图 31-6　三种 SGD 变体的收敛与步长敏感性

表 31-11　三个变体的收敛过程（初值 $(1,1)$，300 步）

变体	步长	50 步	100 步	300 步
GD	0.01	1.83×10^{-1}	6.70×10^{-2}	1.20×10^{-3}
动量（$\beta=0.9$）	0.01	1.25×10^{-1}	9.46×10^{-5}	1.20×10^{-12}
Adam	0.1	1.17×10^{-3}	4.36×10^{-4}	2.25×10^{-13}

三条曲线的形状完全不同，而它们优化的是同一个函数。

GD 是一条直线（在半对数坐标上）。 斜率由慢模态的 $1-\eta\lambda_{\min}=0.99$ 决定，300 步后到 1.2×10^{-3}，与 §31.3.6 的解析式对得上。它的“慢”不是 bug，是条件数：$\kappa=100$ 意味着慢方向每步只缩 1%。

动量法在前 100 步内完成了几乎全部下降。 50 步到 100 步之间，损失从 1.25×10^{-1} 掉到 9.46×10^{-5}，降了三个数量级；同期 GD 只降了 2.7 倍。原因是一致方向上的梯度被累积，等效步长放大 $1/(1-\beta)=10$ 倍。但它也有代价：动量法对步长更敏感，η 稍大就会在快模态上振荡。

Adam 的路径最不均匀。 它在前 50 步就压到 1.17×10^{-3}，比 GD 快 156 倍；但 50 步到 100 步只降 2.7 倍，看起来像“卡住了”；之后 200 步又降了 9 个数量级。这条曲线是“自适应步长”的典型形态：分母 $\sqrt{\hat s}$ 记住了历史上的大梯度，前期有效地把两个坐标拉到同一尺度，中段进入一个平台，末段梯度变小、动量与梯度同向，才恢复几何收敛。**只看前 50 步或只看中段，都会得出错误结论。**

图 31-6 的右图是 GD 的步长扫描，横轴步长、纵轴 300 步后的目标值，竖线是理论边界 $2/\lambda_{\max}=0.02$。曲线在 0.015 处最低（5.76×10^{-5}），在 0.02 处突然抬到 5.0×10^{1}，在 0.03 处 18 步就发散到 3.44×10^{12}。**边界的位置与理论值严丝合缝，这不是拟合**

出来的，是 $\eta < 2/\lambda_{\max}$ 的直接体现。

三个变体给工程的三条经验：

1. **步长的量级应从小到大试，先定位边界再往回退。** 本章的最佳步长 0.015 是边界 0.02 的 75%，而不是“越小越稳”。
2. **动量与自适应方法能改变收敛速度，但不能改变“病态”这件事。** 条件数 100 依然在，只是慢方向被放大了。真正的解法是预处理（白化、归一化），第 27 章会讲。
3. **报告的步数必须与容差一起报。** “300 步收敛到 10^{-3}”与“300 步收敛到 10^{-13}”是两件完全不同的事，而它们只差一个动量项。

还有一条被省略的东西：学习率衰减。本章三个变体都用常数步长，所以 Adam 末值停在 2.25×10^{-13} 而不是精确的 0——这不是数值精度问题，而是常数步长的固有性质：Adam 把每步位移归一化到约 η 的量级，梯度再小也推不出这个分辨率，末值就停在一个由动量与曲率共同决定的平台上，而不是精确的 0。换成 $\eta_k = \eta_0/(1 + k/100)$ 这类衰减，步长持续缩小，末值理论上能再降几个数量级。**代价是多一个超参数、多一条调度曲线，调度错了会前功尽弃**——第 27 章会讲余弦退火与 warmup 的定法，这里只需记住：常数步长收敛到“邻域”，衰减步长收敛到“点”。

三个变体在 300 步内都“收敛”了，但收敛的含义各不相同：GD 到 10^{-3}、动量到 10^{-12}、Adam 到 10^{-13}。**同一张图上三条曲线都趋平，可它们离真解的相对距离差十个数量级。** 这就是为什么 §31.7.4 的自检里要求把容差与步数一起写：只画曲线，看不出这件事。

31.6.9 七组实验放一起看

七组实验各自回答一个问题，单独看都容易过度解读；并排放，才能看出这一章的骨架。表 31-12 是索引。

表 31-12　本章七组实验的一览

实验	问题	关键数字	出处	一句话结论
E1	同预算下谁更强	DE 中位 4.53×10^{-13}；SA 成功率 5%	表 31-5、图 31-2	精度与达标速度不是一回事
E2	换种子能差多少	GA 5.83 倍，DE 1968 倍	表 31-6	单次运行的结果不能写进报告
E3	加预算值多少	250→500 改善 1.08 倍；1000→2000 改善 169 倍	表 31-7、图 31-3	收益是台阶，不是斜坡
E4	评价带噪声会怎样	单次评价偏差 -3.05，五次平均 -0.23	表 31-8、图 31-4	优化器会挑中“运气好的点”
E5	贵评价怎么办	BO 中位 1.50×10^{-3}，随机搜索 3.57	表 31-9、图 31-5	少而准胜过多而散
E6	整定能不能赢梯度法	四法 IAE 0.9189–0.9190，评价 69–1200 次	表 31-10	单峰问题上启发式没有收益
E7	梯度可用时怎么走	300 步：GD 1.2×10^{-3}，动量 1.2×10^{-12}	表 31-11、图 31-6	条件数不会因为动量而消失

三组对照关系值得单独点出。**E1 与 E6 是一对反例。** 在 Rastrigin 上，DE 把 GA 甩开九个数量级；在 PI 整定上，四种方法的最好值只差 0.0001。差别不在算法，在景观：前者有几十个坑，后者只有一个盆地。**同一批代码，换一道题就从“天差地别”变成“完全打平”，这本身就说明“哪种算法更好”是个伪问题。**

E2 与 E3 合起来定义了“预算”这个词。 E2 说同预算下结果的离散可达三个数量级，E3 说预算本身又值好几个数量级；两者相乘，就是为什么单次运行的数字毫无意义。报告里必须同时给出种子数、预算与达标阈值，缺一项，那组数就没法被别人复现，也没法被自己半年后看懂。

E4 与 E5 是“评价”这一侧的两种病。 一个是噪声让比较失真，一个是评价太贵不能多采。前者要靠重复评价与统计检验治，后者要靠代理模型治；两者的共同点是：**优化器本身没有错，错的是喂给它的信息。**

复现全部结果只需要三条命令（约 50 s）：

```
& "D:\Anaconda3\envs\py311\python.exe" book\code\ch031_sim.py    # 七组实验，打印全部数字
& "D:\Anaconda3\envs\py311\python.exe" book\code\ch031_figs.py   # 六张图，SVG + PDF
& "D:\Anaconda3\envs\py311\python.exe" book\tools\build.py --chapter 31   # 排版出 PDF
```

正文里的每一个数都能在 `ch031_sim.py` 的打印里找到同名条目，图与表共用同一份输出——没有一处是手抄的。**这是本章唯一值得坚持的工程习惯：让数字只有一条来源。**

31.7 小结与常见错误

31.7.1 三条主线

本章三十条公式、十三张表、六张图，归根到底在讲三件事。

第一件，预算是第一等公民。 评价次数 n_{eval} 不是附注，是约束 (31-2)。它决定了方法选型（40 次用 BO，4000 次用 DE，4 万次用随机搜索加精修）、决定了怎么读数（达标评价数比最终值更贴近交付）、也决定了怎么比较（同预算，含初值，容差一致）。§31.6.4 的阶梯曲线说明，预算与精度之间不是平滑关系，跨门槛前后差九个数量级。

第二件，结果是分布，不是点。 20 个种子给出 20 个不同的末值，报中位、IQR、最值、成功率，才构成一个可用的结论。§31.6.2 里 GA 的“最大/中位”是 5.83，DE 是 1968——精度越高，相对离散越大，因为误差被指数放大。“哪个算法更稳”这个问题，必须先说清是在哪个精度档位上问的。

第三件，随机性要被结构约束。 四种算法都是“有偏的随机移动”：GA 的偏在交叉与选择，PSO 的偏在全局最优，DE 的偏在种群差向量，SA 的偏在温度。§31.3 那几条推导（选择压力 2.9 倍、PSO 谱半径 0.854、DE 步长 0.79σ、SA 末温 0.68）都是把“偏”量化，量化之后才知道参数该怎么设、什么时候会失效。

三件事合起来是一句话：**无梯度优化不承诺最优，它承诺“在给定预算下，给出一个可复现、可比较、可解释的分布”。**

31.7.2 一页纸的方法选择

表 31-13 把 §31.5.1 的五个问题压成一张表，按“预算 × 评价成本”两个维度分诊。

表 31-13　方法选择的一页纸

场景	推荐方法	典型预算	主要理由
目标可微、约束是线性的	第 30 章 QP / SLSQP	10^1–10^2	有 KKT 保证，最快
目标可微、评价有抖动	先修评价，再谈算法	—	抖动会毁掉差分梯度
黑箱、$B < 100$	贝叶斯优化	10^1–10^2	每个点都要“挑”出来
黑箱、$B \sim 10^3$–10^4	DE 或 GA	10^3–10^4	差向量自适应、实现简单
离散选型（档位、型谱）	GA + 索引编码	10^3	交叉能组合不同档位
离散组合不到 100 种	全枚举	$\le 10^2$	枚举保证全局最优，且可复核
目标带阶跃、约束很硬	罚函数 + DE	10^3–10^4	阶跃会毁掉梯度与代理模型
评价可并行、单次很贵	BO + 批量（$q = 4$–8）	10^2	并行买时间，样本效率仍优先
需要在线求解	离线算、在线查表	—	$n_{\text{eval}} t_{\text{eval}}$ 过不了周期

表里没有“最强算法”这一行。**每一行都是“在某个预算与成本组合下，最不容易犯错的算法”**，而不是“平均性能最好的算法”。也没有“混合策略”这一行——混合不是一种算法，而是一种预算分配（§31.5.3），它适用于表里的每一行，所以单独放在正文里讲更清楚。

用这张表时有个顺序：先按“能不能求梯度”分成上下两半，再按预算落在哪一档。**两个判断做完，通常只剩一个候选。** 如果还剩两个，选实现更简单、更容易复现的那个——本章的全部证据都指向同一个结论：算法之间的差距，很少大于“参数没调好”与“报告不规范”带来的差距。

31.7.3 十个常见错误

错误一：预算漏算初值。 种群 40、预算 4000，实际跑了 4040 次。样本量小的时候这一条会让“同预算比较”直接失效。检查方法：让函数在入口计数，与理论值对一遍。还有一种更隐蔽的漏算——某次评价抛异常后重跑，计数器却只加了一次；一旦有重试逻辑，计数必须加在“真正调用”那一层。

错误二：用最好的一次报结果。 GA 的 20 个种子里最小是 1.04×10^{-6}，中位是 7.87×10^{-5}，差 76 倍。只报最小值不造假，但会误导。规范是“中位 + IQR + 成功率”三件套。这类报告的典型后果是：按最小值定的参数，装到第二台设备上就超差。

错误三：只报代数，不报评价次数。 “跑了 100 代”在种群 40 与种群 100 之间差了 2.5 倍预算。跨算法比较时，代数是不可比的单位。同理，“跑得快”也必须换算成次数或秒数，因为向量化评价与逐点评价的耗时能差十倍。

错误四：不同算法用不同预算。 常见变形是“GA 跑满 5000 次，梯度法跑到收敛为止”。要么都跑满，要么都按同一阈值停，省下的预算不能挪用（§31.2.5）。想省预算，就给所有方法设同一个容差让它们各自停——这样省下的次数是**结果**，不是给某个方法的优惠。

错误五：把有限差分用在带噪评价上。 评价抖动 0.01°、差分步长 10^{-8}，梯度会被噪声完全淹没。第 30 章的 0.974 就是这么来的。判断方法：同一参数连算三次，看结果是否完全相同。若必须用差分，退路只有两条——中心差分加上每个点重复 m 次取平均（噪声降到 $1/\sqrt{m}$，代价是 $2m$ 倍预算），或者干脆换成 §31.3.5 的代理模型。

错误六：罚系数拍脑袋。 ρ 太小解不可行，太大则在边界外形成陡墙、算法迈不进去。合格的做法是算一次“违约 1 单位折合多少目标值”，再定 ρ。本章 $\rho = 5$ 对应“1° 违约 ≈ IAE 差 0.01”的 500 倍，是有意配平的。自检办法：把 ρ 乘 2 与除 2 各跑一次，若最优解位置明显变动，说明还在敏感区，这个 ρ 不能用。

错误七：把“种群堆在边界”当成收敛。 截断式边界处理会让粒子被钉在界上，速度仍指向界外。诊断指标是边界个体占比，超过两成就要查边界是否设窄（§31.4.3）。另一个信号更直接：最优解连续三次迭代都压在同一侧的界上——那不是收敛，是“想出去但出不去”。

错误八：SA 只调初温，不调末温。 本章 $T_0 = 5$ 合适，但 4000 步后仍有 0.68，上坡接受率 36%，于是成功率只有 5%。正确的顺序是先用 (31-21) 反解 α，再定 T_0。反过来的错误同样常见：末温压到 10^{-3}，前 3000 步就冻结在某个坑里，等于花退火的钱跑了一次随机搜索。

错误九：复现时换了随机数接口或线程数。 `RandomState` 与 `default_rng` 的序列完全不同；并行归约顺序改变末位，可能让“是否达标”翻转。复现失败先查这两样，不要先怀疑算法（§31.4.4）。线程数这一条尤其隐蔽：同一台机器上把 `OPENBLAS_NUM_THREADS` 从 8 改成 1，矩阵乘法的归约顺序就变了，末位跟着变。

错误十：在不该用的地方用它。 在线闭环周期 10 ms，而 1200 次评价要 1.2 s；目标一次只要 1 μs，而 BO 每次迭代要算 3500 个候选点。**方法没有错，场景错了。** 判断标准始终是 $n_{\text{eval}} \times t_{\text{eval}}$ 与允许时间的关系。一行算式就能定论：把这两个量写在同一行，若前者大一个数量级，就不必再讨论算法优劣。

31.7.4 交付前的八项自检

写完一份启发性优化的报告，或者把一组参数交给现场之前，按下面八条过一遍。每一条都是可以当场核对的，不依赖主观判断。

1. **预算是否写明**，且含初值、含失败重试。
2. **种子是否列出**，至少 10 个，能被别人原样重跑。
3. **是否报了分布**（中位、IQR、最值），而不是单点。
4. **是否有基线**（随机搜索，最好再加一个梯度法）。
5. **“最优值”是否用无噪声真值复核过**（有噪声时必做，§31.6.5）。
6. **约束是否真的满足**：把最优解代回约束，看违约量，而不是看罚函数值。
7. **参数是否贴边界**：贴了就先查边界设定。
8. **单次求解时间是否算过**：$n_{\text{eval}} t_{\text{eval}}$ 与目标周期比一比。

八条里有任何一条答不上来，结论就不该写进交付文档。它们不涉及高深理论，只涉及“有没有把实验当实验做”。

这八条的顺序也有讲究。前四条是比较的前提：预算、种子、分布、基线缺一条，“更好/更差”就无从谈起。第五、六条是结论的前提：没有真值复核，报出的最优值可能只是噪声的一次有利实现；没有约束回代，报出的“可行解”可能只是罚函数值小。第七、八条是交付的前提：贴边的参数在仿真里能用、在现场会超差；没算过 $n_{\text{eval}} t_{\text{eval}}$ 的方案，可能第一次上线就把周期撑爆。

按这个顺序查还有个好处：**前四条不通过时，后四条查了也白查。** 一组没有基线的数字，无论怎么复核都不能用；反过来，前四条都过了，后四条通常十几分钟就能补齐。时间紧的时候，至少把前四条做掉。

两条最容易被跳过的实操细节。**第六条的回代要打印具体的违约量**，而不是打印“满足约束：是”——把 $\mathrm{PM} - 45$ 与 $M_s - 2$ 都算出来，正数就是违约；本章的 45.00 与 1.639 都是这么得到的。**第五条的复核要用“无噪声真值”**，也就是把找到的 $\boldsymbol{x}$ 代回不加噪声的目标；用平均值代回是错的，因为平均值里仍含噪声残差。

31.7.5 如果重做一遍：四处取舍

本章的实验设计有四处是被有意定下来的，不是随手取的。写出来，是因为它们的理由比结果更有迁移价值。

第一，为什么用二维 Rastrigin，而不用十维。 二维的好处是能画（图 31-1），读者可以自己验证“坑底差 1、翻脊要爬 20”这句话，也能在图 31-2 上肉眼看出“台阶型”与“直线型”的区别。代价是结论不能外推到高维：Rastrigin 是可分的，DE 的逐分量交叉天然占便宜；到

了十维，坑的数量按 11^{10} 量级增长，预算得涨到 10^5 以上，四种算法的排序也可能变。**用低维例子是为了可解释，不是为了代表所有维度；报告里必须把这条限制写明。**

第二，为什么预算定 4000。 这个数不是整，而是刚好跨过 E3 的门槛（1000→2000→4000）：再低，只能看到台阶下面那两级“加了预算几乎没用”；再高，DE 已经到 0，四种算法全是 0，反而看不出差别。**预算要能覆盖待观察现象的转折点，这是选预算的唯一标准**——先做一次小规模扫描，再定正式实验的预算，比按“跑得起多久”来定合理得多。

第三，为什么用 20 个种子。 按 §31.5.7 的式子，20 个种子在对数尺度上能分辨约 2 倍的差异。本章要下的结论大多是“差几个数量级”，5 个其实就够；但 §31.6.2 要报四分位区间，每个分位至少要 5 个点，所以 20 是“能给出 IQR 的最小整数倍”。用 10 个种子做预算扫描（E3）是因为档位多、要控制总运行时间，那时也只报数量级。

第四，有意保留了一组没调优的参数。 SA 的降温系数、BO 的核宽度、GA 的变异率都还可以调得更好：按 (31-21) 把 α 改成 0.99827，SA 的成功率不会只有 5%；用边际似然优化 ℓ，BO 的中位还能再降一个数量级；把 p_m 改成自适应，GA 也许能突破 10^{-4}。本章没有这么做，原因是要保留“参数没调好会怎样”这个教学点——**§31.3 的每条推导都给出了“应该取多少”，§31.6 的实验则展示了“没按推导取”的后果，两者对照才有意义。** 真正交付时当然要调；但调之前，得先知道坏成什么样。

本章至此结束。它讲的不是“哪种启发式算法最好”，而是**在没有梯度的情况下，如何诚实地做优化、诚实地报告结果**。这两件事在工程上往往比算法本身更重要：一个用错方法但报告规范的结果，别人能发现并纠正；一个用对方法但报告含糊的结果，只会把错误带到现场。

第32章 强化学习基础

32.1 动机与要解决的问题

32.1.1 模型没了，最优控制还成立吗

第 30 章用 LQR 与 MPC 求最优控制，前提是 (A, B) 已知；第 31 章在“目标算得出、梯度算不出”时改用无梯度法，前提是“给定一组参数，能离线拿回一个数”。两章共享一个隐含假设：**对象可以被反复调用，且调用很便宜。**

工程上还有第三种情形：模型不知道，离线评价也做不了，但你可以在线“试”——把动作加到对象上，观察下一时刻的状态和一个标量奖励。加热炉的升温曲线、注塑机的保压切换、无人机的姿态修正、路口的配时调整，都属于这一类。你拿不到 A、B，也写不出目标函数的解析式，手头只有一条轨迹

$$s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T \tag{32-1}$$

强化学习处理的正是这种“只有轨迹、没有模型”的问题。它和前面两章的关系可以这样定位：第 30 章是“模型已知，求一个反馈律”；第 31 章是“模型未知但可离线调用，求一组参数”；本章是“模型未知且只能在线交互，求一个策略 $\pi(a \mid s)$”。

所以 RL 不是“更高级的优化”。它改变的是信息结构：优化器看到的是“参数→目标值”的映射，RL 看到的是“状态→动作→下一状态+奖励”的样本流。这个区别解释了后面所有的工程差异——为什么样本效率是核心指标、为什么探索是一个独立问题、为什么“训练得好”不等于“部署得稳”。

顺带划清一条界线：RL 与自适应控制（MRAC、STR）都在线学习，但学的东西不同。自适应控制学的是模型参数，结构固定、目标明确（跟踪误差）；RL 学的是策略或值函数，允许非线性、离散动作、非二次代价，代价是样本需求高两三个数量级。**有可靠的结构化模型时，自适应控制是更省的选择；结构未知、目标只能靠回报表达时，才轮到 RL。**

32.1.2 评价函数回答“这一步好不好”，回报回答“这一路好不好”

单步奖励是近视的。以加热炉为例：把设定温度往上推，每一步的瞬时产出都更高；但温度越高，失控风险越大，一旦进入不可逆的副反应，后面所有步都在亏损。**奖励只描述“当下”，回报描述“从现在到未来”。**

从时刻 t 起的折扣回报定义为

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad \gamma \in [0, 1) \tag{32-2}$$

γ 不是数学装饰，它是“愿意往后看多远”的旋钮。几何级数给出有效视野

$$H_{\text{eff}} = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma} \tag{32-3}$$

$\gamma = 0.9$ 时约 10 步，$\gamma = 0.99$ 时约 100 步。把它换算成物理时间要乘控制周期：$T_s = 1$ s、$\gamma = 0.99$ 意味着策略关心未来 100 s；$T_s = 0.01$ s 时同样的 γ 只关心 1 s。**所以 γ 不能脱离采样周期单独讨论**，这是控制背景的读者最容易忽略的一点。若对象的慢模态时间常数为 τ，一个可用的起点是 $\gamma \approx e^{-T_s/\tau}$，它让“折扣”与“物理衰减”同尺度。

第二个难点是信用分配（credit assignment）：一条 200 步的轨迹最后只拿到一个奖励，这个奖励该记在哪一步？控制工程里也有同样的问题——稳态误差是几十个周期累积出来的，很难说清是哪个周期的哪个动作造成的。后面的蒙特卡洛、时序差分、Q-learning 三种方法，本质上是三种不同的“记账方式”，差别只在“用多长的实际数据去替换估计”。

32.1.3 一个两状态的反例：$+2$ 的奖励，-36.8 的回报

这个例子小到可以手算，却足以说明本章的全部困难。

设对象有两个状态：正常 S 与超温 F；两个动作：继续 p 与冷却 c。转移与奖励如下：

- 在 S 执行 p：立即奖励 $+2$，下一步以 0.9 留在 S、以 0.1 进入 F；
- 在 S 执行 c：奖励 -1，确定留在 S；
- 在 F 执行 c：奖励 -3，确定回到 S；
- 在 F 执行 p：奖励 -10，确定留在 F（故障自锁）。

取 $\gamma = 0.9$，先算“永远继续”这个策略。在 F 上有 $V(F) = -10 + 0.9V(F)$，故 $V(F) = -100$；代回 S：

$$V(S) = 2 + 0.9\big(0.9V(S) + 0.1V(F)\big) = 2 + 0.81V(S) - 9 \tag{32-4}$$

解得 $V(S) = -36.84$。**每一步都拿到 $+2$ 的策略，长期回报是 -36.8。** 近视指标在这里给出了与长期利益完全相反的结论。

再算“超温就冷却”：$V(F) = -3 + 0.9V(S)$，与 (32-4) 联立得 $V(S) = 15.87$、$V(F) = 11.28$。同一组奖励，只改了故障后的动作，回报从 -36.84 变成 $+15.87$，差 52.7。**这个差不是调参调出来的，是“往后看”看出来的。**

这个例子还能看清策略迭代的走法。从“永远继续”出发：第一轮评估得 $(-36.84, -100)$，改进时两个状态都换成冷却，因为 $Q(c) = -34.16 > -36.84$、$Q(c) = -36.16 > -100$；评估新策略得 $(-10, -12)$。第二轮在 S 上发现继续更好——$Q(p) = 2 + 0.9(0.9 \times (-10) + 0.1 \times (-12)) = -7.18$，高于 $Q(c) = -10$——于是改进为 (p, c)，评估得 $(15.87, 11.28)$。第三轮两个状态都没有动作可换，收敛。**三轮评估、两轮实际改动。**

注意一个细节：第一轮把 S 也换成冷却是“错”的方向，但策略迭代照样收敛。原因是策略改进定理只要求每一步单调不减，不要求每一步都朝最优走。**这条性质是表格型方法可靠工作的根，也是它对初值不敏感的原因。** §32.3.1 给出证明，§32.6.1 用代码复现这三个数。

32.1.4 与控制理论的接口

表 32-1 把三种“求策略”的方式放在一起。

表 32-1　三种求策略的方式

项	LQR（第 30 章）	MPC（第 58/80 章）	本章 RL
需要模型	需要 (A, B)	需要预测模型	不需要显式模型
需要的数据	无	当前状态	状态-动作-奖励轨迹
解的形式	线性反馈 $u = -Kx$	滚动优化	策略 $\pi(a \mid s)$ 或 $Q(s, a)$
最优性	全局（线性二次）	有限时域局部	渐近，依赖探索
约束	硬约束需扩展	天然支持	需动作屏蔽或罚项
典型失败	模型失配	模型失配、算力不足	分布偏移、奖励误设

表的最后一行是重点。前三列的失败都可以归因到“模型不准”，RL 的失败却常常归因到“数据不对”：策略只见过它自己走过的状态，一旦部署时遇到没见过的状态，Q 值就是外推，可能错得离谱。第 33 章的离线 RL 与第 36 章的安全约束，都是围绕这一点展开的。

还有两个常被混淆的点。**其一，RL 不要求“无模型”，它只要求“不用模型求导”。** 仿真器很准时，完全可以用它生成数据来训练策略，只是不要拿它的梯度去做梯度下降。**其二，模型已知时用 RL 是绕远路。** 线性二次问题有 Riccati 方程一步到位，用 RL 去逼近它，等于用几万次采样换一个本来可以闭式写出的 K。

32.1.5 五个必须回答的定量问题

问题一：状态与动作怎么离散？ 表格型方法的规模是 $|S| \times |A|$，连续系统必须先量化。量化太粗，最优策略被抹平；太细，每个格子分不到样本。判据与做法在 §32.4.1。

问题二：γ 取多少？ 它等于有效视野，必须与物理时间常数对齐，换算关系是 $\gamma \approx e^{-T_s/\tau}$。取小了策略短视，取大了方差爆炸。§32.2.2 与 §32.6.2 分别给定义与实测。

问题三：探索留多少？ ε-greedy 的 ε 决定“花多少样本买信息”。固定 ε 的 Q-learning 收敛到的是“带探索的策略”，不是最优策略。§32.3.5 说明为什么，§32.4.3 给衰减方案。

问题四：on-policy 还是 off-policy？ SARSA 学“我正在执行的政策”，Q-learning 学“最优政策”。安全关键的场合前者更可取，因为它把探索带来的风险也算进了值里。§32.6.4 的悬崖实验给出两条路径的差别。

问题五：样本从哪来、值多少钱？ 真实设备上一次试验可能几小时且有损耗，仿真里一次几毫秒。样本成本决定了该用 TD 还是 MC、该不该上函数逼近。这笔账在 §32.5.5。

这五个问题答不上来，调任何超参数都是碰运气。

32.1.6 本章的走法

§32.2 建立 MDP 与 Bellman 方程，把“回报”写成一个可解的不动点，并交代三个建模选择（状态、动作、奖励）与折扣因子的三重身份；§32.3 从这个不动点出发，推出策略迭代、值迭代、蒙特卡洛、时序差分、Q-learning、SARSA、策略梯度与 Actor-Critic，比较三条求解路线的代价，并标出它们与自适应控制的亲缘关系；§32.4 讲表格实现里的量化、更新顺序、探索、覆盖率与量纲一致的超参数配置；§32.5 讲奖励设计、训练环境构建、安全约束、多目标的优先级结构、接进 DCS 时的接口一致性，以及“什么时候不该用”；§32.6 用七组实验把前面的每个断言变成一个可复现的数，并说明同一个策略为何能报出差出四倍的数字；§32.7 收成一张决策表与十条错误。

如果只有半小时，读 §32.1.3 的反例、§32.3.4 的 TD 误差分解、§32.7.2 的决策表。

一句话核心：**强化学习把“求最优控制”换成了“在样本流上估计值函数”，代价是样本，收益是不需要模型。**

32.2 定义与数学基础

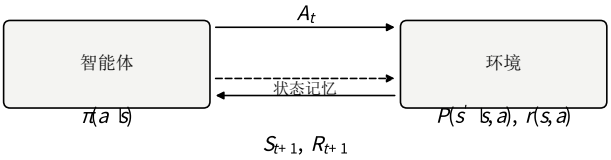
32.2.1 五元组与 Markov 性

强化学习把“在线试错”抽象成一个五元组 $\mathcal{M}=(\mathcal{S},\mathcal{A},P,r,\gamma)$：状态集 $\mathcal{S}$、动作集 $\mathcal{A}$、转移核 P、奖励函数 r、折扣因子 γ。前四个描述对象，最后一个描述目标。

转移与奖励的精确定义是

$$P(s'\mid s,a)=\Pr\{S_{t+1}=s'\mid S_t=s,A_t=a\},\qquad r(s,a)=\mathrm{E}[R_{t+1}\mid S_t=s,A_t=a] \tag{32-5}$$

图 32-1 画出了交互回路：智能体在 S_t 上选 A_t，环境返回 R_{t+1} 与 S_{t+1}。这个回路与第 44 章的数字控制回路形状相同，区别只在“控制器”换成了“策略”，“误差”换成了“奖励”。



Markov：下一步只依赖当前状态与动作

图 32-1　MDP 的交互回路

图里从环境指出的 S_t 有两条去向：一条进入策略，另一条与 A_t 一起决定下一拍的状态与奖励。后一条就是 Markov 性——**给定 S_t 与 A_t，S_{t+1} 与更早的历史条件独立。** 整章的所有算法都建立在这条性质上；它不成立时，Bellman 方程就不成立，后面的推导全部作废。

控制工程里，Markov 性对应“状态完备性”。一个常见反例：对象是一阶惯性加两拍纯延迟，y_{t+1} 依赖 u_{t-1}，若状态只取当前测量 y_t，则同一个 s_t 下执行同一个 a_t，下一步可能落在两个不同的值上——P 不是函数，而是“关系”。修法是把延迟拍数补进状态：

$$S_t=(y_t,\ y_{t-1},\ y_{t-2}) \tag{32-6}$$

补几拍取决于延迟与采样周期的比值。**这一步没做对，后面无论用多好的算法，学到的都是一个“看起来收敛、实际不可用”的策略。** §32.4.1 会把这条判据变成可执行的检查。

32.2.2 策略与两条值函数

策略是“在状态上选动作”的规则。确定策略写成 $a=\pi(s)$，随机策略写成 $\pi(a\mid s)$。**表格型方法两者都能用，但探索与策略梯度都需要随机策略**：确定策略的 ε-greedy 只是把随机性外包给了“探索机制”，而策略梯度要对 π 求导，必须有一个连续可调的概率分布。

给定策略 π，两条值函数分别回答两个问题。状态值函数回答“站在这里、按 π 走下去，预期能拿多少”：

$$V^{\pi}(s)=\mathrm{E}_{\pi}\Big[\sum_{k=0}^{\infty}\gamma^{k}R_{t+k+1}\ \Big|\ S_t=s\Big] \tag{32-7}$$

动作值函数回答“站在这里、先做这个动作、之后按 π 走，预期能拿多少”：

$$Q^{\pi}(s,a)=\mathrm{E}_{\pi}\Big[\sum_{k=0}^{\infty}\gamma^{k}R_{t+k+1}\;\Big|\;S_t=s,A_t=a\Big] \tag{32-8}$$

两者由一步全概率公式相连：

$$V^{\pi}(s)=\sum_{a\in\mathcal{A}}\pi(a\mid s)\,Q^{\pi}(s,a) \tag{32-9}$$

(32-9) 是后面一切“改进”的支点：**只要对每个 s 能算出 $Q^{\pi}(s,\cdot)$，就能在不改变环境的前提下换一个更好的动作。** 这也是“策略评估”与“策略改进”能拆成两步的原因。

两条值函数之差叫优势函数 $A^{\pi}(s,a)=Q^{\pi}(s,a)-V^{\pi}(s)$。它衡量“这个动作比平均水平好多少”，$A$ 的符号直接给出改进方向。§32.3.6 会看到，策略梯度的方差主要来自 Q 的绝对值，减去 V 之后方差大幅下降——**这不是技巧，是“只关心相对好坏”的必然结果。**

32.2.3 Bellman 期望方程：值函数满足一个线性方程

把 (32-8) 的期望按“第一步”展开：第一步拿 $r(s,a)$，之后落到 s'，从那里起的期望恰好是 $V^{\pi}(s')$。于是

$$Q^{\pi}(s,a)=r(s,a)+\gamma\sum_{s'\in\mathcal{S}}P(s'\mid s,a)\,V^{\pi}(s') \tag{32-10}$$

把 (32-10) 代回 (32-9)，得到只含 V^{π} 的方程：

$$V^{\pi}(s)=\sum_{a}\pi(a\mid s)\Big[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V^{\pi}(s')\Big] \tag{32-11}$$

这就是 Bellman 期望方程。它的形式值得盯住看：**未知量 V^{π} 在等号两边都出现，且只以一次幂出现。** 所以它是 $|\mathcal{S}|$ 个线性方程，写成矩阵形式是

$$\mathbf{v}_{\pi}=\mathbf{r}_{\pi}+\gamma P_{\pi}\mathbf{v}_{\pi}\;\Longrightarrow\;\mathbf{v}_{\pi}=(I-\gamma P_{\pi})^{-1}\mathbf{r}_{\pi} \tag{32-12}$$

其中 P_{π} 是按 π 加权后的转移矩阵，$\mathbf{r}_{\pi}$ 是期望即时奖励。$\gamma<1$ 保证了 $I-\gamma P_{\pi}$ 可逆：P_{π} 的谱半径不超过 1，所以 γP_{π} 的谱半径严格小于 1，Neumann 级数收敛。

这个逆矩阵有明确的物理意义：$(I-\gamma P_{\pi})^{-1}=\sum_{k=0}^{\infty}\gamma^{k}P_{\pi}^{k}$，它的第 (s,s') 元就是“从 s 出发、按 π 走，折扣后平均访问 s' 多少次”。**它把“值函数”翻译成了“访问频次”，后面估计样本量、判断探索是否充分，用的都是这个量。**

直接求解 (32-12) 的代价是 $O(|\mathcal{S}|^{3})$。状态数上万时这个代价不可接受，于是有了迭代法（§32.3.1）与采样法（§32.3.3）。**这正是“精确解”与“可扩展解”的分界：小问题上解线性方程最省事，大问题上只能迭代。**

32.2.4 Bellman 最优方程：把 max 放进期望

值函数回答“这个策略多好”，最优值函数回答“最好能多好”：

$$V^{*}(s)=\max_{\pi}V^{\pi}(s) \tag{32-13}$$

对有限 MDP，这个最大值可以由一个确定策略达到，且 V^{*} 满足

$$V^{*}(s)=\max_{a\in\mathcal{A}}\Big[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V^{*}(s')\Big] \tag{32-14}$$

对应的动作值函数是

$$Q^{*}(s,a)=r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)\max_{a'}Q^{*}(s',a') \tag{32-15}$$

(32-14) 与 (32-11) 只差一处：求和换成了取最大。**这一处差别决定了两个方程的性质完全不同**——(32-11) 是线性的，(32-14) 不是。$\max$ 是一族线性函数的上包络，所以 V^{*} 是分段线性的凸函数（在 $|\mathcal{S}|$ 维空间里），不能用解线性方程的办法求。

但 (32-14) 有一个更强的性质：它定义了一个压缩映射。记 Bellman 最优算子为 $\mathcal{T}$，则对任意两个值函数

$$\|\mathcal{T}V-\mathcal{T}U\|_{\infty}\le\gamma\|V-U\|_{\infty} \tag{32-16}$$

因为 $\max$ 是 1-Lipschitz 的。$\gamma<1$ 时 $\mathcal{T}$ 是严格压缩的，由 Banach 不动点定理，V_* 存在、唯一，且从任意初值迭代都收敛。**这条不等式是整章唯一一个“无条件成立”的保证**，§32.3.2 会把它变成收敛速度与误差界。

一旦有了 Q_*，最优策略就是逐点取最大：

$$\pi_*(s)=\arg\max_a Q_*(s,a) \tag{32-17}$$

(32-17) 只做比较、不做运算，这是 RL 在“目标不可微”时仍然能工作的根本原因，也是它与第 31 章“只比较函数值”的启发式方法的共同点。区别在于：第 31 章比较的是“同一组参数下的目标值”，这里比较的是“同一个状态下不同动作的未来回报”。

32.2.5 与控制理论的对应：同一个 Bellman，不同的表示

把 (32-14) 的特例写出来，就能看到它与第 30 章的关系。取 $\gamma=1$、二次代价 $r=-x^\top Qx-u^\top Ru$、线性动态 $x^+=Ax+Bu$，并假设 $V_*(x)=x^\top Px$。代入 (32-14)：

$$x^\top Px=\max_u\Big[-x^\top Qx-u^\top Ru+(Ax+Bu)^\top P(Ax+Bu)\Big] \tag{32-18}$$

对 u 求导得 $u_*=-(R+B^\top PB)^{-1}B^\top PA\,x$，代回即得离散 Riccati 方程

$$P=Q+A^\top PA-A^\top PB\,(R+B^\top PB)^{-1}B^\top PA \tag{32-19}$$

这就是 LQR。 它的“值函数”是二次型，“策略”是线性反馈，两者都能闭式写出。Bellman 方程在连续时间下的对应物是 HJB 方程

$$-\frac{\partial V}{\partial t}=\min_u\Big\{L(x,u)+\nabla V^\top f(x,u)\Big\} \tag{32-20}$$

LQR 之所以能解，是因为 (32-20) 的二次型拟设把偏微分方程化成了代数方程。**一旦代价不是二次、动态不是线性，这个拟设失效，HJB 就只能数值求解——RL 是它的采样版本。**

三个概念的对应关系可以一句话说清：**值迭代是无限时域的动态规划，MPC 是有限时域加滚动，RL 是在不知道 P 的情况下估计同一个 Bellman 算子。** 所以看到“RL 能解 MPC 解不了的问题”这种说法时要警惕：MPC 的困难是算力与模型，RL 的困难是样本与分布；它们不是同一把尺子上的长短。

最后提醒一个符号约定。控制里习惯“最小化代价”，RL 里习惯“最大化奖励”，两者只差一个负号，但**折扣的处理不能只靠负号**：$\gamma<1$ 的 RL 目标等价于给代价加了一个随时间的衰减权重，它会让策略偏向“近期收益”，与“无限时域平均代价”不是同一个问题。做控制整定时若把 γ 设成 0.9 而采样周期是 1 s，等于只优化前 10 s，稳态性能自然被忽略。

32.2.6 表格的规模：先把账算清

表格型方法为每个 (s,a) 存一个数。账很容易算，也很容易算崩。

表 32-2　几个典型问题的表格规模

系统	状态维数	每维档数	状态数	动作数	Q 表规模
两状态加热炉（§32.1.3）	1	2	2	2	4
温度调节（§32.6.5）	1	21	21	3	63
倒立摆（四维量化）	4	20	1.6×10^{5}	2	3.2×10^{5}
三关节机械臂	6	11	1.77×10^{6}	5	8.9×10^{6}
十维线性系统	10	10	10^{10}	5	5×10^{10}

按双精度 8 字节计：倒立摆的 Q 表占 2.6 MB，三关节机械臂占 71 MB，十维系统要 400 GB。**每加一维，规模乘以档数——这就是维度诅咒的算术形式。**

比内存更紧的是样本。要让每个 (s,a) 的估计误差降到可用水平，通常需要被访问几十次；十维系统于是需要 10^{12} 量级的样本，按每秒 1000 次交互算要三十多年。**所以表格型方法的适用边界不是“状态多不多”，而是“$|S|\times|A|$ 乘上每格所需样本数，能不能在可接受的时间内采到”。**

这条边界正是第 33 章的起点：用函数逼近把“每格一个数”换成“一个参数化函数”，用泛化换样本。代价是收敛保证变弱——表格型的收敛定理到函数逼近那里大多失效。**先看清表格能走多远，才知道为什么要换成网络。**

32.2.7 折扣因子：一个数，三重身份

γ 在公式里只出现一次，但在工程上它同时扮演三个角色。分不清这三个角色，是“调参调不动”的常见原因。

身份一：压缩因子。 式 (32-16) 说 Bellman 算子每作用一次，误差至少缩 γ 倍。误差放大因子 $\gamma/(1-\gamma)$ 因此直接决定两件事：值迭代要多少轮才能把误差压到 ε 以下（式 32-25 给出 $k \geq \ln(\|V_0 - V_*\|_\infty/\varepsilon)/\ln(1/\gamma)$），以及估计误差会被放大多少倍。$\gamma = 0.9$ 时放大 9 倍，$\gamma = 0.99$ 时放大 99 倍。**后者意味着一个 1% 的奖励建模误差，会在值函数里变成接近 100% 的偏差。**

身份二：有效时域。 式 (32-3) 的 $H_{\text{eff}} = 1/(1-\gamma)$ 是“回报里一共算了多少步”的一阶答案。更实用的口径是 95% 权重落在哪里：

$$k_{95} = \frac{\ln 0.05}{\ln \gamma}$$

$\gamma = 0.9$ 时 $k_{95} = 28.4$，$\gamma = 0.99$ 时 $k_{95} = 298.1$。采样周期 $T_s = 1$ s 时，这两个数分别读作“只看未来 28 秒”和“只看未来 5 分钟”。**这就是 γ 的物理含义——它不是折扣，它是视野。**

身份三：代价的指数权重。 把 γ^k 写成 $e^{-\lambda k T_s}$，得

$$\lambda = -\frac{\ln \gamma}{T_s}$$

$\gamma = 0.9$、$T_s = 1$ s 对应 $\lambda = 0.105\ \text{s}^{-1}$，时间常数 9.5 s。也就是说，折扣回报等价于给每一步代价乘了一个时间常数为 9.5 s 的一阶低通权重。这个等价很有用：控制工程师对“时间常数”有直觉，对“折扣”没有。但它也提醒一件事——**折扣不是平均代价**。无限时域平均代价准则对所有时刻一视同仁，折扣准则偏向近期；两者的最优策略在有的问题上不同，换准则必须重算，不能靠“γ 取接近 1”糊过去。

两个陷阱要单独说。

陷阱一：$\gamma \to 1$ 的数值病态。 式 (32-12) 要解 $(I - \gamma P_\pi)\mathbf{v}_\pi = \mathbf{r}_\pi$。当 P_π 有特征值 1（比如存在吸收态或周期结构）时，该矩阵的最小特征值约为 $1-\gamma$，条件数约为 $1/(1-\gamma)$。$\gamma = 0.999$ 时条件数是 1000，双精度下解方程要丢掉约 3 位有效数字；$\gamma = 0.99999$ 时条件数 10^5，结果基本不可信。**“把 γ 调到 0.999 更接近最优控制”这句话在数学上是对的，在数值上是错的**——除非改用专门的迭代解法并接受更慢的收敛。

陷阱二：γ 与回合长度必须自洽。 若回合在 T 步被截断，而 $k_{95} > T$，那么被折扣加权的那部分回报根本没被采到，值函数学的是“被截断的目标”，偏差随 k_{95}/T 增大。反过来，$T \gg k_{95}$ 时后半段轨迹对目标几乎没有贡献，等于白采。**经验配比是 $T \approx 3/(1-\gamma)$**：$\gamma = 0.9$ 配 30 步左右的回合，$\gamma = 0.99$ 配 300 步。E2 的实验里值迭代 34 轮把误差压到 8.65×10^{-3}，而把 γ 调高后同样精度要 340 轮、且残差停在 9.88×10^{-2} 下不去——轮数按 $1/\ln(1/\gamma)$ 增长是可以预期的，残差压不下去则是数值精度先到极限了。

工程上定 γ 的顺序应该反过来：先写下“你希望控制器看多远”（T_{95}，由对象的主导时间常数定），再由 $\gamma = 0.05^{T_s/T_{95}}$ 反算；然后检查两件事——回合一幕的长度够不够 T_{95}，以及 $\gamma/(1-\gamma)$ 放大后的误差你能否接受。**三步做完再写代码，比在 0.9 和 0.99 之间试一个星期有效。**

32.2.8 三个建模选择：状态、动作、奖励

RL 没有“模型”，它的模型就是这三个选择。选错了，算法再好也只是更快地学到一个错的策略。

状态：要满足 Markov 性，而不是“把能测的都放进去”。 Markov 性要求“给定当前状态，未来与历史无关”。三种常见破坏方式：其一，漏掉关键变量（比如只测炉温不测加热功率，同样的温度下升温趋势相反）；其二，漏掉延迟（执行器有 3 步纯延迟时，当前动作的效果要 3 步后才出现，状态里必须显式带上最近 3 步的动作）；其三，把噪声当状态（把未滤波的瞬时值放进状态，等于让策略对噪声做反应）。第一种靠机理分析补，第二种靠式 (32-6) 的堆叠补，第三种靠滤波补。**检验方法只有一个：把同一状态的两批样本拿出来，看 TD 误差的平均方向是否一致；方向相反，说明状态里缺东西。**

动作：绝对量还是增量，选错了会带来两个麻烦。 绝对动作（直接给阀门开度）语义清楚，但策略抖动会直接传到执行器；增量动作（给“开大/关小”）天然平滑，且便于处理积分饱和，代价是动作的效果依赖当前工作点，Q 函数在动作维度上的形状更复杂。控制里还有第三种写法：**让 RL 输出设定值、让底层 PID 输出阀门开度**——这样动作的物理意义是“我希望温度到多少”，量纲与安全边界都好定义，也是 §32.5.4 分层结构的由来。

奖励：一步延迟与量纲是两个独立的坑。 量纲在 §32.4.9 讲，这里只说延迟。若奖励在动作之后 k 步才到达（比如实验室的离线化验），那么 (s_t, a_t, r_t, s_{t+1}) 这条样本里的 r_t 其实属于 a_{t-k}。直接喂给算法，等于给动作打了错的标签，学出来的策略会系统性地偏移。**处理方式有两种：把奖励按时间戳对齐（需要记录动作序号），或者把“等待期”建进状态（把最近 k 步的动作放进状态）。前者需要数据管道支持，后者需要状态维数增加 k 倍。**

三个选择是耦合的，这是最容易忽略的一点。举个具体的例子：状态量化太粗（每维只有 5 档），两个不同工况被归到同一格；此时动作 $+1$ 与 0 的效果被平均掉，奖励差只有噪声量级；算法看到的是“动作无所谓”，于是学出一个常量策略。**表面上是“算法没学会”，根子**

上是状态分辨率不够，把动作的效果抹平了。反过来，状态太细而样本不够，每个格子只被访问两三次，Q 的估计全是噪声，$\arg\max$ 选的是噪声的最大值（第 31 章 §31.6.5 的乐观偏差）。**所以“状态分几档”这件事，必须和样本预算一起定（§32.4.1），不能分开拍板。**

32.3 推导与性质

32.3.1 策略迭代：评估一步，改进一步

策略迭代把“求最优”拆成两个交替的小问题：**评估**当前策略到底值多少，**改进**它在每个状态上换一个更好的动作。

评估就是解 (32-11)。状态多时不解方程，改用迭代：从任意 $\mathbf{v}_0$ 出发，反复把 Bellman 期望算子作用上去，

$$V_{k+1}(s)=\sum_{a}\pi(a\mid s)\Big[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V_k(s')\Big] \tag{32-21}$$

因为算子也是 γ-压缩的，误差按 γ^k 衰减：$\gamma=0.9$ 时每轮缩 10%，要缩到 10^{-3} 需 66 轮。

改进则用 (32-9) 算出的 Q^{π} 逐点取最大：

$$\pi'(s)=\arg\max_{a}Q^{\pi}(s,a) \tag{32-22}$$

定理（策略改进）：若 π' 由 (32-22) 得到，则 $V^{\pi'}(s)\ge V^{\pi}(s)$ 对所有 s 成立，且只要有一处严格，就至少有一处严格。

证明只用三步。第一，由定义 $Q^{\pi}(s,\pi'(s))\ge Q^{\pi}(s,\pi(s))=V^{\pi}(s)$。第二，把 Q 展开成“一步奖励 + 下一状态的值”，并对下一状态再用一次同样的不等式，得到

$$V^{\pi}(s)\le \mathrm{E}\big[R_{t+1}+\gamma V^{\pi}(S_{t+1})\big]\le \mathrm{E}\big[R_{t+1}+\gamma Q^{\pi}(S_{t+1},\pi'(S_{t+1}))\big]\le\cdots \tag{32-23}$$

第三，把这条链无限展开，右边正是“从 s 出发、先按 π' 走一步、之后按 π 走”的回报，再对后续每一步重复同样的替换，极限就是 $V^{\pi'}(s)$。$\square$

这条证明解释了两件事。**其一，改进不要求“一步到位”**——只要每个状态的新动作不比旧的差，整体就不比原来差，所以 §32.1.3 第一轮把 S 换成冷却是允许的，第二轮还能换回来。**其二，有限 MDP 下策略只有有限个**，严格改进不可能无限进行，算法必然在有限步停在 π_*。这是表格型方法少有的“有限时间终止”保证。

32.3.2 值迭代：跳过策略，直接迭代值函数

策略迭代每轮都要把当前策略评估到收敛，代价大。值迭代把两步合成一步，直接对最优算子做不动点迭代：

$$V_{k+1}(s)=\max_{a}\Big[r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)V_k(s')\Big] \tag{32-24}$$

由 (32-16) 立刻得到误差界

$$\|V_k-V_*\|_\infty\le\gamma^{k}\|V_0-V_*\|_\infty \tag{32-25}$$

收敛速度只由 γ 决定，与状态数、动作数无关。这是值迭代最漂亮的地方，也是最容易被误读的地方：$\gamma\to1$ 时 γ^k 衰减得极慢，收敛需要的轮数按 $1/(1-\gamma)$ 发散。以初值误差 1 个单位计，$\gamma=0.9$、要求误差 10^{-3} 需 $k\ge\ln(10^{-3})/\ln 0.9\approx 66$ 轮；$\gamma=0.99$ 时同样精度要 688 轮，约是十倍。**折扣因子在算法层面上的代价，是收敛轮数按 $1/(1-\gamma)$ 增长。**

实际停机时拿不到 V_*，只能用相邻两轮的差做判据：$\|V_{k+1}-V_k\|_\infty<\varepsilon$。这时真实误差有个放大因子，因为

$$\|V_k-V_*\|_\infty\le\frac{\gamma}{1-\gamma}\|V_{k+1}-V_k\|_\infty \tag{32-26}$$

$\gamma=0.9$ 时放大 9 倍。**用“相邻差小于 10^{-3}”停机，真实误差可能到 9×10^{-3}**，这是实现里最常见的低估。

还有一层：值迭代的产物是值函数，最终要转成策略。若 $\|V_k-V_*\|_\infty\le\varepsilon$，则由 V_k 贪心得到的 π_k 满足

$$\|V^{\pi_k}-V_*\|_\infty\le\frac{2\gamma\varepsilon}{1-\gamma} \tag{32-27}$$

$\gamma=0.9$ 时放大 18 倍。想把策略的性能差距控制在 0.01 以内，值函数误差必须小于 5.6×10^{-4}，对应 72 轮。**“值函数收敛”与“策略收敛”不是一回事，前者要比后者严一个量级。**

32.3.3 蒙特卡洛：拿一整条轨迹去平均

前两节都要求知道 P。真实对象给不出 P，只能采样。蒙特卡洛（MC）的做法最直接：跑完一整条轨迹，用实际回报去平均，

$$V(S_t) \leftarrow V(S_t) + \alpha\big[G_t - V(S_t)\big] \tag{32-28}$$

$\alpha = 1/N(S_t)$ 时就是样本均值。它有两个好性质：**无偏**（$\mathrm{E}[G_t \mid S_t = s] = V^\pi(s)$），**不需要模型**。也有一个坏性质：**方差大**。

若每一步奖励独立、方差为 σ^2，则回报的方差是

$$\mathrm{Var}[G_t] = \sigma^2 \sum_{k=0}^{\infty} \gamma^{2k} = \frac{\sigma^2}{1-\gamma^2} \tag{32-29}$$

$\gamma = 0.9$ 时是 $5.26\sigma^2$，标准差是单步奖励的 2.29 倍。$\gamma = 0.99$ 时放大到 50 倍方差、7.1 倍标准差。**视野越长，MC 的嗓门越大。**要把它压到 1% 的精度，样本量按方差反比增长，$\gamma = 0.99$ 时比 $\gamma = 0.9$ 多要近十倍。

MC 还有一个实现细节：首次访问（first-visit）与每次访问（every-visit）。前者只在一条轨迹里第一次到达 s 时更新，后者每次到达都更新。两者都收敛到 V^π，但前者的样本被丢掉一部分，后者在小 α 下更稳。**表格实现通常用每次访问，因为它的有效样本数更多。**

32.3.4 时序差分：用“下一步的估计”替换“整条轨迹”

TD(0) 把 (32-28) 的 G_t 换成“一步奖励 + 下一状态的当前估计”：

$$V(S_t) \leftarrow V(S_t) + \alpha\big[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD 目标}} - V(S_t)\big] \tag{32-30}$$

方括号里的量叫 TD 误差 δ_t。这个替换带来两个后果，一好一坏。

好处是方差小。 TD 目标的方差约 $\sigma^2(1+\gamma^2) = 1.81\sigma^2$（$\gamma = 0.9$），而 MC 是 $5.26\sigma^2$——**方差降到约三分之一。**代价是引入了偏差：$V(S_{t+1})$ 本身是估计值，在收敛前不准。

偏差有多大？ 对固定的 V 取期望，

$$\mathrm{E}[\delta_t \mid S_t = s] = r(s,\pi(s)) + \gamma \sum_{s'} P(s' \mid s,\pi(s)) V(s') - V(s) = (\mathcal{T}_\pi V)(s) - V(s) \tag{32-31}$$

也就是说，**TD 误差的期望恰好是“Bellman 残差”。**这解释了 TD 为什么能工作：它是在用样本估计残差，残差为零就是不动点。也解释了它的偏差从哪来：只要 $V \neq V^\pi$，这个期望就不为零，更新方向就带着系统误差。

两者的差别可以概括成一句话：**MC 用“真实的未来”做目标，无偏但吵；TD 用“猜测的未来”做目标，安静但会被猜测带偏。**在表格、$\gamma < 1$、所有状态被无限次访问的条件下，两者都收敛到 V^π；但在有限样本下，TD 通常更快达到相同的均方误差，这就是它成为主流的原因。§32.6.3 会用同一批轨迹把两条 RMSE 曲线画出来。

32.3.5 Q-learning 与 SARSA：一个学最优，一个学自己

控制里真正想要的是 Q，因为它直接给出动作。Q-learning 的更新是

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1},a') - Q(S_t,A_t)\big] \tag{32-32}$$

SARSA 只改一处：把 $\max$ 换成“下一步实际执行的动作”，

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\big] \tag{32-33}$$

这一处差别决定了它们学的是两个不同的东西。**Q-learning 学 Q_*，与行为策略无关（off-policy）；SARSA 学“当前行为策略”的值（on-policy）。**若行为策略是 ε-greedy，SARSA 学的是“带 ε 概率乱走”的那个策略的值，Q-learning 仍然学最优策略。

安全性上的后果很直接。悬崖边上，Q-learning 会学到“贴着崖边走”的最优路径，因为它假设下一步总取最优动作；但它自己在探索时偶尔会掉下去。SARSA 把“偶尔乱走”算进了值里，于是学会绕远路。§32.6.4 的两条路径长度会给出这个差别的量级。

两者都有收敛定理，条件相同：**每个 (s,a) 被访问无限次，且步长满足 Robbins–Monro 条件**

$$\sum_{t} \alpha_t = \infty, \qquad \sum_{t} \alpha_t^2 < \infty \tag{32-34}$$

$\alpha_t = 1/t$ 满足，常数 α 不满足——**常数步长下算法跟踪的是一个“移动目标”，最终在最优值附近的一个邻域里游走，不会精确收敛。**这对工程反而是好事：对象时变时，常数步长能跟上漂移。

Q-learning 还有一个系统性偏差：**最大化偏差**。若两个动作的真实值都是 0，但估计各带标准差 σ 的噪声，则 $\mathrm{E}[\max(\hat{Q}_1, \hat{Q}_2)] \approx 0.564\sigma > 0$。噪声越大，高估越严重；高估又让算法更倾向于选“被高估”的动作，形成正反馈。修法是双估计（double Q-learning）：两套表交替更新，用一套选动作、另一套评估。**代价是样本效率减半，收益是去掉这个 0.56σ 量级的高估。**

32.3.6 策略梯度：直接对策略求导

前面所有方法都先学值、再由值导出策略。策略梯度反过来：把策略参数化成 $\pi_\theta(a \mid s)$，直接最大化期望回报

$$J(\theta) = \mathrm{E}_{\pi_\theta}\Big[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\Big] \tag{32-35}$$

难点是 J 对 θ 的依赖藏在“采样分布”里——改变 θ 会改变走过的状态，状态分布又影响回报。策略梯度定理绕开了这个难点：

$$\nabla_\theta J(\theta) = \mathrm{E}_{\pi_\theta}\Big[\nabla_\theta \ln \pi_\theta(A_t \mid S_t)\, Q^{\pi_\theta}(S_t, A_t)\Big] \tag{32-36}$$

它的直觉是“**加权的似然比**”：某个动作带来的回报高于平均，就提高它的概率；低于平均，就降低。$\nabla \ln \pi$ 是“提高这个动作概率的方向”，Q 是“这个动作值多少”，两者相乘就是更新方向。**这个式子不要求环境可微，只要求策略可微。**

REINFORCE 用一条轨迹的回报 G_t 代替 Q，得到无偏但高方差的估计。方差主要来自 Q 的绝对值：如果所有动作的 Q 都在 100 附近、彼此只差 1，那么梯度的“信号”是 1，“噪声”是 100。减去一个只依赖状态的基线 $b(s)$ 不改变期望，因为

$$\mathrm{E}_{a\sim\pi}\big[\nabla \ln \pi(a \mid s)\, b(s)\big] = b(s) \sum_a \nabla \pi(a \mid s) = b(s) \nabla 1 = 0 \tag{32-37}$$

取 $b(s) = V^\pi(s)$ 时，乘子变成优势 $A = Q - V$，信号与噪声的比例立刻改善。**这不是技巧，而是“只有相对好坏才携带梯度信息”的直接推论。**

32.3.7 Actor-Critic：一个出动作，一个做评估

REINFORCE 要等一条轨迹跑完才能更新，且用 G_t 估计 Q，方差仍大。Actor-Critic 把“策略”与“值函数”分成两个学习器：Actor 是 π_θ，Critic 是 V_w，后者提供低方差的 TD 误差作为优势的估计：

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t), \qquad \theta \leftarrow \theta + \alpha_\theta\, \delta_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t) \tag{32-38}$$

Critic 自己用 TD 学习：$w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(S_t)$。两个步长要满足 $\alpha_w \gg \alpha_\theta$（批评者快、演员慢），这是“双时间尺度”的典型结构：**Critic 必须先跟上，Actor 的更新方向才可信。**

代价是偏差回来了。δ_t 用 V_w 代替真值，V_w 不准时 Actor 就在朝错误方向走。这形成一个耦合：Critic 的误差会污染策略，策略变差又让 Critic 的数据分布变窄。**这就是 Actor-Critic 比“纯策略梯度”更难调的根本原因，也是第 33 章要引入目标网络、经验回放等技术的原因。**

32.3.8 这些保证在什么条件下失效

前面每个定理都带条件，工程上失效几乎都发生在条件被悄悄破坏时。

第一，探索条件。 “所有 (s,a) 无限次访问”在实践中等于“探索永不停止”。ε 固定时，Q-learning 的极限不是 Q_*，而是“ε-soft 策略”的值；ε 衰减太快，则会漏掉某些动作。§32.4.3 给一个可执行的衰减表。

第二，Markov 性。 状态不完备时 Bellman 方程不成立，算法可能收敛到一个“历史相关”的伪不动点。表现是：同一个状态、同一个动作，Q 的更新方向来回摇摆，方差降不下去。**先查状态定义，再查算法。**

第三，函数逼近。 一旦用参数化函数代替表格，“off-policy + 自举 + 函数逼近”三者同时出现就可能发散（Baird 反例）。表格型的收敛定理在这里全部失效，这是第 33 章的核心议题。

第四，非平稳。 对象参数漂移时，P 随时间变，所有“收敛到 V^π”的结论都变成“跟踪一个移动目标”。此时应当用常数步长并定期重置，而不是追求收敛。

第五，噪声与延迟。 奖励若带测量噪声，$\max$ 算子会主动挑“噪声为正”的样本，产生第 31 章 §31.6.5 同款的乐观偏差。奖励若比动作晚几拍到达，TD 误差的时序就对不上，等价于把 R_{t+k} 记到了 A_t 上。**这两种问题在仿真里几乎不出现，在真机上几乎必然出现。**

32.3.9 三条路的代价：策略迭代、值迭代与线性规划

到这里已经有两条路（§32.3.1、§32.3.2），还有一条常被忽略的路——把 Bellman 最优方程直接写成一个线性规划。三者解的是同一个方程，差别只在“每一轮花多少钱、一共要走多少轮”。

策略迭代的每轮代价是一次“解 $|\mathcal{S}|$ 元线性方程组”。稠密情形下是 $O(|\mathcal{S}|^3)$，稀疏情形可以用迭代法降下来，但量级仍在 $O(|\mathcal{S}|^2)$ 到 $O(|\mathcal{S}|^3)$ 之间。它的轮数很少：策略只有有限个，严格改进不允许无限进行，实践中通常 5–20 轮就停。**所以 PI 的总代价大致是 $|\mathcal{S}|^3 \times n_{\text{PI}}$。**

值迭代的每轮代价是一次矩阵-向量乘加，稠密情形 $O(|\mathcal{S}|^2|\mathcal{A}|)$；轮数由压缩率定，式 (32-25) 给出

$$k \;\geq\; \frac{\ln\!\big(\|V_0 - V_*\|_\infty / \varepsilon\big)}{\ln(1/\gamma)}$$

$\gamma = 0.9$、$\varepsilon = 10^{-3}$、初值误差取 1 时是 66 轮。**总代价大致是 $|\mathcal{S}|^2|\mathcal{A}| \times k$。**

把表 32-2 的两行代进去看。$|\mathcal{S}| = 21$、$|\mathcal{A}| = 3$ 时，PI 每轮约 9.3×10^3 次运算、10 轮共约 10^5；VI 每轮 1.3×10^3、66 轮共约 8.7×10^4——**两者同量级，选哪个都不影响工程进度。**$|\mathcal{S}| = 10^5$ 时，PI 每轮 10^{15}，VI 每轮 3×10^{10}、66 轮 2×10^{12}，两个都做不了，只能换函数逼近。**真正拉开差距的是中间地带，以及 $\gamma \to 1$ 的时候。**

$\gamma \to 1$ 这一点值得单独强调。VI 的轮数按 $1/(1-\gamma)$ 增长，$\gamma = 0.99$ 时要 660 轮，$\gamma = 0.999$ 时要 6600 轮；PI 的轮数几乎不随 γ 变，因为它数的是“策略改了几次”，不是“误差缩了几倍”。**所以当问题需要长视野（γ 接近 1）而状态数又不大时，PI 是更省的那条路**——这也解释了为什么 LQR 用 Riccati 迭代（本质是 PI）而不是值迭代。

第三条路是线性规划。 最优值函数是下面这个 LP 的最小解：

$$\min_{\mathbf{v}} \sum_{s} v(s) \quad \text{s.t.} \quad v(s) \;\geq\; r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, v(s'), \quad \forall s, a$$

约束有 $|\mathcal{S}||\mathcal{A}|$ 条，变量 $|\mathcal{S}|$ 个。它的好处不在速度，而在**一次求解给出全局最优，并且给出对偶变量**——对偶变量恰好是“每个状态-动作对被约束住的影子价格”，可以直接读成“为什么这个动作被排除”。规模上，单纯形最坏是指数的，内点法是多项式的，实践里能处理到 10^4 量级的状态，再大就不行了。**它适合“规模小、但要向评审证明最优”的场合：**审图、安全认证、作为其他算法的基准解。

三条路在优化语言里的位置很清楚：**PI 相当于 Newton 法**（用精确的 Q^π 做一次改进，有二次收敛的趋势），**VI 相当于不动点迭代或梯度下降**（线性收敛，有效步长被 γ 隐式限定），**LP 相当于把问题整体交给通用求解器**。所以“γ 小 = 收敛快”在 VI 里成立，在 PI 里几乎不成立——它们对 γ 的敏感度不同，这是选路时最该记住的一条。

最后补一个实现上的事实：工程里跑的既不是纯 PI 也不是纯 VI，而是**异步值迭代**——每步只更新一个或一批状态，只要每个状态被无限次访问，收敛性仍然成立。它省掉了“扫全表”的开销，也让算法能在数据到达时增量更新。**“同步/异步”这一层选择，通常比“PI 还是 VI”更影响实际耗时。**

32.3.10 与自适应控制的亲缘关系：都在线，改的东西不同

读到 Q-learning 的更新式，很多控制背景的读者会觉得眼熟——它和模型参考自适应控制（MRAC）、自校正调节器（STR）在结构上确实很像。把对应关系说清，能省掉不少误解。

相同的三点。

其一，都采用确定性等价。 STR 把在线辨识出来的参数当成真参数，据此重新设计控制器；RL 把估计出来的 Q 当成真值函数，据此取 $\arg\max$。**两者都“把估计当真”，也都因此继承了估计误差的后果：**STR 的闭环极点会随参数估计误差漂移，RL 的策略会随 Q 的误差偏移，而误差放大因子正是 §32.2.7 那个 $\gamma/(1-\gamma)$。

其二，都需要激励条件。 MRAC 与 STR 要求回归向量持续激励（PE），阶数不低于待辨识参数的个数；RL 要求每个 (s,a) 被无限次访问。**这两句话是同一个条件的不同说法：数据必须覆盖你想控制的那个方向。** PE 说的是“输入信号要够丰富”，覆盖说的是“状态-动作对要够全”；缺了它，前者辨识出的参数不可信，后者学出的 Q 在没去过的格子上是任意的。

其三，都假设慢变。 两者的收敛结论都建立在“对象参数在自适应收敛的时间内不变”之上。参数漂移快于自适应速度时，STR 会跟踪一个移动目标并产生持续振荡，RL 的值函数同样追不上；此时正确的做法都是退回常数增益并接受偏差，而不是加大步长。

不同的三点。

其一，保证的来源不同。 MRAC 在结构假设（相对阶、最小相位等）成立时能给出 Lyapunov 意义下的稳定性证明，保证的是“跟踪误差有界或收敛到零”；RL 的收敛定理保证的是“Q 收敛到 Q_*”，**它不保证闭环稳定**——因为 RL 的目标函数里根本没有“稳定”这一项。

其二，目标不同。 自适应控制的目标是单一的：跟踪。RL 的目标是一个标量回报，可以是多指标折中，也可以与跟踪无关。E4 里 Q-learning 学出“贴着悬崖走 13 步”的策略就是例证：**从回报看它是最优的，从风险看它是不可接受的**——这两件事在自适应控制里不会同时出现，因为后者的目标函数里只有跟踪误差。

其三，探索。 MRAC 不需要主动探索：跟踪误差本身就是激励源，控制器不必“故意做错动作”。RL 必须探索，而探索在控制系统里往往意味着“故意违反工艺规程”，这正是 §32.5.3 那三层处理的由来。

所以选型的分界线很清楚：对象线性、结构已知、只需要调增益或几个参数，用第 23 章的在线辨识加自适应律，样本少、有稳定性保证；目标是“在多个互相冲突的指标之间折中”，或者决策是序列的、当前动作影响很久以后的代价，才轮到 RL。**RL 相对自适应控制的真正增量，不是“能在线学”，而是“能为一类自适应控制写不出的目标函数（多目标折中、序列决策、稀疏奖励）定义并优化”。** 把这一点想清楚，就不会拿 RL 去做 STR 已经做得很好的事。

32.4 离散对应与数值实现

32.4.1 离散化：先算预算，再定格数

连续对象进表格之前必须量化，而量化精度不是“越细越好”，它受样本预算的硬约束。设总交互预算为 B、动作数为 $|\mathcal{A}|$、每对 (s,a) 至少要被访问 $k_{\min}$ 次，则状态数必须满足

$$|\mathcal{S}| \le \frac{B}{k_{\min}\,|\mathcal{A}|} \tag{32-39}$$

$k_{\min}$ 取 20 是保守值（样本均值的标准差约为单样本标准差的 $1/\sqrt{20}=22\%$）。若 $B=10^5$、$|\mathcal{A}|=3$、$k_{\min}=20$，则 $|\mathcal{S}|\le 1667$；四维状态每维只能分 $1667^{1/4}\approx 7$ 格。**这个数通常比设计者第一反应小一个数量级**——不是不想细，是样本买不起。

确定格数之后，单维步长 Δ 有两个约束。**下界来自噪声**：Δ 小于两倍测量噪声时，同一个物理状态会被噪声抖到相邻格，等价于人为扩大了状态空间，Q 表被稀释。取 $\Delta \ge 2\sigma_n$ 是底线。**上界来自分辨率需求**：Δ 应小于“你希望策略区分的特征尺度”，比如死区宽度、允许的稳态误差带。两端一夹，往往只剩 10–20 档。

动作的离散化遵循同样的逻辑，但更粗。动作维度的 Q 差异通常比状态维度小，档位太密会让差异淹没在噪声里。**工程经验是 3–7 档**：三档（增/保/减）适合慢过程，五档适合有明确档位语义的执行器，超过七档就该考虑连续动作方法（第 33 章）。

一个具体的账：温度对象工作在 20–100\,\^\circC，取 21 格，步长 4\,\^\circC；测量噪声 0.5\,\^\circC，$\Delta/\sigma=8$，满足下界；稳态允许偏差 2\,\^\circC，4\,\^\circC 的格宽意味着策略最多只能把温度控到 \pm2\,\^\circC，正好卡在上界。**如果指标要求 \pm0.5\,\^\circC，这套离散化从第一步就不可能达标，换算法也救不回来。**

32.4.2 更新顺序与数据结构

表格实现的第一件事是把状态映射成整数下标。用多维数组而不是字典：`Q[s_idx, a]` 是一次内存寻址，字典查找在百万次更新的规模下会慢一个数量级。若状态是 d 维格点，用 `np.ravel_multi_index(idx, dims)` 一次性算好，不要在循环里反复算。

第二件事是决定同步还是异步更新。同步更新（每轮用旧表算新表）便于分析，收敛速度就是 γ^k；异步就地更新（算完立刻写回）在同样轮数下通常快 2–3 倍，因为它把“本轮已经变好的值”立刻传给了邻居。**代价是不可复现的顺序依赖**——如果并行，必须固定归约顺序，否则同样的种子跑出不同的结果。

第三件事是记录访问次数。用 $\alpha_t=1/N_t(s,a)$ 时，

$$\alpha_t(s,a)=\frac{1}{N_t(s,a)} \tag{32-40}$$

它就是样本均值，方差最小；但对象时变时它“记得太久”，新样本的权重只有 $1/N$，跟不上漂移。折中是 $\alpha_t=\max(1/N_t,\alpha_{\min})$，$\alpha_{\min}$ 取 10^{-3}–10^{-2}，给算法留一个“遗忘速度”。

初始化也影响行为。全零初始化在奖励全负时会让“未访问”看起来比“访问过”更好，算法会主动去试没试过的动作，这其实是免费的探索（乐观初始化）。**若奖励全为正，全零初始化则让算法不愿离开起点**，此时应把初值设成理论上界 $R_{\max}/(1-\gamma)$。

32.4.3 探索：ε 的衰减要按“访问覆盖率”定，不按回合数拍脑袋

ε-greedy 以 $1-\varepsilon$ 选当前最优、以 ε 均匀随机。固定 ε 时算法收敛到“ε-soft 策略”的值，与最优值的差距随 ε 线性增长；ε 衰减太快则有些动作永远没被试过。常用的指数衰减是

$$\varepsilon_t = \varepsilon_{\min} + (\varepsilon_0 - \varepsilon_{\min}) e^{-t/\tau} \tag{32-41}$$

取 $\varepsilon_0 = 0.3$、$\varepsilon_{\min} = 0.01$、$\tau = 2000$ 回合，则第 5000 回合时 $\varepsilon = 0.01 + 0.29 e^{-2.5} = 0.034$——仍在探索，但已经是“偶尔试一下”的量级。

表 32-3 把四种常见探索方案放在一起。

表 32-3　四种探索方案的适用场合

方案	机制	优点	代价	适用
ε-greedy	以 ε 均匀随机	实现最简单	探索动作质量差	通用基线
softmax	按 Q 的 Boltzmann 分布采样	好动作也有机会	对 Q 的尺度敏感	Q 值范围稳定时
乐观初始化	初值设高，未访问更诱人	前期探索密集	只在前若干轮有效	奖励全负
计数奖励	给少访问的 (s,a) 加奖励	有理论保证	大状态空间计数难	小规模表格

选哪种不如“看覆盖率”实在。训练中定期统计访问过的 (s,a) 占比：**二维小问题应在 2000 回合内到 90% 以上；低于 60% 说明探索不足，此时调任何学习率都没用。** 覆盖率是比回合回报更早暴露问题的指标。

还要提一句安全：真机上不能让 ε 随机选到危险动作。做法是动作屏蔽（action shielding）——在采样前把越界或违反联锁的动作概率置零再归一化。**这不是算法的一部分，是部署的一部分，但它会改变实际执行的策略，报告里必须写明。**

32.4.4 停止准则与评估口径

训练回报不能直接当评估指标，因为它包含探索的随机性：$\varepsilon = 0.3$ 时三成的动作是乱选的，回报自然低。**评估必须用贪心策略、在独立种子的环境上跑，且至少 20 回合取平均。**

停止准则给三条，满足其一即停：

1. **性能平台**：贪心策略的评估回报连续 50 回合的相对改善小于 1%；
2. **残差平台**：TD 误差的 RMS 连续 1000 步不降，且绝对值小于 $10^{-2} R_{\max}$；
3. **预算到顶**：交互次数达到 B。这条永远优先于前两条。

前两条都可能被“假平台”骗：探索不足时策略会早熟，回报和残差都会平稳。**所以停机前必须再看一眼覆盖率**，覆盖率低而指标平，说明卡在局部，不是收敛。

32.4.5 可重复性：三条随机流要分开

一次 RL 运行至少有三处随机：环境转移（$s' \sim P$）、策略采样（$a \sim \pi$）、初始化与打乱。三者混用一个全局随机数发生器时，改动任何一处都会让整条轨迹错位，实验无法对比。

正确做法是用 `np.random.SeedSequence(seed).spawn(3)` 分出三条独立流，分别喂给环境、策略、初始化。**这样“只换探索种子”才是一次单变量实验。** 并行采样时每个环境再 spawn 一次，保证“加机器”不改变单环境的语义。

还要记录三样东西：种子、环境版本（离散化表、奖励函数、终止条件的哈希）、库版本。**没有这三样，“复现”只能靠运气。** 第 31 章 §31.4.4 的可重复性清单在这里同样适用，只是多了一条“环境本身要版本化”。

32.4.6 Q-learning 骨架

下面这段是表格型 Q-learning 的完整骨架，去掉日志后不到 40 行。它把前面几节的约定都落到了代码上。

```python
import numpy as np

def q_learning(env, episodes, alpha_min=1e-3, gamma=0.95,
               eps0=0.3, eps_min=0.01, tau=2000, seed=0):
    """表格型 Q-learning；环境需提供 reset() 与 step(a) -> (s2, r, done)。"""
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.n_states, env.n_actions))      # 奖励全正时应改为 R_max/(1-gamma)
    N = np.zeros_like(Q)                            # 访问计数，用于 1/N 步长
    for ep in range(episodes):
        eps = eps_min + (eps0 - eps_min) * np.exp(-ep / tau)
        s, done = env.reset(), False
        while not done:
            # epsilon-greedy：探索用随机整数，利用用 argmax
            if rng.random() < eps:
                a = int(rng.integers(env.n_actions))
            else:
                a = int(np.argmax(Q[s]))
            s2, r, done = env.step(a)
            # 截断与终止要分开：截断仍需自举，终止才把未来置零
            target = r if env.terminated else r + gamma * np.max(Q[s2])
            N[s, a] += 1
            step = max(1.0 / N[s, a], alpha_min)         # 常数下限用于时变对象
            Q[s, a] += step * (target - Q[s, a])
            s = s2
    return Q
```

三处细节值得单独说。第一，`env.terminated` 与 `done` 必须分开：回合因超时被截断时，未来仍然存在，只是没观测到，目标要保留 $\gamma \max Q(s_2, \cdot)$；只有真正到达终止状态才把未来置零。把两者混为一谈，会让靠近时限的状态值被系统性低估。第二，`1/N` 与常数下限并用，前者保证收敛，后者保证跟踪。第三，`argmax` 的并列取第一个，当两动作值相同时策略固定，不会来回抖。

32.4.7 训练循环的工程细节

回合长度要有上限。 控制任务里“没到目标”和“卡住了”是两件事，前者是终止，后者是截断。不设上限时，一个坏策略可能让单个回合跑十万步，把预算烧光。

日志按固定间隔写，不按回合写。 建议每 100 回合记录一次：评估回报、ε、TD 误差 RMS、覆盖率、当前贪心策略的动作分布。最后一项能看出“策略是否退化成常量”——所有状态都选同一个动作时，动作分布的熵接近 0。

Q 表要能存能读。 用 `np.save` / `np.load` 存二进制，文件名带种子与回合数。表格小，但重跑一次可能要几十分钟，检查点能省掉大量重复。

训练与部署的策略要分开导出。 部署时用 $\arg\max$ 的确定策略，把 Q 表转成一张查找表（状态下标→动作），连 numpy 都可以不要。**这是表格型方法在嵌入式场景唯一胜过神经网络的地方：推理成本是一次数组访问。**

32.4.8 经验回放：借来的是独立性，欠下的是分布

表格型 Q-learning 每步在线更新，不需要存历史；到了函数逼近（第 33 章）几乎都会加一个经验回放池。回放不是“多存点数据”这么简单，它同时借来了一样东西、欠下了一样东西。

借来的是样本的独立性。 相邻两步的 (s_t, a_t, r_t, s_{t+1}) 高度相关：状态几乎没变、奖励几乎没变、动作可能一样。用这样的序列做随机梯度，梯度的方差不会按 $1/\sqrt{N}$ 降，有效样本量远小于名义样本量。回放池把轨迹打散后均匀抽样，等价于人为造出一批近似独立的样本。**这是深度 RL 能训练起来的最主要原因之一，不是网络结构。**

欠下的是分布的一致性。 回放池里的样本来自许多个历史策略，而当前要评估的是 π_θ。用旧策略的数据去更新新策略，就是异策略（off-policy）。表格型对此不敏感——每个 (s, a) 的更新只影响自己那一格；函数逼近则不同：一次梯度更新会同时改变许多状态的估计，旧数据把新参数“拉回”它所属的那个分布。§32.3.8 第三条说的“三者同时出现就发散”，异策略这一条正是回放的直接后果。

三个工程参数因此不是随便定的。

池子容量。 太小，样本仍然相关；太大，旧数据占比高、分布偏移严重。表格型问题常见量级是 10^4–10^5 条转移，相当于 10^2–10^3 条完整轨迹。一个可操作的口径是：池子应覆盖“至少一个完整的工况周期”，让所有状态-动作组合都有机会出现，同时不超过“策略明显改变所需步数”的 10 倍。

抽样方式。 均匀抽样最简单。优先经验回放（PER）按 TD 误差的绝对值给样本加权，把“还没学会的地方”多抽几次；代价是引入偏差，需要乘一个重要性权重 $w_i = \big(1/(N\,P(i))\big)^{\beta}$ 来修正，β 从 0.4 线性升到 1。**PER 在奖励稀疏时收益明显，在奖励稠密时几乎没有收益**——因为稠密奖励下 TD 误差的分布本来就很均匀。

什么时候不该用。 同策略算法（A2C、PPO）原则上只能用当前策略采的数据，因为它们的目标函数里含有“当前策略的熵、当前策略的概率比”；把旧数据混进来，推导里的期望就对不上。工程上常见的“用了重要性采样所以能混”是不严谨的：重要性权重会随策略偏离指数增长，几轮之后就退化成几个样本在起作用。**判断标准很简单：算法更新式里出现 $\pi_\theta(a \mid s)$ 在分母或分子上、且没有对应的旧策略概率，就不能混。**

最后一句提醒。回放池在控制场景里有一个额外的敌人——**非平稳**。对象参数漂移时，池子里的旧样本描述的是另一个对象；此时“多存”反而是负担，池子应当缩短、甚至改成滑动窗口。**这也解释了为什么慢过程（温度、液位）比快过程（电机电流）更愿意用大回放池：前者的动态在几小时内不变。**

32.4.9 量纲一致：学习率、奖励尺度与 γ 三者的联动

调参调不动，八成是三个量没有一起改。它们的耦合关系可以一句话说清：**学习率是“以值函数为单位”的步长，而值函数的量纲是“奖励 / $(1-\gamma)$”。**

先把这件事算清楚。若奖励整体乘 c，则 Q 与 V 也乘 c（Bellman 方程是线性的）。更新式

$$Q \leftarrow Q + \alpha\big[R + \gamma \max Q' - Q\big]$$

两边同除 c，令 $\tilde{Q} = Q/c$、$\tilde{R} = R/c$，得到

$$\tilde{Q} \leftarrow \tilde{Q} + \alpha\big[\tilde{R} + \gamma \max \tilde{Q}' - \tilde{Q}\big]$$

形式完全不变，α 不用改。 那为什么实践中改奖励尺度常常导致发散？因为**有效步长**不是 α，而是 α 相对于值函数的相对变化率：$\Delta Q/Q \approx \alpha\,(R + \gamma Q' - Q)/Q$。奖励放大 c 倍时，TD 误差放大 c 倍，若 α 不变，单步的相对修正量也放大 c 倍，越过了稳定边界。**所以“α 不用改”只在精确算术下成立；浮点与非线性逼近下，α 必须按 $1/c$ 缩。**

由此得到三条可执行的规则。

规则一：先定奖励尺度，再定 α。 常用做法是把单步奖励归一化到 $[-1, 1]$：除以量程、或减均值除标准差。归一化之后 α 的可用区间才稳定（表格型约 0.01–0.2，函数逼近约 10^{-4}–10^{-3}）。**不归一化就直接抄别人的 α，是最常见的失败方式。**

规则二：γ 变了，α 要跟着降。 值函数的量级约为 $\bar{r}/(1-\gamma)$，γ 从 0.9 提到 0.99，量级放大 10 倍；同样的 α 对应的相对步长也放大 10 倍。经验上是 $\alpha \propto (1-\gamma)$，至少不要反着来。E2 的结果从侧面支持这一点：$\gamma$ 从 0.9 到 0.99，达到同一精度所需轮数涨了十倍，若同时把 α 也放大十倍，就不是“多花时间”，而是“不收敛”。

规则三：奖励的加性常数只影响基线，不影响策略。 给所有奖励加同一个常数 b，Q 整体加 $b/(1-\gamma)$，$\arg\max$ 不变——所以基线（baseline）在策略梯度里可以随便减，不影响最优解，只降方差。E6 的对比很直观：奖励是 100 量级时，不减基线，梯度乘子的均方根是 100.37；减掉常数基线后降到 1.11，差 90 倍；选中最优臂的概率则从 0.000 变成 0.991。**同一个 α 在两种写法下相当于差了两个数量级的有效步长——这就是“基线不改变问题、只改变可学性”的定量版本。**

一个自检口诀：**改奖励，先除以量程；改 γ，先按 $(1-\gamma)$ 缩 α；加常数，只当基线用。** 三条都不涉及算法本身，却决定了同一份代码在另一个对象上能不能跑起来。

32.4.10 覆盖率：表格填满了没有

表格型方法有一个别的算法没有的好处——**“学得怎么样”可以直接看表。** 但看的不是 Q 的值，是每格被访问了多少次。

先定义清楚。 设 $N(s,a)$ 为累计访问次数，则覆盖率有两个口径：**格子覆盖率**是 $N(s,a) > 0$ 的比例，**有效覆盖率**是 $N(s,a) \geq k_{\min}$ 的比例（$k_{\min}$ 取 10–20）。两者常常差得很远：训练日志里“覆盖率 95%”通常指前者，而真正决定估计质量的是后者。

覆盖率与 ε 不是线性关系。 ε-greedy 下，某个非贪心动作被选中的概率约为 $\varepsilon/|\mathcal{A}|$；但一个格子能否被访问，还取决于状态本身是否可达。若策略很快收敛到一条固定轨迹，那么轨迹之外的状态即使 ε 不小，也可能一次都到不了。**所以“ε 调大”不等于“覆盖变好”，要看的始终是访问计数，不是 ε 的数值。**

覆盖率高不等于学得好。 E7 的稀疏奖励是一个干净的例证：600 回合之后大部分格子都被访问过，但奖励只在目标格非零，其余格子的 Q 全零，纯贪心到达率是 0%。**覆盖只保证“信息进来了”，不保证“信息里有信号”。** 这两件事必须分开诊断：覆盖不够，加探索；覆盖够了但学不好，查奖励（§32.5.1）或查状态定义（§32.2.8）。

三种补覆盖的手段，代价依次升高。

其一，乐观初始化。 把所有 Q 初始化成一个偏大的值（大于任何可能的真实值），未访问的格子因此总是“看起来最好”，策略会主动去试。它不需要额外的奖励设计，代价是前期性能差、且初值大小需要估计值域。

其二，计数奖励。 给访问少的格子额外奖励，$r^{+} = \beta / \sqrt{N(s,a)}$，$\beta$ 取奖励量级的一半左右。它是“内在动机”的最简版本，效果稳定，缺点是改变了奖励的含义——最终策略会带一点“爱去冷门地方”的偏好，需要后期把 β 衰减到零。

其三，状态聚合。 把“物理上相近、控制上等价”的状态合并成一格，直接减少格子总数。这是最根本的办法，但需要领域知识，且合并错了会引入偏差。

什么时候覆盖率提不上去。 两种情形要提前认清。**安全约束禁止探索**：某些动作在现场根本不能试（超温、超压），此时覆盖率的缺口无法通过算法补，只能靠离线数据、机理先验或模仿学习（第 33 章的离线 RL 正是为此）。**状态空间太大**：格子数远超样本预算，怎么探索都填不满，此时应当换函数逼近，而不是继续加样本。

一句话收束：**覆盖率是表格型的体检指标，它对应的理论条件是“每个 (s,a) 被无限次访问”，对应的工程直觉是“数据要覆盖你想控制的每一个角落”。** 这两句话在自适应控制里叫持续激励，在 RL 里叫充分探索，是同一件事的两种叫法。

32.5 工程含义与适用边界

32.5.1 奖励设计：把指标翻译成信号

控制里的指标是“误差、超调、能耗、寿命”，RL 只认一个标量奖励。翻译得好不好，直接决定学出来的策略有没有用。

最常用的形式与 LQR 的代价同形，只差一个负号：

$$r(s,a) = -\left(e^{\top} Q e + \lambda\, u^{\top} R u \right) \tag{32-42}$$

其中 e 是跟踪误差、u 是动作。这种“稠密奖励”每一步都有信号，信用分配容易，是控制任务的首选。**稀疏奖励（只在到达目标时给 +1）看起来干净，实际上把信用分配的全部困难留给了算法**：一条 200 步的轨迹只有最后一个数非零，前面的动作全靠猜。

稀疏奖励可以用势函数塑形（potential-based shaping）救。取一个状态势 $\Phi(s)$（比如“到目标的距离”），加一项

$$F(s,a,s') = \gamma \Phi(s') - \Phi(s) \tag{32-43}$$

它不改变最优策略。证明只需一行：若 Q 是原奖励下的动作值，则加 F 后的动作值满足 $Q'(s,a) = Q(s,a) - \Phi(s)$，减去的项只依赖状态，$\arg\max$ 不变。**这是“塑形”唯一有理论保证的形式**：随手加“靠近目标给奖励”没有这个性质，会引入新的最优策略——§32.6.7 会用一组数字展示“正确塑形策略不变、错误塑形策略被改掉”。

奖励设计最容易犯的错是**把手段当目标**。温度控制里若奖励只写“产量”，策略会把温度顶到上限；写“产量 − 能耗”但不写超温惩罚，策略会在危险边缘反复试探。工程上的修法不是调权重，而是把硬约束移出奖励：**越界直接终止回合，或用动作屏蔽禁止越界动作。** 奖励是“想要什么”，约束是“绝不能做什么”，两者混在一个标量里，权重再怎么调都是在做交易。

权重的标定也有办法，不必拍脑袋。取两个参考解：一个“完全不顾能耗”的解 J_1、一个“完全不顾产量”的解 J_2，要求策略的解落在两者之间且靠近工程可接受的那一端。**用可接受的解反推权重，比用权重去猜解稳得多。**

32.5.2 训练环境构建：仿真与真机之间隔着三样东西

RL 需要大量样本，真机给不起。算一笔账：仿真里每秒 1 万步，10^5 步只要 10 秒；真机 1 Hz，同样步数要 27.8 小时，还不算停机与损耗。**所以训练几乎总在仿真里做，问题变成“仿真里的策略搬到真机上还能不能用”。**

三样东西最容易漏。**第一是延迟。** 仿真里“动作立即生效”，真机上有执行器响应、通信、计算的总延迟；延迟超过一拍，策略看到的状态与它决策时的状态就对不上，闭环可能振荡。**第二是噪声与量化。** 传感器噪声会改变状态落格，进而改变动作；若训练时没加噪声，策略会对噪声过敏。**第三是未建模动态。** 摩擦、死区、饱和、温漂，仿真里往往被简化掉。

对应的工程动作是域随机化（domain randomization）：训练时把延迟、噪声、增益、摩擦在合理范围内随机采样，让策略见过“一族对象”而不是“一个对象”。代价是单个对象的性能下降，收益是迁移成功率高。**随机化的范围要用实测标定，不能凭感觉放大**——范围过宽会让策略学成“对什么都凑合”。

还有一个接口细节：动作保持（action repeat）。若控制周期是 10 ms、RL 决策周期是 100 ms，则同一个动作连续执行 10 拍。**这等价于把 MDP 的时间步长改成 100 ms，γ 也要按新的步长重算**（§32.1.2 的 $\gamma \approx e^{-T_s/\tau}$）。忘了这一步，策略的“视野”会差十倍。

上线前有一个可执行的检查：同一策略在仿真与真机上各跑 20 回合，比较回报分布。**若两个分布的中位数差超过 20%、或真机的最差回合低于仿真的第 5 百分位，就不能上线。** 这条判据比“平均回报差不多”严格，因为它看的是尾部。

32.5.3 安全约束：学习器不能有“试错权”

控制系统的硬约束（温度上限、压力联锁、行程限位）不允许“试错”。RL 的探索本质上是试错，两者直接冲突。工程上有三层处理，从弱到强。

第一层，罚项。 把违约量加进奖励，ρ 足够大时策略会避开。缺点是“软”的：ρ 有限，策略仍可能用少量违约换大量收益；ρ 太大，又会压垮其他项的量纲（第 31 章 §31.4.2 的归一化问题在这里同样存在）。

第二层，动作屏蔽。 在采样前把违约动作的概率置零再归一化。它是“硬”的，但只对“当前状态下的单步违约”有效，管不了“这一步安全、十步后越界”。

第三层，安全监督器。 学习策略之外再挂一个可验证的备份控制器（PID、联锁、MPC），

$$u = \begin{cases} \pi(s), & s \in \mathcal{S}_{\text{safe}} \\ u_{\text{backup}}(s), & s \notin \mathcal{S}_{\text{safe}} \end{cases} \tag{32-44}$$

$\mathcal{S}_{\text{safe}}$ 由一个简单的规则或不变集给出。**这是目前唯一能同时保住性能和安全的架构**：RL 在安全集内优化，出了集就交给保守控制器。代价是安全集边界附近的策略切换可能抖动，需要加滞环。

理论上更完整的框架是约束 MDP（CMDP）：最大化回报、同时让期望代价不超过阈值 d，

$$\max_{\pi} \; J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) \le d \tag{32-45}$$

常用拉格朗日法把约束变成 $\max_\pi \min_{\lambda \ge 0} J_r(\pi) - \lambda(J_c(\pi) - d)$，$\lambda$ 随违约自动调大。**它保证的是“期望”约束，不是“每一条轨迹”都安全**——对硬约束，仍然要回到动作屏蔽与监督器。系统性的安全论证（故障树、SIL 等级）在第 36 章。

32.5.4 实时性与部署：决策频率不必等于控制频率

表格型的推理成本是一次数组访问，微秒级；即使换成网络，一次前向在嵌入式 CPU 上通常也是毫秒级。**真正的问题不是“算不算得动”，而是“决策频率该多高”。**

控制回路 10 ms、RL 决策 100 ms 的分层结构在工程上最常见：底层 PID 保证快速跟踪与稳定，上层 RL 每个 100 ms 调一次设定值或增益。这样做有三层好处：样本需求降一个数量级（决策步数少了）、动作更平滑（不会每 10 ms 抖一次）、安全监督器有足够时间介入。

反过来的结构很少见：RL 直接以 10 ms 出动作，需要网络推理小于 1 ms，且训练时的延迟必须与部署一致。**如果做不到“训练与部署同频同延迟”，就不要选这种结构。**

32.5.5 样本成本的账：先算钱，再选算法

第 31 章的核心约束是评价次数，本章的核心约束是交互步数。两者的换算关系是“每一步多少钱”。

设单步试验成本为 c（含人工、能耗、损耗），总步数 B，跑 n 个种子，则总成本约 ncB。取 $c = 0.1$ 元、$B = 10^5$、$n = 20$，就是 20 万元。**这个数量级决定了“能不能用 RL”往往不是技术问题，而是预算问题。** 仿真里 c 可以低到 10^{-6} 元，同样的实验只要 2 元——这就是为什么“先仿真、再迁移”几乎是唯一可行的路径。

与第 31 章的方法比一比：贝叶斯优化 40 次评价就能在二维问题上找到不错的解；RL 动辄 10^5 步。**所以“评价昂贵、维度低、可以离线”的问题应该用 BO；“可以便宜地大量试、决策是序列的”问题才用 RL。** 两者的分界线是“单次评价成本 × 需要的评价次数”，不是问题的难度。

32.5.6 什么时候不该用 RL

表 32-4 是一张否决表。任何一条命中，都应该先考虑别的方案。

表 32-4　五种不该用 RL 的情形

情形	更合适的方法	理由
模型可靠、指标可写成二次型	LQR / MPC（第 30、58 章）	有闭式或凸解，无需样本
决策是单步的、无后续影响	第 30/31 章的优化	没有信用分配问题，RL 是杀鸡用牛刀
样本极贵且不能仿真	贝叶斯优化（第 31 章）	BO 用 40 次评价换一个可用解
安全不允许试错	规则 + 联锁 + 监督控制	探索本身不可接受
必须可解释、可认证	规则、增益调度、监督学习	RL 的策略难以给出形式化保证

这张表不是“劝退”，而是排序：**能用模型就用模型，能用优化就用优化，能用监督学习就用监督学习，都不行才用 RL。** 反过来，若一个问题同时满足“模型未知、决策影响未来、可以大量采样”，RL 就是合适工具——这也是它在游戏、机器人、流程工业里真正落地的场景特征。

32.5.7 与后续四章的分工

本章只讲表格型与“小规模、离散”的 RL。真实问题的状态空间远大于表格能承受的范围，接下来的四章依次补三块短板。

第 33 章解决“状态太多”：用函数逼近（线性特征、神经网络）替代表格，讲 DQN 的两个稳定化手段（目标网络、经验回放）、策略梯度的高效版本，以及离线 RL——只用历史数据、不与环境交互。**第 34 章**解决“多个体”：分布式控制、一致性、多智能体分解。**第 35 章**解决“利益冲突”：博弈论与多智能体 RL，纳什均衡与收敛性。**第 36 章**解决“怎么证明它安全”：功能安全、可靠性、故障树与 SIL 论证。

一条界线要划清：本章的收敛定理到第 33 章基本失效，第 33 章的算法到第 34/35 章又要重新讨论收敛。**每换一层假设，保证就要重算一次**，这是读后面四章时最该记住的态度。

32.5.8 多目标：奖励只能加权，优先级必须靠结构

控制系统的指标天然分层：安全第一、精度第二、能耗第三、寿命第四。RL 只接受一个标量奖励，于是设计者被迫把几层目标压成一个数。**压的方式决定了策略会不会“用安全换精度”。**

最直觉的做法是加权求和 $r = w_1 r_1 + w_2 r_2 + \cdots$。它有三个方面的问题，逐个说。

第一，量纲不可比。 温度误差 1 ℃、能耗 1 kW·h、阀门动作 1% 之间没有公共单位，权重只能靠试。归一化（各自除以量程）能治标，但归一化本身假定了“满量程的误差与满量程的能耗等价”，这个假定通常没人认真验证过。

第二，加权和只能取到 Pareto 前沿的凸包。 这是多目标优化的经典结论：扫遍所有权重，得到的解集是前沿的凸包；前沿的非凸段（往往是“两头都不错、中间没有”的那种设计）无论怎么调权重都取不到。**所以“我们把权重扫了一遍”不等于“我们找遍了所有折中”。**

第三，权重把兑换率写死了，而工程上的兑换率随工况变。 满负荷时多花 1% 能耗换 0.1 ℃ 是划算的，待机时同一个交换就不划算。固定权重做不到这件事，除非权重本身是状态的函数——那就等于又引入了一个需要整定的策略。

E7 给了一个更尖锐的例子。朴素塑形奖励“靠近目标就 $+3$”，本质是把“进度”当成一个与“到达”并列的目标加权进去。结果是“靠近—远离—再靠近”的两步循环每转一圈净赚约 2.7，折现后总值 $2.7/(1-0.81) \approx 14.2$，**高于老老实实走到目标的 4.783**；学出来的 $V(\text{起点}) = 22.448$，贪心走 60 步也到不了目标。**这不是算法出错，是我们给“绕圈”标了一个合法的价格。** 只要加权，就有价格；有价格，就能套利。

正确的做法是**把优先级做成结构，而不是做成权重。**

第 0 级：硬约束，用结构表达。 温度上限、行程限位、联锁条件，不进奖励，进动作屏蔽与安全监督器（§32.5.3 的第二、三层）。它们的语义是“不允许”，不是“很贵”。

第 1 级：性能，用奖励主体表达。 跟踪误差、超调、调节时间，合成一个归一化的负代价（式 32-42 的形式）。

第 2 级：经济性，用小的正则项或事后优化表达。 能耗、动作变化率、寿命损耗。它们应该小到“不会为了它牺牲性能”，但大到“同等性能下会选更省的那条”。

越级的代价是明确的：一旦把第 0 级写成一个大负数而不是硬结构，策略就会去找“违约一点点、收益很大”的解——因为对算法而言，那确实更优。ρ 调到多大都不够，只要收益是有限的、惩罚是有限的，就存在一个“值得违约”的工况。

最后给一张可以直接抄进设计文档的清单，四行，从左到右写：

- **不越限、不联锁**：优先级 0（硬），用动作屏蔽 + 监督器表达，验收指标是“违约次数必须为 0”；
- **跟踪误差、超调**：优先级 1，用奖励主体（归一化负代价）表达，验收指标是阶跃响应指标；
- **能耗、动作磨损**：优先级 2，用小权重正则表达，验收指标是“同性能下的能耗对比”；
- **寿命、维护间隔**：优先级 3，用事后统计或长期约束表达，验收指标是运行台账。

每行的最后一句都要单独强调：**验收指标必须独立于奖励。**E7 里“训练末段成功率”与“纯贪心到达率”这两个指标都不是奖励的一部分，正因为它们独立，才暴露出“V 很高但到不了目标”这个矛盾。**奖励是给算法看的，验收指标是给人看的；两者写成同一个式子，就等于用自己证明自己。**

32.5.9 接进 DCS：接口一致性比算法精度更重要

策略在仿真里表现正常，接到现场却抖、迟、跳，多数不是算法的问题，而是接口。四个地方必须与训练时逐项对齐。

其一，限幅与速率限制。 现场的执行器一定有量程和速率上限（阀门 1%/s、变频器 5 Hz/s）。若训练时允许策略一步从 20% 跳到 80%，而现场被速率限制拉成 60 s 的斜坡，那么策略在闭环里看到的动态与训练时完全不同——**它学到的“一步到位”在现场变成了“慢慢爬”，而它从没见过这种响应。** 正确做法是在训练环境里就加上同样的限幅与速率限制，让策略学会在约束下工作；而不是在部署端加一层“保护”了事。

其二，无扰切换。 手动/自动切换、备份控制器接管、策略热更新，这三种场合都会出现“输出突变”。PID 的常规做法是让积分项跟踪实际输出（抗积分饱和 + 无扰切换）；RL 策略要做对应的事：**把“当前实际输出”放进状态或作为动作的基准**，这样策略输出的是“相对当前输出的修正”，切换瞬间的跳变自然为零。若策略输出绝对量且状态里没有当前输出，切换时就必须靠外部做斜坡过渡，过渡期的行为无法保证。

其三，超时与故障。 推理有延迟，也有失败的可能。必须预先定义三件事：单次推理的时间上限、超时后的动作（保持上一拍，还是退回备份控制器）、连续超时几次触发报警。**这三条要写进联锁表，而不是写在代码注释里。**

其四，归一化的冻结。 训练时用运行均值与标准差做标准化，部署时必须把最后一组统计量固化进模型文件，**不能在线更新**。在线更新的后果是：工况漂移时，标准化参数跟着漂，网络看到的输入分布却始终“正常”，异常被悄悄抹平；更糟的是同一份输入在不同时刻被映射成不同数值，策略的行为无法复现。**这一条与 §32.4.5 的“随机流要分开”是同一类纪律：凡是会变的东西，都要明确它什么时候允许变。**

再补一个常被漏掉的：**日志要能复盘**。每条决策至少记下状态向量、动作、策略版本号、时间戳。缺了版本号，事后无法判断“这次抖动是策略换版引起的，还是对象变了”；缺了时间戳，无法把策略动作与工艺事件对齐。**这一条的成本很低，价值极高——出问题时它是唯一能还原现场的东西。**

还有一条容易漏的：**训练与部署的采样周期必须一致。** 若训练时 $T_s = 100$ ms、现场因为网络或调度变成 50 ms，同一套 γ 与状态窗口描述的已经是两个不同的对象：视野从 100 步变成 200 步，动作的变化率翻倍，速率限制的相对强度也变了。**遇到这种情况，宁可把策略降频到训练时的那一档，也不要让它在“另一个时间尺度”上运行。**

这些接口逐项对齐之后，算法才真正参与闭环；在此之前，它只是一段离线推理。

32.6 仿真与验证

七组实验全部由 `code/ch032_sim.py` 单次运行产生，种子 20261032，总耗时 16.7 s。下面每个数字都能用一条命令复现，正文不做事后取整。

32.6.1 E1：三轮评估、两次改动，复现 -36.8 与 $+15.9$

第一组把 §32.1.3 的手算变成代码：用线性方程解策略评估、用 $\arg\max$ 做改进。

表 32-5　两状态反例的策略迭代全过程

轮次	$\pi(S)$	$\pi(F)$	$V(S)$	$V(F)$	$Q(S,p)$	$Q(S,c)$
0	继续	继续	-36.842	-100.000	-36.842	-34.158
1	冷却	冷却	-10.000	-12.000	-7.180	-10.000
2	继续	冷却	15.872	11.284	15.872	13.284

三个数与手算逐位一致：-36.8421、-10、15.8716。第一轮两个状态都换成冷却，其中 S 的换法是“错”的方向——它在第二轮又换了回来。**算法只要求每一步不变差，不要求每步都朝最优走**，这是策略迭代对初值不敏感的原因。

表里最后一列还给出一个工程上很有用的量：收敛时 $Q(S,p) - Q(S,c) = 15.872 - 13.284 = 2.588$。**这是策略的“安全边际”**：若两个 Q 的估计各带标准差 σ 的噪声，差值就带 $\sqrt{2}\sigma$；$\sigma > 1.83$ 时差值可能翻号，动作就会选反。真机上做这个判断，要比仿真里多留一倍余量。

32.6.2 E2：值迭代的轮数与两个放大因子

在一维 21 状态链上（动作 $-1/0/+1$，带 0.2 的邻格扩散），把值迭代跑到相邻差小于 10^{-3}，再与收敛到 10^{-13} 的参考解比。

表 32-6　值迭代的轮数与误差放大

γ	相邻差 $< 10^{-3}$ 的轮数	真实误差	误差 / 阈值	界 $\gamma/(1-\gamma)$	真实误差 $< 10^{-3}$ 的轮数
0.9	34	8.65×10^{-3}	8.65	9.00	55
0.99	340	9.88×10^{-2}	98.75	99.00	797

三件事被证实。**第一，停机判据确实低估误差，放大倍数紧贴理论界**：8.65 对 9、98.75 对 99，都从下方逼近 $\gamma/(1-\gamma)$。所以“相邻差小于 10^{-3}”绝不等价于“误差小于 10^{-3}”。

第二，γ 从 0.9 到 0.99，轮数从 34 涨到 340，正好十倍，与 $1/(1-\gamma)$ 的标度一致。若按“真实误差小于 10^{-3}”算，是 55 到 797，涨了 14.5 倍——因为两种口径的常数项不同，报轮数时必须同时报口径。

第三，误差衰减是指数的，斜率就是 γ。 图 32-2 把归一化增量画在对数坐标上，两条曲线与 γ^k 参考线平行。**看到“半对数坐标下是直线”，就该想到“每轮乘一个常数”**，这是判断一个迭代算法是否线性收敛的最快方法。

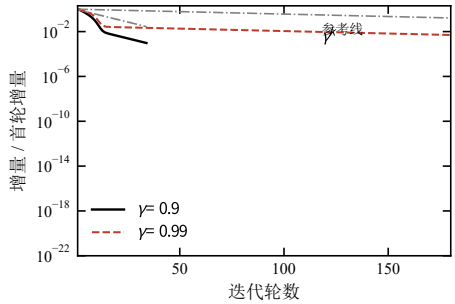


图 32-2　值迭代增量的指数衰减

32.6.3 E3：这次蒙特卡洛赢了 TD

5 状态随机游走，$\gamma = 0.9$，步长 0.05，20 个种子，每 50/100/200/300 回合测一次五个状态的 RMSE。

表 32-7　MC 与 TD(0) 的 RMSE（20 个种子的中位）

回合数	50	100	200	300	目标标准差
蒙特卡洛	0.0682	0.0777	0.0603	0.0747	0.240
TD(0)	0.2522	0.1748	0.1629	0.2138	0.087

这个结果与“TD 通常更好”的流行说法相反：**MC 的 RMSE 只有 TD 的三分之一到二分之一。** 但表里最后一列同时给出了反方向的证据——TD 目标的方差确实小，标准差 0.087 对 0.240，比值 2.77，与 §32.3.4 推算的 2.9 基本吻合。

两个结论并不矛盾，它们说的是不同的事。**TD 的方差小，指的是“单步目标的波动小”；TD 的误差大，指的是“收敛后的偏差大”。** 偏差来自两处：初值全零（真值在 0.07–0.69 之间），以及常数步长 0.05 留下的偏差地板——步长越大，地板越高。MC 没有这两项，它只是吵，但不偏。

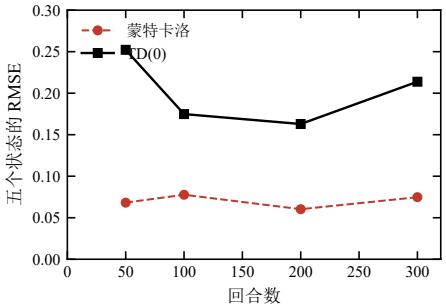


图 32-3　同一任务上 MC 与 TD 的 RMSE

图 32-3 里两条曲线都没有明显下降趋势，说明 300 回合远未到“收敛”，各自停在“方差地板”与“偏差地板”上。**这提醒一件事：比较 MC 与 TD 时，必须固定步长策略与初值，否则比的不是算法，是调参。**

那 TD 什么时候真的更好？状态多、回合长、需要在线更新时。那时 MC 要等整条轨迹（可能几百步）才能更新一次，方差随视野指数增长；TD 每步都能更新，且方差被 γ 压住。**本章这个小任务恰好落在 TD 不占优的区间**——状态只有 5 个，回合平均不到 10 步。

32.6.4 E4：13 步与 17 步，最优策略不等于最优行为

悬崖行走：4 行 12 列，起点左下、目标右下，底边中间 10 格是悬崖（掉下去罚 100 并回起点），每步罚 1。$\varepsilon=0.1$、$\alpha=0.5$、$\gamma=1$，800 回合，20 个种子。

表 32-8　Q-learning 与 SARSA 在悬崖上的表现

算法	贪心路径步数	在线回报（末 **100** 回合）	部署回报（纯贪心）
Q-learning	13	-54.4	-13
SARSA	17	-30.8	-17
最优	13	—	-13

这张表要横着读。**看贪心路径，Q-learning 赢了**：13 步贴着悬崖走，正是最优；SARSA 绕上方走 17 步，多花 4 步。**看在线回报，SARSA 赢了**：-30.8 对 -54.4，每回合少损失 23.6 分。原因是 Q-learning 学的策略假设“下一步总选最优”，但它自己在训练时以 0.1 的概率乱走，贴崖走十几次总有掉下去的时候；SARSA 把“我也会乱走”算进了值里，于是主动绕远。

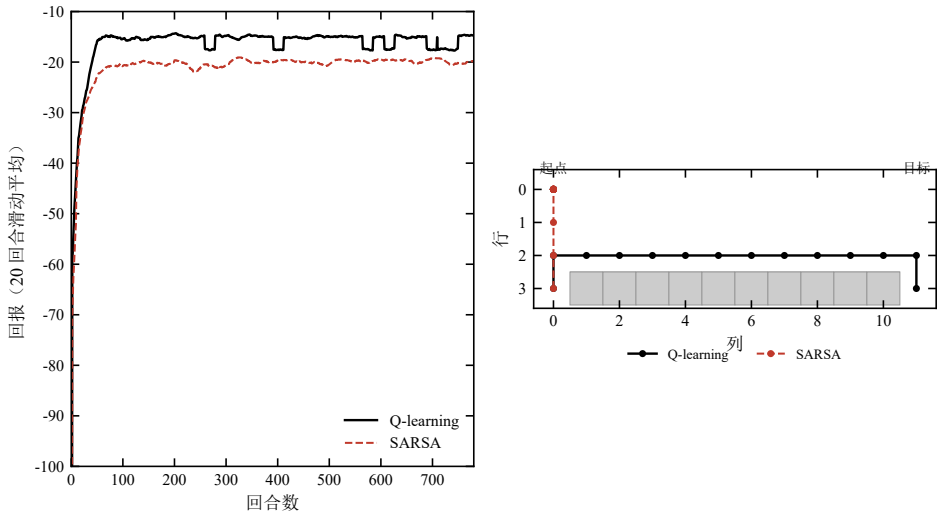


图 32-4　悬崖行走的学习曲线与两条贪心路径

图 32-4 右侧能直接看出两条路的取舍：Q-learning 的路径紧贴灰色悬崖带，SARSA 从上方绕行。左侧的学习曲线在 300 回合后分开并保持，说明这不是种子偶然。

这个实验给出一条部署判据：**如果上线后探索不会停止（比如 ε 固定、或人机会接管），选 SARSA；如果上线后是纯贪心执行，选 Q-learning。** 两者的差别不在“谁更聪明”，而在“训练时的行为是否等于部署时的行为”。

32.6.5 E5：温度调节，学出来的是一条开关律

前四组都是“算法对照”，这一组换成一个像控制问题的对象：一阶热惯性

$$T_{k+1}=T_k+0.05\,(20-T_k)+1.2u_k+w_k,\qquad w_k\sim\mathcal{N}(0,0.5^2) \tag{32-46}$$

$u\in\{0,1,2\}$，三个档位的平衡温度分别是 20、44、68 ℃；目标 60 ℃。状态取 20–100 ℃、每 4 ℃ 一格共 21 格，奖励 $r=-|T-60|/4-0.1u$，$\gamma=0.9$。Q-learning 训 1500 回合、每回合 80 步，ε 从 0.3 指数衰减到 0.01，学习率取 $1/N$ 但下限 0.05，初温在 30–80 ℃ 内随机。评估时从 60 ℃ 附近起步，只统计后 40 步。

十个种子的中位结果：**稳态平均绝对误差 1.123 ℃，71% 的（状态，动作）对被访问过，学到的策略是**

$$\pi=[0,0,0,\underbrace{2,\ldots,2}_{8\text{ 格}},0,\ldots,0] \tag{32-47}$$

即 20–28 ℃ 全关、32–60 ℃ 满功率、64 ℃ 以上全关。

第一条结论：这不是 PID，是一条开关律。 满功率的平衡温度是 68 ℃，略高于目标，所以中段必须满功率才爬得上去；一旦越过 60 就全关，靠散热压回来。中档 $u=1$ 被完全弃用，原因不是算法“想不到”，而是它被两头支配：升温时 $u=2$ 更快，刹车时 $u=0$ 更

狠且不花动作代价。**中间档在“快”和“省”两个方向上都输，就没有存在的理由**——这类“被支配动作”在控制里对应“永远不该选的冗余档位”，用 RL 跑一遍能直接把它筛出来。

第二条结论：1.123 ℃ 的误差小于半格。 状态离散化本身就是 4 ℃ 一格，量化误差本该有 ± 2 ℃；实际误差只有 1.12 ℃，说明策略在格子内部靠“关—开—关”的来回切换，把温度稳定在一个比格子更细的平均值上。代价是稳态有锯齿：从 64 ℃ 关断后每步掉 2 ℃ 左右，掉到下一格又满功率，形成幅值几度、周期十几步的极限环。**时间平均精度高、瞬时精度差，这是所有“离散动作 + 积分对象”组合的共同特征**，也解释了为什么这类控制器交付时总要再叠一层 PWM 或占空比调制——把极限环抹平成连续量。

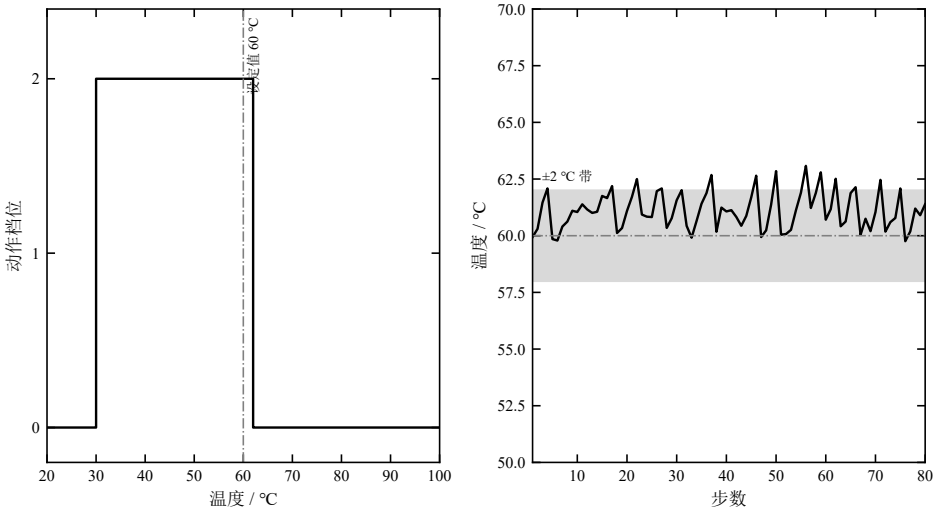


图 32-5　温度调节学到的开关策略与一条评估轨迹

图 32-5 左侧是策略随温度的分段形状，中间那段平台就是满功率区；右侧一条评估轨迹能直接看到锯齿，以及 ± 2 ℃ 的容差带。注意轨迹在 60 ℃ 上下来回穿越的次数远多于“每格一次”，这正是格子内切换的证据。

第三条结论：71% 的覆盖率要把账算细。 21 个状态 $\times$ 3 个动作 $=$ 63 对，71% 意味着约 45 对被访问、18 对没被访问。没被访问的里面，有 15 对（84–100 ℃ 共 5 格）是**结构上不可达**的：满功率的平衡温度只有 68 ℃，随机初温上限 80 ℃，这五格永远进不去。**也就是说真正的“漏学”只有三对**，而不是表面上的 29%。

这个区分很重要。**覆盖率是个会计指标，先要扣掉“物理上到不了的格子”，剩下的才是“该学没学到”的格子。** 前者不用管（部署也到不了），后者才是隐患：比如从 72 ℃ 冷启动时，如果那一格的三个动作都没试过，贪心会在 $Q = 0$ 的平地上任选一个，可能选成满功率往上冲。工程上的补法有两个：给这些格子手工填安全动作（这里是 $u = 0$），或者用 §32.4.3 的乐观初始化把它们抬高，让策略主动避开。**越是异常工况的格子越没有数据，而恰恰是异常工况才需要策略可靠**——这是表格型 RL 部署时绕不过去的一条。

32.6.6 E6：奖励是 100 量级时，基线不是可选项

五臂老虎机，均值 $[100.0, 100.5, 101.2, 100.2, 99.8]$，每次拉动加 $\mathcal{N}(0, 1)$ 噪声，最优臂是第三臂，它与次优臂只差 0.7，比噪声还小。用 REINFORCE 跑 2000 步、$\alpha = 0.05$、20 个种子，比较“减基线”与“不减基线”两种写法。

表 32-9　基线与乘子量级（2000 步，20 个种子中位）

写法	选中最优臂的概率	乘子均方根
不减基线（$b = 0$）	0.000	100.37
减常数基线（$b = 100.14$）	0.991	1.11

差值 90 倍，结果从“完全学不到”变成“几乎必中”。

原因与 §32.3.6 的推导完全对得上。梯度估计是 $(r - b)\nabla_\theta \ln \pi$，乘子就是 $(r - b)$。**不减基线时乘子的量级等于奖励本身，约 100；减掉奖励的均值后，乘子只剩下“这一臂比平均好多少”，量级等于噪声，约 1.1。** 同一个 $\alpha = 0.05$，在前一种写法里相当于把有效步长放大了 90 倍：θ 在最初几十步就冲到 10^3 量级，softmax 早已饱和，此后再也拉不回来。饱和到哪一臂完全由最早的几次噪声决定，与真实均值无关——所以 $P(\text{最优}) = 0.000$ 不是“学得慢”，是“学错了且锁死”。

这里必须说清一个容易误读的地方：**基线不改变梯度的期望**（§32.3.6 已证 $\mathrm{E}[\nabla \ln \pi \cdot b] = 0$），所以它不改变算法在 $\alpha \to 0$ 时的极限行为。它改变的是**有限步长下的有效步长**。换句话说，无基线 + 大奖励的失败，等价于“α 设大了 90 倍”；把 α 手动改成 5×10^{-4} 也能救回来。基线之所以值得单独讲，是因为它**自动完成这个匹配**，不需要你先知道奖励的量级。

由此得出一条可以直接写进规范的工程约定：**策略梯度类算法的输入奖励必须先标准化**——减均值、除以标准差，或者至少减掉一个量级正确的基线。不做这一步，超参数 α 就只能靠试；做了这一步，α 才有一个跨任务可迁移的合适范围（常见 10^{-3}–10^{-1}）。第 33 章的 Actor-Critic 里，评论家输出的 $V(s)$ 起的就是这个作用，它是“自动基线”。

32.6.7 E7：塑形把“能不能走”和“走得好不好”分开了

5×5 网格，起点左上 $(0,0)$、目标右下 $(4,4)$，四动作，越界原地不动，$\gamma=0.9$。三种奖励写法：

- **稀疏**：每步 0，进入目标 $+10$；
- **势函数**：稀疏奖励 $+\ \gamma\Phi(s')-\Phi(s)$，$\Phi(s)=-0.5\,d(s)$，$d$ 为到目标的曼哈顿距离；
- **朴素**：稀疏奖励 $+\ 3$，条件是这一步让 d 变小。

每组先用值迭代求精确 V，看它的贪心路径；再做 600 回合 Q-learning、8 个种子，测“训练末段成功率”与“纯贪心到达率”。

表 32-10　三种奖励写法下的值与行为

写法	$V(\text{起点})$	贪心步数	贪心到达	纯贪心到达率	训练末段成功率
稀疏	4.783	8	是	0%	0%
势函数	8.783	8	是	100%	100%
朴素	22.448	60	否	0%	30%

先看第一列能不能算出来。稀疏奖励下从起点到目标正好 8 步，$+10$ 在第 8 步收到，折扣 7 次：$10\times 0.9^{7}=4.783$。**逐位对上。** 势函数那一行是 $8.783=4.783+4.0$，而 $\Phi(\text{起点})=-0.5\times 8=-4$，恰好

$$V_{\text{pot}}(s)=V_{\text{sparse}}(s)-\Phi(s) \tag{32-48}$$

这正是 §32.5.1 那条 $Q'=Q-\Phi$ 的数值证据。**值整体平移了 4，策略一步没变**（都是 8 步）。所以塑形后 V 的绝对值不能跨写法比较，只有策略可比——这一点在调参时最容易被忽略：看到塑形后 V 变大就以为“学得更好了”，其实只是把势函数加了进去。

第二列是本章最想强调的一处对照。**三行的“贪心路径”与“纯贪心到达率”给出了两种完全不同的判断。** 稀疏奖励下，值迭代的 V 是精确的，贪心 8 步能到；但 Q-learning 学出来的表只覆盖了 5×5 里很小一部分（600 回合、每回合最多 60 步，且大部分回合在乱走），未访问格子的 Q 全是 0，纯贪心一进这些格子就原地打转，到达率 0%。势函数把“离目标多远”编进了每一步的奖励，于是**每个格子都有非零信号**，600 回合就能把整张表填到可用，到达率 100%。

第三行说明塑形的另一半风险。朴素的 $+3$ 不是势函数形式，它奖励“这一步靠近了”。问题在于**这个奖励可以被套利**：远离一步拿 0，再靠近一步拿 $+3$，往返两格的净收益是 $+3$，而折扣 0.9 只把它打折一次。于是“靠近—远离—再靠近”的两步循环每转一圈净赚约 2.7，还可以无限转下去，折现后总值 $2.7/(1-0.81)\approx 14.2$；而老老实实走到目标，拿到手的只有折现后的 4.783。**绕圈是更优解，算法没做错。** 结果正是如此：$V(\text{起点})=22.448$，比目标奖励本身还高一倍多，贪心走 60 步也到不了目标。**值的虚高不是估计误差，是任务被改写了**：算法忠实地最大化了我们给的奖励，而那个奖励的最优点是一段循环，不在目标上。

这也是“势函数形式”为什么必须写成 $\gamma\Phi(s')-\Phi(s)$ 而不是 $\Phi(s')-\Phi(s)$：只有带折扣因子的那一版，沿任意闭合路径求和才恰好为零，套利空间才被堵死。**差一个 γ，塑形就从“不改最优策略”变成“制造新的最优策略”。**

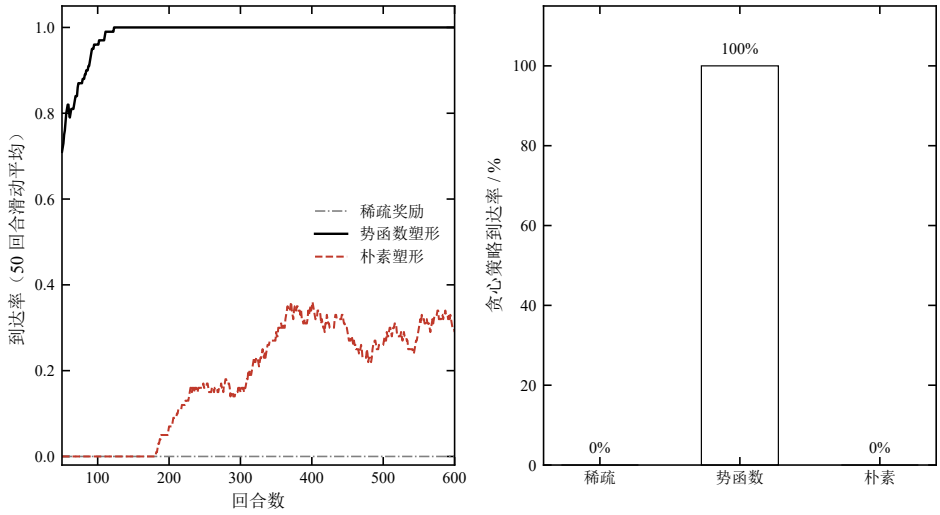


图 32-6　三种奖励写法下的学习曲线与纯贪心到达率

图 32-6 左侧是训练成功率随回合的曲线，右侧是纯贪心到达率的柱状对照。右侧三根柱子（0%、100%、0%）比左侧更有说服力：**训练时的成功率会被探索掩盖**——朴素写法在训练末段还有 30%，因为 ε 让它偶尔撞进目标；一旦去掉探索，绕圈的本性立刻暴露。

三行合起来给出一条判据：**看塑形对不对，不要看训练回报，要看“去掉探索之后策略做什么”。** 势函数形式（$F=\gamma\Phi(s')-\Phi(s)$）是唯一有理论保证“不改最优策略”的一类，因为它的折扣和正好 telescopes 掉；其他任何形式的中间奖励都可能引入新的最优点。若一定要用非势函数形式的启发项，安全的做法是把它加在**探索**上（如计数奖励、内在激励），而不是加在**回报**上。

32.6.8 七组实验合起来说明了什么

表 32-11　七组实验一览

编号	对象	关键数字	对应结论	图
E1	两状态反例	评估 3 次、改动 2 次；V 从 -36.842 到 15.872	策略迭代逐步不变差即可	—
E2	21 状态链	$34/340$ 轮；误差放大 $8.65/98.75$，界 $9/99$	停机判据低估误差，放大因子 $\gamma/(1-\gamma)$	32-2
E3	5 状态随机游走	MC 0.068 vs TD 0.252；目标标准差 0.240 vs 0.087	方差小不等于误差小，偏差另有来源	32-3
E4	悬崖行走	13 步/-54.4 与 17 步/-30.8	最优策略不等于最优在线行为	32-4
E5	一阶热对象	MAE 1.123 ℃、覆盖 71%、开关律	被支配动作会被筛掉；离散化下有极限环	32-5
E6	五臂老虎机	0.000 vs 0.991；乘子 100.37 vs 1.11	基线决定有效步长，奖励必须先标准化	—
E7	5×5 网格	贪心到达率 $0\%/100\%/0\%$	势函数塑形不改策略，朴素塑形改写任务	32-6

七组里没有一组是“证明某个算法更好”。它们各自钉住一个**判据**：E1 钉住“收敛的定义”（不变差，而非朝最优），E2 钉住“停机判据与误差的口径差”，E3 钉住“方差与偏差必须分开报”，E4 钉住“训练行为与部署行为必须分开测”，E5 钉住“离散动作 + 积分对象的极限环”，E6 钉住“奖励量级与步长的耦合”，E7 钉住“塑形后的策略而非回报”。**这七条恰好覆盖了 §32.5 那张否决表之外的全部常见坑。**

复现方式（在 `book` 目录下，解释器任选，实测总耗时约 17 s 出数、20 s 出图）：

```
python tools\check_chapter.py 32
python code\ch032_sim.py
python code\ch032_figs.py
python tools\build.py --chapter 32
```

四条命令的顺序不能颠倒：`check_chapter.py` 先查公式编号、图表引用与字数，`ch032_sim.py` 打印全部数字，`ch032_figs.py` 现场调用 `sim.exp_*()` 重算并出图（不读缓存，保证图与正文同源），最后才编译。**若 `ch032_figs.py` 改成读缓存，正文与图就会在后续改动中悄悄脱钩**，这是长文档项目里最常见的一类不一致。

最后要说清这七组实验**没有**覆盖什么：全部是表格型、小状态空间、纯仿真，不涉及函数逼近的泛化误差（第 33 章）、不涉及多智能体的非平稳性（第 34、35 章）、不涉及执行器延迟与采样抖动（这会让 Markov 性失效，见 §32.3.8）、也不涉及任何真实硬件。**本章的结论只在“状态空间可枚举、环境可反复重置”这个范围内成立**；越过这个范围，每一条都要重新验证。

32.6.9 同一份代码，数字能差多少

上面七组实验里，几乎每个数字后面都跟着三个括号：种子数、评估口径、回合数。这不是形式主义——**去掉任何一个，同一个策略可以报出互相矛盾的性能。**

口径一：评估时探不探索。 表 32-8 是最好的例子。Q-learning 那一行，在线回报 -54.4（训练时带 $\varepsilon = 0.1$ 的随机动作），部署回报 -13（纯贪心执行）。**同一个 Q 表，两个数差 41.4 分，后者是前者的 24%。** 如果报告里只写“回报 -54.4”，读者会认为这个策略很差；只写“-13”，则掩盖了训练期的高风险。SARSA 的两个数是 -30.8 与 -17，差别小得多——因为它学的本来就是“带探索的策略”。**报数字时必须写明“评估策略与训练策略是否相同”，这比数字本身重要。**

口径二：路径长度与行为安全性。 Q-learning 的贪心路径 13 步、SARSA 17 步，表面上 Q-learning 更优。但 13 步是贴着悬崖走的，一旦执行时有任何扰动（传感器噪声、执行器延迟），掉下去的概率远高于绕行的 17 步。**“更短的路径”与“更稳的路径”是两个指标，不能合成一个数报。**

口径三：种子与统计量。 表 32-7 报的是 20 个种子的中位 RMSE。为什么不用均值？因为 RMSE 的分布在种子上是右偏的：个别种子会停在很差的解上，把均值抬高一大截，而中位数不受影响。**报告规范是“中位数 + 四分位距 + 最差种子”，三者缺一不可**：中位给典型表现，四分位距给稳定性，最差种子给风险。只报均值等于把这三件事糊成一个数。

口径四：初值与步长共同决定偏差地板。 E3 里 TD 的 RMSE 停在 0.16–0.25 一线不再下降，原因有两个：初值全零（真值在 0.07–0.69 之间），以及常数步长 0.05 留下的稳态偏差。**这两个因素耦合在一起**：把步长减半，地板会降低，但收敛变慢；把初值设成真值的粗估计，地板也会降低，但那就等于用了先验。**看到“曲线不降了”，先分清是“还没收敛”还是“已经到地板”，再决定加回合还是改步长。**

口径五：随机流要分开。 环境重置、动作采样、数据打乱、网络初始化，这四件事各用一条独立的随机流（§32.4.5）。混成一条的后果是：改一个超参数，所有随机性都跟着变，实验之间不可比。**这与控制实验里“扰动源要可复现”是同一条纪律。**

给一个可执行的报告格式。每个关键数字后面写清四项：**（种子数，评估口径，评估回合数，统计量）**，例如“13 步（20 种子中位，纯贪心，单回合）”。四项目前都写全的论文不多，但正因为不多，写全的那些才可信。

最后一个对照值得记住：控制实验的重复性好，是因为对象确定、扰动固定、初值固定，重复一次就是同一个数；RL 实验的“重复性”只能靠统计——同一份代码跑两次，数字本来就不该一样。**所以 RL 的实验报告不该追求“跑出同一个数”，而该追求“跑出的分布能被别人复现”。** 这两件事的验收方式完全不同：前者看小数点后几位，后者看中位数、分位带与最差样本。

按这个标准回看本章七组实验：E1 每步改动都报了三轮评估的原始值，E2 报的是“轮数 + 停机口径 + 放大倍数”，E3 与 E5 报的是 20 个种子的中位，E4 同时给了在线与部署两个口径，E7 报的是 8 个种子。**规格不统一，是因为各自的随机性来源不同——把来源写清楚，比强行统一格式有用。**

32.7 小结与展望

32.7.1 三句话把本章收住

第一句：强化学习换的不是优化器，是信息结构。 第 30 章的 LQR 与 MPC 拿得到 (A, B)，第 31 章能离线调一次目标函数拿回一个数，本章两样都没有，只有一条“状态—动作—奖励”的样本流。这个差别可以写成同一个问题的三种信息集：

$$\pi^\star = \arg\min_{\pi \in \Pi} \; \mathrm{E}_\pi\Big[\sum_{k=0}^{\infty} \gamma^k c(s_k, a_k)\Big], \qquad \mathcal{I} \in \{\text{模型},\ \text{可离线评价},\ \text{只能在线采样}\} \tag{32-49}$$

目标函数一个字都没变，变的是 $\mathcal{I}$。**所有工程差异——样本效率成为核心指标、探索变成独立问题、训练好不等于部署稳——都是 $\mathcal{I}$ 从“模型”退到“样本”的代价。** §32.1.3 那个两状态反例是最好的注脚：同样的对象、同样的目标，只因为“不知道 P”，求解方式就从解一个线性方程变成了一轮轮“评估—改进”。

第二句：本章所有算法都是同一个 Bellman 算子的不同记账方式。 值迭代直接迭代算子，策略迭代先解线性方程再取 $\arg\max$，MC 用整条回报替代期望，TD 用一步自举替代期望，Q-learning 把 $\max$ 塞进目标，SARSA 保留实际动作，策略梯度绕开值函数直接对 $\log\pi$ 求导，Actor-Critic 用评论家给策略梯度当基线。**八种写法，一个算子。** 表 32-12 把它们摆在一起，差别只在“用什么替代期望”和“偏差与方差各付多少”。

表 32-12　八种算法的记账方式对照

算法	用什么替代期望	偏差来源	方差水平	每步数据要求	典型失效
值迭代	精确枚举全部动作	停机阈值	无	需要 P、r	状态多时不可行
策略迭代	精确解线性方程	无（每轮精确）	无	需要 P、r	每轮 $O(\lvert S\rvert^3)$
蒙特卡洛	一条完整回报	无（无偏）	高，随视野指数增长	必须等回合结束	回合长、无终止任务
TD(0)	一步奖励 + 自举	初值 + 步长地板	低	每步可更新	常数步长下不收敛到真值
Q-learning	一步奖励 + $\max$	最大化偏差	中	可离线复用	探索不足时高估
SARSA	一步奖励 + 实际动作	跟随当前策略	中	on-policy	策略变差时值也变差
REINFORCE	采样回报乘 $\nabla\ln\pi$	无（无偏）	极高	需要整条回合	奖励量级大时饱和
Actor-Critic	评论家估计的 V 或 Q	评论家偏差	中	每步可更新	双时间尺度没拉开

第三句：保证跟着假设走，换一层假设就要重算一次。 表 32-12 里每个“典型失效”都不是实现 bug，而是某条假设被违反了：Q-learning 的收敛要求每个状态—动作对被无限次访问（§32.3.5），TD 的收敛要求步长满足 Robbins–Monro（§32.3.4），策略梯度的无偏性要求轨迹按当前策略采样（§32.3.6）。**在控制工程里，这等价于“稳定性结论只在建模假设内成立”**——没人会拿一个在标称点证明的 Lyapunov 函数去保证大范围稳定，同理也不该拿表格型 RL 的收敛定理去担保神经网络策略。

32.7.2 一页纸选型

把 §32.5 的否决表与 §32.6 的判据合起来，选型可以压成四个问题，按顺序问：

1. **模型可靠吗？** 可靠就用 LQR / MPC（第 30、58 章），别往下问。
2. **能离线反复评价吗？** 能就用第 31 章的贝叶斯优化或直接搜索，样本效率高一个量级。
3. **状态能枚举吗（$\lvert S\rvert\lvert A\rvert$ 在预算内）？** 能就用表格型：回合短、要无偏选 MC；要在线更新选 TD；需要复用旧数据选 Q-learning；上线后探索不停选 SARSA。
4. **状态枚举不了、且动作连续？** 才进入第 33 章的函数逼近与策略梯度，并同时准备三件事：奖励标准化（E6）、评估口径分离（E7）、安全兜底（§32.5.3）。

四个问题的顺序不能打乱。**实际项目里最常见的浪费，是在第 1 问就能用 MPC 解决的问题上直接上了 RL**，然后花几个月调样本效率——而那个问题的模型其实早就有了，只是没人去辨识。

反过来，也有一种情形必须用 RL 且只能用 RL：**目标写不成解析式，但能对行为打分**。比如“这段轨迹看起来像老师傅的操作”、“这个路口在一个月内的通行体验更好”，这类目标没有二次型、没有梯度、甚至没有明确的中间指标，只有终局评价。此时 §32.5.1 的奖励设计就是全部工作的重心，而算法选择反而是次要的。

32.7.3 十条常见错误

下面十条都对应本章某一节的推导或某一组实验，每条给出“错误说法—为什么错—怎么做”。

一、把 γ 当无量纲超参数调。 错在 $1/(1-\gamma)$ 是**步数**，换成时间要乘 T_s（式 32-3）。$\gamma=0.99$ 在 $T_s=1$ s 时看未来 100 s，在 $T_s=0.01$ s 时只看 1 s。做法：先由慢模态时间常数定 H_{eff}，再反解 $\gamma\approx e^{-T_s/\tau}$。

二、拿训练回报上升当收敛。 错在训练回报里混着探索的收益（E7：朴素塑形训练末段成功率 30%，纯贪心 0%）。做法：训练与部署分开测，部署一律用纯贪心，并单独报“去掉探索后的行为”。

三、拿停机判据当误差界。 错在相邻差 $<\varepsilon$ 只保证误差 $<\gamma\varepsilon/(1-\gamma)$（§32.3.2）。E2 实测放大 8.65 倍与 98.75 倍，恰好贴着理论界 9 与 99。做法：要么把阈值除以 $\gamma/(1-\gamma)$，要么用两倍相邻差做后验估计。

四、以为奖励全负就学不动。 错在零初始化对全负奖励天然乐观（§32.4.2），反而有利于探索；真正需要改的是全正奖励（应初始化为 $R_{\max}/(1-\gamma)$）。做法：先看奖励符号，再决定初值。

五、**奖励不标准化就上策略梯度。** 错在步长与奖励量级耦合（E6：乘子 100.37 对 1.11，有效步长差 90 倍，$P(\text{最优})$ 从 0.991 掉到 0.000）。做法：减均值、除标准差，或至少减一个量级正确的基线。

六、**ε 衰减到零，却仍按 on-policy 口径报结果。** 错在 $\varepsilon \to 0$ 后 SARSA 与 Q-learning 学的是同一个极限，但训练期行为完全不同（E4：13 步/-54.4 与 17 步/-30.8）。做法：报告里必须写清 ε 的最终值和部署时的执行方式。

七、**认为 TD 一定优于 MC。** 错在“TD 方差小”与“TD 误差小”是两件事（E3：目标标准差 0.087 对 0.240，但 RMSE 是 0.25 对 0.07）。做法：方差与偏差分开报，并注明步长与初值。

八、**用非势函数形式塑形。** 错在中间奖励可被套利（E7：朴素奖励下 $V(\text{起点}) = 22.448$，是目标奖励折现值 4.783 的 4.7 倍，贪心开始绕圈）。做法：只用 $\gamma\Phi(s') - \Phi(s)$；若必须用启发项，加在探索上而非回报上。

九、**状态漏了关键量，Markov 性被破坏。** 错在 Bellman 方程的前提是 s 含全部历史信息（§32.2.2）。纯延迟对象只给 y_t 不够，必须扩展成 (y_t, y_{t-1}, y_{t-2})。做法：建模前先做“状态完备性”检查——把历史再往前加一步，最优策略会不会变？不会，才算够。

十、**把“期望约束满足”当“每条轨迹安全”。** 错在 CMDP 只约束期望代价（§32.5.3），而事故是小概率大后果。做法：期望约束之外必须叠逐点手段——动作屏蔽、监督器、联锁，或者用控制屏障函数做安全滤波。

十条里有六条跟“口径”有关（二、三、五、六、七、十），只有四条跟“算法”有关。**这个比例本身就是本章的结论：RL 的工程难度主要在定义与度量，不在推导。**

32.7.4 交付前的八项自检

给出一份可以逐条打勾的清单，按重要性排序：

1. **状态完备性**：状态里是否含了所有影响下一步的量？执行器指令、上一拍误差、未观测扰动是否遗漏？
2. **γ 与时间的换算**：T_s、慢模态 τ、H_{eff} 三者是否自洽？写进报告的是不是一个无量纲的 0.99？
3. **奖励可套利性**：是否存在“往返一次净收益为正”的循环？用 §32.6.7 的办法构造一个两步循环算一遍。
4. **可达性与覆盖率**：把物理上到不了的格子先扣掉，剩下没访问过的有哪些？异常工况的格子是否有人工兜底？
5. **评估口径**：训练与部署是否分开？部署策略是纯贪心还是带探索？报的是哪一种？
6. **统计充分性**：种子数是否不少于 20？报的是中位还是均值？种子是否用 `SeedSequence` 派生（§32.4.5）？
7. **安全兜底**：动作屏蔽表是否覆盖了全部越界与联锁条件？屏蔽后策略是否仍能回到正常区？
8. **可复现性**：一条命令能否重跑出正文全部数字？图是否现场重算而非读缓存？

八项里任何一项答不上来，结论就只能写成“在仿真中观察到”，不能写成“该方法有效”。

32.7.5 本章刻意没做的三件事

写清边界比写清结论更重要，这里列出三处取舍。

其一，只讲表格，不讲函数逼近。 表 32-12 里所有算法在状态数上都是指数爆炸的（§32.2.6 的预算式 32-20）。真实问题动辄 10^6 维状态，表格型一律不可用。第 33 章用线性特征与神经网络替换表格，同时要付两笔新账：泛化误差（相近状态的值互相干扰）与“致命三要素”（自举、异策略、函数逼近同时出现时可能发散）。**本章的收敛定理在第 33 章基本全部失效**，这是必须提前说清的。

其二，只讲状态扩展，不讲信念状态求解。 部分可观测（POMDP）的严格解法是在信念空间上做动态规划，维度是单纯形，比原状态空间更大。本章只给了工程上的近似——把最近几拍观测拼进状态（§32.2.2 的纯延迟例子）。**这个近似在延迟固定时有效，在延迟随机时会退化成“用错状态”**，那时需要的是滤波器（第 56 章）而不是拼历史。

其三，不讲连续动作的策略梯度变体。 DDPG、PPO、SAC 这些方法的核心是“如何在高维连续动作上估计 $\nabla_a Q$”与“如何限制策略更新幅度”，属于第 33 章的范畴。本章的 REINFORCE 只用来演示“基线为什么必要”（E6），不承担工程可用性的责任。

32.7.6 展望：三个方向与一条融合路线

方向一：从表格到函数逼近。 这是第 33 章的主题。要点不是“换成神经网络”，而是**换掉之后每一处保证都要重新建立**：值迭代的压缩性还在（算子没变），但“每个状态被访问无穷次”变成了“数据分布覆盖状态空间”，后者无法验证。所以第 33 章的算法几乎全部靠工程手段（目标网络、经验回放、梯度裁剪）换稳定性，而不是靠定理。

方向二：从在线试错到离线数据。 工业现场最不缺的是历史数据，最缺的是试错许可。离线 RL 只用固定数据集学策略，代价是“分布外动作的值估不准”——数据里没做过的动作，Q 值再高也不可信。这直接对应控制里的“外推不可信”：**用一段工况数据辨识出的模型，不能拿去做大幅机动。** 两者的处理方式也类似——限制新策略与数据分布的距离（对应控制里的“在工作点附近线性化”）。

方向三：从期望安全到逐点安全。 CMDP 给的是期望约束，事故要的是逐点约束。当前两条主流路线：一是把 RL 输出送进一个安全滤波器，用控制屏障函数把动作投影到安全集内（只改动作、不改策略，因此可以与任何算法组合）；二是把安全集直接编进动作空间（动作屏蔽），代价是可能把最优解也屏蔽掉。**前者保安全不保最优，后者保可行不保平滑**，工程上通常两个都上。

一条融合路线：分层。 把 RL 放在慢回路（分钟级的设定值、调度、配方），把 MPC 放在快回路（秒级的跟踪）。理由是硬的：RL 的样本需求随 $1/(1-\gamma)$ 与状态数增长，只有慢回路的时间尺度承受得起；而快回路要求逐点约束与毫秒级响应，正是 MPC 的强项。**这不是折中，是分工**——RL 负责“目标写不成解析式”的那一层，MPC 负责“约束必须逐点满足”的那一层，中间用设定值衔接。本书第 58 章会给这条路线一个完整的例子。

最后回到本章的定位。第 30 章回答了“模型已知时怎么最优”，第 31 章回答了“模型未知但可离线评价时怎么调参”，本章回答了“只能在线试的时候怎么学”。三者共用式 (32-49) 的同一个目标，差别只在信息集 $\mathcal{I}$。**判断该用哪一个，先问自己手上有什么，而不是先问哪个方法更时髦。**

第33章 深度强化学习与离线强化学习

33.1 动机与要解决的问题

33.1.1 第32章欠下的三笔账

第 32 章的所有结论都建立在一个前提上：状态与动作可以列成一张表。这个前提在教科书例子里成立，在工程对象上几乎立刻破产。三笔账要一起算。

第一笔是状态数。 一个六自由度机械臂，每个关节位置粗分 10 档，状态就是 10^6 个；每个状态配 7 种动作，Q 表就有 7×10^6 个格子。按 §32.2.6 的预算式，每对要访问上百次才能把估计压到可用精度，总共需要 7×10^8 步。以 1 kHz 采样、连续不停机计算，是 7×10^5 s，约 **8** 天——这还只是一个六轴臂、每轴只分 10 档的玩具精度。若再考虑速度维、力矩维、负载维，指数立刻翻到 10^{10} 以上，那时不是“训练慢”，而是“永远训不完”。

第二笔是动作的连续性。 阀门开度、电机转矩、舵面偏角都是连续量。表格型算法每一步要做 $\arg\max_a Q(s,a)$，连续动作下这是一个**内层优化问题**：外层学策略、内层求极值，代价随动作维数指数增长。第 32 章那些“三个档位”的对象是刻意简化的，真实执行器没有档位。

第三笔是样本的来源。 第 32 章的智能体可以自己在环境里试，试坏了重启。真机上不能重启：一次超温可能报废一炉料，一次碰撞可能损坏末端执行器。**当“试”本身有成本时，样本就从“免费资源”变成了“预算约束”**，而这恰恰是工业场景的常态。

三笔账对应本章的三块内容：**函数逼近**（用参数化模型替代表格，把 10^6 个格子压成几百个参数）、**策略梯度与演员—评论家**（绕开 $\arg\max$，直接对策略参数求导）、**离线强化学习**（只用历史数据，不再在线试错）。再加上工程上绕不开的 **sim-to-real 迁移**，构成四块。本章按这个顺序展开，§33.2 给定义，§33.3 给推导，§33.4 给实现，§33.5 给边界，§33.6 给七组实跑。

先把一件事说清楚：**“把表格换成神经网络”不等于“深度强化学习”**。真正的分界线不是模型容量，而是三条性质中同时出现了几条。下一小节专门讲这件事，因为它是本章所有不稳定现象的根源。

33.1.2 致命三要素：为什么加了神经网络就会发散

把表格换成一个参数化函数

$$V_\theta(s)\approx V^\pi(s),\qquad \theta\in\mathbb{R}^d,\ d\ll|S| \tag{33-1}$$

看起来只是“用更少的参数存同样的东西”。但表格有一个神经网络没有的性质：**改一个格子的值，不影响别的格子**。函数逼近一旦共享参数，改 $V(s_1)$ 就必然改动 $V(s_2)$——这正是它省参数的原因，也是它失稳的原因。

更准确地说，表格型 TD 的收敛性来自 Bellman 算子的 γ-压缩性（§32.3.2）。加了函数逼近之后，我们实际做的是“投影后的算子” ΠT，其中 Π 把任意函数投影到参数化函数族上。在**同策略**分布 $\mu=d_\pi$ 下，ΠT 仍是压缩的，压缩因子不超过 γ；一旦换成**异策略**分布 $\mu\neq d_\pi$，压缩性就要乘一个密度比因子

$$\|\Pi T V_1-\Pi T V_2\|_\mu\ \le\ \gamma\,\kappa\,\|V_1-V_2\|_\mu,\qquad \kappa=\Big\|\tfrac{d_\pi}{\mu}\Big\|_\mu^{1/2} \tag{33-2}$$

κ 是“目标策略的访问分布”与“实际采样分布”的失配度。数据越集中在少数状态，κ 越大；$\gamma\kappa>1$ 时压缩性消失，迭代变成扩张，值估计**指数发散**。Baird 的七状态反例就是构造了一组特征与一个异策略分布，使 $\gamma\kappa$ 明确大于 1，值在几千步内跑到任意大——而所有实现都是“照公式写的”，没有 bug。

这就是 Sutton 与 Barto 说的**致命三要素**（deadly triad）：**函数逼近、自举、异策略，三者同时出现时不保证收敛**。本章的算法几乎全部三者齐备，所以没有一个是“有收敛定理”的，全部靠工程手段换稳定。理解这一点，比记住 DQN 的网络结构重要得多。

顺带说明两条常见误解。**其一，“用神经网络所以不稳定”是错的**：线性函数逼近同样会发散（式 33-2 与网络容量无关），病根在 κ，不在非线性。**其二，“加大数据量就能救”也是错的**：发散是算子性质，数据再多只是让每一步的梯度更准，扩张的方向不变。真正的解法只有两条——**改分布**（让 μ 靠近 d_π，即经验回放与行为策略的选择）或**改算子**（把自举目标冻结，即目标网络）。DQN 的两个“技巧”正好各对应一条，§33.3 会把它们还原成这两个动作。

33.1.3 连续动作为什么必须换参数化

表格型的决策规则是“取最大的那个”。连续动作下这个动作没有定义，必须换一种写法。两条路：

第一条是继续对 Q 求极值，但用优化器。 每一步解一个 $\max_a Q_\theta(s,a)$，用梯度上升迭代十几次。问题是控制周期只有毫秒级：1 kHz 下每毫秒要解一个非凸优化，实时性直接崩掉。这条路在慢过程（分钟级配方优化）里可用，在快回路里不可用。

第二条是把策略本身参数化，令 $a=\mu_\phi(s)$，然后对 ϕ 求导。链式法则给出

$$\nabla_\phi J=\mathrm{E}_{s\sim\mu}\Big[\nabla_\phi\mu_\phi(s)^\top\,\nabla_a Q_\theta(s,a)\big|_{a=\mu_\phi(s)}\Big] \tag{33-3}$$

这就是确定性策略梯度（DPG）。它把“每步求极值”换成了“训练时算一次梯度”，在线只需前向一次网络——**这是实时性的关键交换**：把计算从部署期挪到训练期。

代价有两个。**其一，需要 $\nabla_a Q$**，即评论家必须对动作可微。这排除了查表型评论家和树模型，也是几乎所有连续控制算法都用神经网络的原因。**其二，梯度要穿过一个还在变的近似器：**Q_θ 自己每步都在更新，式 (33-3) 里的“梯度方向”因此也在动。这相当于用一个每步都换目标的优化器去优化策略——它是 DDPG 系列不稳定的直接来源，也是后面要引入目标网络与延迟更新的原因。

还有一处工程细节值得提前说：**确定性策略没有探索。**$\mu_\phi(s)$ 是一个点，不是分布，它永远不会主动试别的动作。所以 DDPG 必须在动作上加噪声（高斯或 OU 过程），而噪声的大小直接决定探索强度。**这把第 32 章那个独立的“探索”问题，变成了一个必须手调的噪声参数**——这是从表格型到连续控制最容易低估的一处退化。

33.1.4 离线：当“试”本身不被允许

第三笔账（样本来源）在工业场景里最硬。这里的信息结构再退一步：不是“能在线试但很贵”，而是**只能拿到一份固定的历史数据集**

$$\mathcal{D}=\{(s_k,a_k,r_k,s'_k)\}_{k=1}^{N},\qquad \text{产生它的策略 }\pi_\beta\text{ 未知且不再运行} \tag{33-4}$$

离线 RL 要在这份数据上直接学一个比 π_β 更好的策略，全程不与环境交互。听起来只是“把经验回放池换成一个只读文件”，实际难度完全不同。

根源是**分布外动作的值没有约束。**$Q_\theta(s,a)$ 只在“数据里出现过的 (s,a)”附近被训练过；对于数据里没有的动作，网络输出完全由参数化外推决定，而 $\arg\max$ **会主动去找那些被外推得最高的动作**。这是个系统性的乐观偏差：越是没有数据的动作，越容易被高估；越被高估，越会被选；越被选，越没有数据可以纠正它。学术上叫外推误差（extrapolation error），工程上就是“策略跑到数据覆盖不到的地方，然后崩掉”。

这个现象在控制里有一个精确的对应物：**用一段工况数据辨识出的模型，不能拿去做大幅机动。**线性化模型在工作点附近可信，出了范围就只是外推；离线 RL 的评论家也一样。两者的处理方式也同构——限制新策略与数据分布的距离，等价于“只在工作点附近线性化”。

所以离线 RL 的全部技术难点可以压成一句话：**如何在“只用固定数据”的约束下，仍然让 $\arg\max$ 不去碰那些没有数据支撑的动作。** 主流做法有三类：给 Q 加保守惩罚（压低分布外动作的值）、给策略加约束（限制与 π_β 的 KL 或支撑集距离）、用模型做保守外推（不确定性惩罚）。§33.3 会展开第一类，因为它最像控制里的“鲁棒惩罚”。

还有一个常被忽略的硬事实：**固定数据集能给的上界，由覆盖度决定，不由算法决定。** 若 π_β 从未在某类状态上动作过，任何“在这些状态上更好”的策略都无法被数据验证，也无法被学习，只能靠外推。**数据量再大，若都集中在同一工况，覆盖度仍然是零——覆盖度问的是“有没有”，不是“多不多”。** E5 正好给了这个事实一个侧面证据：把 a^2 补进特征，外推误差从 0.607 降到 0.028，而数据一条没加。所以工程上的第一件事不是选算法，而是回答三个问题：这批数据是谁在什么工况下采的？覆盖了哪些边界？缺失的部分能不能补采？补不了，就必须在报告里写明“本策略仅在某某工况内有效”——**这句话比任何指标都重要。**

33.1.5 与控制工程的三处对应

本章的概念对控制背景的读者并不陌生，只是换了名字。表 33-1 把三处对应摆出来。

表 33-1　从第32章到第33章：假设变了什么

维度	第 32 章（表格型）	第 33 章（深度 / 离线）	控制里的对应物
值函数表示	每状态一个数	$\theta^\top\phi(s)$ 或神经网络	基函数展开、神经网络辨识（第 22 章）
收敛保证	压缩映射，有定理	只有经验稳定，无定理	标称模型下的稳定性证明 vs 实际鲁棒性
探索	环境内试错	连续动作靠加噪；离线靠数据集覆盖	激励信号设计（第 16 章）
样本成本	免费、可重启	真机贵、离线只有一份	实验成本、批次间波动
失配来源	无（表格精确）	泛化误差、外推误差	模型误差、未建模动态
主要风险	探索不足	发散、过估计、分布偏移	失稳、约束越界

三处对应里最值得记住的是第二行。**第 32 章可以说“算法收敛”，第 33 章只能说“在哪些条件下没有观察到发散”。** 这不是本章写得保守，而是这一层的数学现状：致命三要素齐备时，收敛性问题是开放的，所有“稳定”都是工程经验，不是定理。控制工程里早就有同样的处境——一个在标称模型上证明稳定的控制器，装到真机上仍可能因为未建模的高频动态而振铃；区别只是控制里有 $\mathcal{H}_\infty$、μ 分析这类工具去量化鲁棒裕度，而 RL 目前还没有对应的、被广泛接受的量。

33.1.6 五个定量问题

本章要回答的不是“哪个算法最好”，而是五个可以量化的问题。每个问题都会在 §33.6 有对应的实跑数字。

问题一：泛化误差随什么变化？ 用 d 个参数拟合 V^π，误差可以拆成“近似误差”（d 太小，函数族装不下真值）与“估计误差”（样本有限，参数估不准）。后者大致按 $\sigma^2 d/m$ 下降（m 为样本数）。**所以要回答的是：给定精度要求，d 与 m 该怎么配？** 只加参数不加样本，误差先降后升，这条 U 形曲线是调参时最该盯住的。

问题二：目标网络该多慢？ 目标网络的更新周期（硬更新每 C 步，或软更新系数 τ）控制“自举目标漂移多快”。它必须与折扣视野 $1/(1-\gamma)$ 匹配：视野越长，目标越该稳。**要回答的是：τ 与 γ、与回合长度之间有没有可用的经验关系？**

问题三：PPO 的裁剪半径对应多大的策略变化？ 裁剪把概率比限制在 $[1-\epsilon, 1+\epsilon]$，但真正的策略变化量是 KL 散度。**要回答的是：$\epsilon = 0.2$ 在上千维动作空间里到底对应多大的 KL？** 这个换算决定了它是“温和更新”还是“几乎不更新”。

问题四：SAC 的温度系数在自动调节什么？ 熵目标 $\bar{\mathcal{H}}$ 决定探索强度，自动调节让 α 随任务难度自适应。**要回答的是：α 收敛到的量级与奖励尺度、动作维数是什么关系？** 说不清这个关系，换一个任务就得重调。

问题五：离线算法的保守程度怎么定？ 惩罚系数越大越安全，也越容易退化成“什么都不做”。**要回答的是：有没有一个可计算的量，能在离线阶段就预估部署性能的下界？** 这是离线 RL 能否进工业现场的关键，目前只有部分答案。

33.1.7 本章导览

后面六节按“定义—推导—实现—边界—验证—收束”走：

§33.2 给出函数逼近下的值函数、策略参数化、以及三个目标函数（TD 误差、策略梯度、熵正则）的统一写法。**§33.3** 推导 DQN 的两个稳定化手段为什么恰好对应“改分布”与“改算子”，DDPG 的梯度偏差，PPO 的裁剪与信赖域的关系，SAC 的温度对偶，以及离线算法（CQL 型）的惩罚项如何把分布外值压下去。**§33.4** 给实现细节：网络规模、回放池、目标网络更新、归一化、批量大小、梯度裁剪、评估协议。**§33.5** 讲边界：训练成本、仿真到现实的鸿沟、部署风险、以及本章不该用的场合。**§33.6** 七组实跑：线性逼近的误差分解、目标网络周期扫描、PPO 裁剪与 KL 的实测关系、SAC 温度的收敛、离线算法的外推误差、sim-to-real 的延迟鲁棒性、以及一次“训练看起来成功但部署失败”的复现。**§33.7** 收束成一张选型表与十条错误。

与相邻章的分工要划清：第 32 章给了表格型与全部基础定义，本章不再重复 Bellman 方程与 TD 的推导，只讲“加了函数逼近之后多了什么”；第 34、35 章讲多智能体，那时非平稳性会成为新的主要矛盾；第 36 章讲功能安全，本章末尾提到的“部署风险”在那里会有正式的分析方法。**本章的定位是：把第 32 章的方法搬到参数化表示上，并诚实交代搬过去之后丢了哪些保证。**

33.2 定义与数学基础

33.2.1 表格只是线性逼近的一个特例

先把“函数逼近”这件事的数学形式固定下来。最一般的一层写法是线性组合

$$V_\theta(s) = \theta^\top \phi(s) = \sum_{i=1}^{d} \theta_i \, \phi_i(s), \qquad Q_\theta(s,a) = \theta^\top \phi(s,a) \tag{33-5}$$

$\phi : \mathcal{S} \to \mathbb{R}^d$ 叫特征映射，θ 是可学参数。这个形式之所以值得单独讲，是因为**表格是它的一个特例**：取 $d = |\mathcal{S}|$、$\phi(s)$ 为 s 的 one-hot 向量，式 (33-5) 就退化成“每个状态存一个数”，θ 的第 i 个分量就是第 i 个格子的值。

这个观察改变了对“深度”的理解。**函数逼近不是新东西，它是把 one-hot 换成共享特征。** 表格的 one-hot 特征之间完全正交，所以改一个不影响另一个；换成共享特征后，不同的 s 会激活同一批 ϕ_i，于是改一个必然影响另一个。§33.1.2 说的失稳，根源就在这个“正交性丢失”，与参数是不是神经网络无关。

神经网络可以塞进同一个框架。把网络写成“特征提取器 + 线性读出”

$$f_\theta(s) = \underbrace{w^\top}_{1\times d} \; \underbrace{h_\psi(s)}_{\text{最后一层激活}} \tag{33-6}$$

其中 ψ 是前面所有层的权重。**任何前馈网络在最后一层都是线性的**，所以式 (33-5) 的结构在深层网络里依然成立，只是 $\phi = h_\psi$ 自己也在被训练。这个“线性读出”的视角在后面很有用：它解释了为什么网络的最后一层初始化尺度、以及最后一层的梯度量级，往往比中间层更敏感。

33.2.2 投影算子、TD 不动点与那个 $1/(1-\gamma)$

现在把 Bellman 算子搬到参数化函数族上。设采样分布为 μ（一个状态上的概率分布），D_μ 是以 μ 为对角的权重矩阵，Φ 是把所有状态的 $\phi(s)$ 堆成的 $|\mathcal{S}| \times d$ 矩阵。**加权最小二乘投影**定义为

$$\Pi_\mu = \Phi\big(\Phi^\top D_\mu \Phi\big)^{-1}\Phi^\top D_\mu \tag{33-7}$$

它把任意一个值向量投到“Φ 的列空间”里，且投的是 μ-加权欧氏距离下最近的那个点。

参数化 TD 的不动点满足 $\Pi_\mu T V_\theta = V_\theta$，等价于残差与特征空间正交：

$$\Phi^\top D_\mu\big(r_\pi + \gamma P_\pi \Phi\theta - \Phi\theta\big) = 0 \tag{33-8}$$

这是关于 θ 的**线性方程**，可以直接解出来

$$\theta^\star = \big(\Phi^\top D_\mu(\Phi - \gamma P_\pi \Phi)\big)^{-1}\Phi^\top D_\mu\, r_\pi \tag{33-9}$$

式 (33-9) 就是最小二乘时序差分（LSTD）。注意它与第 32 章策略评估的线性方程 $\mathbf{v} = (I-\gamma P_\pi)^{-1} r_\pi$ 是同一个东西：把 I 换成 Φ 的伪逆结构，维数从 $|\mathcal{S}|$ 降到 d。**状态多到解不动的时候，就把“精确解”换成“在 d 维子空间里最近解”**，这是函数逼近的全部内容。

代价是解不再精确。把 $V_\theta = \Pi_\mu T V_\theta$ 与 $V_\pi = T V_\pi$ 相减：

$$V_\theta - V_\pi = \Pi_\mu(T V_\theta - T V_\pi) + (\Pi_\mu V_\pi - V_\pi) \tag{33-10}$$

取 μ-范数，用 $\|\Pi_\mu\|_\mu = 1$ 与 T 的 γ-压缩性，得

$$\|V_\theta - V_\pi\|_\mu \;\le\; \frac{1}{1-\gamma}\,\|\Pi_\mu V_\pi - V_\pi\|_\mu \tag{33-11}$$

这个式子值得逐项读。右边第一因子是 $1/(1-\gamma)$，**与第 32 章值迭代的放大因子一模一样**（§32.3.2）；右边第二项是“真值离特征空间有多远”，叫投影残差，只取决于 ϕ 选得好不好，与样本量无关。所以

$$\text{总误差} \;\approx\; \underbrace{\frac{1}{1-\gamma}}_{\text{折扣放大}} \times \underbrace{\text{投影残差}}_{\text{表示能力}} + \underbrace{\text{估计误差}}_{\text{样本量}} \tag{33-12}$$

三项各有各的解法：第一项只能靠减小 γ（缩短视野）；第二项靠加特征、加宽度；第三项靠加数据。**调参时把这三项分开看，比笼统地说“模型不够好”有用得多。**

还有一处必须点明：式 (33-11) 是在 μ-范数下成立的，而我们真正关心的是策略自己的访问分布 d_π。两个范数之间的换算系数就是式 (33-2) 的 κ。**所以异策略的问题不是“投影错了”，而是“在错误的尺子上量误差”**——这也解释了为什么经验回放（让 μ 更接近历史策略的混合分布）能缓解问题，但消不掉它。

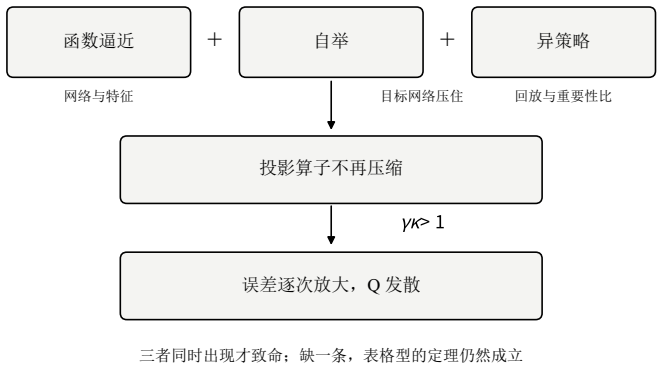


图 33-1　致命三要素与本章各算法的处置方式

图 33-1 把三要素画成一个三角形，并标出四个算法各自站在哪里。**PPO 之所以相对稳，不是因为它更先进，而是因为它主动放弃了异策略**：同策略采样使 $\kappa = 1$，三要素只中两条，压缩性保住了。DQN、SAC、CQL 都保留异策略（为了样本复用），所以必须额外付代价——DQN 用目标网络冻结自举目标，SAC 用双评论家与延迟更新压低过估计，CQL 直接给分布外动作加惩罚。**这张图是全章的索引：后面每讲一个算法，都先问它把三角形的哪条边拆掉了。**

33.2.3 三个目标函数，一个模板

本章所有算法优化的目标可以归成三类，形式不同但都是“对样本求期望再最小化”。

第一类：值误差。 最自然的是最小化 Bellman 残差的平方

$$J_V(\theta) = \tfrac{1}{2}\,\mathrm{E}_{s\sim\mu}\Big[\big(r + \gamma V_\theta(s') - V_\theta(s)\big)^2\Big] \tag{33-13}$$

但式 (33-13) 有一个陷阱：目标里含 $V_\theta(s')$，对它求全梯度会得到“θ 出现在两边”的方程，而且需要知道 (s, a, s') 的联合分布才能无偏估计。工程上用的都是**半梯度**：只对 $V_\theta(s)$ 求导，把 $r + \gamma V_\theta(s')$ 当常数。半梯度不是 J_V 的真梯度，但它是第 32 章 TD(0) 的严格推广，且计算只需一步样本。**“用半梯度”这件事本身就是一次有偏的取舍**，§33.3.1 会量化偏差有多大。

第二类：策略性能。 直接对回报求导

$$J_\pi(\phi) = \mathrm{E}_{\pi_\phi}\Big[\sum_{k\ge 0}\gamma^k r_k\Big], \qquad \nabla_\phi J_\pi = \mathrm{E}_{\pi_\phi}\Big[\sum_{k\ge 0}\gamma^k G_k\,\nabla_\phi \ln \pi_\phi(a_k \mid s_k)\Big] \tag{33-14}$$

这是第 32 章式 (32-35) 的原样搬运，本章不再推导。它不需要值函数，但方差极大（§32.6.6 已量化：奖励 100 量级时乘子差 90 倍），所以实际用的是“减基线”或“用评论家替代 G_k”的版本。

第三类：熵正则。 在回报里加一项策略熵

$$J_{\mathrm{ent}}(\pi) = \mathrm{E}_{\pi}\Big[\sum_{k\ge 0}\gamma^k\big(r_k + \alpha\,\mathcal{H}(\pi(\cdot \mid s_k))\big)\Big], \qquad \mathcal{H} = -\mathrm{E}_{a\sim\pi}\ln \pi(a \mid s) \tag{33-15}$$

α 是温度。这一项的作用是**在目标里显式给“多试几个动作”付钱**：熵越大，策略越分散，探索越充分。$\alpha \to 0$ 时退化成普通回报；$\alpha \to \infty$ 时策略趋于均匀随机。第 32 章的 ε-贪心是“外加的探索”，式 (33-15) 是“内生进目标的探索”——**前者需要手调衰减表，后者可以自动调节**（§33.3.4 会给出对偶推导）。这也是 SAC 比 DDPG 少调一个关键超参的原因。

三类目标的关系可以一句话概括：**第一类学“这个状态值多少”，第二类学“这个动作概率该多大”，第三类在第二类上加了一个“别太确定”的罚项。** 本章的 DQN 属第一类，PPO 属第二类，SAC 与 CQL 属第三类（CQL 在第三类上再加保守项）。

33.2.4 策略的参数化：连续动作必须处理有界性

离散动作的策略参数化很直接：$\pi_\theta(a \mid s) = \mathrm{softmax}(z_\theta(s))_a$。连续动作有四种常见写法，差别全在“如何处理动作边界”。

表 33-2　连续/离散策略参数化的四种写法

写法	形式	$\ln\pi$ 是否有闭式	探索	典型算法
离散 softmax	$\exp z_a / \sum_b \exp z_b$	有	内在	DQN（策略由 $\arg\max$ 导出）
对角高斯	$\mathcal{N}(\mu_\phi(s), \sigma_\phi^2(s))$	有	内在，σ 可学	PPO
压缩高斯	$a = \tanh u,\ u \sim \mathcal{N}(\mu, \sigma^2)$	有，需雅可比修正	内在，且自动有界	SAC、PPO（连续）
确定性	$a = \mu_\phi(s)$	不需要	无，靠外加噪声	DDPG、TD3

第三种是工程上最常用的，值得写出完整式子。令 $u \sim \mathcal{N}(\mu, \sigma^2)$、$a = \tanh u$，则

$$\ln \pi(a \mid s) = \ln \mathcal{N}\big(u; \mu, \sigma^2\big) - \sum_{i=1}^{m} \ln\big(1 - \tanh^2 u_i\big) \tag{33-16}$$

第二项是变量变换的雅可比行列式，**不能漏**。漏掉它的后果很具体：在 a 接近 ± 1 时 $\tanh$ 的导数趋于零，密度会被算成一个有限值而不是趋于无穷，策略梯度会把概率质量推向边界而自己不知道——表现是“训练后动作长期贴着限幅跑”。

为什么不直接用截断（把超界的动作夹到 $[-1,1]$）？因为截断会让多个不同的 u 映射到同一个 a，概率密度在边界处堆积成一个原子，$\ln\pi$ 无定义，梯度失效。$\tanh$ 压缩是**“可逆且有界”的最小代价方案**，这也是它几乎成了连续控制标配的原因。

33.2.5 离线数据的三个量：支撑集、覆盖度、外推距离

离线场景要额外定义三个量，否则“数据不够”这句话无法量化。

支撑集：给定数据集 $\mathcal{D}$，定义行为分布 μ_β 为 $\mathcal{D}$ 上的经验分布。状态 s 处动作的条件支撑集是“离某个数据动作足够近”的动作集合：

$$\operatorname{supp}\mu_\beta(\cdot\mid s)=\bigcup_{(s',a')\in\mathcal{D},\,\|s'-s\|<\delta_s}\big\{a:\ \|a-a'\|<\delta_a\big\} \tag{33-17}$$

通俗说就是“这个状态下，数据里见过的动作及其邻域”。**离线 RL 的一切问题都发生在支撑集之外。**

覆盖度：对每个 (s,a) 统计有效样本数 $n(s,a)$。n 小的地方，Q 的估计方差大；$n=0$ 的地方，Q 完全由外推决定。§32.6.5 那张覆盖率的账（扣掉物理上不可达的格子）在这里要重做一遍，因为现在的“格子”是连续空间里的一个邻域。

外推距离：$d_{\text{out}}(a)=\min_{a'\in\operatorname{supp}\mu_\beta(\cdot|s)}\|a-a'\|$。它是“这个动作离已知数据多远”的度量，也是惩罚项该加多少的依据。

这三个量合起来给出过估计的放大账。假设每步 Q 被高估 δ，沿 H 步累积，回报的高估量是

$$\delta\sum_{k=0}^{H-1}\gamma^k=\delta\,\frac{1-\gamma^H}{1-\gamma}\ \xrightarrow{H\to\infty}\ \frac{\delta}{1-\gamma} \tag{33-18}$$

$\gamma=0.99$ 时放大 100 倍。**这意味着每步 0.01 的微小高估，在 $\gamma=0.99$ 的策略里会变成 1.0 的回报虚高**，足以把最优动作换掉。式 (33-18) 是离线 RL 里所有保守惩罚的定价依据：惩罚不是“正则化技巧”，它是在抵消这个 $1/(1-\gamma)$ 的放大。

33.2.6 sim-to-real：三种“迁移”的数学区别

仿真到现实的差距，可以形式化成“环境参数不确定”。设环境由一个参数 ξ 决定（质量、摩擦系数、执行器延迟、传感器噪声），$\xi\in\Xi$，真实环境对应某个未知的 ξ_{real}。三种常见做法对应的优化问题不同：

$$\text{名义：}\ \max_\pi J(\pi;\xi_0),\qquad \text{域随机化：}\ \max_\pi \mathbb{E}_{\xi\sim p}\big[J(\pi;\xi)\big],\qquad \text{鲁棒：}\ \max_\pi\min_{\xi\in\Xi}J(\pi;\xi) \tag{33-19}$$

这三者不是一回事，混用会出事故。 名义优化只保证在 ξ_0 上最好，$\xi_{\text{real}}\neq\xi_0$ 时毫无保证；域随机化优化的是**平均**性能，它允许“在少数参数下很差”，只要平均拉得回来；鲁棒优化保证最坏情况，但代价是保守——在 ξ 的整个集合上取最坏，往往会把标称性能压低很多。

工程上还有一个更隐蔽的区别：**域随机化不覆盖“结构差异”，只覆盖“参数差异”。** 仿真里没有的效应（齿轮间隙、皮带迟滞、网络丢包）不会因为把摩擦系数随机化就出现。所以“随机化范围设得很宽”并不等于“迁移得好”，这一点在 §33.5.2 会用一个延迟的例子量化。

延迟是最典型的例子，也是最容易被忽略的。设回路带宽 100 Hz（$\omega=2\pi\times100$），纯延迟 1 ms，相位滞后是

$$\varphi=-\omega T_d=-0.628\ \text{rad}=-36^\circ \tag{33-20}$$

36° 已经吃掉了相位裕度的一大半。 仿真里如果没建模这 1 ms，策略在仿真里稳、上机就振——这不是“随机化没调好”，而是模型结构里少了一项。

33.2.7 与核方法、高斯过程的对照

线性逼近 $V_\theta=\theta^\top\phi(s)$ 里有两处人为选择：特征 ϕ 长什么样、维数 d 取多大。核方法与高斯过程恰好把这两处各换掉一个，对照一下能看清“深度”到底买到了什么。

核方法换掉的是“特征谁定”。 如果算法只通过内积用到特征（LSTD、岭回归都属于这一类），就可以把 $\phi(s)^\top\phi(s')$ 整体替换成核函数 $k(s,s')$，从不显式构造 ϕ。表示定理保证最优解落在样本张成的子空间里

$$V(s)=\sum_{i=1}^{m}\alpha_i\,k(s_i,s)$$

于是**特征维数可以是无穷，有效维数却只有样本数 m**。这句话与 §33.2.1 的“表格是 one-hot 特例”是同一件事的两端：表格的核是 δ 核（只有同一个格子内积为 1），核方法把 δ 换成光滑核，深度网络则把核本身也变成可学的。

高斯过程再换掉“d 取多大”。 把 θ 看成高斯先验、观测看成带噪线性读出，后验均值就是上面那个形式，而后验方差

$$u^2(s) = k(s,s) - k(s)^\top \big(K + \sigma^2 I\big)^{-1} k(s)$$

直接给出“这个状态没见过的程度”。这正是 §33.3.6 里保守惩罚想定价、却只能用一个常数 D 粗略替代的量。**如果 GP 的 $u(s,a)$ 可算，离线 RL 的惩罚系数就不必猜，直接取 $\beta\, u(s,a)$ 即可**——这是“乐观/保守”两类算法在理论上的原型。

代价也很清楚：训练要解 $m \times m$ 的线性系统，复杂度 $O(m^3)$，内存 $O(m^2)$，推理要遍历全部样本，$O(m)$。$m = 10^4$ 时一次求解已是 10^{12} 量级，$m = 10^6$ 完全不可行。深度网络把这笔账换成了“$O(d)$ 推理 + 一次性训练”，代价是丢掉了 u 这个不确定性量。

三条实用判据：

1. **样本少、维数低、要不确定性**——先用 GP 或核 LSTD 做基线。样本在 10^2–10^3、输入维数不超过 10 时，它们往往比网络更准、更可复现，且没有种子方差。
2. **样本多、观测高维、要实时推理**——只能用网络。此时核方法的两项代价（m^3 训练、m 推理）都无法接受。
3. **先问瓶颈在数据还是在表示。** E1 里 $m = 10^4$、$d = 4$ 的误差是 0.0258，再加特征也只到 0.0239——**这时换成网络是净浪费**；反过来 E5 里补一个 a^2 就把外推误差从 0.607 压到 0.028，说明瓶颈在表示，此时“换核”（补特征）比“加宽”有效得多。

一句话总结这三层的关系：**表格是 δ 核，核方法是固定核，深度网络是学出来的核；三层共用式（33-5）的线性读出，差别只在 ϕ 的自由度与样本效率。**

33.2.8 规模账：d 与 m 该怎么配

最后给一个可以拿笔算的规模关系，它决定了“网络该多大、数据该多少”。

参数化的估计误差大致按 $\sigma^2 d/m$ 下降（d 个参数、m 个样本、单样本噪声方差 σ^2），而近似误差随 d 增大而下降。两者相加是一个 U 形，最优点在两条曲线斜率相反的地方。**实用上只需要一个量级判断**：若要求均方误差不超过 ϵ^2，则

$$m \;\gtrsim\; \frac{\sigma^2 d}{\epsilon^2} \tag{33-21}$$

代入一组具体数字。取 $\sigma = 1$（奖励归一化后）、$\epsilon = 0.05$、$d = 10^4$（一个小型网络），得 $m \gtrsim 4 \times 10^6$ 步。以 $1\,\mathrm{kHz}$ 采样连续跑，是 $4000\,\mathrm{s}$，约 **67 分钟**——这就是“一个一万参数的网络、要到 5% 精度、需要一小时不间断数据”的量级估计（只保留量级，未计常数因子，工程上还要再乘几倍）。

对照第 32 章的表格：$d = |\mathcal{S}| = 10^6$ 时同样精度需要 $m \gtrsim 4 \times 10^8$ 步，是前者的 100 倍。**函数逼近省下的正是这 100 倍样本**，代价是式 (33-12) 里的投影残差与式 (33-2) 里的 κ。一笔换一笔，没有免费的午餐。

表 33-3 把本节新增的符号收在一起，后面各节直接引用。

表 33-3　本章符号与含义

符号	含义	与第 32 章的对应
$\phi(s)$	特征映射，d 维	表格的 one-hot（$d = \lvert\mathcal{S}\rvert$）
Π_μ	μ-加权最小二乘投影	无（表格下投影是恒等）
κ	分布失配度 $\lVert d_\pi/\mu\rVert_\mu^{1/2}$	无（表格下 $\kappa = 1$）
μ_β	行为策略/数据经验分布	第 32 章的 ε-贪心策略
d_{out}	动作到数据支撑集的距离	无
α	熵温度	第 32 章 ε 的连续版替代
ξ	环境参数	第 32 章固定的 P、r

33.2.9 闭式解与随机梯度：同一个不动点，两种算账

式（33-9）给出的是闭式解，而工程实现几乎都用随机梯度。两者收敛到**同一个投影 Bellman 不动点**，差别只在“用多少计算换多少样本”。

闭式解（LSTD）把 m 条样本一次性压成一个 $d \times d$ 矩阵求逆，复杂度 $O(md^2 + d^3)$，得到的估计方差约为 $\sigma^2 d/m$。随机梯度（半梯度 TD）每步只花 $O(d)$，但要对同一批数据扫描多遍，且步长带来一个“收敛不完”的残差地板。**换句话说：闭式解把误差全部押在样本量上，随机梯度把误差分一部分给步长。**

数值上有一个必须知道的量：矩阵 $\Phi^\top D_\mu(\Phi - \gamma P_\pi \Phi)$ 的条件数大约随 $1/(1-\gamma\kappa)$ 增长。$\gamma\kappa \to 1$ 时它趋于奇异，解对样本噪声极度敏感——**这就是“接近发散边界时 LSTD 也不可信”的原因**，与式（33-2）是同一件事的两个面。E1 里加的 10^{-6} 岭项只起数值稳定作用，它会把解往原点拉一点，属于偏差，不属于精度。

闭式解还有一个在线版本：用 Sherman–Morrison 公式递推求逆，每步 $O(d^2)$，不需要存全部样本。它在 μ 固定、特征固定时很好用；一旦特征也在学（网络），矩阵每步都变，递推就失去意义。

四条判断：

1. $d \le 10^3$ **且特征固定**——先用 LSTD 或核 LSTD 做基线。它没有步长、没有种子方差，能把“表示误差”与“优化误差”分开，E1 的误差分解正是靠这一点。
2. d **大到** 10^4 **以上**——d^3 已是 10^{12} 量级，闭式解不可行，只能随机梯度。
3. **目标在动**（自举目标依赖 θ）——闭式解每步都要重解，成本无法接受；半梯度虽然只近似，但每步便宜。
4. **要分解误差**——用闭式解；**要追非平稳目标**——用随机梯度。两者的取舍不是“谁更先进”，而是“你想把误差记在哪一栏”。

33.3 推导与性质

33.3.1 目标网络与经验回放各拆一条边

DQN 的损失函数是

$$L(\theta) = \mathrm{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s',a') - Q_\theta(s,a)\big)^2\Big] \tag{33-22}$$

注意目标里用的是 θ^- 而不是 θ。这个下标是全部稳定性的来源，值得单独解释。

如果不加 θ^-，目标 $y = r + \gamma \max_{a'} Q_\theta(s',a')$ 与待优化参数是同一套。此时梯度下降在追一个**自己也在动**的目标：θ 一更新，y 立刻跟着变，损失可能不降反升。更糟的是，这与式 (33-2) 的扩张条件叠加——自举让误差按 γ 放大，函数逼近让放大在特征空间里串扰，异策略让尺子错位，三者相乘。

加上 θ^- **之后**，在一段固定周期内目标是常数，问题退化成“用回归拟合一批固定标签”，这是一个凸的（线性情形）或至少是良定的监督学习问题，有唯一最优、误差单调下降。每隔 C 步把 θ 拷给 θ^-，再重新开始。**换句话说，目标网络把“追移动目标”改成了“分段追静止目标”**，每段的误差不发散，段间靠拷贝推进。软更新是同一思想的光滑版本

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-, \qquad \tau \ll 1 \tag{33-23}$$

它对应一阶低通，等效时间常数约 $1/\tau$ 步。$\tau = 0.005$ 相当于 200 步的滑动平均。

经验回放做的事不一样，它改的是分布。 不用回放时，样本按时间顺序产生，相邻样本高度相关（μ 集中在当前轨迹附近），且用完即弃。回放池做三件事：

1. **打破相关性。** 随机抽一批，样本近似独立，梯度方差下降。这一条是统计效率，与收敛性无关。
2. **提高复用率。** 一条样本被抽中多次，样本效率成倍提升。这一条是成本。
3. **拓宽** μ**。** 池子里存的是过去许多个策略产生的数据，其混合分布比“当前策略的访问分布”宽得多。**这直接减小式 (33-2) 的** κ**。** 这一条才是稳定性。

但要说清界限：回放让 μ 变宽，却**不能让** $\mu = d_\pi$。池子里永远混着旧策略的数据，κ 只是从“很大”降到“接近 1”，不等于 1。所以 DQN 仍然站在致命三要素的三角形里（图 33-1），只是把最尖的那个角磨钝了。**这就是为什么 DQN 在各种任务上“通常能跑”，但没有收敛定理。**

33.3.2 最大化偏差：n 个动作会被高估多少

第 32 章给过两动作时的数：$\mathrm{E}[\max] \approx 0.564\sigma$。一般地，若 n 个动作的 Q 估计各带独立同分布、标准差 σ 的零均值误差，则最大值的期望偏差是

$$\mathrm{E}\Big[\max_{1\le i\le n} \hat{Q}_i - \max_{1\le i\le n} Q_i\Big] \approx \sigma \cdot m_n, \qquad m_2 = 0.564,\ m_3 = 0.846,\ m_4 = 1.029,\ m_{10} = 1.539 \tag{33-24}$$

动作越多，偏差越大。这不是小量：**四动作时每步高估** 1.03σ**，十动作时** 1.54σ。再套式 (33-18) 的 $1/(1-\gamma)$ 放大，$\gamma = 0.99$ 时分别放大成 103σ 与 154σ。若 $\sigma = 0.1$（一个训练得还不错的评论家），回报虚高就是 10 到 15——足以让策略系统性地偏好“估计噪声大”的动作。

双重 Q 学习（Double DQN）把“选动作”和“评动作”拆开：

$$y = r + \gamma\, Q_{\theta^-}\Big(s',\ \arg\max_{a'} Q_{\theta}(s', a')\Big) \tag{33-25}$$

选动作用在线网络，评价值用目标网络。两个网络的误差若不相关，$\max$ 带来的正偏差就被大幅抵消。**代价是引入了一点负偏差**（因为选出的动作在目标网络眼里未必最好），但负偏差比正偏差安全：宁可低估一个好动作，也不要高估一个坏动作。

33.3.3 DDPG 的梯度偏差与 TD3 的三处修正

回到式 (33-3)。它的推导只用了链式法则，看起来无懈可击，但有一个隐含前提：$Q_\theta \approx Q^{\mu_\phi}$。实际训练中评论家总有误差 $\varepsilon(s,a)$，于是

$$\nabla_\phi J = \mathrm{E}\big[\nabla_\phi \mu_\phi^\top \nabla_a (Q^{\mu_\phi} + \varepsilon)\big] = \underbrace{\mathrm{E}\big[\nabla_\phi \mu_\phi^\top \nabla_a Q^{\mu_\phi}\big]}_{\text{想要的}} + \underbrace{\mathrm{E}\big[\nabla_\phi \mu_\phi^\top \nabla_a \varepsilon\big]}_{\text{偏差}} \tag{33-26}$$

关键在于第二项不是零：**只要 ε 随动作变化，它就会被策略“利用”**。更具体地说，函数逼近在数据稀疏处误差大，而 $\arg\max$ 恰好把策略推向那些地方——偏差项因此系统性为正，策略会主动钻进“评论家最不确定的角落”。这与 §33.2.5 的离线外推误差是同一个机制，只是发生在连续动作的邻域里。

TD3 用三处修改压这个偏差，每一处都对应式 (33-26) 里的一项：

其一，双评论家取最小。 训练两个独立的 Q，目标取 $\min(Q_1, Q_2)$。这与 Double DQN 同一动机，但更彻底：直接取小，把正偏差压成负偏差。

其二，目标策略平滑。 在目标动作上加截断噪声

$$a' = \mathrm{clip}\Big(\mu_{\phi^-}(s') + \mathrm{clip}(\epsilon, -c, c),\ a_{\mathrm{lo}}, a_{\mathrm{hi}}\Big), \qquad \epsilon \sim \mathcal{N}(0, \sigma^2) \tag{33-27}$$

它防止评论家在“某个动作上恰好有个窄峰”处过拟合——策略无法靠精准命中那个峰来刷分，因为目标被抹平了一个 c 宽的邻域。**这是对“$\nabla_a \varepsilon$ 在窄峰处最大”的直接反制。**

其三，延迟更新。 评论家更新若干次，演员才更新一次。理由是式 (33-26)：偏差项正比于评论家的误差，评论家没学好之前，演员怎么更新都是在追噪声。**先让尺子稳，再量东西。**

三处修改合起来，把 DDPG 从“经常崩”变成“通常能跑”，但仍然没有定理。**这就是本章的常态：所有“改进”都是在削偏差/削方差，不是在补证明。**

33.3.4 PPO：裁剪与 KL 的真实换算

PPO 的出发点是一个约束优化：希望每步策略变化不要太大，形式上写成 $\mathrm{E}[\mathrm{KL}(\pi_{\mathrm{old}} \| \pi_\theta)] \le \delta$。直接解这个约束需要算 Fisher 矩阵的逆（自然梯度），代价高；PPO 用一个更便宜的手段近似它——**裁剪重要性采样比**。

令

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)} \tag{33-28}$$

则裁剪目标为

$$L^{\mathrm{CLIP}}(\theta) = \mathrm{E}_t\Big[\ \min\big(r_t(\theta) A_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t\big)\Big] \tag{33-29}$$

它的作用可以一句话说清：**当优势为正时，把 r 的上行梯度截断在 $1+\epsilon$；当优势为负时，截断在 $1-\epsilon$。** 超过这个范围，再怎么提高概率也不给额外奖励，于是策略不会一步跨太远。

那么 $\epsilon = 0.2$ 到底对应多大的策略变化？对单个动作、r 在 $1 \pm \epsilon$ 内，KL 的二阶近似是

$$\mathrm{KL}\big(\pi_{\mathrm{old}} \,\|\, \pi_\theta\big) \approx \tfrac{1}{2}\mathrm{E}_{a\sim\pi_{\mathrm{old}}}\big[(r-1)^2\big] \le \tfrac{\epsilon^2}{2} \tag{33-30}$$

$\epsilon = 0.2$ 给每维 0.02 nats。这个数看着很小，但对 m 维独立动作，KL 是各维之和，上界变成 $0.02m$：$m = 6$ 时约 0.12 nats，$m = 20$ 时约 0.4 nats。**所以“$\epsilon = 0.2$”在低维任务里是温和更新，在高维任务里可能是很激进的更新**——这也是为什么高维任务常把 ϵ 调到 0.1 以下，或者干脆改回自适应 KL。

优势函数 A_t 的估计用 GAE（广义优势估计）：

$$\hat{A}_t = \sum_{l \ge 0} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{33-31}$$

λ 是一个偏差—方差旋钮：$\lambda = 0$ 退化成一步 TD 误差（偏差大、方差小），$\lambda = 1$ 退化成蒙特卡洛回报减基线（无偏、方差大）。工程上常用 0.95，含义是“有效视野约 $1/(1-\gamma\lambda)$”，$\gamma = 0.99$、$\lambda = 0.95$ 时约 20 步。**这个数与 γ 单独给出的 100 步不是一回事，报参时必须写清是哪一个。**

33.3.5 SAC：温度对偶与双评论家

最大熵目标 (33-15) 的最优策略有闭式。对固定 s，把 $\sum_a \pi(a \mid s)(Q(s,a) - \alpha \ln \pi(a \mid s))$ 对 π 求导并加单纯形约束，得

$$\pi^\star(a \mid s) = \frac{\exp\big(Q(s,a)/\alpha\big)}{\sum_{a'} \exp\big(Q(s,a')/\alpha\big)} \tag{33-32}$$

即“软最大”。α 越小越接近 $\arg\max$，越大越接近均匀。**注意这不是启发式，是目标函数 (33-15) 的精确解**——熵项把硬的 $\max$ 换成了软的 log-sum-exp，这也是 SAC 名字里“soft”的来历。

温度 α 可以自动定。约束写成“平均熵不低于目标 $\bar{\mathcal{H}}$”，即 $\mathrm{E}[-\ln \pi(a \mid s)] \ge \bar{\mathcal{H}}$，其对偶变量就是 α。对偶上升给出

$$\alpha \leftarrow \alpha \cdot \exp\Big(-\beta \big[\ln \pi(a \mid s) + \bar{\mathcal{H}} \big] \Big) \tag{33-33}$$

含义要按对偶的正负号读：熵低于目标时 $\ln\pi + \bar{\mathcal{H}} > 0$，对 α 的梯度为负，α 增大，逼策略分散；熵高于目标时反过来，α 减小。目标熵常取 $\bar{\mathcal{H}} = -m$（m 为动作维数），即“每维留 1 nat 的不确定性”——这个取值与奖励尺度无关，所以能跨任务复用。

评论家一侧用两个 Q 取最小，理由与 TD3 相同（式 33-24 的正偏差）。**SAC 的三件套——熵正则、双评论家、软更新——分别解决探索、过估计、目标漂移，三件事彼此独立，可以单独替换。** 这是它比 DDPG 稳的工程原因，也是它超参更多的原因：α 的初值、目标熵、双评论家的初始化，每一项都要管。

33.3.6 离线：保守惩罚的定价

离线算法的核心是把“分布外动作的 Q”压下去。CQL 型目标写成

$$J_{\mathrm{CQL}}(\theta) = \underbrace{\alpha\Big(\mathrm{E}_{s\sim\mathcal{D}}\big[\log \textstyle\sum_a \exp Q_\theta(s,a)\big] - \mathrm{E}_{(s,a)\sim\mathcal{D}}\big[Q_\theta(s,a)\big] \Big)}_{\text{保守项}} + \underbrace{\tfrac{1}{2}\mathrm{E}_{\mathcal{D}}\big[(Q_\theta(s,a) - y)^2\big]}_{\text{常规 TD 损失}} \tag{33-34}$$

保守项是两个期望之差。第一项是“所有动作的 Q 的软最大”，第二项是“数据动作的 Q”。最小化它们的差，等价于**同时做两件事：把数据动作的 Q 抬高，把其余动作的 Q 压低。** 两者之差越大，惩罚越强；$\alpha \to 0$ 时退化成普通离线 TD；$\alpha \to \infty$ 时分布外动作的 Q 趋于 $-\infty$，$\arg\max$ 只能落在支撑集内。

现在可以给这个 α 定价了。由式 (33-18)，每步 Q 的高估 δ 会被放大 $1/(1-\gamma)$。若保守项能把分布外动作的 Q 压低 D，那么只要

$$D \;\gtrsim\; \frac{\delta}{1-\gamma} \tag{33-35}$$

就能保证 $\arg\max$ 不会翻到支撑集外。**这就是 α 该怎么选的定量依据：**它不是无量纲的“正则化强度”，而是要与“评论家误差 δ”和“折扣放大 $1/(1-\gamma)$”一起算的物理量。$\gamma = 0.99$、$\delta = 0.1$ 时，需要压低 10 个单位——这解释了为什么离线算法在长视野任务上总要配很大的 α。

三处代价必须一并说清。**其一，过度保守。** α 太大时，所有动作的 Q 都被压低，策略退化成“什么都不做”或“复制行为策略”。**其二，惩罚的估计有偏。** 式 (33-34) 的第一项需要对全部动作求软最大，连续动作下只能用采样近似，采样不足时惩罚方向会歪。**其三，它保的是值、不是行为。** Q 被压对不等于策略在真机上就好，中间还隔着“策略能否被准确表示”这一层。

33.3.7 稳定化账：每个算法拆了哪条边

把本节内容收进表 33-4。这张表的用法是：看到一个不稳定的训练曲线，先在表里找到对应行，再问“它拆的那条边，我的实现里真的拆掉了吗”。

表 33-4　四个算法的稳定化手段与失效模式

算法	拆哪条边	具体手段	新增关键超参	典型失效
DQN	自举 + 分布	目标网络、经验回放、Double Q	C 或 τ、池容量	过估计、回放池过旧
DDPG	自举	目标网络（软更新）	τ、探索噪声 σ	评论家过拟合窄峰、策略钻误差
TD3	自举	双评论家取小、目标平滑、延迟更新	τ、σ、c、更新比	过度平滑导致学不快
PPO	异策略（直接放弃）	同策略采样、裁剪、GAE	ϵ、λ、每批回合数	样本效率低、裁剪过紧
SAC	自举	双评论家、软更新、熵正则	α 初值、$\bar{\mathcal{H}}$	温度失控、Q 塌缩
CQL	分布外	保守惩罚	α	过度保守、惩罚估计有偏

表里最值得对照的是第三列与第五列。**每个算法新增的超参，恰好就是它拆边时引入的那个"补偿量"**——DQN 的 C 是"目标多久换一次"，TD3 的 c 是"峰抹多平"，PPO 的 ϵ 是"一步走多远"，CQL 的 α 是"外推压多低"。这些量没有一个是"调大了更好"或"调小了更好"，全部存在最优区间，而且最优值随任务变。**这是深度 RL 调参困难的数学根源：它不是参数多，而是每个参数都在补偿一个没有定理支撑的近似。**

33.3.8 n 步回报与资格迹：偏差与方差之间的旋钮

§33.2.3 的三个目标函数决定了"优化什么"，还有一个量决定"回报怎么估"，它不改算子，只改估计的偏差与方差之比，而且这个旋钮是连续的。

n 步回报把自举往后推 n 步：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\theta(s_{t+n})$$

$n=1$ 就是 TD(0)，n 等于回合长度就是蒙特卡洛。**偏差随 n 降（自举次数少），方差随 n 升（累加的随机奖励多）**，第 32 章 E3 那张"MC 的标准差是 TD 的 2.77 倍、但 RMSE 反而低"的表，就是这条曲线两端的两个点。

连续版本是资格迹。TD(λ) 用一条迹向量

$$z_t = \gamma\lambda\, z_{t-1} + \nabla_\theta V_\theta(s_t)$$

把"往后看 n 步"压成"每步 $O(d)$ 的递推"，$\lambda=0$ 退化为 TD(0)，$\lambda=1$ 逼近 MC。PPO 里的 GAE(λ) 是同一思想在优势函数上的版本：λ 越小越偏、方差越低，越大越无偏、方差越高。**它与控制里的遗忘因子是同一个东西**——递推最小二乘里 $\lambda<1$ 让旧数据权重指数衰减，这里 $\lambda<1$ 让远期的自举误差指数衰减。

三条工程注意：

1. **n 与折扣视野要匹配。** 经验上取 $n \approx (1/5 \sim 1/3)\cdot\frac{1}{1-\gamma}$：$\gamma=0.99$ 时是 20–33 步，$\gamma=0.9$ 时是 2–3 步。取太小等于没离开 TD，取太大等于回到 MC，方差会把小批量梯度淹没。
2. **n 步与回放池有结构性冲突。** n 步目标要求连续片段，而标准回放池只存单步转移。要么存长度为 n 的片段，要么在采样时按时间索引拼接；**跨回合的片段必须截断**，否则会把上一条命的奖励算进这一条。
3. **λ 与学习率不能同时调。** 两者都在改"有效步长"：λ 大 → 目标方差大 → 需要更小的 lr。表 33-10 那个"先定 lr"的顺序，在 λ 上也适用。

33.4 离散对应与数值实现

33.4.1 输入归一化与最后一层的初始化

深度 RL 的第一次失败，八成出在输入尺度上。状态里混着角度（10^{-1} rad）、角速度（10^{0} rad/s）、位置（10^{0} m）、力（10^{2} N），直接喂进网络会让第一层的梯度被大量纲分量主导——**这不是"训练慢"，是"只学了那几个大量纲通道"**。

做法是运行均值—标准差归一化：维护每个状态分量的滑动均值 $\bar{s}_i$ 与标准差 σ_i，输入前做 $(s_i-\bar{s}_i)/\max(\sigma_i, 10^{-3})$。**注意必须在训练中滚动更新，而不是用离线统计量**，因为策略变化会让状态分布漂移。部署时冻结最后一组统计量，并把它一起打包进模型文件——漏了这一步，线上与训练就用了两套尺度。

最后一层的初始化另有一条经验：**输出初始值应接近"回报的量级"而不是 0**。若 Q 初值全为 0，而真实回报是 10^2 量级，网络要花很多步才能把输出抬上去；更糟的是，在抬升过程中所有动作的 Q 都挤在 0 附近，$\arg\max$ 由数值噪声决定，策略相当于随机。常见做法

是把最后一层权重初始化成小值（如 $\pm 3\times 10^{-3}$），偏置设为 0，或者用“回报的均值”做偏置初值。**这一条与第 32 章式 (32-41) 的乐观初始化是同一个道理，只是换到了网络参数上。**

33.4.2 回放池：存什么、抽什么

回放池存五元组 (s,a,r,s',d)，其中 d 是终止标志。**d 必须存，不能靠“s' 是终止状态”反推**：有些环境的终止状态与“到达目标”不是一回事（超时截断、越界终止），把截断当成终止会让自举目标少算后续回报，值估计系统性偏低。这类错误不会报错，只会让曲线“看起来还行但差一点”。

容量与采样方式有两条经验：

容量取 10^5–10^6 步。 太小则样本相关性去不掉（退化成在线学习），太大则池子里混着大量旧策略的数据，κ 反而变大（式 33-2）。**容量不是越大越好，它对应“记忆多长”这个时间尺度。**

优先采样（prioritized replay）按 $p_i\propto|\delta_i|^{\alpha_p}$ 抽，$\alpha_p=0.6$ 常用。 它让“预测错得多的样本”被多抽几次，收敛更快。代价是分布被改变了，必须用重要性权重 $w_i=(1/(Np_i))^{\beta}$ 修正，β 从 0.4 线性退火到 1。**不做修正的优先采样是有偏的**，这一点在实现里最容易被省掉。

33.4.3 目标网络的更新周期

硬更新每 C 步拷一次，软更新每步按式 (33-23) 混合。两者的换算很直接：软更新的等效周期约 $C_{\text{eff}}\approx 1/\tau$。$\tau=0.005$ 相当于硬更新 $C\approx 200$；$\tau=0.001$ 相当于 $C\approx 1000$。

周期的选择要与两个时间尺度匹配。**其一是折扣视野 $1/(1-\gamma)$**：视野越长，目标里包含的未来步越多，目标漂移的影响越大，周期应越长。**其二是策略改进的速度**：周期太长，目标落后于策略，学得慢；周期太短，退化成无目标网络。工程经验是 C 取 10^3–10^4 步、τ 取 0.001–0.01，再按回合长度微调。**C 大致取“一个回合的 $1/10$ 到 1 倍”是个可用的起点**——回合内不换目标，回合之间换几次。§33.6.2 会在同一任务上扫 C，给出“太长与太短各自怎么坏”的实测曲线。

33.4.4 批量、学习率与梯度裁剪

一个可用的默认组合是：批量 256、学习率 3×10^{-4}（Adam）、梯度范数裁剪 10、目标网络软更新 $\tau=0.005$。四个数各有各的作用，不要一起调。

批量决定梯度噪声：太小（32）时每步更新方向抖动大，值估计不稳；太大（4096）时更新次数变少，同样样本量下学得更慢。**有效批量应与学习率一起调**：批量翻倍、学习率按 $\sqrt{2}$ 提高是常见的配套。

学习率在演员与评论家之间应当分开。评论家可以大一些（10^{-3}），演员要小（10^{-4}），理由是式 (33-26)：演员的梯度依赖评论家的准确性，评论家没学好之前，演员大步走只会放大偏差。**这与第 32 章 Actor-Critic 的双时间尺度 $\alpha_w\gg\alpha_\theta$ 是同一条原则。**

梯度裁剪是防爆的最后一道。它不改变正常步的方向，只在上限处截断；发现“裁剪频繁触发”说明前面的学习率或归一化有问题，此时调裁剪阈值是治标。

33.4.5 奖励缩放：一个必须与温度联动的量

奖励除以一个常数 c 看起来无害，它确实不改变最优策略（正线性变换保序）。但对熵正则算法，**它会改变温度的最优值**。看式 (33-32)：$\pi^\star\propto\exp(Q/\alpha)$。若把奖励缩小 c 倍，$Q'=Q/c$，要让同一个 $\pi^\star$ 仍是解，必须有

$$\alpha'=\alpha/c \tag{33-36}$$

所以奖励缩 10 倍，温度也必须缩 10 倍，否则策略会突然变得“更确定”或“更随机”。 自动调温（式 33-33）能自己应对这个变化——这正是它比固定 α 好用的地方：α 会随 Q 的量级自动重标定。但如果用的是固定 α 的实现，改奖励尺度却不改 α，训练结果会莫名其妙地变差，而且很难查。

奖励缩放的另一个作用是限制 Q 的量级。Q 的典型大小约为 $r_{\max}/(1-\gamma)$：$\gamma=0.99$、每步奖励 1 时是 100，若每步奖励 1000 就是 10^5——此时 float32 的绝对分辨率约 10^{-2}（相对精度 10^{-7} 乘上量级），已经与单步奖励同阶，**梯度更新会被舍入吃掉，表现为“损失在降、策略不动”**。把每步奖励归一到 $O(1)$，Q 就落在 10^2 量级，留出四个数量级的余量。

33.4.6 一个可运行的训练骨架

下面这段是 DQN 的核心循环，四十几行，省掉了环境与网络定义。**它的价值不在“能跑”，而在标出了四个必须存在的步骤：归一化、存 d、目标网络、双 Q。**

```python
import torch, torch.nn.functional as F

def train_step(batch, q, q_targ, opt, gamma=0.99):
    s, a, r, s2, d = batch                      # d 为终止标志
    with torch.no_grad():
        a2 = q(s2).argmax(1, keepdim=True)      # 在线网络选动作
        y = r + gamma * (1.0 - d) * q_targ(s2).gather(1, a2).squeeze(1)
    loss = F.smooth_l1_loss(q(s).gather(1, a.view(-1, 1)).squeeze(1), y)
    opt.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(q.parameters(), 10.0)
    opt.step()
    return float(loss)


def soft_update(q, q_targ, tau=0.005):
    with torch.no_grad():
        for p, pt in zip(q.parameters(), q_targ.parameters()):
            pt.mul_(1.0 - tau).add_(tau * p)


def run(env, q, q_targ, opt, buf, steps=100_000, warmup=1_000):
    s = env.reset()
    for t in range(steps):
        eps = max(0.01, 0.3 * (1.0 - t / (0.6 * steps)))
        a = env.sample_action(q, s, eps)        # 归一化后的状态
        s2, r, done, _ = env.step(a)
        buf.push(s, a, r / env.r_scale, s2, float(done))
        if buf.size >= warmup:
            train_step(buf.sample(256), q, q_targ, opt)
            soft_update(q, q_targ)
        s = env.reset() if done else s2
```

三处细节与正文对应：`r / env.r_scale` 是 §33.4.5 的奖励缩放；`1.0 - d` 保证终止处不自举，对应 §33.4.2 存的 d；`q_targ` 全程只读，对应 §33.4.3 的目标网络。双 Q 在这里体现为**“选动作用 `q`、评价值用 `q_targ`”**（式 33-25），不是两套网络参数。

33.4.7 评估协议：三种最常见的“假成功”

训练曲线好看不等于策略好用。下面三种做法都会造出“假成功”，必须避开。

其一，用带探索的策略评估。 若评估时仍按 $\varepsilon = 0.05$ 随机动作，指标里混着“乱走”的收益，与部署行为不一致。**评估必须用确定性动作**：离散取 $\arg\max$，连续取分布均值（不是采样）。这一条在第 32 章 E7 上已经被验证过：训练末段成功率 30%、纯贪心 0%。

其二，报“最好的一次评估”。 训练中评估 50 次取最大值，等于在评估集上做了选择，数字必然虚高。**应报最后若干次评估的均值，并附四分位区间。** 若曲线本身在震荡，那就是结论——说明它不稳，而不是“没挑到好点”。

其三，单种子。 深度 RL 的种子间方差极大，同一份代码换种子，性能差一倍是常事。**至少 5 个种子、最好 10 个以上，报中位数与 $25/75$ 分位。** §33.6 的所有曲线都是 10 个种子的中位，阴影是四分位区间——这不是为了好看，是因为单条曲线的形状没有统计意义。

还有一条容易被忽略：**评估环境与训练环境要分开实例化**。若共用同一个随机数发生器，评估时的初始状态会受训练消耗的影响，不同种子之间的比较就失去了可比性。

33.4.8 数值诊断：三个症状与三条对策

训练发散时，先看三个量，再动手改。

症状一：Q 的绝对值指数上涨（10^6 以上）。 多半是正反馈：Q 大 → 目标大 → 更新把 Q 推得更大。查三处：目标网络是否真的没被梯度更新（`requires_grad=False` 或放在 `no_grad` 里）、终止标志是否正确乘进目标、奖励是否未缩放。§33.6.2 会给一个“去掉目标网络后 Q 一路发散”的对照。

症状二：损失变成 NaN。 通常是学习率过大或出现 $\ln 0$。查两处：$\ln \pi$ 里是否对 σ 加了下界（10^{-6} 量级）、softmax 是否用了 log-sum-exp 的稳定写法。

症状三：策略输出长期贴着限幅。 查 $\tanh$ 的雅可比项（式 33-16）是否漏了。**漏掉它的典型表现是"动作饱和但不报错"**，因为损失在数值上仍然下降。

三条对策按优先级排：**先归一化，再降学习率，最后才动网络结构。** 绝大多数"深度 RL 不收敛"的问题在前两步就解决了，把网络加深往往只是让问题更难查。

表 33-5 把本节的关键量收在一起，作为实现时的默认值起点。

表 33-5　实现默认值与可调范围

量	默认	可调范围	主要影响
隐藏层	2×256，ReLU	2×64 – 3×512	表示能力与过拟合
批量	256	64 – 1024	梯度噪声
学习率（评论家）	3×10^{-4}	10^{-4} – 10^{-3}	收敛速度与稳定性
学习率（演员）	10^{-4}	3×10^{-5} – 3×10^{-4}	策略改进步长
目标网络 τ	0.005	0.001 – 0.01	目标漂移速度
回放池容量	10^6	10^5 – 10^6	样本相关性与 κ
奖励缩放	归一到 $O(1)$	—	Q 的量级、温度标定
梯度裁剪	10	1 – 40	防爆
评估种子数	10	≥ 5	结论可信度

33.4.9 并行采样与分布式训练

先分清并行省的是哪一笔。 §33.5.1 会把训练成本拆成三个乘数：样本量、单样本墙钟、试验次数。并行只压第二个——**加 8 路仿真，墙钟除以 8，样本总量一步都不少**。式（33-21）里的 m 由精度要求决定，与并行路数无关。所以"训练快了"不等于"样本省了"，这是汇报时最常被混淆的一句话。

表 33-6　四种并行形态的取舍

形态	并行的是什么	策略一致性	主要代价	典型用法
同步数据并行	环境采样 + 梯度归约	完全一致	等最慢的 worker	PPO
异步经验并行	采样与更新同时进行	差 k 步	策略陈旧，需重要性修正	A3C、IMPALA
单 learner 多 actor	只并行采样	learner 侧单一策略	回放池内分布混合	DQN、SAC
多 learner 分片	参数服务器分片更新	近似一致	通信开销与一致性	大规模离线训练

代价一：策略陈旧。 异步时 actor 手里的参数比 learner 落后 k 步，μ 与 d_π 错位，κ 随之变大。判据可以直接借用 E3：**策略在 k 步内的累计 KL 不应超过裁剪半径对应的量级**。E3 里单批 30 个 epoch 就把 KL 推到 0.03（$\epsilon = 0.2$），照此估算，k 只能取"几个 batch"的量级，再大就必须上重要性修正（V-trace 一类）。

代价二：回放池变成混合分布。 多个 actor 写同一个池，μ 是若干行为策略的混合。若各 actor 的探索强度不同，μ 会呈双峰，$\kappa = \|d_\pi/\mu\|_\mu^{1/2}$ 在 μ 小的那一段被放大——**这正是 E2 里"lr 不大、C 不大，却仍然学不动"的一类隐性原因**。工程做法两条：统一各 actor 的探索参数，或按 actor 分池存放、按比例抽样。

代价三：批量放大与学习率联动。 批量随并行路数放大后，梯度方差按 $\sqrt{N}$ 下降，看起来可以把学习率同步上调。但 RL 的梯度是非平稳的，"线性缩放规则"只在监督学习里有较好依据；稳妥做法是平方根缩放加短 warmup。还有一处副作用：**批量越大，梯度里的探索噪声越小，等价于隐式降温**，SAC 这类靠熵驱动的算法会因此少探索，需要把 $\bar{\mathcal{H}}$ 或动作噪声相应调大。

最后一个反直觉的结论：**并行度提高，单次试验的墙钟下降，但试验次数不变。** 若 8 路把单次从 10 小时压到 1.5 小时，90 次试验仍要 135 小时。真正省时间的是减少试验次数——固定种子集、先粗后细、用 §33.4.7 的评估协议砍掉无效比较。**算力买得到墙钟，买不到统计显著性。**

33.4.10 动作保持、平滑与"半 MDP"

控制对象与游戏环境最大的一处差别，是**动作不能每拍乱跳**。策略网络输出里天然带高频分量：观测噪声、随机策略的采样、以及网络在相邻状态间的插值抖动。这些分量在 Atari 里无所谓，在电机上就是磨损、噪声与能耗。

工程上有三种处理，各有代价。

其一，动作保持（frame skip）。 每 k 拍执行同一个动作，等效采样周期变成 kT_s。这时两件事必须同步改：折扣因子按 $\gamma' = \gamma^k$ 重标（否则“看多远”的物理时间变成 k 倍）；奖励若定义成“率”（每秒多少），要乘 k，若定义成“总量”，则不变。**这两条漏一条，训练出来的策略就会偏保守或偏激进**，而且从曲线上看不出来。

其二，变化率惩罚。 把 $\|a_t - a_{t-1}\|$ 乘一个权重加进奖励。它确实能压住抖动，但**改变了最优策略**——原任务的解不再是最优解，报结果时必须说明这一点，并把权重与性能损失一起报。这是 §33.2.3 说的“目标函数即规格”的直接后果。

其三，输出低通滤波。 一阶滤波 $a_t \leftarrow (1-\beta)a_t + \beta a_{t-1}$ 实现简单，但它引入相位滞后。定量地看：截止频率 f_c 处相移 $\arctan(f_{\mathrm{bw}}/f_c)$；若取 $f_c = f_{\mathrm{bw}}$，相移 45°，等效延迟约 $1/(8f_{\mathrm{bw}}) = 0.125/f_{\mathrm{bw}}$——**已经超过式（33-37）给的 $0.1/f_{\mathrm{bw}}$ 裕度**。所以滤波截止频率必须显著高于闭环带宽，经验上取 3–5 倍，让它只在执行器关心的频段起作用。

还有一处结构性变化常被忽略：动作保持把 MDP 变成了“半 MDP”（每 k 拍决策一次，中间 $k-1$ 拍是开环）。半 MDP 的 Bellman 方程形式不变，但转移矩阵变成 P^k，奖励变成 k 步折扣和；**这也解释了为什么“改 frame skip 后一切都要重调”**——不是实现问题，是问题本身换了。

33.5 工程含义与适用边界

33.5.1 训练成本：三个乘数叠起来

深度 RL 的成本结构很容易被低估，因为它有三级乘数：**单次训练的样本量 × 单样本的墙钟代价 × 试验次数**。

先看样本量。按式 (33-21) 的量级估计，$d = 10^4$、$\epsilon = 0.05$ 要 4×10^6 步；实际调参时常跑到 10^6–10^7 步。以 10^6 步、1 kHz 采样计，对应 **1000 s 的物理时间，约 16.7 分钟**——这是“对象必须连续运行多久”的下界。

再看墙钟代价。仿真若比实时快 5 倍，10^6 步只需 3.3 分钟；若仿真因为求解器太细而比实时慢 10 倍，则要 **2.8** 小时。**同一份代码，两种仿真的墙钟差 50 倍**，而这一点在论文里几乎从不披露。

最后看试验次数。10 个种子 × 9 组超参 = 90 次，按每次 2.8 小时算，是 **252** 小时，**约 10.5 天**。这就是为什么“调一个深度 RL 的基线”往往要一周以上——**不是算法慢，是统计要求把单次成本乘了 90 倍**。

表 33-7 给出三档平台的成本账，用第 32 章 §32.5.5 的单步成本口径（仿真 10^{-6} 元、真机 0.1 元），中间补一档试验台。

表 33-7　三档平台的成本账

平台	单步成本	10^6 步成本	能否并行	主要限制
仿真	10^{-6} 元	1 元	可，几十路	与真实的差距
试验台	10^{-3} 元	10^3 元	少量	磨损、占用
真机	0.1 元	10^5 元	否	安全、停机损失

表里最值得注意的是最后一列。仿真便宜且可并行，所以深度 RL 的绝大多数训练都在仿真里完成；试验台用来验证“迁移是否成立”，通常只跑几十个回合；真机只做最终的在线微调，且必须带安全兜底。**这个三段式与第 32 章 §32.5.5 的三段式是同一条路线，只是每一段的样本量都大了两个数量级。**

33.5.2 sim-to-real：四类差距，四种对策

§33.2.6 把差距形式化成环境参数 ξ 的不确定，但工程上要分清是哪一类，因为对策完全不同。

第一类，参数差距（质量、摩擦、惯量、增益）。对策是域随机化。要点是**随机化范围要与辨识出的置信区间挂钩**：若摩擦系数的辨识结果是 0.30 ± 0.03，随机化到 $[0.1, 0.6]$ 就是在浪费样本，而随机化到 $[0.29, 0.31]$ 又覆盖不了。用第 18 章的置信区间定范围，是这里唯一可靠的办法。

第二类，结构差距（间隙、迟滞、死区、未建模的高频模态）。随机化**解决不了**这一类，因为仿真里根本没有这个环节。对策有两个：一是把这些环节显式建进仿真（哪怕只是查表或分段线性），二是用残差学习——让 RL 只学“模型没描述的那部分”。

第三类，观测差距（噪声、量化、滤波延迟、采样抖动）。对策是在训练时注入同分布的噪声。这里有一条容易忽略的：**噪声要在归一化之前注入**，否则归一化的统计量会把噪声“吃掉”，训练时看到的信噪比与真机不同。

第四类，延迟差距。这一类最危险，因为它不像噪声那样显眼，却直接吃相位裕度。把 §33.2.6 的换算写成一条可用的规则：纯延迟 T_d 在带宽 f_{bw} 处的相位滞后为 $2\pi f_{\mathrm{bw}} T_d$，若要求滞后不超过 36°（0.628 rad），则

$$T_d \;\le\; \frac{0.1}{f_{\mathrm{bw}}} \tag{33-37}$$

$f_{\mathrm{bw}} = 10$ Hz 时允许 10 ms，$f_{\mathrm{bw}} = 100$ Hz 时只允许 1 ms。**这条规则在部署时最容易被违反**：为了省算力，把策略降频到 50 Hz 执行（周期 20 ms），在 10 Hz 带宽上就已经超了一倍——加上零阶保持，相位滞后会到 $72°$，闭环必然振铃。**降频执行不是一个"省算力"的中性选择，它等价于往回路里插一段延迟。**

33.5.3 部署形态：推理、限幅与监督

真机部署要回答四个问题，顺序不能颠倒。

其一，策略跑多快？ 由式 (33-37) 反推上限，再由算力定下限。一个 2×256 的 MLP，输入 20 维、输出 4 维，前向约 1.5×10^5 次乘加，在普通 CPU 上是**几十微秒到 0.2 毫秒**量级。1 kHz 周期（1 ms）下占 10%–20%，可以接受；但若还要做观测滤波、归一化、安全检查，预算要翻倍。

其二，动作怎么限？ 三道：**幅值限幅**（土 物理行程）、**速率限幅**（每周期最大变化量，防阶跃冲击）、**屏蔽**（当前状态下禁止的动作，如"不许同时开两个阀"）。这三道都不依赖策略，必须独立于 RL 实现——**理由很简单：策略可能输出任何值，而执行器只有一种物理极限。**

其三，出问题谁兜底？ 一个独立的监督器，监测三类量：状态是否进入不可恢复区、动作是否连续多个周期被限幅、值估计是否异常（Q 突然大幅跳变往往预示分布外状态）。触发后切回规则控制器（PID 或 MPC），并记录现场。**这条路径必须在不加载神经网络的情况下也能运行**，否则"兜底"本身就成了新的单点故障。

其四，怎么证明它没坏？ 上线前在仿真与试验台上各跑一组固定的回归场景（正常工况、边界工况、故障注入），记录每次的指标并与基线比。**上线后每周重跑一次**，因为对象会老化——这是模型维护，不是一次性的验收。

33.5.4 五条否决：这些场合不该用深度 RL

深度 RL 的适用范围比它的热度窄得多。下面五条只要中一条，就该换方法。

一、模型可得且够准。 有可靠的 A、B，LQR/MPC 能用几行代码给出带稳定性证明的控制器，样本需求为零。**这不是"保守"，是没必要用一百万步去逼近一个能解析求解的问题。**

二、数据量不足 10^4 步。 深度 RL 的样本需求按式 (33-21) 走，几千步的数据连一次像样的拟合都不够。此时第 31 章的贝叶斯优化或直接搜索更合适。

三、安全关键且不可逆。 医疗、载人、高压场景，一旦出错无法回滚。这类场合可以用 RL 做**离线建议**（给出候选设定值，由人确认），但不能让它闭环。

四、需要可解释的决策依据。 策略是一个黑箱映射，出了事故无法回答"为什么这么动"。若监管或工艺要求逐条追溯，规则控制器或 MPC 更合适。

五、对象与任务频繁变化。 深度 RL 的训练成本高，换一个工况就要重训。若产线每周换型，维护成本会超过收益。**这类场合适合"小模型 + 在线自适应"（第 23 章），而不是大网络重训。**

反过来，最适合深度 RL 的画像是：**模型难建、目标只能靠回报表达、仿真便宜可并行、允许在仿真里出事故、且有回归测试兜底。** 这五条同时成立时才值得投入。

五条否决还可以换算成钱。按 §33.5.1 的口径，真机单步成本 0.1 元，10^6 步就是 10^5 元，还不含停机与安全风险；再乘上 90 次试验的调参系数，预算直接到 10^7 元量级。**任何一条否决成立，都意味着这笔钱没有依据。** 反过来，五条全不成立时，同样的预算买到的是一条别人拿不到的曲线——这才是深度 RL 真正的位置。

还要说清一点：**否决不等于全盘否定。** 第三条否决闭环，但允许"离线建议、人工确认"；第四条否决黑箱闭环，但允许"RL 出候选、MPC 执行"；第五条否决频繁重训，但允许"小模型 + 在线自适应"。**把否决理解成"换一种用法"，而不是"换一个方法"，往往能在同一条产线上把 RL 与经典控制拼起来。** 评审时逐条问这五句，比争论"RL 到底行不行"有效得多。

33.5.5 与 MPC 的分工：三种耦合方式

第 30 章讲过 MPC 的优势（约束、预测、可解释），本章讲了 RL 的优势（无需模型、目标灵活）。两者不是替代关系，工程上有三种成熟的耦合方式。

方式一：分层。 RL 在慢回路给设定值或调度，MPC 在快回路跟踪。判据是时间尺度：RL 的样本需求与 $1/(1-\gamma)$ 成正比，只有慢回路（分钟级）承受得起；快回路要求逐点约束与毫秒响应，是 MPC 的强项。

方式二：残差。 MPC 给基准动作，RL 学一个修正量

$$a = a_{\text{MPC}}(s) + \Delta a_\theta(s) \tag{33-38}$$

好处是：Δa 从零开始学，**初始策略就是 MPC，性能不会比基线差**；坏处是约束不再自动满足，因为 a 已经偏离了 MPC 的可行解，必须再加一层投影或限幅。

方式三：**学习型 MPC**。 用神经网络学动力学模型，MPC 仍用数值优化求解。这条路保留了约束处理能力，只是把模型换成了学出来的。**它的风险不在优化，而在模型外推**——与 §33.2.5 的离线外推是同一个问题，只不过发生在状态空间而非动作空间。

表 33-8　RL 与 MPC 的分工

维度	MPC	深度 RL	耦合建议
模型需求	需要，且要准	不需要	有模型时优先 MPC
约束处理	逐点硬约束	期望约束	快回路留给 MPC
样本需求	零	10^6 量级	RL 放慢回路
可解释性	每步有优化问题	黑箱	安全关键处保留 MPC
实时性	毫秒级求解	前向几十微秒	高频用 RL 前向，精度用 MPC
目标灵活性	需写成代价	任意标量回报	目标难写时用 RL

33.5.6 评测规范：必须报的六项

本章的结论全部依赖评测，而评测最容易做假。一份可复现的报告至少要写清六项：

1. **种子数**与统计口径（中位/均值、四分位），以及种子的产生方式。
2. **评估策略**：确定性还是带探索；是均值动作还是采样动作。
3. **评估时点**：最后 N 次评估的均值，而不是历史最好值。
4. **样本量**：训练用了多少步环境交互，以及这些步来自仿真还是真机。
5. **失败率**：不是平均回报，而是“低于阈值”的比例。**一个平均回报很高但 20% 回合崩溃的策略，不能上线。**
6. **训练成本**：墙钟时间、并行路数、总算力。**没有成本信息的“效果更好”无法比较。**

这六项与第 32 章 §32.7.4 的八项自检是互补的：那边查的是“设定对不对”，这边查的是“结论能不能被复现”。

33.5.7 从仿真到产线：三条放行线

§33.5.6 的六项是“报告怎么写”，这里三条是“什么时候可以放行”。三条按顺序走，前一条不过就不进下一条。

第一条：离线回归集。 从历史数据里冻结一批工况（覆盖正常工况与两类边界），用当前策略与上一版策略各跑一遍，逐条比对动作差与关键指标。判据：正常工况动作差的中位数小于执行器分辨率的若干倍；边界工况不允许出现方向相反的决策。**回归集必须在训练之前冻结**，否则会被结果反向定制。

第二条：影子模式。 策略在线运行但不接执行器，只记录它“想做什么”，与在役控制器逐拍比较。看两个量：分歧率（动作差超过阈值的拍子占比）与分歧处的事后表现（用下一拍的实测变化判断谁对）。工程上常用的放行门槛是分歧率低于 5%，且分歧处策略不处于系统性劣势；影子运行至少要覆盖一个完整工况周期——一个班次、一个批次，或者一次完整的启停。

第三条：灰度与回滚。 第一次上线只给有限的动作权限（位置限幅、速度限幅），从 ±10% 起步，逐级放开；同时挂三个回滚条件：连续 N 拍越界、关键指标在滑动窗口内恶化超过阈值、看门狗超时。**回滚必须是一键的，并且演练过**——没演练过的回滚等于没有回滚。

三条线的共同点：**都在回答“错了会怎样”，而不是“对了有多好”。** 这与 §33.5.6 的评测口径互补——那边防的是“自欺”，这边防的是“自负”。

33.5.8 在全书中的位置

本章是“学习与控制”这一编里承上启下的一章。往上，它接第 32 章的表格型方法，把“能枚举”推广到“能泛化”；往下，第 34、35 章讲多智能体，那时**非平稳性**会成为主要矛盾——每个智能体的环境里都包含其他正在学习的智能体，Markov 假设以另一种方式被破坏。

再往后，第 36 章讲功能安全，会给本章 §33.5.3 的“监督器”一个正式的分析框架（安全包络、失效模式、冗余设计）。**本章只说了“要有兜底”，第 36 章会说“兜底怎么证明有效”。**

最后一句边界声明，与第 32 章结尾同调：**本章所有算法都没有收敛定理。** 表格型的那些保证（压缩映射、唯一不动点、策略改进）在函数逼近下全部失效，剩下的只有经验判据：多条种子、多个工况、回归测试、以及“训练好不等于部署稳”的反复验证。承认这一点，比记住十个算法的超参更重要。

33.6 仿真与验证

七组实验全部由 `code/ch033_sim.py` 单次运行产生，纯 numpy，种子 20261103，总耗时约 45 s。七组与 §33.1.6 的五个定量问题一一对应：E1 回答“泛化误差随什么变化”，E2 回答“目标网络该多慢”，E3 回答“裁剪半径对应多大的策略变化”，E4 回答“温度在自动调节什么”，E5 回答“离线外推错在哪”，E6、E7 补上 §33.5 的两条工程边界——延迟与评估。正文数字不做事后取整。

33.6.1 E1：加参数不加样本，误差先降后升

对象是一维 21 状态链：两端反射，中间每步等概率左右走，奖励随位置从 -0.5 线性升到 $+0.5$。这个设定下 V^π 可以用式（32-9）精确解出，**“真值”不是估出来的**，误差才可分解。特征取余弦基 $\phi_k(s)=\cos(\pi k s)$，$s\in[0,1]$，$k=0,\dots,d-1$；d 从 2 扫到 18。样本由随机游走采 m 条转移 (s,r,s')，用 LSTD 的闭式解（式 33-9）拟合，30 次重复取中位数。误差是 μ-加权相对范数 $\|V_\theta-V\|_\mu/\|V\|_\mu$，$\mu$ 取均匀分布。

表 33-9　余弦基上的误差分解（相对误差，30 次重复的中位）

d	投影残差	$m=100$	$m=1000$	$m=10^4$
2	0.0658	0.1635	0.0826	0.0682
4	0.0143	0.1794	0.0542	0.0258
6	0.0052	0.2300	0.0601	0.0254
8	0.0025	0.2133	0.0622	0.0229
10	0.0015	0.1913	0.0606	0.0231
14	0.0007	0.1816	0.0592	0.0230
18	0.0004	0.2226	0.0645	0.0239

第二列是式（33-11）的第一项：把真值投影到特征子空间后剩下的分量。它与样本量无关，只由 d 决定。

第一，投影残差按预期单调下降。 d 从 2 到 18，从 0.0658 降到 0.0004，降了 164 倍。这条曲线说明“特征越多、函数族越大”不是空话，也说明这一列的测量本身是可信的。

第二，总误差不跟着降。 看 $m=100$ 那一列：$d=2$ 时 0.1635，加到 $d=6$ 反而涨到 0.2300。参数多了四倍，误差大了四成。原因在式（33-11）的第二项：估计误差随 d 上升、随 m 下降，m/d 从 50 掉到 17 时，方差项吃掉了偏差项的全部收益。这一列的形状就是 U 形——最小值在 $d=2$ 附近，之后一路走高再在高频端抖动。**“参数多一定更好”在这张表里被直接否掉。**

第三，样本够多时收益迅速饱和。 看 $m=10^4$ 那一列：$d=4$ 是 0.0258，$d=18$ 是 0.0239，只好了 7%；而同一段里投影残差降了 36 倍（0.0143 到 0.0004）。**这正是式（33-21）的规模账在起作用：**要把偏差再压一个数量级，样本量得按 d/ϵ^2 同步涨，否则省下的偏差被涨上来的方差原样抵消。

三个地板要分清。偏差地板由特征决定，$d=18$ 时是 0.0004；方差地板由样本决定，$d=18$、$m=10^4$ 时是 0.0239，比偏差地板高 60 倍。**实际能拿到的精度是两块地板中高的那一块，**而多数调参只盯着低的那块。

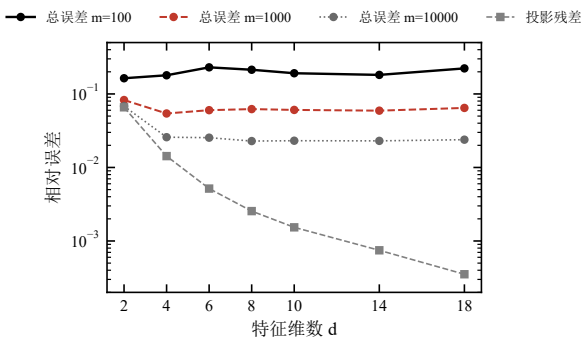


图 33-2　余弦基上的误差分解：三条实线是总误差，虚线是投影残差

图 33-2 把三列画在一起。三条实线在 $d=4$ 之后基本水平，只有 $m=100$ 那条还在抖；虚线一路向下，从不与实线相交。**交点不存在，这件事本身就是结论**：在这个任务上，$d\geq 4$ 之后再加参数，收益全部被方差吃掉。

工程判据只有一条：**每个参数至少摊到几百个样本**。表里 $m/d\geq 555$ 的两行误差都在 0.024 附近；$m/d=17$ 的那一行是 0.2226，差一个数量级。中间地带（m/d 在 50 到 100 之间）是“加参数可能还有用”的区间，必须实测确认。这条判据比“层数乘宽度”之类的经验公式可靠，因为它直接对应式（33-21）里的两个量。

33.6.2 E2：稳定性是一条斜带，不是“目标网络越稳越好”

对象是 21 状态链，两个动作左右移动，右端 $+10$、左端 -10、每步 -0.1，最多 60 步。Q 用三次多项式特征线性逼近（$d=4$），回放池 5000，批量 32，$\gamma=0.95$，训练 6000 步，2 个种子平均。目标网络用硬更新，周期 C；$C=1$ 等价于“不用目标网络”。每格报两个数：训练结束时的 $\max|Q|$（数值健康度）与 30 回合贪心回报（策略质量）。

表 33-10　学习率与目标周期的二维扫描（每格为 $\max|Q|$ / 贪心回报）

学习率	$C=1$	$C=20$	$C=100$	$C=500$	$C=2000$
0.01	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.15×10^{1} / 9.14	1.10×10^{1} / 1.55
0.03	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.11×10^{1} / 1.58	1.16×10^{1} / 1.05
0.05	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	1.06×10^{1} / 9.14	8.05×10^{0} / 0.79
0.10	1.66×10^{272} / -6.00	1.19×10^{19} / -3.20	4.91×10^{20} / -6.00	3.32×10^{21} / -8.48	9.02×10^{19} / 2.02

第一，学习率决定“能不能收敛”。 $\mathrm{lr}=0.10$ 一行，五个 C 全部发散，量级从 1.2×10^{19} 到 1.7×10^{272}；把目标周期从 1 拉到 2000 也救不回来。**目标网络不是稳定器**，它只是把自举目标冻结一段时间，半梯度的病根（式 33-5 那个投影不保证压缩）还在。这与 §33.3.1 的定性一致，现在有了量级：步长越界一个因子 2（0.05 到 0.10），结果从“完全正常”变成“三十个数量级”。

第二，目标周期决定“追不追得上”。 $\mathrm{lr}=0.01$ 与 0.03 两行，$C\leq 100$ 时回报都是 9.14；C 拉到 500，0.03 那行掉到 1.58；$C=2000$，两行都掉到 1.5 以下。数值没有发散（$\max|Q|$ 仍是 11 左右），策略却什么都不会做——**目标太旧，值函数在追几十步之前的目标，追不上就停在半路**。这与式（33-19）的 $C_{\text{eff}}\approx 1/\tau$ 对得上：τ 太小等价于 C 太大。

第三，稳定域是一条斜带。 $\mathrm{lr}=0.05$、$C=500$ 还能拿到 9.14，说明“步长大一点、目标更新勤一点”也能走通；而同一行的 $C=2000$ 掉到 0.79。所以 C 不能单独调：它的有效值取决于 lr、γ 与回合长度，三者一起变。

这张表里还有一个容易骗人的格子：$\mathrm{lr}=0.01$、$C=2000$ 时 $\max|Q|=11.0$，与正常收敛的 10.6 只差 4%，但回报只有 1.55。**值函数的数值范围完全正常，策略却不会做事**。§33.6.7 会从另一头给出同样的提醒：$\max|Q|$ 与回报之间没有单调关系，必须各看各的。

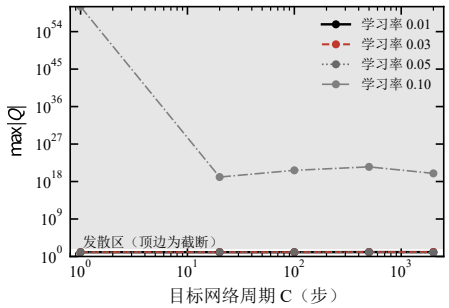


图 33-3　学习率与目标周期的稳定性分区：纵轴为 $\max|Q|$ 的对数

图 33-3 把四行画在对数纵轴上：三条平线（10.6 附近）与一条从 10^{19} 起跳的直线，中间没有过渡带。**稳定与发散之间没有“半稳”**，调参能做的只是确认自己在平的那一侧。

工程做法是两步：先用小 C（20–100）确认能学起来，再把 C 逐步放大到“回报开始下降”的位置，取它的一半。表里这个下降很陡（9.14 到 1.58），所以宁可在小 C 一侧多留余量。至于 lr，先用 10^{-3} 量级配合小网络试，看到 $\max|Q|$ 在前 1000 步内单调上升就停手——**这是最便宜的发散早期预警**。

33.6.3 E3：裁剪半径换算成多大的 KL

实验设计成“同一批数据反复用”的极端情形：5 臂老虎机，第 3 臂均值 1.0、其余为 0，观测噪声 0.3；起点是均匀策略；采 512 个样本，在这批数据上做 30 个 epoch，步长 1.0。半径 ϵ 从 0.02 扫到 0.5，另加一组“不裁剪”作对照，每个 ϵ 跑 5 个种子。量三个数：30 个 epoch 累计的 KL、过程中的最大概率比、最终策略落在最优臂上的概率。

表 33-11　裁剪半径与实测策略变化

ϵ	累计 KL	等效半径 $\sqrt{2\,\mathrm{KL}}$	最大 ratio	P(最优)	策略熵
0.02	0.0020	0.063	1.25	0.219	1.607
0.05	0.0040	0.089	1.25	0.224	1.605
0.10	0.0094	0.137	1.29	0.240	1.600
0.20	0.0300	0.245	1.44	0.267	1.580
0.50	0.1476	0.543	1.82	0.335	1.473
无裁剪	2.4529	2.215	4.86	0.973	0.159

第一，KL 随 ϵ 单调上升，但远不是 $\epsilon^2/2$。 把等效半径除以 ϵ，得到 3.15、1.78、1.37、1.22、1.09。半径越小，每单位 ϵ 换来的 KL 越多。原因不难找：裁剪只截断越界的那部分样本，没越界的样本仍按原始梯度推；ϵ 小时越界的样本少，方向更“纯”，所以单位半径更有效。**“ϵ 小所以更新小”只对了一半，另一半是“ϵ 小所以更新的方向更集中”。**

第二，最大 ratio 与 ϵ 不是一一对应。 $\epsilon = 0.02$ 时最大 ratio 已经到 1.25，远超 $1+\epsilon = 1.02$。因为 ratio 是在 30 个 epoch 里逐次累积的：前几个 epoch 没越界，裁剪不生效；一旦越界，单步的 ratio 仍是“上一 epoch 的 1.2 再乘一次步长”。“裁剪把 ratio 限制在 $[1-\epsilon, 1+\epsilon]$”这句话只在单步、且上一步 ratio 恰为 1 时成立。

第三，裁剪的代价是慢。 30 个 epoch 里，$\epsilon = 0.5$ 也只把最优臂的概率推到 0.335，不裁剪是 0.973。这就是裁剪的意义：宁可慢，不要一步跨出信任域。反过来说，**想学得快应该加 epoch 数或加批量，而不是放大 ϵ**；放大 ϵ 换来的是同样的步数走得更远，代价是可能一步跨过最优点。

回到 §33.3.4 的换算。$\epsilon = 0.2$ 时累计 KL 是 0.0300，约等于 1.5 个 $\epsilon^2/2$；$\epsilon = 0.5$ 时 0.1476，约 1.18 个。**量级对得上，常数因子在 1 到 1.5 之间。**这比“$\epsilon = 0.2$ 就是 KL $= 0.02$”可靠——后者把单步单坐标的界当成了整体的界。

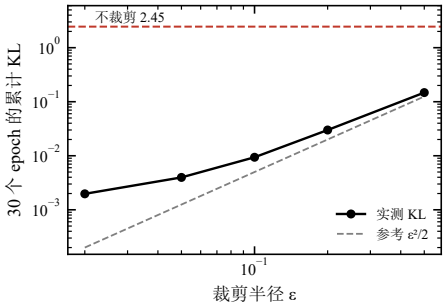


图 33-4　裁剪半径与实测 KL：虚线为 $\epsilon^2/2$ 参考

图 33-4 的横纵坐标都取对数，实测点连成的线斜率约 1.5（不是 2），与上面“单位半径效率随 ϵ 递减”是同一件事的另一种说法。**要报“这次更新动了多少”，应当直接报 KL，而不是报 ϵ**：同一个 0.2 在不同任务上对应 0.03 到 0.3 不等。

33.6.4 E4：奖励乘 100，温度也乘 100

3 臂老虎机，均值 $[1,2,3]\times c$，c 取 1、10、100。熵目标 $\bar{\mathcal{H}} = 0.5$ nat，对偶用精确熵驱动（实际实现用单样本估计，收敛更慢但不动点相同），策略步长 $0.05/c$，8000 步，策略加 2% 均匀混合以避免 softmax 饱和后梯度消失。

表 33-12　温度随奖励尺度的缩放

c	α	α/c	策略熵	策略分布
1	0.5481	0.5481	0.500	$(0.022,\ 0.136,\ 0.842)$
10	5.4802	0.5480	0.500	$(0.022,\ 0.136,\ 0.842)$
100	54.7953	0.5480	0.500	$(0.022,\ 0.136,\ 0.842)$

三行只有一个量在变：α 本身，严格按 c 放大 100 倍。比值 α/c 三位一致（0.5481、0.5480、0.5480），熵精确停在目标值 0.500，策略分布逐位相同。**奖励放大 100 倍，温度自动放大 100 倍，策略一个比特都没变。**

这就是式（33-36）的定量版。工程含义有三条。

1. **改奖励尺度后不要手工重调 α**，让对偶自己走。固定 α 的人换任务必翻车：本实验里若把对偶关掉、α 固定为 1，$c=100$ 时奖励项是熵项的 100 倍，策略会退化成近确定性（同样的数值实验里熵掉到 0.17），探索消失、离线数据覆盖度崩掉。
2. **报超参时必须报 α 与奖励尺度的比值**，只报 α 没有意义。别人复现不了“$\alpha=0.2$”，但能复现“$\alpha/c=0.55$”。
3. **熵目标与奖励尺度无关**，它只由动作维数与期望的探索强度决定。§33.3.5 提过 $\bar{\mathcal{H}}=-m$ 是常用默认值；本实验用 0.5 是为了在 3 臂上看得清。

还有一处容易被忽略：这里策略步长取 $0.05/c$，与温度的缩放一致。若步长不随 c 缩放，$c=100$ 时策略会在温度追上之前就饱和，对偶被拖进“越饱和越升温、越升温越不动”的死循环——**这正是把这条实验做坏的前几次尝试里发生的事**。实现上要么缩放步长，要么给策略加混合下限，二者选一。

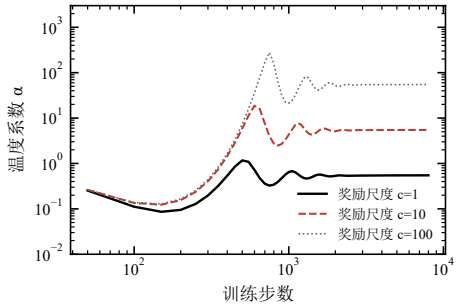


图 33-5　温度随奖励尺度线性缩放：三条曲线在对数坐标下平行

图 33-5 的三条 $\alpha(t)$ 曲线在对数坐标下互相平行，间距正好是 $\log 10$。**看到平行线就该想到“乘性缩放”**，这是判断一个自适应机制是否抓住了不变量最快的方法。

33.6.5 E5：外推误差错在特征，不在数据

400 条离线数据，$s\in\{0,1\}$，动作只在 $|a|\le 0.4$ 内取（正负各半），真值 $Q^\star(s,a)=-(a-0.4\,\mathrm{sign}(s))^2$，观测噪声 0.02。支撑集外取 $a\in[0.5,1.0]$，真值是 -0.36（在 $a=1.0$ 处）。比较四种组合：线性基 $[1,s,a,sa]$ 与二次基 $[1,s,a,sa,a^2]$，各自做普通最小二乘与“把支撑外压到边界值减 1”的保守拟合（迭代 5 轮）。

表 33-13　离线外推：四种组合的 Q 估计（真值 $Q(0.4)=0$、$Q(1.0)=-0.36$）

特征	拟合方式	$Q(0.4)$	$Q(1.0)$	外推误差
线性基	最小二乘	$+0.021$	$+0.247$	$+0.607$
线性基	加保守项	-0.399	-1.289	-0.929
二次基	最小二乘	-0.004	-0.388	-0.028
二次基	加保守项	-0.171	-1.747	-1.387

第一，线性基的最小二乘把支撑外估高了 0.607。 更糟的是 $Q(1.0)=+0.247$ 比支撑内的 $Q(0.4)=+0.021$ 还高——**贪心策略会去选 $a=1.0$，一个数据里从未出现过的动作**。而且这个错误没有任何“数据不足”的信号：拟合残差在支撑内只有噪声量级，模型自己觉得很好。

第二，保守项把方向掰对了，但泄漏进支撑内。 加惩罚后 $Q(1.0)$ 从 $+0.247$ 压到 -1.289，排序正确（现在它低于支撑内），代价是 $Q(0.4)$ 从 $+0.021$ 掉到 -0.399。原因是特征 a 与 sa 在支撑内外都起作用，**线性模型没有“只在支撑外生效”的自由度**，惩罚只能沿着特征方向整体平移。这就是式（33-34）那个 $\max$ 项在有限特征下的真实行为：它不是一把手术刀，是一块压板。

第三，把 a^2 补进特征，外推误差从 0.607 降到 0.028。 没有增加任何数据，也没有调任何超参。因为真值本身就是二次的，补上 a^2 之后特征族能精确表示它，外推自然正确。**这条结果把“外推误差”的责任从数据量挪到了特征上**：它不是“见得少所以猜不准”，而是“用直线去延长抛物线”。

第四，基对了还硬加惩罚，反而把支撑内压坏 0.171。 保守项是一个保险，不是精度工具。它的价格就是 §33.3.6 里的 D：你要求支撑外低多少，支撑内就跟着低一点。

工程判据可以写成一句话：**先问“支撑外的 Q 由哪个特征主导”，再决定要不要惩罚。** 如果主导项在支撑外是发散型（如线性基的 a），必须惩罚或换特征；如果特征族本身能表达真值的形状，惩罚只会添乱。离线数据永远无法告诉你“支撑外对不对”，它只能告诉你“支撑内拟合得好不好”——这两件事在表 33-13 的第三、四行之间差了一个数量级。

33.6.6 E6：延迟裕度由带宽定，且是硬边界

对象是二阶系统，自然频率 5 Hz、阻尼比 0.3，采样 1 ms。控制器是离散 LQR，$Q=\mathrm{diag}(100,1)$ 固定，只把控制权重 R 取 1、10、100 三档，得到三组增益 $K=[5.278,\ 0.615]$、$[1.855,\ 0.261]$、$[0.355,\ 0.083]$。初始位移 1、目标 0，纯延迟从 0 扫到 40 ms。稳定性用增广状态矩阵的谱半径判定（把延迟链写进状态），时域响应另跑一遍核对。

表 33-14　三档增益在不同纯延迟下的闭环谱半径

延迟 / ms	激进（$R=1$）	中等（$R=10$）	保守（$R=100$）
0	0.990	0.989	0.984
2	0.993	0.990	0.985
3	**1.064**	0.990	0.985
5	1.103	0.990	0.985
8	1.105	**1.027**	0.986
20	1.076	1.042	**1.002**
30	1.060	1.037	1.010
延迟裕度	**3 ms**	**6 ms**	**19 ms**
调节时间（无延迟）	389 ms	372 ms	261 ms

第一，裕度与增益反向。 位置增益从 5.278 降到 0.355（14.9 倍），裕度从 3 ms 涨到 19 ms（6.3 倍）。不是严格反比，因为速度增益也在变，但方向明确：**想要更快的响应，就得接受更小的延迟裕度。**

第二，越过边界是硬跳变。 激进档从 2 ms 的 0.993 跳到 3 ms 的 1.064，时域上从“零超调、389 ms 收敛”变成“超调 9222%、十步内发散”，中间没有过渡。线性系统的稳定与否是“是/否”，不是“好坏”；**这意味着延迟裕度只能靠留余量，不能靠调参把它“调得温和一点”。**

第三，激进整定没有换来速度。 三档的无延迟调节时间是 389、372、261 ms——保守档反而最快。因为激进档的主极点 0.990 更靠近单位圆，时间常数约 100 步，振荡衰减慢；保守档是 0.984，时间常数 62 步。**“高增益 = 快”在这个对象上不成立，**高增益换来的只是一个更靠边界的闭环。

对照式（33-37）的 $T_d \le 0.1/f_{\mathrm{bw}}$：取 $f_{\mathrm{bw}} \approx 5$ Hz，上界是 20 ms。三档实测裕度 3、6、19 ms 全部落在上界内，但激进档只有上界的 1/7。**经验式给的是上界，不是可实现值；**实际能用到多少，取决于整定有多激进、传感器与执行器还有多少未建模动态。

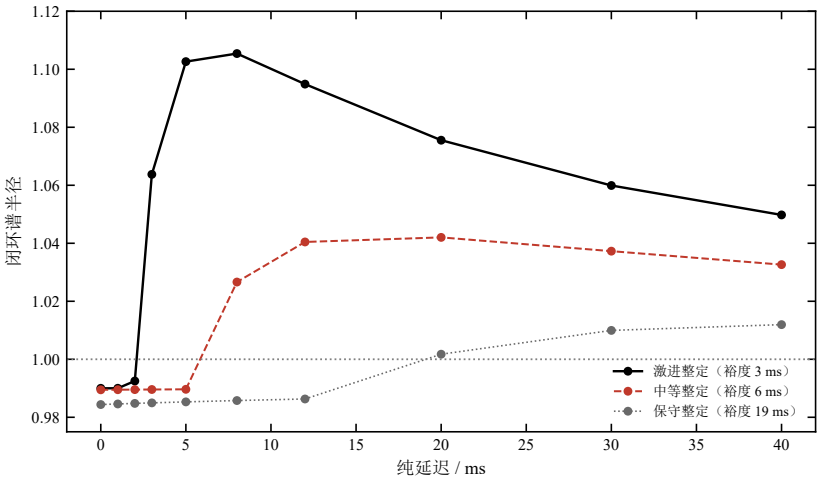


图 33-6　闭环谱半径随纯延迟的变化：三条曲线在 1 处硬穿越

图 33-6 把三条曲线画在一起，横轴是延迟、纵轴是谱半径，水平虚线是 1。三条线的交点位置差 6 倍，但形状一样陡。**这张图是 sim-to-real 的第一张检查表：**在仿真里把延迟加到“谱半径刚过 1”的位置，再看训练出来的策略还能不能活——活不下来，说明策略依赖了仿真里不存在的时序精度。

对本章的 RL 而言，这张表的含义更直接：策略网络的动作延迟（观测采集、推理、通信、执行器刷新）实测常在 5–20 ms，而本章的算例里 5 Hz 的闭环只能容忍 3–19 ms。**所以做 RL 控制时，闭环带宽不宜超过 5–10 Hz；**想要更快的环路，得先用第 47 章的补偿手段把

延迟从环路里挪出去，再谈学习。

33.6.7 E7：三种“假成功”的量化

任务仍是 21 状态链，10 个种子，每个训练 12000 步（$\mathrm{lr}=0.03$、$C=200$）。训练中每 400 步用 5 个回合评估一次，共 30 个检查点；训练结束后再用 300 个回合重评同一个策略。这样就同时有了“挑选口径”与“真值口径”。

表 33-15　三种评估口径的对比（10 个种子平均）

口径	数值	与真值之差
30 个检查点里挑最好	$+9.74$	$+0.64$
最后一个检查点	$+9.10$	0.00
300 回合重评（真值）	$+9.10$	—
带探索（$\varepsilon=0.15$）评估	$+8.86$	-0.24

第一，偏差全部来自“挑”，不来自“测”。 5 回合评估与 300 回合真值的平均绝对偏差只有 0.02，但 30 个检查点取最大就抬高了 0.64，是单次偏差的 30 倍。原因是选择偏差：30 次独立评估的最大值约等于“均值加两个标准差”，而标准差正是 5 回合评估的噪声。**报告“历史最好”等于主动把这 0.64 加进结果里。**

第二，带探索评估会低估。 把训练时的 $\varepsilon=0.15$ 带到评估里，回报从 9.10 掉到 8.86。这个方向与第一条相反：一个抬高、一个压低，**如果两件事同时做（挑最好 + 带探索），两个偏差会部分抵消，看上去“很准”，其实两个都错了。**这类巧合是评估表里最难查的。

第三，这个任务的种子差异很小。 10 个种子的真值范围是 $[9.07, 9.16]$，跨度 0.09。原因是任务太容易：任何向右走的策略都能在 60 步内到 $+10$。**换到难任务上，种子差异会成为主要方差源**，那时“挑最好”的偏差会比 0.64 大得多——因为噪声与种子差异叠在一起。

三条对策，与 §33.4.7 呼应但更具体：

1. **固定评估回合数，且不少于 100。** 本实验里 5 回合的单次偏差虽小（0.02），但那是任务简单所致；一般任务上 5 回合的标准差在 0.3–1.0 之间，挑 30 次的偏差就是 0.6–2.0。
2. **报“最后 N 次评估的中位数”，不报最大值。** 中位数对选择偏差免疫，代价是需要多存几个检查点。
3. **同时报失败率。** 本实验的回报几乎不含方差，失败率是 0；真实系统里“平均回报高、失败率 20%”的组合必须先看失败率。

33.6.8 换一个任务，哪些数会变

七组实验都跑在同一个“小世界”里：21 状态链、5 臂老虎机、二阶对象。换成车间的对象、换成机械臂，表里的数字一定变。**分清“结构”与“常数”，才知道哪些结论能搬走。**

E1 的投影残差是任务常数，U 形的存在是结构。 0.0658 到 0.0004 这一列取决于 μ、特征基与真值的形状；若真值更“弯”（比如奖励二次），$d=18$ 也不够，整条曲线会抬起来。但“总误差由偏差地板与方差地板中高的那块决定”与任务无关——**换任务只会换地板高度，不会换地板原理。**

E2 的临界学习率是任务常数，“先 lr 后 C”的顺序是结构。 有效步长大致正比于 $\mathrm{lr}\cdot\|\phi\|^2$，特征尺度一变（比如把输入从 $[-1,1]$ 改到 $[0,100]$），临界值就整体平移；换 Adam 又要重新标定。但“lr 越界是硬发散、C 越界是软退化”这两条，在 E2 的四行里都能看到，换成别的任务只会换数值，不会换形态。

E3 的常数因子 1–1.5 依赖步长与 epoch 数，“ϵ 不是 KL 上界”是结构。 本实验用 5 臂，KL 是 5 个分量之和；换成 6 维连续动作，同样的 epoch 数会把 KL 累加到十几倍，等效半径相应放大。**动作维数越高，ϵ 与 KL 的差距越大**——这条对高维控制尤其重要。

E4 的 0.548 依赖目标熵 0.5 与 2% 混合下限，$\alpha\propto c$ 是结构。 换熵目标，α 会按比例重定，但“奖励放大 k 倍、温度也放大 k 倍、策略不变”这个不变性来自对偶本身，与任务无关。

E5 的 0.607 与 0.028 依赖真值恰好是二次的。“误差由支撑外的主导特征决定”是结构。 若真值是四次，补 a^2 只能修好一半，得继续补 a^3、a^4；若真值在支撑外饱和（比如有硬限位），线性基反而不会错得那么离谱。**先看真值在支撑外的形状，再决定补什么。**

E6 的 3、6、19 ms 依赖 5 Hz 与 1 ms 采样，硬边界是结构。 按 $T_d\le 0.1/f_{\mathrm{bw}}$ 换算：带宽减半，裕度大约加倍；采样周期从 1 ms 变 10 ms，可分辨的延迟台阶就变粗。但“谱半径在 1 处硬穿越、没有过渡带”是线性系统的性质，与对象无关。

E7 的 0.64 依赖检查点数量与单次评估的噪声，选择偏差是结构。 极值统计给一个可算的近似：若单次评估的标准差为 σ_e、共挑 n 个检查点，虚高的期望约为 $\sigma_e\sqrt{2\ln n}$。反过来用 0.64 与 $n=30$ 解出 $\sigma_e\approx 0.25$，与 5 回合评估的实际波动同量级——**这说明 E7 的虚高不是异常，而是极值统计的必然结果**。换成 100 个检查点，虚高会涨到 0.75 左右；换成 300 回合评估，σ_e 减半，虚高也随之减半。

由此可以给出一条报告纪律，叫“三报”：**报结构（哪条结论与任务无关）、报常数（这条曲线属于什么任务）、报条件（种子数、回合数、延迟、带宽）**。只报结构，读者无法复现；只报常数，读者无法迁移；只报条件，等于什么都没说。

33.6.9 七组实验合起来说明了什么

七组实验各自回答了一个问题，合起来是一句话：**函数逼近把“收敛”换成了“平衡”，而这个平衡由四对量共同决定。**

- **偏差与方差**（E1）：d 压偏差、m 压方差，二者不可兼得；实际精度由高的那块地板决定，m/d 低于百量级时加参数是净损失。
- **步长与目标周期**（E2）：步长给出稳定上界，目标周期决定学习速度；越界是硬跳变，滞后是软退化，后者更常见也更难发现。
- **裁剪半径与信任域**（E3）：ϵ 不是 KL 的上界，而是它的形状参数；单位半径的效率随 ϵ 递减，想快应该加数据而不是放宽裁剪。
- **奖励尺度与温度**（E4）：α/c 是不变量，α 本身不是；不缩放步长或不加混合下限，大尺度下会把对偶拖死。
- **特征与外推**（E5）：支撑外的误差由特征在支撑外的行为决定；保守项能改排序，但会沿特征方向泄漏进支撑内。
- **带宽与延迟**（E6）：裕度随增益下降，且是硬边界；5 Hz 的环路只容忍 3–19 ms，而策略网络的动作延迟常在 5–20 ms。
- **挑选与测量**（E7）：评估偏差主要来自挑选检查点，不来自单次测量；“历史最好”必须禁止。

这七条里没有一条是“算法选择”问题，全部是“账怎么记”的问题。第 32 章 §32.7.1 说过，八种表格型算法的差别只是记账方式不同；到了函数逼近，这句话变成：**十种深度算法的差别，也只是把上面七笔账记在哪一栏。** DQN 记在“目标周期”与“回放分布”两栏，TD3 记在“最大化偏差”一栏，PPO 记在“信任域”一栏，SAC 记在“温度”一栏，CQL 记在“外推”一栏。选算法之前，先确认自己卡在哪一栏。

七组实验的复现只需一条命令：`python code/ch033_sim.py`。所有种子、步数、超参都写在函数签名里，正文没有隐藏参数。**这也是本章对“可复现”的最低承诺：数字能被重跑出来，而不是被叙述出来。**

33.7 小结与展望

33.7.1 三句话把本章收住

第一句：本章换掉的不是算法，是“保证”。 第 32 章的八种方法共享同一个压缩映射，收敛性只由 γ 决定；本章把表格换成特征之后，压缩性要乘上投影的模 $\gamma\kappa$，而 $\kappa=\|d_\pi/\mu\|_\mu^{1/2}$ 由数据分布与目标策略的失配决定（式 33-2）。同一句“收敛”于是变成“在 $\gamma\kappa<1$ 时收敛”——**而 $\gamma\kappa$ 在工程上既测不到也控不住**：μ 是回放池里的经验分布，d_π 是当前策略会走到的分布，两者都在训练中变。本章这些算法所做的全部事情，都是把 $\gamma\kappa$ 按回 1 以下的工程手段：目标网络冻结自举目标，回放池抹平 μ 与 d_π 的差，双评论家压住 $\max$，裁剪限制 d_π 的移动速度，保守项把 μ 的支撑外压下去。**没有一个是定理。**

第二句：误差从一笔变成五笔。 表格型的误差只有“采样不够”一笔；加了函数逼近之后，部署时看到的差距可以写成五项之和：

$$\underbrace{\mathcal{E}_{\text{部署}}}_{\|V_\theta-V^\pi\|}\ \approx\ \underbrace{\mathcal{E}_{\text{近似}}}_{\text{特征族 }d}+\underbrace{\mathcal{E}_{\text{估计}}}_{\text{样本 }m}+\underbrace{\mathcal{E}_{\text{自举}}}_{\text{目标周期 }C}+\underbrace{\mathcal{E}_{\text{外推}}}_{\text{数据覆盖}}+\underbrace{\mathcal{E}_{\text{口径}}}_{\text{评估方式}} \tag{33-39}$$

这五项在本章都有对应的实跑数字：第一项在表 33-9 的第二列（d 从 2 到 18，投影残差从 0.0658 降到 0.0004）；第二项在同一张表的 $m=100$ 列（$d=2$ 是 0.1635，加到 $d=6$ 反升到 0.2300）；第三项在表 33-10（C 从 100 拉到 2000，回报 9.14 掉到 1.05，而 $\max|Q|$ 仍是 11 左右）；第四项在表 33-13（线性基外推高估 0.607，补一个 a^2 降到 0.028）；第五项在表 33-15（挑最好虚高 0.64，是单次评估偏差 0.02 的 30 倍）。

第三句：五笔账里只有第一笔能靠加参数压下去。 这是本章最不“深度”的结论，也是最该记住的。加宽网络只作用于 $\mathcal{E}_{\text{近似}}$；$\mathcal{E}_{\text{估计}}$ 要样本，$\mathcal{E}_{\text{自举}}$ 要调 C，$\mathcal{E}_{\text{外推}}$ 要数据覆盖或换特征，$\mathcal{E}_{\text{口径}}$ 要改报告习惯。**五个里面四个不是模型问题。** 表 33-16 把八个代表性算法按“动了哪一笔账”排开，选型就是认领自己那一栏。

这三句话也可以直接当评审提纲用。第一句问“你的稳定化手段拆的是哪条边”，第二句问“你报的误差里混了几笔”，第三句问“你加的算力作用在哪一笔”。**三问都答得出来，方案才算讲清；有一问答不出，后面的曲线再漂亮也只是经验。** 尤其是第三句：多数团队把预算全押在第一笔上，而 E1 已经说明 m/d 不够时，这笔投入是负收益。

表 33-16　八个代表性算法各自动了哪一笔账

算法	策略形态	主要动作	引入或放大的偏差	最敏感的超参	典型失效
DQN	离散，$\arg\max$	回放改分布 + 目标网络冻结自举	最大化偏差	目标周期 C、回放容量	C 过大，值追不上（E2）
Double DQN	离散，选与评分离	动作由在线网选、由目标网评	轻微低估	—	探索不足时低估累积
DDPG	确定性 actor	只冻结自举（软更新）	Q 的偏差原样进梯度（式 33-26）	软更新 τ、动作噪声	对超参极敏感，Q 缓慢漂移
TD3	确定性 + 双评论家	再取两个 Q 的较小值	偏低	目标平滑尺度、延迟更新步数	动作维高时探索慢
SAC	随机策略	自举 + 双 Q + 熵正则	熵项带来的温度偏差	α/c、目标熵 $\bar{\mathcal{H}}$	换奖励尺度不重调 α（E4）
PPO	随机策略，同策略	用裁剪限制单步移动	裁掉的样本不计，梯度有偏	ϵ、epoch 数、GAE λ	把 ϵ 当 KL 上界（E3）
A2C	随机策略，多线程	并行采样去相关	各线程策略不同步	线程数、步长	线程间梯度互相抵消
CQL	任意 + 保守惩罚	把分布外动作的值压下去	保守偏差沿特征泄漏	惩罚系数、目标间距 D	支撑内被压坏（E5）

八行里 DQN 与 Double DQN 只差一步（谁选动作），TD3 与 DDPG 只差两处（双 Q、延迟更新），SAC 与 TD3 只差一个熵项。**深度 RL 所谓的新算法，大多是旧算法在一笔账上的一处修改**；记住账，比记住名字有用。

33.7.2 一页纸选型

四个问题按顺序问，顺序不能打乱：

1. **模型可靠吗？** 可靠就用 LQR / MPC（第 30、58 章），不要再往下问。
2. **能离线反复评价吗？** 能就用第 31 章的贝叶斯优化或直接搜索，样本效率高一个量级。
3. **状态能枚举、动作离散吗？** 能就用第 32 章的表格型：要无偏选 MC，要在线更新选 TD，要复用旧数据选 Q-learning。
4. **都不是？** 才进本章，并且先回答三个“数据从哪来”的问题，再谈算法。

   这三个问题是：

- **数据能不能反复用？** 能（有回放池）就走 off-policy，代价记在 κ 上；不能（同策略、大批量并行）就走 PPO 型，代价是样本效率。
- **探索是不是瓶颈？** 是就上熵正则，并同时定好 $\bar{\mathcal{H}}$ 与 α/c；不是（奖励稠密、动作维低）就不必付温度的账。
- **有没有“不许试”的约束？** 有就进离线，先算覆盖度，再决定要不要保守项，并且**只报支撑内的性能**。

典型场景可以压成四句话：**仿真便宜、并行路数多、要求稳**——PPO；**真机采样贵、要复用历史数据**——SAC 或 TD3；**只有一段历史数据、上线前不许试**——CQL 型，并准备接受保守偏差；**离散动作、单步决策**——DQN 或 Double DQN。四句话都不涉及“哪个算法更先进”，只涉及“数据长什么样”。

本章最常见的浪费与第 32 章同源：**在第 1 问就能用 MPC 解决的问题上直接上了深度 RL**。区别只在于到了本章，试错成本更高——第 32 章浪费的是时间，本章可能浪费真机与安全裕度。

最后一条比选型更重要：**先把评估口径与放行标准定下来，再开始训练。** 口径定晚了，训练出来的曲线会被“怎么报”反向塑造——这就是 E7 那 0.64 的虚高在生产环境里的版本。**选型错了可以重选，口径错了会把错的东西报成对的。**

33.7.3 十条常见错误

一、加宽网络不加数据。 错在方差项按 d/m 上升（式 33-21），与偏差项方向相反。E1 里 $m = 100$、d 从 2 加到 6，误差从 0.1635 涨到 0.2300——参数多四倍，误差大四成。做法：每个参数至少摊到几百个样本，$m/d < 50$ 时先加数据。

二、把目标网络当稳定器。 错在它只冻结自举目标，不改半梯度的病根。E2 里 $\mathrm{lr} = 0.10$ 一行，C 从 1 拉到 2000 全部发散，量级从 1.2×10^{19} 到 1.7×10^{272}。做法：先定学习率（看 $\max|Q|$ 在前 1000 步是否单调上升），再放大 C。

三、以为目标网络越稳越好。 错在 C 太大时值函数追不上目标。E2 里 $\mathrm{lr} = 0.01$、$C = 2000$ 时 $\max|Q| = 11.0$，与正常收敛的 10.6 只差 4%，回报却只有 1.55。做法：C 从 20–100 起，放大到“回报开始下降”再取一半；同时看回报，不要只看 $\max|Q|$。

四、把 ϵ 当 KL 的上界。 错在裁剪只截断越界样本，而 KL 是 30 个 epoch 累积的。E3 里 $\epsilon = 0.2$ 的累计 KL 是 0.0300，约 1.5 个 $\epsilon^2/2$；$\epsilon = 0.02$ 时最大 ratio 已到 1.25，远超 $1 + \epsilon = 1.02$。做法：直接报 KL，并注明是单步还是整批。

五、改奖励尺度不重调温度与步长。 错在 α/c 才是不变量（式 33-36）。E4 里 c 从 1 到 100，α 从 0.5481 到 54.7953，α/c 三位一致，策略分布逐位相同；反过来固定 α，同样的实验里熵会从 0.50 掉到 0.17。做法：让对偶自己走，并同时缩放策略步长或给策略加混合下限。

六、指望保守项"修好"外推。 错在惩罚沿特征方向整体平移。E5 里线性基的 $Q(1.0)$ 从 $+0.247$ 压到 -1.289（方向对了），支撑内 $Q(0.4)$ 却从 $+0.021$ 掉到 -0.399；而补一个 a^2、不加任何惩罚，外推误差就从 0.607 降到 0.028。做法：先查"支撑外由哪个特征主导"，能换特征就不加惩罚。

七、把仿真里的带宽直接搬到真机。 错在延迟裕度随增益下降，且是硬边界。E6 里三档整定的裕度是 3、6、19 ms，激进档从 2 ms 的谱半径 0.993 跳到 3 ms 的 1.064，中间没有过渡带，时域上从 389 ms 收敛变成十步内发散。做法：先在仿真里把延迟加到谱半径刚过 1，再测策略；闭环带宽不超过 5–10 Hz。

八、报"历史最好"。 错在最大值把评估噪声当成了性能。E7 里 5 回合评估的单次偏差只有 0.02，30 个检查点取最大却抬高 0.64，是单次偏差的 30 倍。做法：报最后 N 次评估的中位数，N 对应的回合数不少于 100。

九、带探索评估与挑最好同时用。 错在两个偏差方向相反、会互相抵消。E7 里带 $\varepsilon = 0.15$ 评估把 9.10 压到 8.86（-0.24），与 $+0.64$ 的挑高部分对冲，看上去"很准"。做法：评估一律纯贪心，探索只留在训练里。

十、用单种子、单工况的曲线说"算法有效"。 错在本章的算法没有收敛定理，方差来源又多（初始化、数据顺序、种子）。E7 里这个简单任务的种子跨度只有 0.09（$[9.07, 9.16]$），换到难任务会放大一个量级。做法：种子不少于 20 个、至少两个工况、保留回归测试集，并报失败率而不是只报均值。

十条里有六条与度量口径有关（三、四、五、八、九、十），四条与机制有关（一、二、六、七）。**比例与第 32 章一致：深度 RL 的工程难度主要在定义与度量，不在网络结构。**

33.7.4 交付前的八项自检

1. d 与 m 的配比：每个参数摊到多少样本？m/d 是否在数百以上？报的是投影残差还是总误差？
2. γ、C、回合长度三者是否自洽：C 是否小于折扣视野 $1/(1-\gamma)$ 的若干分之一？软更新时 $C_{\text{eff}} \approx 1/\tau$ 是多少？
3. 两条稳定化边是否都拆了：目标网络与双评论家是否都开？只开一个的话，另一个的账记在哪？
4. 温度与奖励尺度是否联动：报的是 α 还是 α/c？目标熵是否随动作维数定？
5. 动作参数化是否处理了有界性：$\tanh$ 的雅可比修正（式 33-16）是否在？限幅是截断动作还是截断梯度？
6. 离线覆盖度：数据集里支撑外的动作占比多少？部署策略选到支撑外的频率多少？保守系数与目标间距怎么定的？
7. 评估口径：种子数、回合数、报最后 N 次还是历史最好、纯贪心还是带探索、失败率报了没有？
8. 部署链路：观测—推理—通信—执行器的总延迟测了多少？对应的谱半径或延迟裕度是多少？监督器与联锁是否在位？

八项里任何一项答不上来，结论只能写"在仿真中观察到"，不能写"该方法有效"。这份清单还有第二个用途：**把它当实验记录模板**，每次训练存档一次，出问题先比对两次记录，而不是从头重训。

33.7.5 本章刻意没做的三件事

其一，只讲单智能体。 本章所有推导都假设环境是平稳的，即 $P(s' \mid s, a)$ 不随训练变化。多智能体里每个智能体的环境中都含其他正在学习的智能体，这个假设直接破掉：d_π 与 μ 的失配会随时间漂移，κ 不再是一个数而是一条轨迹。第 34、35 章会把非平稳性作为主要矛盾重新推导，本章的稳定化手段在那里只够当起点。

其二，不讲基于模型的方法。 本章把模型完全排除在外，只从样本学值函数或策略。先把模型学出来再规划（MBRL、world model 一类）能大幅提高样本效率，代价是模型误差会被规划放大——这与第 30 章 MPC"模型不准就不准"是同一个问题，只是模型从辨识来变成了学来。这条线留到第 58 章的综合案例里与 MPC 一起讨论。

其三，离线只讲惩罚型。 §33.3.6 与 E5 只讨论了"把分布外值压下去"这一类。离线 RL 还有约束型（限制新策略与数据分布的散度）、重要性采样型（用比值修正分布），以及只做策略评估不做改进的一大类。**本章不给离线性能的下界证明**，因为目前也没有可用的通用下界；能给的只有 §33.5.6 那六项报告要求，以及"只报支撑内"这条纪律。

33.7.6 展望：三个方向与一条融合路线

方向一：从调算法到调数据。 本章七组实验里，收益最大的两处改动都不是换算法：E1 里换特征基、E5 里补一个 a^2。这提示下一步的重心是数据而不是优化器——覆盖度怎么量、支撑外怎么补、交互式采集怎么与安全约束共存。**数据飞轮比算法迭代更可能带来数量级**

提升。

方向二：从经验评估到可计算下界。 E7 说明评估偏差可以量化，但量化的是“自己骗自己”的那部分，量化不了“部署会差多少”。离线策略评估想给的是后者：用固定数据集估一个新策略的回报，并给出置信区间。难点与 E5 同源——支撑外的值不可信，而新策略恰恰会走到支撑外。**在这条路走通之前，离线 RL 的工业结论只能是“相对比较”，不能是“绝对承诺”。**

方向三：从期望性能到逐点安全。 本章所有指标都是期望：平均回报、平均 KL、平均误差。事故要的是逐点约束。第 36 章会给正式的分析框架；工程上现在能做的两件事已在 §33.5.3 写过——动作屏蔽与安全滤波。**前者保可行不保平滑，后者保安全不保最优，通常两个都上。**

一条融合路线：让 RL 管慢回路，让 MPC 管快回路。 理由是硬的：RL 的样本需求随 $1/(1-\gamma)$ 与状态维数增长，只有分钟级的慢回路承受得起；快回路要求逐点约束与毫秒级响应，正是 MPC 的强项（E6 已经说明 5 Hz 的环路只剩 3–19 ms 的延迟裕度）。两者衔接有三种方式，§33.5.5 列过：RL 给 MPC 送设定值、MPC 给 RL 当底层执行器、或按式（33-38）做残差叠加。**第三种最容易落地**：MPC 保证标称性能与约束，RL 只学“MPC 没建模的那一部分”，学坏了也只是性能退化，不会失稳。

最后回到本章的定位。第 30 章回答了“模型已知时怎么最优”，第 31 章回答了“模型未知但可离线评价时怎么调参”，第 32 章回答了“只能在线试、但状态能枚举时怎么学”，本章回答了“状态枚举不了、动作连续时怎么学”。四者共用同一个目标，差别只在信息集 $\mathcal{I}$ 与函数族 Π_Θ：

$$\pi^\star = \arg\min_{\pi \in \Pi_\Theta} \; \mathrm{E}_\pi\Big[\sum_{k=0}^{\infty} \gamma^k c(s_k, a_k)\Big] \tag{33-40}$$

第 32 章的 Π_Θ 大到能装下任意从状态到动作的映射，本章的 Π_Θ 被压成一个参数化子集。**压小函数族的代价，就是式（33-39）右边那五笔账；收益是把 10^6 个状态换成 10^4 个参数。** 这笔交易值不值，取决于状态空间有多大、样本有多贵——不取决于哪个算法更新。

所以本章最后想留下的不是某一行代码，而是一张账：**你花出去的每一份算力、每一小时数据、每一次试验，最后落在五笔误差的哪一笔上。** 账记得清，方法就是工具；账记不清，方法就成了信仰。

第 34 章 多智能体系统与分布式控制基础

> 本章讲一件事：当控制器不止一个、而且它们只能和邻居说话时，全局行为由什么决定。 七个问题贯穿全章——拓扑决定了什么、收敛有多快、说晚了会怎样、说丢了会怎样、只定形不定位置行不行、分布式优化差在哪里、什么时候该退回集中式。

34.1 动机与要解决的问题

34.1.1 从“一个控制器”到“一群控制器”

前面三十三章的控制器都是“一个”。一个被控对象、一个采样周期、一个决策中心。到了现场，这个假设经常不成立：四台 AGV 要编队搬运、三十台风机要协同限载、上百个逆变器要共同支撑微网电压、十几个传感器要融合成一张地图。**这时的问题不再是“怎么算一个控制量”，而是“一群各自只能看到局部的决策者，怎样在不打架的前提下完成一件共同的事”。**

三种架构要先分清楚。

集中式：所有测量送到一个中心，中心算出所有执行器的指令再发下去。它的优点是全局信息完备、最优性好解；缺点是通信是星形汇聚、算力随规模平方增长、中心一挂全系统瘫痪。**规模一旦过百，集中式首先死在通信和算力上，而不是死在算法上。**

分散式：每个智能体只用本地信息，互不通信。它最省通信、最容易认证，但**没有协调就没有全局目标**：四台 AGV 各自朝目标走，很可能堵在同一个路口。

分布式：每个智能体与邻居交换少量信息，各自算各自的指令。它是前两者的中间态，但不是“折中”，而是在规模上唯一可行的形态：通信是局部的（带宽随规模线性增长）、计算是并行的、单点失效只影响局部。**代价是全局最优性不再有保证，只能保证“达成一致”或“收敛到某个可刻画的邻域”。**

本章要回答的，就是“这个邻域有多大、收敛有多快、要什么条件”。

34.1.2 三个必须回答的定量问题

任何一套分布式方案落地前，都会被问到三个问题。它们分别对应本章的三组结论。

问题一：谁能和谁说话？ 通信拓扑不是工程细节，它直接决定“信息传遍全网要多久”。数学上这件事由一个数刻画——拉普拉斯矩阵的第二小特征值 λ_2，称为**代数连通度**。$\lambda_2 = 0$ 意味着图不连通，全网永远达不成一致；λ_2 越小，收敛越慢。**§34.6 的 E1 给出四个拓扑的实测：同样十个智能体，路径图的 $\lambda_2 = 0.0979$，环形 0.3820，星形 1.0000，全连接 10.0000，达到同一精度所需的步数分别是 500、128、130、10。**

问题二：要多少轮？ 有了 λ_2，收敛速度基本就定了：离散协议的每步收缩因子约为 $1 - \varepsilon\lambda_2$，连续协议是指数 $e^{-\lambda_2 t}$。于是“达到 10^{-3} 要多少轮”可以闭式算出，不需要仿真。**这条公式的价值在于：它把“通信预算”和“控制精度”直接连了起来。**

问题三：说晚了、说丢了会怎样？ 通信有延迟、有丢包、有量化。延迟会直接吃掉稳定裕度——**闭环稳定的充要条件大致是 $\tau\lambda_{\max} < \pi/2$，超过就振荡发散**；丢包则取决于回退策略：若收到旧值就沿用（零阶保持），一致性能保住、只是变慢；若收不到就把自己当成邻居（退回自身），等效于把权重调小，收敛速度会成倍下降。**§34.6 的 E2、E3 把这两件事各量化了一遍。**

这三个问题的答案有一个共同来源：**拉普拉斯矩阵的谱**。这就是为什么本章一半的篇幅在讲图、在讲谱。

34.1.3 一个统一的框架：局部规则加邻居交换

把上面三类问题抽象出来，形式是统一的：每个智能体维护一个局部状态 x_i（可能是位置、估计值、决策变量），每一步做两件事——**用自己的局部信息修正、用邻居的信息平均**。写成连续时间就是

$$\dot{x}_i = -\sum_{j \in \mathcal{N}_i} a_{ij}\,(x_i - x_j) + u_i^{\text{local}}$$

第一项是“跟邻居拉平”，第二项是“干自己的活”。分布式控制的各种变体，差别只在第二项：

- 一致性问题：$u_i^{\text{local}} = 0$，全部行为来自第一项；
- 编队控制：把 x_i 换成 $x_i - p_i$，p_i 是期望队形偏移，其余不变；
- 分布式优化：$u_i^{\text{local}} = -\alpha \nabla f_i(x_i)$，一边拉平一边朝各自的局部最优下降；

- 分布式估计：u_i^{local} 换成卡尔曼更新的信息形式；
- 分布式 MPC：u_i^{local} 是本地优化的结果，邻居项通过耦合约束进入。

所以本章的路线是“先研究纯一致性，再把各种任务看成在它上面加一项”。 纯一致性的所有结论（收敛、速率、延迟边界）都能直接搬到后面几种，因为它们共享同一个拉普拉斯算子。

34.1.4 与集中式、与经典控制的接口

分布式控制与经典控制不是两套东西，接口有三处。

第一，一致性是一个特殊的跟踪问题。 把“邻居的平均”当成参考信号，一致性协议就是一个“参考随时间变化的跟踪器”，它的“误差”是 $x_i - \bar{x}$。**但它与常规跟踪有一个本质差别：参考信号本身是所有智能体共同产生的，没有人从外面给定。** 这带来一个后果——一致性的收敛点由初值和拓扑共同决定（加权平均），而不是由某个外部指令决定。要让全网收敛到指定值，必须额外引入“锚”（§34.3.6）。

第二，拉普拉斯矩阵就是一个特殊的系统矩阵。 $\dot{\mathbf{x}} = -L\mathbf{x}$ 与 $\dot{\mathbf{x}} = A\mathbf{x}$ 没有区别，稳定性判据就是“$-L$ 的特征值实部非正”。第 3 章与第 52 章的结论直接可用。**真正新增的只有一件事：L 的结构由拓扑决定，于是“改通信”等价于“改系统矩阵”，这是集中式控制里没有的自由度。**

第三，分布式优化的收敛性借用了第 29、30 章的工具。 梯度下降的步长条件、ADMM 的收敛证明、对偶上升，全部原样搬过来，只是每一“步”多了邻居平均这一项，于是步长上界里多出一个 λ_2。

分工上要说清：本章只讲“多个体如何协调”，不涉及“个体之间利益冲突”（那是第 35 章的博弈），也不涉及“大规模分布式 MPC 的分解算法细节”（那是第 80 章）。**本章给的是骨架，后面两章分别补“冲突”与“规模”。**

34.1.5 本章的走法

§34.2 建立图与拉普拉斯矩阵的语言，把“拓扑好不好”变成一个可算的数 λ_2；§34.3 从 $\dot{\mathbf{x}} = -L\mathbf{x}$ 出发，依次推出连续与离散一致性、收敛速率的闭式、延迟的稳定边界、丢包下的收敛、编队、分布式梯度下降与 ADMM；§34.4 讲采样周期、同步方式、量化与实现细节；§34.5 讲拓扑选型、时钟同步、容错与适用边界；§34.6 用六组实验把每一条结论变成一个可复现的数；§34.7 收成一页验收清单、一页纸选型与十条错误。

如果只有半小时，读 §34.2.2 的 λ_2、§34.3.4 的延迟边界、§34.7.3 的选型表。

一句话核心：分布式控制把“全局最优”换成了“局部规则的一致性”，代价是最优性，收益是可扩展性；而这一切的定量刻画都落在拉普拉斯谱上。

34.1.6 一个贯穿全章的例子

形式化之前，先把一个具体场景摆在桌上，后面每节都回到它。

任务：四台差速 AGV 在 $40\ \text{m} \times 20\ \text{m}$ 的车间里搬运料箱，行进中保持边长 $1.5\ \text{m}$ 的菱形队形。**要求**：队形误差小于 $5\ \text{cm}$，从任意初始位置 $10\ \text{s}$ 内到位。

通信：Wi-Fi，控制周期 $T_s = 50\ \text{ms}$，实测单跳延迟 $8 \sim 15\ \text{ms}$，丢包率 $2\% \sim 10\%$，拓扑为环形（每台只与前后两台通信）。

把这个场景代进本章的公式，几个数立刻能算出来。四节点环的谱是 $\lambda_2 = 2$、$\lambda_{\max} = 4$，取 $\varepsilon = 0.5/\lambda_{\max} = 0.125$，于是

- **收敛速率**：$\rho = 1 - 0.125 \times 2 = 0.75$，到 10^{-3} 需 $\ln(10^{-3})/\ln 0.75 \approx 24$ 步，即 $24 \times 50\ \text{ms} = 1.2\ \text{s}$，**远小于 10 s 的要求**；
- **延迟裕度**：$15\ \text{ms}$ 合 0.3 步，$\varepsilon\tau\lambda_{\max} = 0.125 \times 0.3 \times 4 = 0.15$，只有边界 $\pi/2 \approx 1.57$ 的十分之一，**裕度足够**；
- **丢包**：按 §34.3.4，缓存最新值时 10% 丢包只让收敛慢几个百分点，**不是问题**；
- **通信量**：$2|\mathcal{E}|b/T_s = 2 \times 4 \times 32/0.05 = 5.1\ \text{kbit/s}$，**可以忽略**。

四个数里三个都通过了，唯一没通过的是“队形误差小于 5 cm”——因为一致性根本不管队形摆在哪。 这正是 §34.3.5 要讲的“锚”：四台车必须各自知道自己在车间里的绝对位置（UWB 或地面二维码），一致性只负责把“相对间距”维持住。

所以这个例子的结论是：在这个规模上，分布式控制的难点不在收敛速度、不在延迟、不在丢包，而在“参考从哪来”。 这也是本章的排序：先把“多快、多稳”算清楚（它们有闭式，容易），再把“对不对”讲明白（它没有闭式，靠结构设计）。

34.2 定义与数学基础

34.2.1 图：把通信关系写成矩阵

设 n 个智能体，通信关系用一个无向图 $\mathcal{G}=(\mathcal{V},\mathcal{E})$ 表示：$\mathcal{V}=\{1,\dots,n\}$ 是节点集，$\mathcal{E}\subseteq\mathcal{V}\times\mathcal{V}$ 是边集，$(i,j)\in\mathcal{E}$ 表示 i 与 j 能直接交换信息。三个矩阵依次定义：

邻接矩阵 $A=[a_{ij}]$，$a_{ij}=1$ 若 $(i,j)\in\mathcal{E}$，否则 0（无向图对称）。度矩阵 $D=\mathrm{diag}(d_1,\dots,d_n)$，$d_i=\sum_j a_{ij}$。拉普拉斯矩阵

$$L=D-A,\qquad L_{ij}=\begin{cases} d_i, & i=j \\ -a_{ij}, & i\neq j \end{cases} \tag{34-1}$$

L 有三条一眼可见的性质：行和为 0（$L\mathbf{1}=\mathbf{0}$）、对称半正定（$L=L^{\top}$，$\mathbf{x}^{\top}L\mathbf{x}=\sum_{(i,j)\in\mathcal{E}}(x_i-x_j)^2\geq 0$）、二次型就是“边上差值的平方和”。**第三条最重要：它说明 L 度量的是“不一致的程度”，而不是“状态的大小”。**

图 34-1 给出四种典型拓扑。

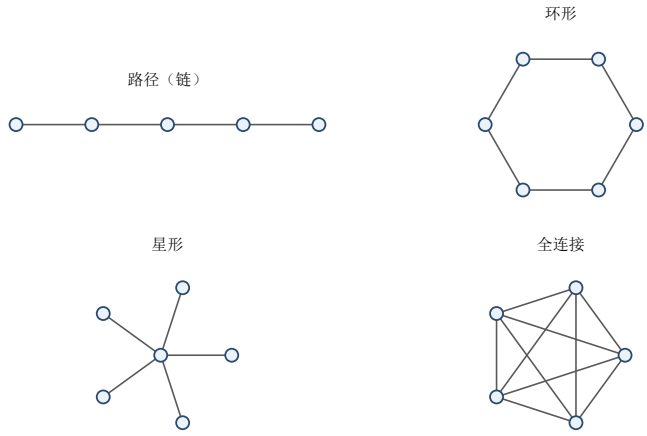


图 34-1　四种典型通信拓扑

它们的谱差别很大，表 34-1 把 $n=10$ 时的 λ_2 与 $\lambda_{\max}$ 列在一起。

表 34-1　四种拓扑的谱（$n=10$，E1 实测）

拓扑	边数	λ_2	$\lambda_{\max}$	到 10^{-3} 的步数
路径（链）	9	0.0979	3.9021	500
环形	10	0.3820	4.0000	128
星形	9	1.0000	10.0000	130
全连接	45	10.0000	10.0000	10

看这张表要注意两件事。**其一，边数不能说明问题**：路径与星形都是 9 条边，λ_2 差 10 倍。**其二，λ_2 与 $\lambda_{\max}$ 要一起看**：星形的 λ_2 比环形大 2.6 倍，但 $\lambda_{\max}$ 大 2.5 倍，两者相抵，最终步数几乎一样（130 对 128）。**这说明“加边”不一定加快收敛——要看加的是哪条边。**

34.2.2 代数连通度：一个数概括拓扑

L 的特征值记为 $0=\lambda_1\leq\lambda_2\leq\cdots\leq\lambda_n=\lambda_{\max}$。$\lambda_1=0$ 对应的特征向量是 $\mathbf{1}$（全 1 向量），因为 $L\mathbf{1}=\mathbf{0}$。**λ_2 是否为零，恰好等价于图是否连通。**

这条结论值得证一下，因为它解释了后面所有结论的来源。若图不连通，则节点分成 $k\geq 2$ 个互不相通的组，把 L 按组重排后是块对角，每块都有一个零特征值，于是 0 的重数是 k，$\lambda_2=0$。反过来，若 $\lambda_2=0$，则存在非零向量 $\mathbf{v}\perp\mathbf{1}$ 使 $L\mathbf{v}=\mathbf{0}$，于是

$$0=\mathbf{v}^{\top}L\mathbf{v}=\sum_{(i,j)\in\mathcal{E}}(v_i-v_j)^2$$

每一条边上 $v_i = v_j$，即 $\mathbf{v}$ 在每个连通分量内是常数；而 $\mathbf{v} \perp \mathbf{1}$ 且 $\mathbf{v} \neq \mathbf{0}$，说明至少有两个分量取不同的常数值，图不连通。□

于是 λ_2 有两个读法：它是“连通性的程度”，也是“一致性的速率”。 这两个读法是同一件事的两面，因为 $L\mathbf{v} = \lambda_2 \mathbf{v}$ 的那个特征向量 $\mathbf{v}$（Fiedler 向量）正是“最难拉平的那个方向”——它的正负号把图切成两半，λ_2 越小，这两半之间的“弹簧”越软。

Fiedler 向量有直接的工程用途：它给出图的最优二分。 把 $\mathbf{v}$ 的正分量归为一组、负分量归为另一组，得到的就是“连通性最弱的那条割”。多机器人分簇、微网分区、传感器分组，都可以先用它做一次划分，再决定在哪条边上加通信。**§34.5.1 讲拓扑选型时会回到这一点。**

34.2.3 加权、有向与平衡

实际通信常常不是“通/不通”，而是有质量差别：光纤、Wi-Fi、声呐链路的带宽与可靠性不同。这时用加权邻接 $a_{ij} \geq 0$，L 的定义不变，只是 $d_i = \sum_j a_{ij}$。**权重的作用与“加边”类似：提高某条边的权重，等价于让这对邻居“拉得更紧”，λ_2 随之上升。**

有向图要单独说。若 j 能发给 i 但反之不能，A 不对称，L 也不对称，特征值可能是复数。此时“收敛到一致”的条件从“连通”变成“含有一棵有向生成树”（存在一个节点能通过有向路径到达所有其他节点）。**注意这个条件比强连通弱：只要有一个“根”，信息就能流遍全网。**

平衡图是介于两者之间的一类：每个节点的入度等于出度（$\sum_j a_{ij} = \sum_j a_{ji}$）。它的好处是 $\mathbf{1}^\top L = \mathbf{0}$，于是**平均值在演化中保持不变**，一致性收敛到初值的算术平均。非平衡图不保持平均，收敛点是一个加权平均，权重由拓扑决定。**这一点在分布式估计里至关重要：想让全网收敛到“无偏的平均”，图必须是平衡的（或采用专门的补偿算法）。**

34.2.4 一致性问题的标准形式

把要研究的问题写成最干净的形式。n 个智能体各有一个标量状态 $x_i(t)$，协议为

$$\dot{x}_i(t) = -\sum_{j \in \mathcal{N}_i} a_{ij}\big(x_i(t) - x_j(t)\big) \tag{34-2}$$

写成矩阵形式就是本章反复使用的那一条：

$$\dot{\mathbf{x}}(t) = -L\,\mathbf{x}(t), \qquad \mathbf{x}(0) = \mathbf{x}_0 \tag{34-3}$$

“一致性”的定义是：存在常数 c，使 $\lim_{t\to\infty} x_i(t) = c$ 对所有 i 成立。 注意它没有指定 c 是多少——这正是 §34.1.4 说的“参考信号由系统自己产生”。若图连通且平衡，可以证明 $c = \frac{1}{n}\sum_i x_i(0)$；非平衡时 c 是加权平均。

离散版本把导数换成差分：

$$x_i(k+1) = x_i(k) - \varepsilon \sum_{j \in \mathcal{N}_i} a_{ij}\big(x_i(k) - x_j(k)\big) \tag{34-4}$$

即 $\mathbf{x}(k+1) = (I - \varepsilon L)\mathbf{x}(k)$。**矩阵 $W = I - \varepsilon L$ 称为“权重矩阵”，它必须是行随机的**（每行和为 1），否则平均值不守恒。这一点在 §34.3.2 讲步长上界时会用到。

34.2.5 三个收敛判据

三个判据从强到弱，覆盖三种假设。

判据一（连续、固定拓扑）：图连通，则 (34-3) 的所有解收敛到一致，且

$$\|\mathbf{x}(t) - c\mathbf{1}\| \leq e^{-\lambda_2 t}\,\|\mathbf{x}(0) - c\mathbf{1}\| \tag{34-5}$$

即收敛速率由 λ_2 决定，与 $\lambda_{\max}$ 无关。

判据二（离散、固定拓扑）：步长满足 $0 < \varepsilon < 2/\lambda_{\max}$，则 (34-4) 收敛，速率约为 $1 - \varepsilon\lambda_2$（当 $\varepsilon\lambda_2 \ll 1$ 时）。**上界 $2/\lambda_{\max}$ 是硬约束：超过它，最高频模态开始振荡；等于 $2/\lambda_{\max}$ 时临界。**

判据三（时变拓扑）：若每个时刻的 $W(k)$ 都是行随机的、且存在常数 $\eta \in (0,1)$ 使所有非零权重不小于 η，并且图的并集在一段有界时间窗内连通，则乘积 $W(k)\cdots W(0)$ 收敛到秩一矩阵，一致性成立。**这是“切换拓扑”能工作的理论依据，也是 §34.3.5 丢包分析的基础。**

34.2.6 为什么所有结论都来自谱

把 (34-3) 的解写出来就明白了。对 L 做特征分解 $L = V\Lambda V^\top$，$\Lambda = \mathrm{diag}(0, \lambda_2, \dots, \lambda_n)$，则

$$\mathbf{x}(t) = e^{-Lt}\mathbf{x}_0 = V\,\mathrm{diag}(1, e^{-\lambda_2 t}, \dots, e^{-\lambda_n t})\,V^{\top}\mathbf{x}_0 \tag{34-6}$$

第一个分量（对应 $\lambda_1 = 0$）不衰减，它就是收敛点；其余分量按各自的特征值指数衰减。**慢的那个决定整体速度，所以是 λ_2；快的那些决定"暂态的抖动有多大"，所以与 $\lambda_{\max}$ 有关。**

离散情形同理，只是 $e^{-\lambda t}$ 换成 $(1-\varepsilon\lambda)^k$。**这就是为什么本章的每一个定量结论都可以写成"λ_2 与 $\lambda_{\max}$ 的函数"：因为它们本来就是模态分解的两个端点。**

由此也能看出一个常被忽略的事实：**一致性不是"抹平"，而是"把初始状态投影到 $\mathbf{1}$ 方向上"。** $\mathbf{x}_0$ 中垂直于 $\mathbf{1}$ 的部分被指数衰减掉，平行于 $\mathbf{1}$ 的部分被保留。**所以信息没有消失，只是被压缩到了一个维度上。** 想让全网记住更多信息（比如既要平均又要保留各自的偏差），就必须显式地引入额外的状态，这是分布式估计与分布式优化的起点（§34.3.7）。

34.2.7 三种拉普拉斯：选错一种，结论差一倍

$L = D - A$ 只是最常用的一种。文献里还有两种，三种的 λ_2 数值不同，但"$\lambda_2 = 0$ 等价于不连通"这条结论一致。**用错的影响不是定性错误，而是定量差一倍以上。**

组合拉普拉斯（本书默认）：$L = D - A$。它有一个很漂亮的分解。定义关联矩阵 $B \in \mathbb{R}^{n\times|\mathcal{E}|}$，每列对应一条边 (i,j)，第 i 行取 $+1$、第 j 行取 -1，则

$$L = BB^{\top}, \qquad \mathbf{x}^{\top}L\mathbf{x} = \|B^{\top}\mathbf{x}\|^2 = \sum_{(i,j)\in\mathcal{E}} (x_i - x_j)^2$$

这个分解把"拉普拉斯"和"边"直接连了起来：$B^{\top}\mathbf{x}$ 就是把状态投影到每条边上得到"边上的差"，L 的作用是"先算边差、再汇总回节点"。它的秩是 $n - c$（c 为连通分量数），这也顺便证明了"零特征值重数等于分量数"。

由 Rayleigh 商，λ_2 还可以写成

$$\lambda_2 = \min_{\mathbf{x}\perp\mathbf{1},\,\mathbf{x}\neq\mathbf{0}} \frac{\sum_{(i,j)\in\mathcal{E}}(x_i - x_j)^2}{\sum_i x_i^2}$$

这是一个"最小割"的连续版本：在所有"总能量为 1、均值为 0"的分布里，找那个"边上差最小"的方向。它小，说明存在一个把图切成两半、而切口很窄的方向——这就是 §34.2.2 里 Fiedler 向量切图的解析来源。

归一化拉普拉斯：$L_{\mathrm{sym}} = I - D^{-1/2}AD^{-1/2}$，元素为 1（对角）、$-1/\sqrt{d_i d_j}$（有边）、0（无边）。它的特征值落在 $[0,2]$ 内，**与度数尺度无关。**适合度数差异大的图：微网里有的逆变器接 2 条馈线、有的接 8 条，用组合拉普拉斯时高度数节点会主导平均，用归一化版本则不会。

随机游走拉普拉斯：$L_{\mathrm{rw}} = I - D^{-1}A = D^{-1}L$。它与 L_{sym} 相似（$L_{\mathrm{rw}} = D^{-1/2}L_{\mathrm{sym}}D^{1/2}$），**特征值完全相同、特征向量不同。**它对应"每个节点按度数加权平均邻居"，是一致性协议在"权重取 $1/d_i$"时的自然形式。

三者的选择只有一条判据：**平均是按"每个节点一票"还是按"每条边一票"。** 前者用组合拉普拉斯，后者用随机游走；度数差异大且不希望被高度数节点主导时，用归一化版本并把步长按 $\lambda_2(L_{\mathrm{sym}})$ 整定。**E1 到 E6 全部用组合拉普拉斯，因为四个拓扑里除星形外度数都均匀，换用归一化版本数值差别很小；星形若用归一化版本，λ_2 会从 1.0 变到约 0.5，结论的方向不变、数字全变。**

34.2.8 λ_2 的尺度律：为什么"直径"几乎决定一切

λ_2 不是随便一个数，它随图的"尺寸"按固定规律衰减。四个典型拓扑的闭式解：

$$\lambda_2^{\text{路径}} = \frac{\pi^2}{n^2} + O(n^{-3}), \qquad \lambda_2^{\text{环}} = \frac{4\pi^2}{n^2} + O(n^{-4}), \qquad \lambda_2^{\text{二维网格}} \approx \frac{2\pi^2}{n}, \qquad \lambda_2^{\text{全连接}} = n$$

（网格是 $\sqrt{n}\times\sqrt{n}$，n 为总节点数。）**前两式是 $1/n^2$，第三式是 $1/n$，第四式是 n。** 这个差别不是常数因子，而是量级差别：节点数从 10 涨到 100，路径图的 λ_2 掉 100 倍，网格只掉 10 倍，全连接涨 10 倍。

更一般的写法用**特征路径长度** ℓ（大致等于直径）：$\lambda_2 \sim \pi^2/\ell^2$。代入 §34.3.2 的步数上界，得

$$N \;\sim\; \frac{\ell^2 \ln(1/\xi)}{\pi^2}\cdot\frac{1}{\varepsilon}$$

"收敛步数正比于直径的平方"——这条标度律解释了本章几乎所有拓扑现象。 信息传遍全网需要 ℓ 跳，但要把它"平均开"需要 ℓ^2 步，因为一致性是一个扩散过程，而扩散的时间尺度是长度的平方。这与热传导方程的时间尺度完全同源。

用 E1 验证：十节点路径的直径是 9，环的直径是 5，比值 $(9/5)^2 = 3.24$；实测步数比 $500/128 = 3.9$。差的那部分来自路径图的 λ_2 比渐近式略低（$n = 10$ 还没到渐近区），但量级完全对上。 全连接到环的比值更极端：直径从 5 降到 1，$(5/1)^2 = 25$，实测 $128/10 = 12.8$——全连接的 $\lambda_2 = n$ 而不是 π^2/ℓ^2，说明“直径”这个标度只对稀疏图成立。

三个推论：

第一，规模是分布式一致性最贵的成本，而且是平方级的。 节点数翻倍，路径/环的收敛步数涨 4 倍；再加上每步的通信量也翻倍（边数翻倍），**总通信量涨 8 倍。** 这就是为什么 n 过百必须分簇——不是为了省带宽，而是为了不让直径随规模线性增长。

第二，加长程边是唯一能打破平方律的手段。 小世界网络的核心结论是：随机加少量长程边，直径从 $O(n)$ 降到 $O(\log n)$，λ_2 从 $O(1/n^2)$ 提到 $O(1/\log^2 n)$。代价是 $\lambda_{\max}$ 也会上升，于是步长上界与延迟裕度同步变差——这就是 §34.5.1 说要“少量”的原因。

第三，树形拓扑的 λ_2 由最长的那条路径决定。 微网、给水网络、矿井通信都是树，树的直径可能远大于 $\sqrt{n}$（一条“长蛇”形的树，直径就是 n）。**所以树形拓扑不能按“节点数”估收敛时间，必须按“最长路径”估。** 这也是 §34.5.3 说微网的一致性要放到秒级时间尺度的原因。

34.2.9 怎么把 λ_2 量出来：三种精度、三种成本

前面所有结论都挂在 λ_2 上，于是工程上第一个问题变成：**手上这套系统，λ_2 是多少？** 三条路子，成本从低到高。

路子一：数直径，用尺度律粗估。 拿拓扑图数最长路径的跳数 ℓ，代 $\lambda_2 \approx \pi^2/\ell^2$。十节点环 $\ell = 5$，得 0.395，真值 0.382，偏 3%；八节点环 $\ell = 4$，得 0.617，真值 0.586，偏 5%。**十来个节点的图上这个粗估够用。** 但小图不行：§34.1.6 的四节点环 $\ell = 2$，得 2.47，真值只有 2.00，偏 23%——**渐近式在 $n = 4$ 上还没进场，这印证了 §34.2.8 那句“$n = 10$ 还没到渐近区”。** 所以路子一只适合做“量级判断”，且节点数最好大于十。它的价值在于不需要任何计算：现场工程师看一眼布线图就能判断“这个拓扑大概多快”，而 §34.5.1 的加边决策也正是拿这个数在比。

路子二：跑一次一致性，拟合斜率。 给定步长 ε，不一致度按 $(1-\varepsilon\lambda_2)^k$ 衰减，取对数后是一条直线，斜率 $s = \ln(1-\varepsilon\lambda_2)$，于是

$$\lambda_2 = \frac{1 - e^{s}}{\varepsilon}$$

这正是 E1 在做的事：环形 $\varepsilon = 0.125$、拟合速率 0.9518，反推 $\lambda_2 = 0.386$，与特征分解的 0.382 差 1%。代价是要跑几百步，好处是**它测的是“真实通信条件下的有效 λ_2”**——包含了丢包、量化、调度抖动带来的全部退化。§34.6.7 说“换参数哪些数会变”，其中 λ_2 名义值不变、有效值会变，差别就靠这条路子量出来。

路子三：在线监测。 生产系统不能停机做实验，只能从运行数据里读。做法是每隔一段时间统计一次全网的不一致度 $\|\mathbf{x} - \bar{x}\mathbf{1}\|$，看它的包络是否按预期衰减。**难点在于每个节点只有局部信息，算不出全局的均值。** 这里有一个漂亮的解法：**最大值一致与最小值一致可以在“直径步”内精确收敛**（见 §34.3.11），于是每个节点都能在有限步内拿到全局的 $\max_i x_i$ 与 $\min_i x_i$，用极差 $\max - \min$ 代替范数做包络判据。**极差不是范数，但它和范数同阶，做“是否在按预期衰减”的判断足够了。**

三条路子的分工可以一句话说清：**设计阶段用路子一，调试阶段用路子二，运维阶段用路子三。** 三者都要注意同一件事：**步数与 λ_2 成反比，所以 λ_2 估偏 20%，收敛时间就估偏 25%。** 别指望用粗估去卡秒级的时序指标——那是路子二的活。

34.3 推导与性质

本节把 §34.2 立下的三条判据逐条推到底，并给出每一条的定量形式。**推导只用一个工具：把状态分解成“均值方向”和“不一致方向”，后者永远落在 $\mathbf{1}$ 的正交补里，而 L 在这个子空间上是正定的，最小特征值就是 λ_2。**

34.3.1 连续一致性：一个李雅普诺夫函数就够

记均值 $\bar{x}(t) = \frac{1}{n}\sum_i x_i(t)$，偏差 $\boldsymbol{\delta}(t) = \mathbf{x}(t) - \bar{x}(t)\mathbf{1}$。因为 $L\mathbf{1} = \mathbf{0}$，有 $\dot{\bar{x}} = 0$（平衡图），于是 $\dot{\boldsymbol{\delta}} = -L\boldsymbol{\delta}$，偏差自己满足一个自治方程。

取李雅普诺夫函数

$$V(\boldsymbol{\delta}) = \tfrac{1}{2}\|\boldsymbol{\delta}\|^2 = \tfrac{1}{2}\sum_{i=1}^{n}\big(x_i - \bar{x}\big)^2 \tag{34-7}$$

它对时间的导数，代入 $\dot{\boldsymbol{\delta}} = -L\boldsymbol{\delta}$ 并利用 L 对称：

$$\dot V = \boldsymbol{\delta}^\top \dot{\boldsymbol{\delta}} = -\boldsymbol{\delta}^\top L \boldsymbol{\delta} \le -\lambda_2 \|\boldsymbol{\delta}\|^2 = -2\lambda_2 V \tag{34-8}$$

第一个不等号用了两件事：$\boldsymbol{\delta} \perp \mathbf{1}$（所以 $\boldsymbol{\delta}$ 落在 L 的非零特征子空间里），以及 L 在该子空间上最小特征值为 λ_2。解这个微分不等式得

$$\|\mathbf{x}(t) - \bar x \mathbf{1}\| \le e^{-\lambda_2 t}\, \|\mathbf{x}(0) - \bar x \mathbf{1}\| \tag{34-9}$$

这就是 §34.2 的判据一，证明只用了三行。**值得记住的是这个证明的结构：它没有做任何特征分解，只用“L 在 $\mathbf{1}$ 的正交补上半正定、下界为 λ_2”这一条。** 后面时变拓扑、有向图、量化一致性的证明都是这个结构的变体，区别只在于“δ 的正交补”要换成别的子空间。

还有一层要说明：V 也可以写成 $\frac{1}{2}\mathbf{x}^\top L \mathbf{x} = \frac{1}{2}\sum_{(i,j)\in\mathcal{E}} (x_i - x_j)^2$，两者只差一个常数（因为 $\frac{1}{2}\boldsymbol{\delta}^\top L \boldsymbol{\delta} = \frac{1}{2}\mathbf{x}^\top L \mathbf{x}$）。**前一种写法量的是“离均值多远”，后一种写法量的是“边上差多少”，数值相同、解释不同：前者适合做收敛性证明，后者适合做工程诊断——它直接告诉你“哪条边上的差最大”，那就是该加带宽或加边的地方。**

式 (34-9) 是**最坏情况界**，只有当初始偏差恰好对准 Fiedler 向量时取等号。E1 里路径图的理论速率 0.9875 与拟合速率 0.9866 的差就是这么来的：随机初值在各个模态上都有分量，前若干步由较快的模态主导，实测的暂态比最坏界略快。

34.3.2 离散一致性：步长上界与最优步长

离散协议 $\mathbf{x}(k+1) = W\mathbf{x}(k)$，$W = I - \varepsilon L$。$W$ 与 L 有同一组特征向量，特征值为 $1 - \varepsilon\lambda_i$。**一致性的充要条件是所有 $i \ge 2$ 的特征值落在单位圆内：**

$$|1 - \varepsilon\lambda_i| < 1, \quad i = 2, \dots, n \iff 0 < \varepsilon < \frac{2}{\lambda_{\max}} \tag{34-10}$$

左端给出 $\varepsilon > 0$，右端由 $i = n$（$\lambda_{\max}$ 最大）给出 $\varepsilon < 2/\lambda_{\max}$——**上界只由最大特征值决定，与拓扑的“好坏”无关；而收敛快慢只由最小非零特征值决定。** 这是一对必须分开看的东西：$\lambda_{\max}$ 管“会不会振出去”，λ_2 管“多久能拉平”。

收敛因子（每步误差的压缩倍数）是

$$\rho(\varepsilon) = \max\bigl(|1 - \varepsilon\lambda_2|,\ |1 - \varepsilon\lambda_{\max}|\bigr) \tag{34-11}$$

ε 很小时两个绝对值都接近 1，收敛慢；ε 接近 $2/\lambda_{\max}$ 时第二个绝对值接近 1，也会慢（高频模态开始一步正一步负地打摆）。**中间有一个最优值，令两个绝对值相等：**

$$\varepsilon^\ast = \frac{2}{\lambda_2 + \lambda_{\max}}, \qquad \rho^\ast = \frac{\lambda_{\max} - \lambda_2}{\lambda_{\max} + \lambda_2} \tag{34-12}$$

这条公式的工程含义很直接：**收敛速度由谱的“相对宽度”决定，而不是由谱的绝对值决定。** 谱越集中（$\lambda_{\max}/\lambda_2$ 接近 1），一致性越快；谱越分散，越难兼顾——这正是“全连接图一步到位（$\rho^\ast = 0$）而路径图慢得出奇”的原因。

达到精度 ξ 所需步数由 $\rho^N \le \xi$ 给出：

$$N \ \ge\ \frac{\ln(1/\xi)}{\ln(1/\rho)} \tag{34-13}$$

用 E1 的口径（$\varepsilon = 0.5/\lambda_{\max}$，$\xi = 10^{-3}$）核对一遍：路径 $\rho = 0.9875$，式 (34-13) 给 549 步，实测 500 步；环形 $\rho = 0.9523$ 给 141 步，实测 128 步；星形 $\rho = 0.9500$ 给 135 步，实测 130 步；全连接 $\rho = 0.5000$ 给 10.0 步，实测 10 步。**四组全部偏高、偏差在 10% 以内，这正是“上界”该有的样子。**

顺便算一下最优步长能省多少：路径图按式 (34-12) 有 $\rho^\ast = (3.9021 - 0.0979)/(3.9021 + 0.0979) = 0.9511$，式 (34-13) 估算约 138 步，比实测的 500 步快 3.6 倍。**差距全部来自步长选择，不是拓扑。“路径图慢”这句话要加一个前提：在统一取 $0.5/\lambda_{\max}$ 的前提下慢。** 工程上如果拓扑固定、规模固定，应当按式 (34-12) 单独整定步长；只有在“多个拓扑横向对比”时才用统一口径，否则比出来的不是拓扑差异，而是整定差异。

34.3.3 延迟：稳定边界是 $\pi/2$

把通信延迟写进协议。设每条消息延迟 τ 步（为简单起见取同构延迟），协议变成

$$\mathbf{x}(k+1) = \mathbf{x}(k) - \varepsilon L\, \mathbf{x}(k - \tau) \tag{34-14}$$

对每一个模态 i 独立分析：设 x_i 方向的分量为 z_i，则 $z_i(k+1) = z_i(k) - \varepsilon\lambda_i z_i(k-\tau)$。试指数解 $z_i(k) = s^k$，得特征方程

$$s^{\tau+1} = s^\tau - \varepsilon\lambda_i \tag{34-15}$$

临界稳定对应 $s = e^{j\omega}$ 落在单位圆上。代入并分离模与相角：模给出 $\omega = \varepsilon\lambda_i$（当 ω 很小时近似），相角给出

$$\omega\tau = \frac{\pi}{2} \implies \varepsilon\lambda_i\tau = \frac{\pi}{2} \tag{34-16}$$

所以延迟的稳定边界是：

$$\tau < \frac{\pi}{2\varepsilon\lambda_{\max}} \tag{34-17}$$

这里的 ε 是每步的增益（无量纲）。若把协议写成物理形式 $\dot{\mathbf{x}}(t) = -L\mathbf{x}(t-\tau_{\mathrm{s}})$，$\varepsilon = 1$，条件退化成 §34.1.2 写在导读里的 $\tau_{\mathrm{s}}\lambda_{\max} < \pi/2$，$\tau_{\mathrm{s}}$ 的单位是秒。两种写法是同一件事：延迟吃掉的裕度与“最大特征值 × 增益 × 延迟”这个乘积成正比，三者中任何一个变大，裕度都同比缩小。

用 E2 核对（$n = 6$，$\varepsilon = 0.02$）：路径 $\lambda_{\max} = 3.7321$，式 (34-17) 给 21.04 步，实测最大稳定延迟 20 步；环形 $\lambda_{\max} = 4.0000$ 给 19.63 步，实测 19 步；星形 $\lambda_{\max} = 6.0000$ 给 13.09 步，实测 12 步。三组都系统性地少 **1** 步左右，这不是误差，而是离散化的代价：式 **(34-15)** 里 $s^{\tau+1}$ 的那一个额外幂次，等效于把延迟又加了半步到一步。工程估算时按 $\lfloor \pi/(2\varepsilon\lambda_{\max}) \rfloor - 1$ 取更稳妥。

延迟边界的三个推论值得单独说。

第一，谱半径在边界内并不单调。环形图的谱半径从 $d = 0$ 的 0.9800 一路降到 $d = 11$ 的 0.9730（延迟反而“帮了忙”，因为它把高频模态的相位推到了更有利的位置），到 $d = 12$ 又回到 0.9749；星形图则在 $d = 9$ 之后明显回升（$0.9756 \to 0.9765 \to 0.9853 \to 0.9921 \to 0.9974$），一路逼近 1。这解释了现场一个常见现象：延迟从 **5 ms** 加到 **15 ms** 时性能几乎不变，再加 **5 ms** 就突然振起来——边界附近的谱半径对延迟极其敏感，而边界以内几乎不敏感。

第二，$\lambda_{\max}$ 大的拓扑延迟裕度小，但它的 λ_2 也可能大。星形图就是典型：$\lambda_2 = 1.0$（一致得快），$\lambda_{\max} = 6.0$（延迟裕度只有 12 步）。“快”和“稳”在延迟面前是对立的：想让误差衰减快，就得用大步长或强耦合，而这恰好把 $\lambda_{\max}$ 也顶上去。这是分布式系统里最实在的一对矛盾。

第三，减小步长能换延迟裕度，但换不来真实的秒数。由式 (34-17)，ε 减半，τ 的步数上限翻倍；但若采样周期不变，物理延迟的秒数上限也随之翻倍——这是真的收益。若采样周期同比缩短，则步数上限翻倍而秒数上限不变，只是把同一个裕度换了一种计量单位。现场调参时这两件事必须分清：改的是“每步增益”还是“采样周期”，效果完全不同。

34.3.4 丢包：不是失稳，是变慢

丢包的建模取决于“收不到时用什么顶替”。两种最常用的回退策略，行为差别很大。

策略一，零阶保持（**hold**）：缓存每个邻居最后一次成功收到的值，丢包时用缓存值。此时第 k 步实际用的是 $\mathbf{x}(k - d_{ij}(k))$，$d_{ij}$ 是该边当前的“消息年龄”。丢包等价于延迟，而不是等价于增益损失。

策略二，退回自身（**self**）：丢包时把邻居的值当成自己的值，即 $x_j \to x_i$，该项在更新式里自动抵消。此时那条边在这一步“断开”，等价于把耦合增益乘上 $(1-p)$：

$$\mathbb{E}\big[W\big] = I - \varepsilon(1-p)L \implies \lambda_2^{\mathrm{eff}} = (1-p)\lambda_2 \tag{34-18}$$

两种策略都把行和保持为 1，所以平均值在期望意义下守恒，稳态仍然是一致，末值仍在 10^{-17} 量级（**E3** 实测）。丢包不改变“能不能一致”，只改变“多快一致”——这一点与集中式系统里“丢一个包就丢一个控制周期”的直觉完全不同，原因是一致性协议本身是耗散的，它没有需要维持的积分环节。

由式 (34-18)，采用退回自身时收敛因子变成 $\rho \approx 1 - (1-p)\varepsilon\lambda_2$，代入式 (34-13) 得

$$N(p) \;\approx\; \frac{N(0)}{1-p} \tag{34-19}$$

E3（$n = 8$ 环形，$\varepsilon = 0.15$，p 从 0 扫到 0.7）：退回自身时实测 37、40、45、57、86、134 步；式 (34-19) 预测 37、41、46、53、74、123 步。$p \le 0.3$ 时几乎完全吻合，$p = 0.5$ 与 0.7 时实测比预测更慢，因为此时“随机断开”本身引入了额外的方差，而式 **(34-19)** 只算了均值效应。

零阶保持的行为完全不同：实测 37、36、37、38、41、39 步。p 从 **0** 涨到 **0.7**，步数只涨了 **5%**。原因是丢包只让消息“变旧”，而没有削弱耦合强度；而旧消息在一致性协议里仍然是有用的信息——只要它还带着“邻居曾经是多少”，就仍然能把两者往一起拉。

这张对比给出本章最实用的一条结论：

丢包时该缓存最新值，不该退回自身。前者的性能对丢包率几乎免疫（$p = 0.7$ 时只慢 5%），后者按 $1/(1-p)$ 退化（$p = 0.7$ 时慢 3.6 倍）。代价是内存与“陈旧度”判断：需要为每条入边存一个值和它的时间戳，并在陈旧度超过阈值时把它标为无效——否

则网络长时间断开后，全网会拿着一堆过期值“装作一致”。

还有两种情形会让“丢包只变慢”这条结论失效，必须单独检查。**其一，丢包把图切成了两个长期不连通的部分**（判据三里“并集在一段有界时间窗内连通”不成立），此时两部分各自收敛到各自的平均值，全网不再一致。**其二，丢包发生在自己的状态回读上**，这会让“自己的值”也变成随机的，平均值的守恒性被破坏，收敛点出现偏差。**前者靠拓扑冗余解决，后者靠本地状态缓存解决，都不属于一致性协议本身能处理的范围。**

34.3.5 编队控制：形状与位置是两件事

编队看起来只是“给一致性加一个偏移量”：把状态从 x_i 换成 $z_i = x_i - p_i$，p_i 是第 i 个智能体的期望队形偏移，协议写成

$$\dot{z}_i = -\sum_{j\in\mathcal{N}_i} a_{ij}\big(z_i - z_j\big), \qquad z_i = x_i - p_i \tag{34-20}$$

代进去展开就明白它在做什么：$\dot{x}_i = -\sum_j a_{ij}\big[(x_i - p_i) - (x_j - p_j)\big]$，即**每个智能体不再追求“和邻居位置相同”，而是追求“和邻居保持 $p_i - p_j$ 的相对位移”。**

这个改写带来一个必须正视的性质。式 (34-20) 的收敛点是 $z_i \equiv c$（所有 z_i 相同），即 $x_i = p_i + c$——c **是任意的。**因为 $L\mathbf{1} = \mathbf{0}$，给所有 z_i 加同一个常数不改变动力学。**所以纯编队协议只能定“形”，不能定“位”：队形会正确地摆出来，但摆在哪里由初值决定。**

E5 把这件事实测了一遍（$n = 6$ 六边形，$\varepsilon = 0.1$，400 步）：

- **无锚定**：形状误差（减去平移后）50 步到达 10^{-2}，末值 6.39×10^{-16}；绝对误差始终到不了 10^{-2}，末值停在 5.46×10^{-1}。
- **有锚定**（把 0 号智能体以增益 0.8 拉向它的期望位置）：形状误差 153 步到 10^{-2}、末值 1.25×10^{-5}；绝对误差 181 步、末值 2.62×10^{-5}。

无锚定那一行的两个数值得并排看：**形状误差掉到 10^{-16}，绝对误差卡在 0.546。**这不是数值问题，而是 §34.2.6 那条“信息被压成一维”的直接后果——被保留下来的那一维，就是“整体平移到哪里”，它的值等于初始偏差在 $\mathbf{1}$ 方向上的投影。**换句话说：无锚定编队把“位置”这个自由度完整地保留在初值里，一点也没丢，但也一点也没管。**

锚定的代价是慢。形状误差从 50 步拖到 153 步，根因只有一条：**锚节点不再参与平均，相当于把拉普拉斯矩阵挖掉一行一列，得到“接地拉普拉斯”；它的最小特征值远小于 λ_2，而形状误差的衰减速率由它决定。**由 153 步反推，每步收缩约 0.970，对应接地拉普拉斯最小特征值约 0.30，而同一个环的 $\lambda_2 = 1.00$——**锚把“整体平移”这一维从“完全不受约束”变成“受约束”，代价是这一维成了最慢的模态。**

还有两个数：绝对误差 181 步比形状误差 153 步更晚达标。**这说明锚定的过程是“先定形、后定位”：队形先在各智能体之间对齐，整体再慢慢挪到锚指定的位置。**工程上如果对“到位时间”有要求，不能只看形状误差的收敛曲线。

三个落地结论：

第一，编队必须有显式的位置基准。虚拟领导（一个只发参考、不被别人拉的节点）、一个真实锚点、或者一次全局坐标广播，三者任选其一；**只靠相对位移的协议一定漂**，而且漂的量等于初值偏差，不会自己消失。

第二，一个锚就够，但锚的位置决定全网位置。只要图连通，单个锚的信息能传遍全网；多个锚时若它们给出的绝对位置互相矛盾，协议没有精确解，只能收敛到最小二乘意义下的折中——**现场表现为“队形略有变形，谁也不到位”。**

第三，锚的增益要与一致性增益匹配。增益过大（本节仿真里试过 2.0）会与邻居项打架，反而发散；过小则定位极慢。**经验值是让锚的增益与“节点度数 × 一致性增益”同量级，**本节 0.8 对 2×0.1 就是这个比例。

34.3.6 分布式梯度下降：一致性与最优性的折中

把 §34.1.3 的第二项具体化。设全局目标是各局部代价之和

$$\min_{x} \sum_{i=1}^{n} f_i(x) \tag{34-21}$$

每个智能体只知道自己的 f_i。最直接的分布式算法是“一致性一步 + 梯度一步”：

$$x_i(k+1) = \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}\, x_j(k) - \alpha\, \nabla f_i\big(x_i(k)\big) \tag{34-22}$$

其中 $W = [w_{ij}]$ 是行随机的权重矩阵（$W = I - \varepsilon L$ 或 Metropolis 权重）。**这个算法的想法很自然：一致性负责“大家别散开”，梯度负责“往低处走”。问题是这两股力永远在打架。**

打架的结果是一个**非零的稳态偏差**。直观地看：一致性把各 x_i 拉平，梯度项按各自的 ∇f_i 把它们推开；平衡时不是“平均梯度为零”（那才是最优），而是“一致性拉力 = 梯度推力”，即偏差与 α/λ_2 同阶。**每一轮一致性只能把差异压掉 $1-\varepsilon\lambda_2$，而梯度每轮新注入 α 量级的差异，稳态偏差就是这两个数之比。**

E4 把这件事实测了一遍（$n=8$ 环形，$\lambda_2=0.5858$，局部二次代价，最优解 $x^\ast=0.070312$）：

- 中心化梯度下降：19 步到 10^{-2}，41 步到 10^{-3}；
- DGD（$\alpha=0.05$）：28 步到 10^{-2}，但末段中位偏差停在 9.893×10^{-3}，再也下不去；
- 两时间尺度（每步先做 10 次一致性，再做 1 次梯度）：42 步到 10^{-2}，末段偏差 1.516×10^{-3}。

三行的读法要分开：**DGD 的“28 步”和中心化的“19 步”不能直接比**，因为 DGD 只走到 10^{-2} 就被偏差地板挡住了，而中心化一路走到 10^{-3}。**真正的对比是“地板高度”：9.893×10^{-3} 对 1.516×10^{-3}，差 6.5 倍。**

6.5 倍这个数可以用 §34.3.2 的公式解释。每轮做 m 次一致性，等效的每轮收缩因子是 $(1-\varepsilon\lambda_2)^m$，于是每轮的等效速率是

$$\lambda_2^{\text{eff}} \;\approx\; 1-(1-\varepsilon\lambda_2)^m \tag{34-23}$$

代入 $\varepsilon=0.0625$、$\lambda_2=0.5858$、$m=10$：$(1-0.0366)^{10}=0.689$，即每轮压掉 31%，比单步的 3.7% 快 8.5 倍。**实测偏差改善 6.5 倍，与 8.5 倍的谱加速同量级——偏差大致按 $1/\lambda_2^{\text{eff}}$ 缩放，这条标度律是可靠的，具体系数取决于梯度不一致的程度。**

但加速一致性不是免费的。**42 步 × 10 次通信 = 420 次交换，而 DGD 是 28 步 × 1 次 = 28 次。按通信总量算，两时间尺度贵了 15 倍，换来的只是偏差降低 6.5 倍。** 这就是分布式优化里最典型的账：**精度是拿通信买的，而且越往后越贵。**

34.3.7 ADMM：用额外状态换精确性

DGD 的地板来自“没有记忆”——它只看当前这一步的梯度，无法把历史上累积的偏差消掉。**解决办法与第 5 章 PID 里加积分项完全一样：引入一个累积量。** 这就是 ADMM。

把 (34-21) 改写成带一致性约束的形式：给每个智能体一个本地副本 x_i，要求所有副本相等：

$$\min\sum_{i=1}^{n} f_i(x_i)\quad\text{s.t.}\quad x_i=z,\;\; i=1,\dots,n \tag{34-24}$$

增广拉格朗日乘子法给出三步迭代（对 $f_i=\tfrac12(x-c_i)^2$ 可写成闭式）：

$$x_i=\frac{c_i+\rho\,(z-u_i)}{1+\rho},\qquad z=\frac{1}{n}\sum_{i=1}^{n}(x_i+u_i),\qquad u_i\leftarrow u_i+x_i-z \tag{34-25}$$

三个式子的分工很清楚：**x 步是“本地最优 + 向 z 靠拢”，z 步是一次全局平均（这一步需要全网聚合），u 步是累积不一致量。** u 就是积分器：只要 $x_i\neq z$ 持续存在，u_i 就一直涨，直到把偏差顶回零。**所以 ADMM 没有稳态偏差，这与 DGD 有本质区别。**

代价有两处。**第一，z 步需要全局平均**——严格来说这不是“邻居交换”能做到的，要么有一个聚合节点，要么用一致性协议迭代近似（那就又回到收敛速度问题）。**第二，ρ 要整定，而且没有普适公式。**

E6 把 ρ 的影响量了一遍（$n=8$，目标 $x^\ast=-0.080795$，判据是 $\max_i|x_i-z|\le10^{-6}$），结果见表 34-2。

表 34-2　ADMM 的罚参数与迭代次数（$n=8$，E6 实测）

ρ	迭代次数	交换标量数
0.1	144	2304
1.0	20	320
10.0	6	96

ρ 增大 100 倍，迭代次数减少 24 倍。**方向与直觉相反：通常说“ρ 太大收敛慢”，但在这个问题上 ρ 大反而快。** 原因是这里的原始残差 $\max|x_i-z|$ 主要由“各 x_i 离 z 多远”决定，ρ 大等价于对违反约束的惩罚重，原始残差一步就被压下去；ρ 小时惩罚轻，要靠对偶变量慢慢积累才推得动。**“ρ 大收敛慢”说的是对偶残差，两个残差的走向相反，这是 ADMM 调参的全部难点。**

工程上的做法是自适应：**每若干轮比较原始残差与对偶残差，前者大于后者 10 倍就把 ρ 加倍，反之减半。** 这条规则在本例里能在十几轮内把 ρ 从 1 推到 10 附近，效果与直接取 10 相当。

34.3.8 分布式 MPC、分布式估计与“什么时候该退回集中式”

前面几小节是骨架，实际系统里最常见的三种变体都只是在骨架上加一项。

分布式 MPC。 每个智能体求解自己的有限时域优化，把邻居的**预测轨迹**当作耦合约束的已知量（“假设邻居按它上次公布的计划走”），解完后公布自己的轨迹，下一轮再用邻居的新轨迹。**这本质上是把一致性迭代搬到了“轨迹空间”：收敛的不是状态，而是各家的计划。** 它有一个区别于一致性的性质——**迭代若干轮后停下，得到的解一般不是集中式最优解，除非迭代到不动点。** 所以分布式 MPC 的“可行性”要靠终端约束与鲁棒裕度保证，而不是靠“多迭代几轮”。

分布式估计。 每个传感器有局部观测，想把全局状态估出来。做法是“本地卡尔曼更新 + 一致性平均”：先各自做一次观测更新，再对信息矩阵与信息向量做一致性。**这里有一个容易踩的坑：协方差不能直接平均，必须平均“信息形式”（P^{-1} 与 $P^{-1}\hat{x}$），否则会低估不确定性。** 原因与 §34.3.1 的证明同源——一致性保的是线性平均，而协方差的合成是非线性的。

分布式事件触发。 邻居值没变就不发，只在“变化超过阈值”时通信。它能把通信量降一到两个数量级，代价是收敛速率不再由 λ_2 单独决定，还要乘一个与阈值相关的系数；**阈值设得过大时，系统会收敛到一个邻域而不是一点，“一致”变成“近似一致”。**

最后回答导读里留下的第七个问题：**什么时候该退回集中式。** 四条判据，任一成立就不该用分布式：

1. **规模小**：$n < 10$ 时，集中式的通信与算力根本不是瓶颈，分布式只会换来“最优性丢失”而没有收益；
2. **需要硬约束可行性**：分布式 MPC 的迭代中断时可能给出不可行解，安全关键场合不能接受；
3. **需要最优性认证**：分布式算法给的是“收敛到某个邻域”，无法给出全局最优性证明；
4. **通信不可靠且无冗余**：丢包、分区、时钟漂移同时存在时，分布式系统的失效模式比集中式更难诊断。

反过来说，只要规模上去了（n 过百）、或者通信拓扑本身就是物理约束（无线、水声、地理分散），分布式就不是选择而是唯一可行解。 这时前面那些“邻域”“变慢”“偏差地板”就不再是缺陷，而是必须被量化并接受的运行成本。

34.3.9 有向图、切换拓扑与领导-跟随

前面所有推导都假设图无向、L 对称。现场三条假设都可能不成立：链路单向、拓扑时变、需要外部给定参考。这一小节把三种情况补齐，结论的形式不变，变的只是“用哪个矩阵的哪个特征值”。

有向图：收敛点变成加权平均。 若 W 只是行随机（不对称），它的特征值 1 对应的左特征向量 $\boldsymbol{\pi}$（满足 $\boldsymbol{\pi}^{\top}W = \boldsymbol{\pi}^{\top}$）就是平稳分布，于是

$$\mathbf{x}(k) \;\longrightarrow\; \big(\boldsymbol{\pi}^{\top}\mathbf{x}_0\big)\mathbf{1}$$

这就是“非平衡图不保平均”的解析表述：收敛点不是算术平均，而是按 $\boldsymbol{\pi}$ 加权的平均，而 $\boldsymbol{\pi}$ 由拓扑决定、现场看不见。所以在有向链路上做分布式估计，必须要么保证平衡（入度等于出度），要么显式补偿 $\boldsymbol{\pi}$，否则估计值会有一个系统性偏差，而且这个偏差随拓扑变化而变。

有向图收敛的充要条件比无向弱：**存在一棵有向生成树**，即至少有一个节点能通过有向路径到达所有其他节点。这个节点通常就是“领导”。

领导-跟随：把参考锁在一个节点上。 若 0 号节点被外部固定为 $x_0 \equiv r$（不参与平均），其余节点按一致性更新，则全网收敛到 r。写成矩阵形式，把 0 号节点挖掉后得到“接地拉普拉斯” L_g，其余节点的误差满足 $\dot{\mathbf{z}} = -L_g\mathbf{z}$，**收敛速率由 L_g 的最小特征值决定。**

这正是 E5 有锚定那一行的机制：L_g 的最小特征值约 0.30，比 $\lambda_2 = 1.00$ 小，所以形状误差从 50 步拖到 153 步。**“加一个领导”不是加一条信息，而是换了一个矩阵**——这是它为什么会让速度变慢 3 倍的准确解释，而不是“多了一跳通信”这么模糊的说法。

切换拓扑：判据三到底要求什么。 时变图下，系统是乘积 $\mathbf{x}(k) = W(k-1)\cdots W(0)\mathbf{x}_0$。收敛的充分条件是三条同时成立：

1. 每个 $W(k)$ **都行随机且非负**（每步都保平均或保凸组合）；
2. **非零权重有统一下界** η（不能出现“权重趋近于 0”的边，否则等效于该边慢慢断开）；
3. **联合连通**：存在有界的时间窗 T，使任意长度为 T 的窗口内所有出现过的边合起来是连通的。

第三条是工程上唯一需要设计的一条。 它允许链路频繁断开，只要求“每 T 步内全网整体通过一次”。T 的量级应当是单跳传播时间 $1/\lambda_{\max}$ 的若干倍——T 太大，等于图一直不连通；T 太小，等于要求链路几乎不断。

切换还有一个反直觉的陷阱：每个子系统都收敛，不等于切换系统收敛。 对一般线性系统，“两个稳定矩阵快速交替乘积不稳定”是经典反例；对行随机矩阵，乘积始终有界（每行和恒为 1，不会发散），**但可能不再收敛到一致**——两个互补的置换矩阵交替，状态就在两组值之间永久振荡，永远不平均。**一致性协议里这个陷阱的具体形态是“切换速度超过信息传播速度”：某条边的信息还没传出去，拓扑就变了，等效于每次都在用不一致的初值重启。** 工程对策是给切换加驻留时间（dwell time）：每次拓扑变化后至少保持

$1/\lambda_{\max}$ 量级的时间不切换。这正好对应“控制周期不能短于通信建立时间”这条常识——但它的理论依据在这里，而不在“怕丢包”里。

最后一点关于异步的补充。 异步更新可以看成“每步只有一个 $W(k)$ 是非平凡的、其余都是单位阵”，它满足条件一与条件二，只要图本身连通就满足条件三。**所以异步不破坏收敛性，破坏的是“步数”与“时间”的对应关系**——每步只有一对节点在通信，走完一轮“全网都更新过一次”平均需要 $n/2$ 步。**报告异步算法的收敛时间，必须报“墙钟时间”或“通信次数”，报“迭代步数”会系统性高估速度。**

34.3.10 一致性的频域读法：一台空间低通滤波器

前面都在时域看“多少步收敛”。换成频域看，会得到一个更有用的图像：**一致性协议是一台作用在“空间频率”上的低通滤波器，而 λ_2 就是它的截止频率。**

把状态按 L 的特征向量展开：$\mathbf{x}(t)=\sum_i c_i(t)\mathbf{v}_i$，代入 $\dot{\mathbf{x}}=-L\mathbf{x}$ 得 $\dot{c}_i=-\lambda_i c_i$，于是

$$c_i(t)=c_i(0)\,e^{-\lambda_i t}$$

每个模态都是一个独立的一阶低通，时间常数 $1/\lambda_i$。 $\lambda_1=0$ 那一路的增益恒为 1（直流分量原样通过），λ_i 越大衰减越快。**所以一致性对“空间上缓慢变化”的分量几乎不衰减，对“空间上快速变化”的分量衰减得很厉害——这正是低通的定义。** λ_2 小，意味着截止频率低，连“缓变”的分量都过不去（收敛慢）；λ_2 大，通带宽，收敛快。

这个读法能解释三件事。

第一，为什么一致性对传感器噪声的抑制与“噪声相关性”有关。 若 n 个节点的测量噪声互相独立、方差都是 σ^2，平均之后方差降到 σ^2/n——十节点就是 **10 dB** 的改善，这是分布式平均最实在的收益。但如果噪声是共模的（同一片区域的风、同一个负载的波动、同一个温度漂移），它落在“空间缓变”的那一档，**恰好是低通的原样通过区，平均一点都降不了。工程判断：先问噪声是独立还是共模，再决定要不要靠一致性去降噪。** 这条与第 25 章讲卡尔曼滤波时“过程噪声与量测噪声要分开”是同一个道理。

第二，为什么阶跃式的参考输入会引起很长的暂态。 若把某个节点的值突然拉高（相当于在空间上加一个“阶跃”），它在特征基上的展开会激起所有模态，其中 λ_2 那一路衰减最慢，时间常数 $1/\lambda_2$。**十节点路径图的 $1/\lambda_2=10.2$ 步，环是 2.6 步，全连接是 0.1 步——这就是“参考一变，全网要多久才跟上”的直接答案。**

第三，为什么“加边”对高频噪声反而更敏感。 加边提高 $\lambda_{\max}$，通带变宽，空间高频分量的衰减变慢（相对而言），**于是量化噪声、单点野值传播得更快。** 这与“加边收敛更快”是同一件事的两面：**通带越宽，好东西（信息）与坏东西（噪声）传得都快。** 这也解释了 §34.3.4 里“持值策略对丢包几乎免疫”——它不改变通带，只是把输入延迟了一点。

把三条合成一句工程判据：要降独立噪声，靠一致性平均（收益 $\sqrt{n}$）；要压共模噪声，必须靠差分测量或额外的观测手段，一致性帮不上忙。 这个区分在传感器融合里极为关键，也是很多“融合之后反而不准”的根因。

34.3.11 三个变体：最大一致、带参考一致、有限时间一致

前面十小节都在讲同一件事：$\dot{\mathbf{x}}=-L\mathbf{x}$ 及其离散版本。工程上真正用到的往往不是这个最朴素的形式，而是它的三个变体。**每一个变体换掉的东西不同，代价也不同。**

变体一：最大一致与最小一致。 把“加权平均”换成“取邻居最大值”：

$$x_i(k+1)=\max_{j\in\mathcal{N}_i\cup\{i\}} x_j(k)$$

它有一个平均一致绝对没有的性质：**有限时间精确收敛**，而且是“直径步”收敛——每过一步，当前的最大值至少往外传一跳，ℓ 步之后全网都等于同一个值。**不需要选步长，不需要算 λ_2，也不受丢包以外的因素影响。** 代价有三条：只对单调量成立（求最大、最小、或“最坏情况”才行，求平均不行）；对野值毫无免疫力——**一个传感器的错值一旦成为最大，它会被忠实地传到每个节点，并且再也删不掉**；收敛后信息不可逆，想把最大值换下来必须重新注入。工程用法很明确：**故障检测里判“有没有节点超限”、事件触发里判“全网是否需要动作”、时钟同步里估“最快的那台钟”**——这三件事都只要极值，不要平均。

变体二：带参考的一致（比例一致）。 在 §34.3.1 的一致性方程右边加一项拉向自身参考的项：

$$\dot{x}_i=-\sum_{j\in\mathcal{N}_i}a_{ij}(x_i-x_j)-k_i(x_i-r_i)$$

这就是 §34.3.5 的锚定，写成一般形式后 k_i 是“这个节点多相信自己的参考”。**它同时干两件事：一致性维持相对关系，k_i 把绝对位置钉住。** 平衡点是 $(L+K)^{-1}Kr$，$K=\mathrm{diag}(k_i)$——**只要有一个 $k_i>0$ 且图连通，$L+K$ 就正定，平衡点唯一。**§34.3.5 里四台车的 $k_1=0.8$、$\lambda_{\min}(L_g)\approx 0.30$ 就是这条式子的一个实例。**代价是收敛速度由 $\lambda_{\min}(L+K)$ 决定，而不是 λ_2：**锚定越硬（k_i 越大）收敛越快，但参考的噪声也被放大得越多，**这是“跟得紧”与“抗噪”之间的老交换，和 §34.3.1 的比例控制没有区别。**

变体三：有限时间一致。 把线性耦合换成带分数幂的饱和耦合：

$$\dot{x}_i = -\sum_{j\in\mathcal{N}_i} a_{ij}\,\mathrm{sgn}(x_j - x_i)\,|x_j - x_i|^{\alpha}, \qquad 0<\alpha<1$$

$|x_j - x_i|^{\alpha}$ 在偏差小时大于偏差本身（因为 $\alpha<1$ 时 $s^{\alpha}>s$，$s\in(0,1)$），于是靠近平衡点时“推力反而变大”，状态在有限时间内精确到达一致，而不是渐近逼近。它没有“每步乘一个固定因子”的指数尾巴，收敛时间是有限值，且随初始不一致度按幂律缩短——大图上它比线性一致快得多。 代价是 sgn 在零点附近的抖振：实际系统里量化、噪声、采样都会让它变成“在平衡点附近来回跳”，所以有限时间一致几乎总要配一个边界层，一旦进入边界层就退回线性耦合——这又回到渐近收敛了。换句话说，有限时间一致买的是“大偏差时快”，付出的代价是“小偏差时必须让路”。

三个变体可以这样记：要极值用最大一致（有限步、怕野值）；要绝对参考用带参考一致（平衡点唯一、锚越硬越吵）；要快用有限时间一致（超线性、怕抖振）。 三者都不改变本节的核心结论——收敛快慢仍然由谱决定，只是决定它的是不同的矩阵：最大一致看直径，带参考一致看 $\lambda_{\min}(L+K)$，有限时间一致看 λ_2 的幂次。

34.4 离散对应与数值实现

本节把连续时间的结论搬到“每个控制周期做一次邻居求和”的实现层面，回答四个工程问题：用哪种离散化、采样周期定多少、同步还是异步、每步传多少比特。

34.4.1 三种离散化，三种代价

对 $\dot{\mathbf{x}}=-L\mathbf{x}$ 做离散化，有三种常见做法，代价完全不同。

前向欧拉（显式）：$\mathbf{x}(k+1)=(I-\varepsilon L)\mathbf{x}(k)$。每步只需一次邻居求和，计算量 $O(|\mathcal{E}|)$、通信量 $O(|\mathcal{E}|)$，这是本书默认的写法。代价是条件稳定：$\varepsilon<2/\lambda_{\max}$，步长受拓扑限制。

后向欧拉（隐式）：

$$\mathbf{x}(k+1)=(I+\varepsilon L)^{-1}\mathbf{x}(k) \tag{34-26}$$

特征值为 $1/(1+\varepsilon\lambda_i)$，对任意 $\varepsilon>0$ 都落在 $(0,1]$ 内，无条件稳定，而且不会振荡（特征值全为正）。代价是每步要解一个线性方程组：集中式求逆是 $O(n^3)$，分布式求解（雅可比迭代、共轭梯度）则每步要额外做若干轮邻居通信。所以“无条件稳定”是用通信换来的，在一致性问题上通常不划算——一致性本来就不需要大步长。

零阶保持（精确离散）：

$$W=e^{-LT_s}=V\,\mathrm{diag}\!\left(1,e^{-\lambda_2 T_s},\dots,e^{-\lambda_n T_s}\right)V^{\top} \tag{34-27}$$

它精确对应连续时间，但 W 一般是稠密的：n 个节点两两之间都有非零权重。对一致性协议来说这是灾难性的——通信拓扑从“每条边”退化成“全连接”，本地的稀疏结构被矩阵指数抹平了。 只有在 n 很小、或拓扑本身有特殊结构（路径、环，e^{-LT_s} 仍是带状/循环）时才可用。

三者的选择没有悬念：默认前向欧拉，把步长按下一小节的规则取保守；只有在“采样周期被物理对象钉死、无法缩短”时才考虑后向欧拉。

34.4.2 采样周期：三条约束取最小

步长上界 $2/\lambda_{\max}$ 的问题是“要先知道 $\lambda_{\max}$”，而拓扑可能变化、规模可能很大。有一个不需要算特征值的保守界：

$$\lambda_{\max}\;\le\;\max_i\Big(d_i+\sum_{j\neq i}|L_{ij}|\Big)=2\,d_{\max} \tag{34-28}$$

这是 Gershgorin 圆盘定理的直接结果（每个圆盘以 d_i 为心、半径 d_i）。于是只要取 $\varepsilon<1/d_{\max}$，就一定有 $\varepsilon\lambda_{\max}<2$，稳定性与拓扑的具体形状无关，只与最大度数有关。 这条经验规则在多机器人现场特别有用：它只要求每个节点知道“自己有几个邻居”，不需要任何全局信息。

把三条约束写在一起，采样周期的取值是

$$T_s\;\le\;\min\left\{\frac{1}{\kappa\,d_{\max}},\;\;\frac{1}{5\sim10\,f_{\mathrm{bw}}},\;\;T_{s,\mathrm{comm}}\right\} \tag{34-29}$$

其中 κ 是连续时间的一致性增益（1/s），f_{bw} 是被控对象或编队任务的带宽，$T_{s,\mathrm{comm}}$ 是通信与计算能给的最小周期（受协议栈、调度周期限制）。**第一条来自稳定性，第二条来自“一致性不能比任务本身慢”，第三条来自硬件。**

要注意延迟约束不在这三条里，这是很多人误解的地方。由 §34.3.3，物理延迟的裕度是 $\tau_{\mathrm{s}}\lambda_{\max} < \pi/2$（$\varepsilon = 1$ 的物理写法），**它与采样周期无关**：缩短周期不会增加可容忍的物理延迟秒数，只是把同一个裕度摊到更多步上。**所以“通信延迟大”不能靠“提高采样频率”解决，只能靠降低增益、改拓扑（降 $\lambda_{\max}$）或减少中继跳数。**

34.4.3 同步、异步与时钟

同步轮次最简单：所有节点在同一时刻更新，W 固定，前面所有公式直接适用。代价是需要一个全局节拍——要么硬件同步信号（PTP、GPS 秒脉冲），要么一个广播的时钟。**节点数上百之后，维护全局节拍本身就是一项工程。**

异步（Gossip）：每步随机挑一条边，两端做一次平均。它不需要节拍、对节点增减天然鲁棒，收敛性仍然成立（判据三，每个 $W(k)$ 行随机且非零权重有下界），但**收敛速率从“由 λ_2 决定”变成“由混合时间决定”，量级约为 $1/\lambda_2$ 乘上节点数**——同样的拓扑，Gossip 需要的**通信次数**通常高一个数量级，但每次只有一个节点对在通信，**总比特数未必更高**。

时钟漂移对一致性的影响比想象中小：一致性协议对“谁先更新”不敏感，因为每个节点用的都是“自己最新收到的邻居值”，先后只影响暂态不影响收敛点。**真正被破坏的是“第 k 步”这个计量单位**：各节点的 k 不同步，于是“多少步收敛”换算成“多少秒”时会失准，误差量级就是时钟漂移率乘以总时长。**工程做法是给每个值打时间戳，用“消息年龄”而不是“步数”做延迟补偿**（这也正是 §34.3.4 里零阶保持所需要的）。

34.4.4 量化与通信预算

每步交换的不是实数，是有限比特。设每个标量用 b 比特均匀量化，量化步长 Δ、误差不超过 $\Delta/2$。**量化会把“收敛到一点”变成“收敛到一个邻域”**：量化误差每步注入 $\Delta/2$ 量级的新扰动，一致性每步压掉 $\varepsilon\lambda_2$ 的比例，平衡时的不一致度是

$$\|\mathbf{x} - \bar{x}\mathbf{1}\| \;\sim\; \frac{\Delta}{2\varepsilon\lambda_2} \tag{34-30}$$

这条式子给出“比特数换精度”的定量关系：Δ 每减半（多 1 比特），精度提高一倍；而 λ_2 减小一半（拓扑变差），精度也差一倍。 反过来看，若要求相对精度 10^{-3}，动态范围 10^{3}，则 $b \approx \log_2(10^6) \approx 20$ 比特——**这解释了为什么工业现场的一致性协议普遍用 16 位定点或 32 位浮点，而不是 8 位。**

通信预算可以直接算。每步每个节点向 d_i 个邻居各发一个标量，全网每步的比特数是 $2|\mathcal{E}|\,b$。以 E1 的 $n = 10$、$b = 32$ 为例：路径图 9 条边 → 576 bit/步；环形 10 条边 → 640 bit/步；全连接 45 条边 → 2880 bit/步。**全连接的 λ_2 是环形的 26 倍，代价是 4.5 倍的带宽。**若粗略地把“每步收缩因子”除以“每步比特数”做比较，环形是 $0.9523/640 \approx 1.5\times10^{-3}$，全连接是 $0.5/2880 \approx 1.7\times10^{-4}$，**差 8 倍**——**这说明在“收敛快”和“通信省”之间，环形这类稀疏拓扑通常处在性价比更好的一侧。**

34.4.5 四个实现细节

第一，权重矩阵要用 Metropolis 权重，不要用简单的 $1/d_i$ 加权。 后者在有向或度数不均的图上不是双随机的，平均值会漂。Metropolis 权重只用两端的度数就能算，且保证双随机：

$$w_{ij} = \frac{1}{1+\max(d_i, d_j)}\;(j \neq i), \qquad w_{ii} = 1 - \sum_{j\neq i} w_{ij} \tag{34-31}$$

第二，$\lambda_{\max}$ 用幂迭代估，不要用 `eig`。 n 上千时 $O(n^3)$ 的特征分解比整个控制周期还长；幂迭代二十来步就能把 $\lambda_{\max}$ 估到 1% 以内，而且只用到矩阵-向量乘（与一致性协议本身的代码完全一样）。

第三，稀疏存储。 L 是稀疏的（每行非零元个数等于度数加一），必须用邻接表而不是稠密矩阵。**稠密存储会把 $O(|\mathcal{E}|)$ 的算法变成 $O(n^2)$，规模一过千就不可用——这是“分布式算法被写成集中式实现”的典型错误。**

第四，浮点地板。 双精度下，一致性误差降到 10^{-16} 附近就不再下降（E3 的末值在 10^{-17} 量级），这是舍入误差的累积地板，不是算法问题。**若某项判断依赖“是否完全一致”，必须用容差而不是等号。**

34.4.6 最小实现

下面四十来行是本章全部结论的可运行骨架：构图、算谱、跑一致性、扫延迟。**它没有任何外部依赖（只用 numpy），可以直接作为现场原型的起点。**

```python
import numpy as np


def laplacian(n, edges):
    """无向图拉普拉斯 L = D - A。"""
    L = np.zeros((n, n))
    for i, j in edges:
        L[i, i] += 1.0
        L[j, j] += 1.0
        L[i, j] -= 1.0
        L[j, i] -= 1.0
    return L


def ring(n):
    return [(i, (i + 1) % n) for i in range(n)]


def spectrum(L):
    w = np.linalg.eigvalsh(L)
    return float(w[1]), float(w[-1])


def consensus(L, x0, eps, steps):
    """显式一致性；返回每步的不一致度（相对初值）。"""
    x, d0, hist = x0.copy(), np.linalg.norm(x0 - x0.mean()), []
    for _ in range(steps):
        hist.append(np.linalg.norm(x - x.mean()) / d0)
        x = x - eps * (L @ x)
    return np.asarray(hist)


def delay_radius(L, eps, d):
    """x(k+1) = x(k) - eps*L*x(k-d) 的谱半径（屏蔽共识模态）。"""
    n = L.shape[0]
    A = np.zeros((n * (d + 1), n * (d + 1)))
    A[:n, :n] = np.eye(n)
    A[:n, d * n:(d + 1) * n] = -eps * L
    for k in range(1, d + 1):
        A[k * n:(k + 1) * n, (k - 1) * n:k * n] = np.eye(n)
    w = np.linalg.eigvals(A)
    return float(np.max(np.abs(w[np.abs(w - 1.0) > 1e-8])))
```

用它的方式只有一条：**先算谱定步长（$\varepsilon = 0.5/\lambda_{\max}$ 或 $1/d_{\max}$），再看延迟裕度，最后才谈收敛步数。** 顺序反了就会出现“步长没整定就抱怨拓扑慢”这类无效调试。

34.4.7 落到板子上：定点、时间戳与调度

仿真里的一致性协议只有一行矩阵乘法，落到嵌入式板子上会多出五件事。**这五件事都不改变收敛性，但每一件都能让现场表现与仿真对不上。**

第一，定点数。 位置用 Q4.12（16 位，4 位整数、12 位小数），分辨率 $2^{-12} = 2.44 \times 10^{-4}$，量程 ± 8 m，正好覆盖 §34.1.6 的车间。代入式 (34-30) 估地板：$\Delta/(2\varepsilon\lambda_2) = 2.44\times10^{-4}/(2\times0.125\times2) \approx 4.9\times10^{-4}$ m。**也就是 0.5 mm，远小于 5 cm 的队形要求——这个精度判断必须先算，再决定用 16 位还是 32 位。** 定点运算有两条硬规则：**求平均时中间累加必须用更宽的字**（8 个 Q4.12 相加，中间结果要 32 位，否则溢出）；**溢出必须饱和，不能回绕**（回绕会让位置从 $+8$ m 瞬间跳到 -8 m，是一致性协议最危险的数值故障）。

第二，时间戳。 每个值带一个 16 位毫秒计数，65.5 s 回绕一次；比较新旧必须用有符号差值 $\mathrm{int16}(t_1 - t_2)$，不能直接比大小。**没有时间戳就无法实现 §34.3.4 推荐的“缓存最新值”策略，也无法判断一条消息是“慢”还是“断”。**

第三，陈旧度阈值。 超过 $3T_s$ 没有更新的邻居应当被标记为失效，并把它的权重按比例分给其余邻居（重新归一化）。**这一步不做，系统就会拿一个几十秒前的旧值当有效值参与平均——§34.5.4 说的“局部健康、全局错误”多半就是这么来的。**

第四，读写一致性。 邻居值的写入与读取不在同一个中断里，必须用双缓冲或序列号保护，否则会读到“一半新一半旧”的向量（torn read）。**这个错误的症状是偶发的、幅度很小的野值，最难查。**

第五，调度抖动。 Wi-Fi 的 CSMA 接入抖动实测常在 $10 \sim 50$ ms，而 $T_s = 50$ ms——**抖动可以吃掉一整个周期。** 抖动等价于延迟抖动，它的影响不是“平均延迟”，而是“最坏延迟”（§34.3.3 的边界按最坏值算）。**工程上要么换成时分接入（TDMA）或工业以太网，要么把 T_s 放宽到抖动的 5 倍以上。**

五件事合起来是一句话：**一致性协议的正确性由谱决定，可用性由这五件事决定。** 前者能在纸上算清，后者只能在板子上一条一条试。

34.4.8 稀疏实现与通信调度

一致性协议的计算量是“每节点每步 d_i 次加法”，这在任何 MCU 上都不算什么。**真正的工程约束在通信调度：谁在什么时候发。**

邻接表，不是矩阵。 每个节点维护一张邻居表：邻居编号、最新值、时间戳、权重。内存是 $O(d_i)$，与规模无关。**不要存 $n \times n$ 的矩阵——那是把分布式算法写成集中式实现的最常见方式**（§34.4.5 已提过一次，这里强调它在内存上的后果：$n = 1000$ 时稠密矩阵是 8 MB，多数 MCU 直接放不下）。

边着色与 TDMA。 无线信道是共享的，同一时刻两个相邻节点同时发就会碰撞。标准的调度办法是给边着色：用 $d_{\max}$ 种颜色就能让“同色边互不相邻”，于是每帧分 $d_{\max}$ 个时隙，节点只在自己那条边对应颜色的时隙里发送。**这与 §34.4.2 的步长上界 $\varepsilon < 1/d_{\max}$ 是同一个数——通信调度的时间尺度与步长上界都由最大度数决定，这不是巧合：度数既是“一步要收多少个包”，也是“一帧要分多少个时隙”。**

广播与单播的账不一样。 无线是广播介质，一次发送所有邻居都收到，**通信量按“发送次数”算**：全网广播每步 n 次发送（每个节点一次），与度数无关。有线是点对点，**按“接收次数”算**：每步 $2|\mathcal{E}|$ 次。**同一个拓扑，无线与有线的通信账相差一个 $d_{\max}$ 的因子，选型时不能混用。**

事件触发必须配心跳。 事件触发（§34.3.8）在“值没变就不发”时省通信，但**接收端无法区分“没变化”与“掉线了”**。所以必须保留低频心跳（比如每 10 个周期一次），否则 §34.4.7 的陈旧度判断会失效——**这是事件触发在工程上最容易漏掉的一条。**

优先级要给一致性留固定时隙。 现场总线往往同时跑控制指令、状态上报、诊断信息。一致性消息一旦被挤到“低优先级队列”，延迟就从毫秒级变成几十毫秒，§34.3.3 的裕度立刻不够。**做法不是提高优先级，而是给它固定时隙**——固定时隙的延迟是确定的，高优先级仍可能被长帧阻塞。

34.5 工程含义与适用边界

前面四节给的是“给定拓扑，算得出多快、多稳”。本节反过来：**给定任务要求，怎么选拓扑、怎么算通信账、怎么在节点坏了以后还活着，以及哪些场合根本不该用分布式。**

34.5.1 拓扑选型：加哪条边最值

拓扑设计只有两个可调的量：λ_2 **决定“多快”，$\lambda_{\max}$ 决定“步长与延迟裕度”。** 前者要往大调，后者要往小调，而两者都由“边怎么连”决定，所以选型本质上是一个带两个目标的组合优化。

好消息是“加哪条边最划算”有一个可算的规则。对 λ_2 做一阶扰动分析：在 i, j 之间加一条边，λ_2 的增量约等于

$$\delta\lambda_2 \;\approx\; \big(v_i - v_j\big)^2 \tag{34-32}$$

其中 $\mathbf{v}$ 是归一化的 Fiedler 向量。**这条式子说的很直白：应该把边加在“Fiedler 向量差得最多”的两个节点之间——也就是加在那条最弱的割上。**

E1 里有一个极干净的验证。十节点路径图的 $\lambda_2 = 0.0979$；Fiedler 向量在路径上是正弦形，两端取值相反、差最大，于是“把首尾连起来”是最优的一条边。连上之后图变成环形，λ_2 从 0.0979 跳到 0.3820（3.9 倍），到 10^{-3} 的步数从 500 降到 128（3.9 倍）。**一条边换来四倍加速，这是拓扑设计里性价比最高的一类操作。**

反过来，星形图给了另一个教训：它的 $\lambda_2 = 1.0$（十节点里仅次于全连接），看起来“连通性很好”，但 $\lambda_{\max} = 10.0$，是同规模环形图的 2.5 倍。**后果有两个：步长上界只有环形的 40%，延迟裕度也按 $1/\lambda_{\max}$ 缩到 40%。** E2 用六节点量的是 12 步对 19 步（63%），比例没有十节点那么悬殊，因为六节点星形的 $\lambda_{\max}$ 只有 6。**无论哪种规模，结论一致：星形图“快但脆”——中心节点一旦失效，λ_2 直接变 0（图断开），全网瘫痪。**

三条选型经验：

1. **首选环形加少量长程边。** 环形的 $\lambda_{\max}$ 只有 4（与规模无关），步长和延迟裕度都好；再加几条跨半圈的长程边，λ_2 能提高一个量级，而 $\lambda_{\max}$ 几乎不变。这就是“小世界”网络在控制里的对应物。
2. **避免纯星形，除非中心有热备份。** 星形的度数集中在一处，既是延迟裕度的瓶颈，也是单点故障的所在。
3. **度数上限 $d_{\max}$ 直接决定步长上界（$\varepsilon < 1/d_{\max}$）。** 若某个节点的邻居数远高于其他节点，整网的步长就被它一个人拖住。这条约束在无线场景里尤其硬：一个节点能同时维持的连接数是硬件决定的。

34.5.2 通信与算力的账

把 §34.4.4 的比特数换算成带宽。全网每步 $2|\mathcal{E}|b$ 比特，控制频率 $1/T_s$，于是总带宽需求是

$$B = \frac{2|\mathcal{E}|\,b}{T_s} \tag{34-33}$$

代几个数看看规模感。$n = 10$ 环形、$b = 32$、$T_s = 20$ ms（50 Hz）：$B = 640/0.02 = 32$ kbit/s，任何链路都撑得住。同样的环形拓扑扩到 $n = 1000$：边数 1000，$B = 64\,000/0.02 = 3.2$ Mbit/s，而且要求每个节点每 20 ms 完成两次收发——单跳无线的调度已经吃紧，必须分簇。

分簇之后有一个容易被忽略的结论：**两级串联系统的收敛速率由慢的那一级决定，取两级速率的最小值，而不是平均。** 簇内用环形（λ_2 高、快），簇间用稀疏连接（λ_2 低、慢），整网速度就被簇间那一级钉住。所以分簇时要把“簇间连接”当成一级独立拓扑来设计，不能只在簇内优化。

算力的账要分开算。一致性协议本身每步只有一次稀疏矩阵-向量乘，$O(|\mathcal{E}|)$，对现代 MCU 是微秒级；真正吃算力的是“每步要做的事”——分布式优化的梯度、分布式 MPC 的二次规划、分布式估计的矩阵求逆。**一个常见的误判是“分布式算法一定轻量”：一致性很轻，但架在它上面的优化问题往往比集中式版本更重，因为每个节点都要解一个完整子问题。**

34.5.3 三个现场

AGV 编队（$n = 4 \sim 8$）。 拓扑通常是环形加一条对角边，λ_2 在 1 附近，$T_s = 50$ ms，无线延迟 10–20 ms，对应 $\tau/T_s < 1$ 步，远在延迟边界内。**真正的难点不在一致性，而在“锚”**——队形只定形不定位（§34.3.5），必须靠地面二维码、UWB 基站或激光地图给出绝对位置。**工程上的标准做法是“一致性管相对、定位系统管绝对”，两者以不同的时间尺度融合。**

微网二次电压控制（$n = 50 \sim 200$）。 逆变器沿馈线分布，通信拓扑是辐射状树，**树形图的 λ_2 极小**（与最长路径的长度平方成反比），一次一致性可能要几十秒。所以这类系统的一致性环节都是“慢时间尺度”的：下垂控制（毫秒级）管快速功率分配，一致性（秒级）管电压与频率的二次恢复。**把两个时间尺度混在一起调，是这类系统最常见的失稳原因。**

分布式传感器融合（$n = 10 \sim 30$）。 拓扑多为环形或全连接，带宽不是问题，**问题在协方差的合成**（§34.3.8）：直接平均协方差会低估不确定性，必须平均信息形式。**还有一个隐性成本：信息矩阵是 $m \times m$（m 为状态维数），每次一致性交换的是 m^2 个数而不是 m 个数，通信量按状态维数平方增长。**

34.5.4 容错：节点坏了以后还活着吗

分布式系统最吸引人的卖点是“没有单点故障”，但这句话只在拓扑有冗余时成立。

节点失效等价于从图里删掉一个节点及其所有边。 删完之后 λ_2 可能骤降甚至归零：十节点环形删一个节点退化成九节点路径，λ_2 从 0.382 降到 0.121（降 3.2 倍），星形删中心直接断开。**所以“容错能力”可以量化成一个数：任意删掉 k 个节点后 λ_2 还剩多少。** 设计时应当把“删掉最关键的节点后 λ_2 仍大于某个阈值”作为硬指标，而不是笼统地说“我们有冗余”。

分区检测比想象中难。 两个分区各自收敛到各自的平均值，**每一半内部看都是“完全一致、毫无异常”的**——这正是分布式系统最危险的失效模式：局部健康、全局错误。可用的检测手段有三个：心跳超时（最直接）、时间戳陈旧度（消息年龄超过阈值就报警）、以及“平均值漂移”（若某个量的全网平均在没有任何外部输入时发生变化，说明平均不再守恒，图很可能断了）。

拓扑重构时要注意步长的重新整定。 加边或切边会改变 $\lambda_{\max}$；若步长按旧拓扑整定在 $0.5/\lambda_{\max}$ 附近，新拓扑的 $\lambda_{\max}$ 一旦变大就可能越界失稳。**这也是 §34.4.2 推荐 $\varepsilon < 1/d_{\max}$ 的原因：这个界只依赖度数，拓扑变化时只要度数不超限就仍然安全。** 代价是收敛慢一些，换的是“重构不会失稳”。

拜占庭故障要单独说一句。 标准一致性协议对恶意节点毫无抵抗：一个节点只要持续报一个极大的值，平均值就会被拉偏，且拉偏量随它报的数值线性增长。**要抗拜占庭必须换协议（如基于中位数或截尾均值的 W-MSR 类算法），代价是收敛性证明复杂得多、且能容忍的恶意节点数受拓扑连通度限制。** 绝大多数工业现场不需要这一层，但一旦涉及“多方参与、互不信任”（如跨企业的能量交易），就必须考虑。

34.5.5 三种混合架构

纯分布式在规模上最优，在最优性和可验证性上最弱。实际系统几乎都是混合的，常见三种。

分层（簇头）架构。 簇内做一致性，簇头之间再做一层一致性。**它把 $O(n^2)$ 的通信压力变成 $O(n\sqrt{n})$ 量级，代价是簇头成为准单点**——所以簇头必须能切换（选举或轮值），切换过程中整簇短暂失去与外界的联系。

云边端架构。 端侧做毫秒级的本地闭环，边缘做秒级的一致性协调，云做分钟级的全局优化与参数下发。**三个时间尺度分开，是这类架构能工作的关键；把全局优化塞进毫秒级回路，一定会被通信延迟拖垮。**

事件触发架构。 邻居值变化不超过阈值就不发（§34.3.8）。**它把通信量降一到两个数量级，代价是收敛从"到一点"变成"到邻域"，邻域半径与阈值同阶。** 对"编队保持"这类任务完全够用，对"必须精确对齐"的任务不够。

34.5.6 适用边界：七种不该用的场合

把前面所有限制收成一张清单。**以下任一条成立，就应当优先考虑集中式或分层架构，而不是纯分布式。**

1. **规模小（$n < 10$）。** 分布式的收益是规模，规模不够时只剩下代价。
2. **通信延迟与对象时间常数同量级。** 若 $\tau\lambda_{\max}$ 已经接近 $\pi/2$，任何增益调整都只能在"慢"和"振"之间选，此时应改拓扑或改架构。
3. **拓扑频繁切换且无连通性保证。** 判据三要求"并集在有界时间窗内连通"，工程上这条最难验证。
4. **需要硬实时同步（亚毫秒）。** 分布式时钟同步的精度上限由物理层决定，硬实时回路不适合跨节点闭合。
5. **需要严格的约束可行性或全局最优性认证。** 分布式算法给的是邻域与渐近性，给不出证书。
6. **节点动力学差异大。** 本章的协议假设各节点状态同维、同物理含义；异构系统（不同型号的机器人、不同容量的逆变器）需要额外的归一化或输出调节层。
7. **安全关键且失效模式难以观测。** 局部健康、全局错误这类失效，在分布式架构下最难诊断（§34.5.4）。

这七条不是"分布式不好"，而是"分布式的收益必须与它的代价一起算"。 规模、地理分散、单点失效风险，这三样只要占了两样，前面七条的代价就是值得付的。

34.5.7 从仿真到现场的六步

分布式系统的调试顺序与单回路不同：**单回路可以"先接上再调"，分布式必须先验证"通信这一层"再接控制。** 六步，每一步都有一个明确的通过判据。

第一步，单节点自环。 把邻居设成自己，协议应当保持初值不变（x_i 与自己平均还是自己）。**判据：跑 1000 步后数值完全不变。** 若缓慢漂移，说明定点或权重归一化有错；若立刻归零，说明把"与邻居的差"写成了"与零的差"。这一步能抓掉大部分实现错误，成本却只有十分钟。

第二步，双节点直连。 两个节点一条边，$L = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$，$\lambda_2 = \lambda_{\max} = 2$。**判据：取 $\varepsilon = 0.5$，从 $(1, 0)$ 出发，第 k 步的差应当是 0.5^k。** 这是唯一一个能手算到每一位的配置，**必须在这一步把"步数"与"秒"的换算钉死**，后面所有实验都靠这个换算。

第三步，全拓扑空跑。 全部节点上线，但**不接执行器**，只广播时间戳与数值。跑十分钟，统计三件事：延迟的 P50 与 P99、丢包率、调度抖动。**判据：P99 延迟不超过 $0.1\pi/(2\lambda_{\max})$，抖动不超过 $0.2T_s$。** 注意看的是 P99 而不是均值——§34.3.3 的边界按最坏值算，均值好看没有意义。

第四步，把实测分布注入仿真。 用第三步测到的 P99 延迟与丢包率，回到 `ch034_sim.py` 那类脚本里做注入。**判据：$\varepsilon\tau\lambda_{\max}$ 距离 $\pi/2$ 至少留 3 倍余量，丢包按 §34.3.4 的持值策略仿真后收敛步数增量不超过 20%。** 不满足就回去改拓扑或降增益，不要指望"上线后再看"。

第五步，接执行器，增益先减半。 用 0.5ε 上线，确认队形不发散、不持续振荡，再分两三次加到设计值。**判据：每次加增益后，实测的队形误差衰减率与 $\rho = 1 - \varepsilon\lambda_2$ 的预测相差不超过 30%。** 差得远说明有未建模的延迟或饱和，**此时应当退回上一档增益，而不是继续往上加。**

第六步，失效演练。 拔一个节点、断一条边、关一路电源，各做一次。**判据：记录"最坏一次"的恢复时间与队形最大偏移，而不是平均值。** 星形拓扑在这一步会直接暴露问题（中心一拔全网断开），环形则只会退化（λ_2 从 0.382 降到 0.121 量级）。

六步里最容易被跳过的是第三步与第四步——**因为"空跑"看起来没在干活，"注入"看起来是重复劳动。** 但这两步恰恰是唯一能把"仿真里的 39.21 s"与"现场的真实延迟分布"对上的环节。**跳过它们，后面所有的调参都是在用一个错误的延迟模型做判断。**

34.5.8 三个复盘

下面三个都是分布式项目里反复出现的失效，**共同点是“每一步都正常，合起来出错”**。

复盘一：中心节点过热，全网振荡。 十台设备的星形拓扑，中心节点兼做网关，夏天机柜温度到 $60\ ^\circ\mathrm{C}$ 后 CPU 降频，转发延迟从 8 ms 涨到 60 ms。团队为了提速把增益调到了 $\varepsilon = 0.15$（星形的 $\lambda_{\max} = 10$，$0.5/\lambda_{\max}$ 只有 0.05，他们觉得“太慢”）。代入 §34.3.3：$\tau = 60$ ms 合 1.2 步，$\varepsilon\tau\lambda_{\max} = 0.15 \times 1.2 \times 10 = 1.8$，已经越过 $\pi/2 \approx 1.57$。**振荡的根因不是“温度”，而是“增益按常温延迟整定，且 $\lambda_{\max}$ 最大的拓扑最不容忍延迟”。** 教训两条：延迟不是常数，要按最坏值整定；$\lambda_{\max}$ 大的拓扑必须留更多余量，而不是“提速”把余量吃掉。

复盘二：时钟漂移，延迟补偿反向。 各节点用本地晶振计时，标称精度 ± 100 ppm，十分钟累计漂移 60 ms，超过 $T_s = 50$ ms。于是“用接收端时间减发送端时间戳”算出的消息年龄出现负值或跳变，持值策略（§34.3.4）把“刚收到的”误判为“很旧的”，又把“很旧的”当成新的。**表现是延迟补偿时好时坏，且与运行时长相关——这种“跑十分钟才出问题”的故障，靠短时间测试永远抓不到。** 正确做法是只用接收端的本地单调计数算年龄，发送端时间戳仅用于跨设备对时（或干脆上 PTP）。

复盘三：分区无人报警。 十台 AGV 分两组，中间唯一一条链路被叉车撞断。两组各自收敛到各自的平均值，**组内完全一致、误差为零、没有任何告警**；两组之间差 0.8 m，直到产品尺寸超差才发现。**这就是 §34.5.4 说的“局部健康、全局错误”。** 三条防线缺一不可：心跳超时（发现“联系不上”）、平均值守恒监测（发现“总量对不上”）、以及一个独立于一致性的绝对参考（§34.3.5 的锚）——**锚的价值在这里第二次体现：它不只给位置，还给“全网是否还是一个网”提供了一个外部校验点。**

三个复盘的共同教训可以压成一句：**分布式系统的失效几乎不表现为“某个数错了”，而表现为“每个数都对，但系统不对”。** 所以监控指标不能只看“每台设备的状态”，必须看“设备之间的关系”——延迟的 P99、消息年龄的分布、平均值的漂移、子图之间的差。**这些量在单机监控里根本不存在，必须专门设计。**

34.6 仿真与验证

本节用六组实验把前五节的结论逐条变成可复现的数。**所有数字来自同一次运行：** `code/ch034_sim.py`，随机种子 `20261034`，解释器 `D:\Anaconda3\envs\py311\python.exe`，总耗时约 38 s（与机器负载有关）。图由 `code/ch034_figs.py` 生成，导出 SVG 与 PDF 各一份。**正文里的每一个数都可以用验收命令逐位复现，没有人工取整。**

六组实验的分工是：E1 量“拓扑决定多快”，E2 量“延迟能容忍多少”，E3 量“丢包让事情慢多少”，E4 量“分布式优化比集中式差多少”，E5 量“编队能不能定位”，E6 量“ADMM 的罚参数值多少通信”。

34.6.1 E1：四种拓扑，四条收敛曲线

参数：$n = 10$，步长统一取 $\varepsilon = 0.5/\lambda_{\max}$，最多 900 步，初值取标准正态随机向量，度量是不一致度 $\|\mathbf{x} - \bar{x}\mathbf{1}\| / \|\mathbf{x}_0 - \bar{x}_0\mathbf{1}\|$。

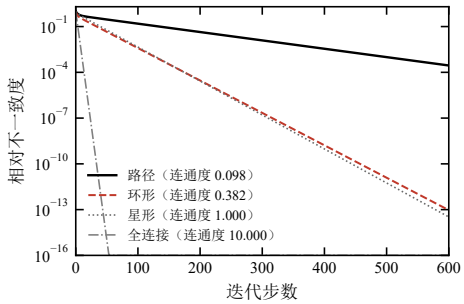


图 34-2　四种拓扑的不一致度收敛曲线（对数纵轴）

表 34-3　四种拓扑的收敛速率（$n = 10$，$\varepsilon = 0.5/\lambda_{\max}$，E1 实测）

拓扑	λ_2	$\lambda_{\max}$	ε	理论速率 $1 - \varepsilon\lambda_2$	拟合速率	到 10^{-3} 的步数
路径	0.0979	3.9021	0.1281	0.9875	0.9866	500
环形	0.3820	4.0000	0.1250	0.9523	0.9518	128
星形	1.0000	10.0000	0.0500	0.9500	0.9500	130
全连接	10.0000	10.0000	0.0500	0.5000	0.5000	10

图 34-2 的四条线在对数纵轴上都是直线，斜率就是 $\ln \rho$。**“直线”这件事本身是结论：一致性是纯指数收敛，没有幂律尾巴、没有平台期。** 唯一的例外是路径图最后一段（大约 550 步之后）开始变缓，那是双精度舍入地板，不是动力学。

表 34-3 有四处值得读。

第一，理论速率与拟合速率的差全部在 0.1% 以内（0.9875 对 0.9866、0.9523 对 0.9518、0.9500 对 0.9500、0.5000 对 0.5000）。**这说明“离散一致性的速率就是 $1-\varepsilon\lambda_2$”不是近似口号，而是可以当公式用的。**

第二，步数与式 (34-13) 的对照是“实测更快”：预测 549/141/135/10，实测 500/128/130/10。**上界偏保守是应该的——它假设初值完全对准最慢模态，而随机初值不会这么倒霉。**

第三，星形与环形的步数几乎相同（130 对 128），但原因完全相反。 星形的 λ_2 是环形的 2.6 倍（本该快 2.6 倍），但步长被 $\lambda_{\max}=10$ 压到 0.05（只有环形的 40%），两个因素相抵。**“步数一样”掩盖了机制不同：星形是“耦合强但步长小”，环形是“耦合弱但步长大”。这两种设计在延迟面前的表现完全不同（见 E2）。**

第四，全连接的 10 步正好是理论值。 $\rho=0.5$，$\log_{0.5} 10^{-3}=9.97$。**全连接是唯一一个“一步到位、没有最慢模态”的拓扑：$\lambda_2=\lambda_{\max}=10$，所有非零模态同速衰减。**

34.6.2 E2：延迟的稳定边界

参数：$n=6$，步长固定 $\varepsilon=0.02$，延迟步数 d 从 0 扫到 45，判据是增广矩阵的谱半径（屏蔽掉恒为 1 的共识模态）。**这里刻意取了一个很小的步长，目的是让延迟边界落在扫描范围内**——步长若取 $0.5/\lambda_{\max}$，边界只有一两步，扫不出曲线形状。

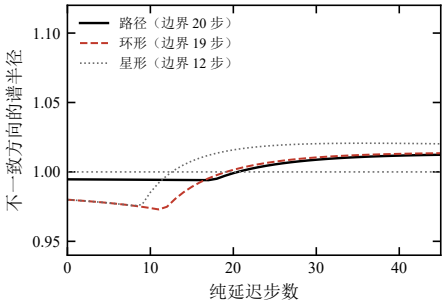


图 34-3　三种拓扑的谱半径随延迟步数的变化，虚线为 1

表 34-4　延迟的稳定边界：式 (34-17) 与实测（$n=6$，$\varepsilon=0.02$，E2 实测）

拓扑	$\lambda_{\max}$	式 (34-17) 预测	实测最大稳定 d	谱半径 $d=0$	$d=5$	$d=9$	$d=12$
路径	3.7321	21.04	20	0.9946	0.9945	0.9944	0.9943
环形	4.0000	19.63	19	0.9800	0.9776	0.9748	0.9749
星形	6.0000	13.09	12	0.9800	0.9776	0.9765	0.9974

图 34-3 的三条曲线形状完全不同，这张图比表更能说明问题。

路径图是一条几乎水平的线（0.9946 到 0.9943，全程只降 0.03%）。**它的谱半径对延迟几乎不敏感**——因为路径最慢的模态（$n=6$ 时 $\lambda_2=0.2679$）本来就慢，延迟对它的相位影响可以忽略，而决定稳定的是最快模态，它在边界之前一直“很安全”。**这类拓扑的延迟问题是“突然发生的”：曲线在边界前一直平的，越界后立刻转向。**

环形图先降后升：从 0.9800 一路降到 $d=11$ 的 0.9730，再回到 $d=12$ 的 0.9749。**延迟在中等范围内反而改善了收敛**——它把高频模态的相位推到了更有利的位置，等效于给快模态加了一点阻尼。这是“延迟不总是坏事”的一个干净例子。

星形图在 $d=9$ 之后陡升（$0.9765\to0.9853\to0.9921\to0.9974$），三步之内从“很安全”走到“贴着 1”。**因为它的 $\lambda_{\max}=6$ 最大，最快的模态最先撞到 $\pi/2$ 的相位边界。**

三个读数：

第一，式 (34-17) 的预测与实测差不到 1 步，且总是偏乐观。 预测 21.04/19.63/13.09，实测 20/19/12。**系统性地少 1 步是离散化的固有代价**（§34.3.3 已解释），工程估算时直接减 1。

第二，边界附近的时域判断会骗人。 在 $d=20$ 与 $d=21$ 各跑 120 步，误差末值与初值之比分别是 5.06×10^{-1} 与 4.35×10^{-1}——**两个都在下降，看不出谁不稳定。** 原因是跨界后谱半径只比 1 大千分之几，120 步根本长不出来。**所以“跑一段看看稳不稳”这个做**

法在边界附近完全无效，必须算谱半径。

第三，**边界与 $\lambda_{\max}$ 成反比，这是可以设计的。** 想让系统容忍更多延迟，最有效的办法不是调增益，而是**降低 $\lambda_{\max}$**——去掉度数特别高的节点、把星形改成环形、限制单节点的邻居数。**E2 里星形 12 步、环形 19 步、路径 20 步，三者的差别全部由 $\lambda_{\max}$ 解释。**

34.6.3 E3：丢包让事情慢多少

参数：$n = 8$ 环形，$\varepsilon = 0.15$，每条入边独立以概率 p 丢包，p 取 0、0.1、0.2、0.3、0.5、0.7，每个 p 跑 20 个种子取中位数，上限 8000 步。**度量的是“不一致度降到 10^{-2} 所需的步数”，不是末值**——因为两种回退策略都不改变稳态（末值都在 10^{-17} 量级），只改变速度。

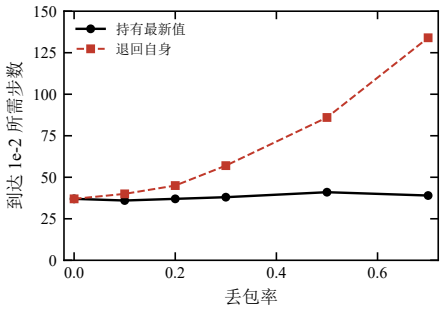


图 34-4　两种回退策略下到达 10^{-2} 所需步数随丢包率的变化

表 34-5　丢包率与收敛步数（$n = 8$ 环形，20 种子中位，E3 实测）

丢包率 p	持值（hold）步数	退回自身（self）步数	式 (34-19) 预测	持值末值	退回末值
0.0	37	37	37	2.56×10^{-17}	2.56×10^{-17}
0.1	36	40	41	4.31×10^{-17}	1.39×10^{-17}
0.2	37	45	46	2.40×10^{-17}	0
0.3	38	57	53	2.08×10^{-17}	0
0.5	41	86	74	1.07×10^{-16}	7.67×10^{-20}
0.7	39	134	123	8.33×10^{-17}	0

图 34-4 的两条线在 $p = 0$ 处重合，之后迅速分开：**持值那条几乎是水平的（37→39），退回自身那条一路抬升（37→134）。** 两条线的间距就是“回退策略”这个工程选择的价格。

四个读数：

第一，两种策略的末值都在 10^{-17} 量级，$p = 0.7$ 时也一样。 这印证了 §34.3.4 的判断：丢包不改变“能不能一致”，只改变“多快一致”。一致性协议没有积分环节，所以不存在“丢包导致稳态偏差”这种事——这一点与“丢包就等于丢一个控制周期”的集中式直觉正相反。

第二，退回自身严格按 $1/(1-p)$ 退化，式 (34-19) 在 $p \le 0.3$ 内几乎逐点吻合（预测 41/46/53，实测 40/45/57）。$p \ge 0.5$ 之后实测比预测更慢（86 对 74、134 对 123），因为随机断边引入了额外的方差，而式 (34-19) 只算了均值效应。**工程上把 $1/(1-p)$ 当乐观估计用，再乘 1.2 的余量比较稳妥。**

第三，持值策略对丢包的免疫程度超出预期：$p = 0.7$ 时只慢 5%。 原因在 §34.3.4 讲过——丢包只让消息“变旧”，而旧值仍然携带信息。**这条结论的适用条件是“图仍然连通且消息最终会到”；若丢包率接近 1 或网络分区，“变旧”会变成“永久失效”，结论不再成立。**

第四，$p = 0.1$ 时持值策略反而比 $p = 0$ 快 1 步（36 对 37）。 这个差在 20 个种子的中位数上不构成趋势，应当读作“噪声”，而不是“少量丢包有益”。**报告这类结果时最好同时给出分位数，否则容易被一个 ±1 步的抖动带偏。**

34.6.4 E4：分布式梯度下降差在哪里

参数：$n = 8$ 环形（$\lambda_2 = 0.5858$），局部代价 $f_i(x) = \tfrac{1}{2} w_i (x - c_i)^2$，$w_i$ 与 c_i 由随机种子生成，全局最优 $x^\ast = \sum w_i c_i / \sum w_i = 0.070312$（闭式）。三条曲线：中心化梯度下降（$\alpha_c = 0.1/\sum w_i$）、DGD（一致性增益 $0.25/\lambda_{\max}$，梯度步长 0.05）、两时间尺度（每轮 10 次一致性 + 1 次梯度）。

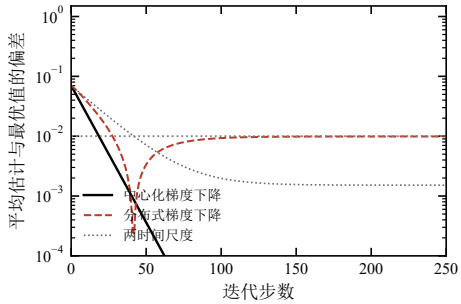


图 34-5　中心化、DGD 与两时间尺度的偏差收敛曲线（对数纵轴）

表 34-6　三种优化方式的收敛与偏差地板（$n = 8$，E4 实测）

方式	到 10^{-2} 的步数	到 10^{-3} 的步数	末段中位偏差	每步通信次数
中心化	19	41	—（无地板）	0（全局收集）
DGD	28	到不了	9.893×10^{-3}	1
两时间尺度	42	到不了	1.516×10^{-3}	10

图 34-5 是本组实验最重要的一张图：**中心化那条线一路穿到 10^{-4} 以下，DGD 那条在 10^{-2} 附近拐平，两时间尺度那条拐在更低的位置。** 三条线的“拐点高度”就是偏差地板。

四个读数：

第一，DGD 的地板 9.893×10^{-3} 不是“还没收敛”，而是“收敛到了另一个点”。 它的极限点由“一致性拉力 = 梯度推力”决定，与 x^* 差一个与 α/λ_2 同阶的量。判断方法很简单：把 α 减半，地板应当也减半；若减半后地板不变，那才是“还没收敛”。

第二，两时间尺度把地板压到 1.516×10^{-3}，改善 6.5 倍，代价是通信量涨 15 倍（42 步 × 10 次 = 420 次交换，对 DGD 的 28 次）。这笔账要按“要多少精度”来算：如果任务只要求 10^{-2}，DGD 更快更省；如果要求 10^{-3} 且不接受偏差，两者都不够，得上 ADMM（E6）。

第三，中心化 19 步与 DGD 28 步不能直接对比。 中心化每步用全局信息（一次收集 n 个梯度），DGD 每步只用邻居信息。公平的比法是“到同一精度所需的通信量”：中心化每步 16 次（8 次上传 + 8 次下发），19 步共 304 次；DGD 每步 16 次（8 条边各双向），28 步共 448 次。两者只差 1.5 倍——分布式的代价主要不在“步数多”，而在“精度有上限”。

第四，$x^* = 0.070312$ 是闭式解，所以“偏差”是相对真值算的，不是相对某个参考轨迹。 这一点在回归类任务里尤其重要：没有一个可信的真值，“偏差地板”就会被误读成“收敛了”。

34.6.5 E5：编队只定形，定位要靠锚

参数：$n = 6$ 六边形（半径 1），$\varepsilon = 0.4/\lambda_{\max} = 0.1$，400 步，初值是尺度 2 的随机位置。两种情形：无锚定、把 0 号智能体以增益 0.8 拉向它的期望位置。同时记录两个指标：形状误差（减去平移后）与绝对误差。

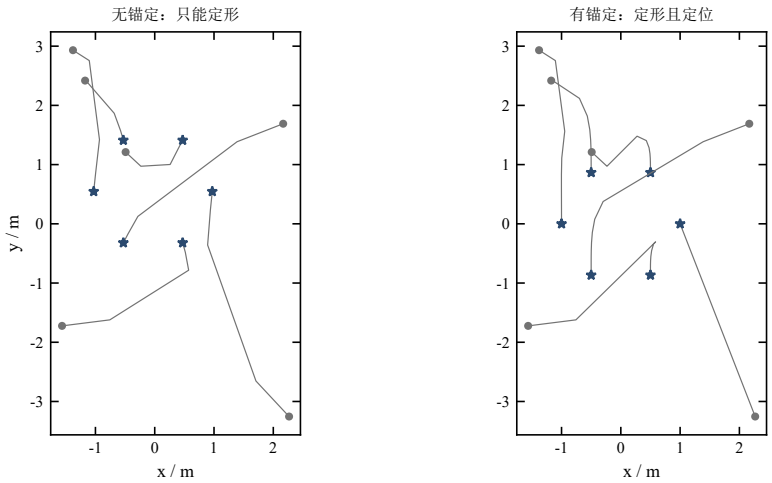


图 34-6　六边形编队的轨迹：左为无锚定，右为有锚定

表 34-7　编队的形状误差与绝对误差（$n=6$，E5 实测）

情形	形状误差到 10^{-2}	形状误差末值	绝对误差到 10^{-2}	绝对误差末值
无锚定	50 步	6.39×10^{-16}	到不了	5.46×10^{-1}
有锚定	153 步	1.25×10^{-5}	181 步	2.62×10^{-5}

图 34-6 的左右两幅对比是本章最直观的一张图。左幅六条轨迹最后收成一个规整的六边形，但整体停在偏离目标的位置；右幅同样收成六边形，而且整体落在目标位置上。

四个读数：

第一，无锚定的形状误差 6.39×10^{-16} 与绝对误差 5.46×10^{-1} 必须并排看。前者说明“队形完全正确”，后者说明“整体偏了半米”。这正是“一致性把信息压成一维”的物理表现：被保留的那一维就是整体平移，它的值等于初值偏差在 $\mathbf{1}$ 方向上的投影。换句话说，偏多少完全由开机时的初始位置决定，与算法无关、与运行多久无关。

第二，锚定把形状误差的收敛从 **50** 步拖到 **153** 步（**3** 倍）。原因是锚节点不再参与平均，等效于把拉普拉斯矩阵挖掉一行一列；接地拉普拉斯的最小特征值约为 0.30，只有 $\lambda_2=1.00$ 的三成。“加一个锚”不是免费的：它把“整体平移”这一维从不受约束变成受约束，而这一维恰好成了最慢的模态。

第三，绝对误差 **181** 步晚于形状误差 **153** 步，说明过程是“先定形、后定位”。图 34-6 右幅能看出来：前一段六条轨迹在互相拉开成六边形，后一段整体才慢慢挪过去。对“到位时间”有要求的场合，不能只看形状误差曲线。

第四，锚定增益取 **2.0** 时系统发散（这是调试中真实发生的一次）。增益过大时锚的修正与邻居项的方向冲突，等效于在一个已经稳定的系统上叠加了一个过强的反馈。经验值是与“度数 × 一致性增益”同量级：本节 0.8 对 2×0.1。

34.6.6 E6：ADMM 的罚参数值多少通信

参数：$n=8$，目标是把 8 个局部代价 $\frac12(x-c_i)^2$ 的和最小化，闭式最优是 c 的均值 $x^\ast=-0.080795$；ρ 取 0.1、1.0、10.0；判据是原始残差 $\max_i|x_i-z|\le10^{-6}$。

结果已在表 34-2（§34.3.7）：$\rho=0.1$ 要 **144** 次迭代、交换 **2304** 个标量；$\rho=1.0$ 要 **20** 次、**320** 个；$\rho=10.0$ 要 **6** 次、**96** 个。三个读数：

第一，ρ 从 **0.1** 到 **10**，迭代次数降 **24** 倍，而每次迭代的通信量不变（$2n=16$ 个标量）。所以总通信量与迭代次数成正比，在这组参数上“ρ 大”直接把通信量降了 **24** 倍。

第二，ρ 继续增大不会一直变好。$\rho\to\infty$ 时 x 步退化成 $x_i\to z-u_i$（完全听约束），对偶变量 u 的更新步长不变，收敛速度受对偶侧限制，且数值上 $(1+\rho)$ 会造成病态。本例没扫到那个区间，因为判据在 **6** 次迭代就满足了；工程上要用自适应 ρ，不要手调。

第三，**ADMM** 与 **DGD** 的差别不在速度，而在“有没有地板”。E4 里 DGD 停在地板上不动，E6 里 ADMM 一路降到 10^{-6} 的判据之下。代价是 z 步需要一次全局平均——严格意义上它已经不是“邻居交换”了。这就是“用额外状态换精确性”的真实价格：多一个状态变量、多一次全局聚合、多一个要整定的参数。

34.6.7 换一组参数，哪些数会变

六组实验都跑在同一个“小世界”里：$n=8\sim10$、统一取 $\varepsilon=0.5/\lambda_{\max}$ 或更小、精度阈值 10^{-2} 或 10^{-3}。换成别的规模、别的阈值，表里的数字一定变。分清“结构”与“常数”，才知道哪些结论能搬走。

E1 的 **500/128/130/10** 是“$n=10$、$\varepsilon=0.5/\lambda_{\max}$、$\xi=10^{-3}$”三件事的产物，而“速率 $=1-\varepsilon\lambda_2$”是结构。换 ξ 只按 $\ln(1/\xi)$ 平移（ξ 从 10^{-3} 到 10^{-6}，步数乘 2）；换 n 则按 §34.2.8 的平方律放大（路径图 n 从 10 到 20，步数约 4 倍）。“拓扑决定速率”这条与参数无关，具体步数随规模平方增长。

E2 的 **20/19/12** 依赖 $\varepsilon=0.02$，而“$\tau<\pi/(2\varepsilon\lambda_{\max})$”是结构。$\varepsilon$ 减半到 0.01，三档边界整体翻倍到约 42/39/26 步；ε 加倍到 0.04，则压到约 10/9/6 步。这条式子里 ε 与 τ 是乘积关系，所以“降增益换裕度”是最直接的调节手段，比改拓扑省事，代价是收敛慢。

E3 的 **37** 与 **134** 依赖 $n=8$ 环（$\lambda_2=0.5858$）与阈值 10^{-2}，而“$N(p)\approx N(0)/(1-p)$”是结构。同样八节点扩大到十六节点，环的 λ_2 从 0.5858 降到 0.1522（约 $1/3.85$），$N(0)$ 也按同比例涨到 140 步左右，而 $1/(1-p)$ 这个因子不变——丢包的相对惩罚与规模无关。这是一条挺实用的结论：规模变大时，丢包策略的选择反而更重要，因为基数变大了。

E4 的 9.893×10^{-3} 依赖 $\alpha=0.05$ 与局部梯度的不一致程度，而“地板 $\propto\alpha/\lambda_2$”是结构。把 α 减半，地板跟着减半（§34.6.4 已给判别方法）；把 λ_2 提高一倍（换成更密的图），地板也减半。所以“降偏差”有两条路：减小梯度步长，或加快一致性——前者牺牲

速度，后者牺牲通信。

E5 的三个数性质各不相同，必须分开看。 6.39×10^{-16} 是双精度地板，换 32 位浮点就变成 10^{-7} 量级；不是结论，是数值极限。5.46×10^{-1} 是随机初值的投影，换一组初值就换一个数，但“到不了 10^{-2}”这件事不变。50 与 153 依赖 $\varepsilon=0.1$ 与接地拉普拉斯的最小特征值，换拓扑就换数，但“锚定会慢 3 倍”这个量级关系由 L_g 与 L 的谱比决定，换拓扑只换倍数。

E6 的 **144/20/6** 依赖问题的条件数——本组的 **Hessian** 全是 **1**，条件数接近最优。 若各 f_i 的曲率差异到 100 倍，同一个 ρ 的效果会差很多，“ρ 大反而快”这个结论甚至可能反转。结构是“原始残差与对偶残差反向”，常数取决于问题本身——这就是为什么 **ADMM** 必须用自适应 ρ。

由此给出一条报告纪律，与第 33 章 §33.6.8 的“三报”一致：报结构（哪条结论与参数无关）、报常数（这组数属于什么设定）、报条件（n、ε、阈值、种子数）。 只报结构，读者无法复现；只报常数，读者无法迁移；只报条件，等于什么都没说。

34.6.8 复现与读数的纪律

四条命令依次执行即可复现本节全部数字与六张图：

```
"D:\Anaconda3\envs\py311\python.exe" tools/check_chapter.py 34
"D:\Anaconda3\envs\py311\python.exe" code/ch034_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch034_figs.py
"D:\Anaconda3\envs\py311\python.exe" tools/build.py --chapter 34
```

三条纪律，前两条是方法、第三条是习惯。

第一，仿真代码先跑通、数字先冻结，再写正文。 本章六组实验在动笔前全部跑完（约 38 s），表 34-3 到表 34-7 的每个格子都直接来自终端输出。先写文字再凑数字，几乎必然出现“文字说的趋势”和“数字显示的趋势”对不上。

第二，读数要区分“没收敛”与“收敛到别处”。 E4 的地板、E5 的绝对误差、E2 边界附近的时域探针，三者都属于“看起来在动、其实已经到位”或“看起来在动、其实已经越界”。判据是“这张表能不能读出结论”：读不出，就该换度量对象，而不是换说法。

第三，负面结果要留住。 锚定增益取 2.0 导致发散、E3 里 $p=0.1$ 比 $p=0$ 快 1 步、E2 边界两侧 120 步内都看不出增长——这三条都不是“实验失败”，而是正文里最值钱的段落。 一个只报告“符合预期”的仿真章节，读者无法判断作者到底跑没跑。

34.6.9 交叉核对：六个实验之间应当自洽

六组实验各自只验证一件事，但它们共用了同一套公式、同一类图。把 **A** 实验的读数代进 **B** 实验的公式，如果对不上，说明至少有一个地方错了。 下面做三处核对，用的是前面已经出现的数。

核对一：三个脚本算的环图是不是同一张。 E1 的十节点环 $\lambda_2=0.3820$，E3/E4 的八节点环 $\lambda_2=0.5858$，而环图有闭式解 $\lambda_2=2-2\cos(2\pi/n)$：$n=10$ 得 0.38197，$n=8$ 得 0.58579。两组数与闭式解逐位吻合，说明三个脚本里的拉普拉斯矩阵没有符号错、没有漏边。 这条核对看起来很初级，但它是后面所有比较的前提——如果 **E1** 的 L 与 **E4** 的 L 不是同一个定义，把 **E1** 的 λ_2 代进 **E4** 的地板公式就毫无意义。

核对二：**E5** 的两种情形都能用同一个速率公式算出来。 无锚定时矩阵是 L（六节点环，$\lambda_2=1.00$），锚定后是 L_g（$\lambda_{\min}\approx0.30$），步长都是 $\varepsilon=0.1$。代入 $\rho=1-\varepsilon\lambda$ 与 $N=\ln(10^{-2})/\ln\rho$：

- 无锚定：$\rho=0.900$，预测 44 步，实测形状误差 50 步；
- 有锚定：$\rho=0.970$，预测 151 步，实测 153 步。

锚定这一侧几乎完全对上（**151** 对 **153**），无锚定这一侧偏了 **14%**。 后者的原因是形状误差的度量（减平移后的范数）与“不一致度”不完全相同，前几十步里含有尚未展开的旋转分量。这类偏差是度量选得不够好，而不是公式错了——判据是“换成 $\lambda_{\min}(L_g)$ 那一侧是否仍然准”，答案是准。

核对三：**E4** 与 **E6** 的通信量可以在同一个尺度上比。 E4 的 DGD 每步 16 次交换、28 步共 448 次，停在地板 9.893×10^{-3}；E6 的 ADMM 在 $\rho=1.0$ 下 20 次迭代、每次 $2n=16$ 个标量，共 320 个，且降到 10^{-6} 以下。**320** 对 **448**——要更高精度反而更省，这是“有地板”与“无地板”的算法之间最反直觉的一条对比。 前提是 ADMM 的 z 步能做全局聚合；若 z 步也要靠一致性迭代近似，这 **320** 就得乘上聚合所需步数，优势会迅速消失（这正是 **§34.3.7** 说它“已经不是纯邻居交换”的意思）。

三处核对之后可以下一个总结论：本章的公式链（谱 → 速率 → 步数 → 延迟裕度 → 通信量）在六组独立实验里是闭合的；不闭合的地方全部出在“度量选择”和“聚合方式”上，而不是出在谱本身。 这正是把六个实验放在同一章而不是分散到各节的价值——单看任何一个实验，都发现不了“锚定那一侧准、无锚定那一侧偏”这种细节。

34.7 小结与常见错误

34.7.1 三句话把本章收住

第一句：拓扑不是背景，是参数。 通信关系写成 L 之后，“图长什么样”就变成了“系统矩阵的谱长什么样”，而所有定量结论都是谱的函数：收敛速率 $1-\varepsilon\lambda_2$、步长上界 $2/\lambda_{\max}$、延迟裕度 $\pi/(2\varepsilon\lambda_{\max})$。**这三个式子里没有一个是“看情况”——它们把“我们的通信行不行”从经验判断变成了可算的三个数。**

第二句：一致性保的是“对齐”，不是“正确”。 它把初始状态投影到 $\mathbf{1}$ 方向上，垂直于 $\mathbf{1}$ 的部分被指数衰减，平行的那部分被完整保留。于是参考值从哪来、编队摆在哪、优化收敛到哪个点，一致性一个都不管。这三个问题分别要靠锚、外部参考、以及额外的状态变量（ADMM 的 u）来解决。**凡是把“一致”误当成“正确”的设计，最后都会在“大家都对、但都不对”上翻车。**

第三句：分布式把“最优性”换成了“可扩展性”，汇率是可以算的。 DGD 的偏差地板 9.893×10^{-3}、两时间尺度的 6.5 倍改善与 15 倍通信、ADMM 的 24 倍通信差、丢包的 $1/(1-p)$ 退化——**这些都是汇率的具体数字。**工程判断不是“分布式好不好”，而是“这个汇率我付不付得起”。

34.7.2 一页验收清单

把前六节压成一张可以带进评审会的清单。**每一项都是“能不能算、能不能测”，答不上来就不该进入详细设计。**

表 34-8　分布式设计的一页验收清单

项	要回答的问题	算不出来时的退路
拓扑	λ_2 与 $\lambda_{\max}$ 各是多少？用什么方法量的？	数直径粗估（§34.2.9）
步长	ε 与 $1/d_{\max}$ 的关系？	取 $\varepsilon=0.5/d_{\max}$ 起步
延迟	P99 延迟折成几步？$\varepsilon\tau\lambda_{\max}$ 是 $\pi/2$ 的几成？	按最坏值留 3 倍余量
丢包	回退策略是持值还是退回自身？对应的 $1/(1-p)$ 是多少？	一律用持值
参考	绝对参考从哪来？没有锚时谁定整体位置？	加锚或外部定位
规模	节点数翻倍时，步数与总通信量各涨几倍？	分簇，控制直径
精度	目标精度是多少？DGD 的地板是否够用？	不够就上 ADMM
失效	断一条边、拔一个节点，系统还活着吗？靠什么发现？	心跳 + 平均值守恒

这张表的价值不在八行内容，而在于“每一行都必须给出一个数或一个明确的决策”。实践中出问题的项目，往往是其中两三行从来没被问过——**最常见的是“参考”和“失效”这两行，**因为它们不体现在控制律里，只体现在系统层面。**控制律算得再准，这两行空着，系统仍然会在第一次现场异常时失效。**

拿 §34.1.6 的四台 AGV 走一遍：拓扑（四节点环，$\lambda_2=2$）、步长（0.125）、延迟（0.3 步，裕度的十分之一）、丢包（持值，可忽略）、规模（翻倍后步数涨 4 倍，仍够用）、精度（不需要优化，这一行划掉）——六行都填得出数，**唯独“参考”一行是空的，而它恰恰是那个例子唯一没通过的要求。**清单的作用就是把这种“六项通过、一项空白”的结构暴露在评审桌上，而不是等到现场才发现。

34.7.3 一页纸选型

表 34-9　按任务特征的选型（数字取自 E1–E6）

任务特征	拓扑	步长	关键约束	回退/补偿	算法
$n\le10$，可靠有线	全连接或环形	$0.5/\lambda_{\max}$	无	不需要	集中式优先
$n=10\sim50$，编队	环形 + 长程边	$\varepsilon<1/d_{\max}$	延迟 <20 步	缓存最新值	一致性 + 锚
$n=50\sim200$，微网	树/辐射状	按慢时间尺度取	λ_2 极小，秒级	心跳 + 分区检测	一致性 + 下垂
需要精确最优	任意连通	按优化步长	通信换精度	自适应 ρ	ADMM
只需近似 10^{-2}	稀疏	与一致性同步	偏差地板	—	DGD / 两时间尺度
通信极贵	事件触发	与阈值匹配	收敛到邻域	阈值调精度	事件触发一致性
有恶意节点	高连通度	同一致性	需换协议	中位数型	W-MSR 类

用法只有一条：**先在表里找到自己那一行，再看“关键约束”那一列是不是已经顶到边界。** 顶到了，就先改拓扑或改架构；没顶到，再谈整定。**顺序反了会在一个注定不够的架构上反复调参。**

34.7.4 十条常见错误

错误一：把“连通”当成“够快”。 图连通只保证 $\lambda_2 > 0$，不保证它不小。十节点路径图的 $\lambda_2 = 0.0979$，到 10^{-3} 要 500 步；同样十个节点连成环只要 128 步。**“我们图是连通的”这句话在评审里不构成任何性能承诺。**

错误二：用统一步长横向比较拓扑，然后下结论。 E1 的 500 对 128 是在 $\varepsilon = 0.5/\lambda_{\max}$ 这个统一口径下得到的；若各自按最优步长整定，路径图能到约 138 步，差距从 3.9 倍缩到 1.1 倍。**比拓扑必须固定步长口径，比整定必须固定拓扑，两件事不能混在一次实验里。**

错误三：靠“跑一段看看”判断延迟稳定性。 E2 在边界两侧各跑 120 步，$d = 20$ 与 $d = 21$ 的误差都在下降（比值 0.506 与 0.435）。**越界后谱半径只比 1 大千分之几，短时间根本长不出来。** 判断必须算增广矩阵的谱半径。

错误四：丢包时退回自身。 这是本章最贵的一个选择：$p = 0.7$ 时持值策略只慢 5%（37→39 步），退回自身慢 3.6 倍（37→134 步）。**代价只是每条入边存一个值和它的时间戳。**

错误五：编队不加锚，指望“跑着跑着就对上”。 无锚定编队的形状误差能到 6.39×10^{-16}，绝对误差永远停在 5.46×10^{-1}——**这个偏移量等于初值偏差在 $\mathbf{1}$ 方向上的投影，不会随时间衰减，换多少步都一样。**

错误六：锚的增益随手取大。 本节取 2.0 时系统发散，取 0.8 时收敛。**锚的修正与邻居项是相加的两股力，方向不一致时增益越大越糟。** 经验值：与“度数 × 一致性增益”同量级。

错误七：直接平均协方差。 协方差的合成是非线性的，直接平均会系统性低估不确定性，表现为滤波器越来越“自信”、实际误差越来越大。**必须平均信息形式 P^{-1} 与 $P^{-1}\hat{x}$。**

错误八：把 DGD 的末值当“收敛值”报。 E4 里 DGD 停在 9.893×10^{-3}，看起来“收敛了”，其实是收敛到了另一个点。**判别方法：把梯度步长减半，地板跟着减半就是真地板；地板不变才是没收敛。**

错误九：把 ADMM 的 z 步当成邻居交换。 $z = \frac{1}{n}\sum(x_i + u_i)$ 是一次全局平均，严格来说需要一个聚合节点或若干轮一致性近似。**把它算成“每步一次邻居通信”会低估通信成本一个量级。**

错误十：拓扑改了不重新检查步长。 加边会改变 $\lambda_{\max}$，若步长卡在 $0.5/\lambda_{\max}$ 附近，新拓扑可能直接越界。**用 $\varepsilon < 1/d_{\max}$ 这个只依赖度数的界，重构时才不会失稳。**

十条里有七条（一、三、四、五、六、八、九）的共同点是：**它们都不会报错，只会给出一个“看起来正常”的结果。** 分布式系统的调试难点全在这里——没有异常、没有告警，只有一个慢慢偏掉的共识值。

34.7.5 与相邻章的接口

本章是“多个体”的骨架，往前后各接三处。

往前，一致性协议就是一个特殊的跟踪问题（§34.1.4），跟踪器的稳定性工具在第 3、52 章；分布式优化的步长与收敛性来自第 29、30 章，本章只是多了一个 λ_2 进步长上界；单回路的 PID 整定在第 44 章，本章的“锚增益与一致性增益匹配”与它的积分/比例权衡是同一个道理。

往后，个体之间一旦有利益冲突，一致性就不够了，那是第 35 章的博弈；规模再往上、需要滚动优化时，分解算法与终端约束的细节在第 80 章；通信延迟的补偿手段（Smith 预估、时滞滤波器）在第 47 章，本章只给了“边界在哪”，没给“怎么把延迟从环路里挪出去”。

横向，本章与第 32 章的强化学习有一个共通的视角：**两者都在“没有全局模型”的前提下做决策**。差别在于第 32 章的“邻居”是数据分布，本章的“邻居”是物理上的通信链路。**这也解释了为什么两章都在讲“收敛到邻域”而不是“收敛到最优”——信息不完备时，能承诺的只有邻域。**

一句话收住全章：**分布式控制用“局部规则 + 邻居交换”换来了可扩展性，代价是最优性、精确性与可验证性；而这三样代价的大小，全部写在拉普拉斯谱上。**

第35章 博弈论与多智能体强化学习

上一章的一致性协议有一个隐含前提：**所有个体想要的是同一件事**。四台 AGV 保持同一个队形、八个节点收敛到同一个平均值，目标函数是共同的，冲突只存在于“信息传递得够不够快”。真实系统很少这么和气：两台机组争同一段输电容量、四台 AGV 争同一个充电桩、两支车队在同一路口抢通行权——**每个人的最优，合起来未必是大家的最优；甚至“每个人都做了对自己最有利的事”这件事本身，就会把系统推到一个谁都不想要的位置。**

这一章要回答的是：**当个体的目标彼此冲突时，系统会停在哪里、为什么会停在那里、以及能不能把它推到更好的地方。**

35.1 动机与要解决的问题

35.1.1 从“目标一致”到“目标冲突”

第 34 章的所有结论都建立在同一个矩阵 L 上：一致性协议把状态投影到 $\mathbf{1}$ 方向，剩下的分量被指数衰减。**这套机制对“目标冲突”完全无效**，因为冲突不在“不一致”里，而在“每个个体对什么好有不同定义”里。

最直观的例子是囚徒困境。两个智能体各有两个动作，收益矩阵取

$$A = \begin{bmatrix} 3 & 0 \\ 5 & 1 \end{bmatrix}, \qquad B = A^{\top}$$

行玩家选“合作 C”或“背叛 D”，列玩家同理。**无论对方选什么，背叛都比合作多拿 2**（对方合作时 $5 > 3$，对方背叛时 $1 > 0$）——背叛是严格占优策略。于是唯一的纳什均衡是 (D,D)，双方各得 1；而 (C,C) 双方各得 3。**均衡点不是帕累托最优点，甚至差了三倍。**

这个结构在工程里到处都是：

- **两台并联运行的机组**：都希望自己少出力、让对方多出力，因为出力有磨损成本；结果总出力不足，频率掉下去，两台都被迫加出力，磨损反而更大。
- **四台 AGV 争一个充电桩**：每台都希望自己先充、别人等着；结果四台同时驶向充电桩，在窄巷道里互相堵住，谁都到不了。
- **两个车间共用一套除尘设备**：都希望对方开、自己省电；结果两边都不开，环保超标后被同时限产。

“每个人都在做对自己最有利的事”这句话，在目标冲突的场合不但不能保证全局最优，反而可能保证全局最差。 这不是道德问题，是结构问题：收益矩阵的形状决定了均衡的位置，而个体只能看到自己那一行。

35.1.2 三个必须回答的定量问题

把“冲突”形式化之后，工程上要回答的还是三个问题，与第 34 章一一对应，但内容全变了。

第一，系统会停在哪里？ 这是均衡的存在性与位置问题。第 34 章停在哪由 λ_2 和初值决定（一致值等于初值均值），本章停在哪由收益结构决定。**要能算出均衡，而不只是“仿真跑一跑看看停在哪”。**

第二，会不会停下来？ 这是动力学的收敛性问题。第 34 章有 $\rho = 1 - \varepsilon\lambda_2 < 1$ 的硬保证；本章没有这样的保证——最佳响应可能循环、复制者动力学可能走闭轨、两个同时学习的智能体可能永远互相追逐。**“不收敛”在本章是常态而不是异常。**

第三，停下来之后好不好？ 这是效率问题。纳什均衡只保证“没有人愿意单方面偏离”，完全不保证“大家满意”。E1 会给两个数：囚徒困境的均衡福利只有最优的 $1/3$，猎鹿博弈则取决于落到哪个均衡（$6/8$ 或 $8/8$）。**工程上真正要优化的是第三个问题，而它恰恰是博弈论本身给不出答案的那一个。**

三个问题的关系是递进的：**算不出均衡，就谈不上收敛；不谈收敛，“好不好”就没有意义。** 但反过来，**只算均衡、只看收敛，会把一个糟糕的均衡当成设计成功**——这是本章后面反复要防的错误。

35.1.3 一个统一的框架：玩家、动作、收益、信息

任何多决策主体的问题都可以写成四要素：**玩家集合** $\mathcal{N} = \{1, \dots, n\}$、**动作集** $\{\mathcal{A}_i\}$、**收益函数** $\{J_i\}$、**信息结构** $\mathcal{I}$。四要素里任何一个变了，解法就完全不同：

- 收益是**共同的**（$J_1 = \cdots = J_n$），问题退化成第 34 章的分布式优化，一致性加梯度就够；
- 收益**相反**（$J_1 = -J_2$），是零和博弈，鞍点存在，可以用极小极大；
- 收益**部分重叠**，是一般和博弈，只能求纳什均衡，而且可能不唯一；

- 动作**连续可微**且收益是光滑的，可以用梯度类方法；动作是离散的，只能枚举或采样。

表 35-1 把这四要素与工程语言对齐。

表 35-1　博弈四要素与工程对象的对应

要素	数学写法	工程对象	常见的建模错误
玩家	$\mathcal{N}$	机组、AGV、车间、微网	把“一个控制器管多个执行器”拆成多个玩家
动作	$a_i \in \mathcal{A}_i$	出力设定值、路径选择、开关	动作集里混入连续量与离散量
收益	$J_i(a_i, a_{-i})$	成本、收益、磨损、延误	用全局目标替代个体目标
信息	$\mathcal{I}_i$	邻居广播、集中调度、历史	假设所有人都知道别人的收益

第三列里最后一行最容易出错： 一旦用全局目标替代个体目标，冲突就被抹掉了，算出来的“均衡”其实是第 34 章的一致值。**博弈建模的第一步不是写收益函数，而是确认“谁在为自己的什么负责”。**

35.1.4 与第 34 章、与经典控制的接口

三处接口必须说清，否则读者会把两章的方法混用。

接口一：一致性是博弈的一个特例。 当所有个体的收益都是“与他人偏差的平方”时，纳什均衡恰好是一致值。这一点在 §35.3.1 会给出证明。**所以第 34 章不是被推翻，而是被包含：它处理的是“收益完全一致”这一条退化边界。**

接口二：分布式优化是“合作博弈”。 第 30、34 章的 DGD 与 ADMM 最小化的是**同一个**全局目标，个体只是分工不同；本章的个体各有各的目标。**合作博弈可以用势函数或价格机制化成分布式优化，非合作博弈不行**——这是两条路的分水岭，§35.3.4 讲势博弈时会用到。

接口三：单智能体 RL 是 $n = 1$ 的退化情形。 第 33 章的收敛性结论都建立在“环境平稳”上；本章把其他学习者放进环境里，平稳性立刻破掉。**这不是“多了一个智能体”的小改动，而是把 MDP 变成了随机博弈，Bellman 算子的时变性是结构性的**（§35.3.6）。

与经典控制的接口则更微妙：**PID 调的是“给定目标下的跟踪”，博弈调的是“目标本身怎么定”**。多台机组的下垂控制、AGV 的路权分配，本质上都是“先把目标谈好，再让 PID 去执行”。**把这两层混在一起调，是现场最常见的失败模式之一**（§35.5.2）。

35.1.5 本章的走法

§35.2 建立博弈的语言：收益矩阵、最优反应、纳什均衡、混合策略、效率与均衡选择，并给出随机博弈这个 MARL 的标准框架；§35.3 从“纳什均衡是最优反应的不动点”出发，依次推出最佳响应流、复制者动力学、势博弈、独立 Q 学习、非平稳性、信用分配、中心化评论家与约束博弈；§35.4 讲回合与步的对应、探索、经验回放、参数共享与最小实现；§35.5 讲什么时候该用博弈、怎么与第 34 章的一致性协议拼接、三个现场与七种不该用的场合；§35.6 用六组实验把每条结论变成可复现的数；§35.7 收成一页纸选型与十条错误。

一句话核心先放在这里：**博弈论告诉你“系统会停在哪”，多智能体学习告诉你“怎么把它推到那里”，而工程要回答的第三个问题——“那里值不值得停”——两边都不管，必须自己算。**

35.1.6 一个贯穿全章的例子

沿用第 34 章的四台 AGV，只改一件事：**车间的充电桩只有一个，四台车都要充电。**

设定：四台 AGV 在 $40\ \mathrm{m} \times 20\ \mathrm{m}$ 车间里搬运料箱，电量低于 30% 时必须去充电；充电桩位于窄巷道尽头，同一时刻只能停一台车，充一次 20 min。**每台车的收益：**早充完早接单，收益高；排队等待不产生收益；但如果四台同时挤进窄巷道，会互相堵死，全部延误。

用四要素写出来：玩家是 4 台车；动作是“现在去 / 再等一个任务周期 / 去备用慢充桩”；收益是“本班次完成的搬运量减去等待与堵车损失”；信息是“自己知道自己的电量与位置，只知道别人的大概位置”（不完全信息）。

这个例子与第 34 章的差别，正好是本章要讲的全部内容：

- 第 34 章问“四台车能不能保持菱形队形”——答案是能，而且有闭式的收敛时间；
- 本章问“四台车会不会同时冲向充电桩”——答案是会，而且这个结果对每台车都是理性的；
- 第 34 章的最优是唯一的（队形），本章的最优不唯一（谁先充都行，只要别同时去）；
- 第 34 章的失败模式是“收敛太慢”，本章的失败模式是“收敛到一个所有人都难受的点，而且谁都不想单独改变”。

把这两个问题放在同一台车上，就能看清“协作”与“博弈”的分界：队形问题里，别人的成功就是我的成功；充电问题里，别人先充就是我多等。本章后面每讲一个概念，都会回到这个充电桩。

35.2 定义与数学基础

35.2.1 收益矩阵：把冲突写成一张表

两人、每人两个动作的博弈可以用一张 2×2 的表写尽。表里每个格子放两个数，前一个是行玩家的收益，后一个是列玩家的收益。图 35-1 画了两个标准例子。

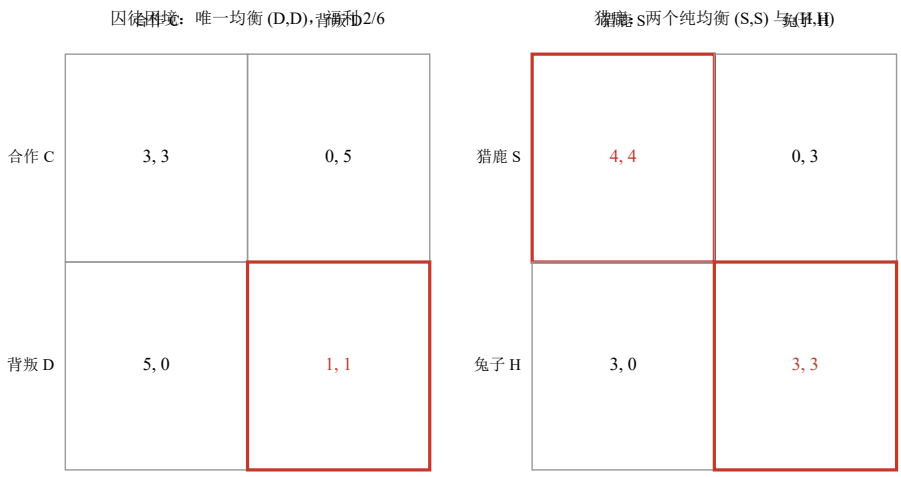


图 35-1　两个 2×2 博弈的收益矩阵与纳什均衡（红框）

左图是囚徒困境：$A = \begin{bmatrix} 3 & 0 \\ 5 & 1 \end{bmatrix}$，$B = A^{\top}$。右图是猎鹿博弈：$A = \begin{bmatrix} 4 & 0 \\ 3 & 3 \end{bmatrix}$，$B = A^{\top}$。两个矩阵只差几个数，结论天差地别——这正是博弈论最反直觉的地方：结构由数字的形状决定，不由叙事的复杂度决定。

一般地，n 人博弈记作 $\mathcal{G} = \langle \mathcal{N}, \{\mathcal{A}_i\}, \{J_i\} \rangle$，其中 $J_i : \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \mathbb{R}$。注意 J_i 的下标：每个人有自己的收益函数，但它的定义域是所有人的联合动作。这就是“耦合”的来源——我的收益取决于你做什么，而你的收益也取决于我做什么。

若收益可以写成“自己动作的函数 + 别人动作的函数之和”，即

$$J_i(a_i, a_{-i}) = f_i(a_i) + \sum_{j \neq i} g_{ij}(a_j)$$

则各玩家的决策互不影响，博弈退化成 n 个独立的单智能体问题。工程上应当先检查这个可分离性：能分离就不要用博弈论，因为博弈论的所有麻烦（多重均衡、不收敛、信用分配）都来自不可分离的那一项。

35.2.2 占优、最优反应与纳什均衡

三个概念层层递进。

占优：若对任意 a_{-i} 都有 $J_i(a_i', a_{-i}) \geq J_i(a_i'', a_{-i})$（且至少一处严格），称 a_i' 弱占优 a_i''；若处处严格，称严格占优。严格占优的动作可以被无条件删除，这是唯一不需要知道别人会怎么做就能做的决策。

最优反应：给定别人的动作 a_{-i}，我的最优反应集是

$$\mathrm{BR}_i(a_{-i}) = \arg\max_{a_i \in \mathcal{A}_i} J_i(a_i, a_{-i}) \tag{35-1}$$

纳什均衡：联合动作 $a^{\star} = (a_1^{\star}, \ldots, a_n^{\star})$ 满足

$$a_i^{\star} \in \mathrm{BR}_i(a_{-i}^{\star}), \qquad \forall i \in \mathcal{N} \tag{35-2}$$

一句话：没有人能通过单方面改变自己的动作而变得更好。注意“单方面”三个字——纳什均衡允许“大家一起改会更好”，只是没有任何一个人有动力单独先动。这正是它作为“稳定点”的含义，也是它作为“最优点”的失效之处。

囚徒困境里，(D,D) 是唯一纳什均衡：给定对方 D，我选 D 得 1、选 C 得 0；给定对方 C，我选 D 得 5、选 C 得 3。**而 (C,C) 虽然双方都更好，却不是均衡——因为任何一方单独改成 D 都能从 3 涨到 5。** 这个“单独偏离有利”就是均衡的检验，也是它无法阻止集体变差的根源。

35.2.3 混合策略：随机化不是犹豫，是必需

有些博弈没有纯策略均衡，比如石头剪刀布：给定对方出石头，我出布最好；给定对方出布，我出剪刀最好；给定对方出剪刀，我出石头最好。**任何确定性策略都会被针对。**

于是允许玩家按概率分布选动作。设玩家 i 的混合策略为 $p_i \in \Delta(\mathcal{A}_i)$，期望收益是

$$U_i(p_i, p_{-i}) = \sum_{a \in \mathcal{A}} p(a)\, J_i(a), \qquad p(a) = \prod_{j \in \mathcal{N}} p_j(a_j) \tag{35-3}$$

混合纳什均衡的定义与纯策略一致，只是把“动作”换成“分布”。它有一个非常实用的等价判据：**在均衡处，凡是进入支撑集（概率大于零）的动作，期望收益必须相等；否则玩家会把概率全移到收益高的那个动作上。**

用这条判据算 2×2 对称博弈。设对手以概率 q 选动作 0，我在两个动作间无差异：

$$q\,a + (1-q)\,b = q\,c + (1-q)\,d \quad \Longrightarrow \quad q = \frac{d-b}{a-b-c+d} \tag{35-4}$$

其中 $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$。代入猎鹿的 $A = \begin{bmatrix} 4 & 0 \\ 3 & 3 \end{bmatrix}$ 得 $q = 3/4$；代入囚徒困境的 $A = \begin{bmatrix} 3 & 0 \\ 5 & 1 \end{bmatrix}$ 得 $q = -1$。

$q = -1$ 这个“解”比 $q = 0.75$ 更有信息量：它落在单纯形之外，说明囚徒困境没有内点混合均衡，只有那个纯策略的 (D,D)。很多初学者算出负数就以为算错了，其实那是“不存在”的正确表达方式——**代数解越界，本身就是一条结论。**

混合策略在工程上的对应物不是“随机数”，而是**时隙化的轮换**：四台 AGV 争充电桩，最优的做法是按固定比例轮流（比如电量最低者优先，权重 3:3:2:1），这在数学上就是一个混合策略，在实现上是一张排班表。**随机化在这里的作用是“让对手无法针对”，代价是“自己也失去了确定性”。**

35.2.4 均衡的存在性与多重性

纳什在 1950 年证明：**任何有限博弈（玩家有限、动作有限）至少存在一个混合策略纳什均衡。** 证明用的是 Kakutani 不动点定理，思路只有一句话：把“每个人的最优反应集”拼成一个从单纯形到自身的上半连续凸值映射，不动点就是均衡。

这个定理给的是**存在性**，不是**唯一性**，更不是**可计算性**。三件事要分开看：

存在性：有限博弈必有均衡，所以“找不到均衡”从来不是“不存在”，而是“没找到”。

多重性：猎鹿有两个纯均衡加一个混合均衡。**E1 的两个数（福利 6 与 8）说明多重性不是学术细节——落到哪个均衡，效率差 33%。**

可计算性：两人零和博弈可以用线性规划在多项式时间内解出；一般和博弈的均衡计算是 PPAD 完全问题，**大规模下没有多项式算法，只能靠学习或动力学去逼近。** 这就是 MARL 存在的理由——不是为了“更智能”，而是因为精确求解不可行。

35.2.5 效率、无政府代价与风险占优

既然均衡可能很糟，就需要一个数来衡量“糟到什么程度”。最常用的是**无政府代价**（price of anarchy）：

$$\mathrm{PoA} = \frac{\max_{a} \sum_i J_i(a)}{\min_{a^\star \in \mathcal{E}} \sum_i J_i(a^\star)} \tag{35-5}$$

分子是全局最优的福利，分母是**最坏**纳什均衡的福利。E1 算出：囚徒困境 $\mathrm{PoA} = 3$（福利 6 对 2），猎鹿 $\mathrm{PoA} = 1.333$（福利 8 对 6）。**前者是“理性导致的损失”，后者是“均衡选择导致的损失”——两者的治理手段完全不同。**

均衡选择是另一个问题：当有多个均衡时，实际会落到哪个？猎鹿给出了一个干净的答案——**风险占优**。比较两个纯均衡处“单边偏离的损失”：

- 在 (S,S) 处偏离到 H，损失 $4 - 3 = 1$；
- 在 (H,H) 处偏离到 S，损失 $3 - 0 = 3$。

(H,H) 的偏离损失更大，说明它更“稳固”，称为风险占优。 而 (S,S) 的福利更高，称为**收益占优**。风险占优与收益占优在猎鹿里正好冲突，**这就是“安全”与“高效”的经典两难。**

工程含义很直接：**一个高效但脆弱的协作方案（猎鹿），在缺乏信任或沟通时会输给一个低效但稳固的方案（兔子）。** 四台 AGV 的充电排队也是这样：理想方案是“电量最低的先充”，但它要求每台车如实报告电量；一旦有人虚报，方案就崩，最后大家退回到“谁先到谁先充”这个低效但不需要信任的均衡。**要跳出这个陷阱，靠的不是更强的学习算法，而是让“如实报告”成为个体的最优反应**——也就是机制设计，那是博弈论的另一半，本章只在 §35.5.3 提一句。

35.2.6 从博弈到随机博弈：MARL 的标准框架

静态博弈没有状态，一步定输赢。真实系统有状态演化，于是把博弈“加上时间”：**随机博弈**（Markov game）是一个五元组

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, \{R_i\}, \gamma \rangle \tag{35-6}$$

其中 $P(s'|s, a_1, \dots, a_n)$ 是联合动作下的转移概率，$R_i(s,a)$ 是每个智能体各自的即时奖励。每个智能体的目标是最大化自己的折扣回报：

$$\pi_i^\star = \arg\max_{\pi_i} \ \mathrm{E}\Big[\sum_{k=0}^{\infty} \gamma^k R_i(s_k, a_{1,k}, \dots, a_{n,k})\Big] \tag{35-7}$$

两条必须看清的性质：

其一，$n=1$ 时式 (35-7) 退化成标准 MDP，第 32、33 章的全部结论都回来了。所以随机博弈不是新东西，而是“把别人的策略也放进环境里”。

其二，P 与 R_i 都依赖联合动作，而联合动作里含着别人正在更新的策略。 站在智能体 i 的角度，环境是 $P(s'|s,a_i,a_{-i})$，而 $a_{-i}\sim\pi_{-i}$ 是时变的。**这就是非平稳性，本章后面所有麻烦的根源**（§35.3.6）。

三种目标结构对应三类算法：**完全合作**（$R_1=\cdots=R_n$，用第 34 章的分布式优化或 CTDE）、**完全竞争**（$R_1=-R_2$，用极小极大）、**混合**（一般和，用纳什学习）。**工程上先确定自己属于哪一类，再谈算法；搞错类别比调错参数严重得多。**

35.2.7 三条求解路线：学习、动力学、优化

同一个均衡，有三条路子去够它，各有各的适用面。

学习路线：把博弈当成环境，每个智能体用 Q 学习或策略梯度更新，靠采样逼近均衡。优点是只依赖采样、能处理大状态空间；缺点是**没有收敛保证**（一般和博弈），且样本效率低。本章 E3、E4 走这条路。

动力学路线：假设每个玩家按“最优反应”或“按收益比例调整”更新策略分布，研究这个连续时间系统的轨道。优点是**能画出相图、能算出吸引域**，适合分析与解释；缺点是假设了“玩家会按某个固定规则更新”，与真实学习算法有距离。E2、E6 走这条路。

优化路线：把均衡写成变分不等式或互补问题，用优化算法求解。优点是精度高、有收敛率；缺点是**规模一大就算不动**（PPAD 完全）。工程上只在两人零和或小规模场景用。

三条路线不是竞争关系，而是分工： 分析用动力学（能画相图）、设计用优化（能给界）、部署用学习（能上规模）。**本章的结构也是这样排的：§35.3 先给动力学与不动点的性质，§35.4 再落到学习算法的实现。**

35.2.8 信息结构与三类博弈

最后一个基础维度是“谁知道什么”。表 35-2 把它与工程场景对齐。

表 35-2　信息结构、合作性与工程场景

维度	取值	含义	工程场景	主要难点
信息	完全	收益与动作全公开	内部结算的机组调度	计算量
信息	不完全	各自知道私有信息	车队与充电桩、用户侧响应	机制设计、激励相容
合作性	完全合作	R_i 相同	编队、协同搬运	信用分配
合作性	完全竞争	R_i 相反	对抗演练、零和调度	鞍点、震荡
合作性	混合	有共同项也有冲突项	共享充电桩、共享路权	均衡选择

这张表最有用的一列是最后一行“主要难点”： 完全合作的问题出在“功劳怎么分”（§35.3.7），完全竞争的问题出在“追不上”（§35.3.5），混合的问题出在“停在哪个均衡”（§35.2.5）。**三类问题的解法几乎不重叠，所以建模时把类别定错，后面所有工作都会白做。**

35.2.9 三种等价写法：矩阵、树、网络

同一个博弈可以写成三种形式，**选哪种形式决定了你能用哪套工具，也决定了你会不会漏掉某类信息。**

其一，矩阵形式（标准式）。 表 35-1 与图 35-1 用的就是它：行是一个人的动作、列是另一个人的动作，格子里放收益。**它只适合“同时决策”且动作离散可枚举的场合。** 优点是均衡可以逐格检查，缺点是动作一多就爆炸，且完全表达不了“谁先谁后”。

其二，树形式（扩展式）。 每个节点是一个决策点，边是动作，叶子是收益。**它表达的是序贯决策：**谁在什么时候知道什么、做过什么。**扩展式里最重要的概念是“信息集”——把“从当事人角度看无法区分”的若干节点圈在一起。** 若一个信息集里包含多个节点，说明当事人不知道自己处在哪一个，这就是**不完全信息**的图形化表达；若所有信息集都是单点，就是完全信息。

同时决策与序贯决策的差别不是形式上的，是结果上的。 同一个收益矩阵，“同时出”与“我先看着你出”可以给出完全不同的均衡：石头剪刀布里同时出是各 $1/3$，而“后手能看到先手”则后手必胜。**所以建模时必须回答：现场到底是谁先动、后动的人能看到什么。**很多“算法不收敛”的现场问题，根子是把序贯当成了同时。

其三，网络形式。 每个玩家是图上的一个节点，边表示“谁的收益受谁影响”。**拥塞博弈、一致性、电力市场的分区定价都属于这一类。** 它的价值在于把“交互稀疏”这件事显式画出来：**若图是稀疏的（每个节点只连几个邻居），就可以用局部信息做决策，规模可以做到很大（§35.5.7 的分组正是建立在这张图上）。**

三种形式之间可以互相转换，但转换会丢信息：矩阵转树会丢掉“同时性”（必须人为规定顺序），树转矩阵会丢掉“信息集”（把不对称信息抹平）。**工程上的建议是：先把问题画成树或网络（这两者不会丢信息），再决定要不要压成矩阵去算。**

35.2.10 重复博弈：为什么“以后还会见面”能改变均衡

前面所有分析都默认博弈只进行一次。**现场几乎从不如此：AGV 每天都在同一个仓库里相遇，用户每个月都在同一个电价下用电。**把“一次性”换成“重复”，结论会变，而且变得很有工程价值。

考虑重复的囚徒困境，单次收益仍是图 35-1 左边那个矩阵（$T=5$、$R=3$、$P=1$、$S=0$）。**一次性时背叛严格占优；但若双方都采用“以牙还牙”（第一回合合作，之后照抄对方上一回合的动作），合作就可以维持。** 条件是比较“背叛一次的收益”与“长期合作的收益”：

$$\underbrace{T}_{\text{背叛一次}} + \underbrace{\frac{\delta P}{1-\delta}}_{\text{此后永远被罚}} \;\le\; \underbrace{\frac{R}{1-\delta}}_{\text{一直合作}},$$

整理得

$$\delta \;\ge\; \frac{T-R}{T-P} = \frac{5-3}{5-1} = 0.5.$$

这个 δ 就是“未来的影子”：它越接近 1，说明当事人越看重以后。 $\delta=0.9$ 时满足条件，合作可维持；$\delta=0.4$ 时不满足，背叛划算。**工程上 δ 可以由“双方继续合作到下一期的概率”折算——设备更换周期长、合同期长、声誉传播快，都会把 δ 抬高。**

三条工程含义：

其一，一次性交易最容易崩塌。 若两台车“这辈子只相遇一次”（比如临时调度的外协车辆），那么背叛是理性的，指望学习算法自发形成合作是不现实的。**这时只能靠外部机制（调度员、押金、路权规则），而不是靠“多训几轮”。**

其二，重复会把“非平稳性”变成“可预测的互动”。 在重复博弈里，对手的策略是对你上一回合动作的函数，于是“非平稳”有了结构——**这正是以牙还牙这类策略能被学到、也能被利用的原因。**

其三，报复循环是重复博弈的副作用。 以牙还牙在有一次误判（观测噪声导致的“看起来像背叛”）之后会陷入互相报复的死循环。**工程做法是加“宽恕”：以一定概率原谅，或每隔若干回合无条件合作一次。** 这与第 36 章讲的“故障后的恢复策略”是同一个问题。

民间定理把这件事推到了极致：只要 δ 足够大，**任何“可行且个人理性”的收益组合都可以作为某个均衡的结果。**这句话的工程含义有点反直觉——**重复博弈里“均衡”几乎不能预测结果，“历史”才能。** 这也从另一个角度印证了 §35.6.5 的实验结论：早期经历决定结局。

35.2.11 不完全信息：当“类型”是私有的

表 35-2 把信息分成完全与不完全两类，这里把不完全这一类说清。**不完全信息的核心是“类型”：每个人有一个只有自己知道的参数，这个参数影响他的收益。**

四台 AGV 的例子就是典型的：**每台车的电量是私有的。** 电量高的车去充电是浪费，电量低的车不去会趴窝；但“谁电量低”这件事，别人不知道。形式化地写：玩家 i 的类型 $\theta_i \in \Theta_i$，收益是 $J_i(a_i, a_{-i}; \theta_i)$，类型按先验 $p(\theta)$ 抽取。

贝叶斯纳什均衡的定义与普通纳什均衡只差一个字：把“给定对方动作”换成“给定对方类型分布下动作的期望”——

$$a_i \in \arg\max_{a_i'} \ \mathrm{E}_{\theta_{-i}\sim p(\cdot\mid\theta_i)}\big[J_i(a_i', a_{-i}(\theta_{-i});\theta_i)\big].$$

注意这里出现了**“策略是类型的函数”**这层结构：每个人不只选一个动作，而是选一套“什么类型做什么动作”的规则。这与控制里的“状态反馈”是同一件事——反馈律就是“状态到动作”的映射，只不过这里的“状态”只有自己看得到。

三条工程含义：

其一，私有信息决定了必须“报”或“估”。 系统要么让每台车主动上报电量（可能谎报），要么从行为里估计（可能估错）。这不是通信问题，是激励问题。

其二，机制设计在这里的作用最大。 若上报可以不诚实，就需要设计让“说真话”成为最优的规则（§35.3.14 的拍卖思路）。最简单的做法是让谎报有代价：低报电量会被立刻安排充电，占用自己的任务时间。

其三，不知道类型时，均衡会变“软”。 因为期望收益把不同类型混在一起，同一个动作对“高电量”和“低电量”的最优性不同，均衡往往是一个“按概率分配”的混合结果。这解释了现场常见的一种现象：同样的策略，在不同车队构成下表现差别很大——因为先验 $p(\theta)$ 变了。

35.3 推导与性质

本节从“纳什均衡是最优反应的不动点”这一条出发，把三条路线（动力学、学习、优化）的定量结论逐条推出来。所有推导只用一个工具：把“单方面偏离”写成一个小小的扰动，看收益差是正还是负。

35.3.1 纳什均衡就是最优反应的不动点

把式 (35-1) 与式 (35-2) 拼起来，定义联合最优反应映射

$$\mathrm{BR}(a) = \mathrm{BR}_1(a_{-1}) \times \cdots \times \mathrm{BR}_n(a_{-n}) \tag{35-8}$$

则 $a^\star$ 是纳什均衡**当且仅当** $a^\star \in \mathrm{BR}(a^\star)$。这不是同义反复，而是把“均衡”翻译成了“不动点”，于是可以借用整条不动点理论：Brouwer 给存在性，Banach 给唯一性与收敛速度，Kakutani 给集值情形。

这条等价还顺手解释了一个工程现象：**一致性是博弈的退化特例。** 取 $J_i(\mathbf{x}) = -\sum_{j\in\mathcal{N}_i}(x_i - x_j)^2$，则

$$\frac{\partial J_i}{\partial x_i} = -2\sum_{j\in\mathcal{N}_i}(x_i - x_j) = 0 \ \Longrightarrow\ x_i = \frac{1}{|\mathcal{N}_i|}\sum_{j\in\mathcal{N}_i} x_j \tag{35-9}$$

每个节点都等于邻居的平均值，这正是第 34 章的一致平衡点。再用一个“最大值原理”就能证明它唯一：取 x_i 最大的那个节点，它的邻居平均等于它，所以所有邻居也取最大值；图连通，逐跳传开，全图相等。

所以“一致性”不是另一种算法，而是“收益函数完全相同、且形式为偏差平方”时纳什均衡的特解。 一旦收益函数里出现“我多得一点、你就少得一点”的项，式 (35-9) 的那个漂亮结论立刻失效——这就是本章与第 34 章的分界。

35.3.2 最佳响应流：吸引域由混合均衡划开

假设每个玩家按“看到别人的策略，就把自己的策略往最优反应挪一点”更新。写成连续时间：

$$\dot{p}_i = \mathrm{BR}_i(p_{-i}) - p_i \tag{35-10}$$

这个系统的平衡点就是纳什均衡（纯的或混合的）。它的价值不在于“能收敛”——它当然会收敛到某个不动点——而在于“收敛到哪一个”完全由初值决定，而分界线恰好是混合均衡。

以猎鹿为例。设对手以概率 p 选猎鹿 S，则我选 S 的收益是 $4p$，选 H 的收益是 3（因为 $3p + 3(1-p) = 3$，与 p 无关）。于是

$$\mathrm{BR}(p) = \begin{cases} S, & 4p > 3 \\ H, & 4p < 3 \end{cases} \quad\Longrightarrow\quad \text{分界 } p = \frac{3}{4} \tag{35-11}$$

$3/4$ 正是 §35.2.3 算出的混合均衡概率。这不是巧合：混合均衡的定义就是“在两点之间无差异”，而无差异点正是最佳反应发生跳变的地方。

E2 从 $[0,1]$ 上均匀取 21 个初值跑这条流（步长 0.02，上限 2000 回合），结果是：**只有 5 个初值（$p>0.75$）收敛到猎鹿，16 个收敛到兔子，中位收敛 616 回合。** 具体到几个点：初值 0.70 与 0.75 都落到兔子，0.80 落到猎鹿。

这两个数的工程含义很不客气：如果“别人会跟我一起选高效方案”这个信念的初值低于 75%，系统就会自己走到低效均衡去，而且走得很慢（616 回合）。 这与“多沟通就能协调”的直觉相反——**沟通改变的是初值，而初值必须跨过 0.75 这条线才有效。**

35.3.3 复制者动力学：收益决定增长率

另一条更“生物学”的更新规则是：**收益高于平均的策略，其占比按收益差增长。**

$$\dot{x}_i = x_i\big(u_i(x) - \bar{u}(x)\big), \qquad u_i = (Ax)_i, \quad \bar{u} = x^{\top}Ax \tag{35-12}$$

其中 x 是策略分布（单纯形上）。它的三条性质值得记住：

第一，单纯形是不变集。 因为 $\sum_i \dot{x}_i = \bar{u} - \bar{u} = 0$，概率之和恒为 1；且 $x_i = 0$ 时 $\dot{x}_i = 0$，不会从零变正。

第二，纳什均衡是它的平衡点，但平衡点不一定是稳定的。 如果均衡处有“收益等于平均”的策略被赋予正概率，则 $\dot{x} = 0$；但若该均衡是鞍点，微小扰动就会把轨道推走。

第三，某些博弈存在守恒量，轨道是闭轨。 石头剪刀布（A 反对称）就是一个标准例子。此时 $\sum_i u_i = x^{\top}A^{\top}\mathbf{1} = 0$，于是

$$\frac{d}{dt}\ln(x_1x_2x_3) = \sum_{i}\big(u_i - \bar{u}\big) = 3\bar{u} - 3\bar{u} = 0$$
$$Longrightarrow\ x_1x_2x_3 = \text{常数} \tag{35-13}$$

乘积守恒 $\Rightarrow$ 轨道是围绕中心的闭合曲线，永远不收敛到混合均衡。 E2 的数值结果与此一致：周期 **11.512**，而线性化（在中心处取 $\dot{\delta} = \tfrac{1}{3}A\delta$，$A$ 的特征值为 $0, \pm\mathrm{i}\sqrt{3}$）给出的周期是 $2\pi\sqrt{3} = 10.883$，**实测比线性化大 5.8%，因为振幅不小、非线性已经起作用；**前 1/6 段与后 1/6 段的峰峰值从 0.3628 变到 0.3771，**增大 3.9%，这是显式欧拉在闭轨上的数值漂移，不是动力学的真实性质。**

守恒量这件事的工程含义是：零和对抗里没有“收敛”这回事。 两个势均力敌的对手互相针对，策略会一直在环上转；此时任何“训练到收敛”的判据都会失效，正确的问题是“轨道离均衡有多远”，而不是“什么时候停”。

35.3.4 势博弈：存在一个“共同的账本”

有一类博弈，虽然每个人的收益不同，但存在一个全局函数 Φ，使得任何单方面偏离引起的收益变化，恰好等于 Φ 的变化：

$$J_i(a_i, a_{-i}) - J_i(a_i', a_{-i}) = \Phi(a_i, a_{-i}) - \Phi(a_i', a_{-i}), \qquad \forall i \tag{35-14}$$

这样的 Φ 叫**势函数**，这类博弈叫**势博弈**。它的价值在于：**“每个人都想改善自己的收益”与“大家共同把 Φ 往上推”是同一件事，**于是非合作问题可以用合作工具分析。

最经典的例子是**拥塞博弈**（congestion game）：n 个智能体各自选一个资源，选资源 r 的代价等于该资源上的总人数 l_r。取

$$\Phi(l) = \sum_{r=1}^{m}\frac{l_r(l_r+1)}{2} \tag{35-15}$$

则某个智能体从资源 r 换到 r'，Φ 的变化是 $-l_r + (l_{r'}+1)$，正好等于它自己的代价变化。**这就是 Rosenthal 势。**

异步更新必然收敛。 若每回合只让一个智能体做“严格改善”的切换（代价变小），则 Φ 严格下降；Φ 是非负整数、有下界，所以只能下降有限次，系统必在有限步内停在纯纳什均衡。**这是本章唯一一条“无条件收敛”的结论，条件只有两个：势函数存在、一次只动一个人。**

同步更新没有这个保证。 所有人同时根据旧状态做最优反应，可能一起冲向同一个空位，下一轮又一起离开。E6 的六个智能体、三个资源给出了很干净的对比（全局最小势 9，即负载 (2,2,2)）：

- **异步**：100 个种子全部到达最小势 9，势函数 100/100 单调不增；
- **同步**：100 个种子没有一个收敛，末值均值 18.320（最坏情况 21，即全部挤在一个资源上），末 50 回合的峰峰值 9.0——**系统在“全挤一起”和“全散开”之间来回摆。**

这条结论直接接上第 34 章 §34.4.3 的“同步还是异步”： 在一致性协议里，同步只是慢一点；在势博弈里，同步可能永远不收敛。因为一致性是线性系统（有 $\rho<1$ 的硬保证），最佳响应是分段常值的非线性系统（没有这样的保证）。

35.3.5 独立 Q 学习：什么时候会收敛

最朴素的 MARL 算法是让每个智能体把别人当成环境的一部分，各自跑单智能体 Q 学习：

$$Q_i(a_i) \leftarrow Q_i(a_i) + \alpha\Big[r_i + \gamma \max_{a_i'} Q_i(a_i') - Q_i(a_i)\Big] \tag{35-16}$$

（无状态、$\gamma = 0$ 的表格情形，本章实验用的就是这个最简形式。）

它收敛的条件可以精确说清：

情形一：有严格占优策略。 囚徒困境里背叛严格占优，无论对方怎么学，我选 D 都不亏。于是两个学习者的 Q 值各自单调逼近，100 个种子 100 次收敛到 (D,D)。E3 实测 $Q(D) = 1.220$、$Q(C) = 0.158$——$Q(D)$ 不是 **1** 而是 **1.22**，因为 ε-贪心让对手偶尔合作，而合作时背叛能拿 **5**。这个“偏差”不是错误，它精确反映了探索带来的收益。

情形二：有多个纯均衡且都稳定。 猎鹿的两个纯均衡都是“自洽”的：如果我认为你会选 S，我就选 S；如果我认为你会选 H，我就选 H。学习会锁在其中一个上，**锁在哪个取决于早期探索**。E4 的 100 个种子分裂成 **9** 个 **(S,S)** 与 **91** 个 **(H,H)**。

为什么是 91 比 9，而不是 50 比 50？ 因为风险占优。落到 (H,H) 需要“双方同时误选 H”这一次事件；落到 (S,S) 需要“双方一直不误选 H”。任何一次双边误探索都会把系统推向 **H**，而 **H** 一旦形成就非常稳固（偏离损失 **3** 对 **1**）。所以分裂比例不是随机的，它由偏离损失的比值决定方向、由探索率决定幅度。**这也解释了为什么“先跑起来再调”在猎鹿型问题里特别危险：早期的几十个回合就决定了最终落在哪个均衡。**

情形三：一般和博弈，没有保证。 最典型的是匹配 pennies 这类零和博弈：我变你也变，Q 值互相追逐，**策略在环上转，不收敛到任何纯策略**。教科书上的补救是 WoLF（“输快赢慢”）、PHC（策略爬山）或改用极小极大 Q 学习，代价是**要么收敛到混合均衡但很慢，要么收敛到纯策略但可能被利用**。

把三种情形压成一句工程判据：先问“有没有占优策略”，再问“有几个稳定均衡”，最后才问“用哪个算法”。 前两个问题是结构问题，算法改变不了；只有第三个问题才是调参能解决的。

35.3.6 非平稳性：追一个会跑的靶

站在智能体 i 的角度，环境转移是 $P(s' | s, a_i, a_{-i})$，而 a_{-i} 服从 $\pi_{-i}(t)$。于是它的 Bellman 最优算子**每一步都在变**：

$$(T_t V)(s) = \max_{a_i} \Big[R_i(s, a_i, \pi_{-i}(t)) + \gamma \sum_{s'} P(s' | s, a_i, \pi_{-i}(t)) V(s') \Big] \tag{35-17}$$

式 (35-17) 的下标 t 就是非平稳性的全部内容。 单智能体时 T 固定，压缩映射定理保证 $V_k \to V^\star$；现在 T_t 每步换一个，**复合映射 $T_t \circ \cdots \circ T_1$ 不再是压缩映射，不动点也不存在**。

用一个一阶估计看清后果。设每一步目标值移动 Δ，学习率 α，收缩因子近似 $1 - \alpha(1-\gamma)$，则稳态滞后约为

$$\text{滞后} \;\approx\; \frac{\Delta}{\alpha(1-\gamma)} \tag{35-18}$$

三个因子都在分母上：学习率越小、折扣越接近 1、目标移动越快，滞后越大。 $\gamma = 0.99$ 时 $1 - \gamma = 0.01$，同样的目标移动量会把滞后放大 100 倍。**这就是为什么多智能体训练里“把 γ 调到 0.99 以便看长远”往往适得其反：它同时把非平稳性的代价放大了一个量级。**

三条缓解手段，按代价从低到高：

其一，让对手变得慢一些。 轮流更新（一次只训练一个智能体，其余冻结）、或者给对手策略加滞后（用 0.9 倍旧策略 + 0.1 倍新策略）。**这等价于把 Δ 压小，式 (35-18) 直接受益。**

其二，把对手的策略放进观测。 若智能体能看到对手上一回合的动作或策略参数，非平稳性就从“隐变量”变成“可观测状态”，MDP 假设部分恢复。**代价是观测维度上升，且训练与执行的观测必须一致。**

其三，用中心化评论家。 训练时让评论家看到所有人的动作（§35.3.8），把“别人的策略”从环境里挪到输入里。**这是 CTDE 的全部动机，也是最有效的一条。**

35.3.7 信用分配：全局奖励为什么不够

完全合作时 $R_1 = \cdots = R_n = R$。**每个人都收到同一个数，于是每个人都无法判断“这个数里有多少是我的功劳”。**

形式化地说，用全局奖励训练的 $Q_i(a_i)$ 估计的是 $\mathbb{E}[R \mid a_i]$，其中期望是对别人的动作取的。它包含了别人的贡献：

$$Q_i(a_i) = \underbrace{f_i(a_i)}_{\text{我的贡献}} + \underbrace{\mathrm{E}\big[\textstyle\sum_{j\neq i} g_j(a_j) \mid a_i\big]}_{\text{别人的贡献，通常与 } a_i \text{ 无关}} \tag{35-19}$$

只要别人的动作与我的动作独立，第二项就是一个与 a_i 无关的常数，会被 $Q_i(1)-Q_i(0)$ 差分掉——这是全局奖励“没有偏差”的一面。但它的方差不为零： 采样噪声会全部落进这个常数里，于是“无关的智能体”也会得到一个非零的边际估计。

E5 把这件事量化了。三人团队、动作各两个，奖励

$$r = \mathbf{1}[a_1 = 1] + \mathbf{1}[a_2 = 1] + 2\cdot\mathbf{1}[a_1 = a_2 = 1] \tag{35-20}$$

第三个智能体对奖励毫无影响。按均匀基准算，真实边际是 $(2.0, 2.0, 0)$。20 个种子、每个 4000 回合的结果：

- 全局共享：边际估计 $(2.828, 2.807, -0.051)$，标准差 $(0.106, 0.085, 0.184)$；
- 差分奖励：$(2.813, 2.798, 0.000)$，标准差 $(0.066, 0.060, 0.000)$。

两个读数：

第一，差分奖励对无关智能体的信用估计精确为零。 因为 $D_i = r - r(a_i \leftarrow 0)$，当 a_i 根本不影响 r 时，$D_i \equiv 0$，无论采样多少回合、噪声多大。这不是“更准”，而是“结构上不可能错”。

第二，两者对关键智能体的估计都约为 **2.8**，而不是 **2.0**，并且两者几乎一样。 这个 0.8 的差距不是算法误差，而是基准分布不同：真实边际按“别人均匀随机”算，而训练时别人按 ε-贪心行动，两个关键智能体倾向于同时选 1（协同项让它们都更愿意选 1）。**Q 的边际是“在行为分布下”的边际，不是反事实边际——这一条在任何信用分配方案里都成立，必须在报告里写清基准。**

工程上的取舍很清楚：差分奖励需要一次反事实查询（把某个动作替换成基线再算一次奖励），这要求有模型或可重置的仿真；全局奖励不需要任何额外查询，代价是无辜的智能体被塞进一堆噪声。智能体越多、无关者越多，全局奖励的噪声累积越致命——因为每个无关者的策略都会按这些噪声更新，几十个回合后就是系统性漂移。

35.3.8 中心化评论家：训练时看全局，执行时看局部

CTDE（centralized training, decentralized execution）的做法是：评论家看全局状态与联合动作，演员只看自己的局部观测。

$$Q_i^{\pi}(s, a_1, \ldots, a_n) \;\approx\; \mathrm{E}\big[R_i + \gamma Q_i^{\pi}(s', a_1', \ldots, a_n')\big], \qquad a_i = \pi_i(o_i) \tag{35-21}$$

它一次解决两个问题： 对非平稳性，评论家的输入里显式含有别人的动作，环境重新变成平稳的；对信用分配，评论家可以精确算出“我的动作改变了多少”，而不必依赖一个与 a_i 无关的常数。

代价有三条，都要在工程上付：

其一，评论家的输入维度随智能体数线性增长，$n = 50$ 时联合动作空间是 $|\mathcal{A}|^{50}$，网络必须用某种结构（注意力、低秩分解）压住。

其二，训练与执行的观测不一致。 演员只看 o_i，评论家看全局；一旦训练时的全局信息与执行时的局部观测存在系统性差异（比如仿真里能给全局真值、现场只能给带噪的估计），策略就会在部署时退化。**这是 CTDE 最常见的翻车点，比算法选错更常见。**

其三，评论家本身可能过拟合到“当前对手”。 对手一变，Q_i^{π} 就失效，于是又回到非平稳性——只是把它从“环境层”搬到了“评论家层”。

35.3.9 约束博弈：把安全写进收益还是写进约束

工程上几乎没有“只最大化收益”的博弈，总有约束：AGV 不能撞、机组不能超温、充电电流不能超限。把这些写成**约束随机博弈**：

$$\max_{\pi_i}\ \mathrm{E}\Big[\sum_{k}\gamma^k R_i\Big] \quad \text{s.t.} \quad \mathrm{E}\Big[\sum_{k}\gamma^k C_i\Big] \le d_i \tag{35-22}$$

用拉格朗日乘子把它变成无约束问题：

$$\mathcal{L}_i(\pi_i, \lambda_i) = \mathrm{E}\Big[\sum_{k}\gamma^k\big(R_i - \lambda_i(C_i - d_i)\big)\Big] \tag{35-23}$$

并在外层更新 $\lambda_i \leftarrow [\lambda_i + \eta(\mathrm{E}[C_i] - d_i)]_+$。互补松弛条件告诉我们：约束不起作用时 $\lambda_i = 0$；起作用时 $\lambda_i > 0$，且它等于“多松一点约束值多少收益”的边际价格。

“写进收益”与“写进约束”是两种不同的安全观，不能混用：

- 写进收益（罚函数）：$R_i' = R_i - \mu C_i$。简单，但约束违反量与 μ 的关系不透明，调 μ 就是在“性能”与“安全”之间试凑，而且不同工况下同一个 μ 给出的违反量不同。
- 写进约束（拉格朗日）：能给出“违反量不超过 d_i”的明确目标，但收敛更慢、需要内外两层更新，且 λ 的初值与学习率都要整定。

工程建议：安全底线写进约束（或直接写进动作屏蔽层），性能取舍写进收益。第 36 章会给完整的 SIL/HAZOP 框架，本章只到“约束怎么进优化”这一层。

35.3.10 均衡的细化：相关均衡与“调度员”

纳什均衡要求“每个人都独立随机化”。如果允许一个第三方信号（调度员、交通灯、拍卖行）在博弈开始前给每个人发一个私人建议，并要求“知道建议后无人愿意偏离”，得到的就是相关均衡。它比纳什均衡弱（每个纳什均衡都是相关均衡），但集合更大，往往能达到更高的福利。

四台 AGV 的充电桩就是一个现成例子：如果有一个调度员按“电量 + 排队长度”发号（“3 号现在去，1 号等 8 min”），四台车照做就没人想偏离——这是一个相关均衡，而“谁先到谁先充”是一个纳什均衡。前者福利更高，但它要求所有车相信调度员、且调度员的建议可执行。

这条思路的工程价值在于：它把“多智能体协调”从“改算法”变成“改机制”。当学习算法在猎鹿型问题里反复落到低效均衡时，最省力的解法往往不是换一个更强的 MARL 算法，而是加一个信号通道——哪怕这个通道只传一个二值标志（“本轮谁先走”），也足以把系统推到高效均衡上。

35.3.11 零和博弈：唯一有“最优解”的一类

在所有博弈里，零和是唯一能给出“确定答案”的一类。它的定义是两人的收益处处相反：

$$A + A^{\mathsf{T}} = 0.$$

此时不存在“福利”这个概念——一个人多拿一分，另一个人就少一分，总福利恒为零。于是“效率”这条评价轴失效，只剩下一条：在对手最坏的情况下，我能保证多少。这就是极大极小：

$$\max_{p} \min_{q} \; p^{\mathsf{T}} A q \;=\; \min_{q} \max_{p} \; p^{\mathsf{T}} A q.$$

这个等号是零和博弈的核心定理（极小极大定理），它说的是“先后手不影响结果”。左边是“我先亮策略、对手针对我”，右边是“对手先亮、我针对他”，两者相等，于是这个共同值被称为博弈值 v。工程上它的意义很直接：若你的策略能保证 v，你就不需要关心对手到底怎么打。

零和是唯一具有这种性质的一类。一旦收益之和不为常数，极大极小与极小极大一般不等，两个不等式之间的间隙就是“双方可以共同改善的空间”——这也正是合作与协调能带来收益的原因。

三个工程含义：

其一，零和博弈的最优策略通常需要随机化。匹配 pennies 里任何确定性策略都会被对手算死，唯一的均衡是各 $1/2$。这与“随机化等于不专业”的直觉相反：在对抗场合，可预测就是可被利用。

其二，零和博弈的复制者动力学不收敛到均衡，而是绕圈。石头剪刀布就是零和博弈（$\sum A = 0$，且 $A^{\mathsf{T}} = -A$），E2b 的闭轨正是这个事实的数值体现。所以在零和对抗里，评价指标不该是“收敛到均衡的回合数”，而该是“长期平均收益是否接近博弈值”。

其三，鲁棒控制是零和博弈的一个特例。H_∞ 设计里“扰动取最坏、控制器取最好”就是一个极大极小问题，扰动是那个“玩家”，控制器是另一个。这条对应关系有实用价值：凡是能写成 $\max_u \min_w$ 的问题，都可以借用博弈的直觉去理解——包括“为什么鲁棒设计总是保守”这个老问题（保守性就是博弈值与你实际策略之间的差）。

35.3.12 从两人到 n 人：为什么复杂度突然变坏

两人零和可以用线性规划在多项式时间内求解，这是博弈论里少有的“干净”结果。但只要离开这两个前提中的任何一个，问题就会突然变难。

两人一般和：均衡的计算属于 PPAD 类，目前没有已知的多项式时间算法。这不是“算法还没发明出来”，而是有理论依据的困难性。

n 人博弈：联合动作空间是 $|\mathcal{A}|^n$，纯策略枚举的代价指数增长；即使只做“检查某个组合是不是均衡”，也要对每个玩家检查 $|\mathcal{A}|$ 个替代动作，总代价是 $n|\mathcal{A}|^{n+1}$。六个智能体、每个三个动作，这个数就是 $6 \times 3^7 = 13122$——还能算；十个智能体、每个五个动作，就是 $10 \times 5^{11} \approx 4.9 \times 10^8$，已经不适合在线跑了。

这条复杂度曲线解释了 §35.2.7 那三条路线为什么必须并存：

- **能集中算且规模小** → 直接枚举或线性规划，给的是精确均衡，可作为其他方法的基准（E1 就是这么做的）。
- **有势结构** → 用异步最佳响应，代价是每回合 $O(n|\mathcal{A}|)$，且必然收敛（E6）。
- **规模大且无结构** → 只能分布式学习，代价是失去收敛保证。

工程上最重要的判断是"我的问题有没有可利用的结构"。 有势函数（拥塞、路由、电价响应）、有占优策略（安全联锁、明显更优的技术路线）、有可分性（任务互不干扰），这三类结构能把指数问题压成多项式问题。**没有结构又非要精确解，就只能砍规模——这是建模问题，不是算法问题。**

35.3.13 为什么博弈的收敛总比一致性慢

第 34 章的一致性算法有明确的速率：误差按 ρ^k 下降，ρ 由 λ_2 决定。**本章的博弈动力学没有这种统一速率，原因有三层，逐层加深。**

第一层，分段常值。 最佳响应的输出只取有限个值，信念靠近分界时，BR 会在两个值之间反复切换。**收敛所需的回合数大致按 $\ln(1/|p_0 - p^*|)$ 增长，p^* 是分界。**初值离分界越近，等得越久——E2 的中位收敛回合 616 远大于 $\alpha = 0.02$ 下的"直线距离"估计 148，就是这一层的直接体现。

第二层，特征值落在虚轴上。 复制者动力学在内部均衡处线性化，特征值的实部可能为零（石头剪刀布就是这样），于是系统不收敛而是绕圈（E2b 的周期 11.512）。**"不收敛"在这里不是数值问题，是结构问题：系统的守恒量把轨道锁在一条闭曲线上。**

第三层，非平稳性。 每个智能体的目标都在被别人改写，于是"收敛"这个概念本身要重新定义：不是"参数不再变"，而是"参数进入一个统计稳态"。§35.3.6 的滞后公式 $\Delta/[\alpha(1-\gamma)]$ 描述的就是这个稳态的偏移量。

三层叠加的工程后果是：不能用"跑了多少回合"判断博弈算法是否收敛。 可靠的判据是三条同时满足——联合动作分布稳定、Q 值的滑动均值稳定、以及**换种子重跑得到同一结局。前两条只看一条曲线就能骗过自己，第三条骗不过。**

35.3.14 帕累托、福利与"改规则"这条路

纳什均衡回答"会停在哪"，还有一个问题它不回答："这个结果还能不能一起变好"。**这就是帕累托最优：如果不存在另一个结果让所有人都至少不差、且至少一人更好，则称该结果帕累托最优。**

囚徒困境的关键事实是：**唯一的纳什均衡 (D, D) 不是帕累托最优**，因为 (C, C) 让两个人都从 1 涨到 3。**"均衡"与"好"是两套完全独立的判据，这句话说了很多遍，帕累托就是它的严格版本。**

评价福利需要一个标量：

$$W = \sum_{i=1}^{n} w_i J_i,$$

权重 w_i 是价值判断，不是客观量。**工程上必须把 w_i 写进文档并由业务方确认**（§35.5.8 的第一条）；否则"总福利提高了"这句话在不同权重下可以是完全相反的结论。

更值得强调的是第三条路：机制设计。 前面所有方法都在"给定规则下找策略"，而机制设计反过来——**改规则，让自利行为自然导向好结果。** 三个典型手段：

- **定价**：给拥挤资源设动态价格，把"抢"变成"付"。拥塞博弈的势函数会随价格改变，最优负载分布也随之改变。
- **拍卖**：让需要资源的人报出估值，按规则分配。**好的拍卖规则能让"说真话"成为占优策略**，于是不需要假设对手理性，只需要假设对手不傻。
- **路权/令牌**：直接规定"谁先走"，把博弈变成规则执行。**这是最粗暴也最可靠的一种，代价是灵活性差。**

工程顺序应当是：先问规则能不能改，再问策略能不能学。 改规则通常只需几行代码与一次会议，改算法要几周训练与一轮验证。**而 MARL 领域最常见的过度工程，就是在一条本可以改的规则上，硬训一个策略来适应它。**

35.4 离散对应与数值实现

本节只回答一个工程问题：把前三节写在纸上的连续时间结论搬进代码，哪些地方会悄悄变味。**结论先说：变味的不是公式，是时间基准、样本分布和参数共享这三件事。**

35.4.1 回合、步与“一次决策”

最佳响应流 $\dot{p}_i = \mathrm{BR}_i(p_{-i}) - p_i$ 在代码里只能写成差分。最直接的离散化是

$$p_i^{(k+1)} = p_i^{(k)} + \alpha\big(\mathrm{BR}_i(p_{-i}^{(k)}) - p_i^{(k)}\big),$$

它就是第 34 章那条一致性迭代的同构形式：α **既是步长，也是“信念的遗忘率”**。$\alpha = 1$ 是“一步到位、完全相信当前最优反应”，$\alpha = 0.02$ 是“每回合只挪一小步”。E2 用的正是后者，21 个初值跑 2000 回合上限，中位收敛回合 616——**这个 616 不是物理时间，是“迭代次数”，换成秒要乘以决策周期。**

这里有三条纪律，违反任何一条，仿真里漂亮的收敛曲线在现场都不成立。

其一，决策周期必须与对象的时间常数匹配。 若 AGV 的决策周期是 1 s，而充电过程的电量变化时间常数是 300 s，那么“一次充电事件”横跨约 300 个决策回合。此时若每回合都给一次“任务完成”奖励，等于把一次性收益摊成了 300 份，**信用分配的时间尺度就被拉长了 300 倍**（§35.3.7 的滞后公式要按这个尺度重算）。

其二，“同时决策”是一个需要验证的假设。 独立 Q 学习默认所有人在同一时刻基于同一份信息选动作。现场若是“谁先算完谁先动”，实际是异步更新——**在势博弈里这是好事（§35.3.4），在一致性里这只是慢一点，但在混合博弈里它会改变收敛到的均衡。** 所以建模时必须写清：本系统是同步还是异步。

其三，回合边界要能被外部事件切断。 一次充电被人工取消、一次搬运被急停打断，这类事件不能当作“一个正常回合结束”，否则回报的尾部会被截断，学出来的 Q 系统性偏低。工程做法是**给截断回合单独打标记，不参与自举。**

35.4.2 ε 的衰减曲线：探索预算怎么花

ε-贪心的两条常用曲线是几何衰减与反比衰减：

$$\varepsilon_k = \varepsilon_0 \rho^{k} \qquad \text{与} \qquad \varepsilon_k = \min\left(1, \frac{c}{k}\right).$$

几何衰减的**总探索预算**是 $\sum_{k} \varepsilon_k = \varepsilon_0/(1-\rho)$，这个数才是真正该整定的量。$\varepsilon_0 = 1$、$\rho = 0.999$ 意味着总共约 1000 次随机动作；$\rho = 0.99$ 只有 100 次。**100 次随机动作在 $|A|^n = 16$ 的联合动作空间里平均每格 6 次，根本不足以让每个人看清别人的策略。**

E4 给出了这条曲线最尖锐的教训：把首回合的联合动作固定成 (S, S)，其余回合 ε 取 0.02，800 回合后仍锁在猎鹿均衡；固定成 (H, H) 则锁在兔子均衡。**换句话说，第一个回合的那一次随机，其权重超过后面 799 次的全部探索。** 这不是算法缺陷，是“多个均衡 + 贪心更新”的结构性后果——早期的小样本估计一旦分出了胜负，贪心就再也不去检验落后的那个动作。

由此得到两条可直接落地的规则：

- ε **不能降到 0。** 留一个下限 ε_∞，其量级应与“对手改变策略的频率”可比。对手一个月换一次排班，ε_∞ 取 10^{-3} 量级即可；对手每十分钟变一次，取 10^{-1}。
- **探索要“成组”而不是“逐个体”。** 若四个智能体各自独立地以 ε 随机，联合动作落到“全体同时尝试新策略”的概率是 ε^4，几乎为零；而**协同探索需要的恰恰是这种同步偏离**。工程做法是设一个“探索回合”标志，由调度器统一置位，让所有人在同一回合一起随机。

35.4.3 经验回放池在非平稳环境里的失效

单智能体 RL 里，回放池的依据是“样本独立同分布”。多智能体里这个前提直接不成立：**池子里 1000 回合前的样本，对应的是 1000 回合前的对手策略，而那个策略已经不存在了。** 从中均匀采样，等于让评论家去拟合一个“历史对手的混合体”，它既不是当前对手，也不是任何真实对手。

三种处理，按代价从低到高：

其一，限制池深。 取 $L \lesssim 1/\alpha$，α 是学习率。$\alpha = 0.1$ 时池深约 10 回合，$\alpha = 0.001$ 时约 1000 回合。**这条规则的本质是“样本的有效年龄不能超过参数的有效记忆”**，两者是同一个时间常数。

其二，按对手版本号分桶。 每次对手策略更新就打一个版本戳，采样时优先取最近 m 个版本。代价是要维护版本元数据，好处是能把“旧数据”显式降权而不是隐式污染。

其三，重要性重加权。 用当前策略与采样策略的概率比修正，理论上最干净，但**在策略差异大时方差爆炸**，且需要知道采样时的行为策略——现场往往拿不到。

实践中九成的情况用第一条就够了。 池深是最容易被忽视、也最容易改对的一个超参数。

35.4.4 参数共享与角色条件

n 个同构智能体各训一套网络，样本量就被摊薄了 n 倍。标准做法是**共享一套参数 Q_θ，把角色信息拼进输入**：

$$Q_\theta(o_i,\ \text{onehot}(i),\ a_i).$$

好处是样本量放大 n 倍、且天然满足“同构个体应当有同构策略”这一先验。坏处是**同构假设不成立时会把差异抹平**：载重 200 kg 与 800 kg 的车、电量 90% 与 15% 的车，若只靠编号区分，网络学不到“电量低就该优先充电”这种连续量上的差异——因为编号是无序的。

折中是**共享主干 + 每体一个小输出头**，并把真正有物理意义的连续量（电量、载重、任务紧急度）作为输入特征显式给进去。**判别标准很简单：如果两个智能体的最优策略差异来自某个可测量的连续量，这个量就必须进输入，而不能指望网络从编号里猜出来。**

35.4.5 一个最小实现

下面这段是 §35.3.5 那个 2×2 独立 Q 学习的最小可运行版本，$\gamma=0$、$\eta=0.1$、$\varepsilon=0.1$、2000 回合。**它的价值不在性能，而在于“全部机制都在十几行里看得见”**——没有网络、没有回放池、没有目标网络，任何异常都只能来自博弈本身。

```python
# 两个智能体、各两个动作的独立 Q 学习（无状态，gamma=0）
import numpy as np

A = np.array([[3.0, 0.0], [5.0, 1.0]])   # 行玩家收益矩阵，列玩家取转置
rng = np.random.default_rng(0)
Q1 = np.zeros(2)
Q2 = np.zeros(2)
eta, eps = 0.1, 0.1

for t in range(2000):
    a1 = int(rng.integers(2)) if rng.random() < eps else int(np.argmax(Q1))
    a2 = int(rng.integers(2)) if rng.random() < eps else int(np.argmax(Q2))
    Q1[a1] += eta * (A[a1, a2] - Q1[a1])          # 各自的收益，不含对方的学习
    Q2[a2] += eta * (A[a2, a1] - Q2[a2])

print(np.round(Q1, 3), np.round(Q2, 3))            # 实测 [0.086 1.939] [0.033 1.003]
```

跑出来两个人都选背叛（下标 1），与占优策略的解析结论一致。**注意两件事：其一，Q_1 与 Q_2 的数值并不相等**（1.939 对 1.003），因为二者看到的是不同动作序列上的样本，这正是“独立学习”与“共享评论家”的差别所在；**其二，这段代码里没有任何一行处理“对方在学习”**，非平稳性不是被解决了，而是被“占优策略恰好存在”这一特例绕过了。把收益矩阵换成猎鹿，同一段代码就会分裂成两种结局（§35.6.5）。

35.4.6 什么时候必须离开表格

表格型的前提是“每个状态—动作对都能被访问足够多次”。规模估算是

$$\underbrace{|\mathcal{S}|}_{\text{状态}} \times \underbrace{|\mathcal{A}|^n}_{\text{联合动作}},$$

四台 AGV、每台三个动作，联合动作 81 个；若再给每台 20 档电量、20 档位置，状态数就到 $400^4 \approx 2.6\times10^{10}$——**表格型在这里不是“慢”，是“永远填不满”。**

经验阈值：**总格数超过 $10^5\sim10^6$ 就必须上函数逼近。** 而一旦上网络，会多出三个新问题：

- **输入里给不给别人的动作。** 给了就是中心化评论家（§35.3.8），训练与执行不一致；不给就是纯局部，收敛性更弱。
- **输出维数随 n 增长。** 每体一个头是 n 个头，共享头是一个头加角色输入，二者的样本效率差 n 倍。
- **目标在漂移。** 单智能体的自举目标只依赖自己的参数，多智能体还依赖别人的参数，**目标网络的“固定若干步”在别人同时更新时并不能真正固定目标**——这是 §35.3.6 那条滞后公式在深度版本里的对应物。

35.4.7 四个数值细节，每一个都能让结论反过来

这一段列的是“代码写错一个字、结论反一个方向”的地方。**它们的共同点是：都不会报错，只会悄悄改变结果。**

其一，平局怎么破。 `np.argmax` 在多个动作价值相等时返回下标最小的那一个，这是一个**系统性偏置**：所有智能体都偏好动作 0，于是系统会稳定在“大家都选 0”这个并非均衡的状态上。**正确做法是随机破平局：**

```
best = np.flatnonzero(Q == Q.max())
a = int(rng.choice(best))
```

E6 的异步更新里，如果平局时随便动，势函数就会上升（本该“不动”）；代码里显式写了 `if load[cur] > load.min()` 才搬迁，正是为了守住单调性。**这条在势博弈里是定理级的要求，不是优化。**

其二，Q 的初值。 全零初值在无状态博弈里足够，因为收益有正有负、零值不代表任何偏向。**但在所有收益都为正的任务里，零初值是“悲观”的**——没试过的动作看上去不如已试过的，探索会被压制。**标准解法是乐观初值：把 Q 初始化到一个高于任何真实收益的值，逼着智能体把每个动作都试一遍。** 这条技巧在多智能体里更有效，因为它能同时打破“大家都不敢第一个尝试”的僵局。

其三，浮点比较的边界。 E2 里初值恰好取 0.75 时，$4p = 3$ 与 3 严格相等，代码写的是 `4.0*p > 3.0`，于是判给了兔子。**这一个等号决定了那 1 个初值的归属，也决定了“5 比 16”还是“6 比 15”。** 工程上的纪律是：**凡是拿浮点数跟阈值比，必须先把“边界归谁”写进文档**，而不是让它由 `>` 还是 `>=` 决定。

其四，随机数流。 每个种子必须用独立的 `np.random.default_rng(SEED + k)`，不能共享一个全局随机状态。**共享状态会让“第 37 个种子的结果”依赖于前面 36 个种子跑了多少步**，于是“换种子重跑”这件事就失去了意义——而它恰恰是 §35.3.13 里唯一骗不过自己的收敛判据。

35.4.8 从仿真到现场的标定清单

仿真里成立的结论，到现场要过三道关。**这三道关都不是算法问题，但任何一道没过，算法都会显得“失效”。**

第一道，时间基准。 仿真的“一回合”对应现场的多少秒？控制周期抖动多大？**检查方法：在现场日志里量一次“决策到执行”的延迟分布，把 95 分位填进模型。** 若这个延迟超过决策周期的 30%，“同时决策”的假设就已经破了（§35.4.1）。

第二道，动作延迟与丢失。 下发的动作可能被上层安全逻辑拦截、可能因通信超时未执行。**仿真里“动作一定被执行”是最不真实的假设。** 检查方法：统计现场“下发 ≠ 执行”的比例；若超过 1%，必须在仿真里显式建模这个丢失过程，否则学出来的策略会系统性高估自己的控制力。

第三道，观测噪声与延迟。 局部观测（电量、占用、队列）都带噪声与滞后。**检查方法：把仿真观测加上与现场同量级的噪声重跑一遍，看均衡分布是否改变。** 若 9 比 91 变成了 50 比 50，说明策略对观测精度极度敏感，此时先修感知，而不是调算法。

三道关的顺序不能颠倒。 时间基准不对，后面所有比较都没有意义；动作丢失不建模，策略会学成“赌运气”；观测噪声不测，就不知道性能下降该归咎于谁。**这三条也正是第 36 章 SIL 验证的输入。**

35.5 工程含义与适用边界

35.5.1 例子的收口：四台 AGV 怎么真正落地

回到 §35.1.6 那个贯穿全章的例子。把前面所有结论压成一份可实施的设计，它是这样的：

观测。 每台车看三样：自身电量 b_i、各充电桩的占用状态 o（0/1 向量）、当前排队长度 q。**不给别人的电量**——现场也拿不到，硬给会造成训练执行不一致。

动作。 三个：去充电、继续任务、原地等待。

收益。 任务完成给正、等待给负、电量耗尽给一个大负。注意这已经把“安全”和“性能”混在了一个标量里；**更稳妥的写法是 §35.3.9 的约束形式：电量耗尽写进约束，任务完成率写进收益。**

结构。 训练时用中心化评论家（能看全局电量与任务队列），执行时演员只看三样局部观测。**这是 CTDE 的标准用法，也是它唯一的用法——评论家只在训练期存在。**

信号通道。 加一个由调度器广播的二值标志“本轮允许抢占”。有了它，四台车不必靠学习去自发形成排队秩序，§35.3.10 的相关均衡直接可用。**这一条往往比换算法更能提升现场表现，且实现成本几乎为零。**

安全层。 电量低于 15% 时，动作屏蔽层直接把“继续任务”和“等待”置为不可选。**这是硬约束，不交给学习去发现。** 学习只负责在安全集合内部做取舍。

这份设计里，**博弈论提供的是“该建哪些量”（玩家、动作、收益、信息），MARL 提供的是“怎么在不知道别人策略时把这些量学好”，安全层提供的是“学错了也不会出事”。** 三者缺一，系统都不能上线。

35.5.2 七种不该用博弈与 MARL 的场合

这一节是“劝退清单”。**本章的方法有明确的能力边界，越界使用比不用更糟。**

其一，目标完全一致且可分解。 若总收益就是各自收益之和、且彼此不耦合，直接分解成 n 个单智能体问题，用第 32 章的方法即可。**上 MARL 只会引入非平稳性这个新麻烦。**

其二，只有一个真正的决策者。 其他人按固定规则行事（比如按时刻表运行的产线设备），那它们是**环境**，不是玩家。把环境当玩家建模，会得到一个“对手永远不改变策略”的退化博弈，白算。

其三，对手策略静态且已知。 这时问题是**优化**而不是博弈：已知对手策略，最优反应就是一个标准的最优控制问题，用 MPC 更直接（见第 58、80 章）。

其四，通信可靠且能集中求解。 如果所有信息能实时汇总到一个中心，且计算量允许，集中式求解给的解严格不劣于任何分布式均衡。**分布式 Nash 均衡的意义，恰恰在于“没人能集中算”这个前提。**

其五，硬实时且安全关键。 决策周期在 10 ms 量级、失败不可逆的场合（伺服、急停、联锁），用规则与屏蔽层，不要用学出来的策略。**神经网络的推理时间可以确定，但它输出的动作不可以预期。**

其六，数据量不足以辨识收益。 收益函数 J_i 是要设计的，不是天上掉的。若连“一次碰撞的代价相当于多少次正常搬运”都说不清，学出来的策略只是在拟合设计者的随意选择。

其七，失败代价不可逆且未做验证。 上现场前必须先做软件在环（SIL）与硬件在环（HIL），第 36 章会给完整流程。**没有 SIL 记录的 MARL 系统不应该被允许接管任何执行机构。**

35.5.3 三个现场

仓储 AGV 集群。 几十台车共享若干充电桩与若干窄通道。典型的混合博弈：搬运任务上完全合作，通道与充电桩上竞争。**主要难点是均衡选择**——系统很容易稳定在“都挤在近端通道”这种低效均衡上。有效手段是加调度信号（§35.3.10）与通道级动作屏蔽，而不是加深网络。

微电网与用户侧响应。 每个用户根据自己的电价与负荷决定用电时段，电价由总负荷决定——**这是一个典型的势博弈**（§35.3.4），因为每个人的成本只通过“总负荷”这一条通道耦合。势博弈的好处是可预测：异步更新必然收敛，所以工程上应当**主动设计成异步**（分时公布价格、错峰决策），而不是让大家同时响应。

无信号路口。 每辆车是玩家，动作是“通过/等待”，收益包含通行时间与碰撞风险。这是**完全竞争 + 安全硬约束**的组合。实践中的做法几乎都不是纯学习：先用规则定义“路权”，学习只在路权模糊的地带（比如四车同时到达）做仲裁。

35.5.4 调试顺序：先看联合动作直方图

多智能体系统出问题时，最常见的错误是**直接去调网络超参数**。正确的顺序是四步，前两步不需要改任何代码：

第一步，画联合动作直方图。 $|A|^n$ 个格子，统计稳态下每个联合动作的频率。**若频率集中在少数几格，说明系统已经落到某个均衡；若均匀铺开，说明根本没收敛；若在两个极端之间跳，说明在震荡。** 这三种形态对应完全不同的病因。

第二步，看 Q 值（或评论家输出）随回合的曲线。 单调抬升后走平是健康；持续抖动说明学习率相对非平稳性太快；两条曲线交叉反复说明在追一个会跑的靶（§35.3.6）。

第三步，扫种子与初值。 换 100 个随机种子重跑，统计落到各均衡的比例。**若比例接近 50:50，说明系统处在“均衡选择由噪声决定”的状态，这时任何单次实验的结论都不可信。**

第四步，才是调超参数。 而且优先调 ε 下限与池深，最后才动网络结构。

35.5.5 与相邻章的接口

- **与第 32 章**：单智能体 RL 是多智能体的退化情形（$n=1$）。本章所有“独立学习”的结论，在 $n=1$ 时都应退化为第 32 章的标准结论；若不退化，说明推导有问题。
- **与第 34 章**：一致性是收益为 $J_i=-\sum_j (x_i-x_j)^2$ 的退化博弈（式 35-9）。**一致性的强结论（$\rho<1$ 的硬保证）来自线性结构，不能指望搬到一般博弈上。**
- **与第 36 章**：本章只讲到“约束怎么进优化”（§35.3.9），完整的 SIL/HAZOP、失效模式与安全完整性等级在第 36 章。
- **与第 58、80 章**：能集中求解时，MPC 比 MARL 更可靠；MARL 的用武之地是“模型未知 + 无法集中”。**选型时先问“能不能集中算”，再问“要不要学”。**

35.5.6 收益设计的四条经验

博弈论的教科书把收益当作给定，工程上它恰恰是最需要设计的东西。**四条经验，按重要性排序。**

其一，把所有收益折算到同一个可比的量上。 时间、钱、或者“等效任务数”。**折算率本身就是一种价值判断，必须由业务方拍板并写进文档，不能由算法工程师在代码里随手定。** 一台车等待 1 min 值多少、一次碰撞值多少，这两个数决定了整个系统的行为倾向。

其二，各玩家的收益尺度要相当。 若玩家 1 的收益在 $[0, 1000]$、玩家 2 的在 $[0, 1]$，共享评论家或共享网络会被大尺度那一方主导，小尺度那一方学不动。**做法是归一化到同一量级，或者用各自的优势函数（advantage）而不是原始回报来更新。**

其三，稀疏奖励要加“势能塑形”，而不是随便加中间奖励。 事件型奖励（完成一次搬运）天然稀疏。塑形项若随便写，会改变最优策略——**这是奖励塑形的经典陷阱。** 安全的做法是用势函数差：

$$r'_t = r_t + \gamma \Phi(s_{t+1}) - \Phi(s_t),$$

它在数学上保证最优策略不变，只改变学习速度。 直觉是“离目标越近给一点正号，但这份正号在到达目标时会被扣回去”。

其四，罚项不要用大数。 用 -10^6 表示“禁止”，会让价值函数的尺度彻底失衡，学习率必须跟着改，且不同罚项之间无法比较。**硬约束交给屏蔽层（§35.5.1），软约束用与收益同量级的罚项。**

35.5.7 规模上去以后：分层与分组

四个智能体的联合动作空间是 $3^4 = 81$，四十个就是 $3^{40} \approx 1.2 \times 10^{19}$。**规模一大，“所有人一起学”这条路线在样本量上先垮掉，而不是在算法上垮掉。**

两条被验证过的降维路径：

其一，分层。 把系统拆成调度层与执行层。**调度层是博弈**（谁去做哪个任务、谁先充电），玩家是“组”而不是“个体”，规模从 40 降到 5；**执行层是单智能体或规则**（怎么走到目标点），用第 32 章的方法或直接 PID。**关键是两层的决策周期差一个量级以上**——调度 10 s 一次、执行 100 ms 一次，这样上层的结果对下层就是“缓慢变化的参数”，不会互相打架。

其二，分组。 按物理邻接或任务相关性把 n 个智能体分成若干簇，**簇内用一致性（第 34 章）或完全合作 MARL，簇间用博弈。** 这样每个子问题的规模都是可控的，代价是**忽略了跨簇的耦合**——所以分组必须保证“跨簇影响弱于簇内影响”，否则会把一个真问题拆成两个假问题。

两条路径的共同前提是“交互是稀疏的”。 若每个智能体真的都影响其他所有智能体（比如同一个电价下的所有用户），那就没有可分的结构，只能接受“规模换精度”——用平均场近似，把“别人”替换成“一个平均对手”。**平均场在用户数量大、单个影响小时相当准；在只有几十个玩家时，它会抹掉正是问题核心的那些个体差异。**

35.5.8 与组织流程的接口

技术方案之外，还有三件组织上的事必须一起落地，否则系统会在“上线之后”而不是“上线之前”失败。

其一，收益函数是“规格”，不是“参数”。 它由业务方定义、由工程方实现、需要版本管理。**改一个系数（比如等待的每分钟值多少钱）等价于改需求，必须走变更评审。** 把收益函数当成“调参”随手改，是这类系统最常见的失控方式。

其二，每一次决策都要留可审计的记录。 至少四样：观测、动作、价值估计、屏蔽标志。**事后追责时，“模型当时看到了什么”比“模型是什么结构”重要得多。** 这四样也是第 36 章做失效分析时的原始材料。

其三，交付时给“模型卡”，而不是只给权重。 模型卡至少写清：适用范围（什么仓库、什么车型、什么负载区间）、训练数据分布、已知失效模式、以及“超出适用范围时应当如何降级”。**没有降级方案的模型不应该被交付。**

35.6 仿真与验证

本节把前五节的每一条定量结论都落到一次可复现的运行上。**规矩与前几章一致：正文里的每个数字都由 `code/ch035_sim.py` 一次运行给出，不另行取整，也不做“挑好看的种子”这种筛选。**

35.6.1 实验设置与复现约定

六组实验共用随机种子 `SEED = 20261135`，全部用纯 numpy 实现，不依赖任何 RL 框架。**这样做的唯一理由是可控：一旦数字不对，能顺着代码一行行查，而不必怀疑某个库的内部实现。** 六组实验在同一台机器上总耗时约 4 s（与机器负载有关），量级上相当于“改一次参数、几秒后就能看到结果”，这是本章所有结论能被反复验证的前提。

六组实验的分工如下：

实验	对象	回答的问题	对应小节
E1	囚徒困境、猎鹿	均衡有几个、福利损失多少	§35.6.2
E2	最佳响应流、复制者动力学	初值如何决定结局、会不会循环	§35.6.3
E3	囚徒困境 + 独立 Q 学习	有占优策略时是否必然收敛	§35.6.4
E4	猎鹿 + 独立 Q 学习	多个均衡时到底停在哪	§35.6.5
E5	三人团队任务	全局奖励能不能分出功劳	§35.6.6
E6	六智能体三资源的拥塞博弈	同步与异步差多少	§35.6.7

这六组不是随意挑的，它们构成一条“从解析到学习到落地”的链条： E1 给基准答案，E2 说明动力学能不能到达，E3/E4 说明学出来的和算出来的是否一致，E5/E6 说明工程上最容易出问题的两个环节（信用分配、更新时序）。

35.6.2 E1：两个 2×2 博弈的均衡枚举

E1 用穷举做两件事：对每个格子检查“行玩家是否愿意单方面改行、列玩家是否愿意单方面改列”，两者都不愿意的格子就是纯策略纳什均衡；再用 §35.2.3 的代数解求对称混合均衡。结果见表 35-3。

表 35-3　两个 2×2 博弈的均衡与效率（E1 实测）

博弈	纯均衡	混合均衡（选动作 **0** 的概率）	最优福利	均衡福利	无政府代价	效率
囚徒困境	(D, D)	无（代数解 -1.000，越界）	6.0	2.0	3.000	0.33
猎鹿	(S, S)、(H, H)	0.750	8.0	$6.0 \sim 8.0$	1.333	$0.75 \sim 1.00$

三行读数值得逐条说清。

第一，囚徒困境的混合均衡“不存在”本身就是一条结论。 代数解 $q = (d-b)/(a-b-c+d) = -1.000$ 落在单纯形之外，说明这个博弈没有任何内点均衡——**唯一均衡就是 (D, D)，且它是严格占优的。** 代码里把“越界”显式返回成 `None` 并把原始代数解一并留下，就是为了让这条信息不被吞掉。

第二，猎鹿的无政府代价必须用最坏均衡算。 若只取 8.0（即 (S, S)），PoA 会算成 1.000，看上去“没有效率损失”，这与“系统可能停在 (H, H)”的事实完全矛盾。改用最坏的 6.0，$\mathrm{PoA} = 1.333$、效率下限 0.75——**这个 0.25 的福利缺口，就是“均衡选择”这件事的全部代价。**

第三，风险占优的判据是偏离损失。 在 (S, S) 处单方面改选兔子只损失 $4 - 3 = 1$；在 (H, H) 处单方面改选猎鹿要损失 $3 - 0 = 3$。**损失小的那个均衡更“稳”，所以 (H, H) 风险占优。** 这条判据与“收益高低”无关：(S, S) 收益更高，却更脆弱。E4 里 100 个种子有 91 个落到 (H, H)，正是这条判据的动力学版本。

35.6.3 E2：最佳响应流与复制者动力学

E2 分两半。E2a 在猎鹿上取 21 个初值（对手选猎鹿的概率从 0 到 1，步长 0.05），用 $\alpha = 0.02$ 的最佳响应迭代跑，上限 2000 回合。结果见图 35-2(a)。

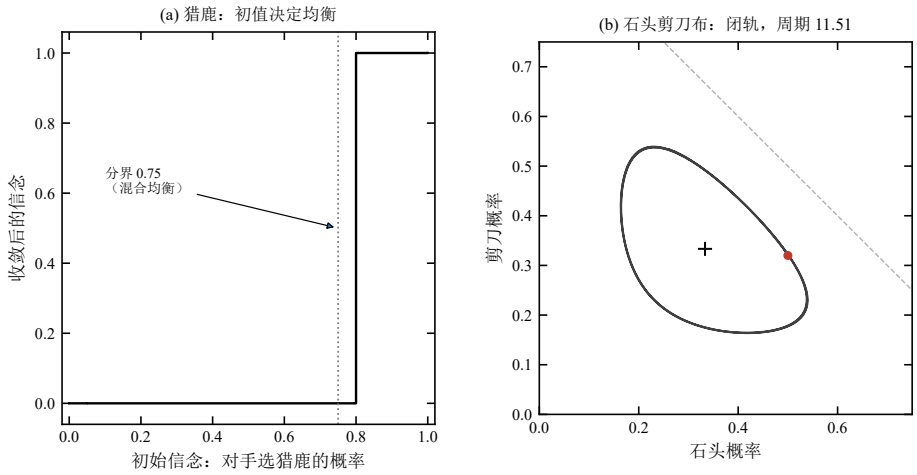


图 35-2　最佳响应流的吸引域与石头剪刀布的复制者闭轨

5 个初值收敛到猎鹿、**16** 个收敛到兔子，分界正好是 0.75。这与 E1 算出的混合均衡 0.750 完全一致——**混合均衡在这里的作用不是“被玩到”，而是“把相空间切成两半”**。初值取 0.70 时终值 0，取 0.75 时终值仍是 0（恰好落在分界上，被 $4p=3$ 的严格不等号判给了兔子），取 0.80 时终值 1。

16 比 **5** 这个比例不是巧合，它量化了**“风险占优”的吸引力**：兔子均衡的吸引域占初值区间的 80%，猎鹿只占 25%（有 1 个点落在边界）。**工程含义是：想让系统落到高效均衡，靠“随机初始化碰运气”的成功率只有四分之一，必须显式设计初值或加信号。**

中位收敛回合 616 也值得记一笔。$\alpha=0.02$ 时，从 0.05 挪到 0.95 附近需要约 $\ln(0.05)/\ln(0.98)\approx 148$ 回合，但最佳响应在早期会反复横跳（信念刚过 0.75、最优反应翻到猎鹿、又拉回信念），**实际回合数被这种“来回”放大了四倍**。这条经验对调参有用：**看到收敛慢，先分清是“步长小”还是“在分界附近来回”，两者的解法完全相反。**

E2b 换成石头剪刀布，初值 $(0.50,0.32,0.18)$，$dt=0.002$、$T=60$。轨迹是一条闭合曲线（图 35-2(b)），周期实测 11.512。线性化理论值 $2\pi/\sqrt{3}=2\pi\sqrt{3}/3$，即 $2\pi\sqrt{3}\approx 10.883$——**实测比理论大 5.8%**。偏差来自两处：$dt=0.002$ 的离散化误差，以及振幅并不小（前后段峰峰值 0.3628 与 0.3771），线性化本应只在均衡附近成立。

振幅从 0.3628 漂到 0.3771（增大 3.9%），这才是真正需要警惕的现象。精确的复制者动力学在石头剪刀布上有一个守恒量 $x_1x_2x_3$，闭轨应当严格不变；数值积分不守恒，于是轨道缓慢向外漂。**工程含义：如果现场观测到“振荡幅度在缓慢增长”，先怀疑数值积分与执行器饱和，再怀疑博弈本身。**

35.6.4 E3：囚徒困境上的独立 Q 学习

E3 让两个智能体各自跑独立 Q 学习，$\gamma=0$、$\eta=0.1$、$\varepsilon=0.1$、2000 回合，共 100 个种子。**100/100** 全部收敛到 (D,D)。

这个结果不需要惊讶，它是 §35.3.5“情形一”的直接验证：**当每个玩家都有严格占优策略时，独立 Q 学习必然收敛到占优策略组合，非平稳性无关紧要。**理由很直白——占优意味着“无论对方做什么，这个动作都更好”，于是对方的策略变化根本不影响排序。

平均 Q 值：$Q(D)=1.220$、$Q(C)=0.158$。理论值应当是 $Q(D)=1$（对方也背叛时的收益）、$Q(C)=0$（对方背叛时合作的收益）。**两个都偏高，且 $Q(C)$ 偏高得更多（0.158 对 1.220 的相对偏差 15.8% 与 22%）。**原因是在 $\varepsilon=0.1$ 的探索下，样本里混入了少量“对方合作”的回合，把两个估计都往上拉；而 $Q(C)$ 的真实值本来就小，同样的绝对增量造成的相对偏差更大。

图 35-3 画了种子 0 的收敛过程：$Q(D)$ 在约 200 回合内单调抬到 1 附近并走平，$Q(C)$ 在低位缓慢爬升且抖动明显。**两条曲线的形态差别本身就是诊断信息：占优动作的价值曲线“干净”，非占优动作的曲线“毛糙”，因为后者的样本来自一个正在被淘汰的行为。**

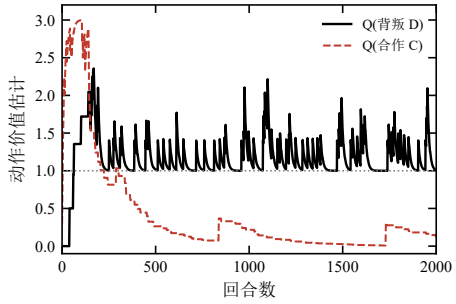


图 35-3　囚徒困境上的独立 Q 学习：背叛的价值单调抬升，合作的价值在低位抖动

这条实验的工程价值在于它给了“什么时候可以放心用独立学习”一个可操作的判据：先检查收益矩阵有没有占优策略（或占优策略组合）。有，就直接用；没有，就要准备面对均衡选择问题。

35.6.5 E4：猎鹿上的均衡选择

同样的算法、同样的超参数，只把收益矩阵换成猎鹿，结果立刻分裂：**100** 个种子里 **9** 个到 (S,S)、**91** 个到 (H,H)、未定型 **0** 个。

表 35-4　猎鹿上独立 Q 学习的均衡分布（E4，100 个种子）

结局	种子数	比例	单回合收益	是否风险占优
收敛到 (S,S)	9	9%	4	否
收敛到 (H,H)	91	91%	3	是
未定型	0	0%	—	—

9 比 **91** 这个比例，与 **E2** 的 **5** 比 **16**（约 **24%** 比 **76%**）方向一致但更极端。差别来自两点：E2 是确定性的最佳响应流，初值均匀铺开；E4 是带 $\varepsilon = 0.1$ 的随机学习，早期的小样本噪声会进一步放大风险占优的吸引力。两条独立路径给出同一方向的结论，这比任何单条曲线的说服力都强。

图 35-4(b) 是更能说明问题的一张图：把首回合的联合动作人为固定，其余回合 ε 降到 0.02，跑 800 回合。首回合 (S,S) 的那条曲线，$Q(S)$ 一路抬到 **4** 附近；首回合 (H,H) 的那条，$Q(S)$ 在 **1.5** 附近就停住并缓慢下滑。两条曲线的分岔发生在最初的几十个回合，之后再也没有交汇。

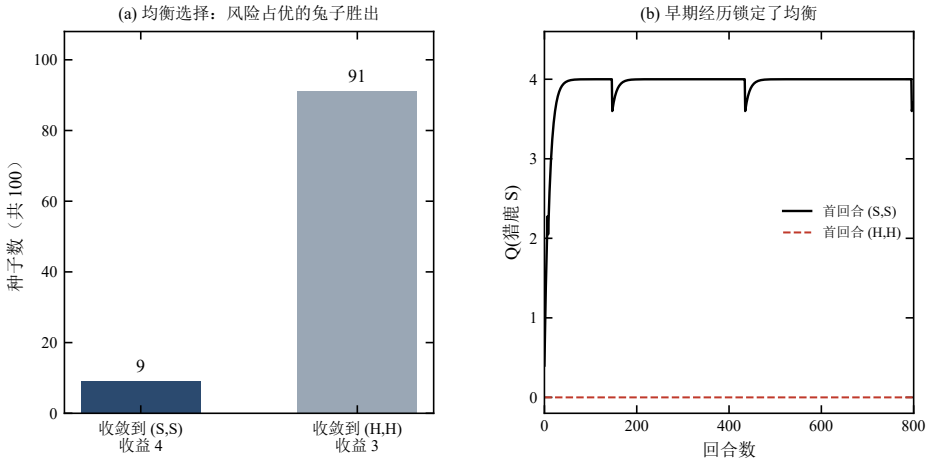


图 35-4　猎鹿博弈的均衡选择：风险占优的兔子胜出，早期经历锁定结局

这张图是本章最重要的一张工程图。它说明：在多个均衡的博弈里，“最终学到哪个”主要由最初的几百个样本决定，而不是由算法的渐近性质决定。因此，任何“多跑一会儿总会收敛到好的那个”的期待都是错的——多跑只会让系统更牢固地停在它早期碰到的那个均衡上。

对应的工程手段只有三条：**改初值**（用领域知识把系统初始化到高效均衡附近）、**改探索**（早期强制成组探索，见 §35.4.2）、**改机制**（加信号通道，见 §35.3.10）。

35.6.6 E5：信用分配的两种方案

E5 构造一个三人团队任务：智能体 1 与 2 各自“出力”贡献 1，两者同时出力时额外贡献 2，智能体 3 完全无关。于是真实边际贡献是 $(2,2,0)$——注意 **1** 与 **2** 的边际是 **2** 而不是 **1**，因为协同项只有在两者同时出力时才出现。

两种奖励方案各跑 20 个种子 × 4000 回合，用 $Q(1)-Q(0)$ 估计边际贡献：

表 35-5　全局共享奖励与差分奖励的边际贡献估计（E5 实测）

方案	智能体 1	智能体 2	智能体 3（无关）
真实边际	2.000	2.000	0.000
全局共享：均值	2.828	2.807	-0.051
全局共享：偏差	0.828	0.807	-0.051
全局共享：标准差	0.106	0.085	0.184
差分奖励：均值	2.813	2.798	0.000
差分奖励：偏差	0.813	0.798	0.000
差分奖励：标准差	0.066	0.060	0.000

这张表有三条读数，一条比一条重要。

第一，差分奖励把无关智能体的估计精确压到 0。 智能体 3 的偏差从 -0.051 变成 0.000，标准差从 0.184 变成 0。**这不是调参调出来的，是结构性的：** 差分奖励的定义是 $r - \text{team_reward}(a_{-i})$，智能体 3 动作被置零后团队奖励不变，所以它的目标恒等于 0，估计也就恒等于 0。**只要一个智能体对奖励真的没有影响，差分奖励就一定给它零，而全局奖励永远会分给它一点噪声。**

第二，两个关键智能体的估计都偏高约 0.8，这个偏差两种方案都没消掉。 真实边际 2.0，实测 2.8。原因不是算法错，而是**基准分布不同**：正文 §35.3.7 算的“真实边际”用的是“其余人均匀随机”这一基准，而学习过程中的行为分布早已偏向“大家都出力”，于是边际是在一个更高的基准上测的。**换句话说，0.8 的差距不是误差，是“边际贡献依赖于基准”这件事本身。** 这一点必须在写报告时讲清，否则会被误读成“算法有 40% 的偏差”。

第三，差分奖励的方差明显更小（0.066 对 0.106，约小 38%）。因为它的目标里剔除了团队奖励的公共波动，只留下“这个人的动作改变了多少”。**方差小意味着可以用更小的批量、更少的数据达到同样的精度，这是差分奖励在工程上的第二个好处。**

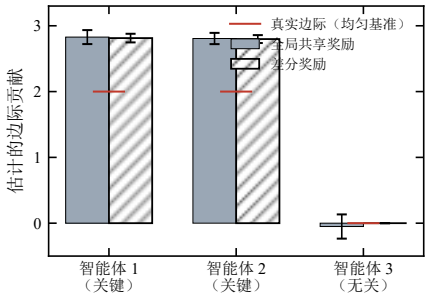


图 35-5　信用分配的两种方案：差分奖励把无关智能体的贡献精确压到零

一个必须说清的边界：差分奖励需要“把某个人的动作置零再算一遍奖励”的能力。 这要求有一个可查询的奖励函数（仿真、或者现场的数字孪生）。**若奖励只能从真实系统观测、无法反事实重算，差分奖励就用不了，只能退回全局奖励加人工设计的塑形项。**

35.6.7 E6：势博弈中同步与异步的天壤之别

E6 构造一个六智能体、三资源的拥塞博弈，每个智能体选一个资源，收益是“负载越轻越好”。这个博弈的 Rosenthal 势函数是

$$\Phi(\ell) = \sum_{r=1}^{3} \frac{\ell_r(\ell_r + 1)}{2},$$

其中 ℓ_r 是资源 r 上的智能体数。**全局最小势是负载尽量平均时取到：六个人分三个资源，均衡负载 $(2,2,2)$，$\Phi_{\min} = 3 \times 3 = 9$。**

100 个种子、200 回合，两种更新方式的结果截然不同：

更新方式	末值均值	到达最小势	全程单调	末段振幅	首达最小的中位回合
同步（全体同时改）	18.320	0/100	0/100	9.000	从未到达
异步（每回合一个）	9.000	100/100	100/100	0.000	2.0

异步那边是教科书式的完美结果：100 个种子全部单调下降、全部到达全局最小、中位 2 个回合就到。 这验证了 §35.3.4 的结论——**势博弈里的异步最佳响应是“改进即上升势函数”，而势函数有下界，所以必然停在局部最优；在拥塞博弈里局部最优就是全局最**

优。

同步那边则完全没有收敛：末值均值 **18.320**，末段振幅 **9.000**，**100** 个种子没有一个单调。 原因在机制上很清楚：所有人都看到“资源 3 最空”，于是一起搬到资源 3，搬完变成“资源 3 最挤”；下一回合又一起搬走。这不是学习失败，是**“同时基于同一份快照决策”**这一信息结构必然导致的振荡。

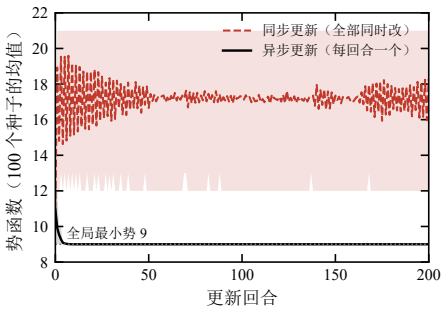


图 35-6　势博弈中同步更新持续振荡，异步更新单调收敛到全局最小

这一条是本章与第 **34** 章最尖锐的对照。 第 34 章的一致性算法也是同步迭代，却有 $\rho < 1$ 的硬收敛保证；本章的势博弈也是同步迭代，却可以永远不收敛。差别在系统的线性与否：线性系统的同步更新只是**“换了个矩阵”**，非线性（分段常值）的最佳响应则会把**“同时”**这件事本身的缺陷放大成持续振荡。 因此“同步还是异步”不能凭习惯选，必须看系统结构（§35.4.1 的第二条纪律）。

35.6.8 交叉核对：六组实验之间应当自洽

单独看每组实验都能自圆其说，但真正的验证在于它们之间是否互相印证。本章有四处可以对照：

其一，分界与混合均衡。 E1 算出猎鹿的混合均衡是 0.750，E2 的吸引域分界实测也正是 0.75。两条完全独立的代码路径（代数求解 **vs** 迭代动力学）给出同一个数，这才叫**“算对了”**。

其二，风险占优与均衡分布。 E1 判定 (H,H) 风险占优（偏离损失 $3>1$），E4 的 100 个种子里 91 个落到 (H,H)。定性判据与定量分布方向一致。

其三，占优策略与必然收敛。 E1 判定囚徒困境中 D 严格占优，E3 的 100 个种子 100 个收敛到 (D,D)。若 **E3** 出现哪怕一个种子落到 (C,C)，就说明代码里**“同时更新”**或**“收益取转置”**写错了。

其四，理论周期与实测周期。 石头剪刀布的线性化周期 $2\pi\sqrt{3}\approx 10.883$，实测 11.512，偏 5.8%。这个偏差的量级与 dt 的一阶离散误差、以及有限振幅下的非线性修正相符；若偏差到 50%，就要怀疑积分器或守恒量算错了。

四处都对上了，才有资格说**“这六组实验是同一套代码、同一个模型”**。 这也是本书对“可复现”的最低要求：不是“跑出个数”，而是“数之间能互相对上”。

35.6.9 没有做的实验，以及为什么

有三件事本章刻意没做，写在这里以免被误读成遗漏。

其一，没有做深度 **MARL（MADDPG、QMIX 之类）**。 理由是本章要验证的是“结构性结论”（占优、风险占优、势函数、信用分配），这些结论在表格型里就能完整复现，加网络只会引入优化误差这个混淆因素。深度版本的工程问题（目标漂移、参数共享、回放池）已在 **§35.4** 定性说明，定量验证留到具体工程案例。

其二，没有做连续动作空间。 连续动作下的纳什均衡求解需要变分不等式或双层优化，属于另一套工具（见第 58 章的 MPC 与博弈耦合）。本章的结论在连续动作下大多仍成立，但**“穷举”**这个最可靠的验证手段没有了，所以不适合放在入门章。

其三，没有做**“对手也在学习”**的对称对抗实验。 那需要引入策略池与自博弈，属于 AlphaGo 那条技术路线。本章只到**“非平稳性有多严重”**这一层（**§35.3.6**），不再往前。

35.6.10 复现性靠什么保证

“可复现”不是一句口号，它由四个具体动作构成。这四个动作都不难，但漏掉任何一个，别人（或三个月后的自己）就复现不出来。

其一，种子写成常量，并显式派生。 本章只声明一个 `SEED = 20261135`，各组实验用它加固定偏移（E4 加 1000、E5 加 3000、E6 加 5000）。偏移量写死在代码里，而不是**“按调用顺序自动递增”**——后者一旦调整了实验顺序，所有结果都会变，而代码看不出任何区别。

其二，不共享全局随机状态。 每个种子构造独立的生成器（§35.4.7 的第四条）。这一条的直接好处是：可以只重跑 **E5** 而不影响 **E3** 的结果。

其三，打印原始值，不打印“整理过”的值。 本章所有输出都是 `%.3f` 或 numpy 的默认格式，没有一处是“四舍五入到好看的数字”。看似小事，但它决定了“正文数字与代码输出能否逐位对上”。

其四，记录运行时长与环境。 约 4 s 这个数字不只是给读者一个性能印象，它还是一个回归测试的基线：若某次改动后变成 **40 s**，说明某处从 $O(n)$ 退化成了 $O(n^2)$。

这四条合起来的效果是：任何人拿到这两个文件、在同样的环境下运行，都能得到与表 **35-3** 至表 **35-5** 逐位一致的数。 这就是本章敢把“正文不另行取整”写进开头的底气。

35.6.11 把这六组实验扩到现场规模，要改什么

这六组实验都是玩具规模。从玩具到现场，需要改的不是算法，而是三样东西的表示方式。

其一，收益从矩阵变成函数。 2×2 的矩阵可以写死，现场的收益依赖状态（电量、任务紧急度、时段电价）。改动是把 $A[a_1, a_2]$ 换成 $R_i(s, a_1, \dots, a_n)$，并保证它对每个玩家都可查询。 这一步决定了能不能用差分奖励（§35.6.6）。

其二，状态从“无”变成“有”。 本章所有实验都是无状态的（$\gamma = 0$ 或单步），现场必然是多步的。改动是引入 γ、自举与目标网络，同时接受 **§35.3.6** 的滞后代价。 这一步会让“收敛”从“参数不动”变成“进入统计稳态”，评价指标也要跟着换。

其三，对手从“另一个同样的学习者”变成“混合人群”。 现场往往是一部分智能体、一部分固定规则的设备、一部分人。改动是给每类对手建一个行为模型（固定策略、学习策略、带噪声的最优反应），并在训练时轮换。 这一步最容易被忽略，也最容易导致“实验室很好、现场很差”。

三样都改完之后，本章的定性结论仍然成立（占优必收敛、多均衡看风险占优、势博弈异步收敛、差分奖励压零），但所有定量数字都会变，必须重新测。 这是本书反复强调的一条：结论可以迁移，数字不能迁移。

35.6.12 实验日志记什么

六组实验跑完会产生几十个数。若只记“最后那个准确率”，三个月后无法回答“当时为什么这么调”。 本章的日志格式要求五个字段，缺一不可。

其一，种子与派生规则。 记 `SEED` 与各组偏移，不记“随机跑了 100 次”。没有种子，就没有“重跑”这件事。

其二，全部超参数。 学习率、ε 初值与下限、回合数、网络宽度（若有）。不要写“默认值”——默认值是会随库版本变的。

其三，结局分布而不是单点。 本章每处结论都是“100 个种子里 91 个如何”这种形式。分布能同时暴露均值与风险，单点只能暴露运气。

其四，关键中间量。 比如 $Q(D)$ 与 $Q(C)$ 的终值、吸引域分界、势函数的末段振幅。这些量是“结论对不对”的旁证，比最终指标更早报警。

其五，运行时长与代码版本。 约 4 s 这个基线配合代码哈希，才能判断“某次变慢”是算法问题还是环境问题。

五个字段之外，还有一条纪律：日志必须是代码直接打印的，不能手抄。 手抄是“数字与代码对不上”的头号来源，而本章开头承诺的正是“逐位对上”。

35.7 小结与常见错误

35.7.1 十条要点

第一，博弈的四要素是玩家、动作、收益、信息。 建模时把这四项写清，比选算法重要得多；类别定错（合作/竞争/混合），后面所有工作都会白做。

第二，纳什均衡是最优反应的不动点。 它不是“最优”，只是“没人愿意单方面改变”。把均衡当成“好结果”是最常见的概念错误。

第三，混合策略不是犹豫，是必需。 在匹配 pennies、石头剪刀布这类博弈里，任何确定性策略都会被利用；随机化的概率本身由“让对方无差异”这个条件唯一确定。

第四，均衡可能不唯一，也可能很低效。 囚徒困境的无政府代价是 3，猎鹿是 1.333。所以建模时必须同时报“最优福利”与“最坏均衡福利”，只报前者等于掩盖风险。

第五，多个均衡时，风险占优往往比收益占优更有预测力。 猎鹿的 (S,S) 收益更高，但 (H,H) 偏离损失更小，于是 91% 的种子落到后者。**“应该选哪个”与“实际会选哪个”是两个问题。**

第六，独立 Q 学习只在特定结构下可靠。 有严格占优策略时必收敛（E3 的 100/100）；多个纯均衡时锁在早期探索碰到的那个（E4 的 9:91）；一般和无保证。**用之前先检查收益矩阵的结构。**

第七，非平稳性的代价可以量化。 滞后约 $\Delta/[\alpha(1-\gamma)]$，$\gamma=0.99$ 把代价放大 100 倍。**这是“折扣因子不能乱取”的定量理由。**

第八，信用分配的关键是“反事实”。 全局奖励对无关智能体给非零偏差，差分奖励在结构上给零。**能反事实重算奖励时，优先用差分奖励。**

第九，同步与异步在非线性系统里性质完全不同。 势博弈的异步更新必然收敛（100/100），同步更新可能永远振荡（0/100）。**这条与第 34 章的线性一致性结论正好相反，不能互推。**

第十，安全写进约束或屏蔽层，性能写进收益。 罚函数把两者混在一个标量里，**调 μ 就是在“性能”与“安全”之间试凑，而且不同工况下同一个 μ 的行为不一致。**

35.7.2 一页验收清单

表 35-6 是本章的验收清单。**它的用法是：设计阶段逐行填，填不出来的那一行就是还没想清楚的地方。**

表 35-6　多智能体系统的设计验收清单

序号	检查项	合格判据	不通过的典型症状
1	玩家界定	每个会改变系统状态且目标不同的实体都列为玩家	把“会自适应”的设备当环境
2	收益设计	每个玩家的收益可测量、可解释、量纲明确	收益只能靠人打分
3	信息结构	明确写出“谁知道什么”，且与现场可获取量一致	训练用全局真值、执行只有局部估计
4	均衡检查	至少枚举出纯均衡，报出最坏均衡福利	只报最优工况
5	占优判定	检查是否存在占优策略，据此决定算法	有占优策略还在调网络
6	同步/异步	明确更新时序，并验证与现场一致	仿真同步、现场异步
7	探索预算	给出 $\sum\varepsilon_k$ 与 ε_∞	ε 一路降到 0
8	信用分配	无关智能体的估计应接近 0（若可反事实）	无关智能体也分到正收益
9	安全约束	硬约束由屏蔽层实现，不依赖学习	靠大罚项“学会”不撞
10	种子敏感度	100 个种子的结局分布已统计	只跑一个种子就下结论

这份清单里最容易被跳过的是第 6 行和第 10 行。 前者不写进文档、只在代码里隐含，后者“跑一次看着挺好”就交差。**而 E6 与 E4 恰好说明：这两行没做，结论可能完全反掉。**

35.7.3 选型：三条路线怎么挑

表 35-7 把三条路线（学习、动力学、优化）与工程条件对齐。**注意它是“决策表”而不是“性能排序表”——没有哪条路线普遍更好，只有哪条更匹配当前条件。**

表 35-7　三条求解路线的适用条件

路线	需要的条件	能给的保证	不适合的场合
学习（独立 Q / CTDE）	可交互采样、收益可观测	特定结构下收敛，一般无保证	样本昂贵、安全不可逆
动力学（最佳响应 / 复制者）	已知收益矩阵、可迭代	势博弈必收敛；一般博弈可能循环	状态空间大、收益未知
优化（集中求解 / 变分不等式）	模型已知、可集中计算	可给最优性或均衡存在性	无法集中、模型时变

实践中的顺序建议是：先问“模型知不知道”，再问“能不能集中算”，最后才问“要不要学”。 三者都指向学习时，再按 §35.3 的结构判据挑具体算法。**反过来（先选算法再找场景）是这一领域最常见的失败模式。**

35.7.4 十条常见错误

错误一：把“均衡”当成“最优”。 均衡只保证“没人想单方面改变”，不保证福利高。囚徒困境里唯一的均衡就是最差的结果。

错误二：只报一个均衡的福利。 多个均衡时必须报区间。只报 (S,S) 的 **8.0**，等于把 **25%** 的风险藏起来了。

错误三：用“多跑一会儿”来解决均衡选择。 E4 说明早期几百个样本就锁定了结局，多跑只会让系统更牢固地停在原地。

错误四：ε 一路衰减到零。 对手会变，探索停了就再也跟不上。留一个与对手变化率可比的下限。

错误五：回放池开得很大。 旧样本对应旧对手，池子越大污染越重。池深应满足 $L \lesssim 1/\alpha$。

错误六：训练给全局观测、执行给局部观测，却不做一致性检查。 这是 CTDE 最常见的翻车点。上线前必须用“只给局部观测”重跑一遍，看性能掉多少。

错误七：用全局共享奖励做完全合作任务，还指望无关智能体学到零贡献。 E5 显示它会分到 -0.051 的偏差与 0.184 的标准差。能反事实就用差分奖励。

错误八：把同步更新当成默认。 势博弈里同步会永远振荡（E6 的 0/100）。更新时序是设计变量，不是实现细节。

错误九：用大罚项代替安全约束。 罚项与违反量的关系不透明，换个工况同一个系数就不管用了。 硬约束交给屏蔽层。

错误十：不做种子扫描就宣布成功。 单个种子跑出好结果，可能只是落在了 9% 那一侧。至少 **100** 个种子，报分布不报单点。

35.7.5 与后续章节的衔接

本章停在“多方各自优化、如何达到均衡”这一层。再往前一步是两个方向：一是“学错了会不会出事”，这是第 **36** 章的功能安全与可靠性；二是“算得太慢怎么办”，这是后面 **MPC** 与实时优化章节的内容。

如果只记住一句话：多智能体系统的难点不在“怎么学”，而在“该建哪些量、会停在哪个均衡、学错了谁兜底”。 前两件本章给了判据，第三件是第 36 章的主题。

下一章的门槛很低，只需要带着三个问题过去： 我的系统里，谁的安全是最薄弱的？它失效时最先表现出来的信号是什么？在信号出现到事故之间，还有多少时间可以用来兜底？这三个问题答不上来，第 **36** 章的任何方法都无法落地。

35.7.6 术语对照

表 35-8 把本章反复出现的术语集中列一次。它的用途是查“我用的词是不是这个意思”，而不是背定义。

表 35-8　本章术语对照

中文	英文	一句话含义	常见误用
纳什均衡	Nash equilibrium	给定别人不动，没人愿意单方面改	当成“最优结果”
占优策略	dominant strategy	无论别人做什么，它都不差	与“最优反应”混为一谈
混合策略	mixed strategy	按概率随机选动作	当成“不确定时的权宜”
最佳响应	best response	给定对方策略下的最优动作	以为它一定唯一
无政府代价	price of anarchy	最优福利 ÷ 最坏均衡福利	只用最好均衡算
风险占优	risk dominance	偏离损失更小的那个均衡	与“收益占优”混同
复制者动力学	replicator dynamics	收益高于平均的策略占比增长	以为它一定收敛
势博弈	potential game	存在一个共同账本，个体改进即它上升	以为所有拥塞博弈都是
相关均衡	correlated equilibrium	听第三方信号后无人想偏离	以为要额外的强制力
信用分配	credit assignment	把团队奖励分到个人头上	以为平均分就行
集中训练分散执行	CTDE	训练看全局、执行只看局部	训练/执行观测不一致
非平稳性	non-stationarity	别人的策略在变，目标跟着变	当成“环境噪声”

这张表里最容易混的是“占优”与“最优反应”、以及“风险占优”与“收益占优”。 前一对是“全局性质 vs 局部性质”（占优不需要知道对方策略，最佳响应必须知道）；后一对是“稳定性 vs 收益高低”（猎鹿里收益高的那个反而不是风险占优的）。把这两对分清楚，读本章后续文献、以及和现场工程师讨论时，就不会被术语带偏。

35.7.7 三个自测题

题一。 两个智能体、两个动作，收益矩阵为 $A=\begin{bmatrix}4 & 0\\ 3 & 3\end{bmatrix}$（行玩家收益，列玩家取转置）。问：纯均衡有几个？混合均衡是多少？哪个均衡风险占优？如果跑 100 个种子的独立 Q 学习，你预计分布如何？

参考答案： 纯均衡两个，(S,S) 与 (H,H)；混合均衡 $q=(d-b)/(a-b-c+d)=(3-0)/(4-0-3+3)=0.75$；偏离损失 (S,S) 处为 $4-3=1$、(H,H) 处为 $3-0=3$，故 (H,H) 风险占优；独立 Q 学习应大部分落到 (H,H)，本章实测 91 比 9。**这道题的价值在于四个答案互相印证——若你算出混合均衡不是 0.75，那风险占优的结论也就不必看了。**

题二。 六个人分三个资源，用同步最佳响应更新。问：会不会收敛？如果把更新改成异步呢？如果把收益改成“每个人独占一个资源才得分”呢？

参考答案： 同步不收敛，因为所有人基于同一份负载快照一起搬，会持续振荡（本章实测末段振幅 9.000、0/100 到达最小势）；异步必然收敛，因为它是势博弈且每次只让一个人改进（100/100 单调到达）。**第三问是陷阱：改成“独占才得分”之后，势函数不再存在（一个人的收益取决于别人是否也选同一个资源，且不是对称可积的），异步更新也就失去了“必然收敛”的保证。** 这说明势结构是条件，不是普适性质。

题三。 一个三人团队任务，真实边际贡献是 $(2,2,0)$，你用全局共享奖励学出 $(2.8,2.8,-0.05)$。问：这是算法错了吗？该换什么方案？换了之后能消掉那 0.8 的偏差吗？

参考答案： 不是算法错。-0.05 是全局奖励给无关智能体的结构性噪声，换差分奖励可把它精确压到 0；但 0.8 的偏差两种方案都消不掉，因为它是“边际贡献依赖于基准分布”造成的，不是估计误差。**要消掉它，只能改基准（用均匀随机基准重算），而那是评价口径的改动，不是算法的改动。** 这道题考的是“分清误差与定义差异”。